VERDE = facut
GALBEN = necesita completari / retusari
ROSU = no idea if it's good

!! putem scrie de acum incolo cu culori / fonturi diferite ca sa ne dam seama repede de ce e in plus?

SUBIECT 23 IAN 2019

1A. In cazul unui proiect ce are un grad foarte inalt de paralelizare se studiaza ipoteza dublarii resursei umane. Ce se va intampla cu efortul planificat pentru dezvoltarea proiectului?

Informal se poate spune ca se va reduce timpul de realizare al proiectului, dar va creste costul acestuia. Ca sa poti sa-ti dai seama si in ce masura, cred ca te poti lega de ce e prin curs:

Aici se poate spune despre Monitorizarea planificarii(pag. 406) si despre Earned Value Analysis(pag.414).

Pasii unui proces de planificare:

1.Construirea planului
2.Stabilirea unei baze
3.Evaluarea planului în mod regulat
    1.Începutul real și sfârșitul real al unei activități
    2.Efortul real consumat pe activitate
    3.Progresul tehnic poate fi dificil de evaluat
4.**Replanificare**
    1.Estimarea efortului și a duratei până la sfârșit
    2.Tehnica 1: eficiență prin care efortul real a fost exprimat cu privire la efortul planificat
    3.Tehnica 2: eficiență prin care progresul tehnic a fost exprimat cu privire la efortul planificat
    4.Împărtășirea noului plan și avansare la punctul 2

Colectarea datelor de efort
• Depinde de nivelul de formalitate oamenii pot fi nevoiți să furnizeze date despre efortul consumat pe activități
• De obicei indicat să fie realizat saptămânal
• Activitățile planului trebuie sa aibă referințe
• Va conține și zgomot

va urez restanta usoara zgarcitilor

Earned Value Analysis

- Earned Value Analysis este o metodă ce furnizează o viziune de ansamblu asupra proiectului, măsurând efortul planificat (costuri), progresul real (valoarea câștigată) și efortul (costurile actuale) în materie de valori monetare
- Măsurarea planului, a muncii și a progresului folosind aceeasi unitate le face comparabile
- Util pentru că:
  - Progresul devine comparabil cu efortul
  - Bugetul și costurile actuale sunt puse în context (a fi sub buget nu este întotdeauna bine, dacă progresul tehnic este și el scăzut)

22

Presupuneri și definiții

- Presupuneri:
  - Forță de muncă = Cost → reprezentările grafice ale efortului sau costului sunt echivalente
  - Corolar: Forța de muncă reală = Costul real
  - Progres = Bani
- Definitii:
  - Valoarea Planificată → costurile cumulate planificate pentru proiect; numite de asemenea: Costuri Bugetate ale Muncii Planificate
  - Costuri Reale → costurile cumulate ce sunt suportate în realitate; numite de asemenea Costurile Reale ale Muncii Efectuate
  - Valoarea Câștigată → progresul real, exprimat drept cantitatea din valoarea planificată ce a produs rezultate

23

2A. Daca suntem siguri ca un eveniment neplacut (spre exemplu unul dintre cei mai valorosi dezvoltatori va parasi echipa la mijlocul termenului planificat) se va intampla pe parcusrul dezvoltarii proiectului, il consideram si tratam ca un risc in cadrul proiectului, in cadrul procedurilor de management al riscului?

Da, il consideram ca un risc in cadrul proiectului, iar argumentele le poti lua din curs.

Aici te poti lega (intr-un mod firesc si natural, de altfel) de **Managementul Riscului (pag. 468), Strategiile managementului de risc (pag. 498)** si altele tot de pe acolo(e.g Evaluarea riscurilor)**.**

tanta usoara zgarcitilor

Managementul riscului

Managementul riscurilor colectează tehnici, metode cunoscute și procese care ajută la identificarea, evaluarea și monitorizarea riscurilor

- Obiectivele managementului riscurilor de proiect sunt:
  - Creșterea probabilității impactului evenimentelor positive
  - Scăderea probabilității și impactului evenimentelor nefavorabile proiectului.

Definiții

Traditional riscul este definit ca probabilitatea de a suferi pierderi

În managementul proiectelor riscul este un eveniment sau o condiție, care dacă se produce va avea un impact negativ sau pozitiv asupra proiectului

Cele mai importante 10 cauze în eșecul unui proiect ale lui Boehm

- Scăderea personalului sau a subcontractanților
- Programul și bugetul nerealist
- Dezvoltarea unor funcții software / interfața cu utilizatorul greșite
- Gold Plating (greșirea priorităților)
- Controlul schimbării ineficient
- Riscuri tehnice

Matricea de risc

Roșu necesită tratament special (sau renunță la proiect)

Portocaliu Necesită monitorizare

Verde stardarul unui proiect

	Neglijabil	Mic	Moderat	Sever	Catastrofic
Foarte înalt	R1				R5
Înalt			R2	R6, R7, R8	
Moderat		R3			
Mic				R4	
Foarte mic		R9, R10			

27

3A. Cum ati defini ca un proiect se afla “sub control”?

Aici se pot lua informatii din cursul **Monitorizarea si controlul proiectului(pag.401)** si poti spune ceva de genul.

Un proiect se afla sub control daca sunt realizate periodic evaluarea stausului proiectului (scop, timp, cost, calitate, etc), analiza deviatiilor, luarea deciziilor corective, daca este nevoie.
S.A.M.D

Introducere

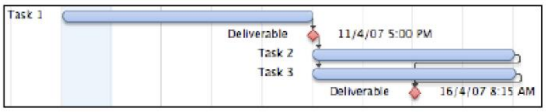
- Obiective:
  - **Pentru proiect** → evaluarea statusului proiectului (scop, timp, cost, calitate etc.), analizarea deviațiilor, luarea de decizii corective, dacă este nevoie
  - **Pentru organizație** → colectarea datelor ajuta la construirea unui plan mai bun și mai exact pentru proiectele viitoare
- Proces (în mod regulat):
  - Colectează → Strânge datele despre statusul curent al proiectului
  - Masoară și compară → Compară cu un plan de baza, subliniază orice deviație, realizează o proiecție pe baza datelor curente
  - Evaluează și replanifică → Decide dacă sunt necesare acțiuni corective. Dacă da, planifică, documentează și ia acele acțiuni

4A. In managementul de proiect putem “interschimba” intr-o oarecare masura oamenii si timpul de dezvoltare. Astfel, putem adauga oameni pentru a dezvolta in timp redus sau putem mari timpul si reduce numarul de membri in echipa. Care varianta o considerati mai usor de implementat si de ce?
Aici poti spune ca e mai usor sa reduci timpul si sa maresti numarul de resurse umane, ca mai apoi sa te legi de **Project Crashing(pag. 349)**

Project Crashing

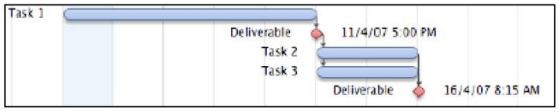
- Durata proiectului poate fi (adesea) redusă prin alocarea de mai multă muncă pentru activitățile proiectului, sub forma de lucru peste program, și prin alocarea de mai multe resurse, precum materiale, echipamente etc.
- Totuși, munca și resursele adiționale cresc costurile proiectului
- Astfel, decizia de reducere a duratei proiectului trebuie să fie bazată pe o analiză și pe compromisul dintre timp și cost
- Project Crashing reprezintă o metoda de scurtare a duratei proiectului prin reducerea timpului uneia sau a mai multor activități critice de proiect, obținând astfel un timp mai scurt decât timpul normal de activitate. Scopul este de a reduce durata proiectului și minimizarea costului crashing-ului

Project Crashing - Exemplu



cost = $ 3000
cost = $ 4000

- Project Crashing: alocăm mai multi oameni pentru Task-urile 2 și 3



cost = $ 6000
cost = $ 8000

- Timp de Crash = 2 unități de calendar (ex. luni)
- Costul de Crash = (6000 + 8000) - (3000 + 4000) = 14000 - 7000 = 7000
- Crashing convenabil conform costurilor livrării întârziate

8

1B. Care sunt asemanarile si deosebirile dintre "lines-of-code" si function-points? Care sunt avantajele si dezavantajele specifice ale fiecareia?

Asemanari:
- ambele sunt metrici ale unui produs
- apreciaza cantitativ modul in care obiectivele proiectului sunt atinse

Deosebiri:
- LoC este o metrica orientata pe dimensiune, pe cand FP este o metrica orientata pe functionalitate
- LoC evidentieaza numarul liniilor de cod, pe cand FP cuantifica functionalitatea oferita de sistem

Avantaje SLOC:
- poate fi colectat automat
- nu necesita personal calificat

Dezavantaje SLOC:
- inconsistenta

Avantaje FP:

- aduce o masura obiectiva in complexitatea sistemului
- pot fi calculate dupa cerinte

Dezavantaje FP:

- necesita training specific pentru a fi aplicat corect
- poate introduce rigididate in procesul de dezvoltare

2B. In cadrul unui proiect consideram urmatoarele optiuni de dezvoltare iterativa:
Ce varianta sau variante recomandati, care sunt avantajele si dezavantajele fiecareia?

A. Se identifica principalele functionalitati si se sorteaza in functie de importanta lor acordata de catre utilizatorii finali. Se implementeaza grupuri de functionalitati, in ordinea descrescatoare a prioritatilor, utilizand atatea iteratii cate grupuri de functionalitati interdependente sunt identificate;

B. Se identifica principalele trasaturi ale tehnologiilor si arhitecturilor folosite. Se ordoneaza functionalitatile dupa riscul estimat. Se dezvolta grupuri de functionalitati incepand cu cele mai riscante.

C. Planificam versiuni succesive: alfa, beta, etc. Fiecare versiune va avea un set de functionalitati complete, dar nivelul calitativ va creste odata cu versiunile superioare.

R:

A. Aici cred ca e **modelul incremental**.”la inceput sunt livrate incrementuri cu cerinte mai importante;

Avantaje

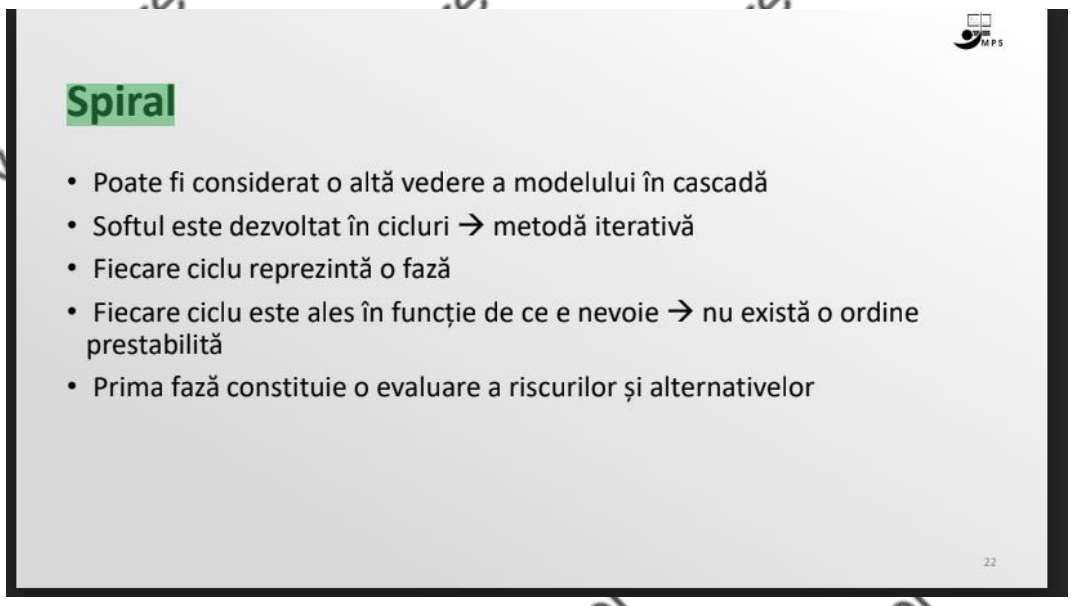
• Se obține un sistem funcțional mai rapid deoarece la început sunt livrate cerințele critice
• Incrementurile inițiale servesc drept prototip ajutând pentru elaborarea viitoarelor cerințe
• Risc scăzut de eșec al proiectului
• Cerințele cu prioritatea cea mai amre au parte de mai multă atenție (testare)

Dezavantaje

• Arhitecturile de sistem tind să se degradeze
• Unele incrementuri pot conduce la reimplementări semnificativ

B. Aici cred ca e modelul **Spiral**

va urez restanta usoara zgarcitilor



va urez restanta usoara zgarcitilor

Avantaje: (din curs)
- Bazata pe analiza de risc si alternative
- Functionalitati suplimentare sau modificari pot fi facute ulterior

Dezavantaje:
- Dificultati in a face estimari la inceput
- Flexibilitate scazuta
- Respectarea cerintelor de soft din contract

C. Descrierea corespunde metodologiei **Agile**.

Avantaje Agile:
- Cu fiecare versiune lansata rapid, clientul poate da feedback mai rapid si astfel proiectul va fi modificat inainte de faza finala.
- Se pierd mai putine resurse (de toate felurile), deoarece se lucreaza intotdeauna la sarcini actualizate.

Dezavantaje Agile:
- Proiectele pot dura mult, deoarece nu exista un final clar. Acest lucru nu este benefic nici pentru clienti.
- Este nevoie de o viziune pe termen lung a proiectului.
- Functionalitatile care sunt prea mari ca sa incapa intr-un ciclu sau mai multe (versiuni) sunt evitate.

va urez restanta usoara zgarcitilor

Subiecte MPS 2017-2018 Part 1

Subiect 1A

Aveți la dispoziție 4 luni pentru a dezvolta o aplicație din categoria celor pentru care nu există un rezultat optim, ci doar încercări/aproximări (rezultatul unor cercetări) din ce în ce mai reușite (ca exemple putem menționa: dublarea pe cale artificială a rezoluției unei imagini, un „engine” de “Optical Character Recognition”/OCR, etc…). Aveți la dispoziție o echipă formată din 4 programatori și 2 testeri.
Cerință: Cum ați planifica activitățile, ce riscuri potențiale întrevedeți și cum doriți să le contracarați?

caca maca
aici oare trebuie sa ii scriem iar de o metoda de dezvoltare?

Cred ca putem aborda Open Unified Proces (dedicat echipelor mici 3-6 persoane cu o abordare agile si iterativa -> 3-6 luni de dezvoltare);
slide-urile 631 - 632 - 633 - 634 din merged pdf.
uh..sau V-Cycle.. ca avem si programatori si testeri..si putem merge in paralel cu ele ca sa nu pierdem timp; (V-Cycle e focusat pe testare .. atata timp cat nu trb sa avem rezultate clare/fixe cred ca e useless sa testam)
Riscuri:

- program si buget nerealist
- gold plating (gresirea prioritatilor)
- riscuri tehnice?!

la strategii poate gasim ceva potrivit prin slide-urile 501, 502, 503

- erori comune in slide-urile urmatoare
  ex: in timpul fazei de planificare: nescrierea unui plan balansat;
  in timpul fazei de monitorizare: inconsecvente, plan economic incorect, neajustarea planului pe parcursul proiectului...etc

Subiect 2A

Un software necesită N unități de timp (UT) pentru dezvoltare. Dezvoltarea este paralelizabilă, 2 oameni o pot termina în N/2 UT, 3 oameni în N/3 UT, etc. Dezvoltarea este precedată de o sesiune ce presupune luarea deciziilor necesare prin discuții libere.
Cazul A: Fiecare persoană va spune ce dorește să facă în X UT și va necesita Y UT în total ca sa răspundă preferințelor tuturor celorlalți.
Cazul B: Fiecare persoană va spune ce dorește să facă în X UT și va răspunde preferințelor celorlalți, fiecăruia în parte Y UT.

Cerință: Care este numărul optim de persoane implicate pentru a termina în timpul cel mai scurt în cazul A și în cazul B? Ce observații/recomandări ați face referitor la relațiile dintre N, X, Y?

Daca N e mare => Cazul A are timpul: X + Y (e mai eficient)
Cazul B : X + (n - 1) Y, n = numarul de persoane

Subiect 3A

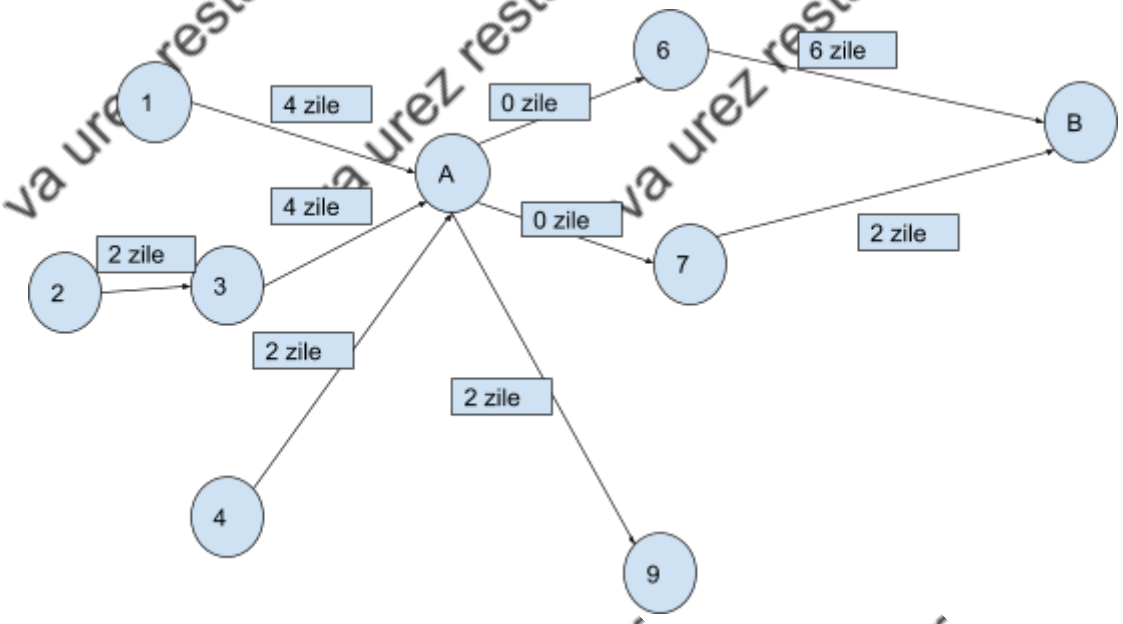
Calculați drumul critic pe baza planului de mai jos:

Task	**Durată**	**Dependență**
Task 1	4 zile	
Task 2	2 zile	
Task 3	4 zile	2
Task 4	2 zile	
Livrabil A		1, 3, 4
Task 6	6 zile	5
Task 7	2 zile	5
Livrabil B		6, 7
Task 9	2 zile	1SS

Nota: *A* SS *B* înseamnă ca *B nu poate începe înaintea începerii A* sau altfel spus activitatea *B poate începe după ce activitatea A începe* (notație pentru dependența tip Start-to-Start).

cred ca vine ceva de genul:

va urez restanta usoara zgarcitilor



no idea daca e corect… cred ca critical path ar fi:

2 -> 3 -> A -> 6 -> B => 12 zile

nu stiu exact unde vine Task 9...

Subiect 1B

Presupuneți ca aveți de decis calea A sau B într-un proiect. Dacă alegeți A, aveți șanse mici (estimativ 1 din 10) de manifestare a unui risc major (9 din 10). Dacă alegeți B aveți șanse mai mari (estimativ 3 din 10) de manifestare a unui risc mediu (4 din 10). Din punct de vedere al managementului riscului ce decizie ați lua și de ce?

Avem asa.. Score = probabilitate * impact
caz A: 1 / 10 * 9 / 10 = 0.09
caz B: 3 / 10 * 4 / 10 = 0.12
nu stiu care ar fi mai buna.. teoretic.. in matricea riscurilor cazul A e in zona rosie.. iar Cazul B e in zona verde spre moderata..deci presupun ca al doilea caz e cel bun.. nu zicea nicaieri in curs ce inseamna scorurile astea
(slide 495 jos)

Mie mi se pare ca ambele cazuri se afla in zona portocalie in matricea riscurilor, si evident scorul de risc il vrei cat mai mic, deci as alege varianta A.

Subiect 2B

Care sunt, în opinia voastră, cele mai importante dezavantaje în dezvoltarea proiectelor utilizând Scrum?

Dezavantaje:

- Daca un membru cu experienta al echipei de Scrum paraseste proiectul in timpul unui sprint/release e foarte greu ca restul sa ii preia taskurile si sa le rezolve si pe ale lor
- Termenul de finalizare este unul necunoscut

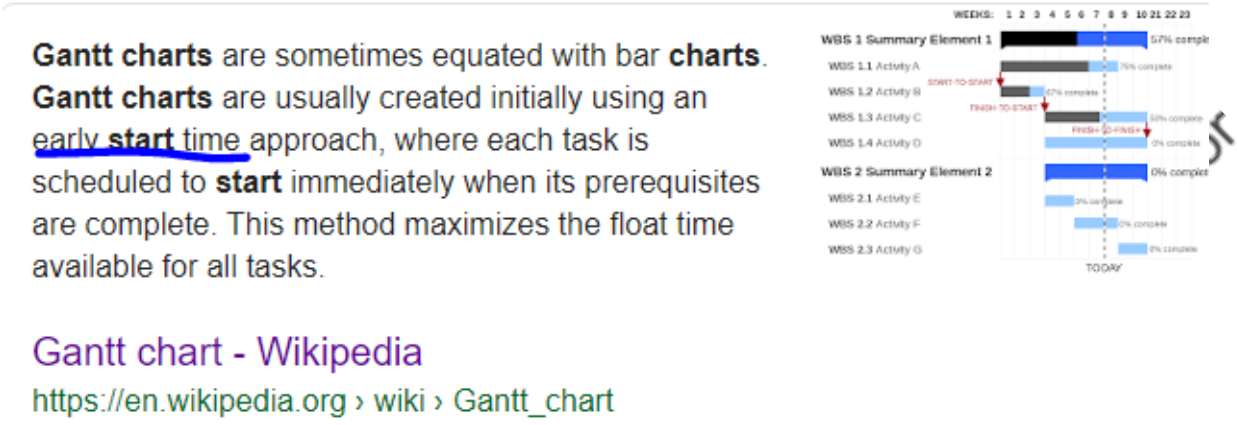
va urez restanta usoara zgarcitilor

- Lucrul in echipe de 5-8 persoane; daca proiectul are mai multe persoane implicate este greu sa imparti sau sa formezi echipe
- In multe cazuri timpul petrecut in sedinte ocupa prea mult timp ce putea fi folosit in dezvoltare

Subiect 3B (mai trebuie adaugat argumentul)

Când încep activitățile într-o diagramă Gantt: cel mai devreme posibil, cel mai târziu posibil sau în cel mai probabil moment? Argumentați răspunsul.

R: cel mai devreme posibil



Float time = slack -> perioada de timp de cand poate sa inceapa proiectul cel mai devreme pana cand poate sa inceapa proiectul cel mai tarziu fara sa afecteze/influenteze durata proiectului

Subiect 4B

Care este diferența dintre un milestone și un livrabil?

R:
Livrabil: exista surse, documente care pot fi livrate.
Milestone: criterii care ne arata faptul ca s-a ajuns in acel punct in dezvoltarea proiectului.

Subiecte MPS 2017-2018 Part 2

Subiect 1A

Aveți la dispoziție 4 luni pentru a dezvolta două proiecte. Primul proiect este o aplicație complexă, de timp real (cu constrângeri clare) dar cu specificații bine construite și pentru care nu vă așteptați la modificări/cerințe suplimentare din partea clientului. Al doilea proiect este o aplicație mai comună (“straightforward”), pentru care vă așteptați să aveți un necesar de timp de lucru redus în comparație cu primul proiect dar, de asemenea, vă așteptați să

aveți mai mult “de alergat” cu specificațiile finale, dată fiind nehotărârea clientului. Aveți la dispoziție 5 programatori și 3 testeri.
Cerință: cum ați planifica activitățile, ce riscuri potențiale întrevedeți și cum doriți să le contracarați?

Planificarea activitatilor:
Specificarea si impartirea task-urilor pentru primul proiect, cat timp pentru cel de-al 2lea sunt definitivate cerintele/specificatiile. La inceput toate resursele umane vor fi asignate primului proiect.
Dupa ce activitatile sunt puse la punct pentru al 2-lea proiect, 3 dintre programatori vor schimba proiectele, ramanand pentru primul doar 2 pentru finalizarea task-urilor.
La inceput, testerii se vor ocupa tot de primul proiect, pana cand cel de-al 2lea are o finalitate. Apoi dupa 2-3 saptamani de la inceperea celui de-al 2lea proiect 2 dintre ei vor schimba proiectele pentru a testa.

Riscuri:

- nefinalizarea in timp util a proiectului al 2-lea din cauza intarzierii clarificarii cerintelor => proiectul a 2lea nu va fi platit integral; SOLUTIE: discutia cu clientul si expunerea problemei
- nealocarea unui interval de timp pentru problemele/bugurile aparute in timpul dezvoltarii proiectelor; SOLUTIE: unul dintre programatori (din fiecare proiect) sa poata fi disponibil pentru bug fixing

Subiect 2A

Considerați următoarele principii/valori:

A. Acționează simplu și păstrează mereu lucrurile simple!
B. Testează-ți codul permanent!
C. Clienții nu știu de fapt ceea ce-și doresc!
D. Nu merge pe încredere, semnează înainte de a porni la drum!
E. Nu munci pe gratis, precizează clar cat valorezi!
F. Fii conectat, mergi des la conferințe și întâlniri profesionale!
G. Așteaptă și relaxează-te, ți se va revela soluția!
H. Împărtășește-ți cunoștințele!
I. Dacă ai ideea corecta, restul vine de la sine!
J. Fii cu capul pe umeri!
K. Învață continuu!
L. Depășește-ți limitele!
M. Fii perfecționist și meticulos!
N. Grăbește-te, timpul trece chiar acum!
O. Fii exact, precis, punctual, profesionist!
P. Creativitatea este cea care face diferența!
Q. Încearcă sa-i înțelegi pe ceilalți!
R. Încearcă sa te faci înțeles!
S. Impune-te, ești cel mai valoros!
T. Visează, doar astfel vei evolua!

va urez restanta usoara zgarcitilor

U. Meseria este brățara mea de aur!
V. Numai cine nu riscă nu câștigă!
W. Succesul este 1% inspirație și 99% transpirație!
X. Comunicarea este cea mai importantă!
Y. Clientul nostru, stăpânul nostru!
Z. Fii adaptabil, valorile și principiile nu vin scrise, ți le creezi din mers!

Considerați pozițiile de mai jos:

1. Dezvoltator software
2. Tester
3. Arhitect software
4. Manager de proiect
5. Cercetător
6. Suport documentație
7. Team leader
8. Freelancer

Pentru fiecare poziție în parte, care credeți că este principiul cel mai important și cel cu potențialul cel mai nociv, în acest din urmă caz precizând care este capcana în care se poate cădea?

wtf ce e prostia asta

soft dev: Important: Nociv: E

tester:. Important: B Nociv:

arhitect soft: important: Nociv : C

PM: important F nociv: S

cercetator: important: K nociv: G

suport documentatie:

team leader: important: H nociv:

freelancer: important: Y nociv: V (poti sa nu castigi….) sau C

Subiect 3A

Este necesară alegerea între două strategii de implementare de proiect diferite ale unui software necesar în compania noastră.
Prima opțiune este bazată pe o dezvoltare internă, planul de proiect propus are următoarele caracteristici:

- Proces de dezvoltare: cascadă
- Echipa:
    - 1PM care lucrează la 50% din timp pentru toată durata proiectului (salariu lunar 5000 credite)
    - 2 analiști care lucrează full-time 2 luni calendaristice pentru fazele de analiză și definire cerințe (salariu lunar 4000 credite)
    - 4 programatori care lucrează 50% din timp pentru 6 luni calendaristice la restul fazelor din procesul de dezvoltare cascadă (design, implementare, integrare și testare, exploatare și întreținere) (salariu lunar 3000 credite)

va urez restanta usoara zgarcitilor

A doua opțiune este bazată pe externalizarea activităților de dezvoltare. Furnizorul nostru de soluții software ne propune următoarele:

- Proces de dezvoltare: SCRUM (8 sprint-uri a cate o lună fiecare)
- Echipa:
  - 4 programatori full-time (salariu lunar 4000 credite)
  - Unul dintre angajați va trebui să fie disponibil ca și 'client'. Alocam pentru acest scop un PM care lucrează 50% din timp pe întreaga durata a proiectului (salariu lunar 5000 credite)

Cerință: Ce strategie de implementare proiect sa alegem și de ce? Efectuați o comparație din punct de vedere: costuri, durată, riscuri.

Deci: ambele proiecte sunt de 8 luni; analizam COSTURILE (adica salariile):
A: 5000 * 8 + 4000 * 2 * 2 + 3000 * 4 * 6 = 128k
B: 4 * 4000 * 8 (programatorii) + 8 * 5000 (pm-ul) = 168k
concluzie so far: depinde
analizam DURATA: 160 de zile lucratoare in 8 luni
A: PM cu 50% din timp => 4h / zi timp de 8 luni => 160 * 4 +
2 * 8 * 40 (analistii) +
4 * 4 * 120 +
total: 3200 ore
B: 4 * 160 * 8 + 4 * 160 = 5760 ore
concluzie: tot mai ok e prima varianta.. per total mai putine ore lucrate;

Acum ar trb sa vb de riscuri. Ala e **Waterfall**, deci nu se schimba req, nu interactionezi asa de mult cu clientul ca in al doilea, bug-urile inseamna rework, daca sunt gasite bugurile abia la sf cresc si costurile. In Agile bugurile pot fi gasite in timp si rework se poate face in perioada de dezvoltare, nu e nevoie de o investitie mai mare cum ar fi la waterfall.

Riscuri Scrum: prea multe modificari = costuri ridicate; genereaza stres; cerintele se modifica des => poate rezulta in munca ineficienta; poate fi foarte dificil de implementat din cauza lipsei de documentatie si structura oferita de la inceput

Subiect 1B

Care sunt, în opinia voastră, cele mai importante dezavantaje în dezvoltarea time-boxed?

Time-boxed:
In agile, daca nu reusesti sa termini ceva in sprintul curent, user story-ul revine pe backlog(asta e tabla aia cu user story-uri in care gasesti si the walking skeleton)
Dezavantaje:

Drawbacks of Timeboxing
Using timeboxing to manage your workload may not be appropriate for everyone. For example, you may find it hard to stop and switch to another task when you're in **flow** , because you worry that you might lose your train of thought. And you may find it difficult to stick to the schedule because of frequent **interruptions** , such as phone calls or colleagues unexpectedly stopping by to chat.

va urez restanta usoara zgarcitilor

Also, some jobs have to be completed to a high level of quality, however long they take. Timeboxing can cause you to rush, meaning that you fail in your objective.
Finally, there's a risk that you may not set appropriate lengths of time for your timeboxes. If they are too short, you may not accomplish much; if they're too long, you might lose focus or procrastinate until the end of the period. So, make a note of how long different tasks take you, and organize your workload according to this.
Timeboxing is a powerful approach, but only use it when it's appropriate!

Setarea unui interval de timp si realizarea a cat mai multe feature-uri
• Accent pe timp
Raspuns:
o Realizarea feature-urilor de dimensiuni mici
o NU ajungem sa implementam feature-uri majore

Subiect 2B

Care sunt, în opinia voastră, dezavantajele de a avea prea puține sau prea multe milestone-uri într-un proiect?
Prea multe:

- poti ajunge la sfarsitul unora fara nicio functionalitate completa; problema cand lucrezi cu stakeholders:
  "This comes in handy when you are dealing with stakeholders. Stakeholders are not interested in a granular, detailed report on the project's progress. They want broad strokes that indicate whether or not the project is moving along as scheduled. Milestones are ideal for this kind of reporting because milestones show the major phases you've finished at this point in the project, according to your plan."

- daca fiecare e decalat => intarzieri foarte mari ale deadline ului

Poate fi si ceva de aici:
"Milestones, like tasks, can be linked. That is when the phase of one milestone cannot begin until the completion of the phase before it. That way you're not blocking team members by having them wait or by not allowing them what they need to move forward with their tasks."

Prea putine:

- durata foarte mare pana ajungi la ceva functional care este aratat clientilor

Raspuns:
o Prea putine: nu stim unde ne aflam cu dezvoltarea
o Prea multe: scade semnificatia milestone-ului -> overhead
Obs: Managementul nu este un scop in sine.

Subiect 3B

Este posibilă existența a două drumuri critice într-un graf de activități al unui proiect? Dacă da, dați un exemplu. Dacă nu, argumentați.

Drumul critic da durata proiectului.
Raspuns:
Da. De exemplu, in situatii simetrice.
! Intotdeauna avem un drum critic.

va ureze restanta usoara zgarcitilor

Subiect 4B

Construim structura unei echipe. Trebuie să urmărim să minimizăm sau să maximizăm comunicația între membri? Există un răspuns general sau depinde de context?

Raspuns:
E dependent de proiect.
Ceea ce nu este relevant, trebuie minimizat.

In cazul proiectelor care se bazeaza pe creativitate, comunicarea trebuie maximizata.
Daca nu este parte din strategia creativa a proiectului, comunicatia trebuie minimizata.

Subiecte din poze:

Sub1:

Aveti la dispozitie N programatori pe care doriti sa ii impartiti in echipe de lucru de dimensiuni cat mai egale cu o coeziune in cadrul echipei cat mai buna. In acest sens ii evaluati cum lucreaza 2 cate 2 impreuna in cadrul unor sesiuni de pair-programming si notati performantele lor pe o scara de la 1 la 10 intr-o matrice simetrica M de dimensiuni N x N.

Ce algoritm de grupare ati folosi pentru a crea echipe de lucru?
Cum ati alege cardinalitatea echipelor?
Cum ati proceda in cazul in care N este mare si este imposibil sa evaluati toate elementele matricei?

	d1	d2	d3	d4	d5	d6	d7	d8
d1 - 4	0	1	2	3	4	5	6	7
d2 - 3, 3	1	0	1	2	3	4	5	6
d3 - 2, 6	2	1	0	1	2	3	4	5
d4 - 2, 3	3	2	1	0	1	2	3	4

va urez restanta usoara zgarcitilor

d5 - 2,3	4	3	2	1	0	1	2	3
d6 - 2, 6	5	4	3	2	1	0	1	2
d7 - 3, 3	6	5	4	3	2	1	0	1
d8 - 4	7	6	5	4	3	2	1	0

d4 - d8 = 4
d5 - d1 = 4
d3 - d7 = 4
d6 - d2 = 4

d4 - d8 - d1 - d7
d2 - d3 - d5 - d6

Putem folosi structura matriceala dintr-un proiect
slide-urile 581, 582, 583, 584

Pare o problema NP-complete… idk… e grea

Sub2:

Aveti la dispozitie un istoric bogat al dezvoltarii de software in compania in care lucrati. Puteti obtine statistici referitoare la grupurile de programatori ce au contribuit la realizarea unor task-uri si o nota a eficientei grupului respectiv, de la 1-10. Grupurile aveau in componenta 3-8 persoane si au fost evaluate doar la nivel de echipa. Doriti ca in cadrul urmatorului proiect sa apelati la o dezvoltare pair-programming si in acest scop este necesar sa determinati care sunt cele mai eficiente perechi, iar cerinta este sa propuneti o solutie algoritmica destinata sa estimeze cat mai precis asta, utilizand toate datele puse la dispozitie. Se poate spune ca datele istorice sunt suficient de acoperitoare astfel incat orice pereche sa se gaseasca in cel putin un grup anterior evaluat.

aici e mai simplu… facem media..
daca fiecare pereche e in cel putin un grup.. luam notele grupurilor din care face parte perechea si facem media la final..

Descrieti utilizand diagrame UML structura si functionarea cat mai completa a
structurii de conducere a statului roman: Presedentie, Guvern, Ministri, Parlament,
Constitutie, Legi, etc.

Eric e de vina

INTREBARI ADITIONALE:

va urez restanta usoara zgarcitilor

Cum pot estima cat de mult va dura completarea unui task?

Care este suma de bani pe care o voi solicita pentru un proiect?

Cum pot sa mentin echipa motivata si sa ma asigur ca proiectele reprezinta pentru angajati o ocazie de a invata, creste si avansa in cariera?

Cum pot gestiona riscurile unui proiect?

Cum evaluez daca proiectul respecta timpul, bugetul si deadline-ul?

Cum controlez calitatea produsului final?

va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor

Subiect 1

Modelul cascada vs Modelul spirala vs Modelul V vs Agile etc

a. Avantaje si dezavantaje?

Modelul cascada:

Avantaje:

- Sistem bine documentat
- Permite controlul efficient al proiectelor si estimarea exacta a timpilor de executie
- bun management al proiectului

Dezavantaje:

- un produs executabil care sa demonstreze functionalitatea este disponibil destul de tarziu, dupa integrare
- multe erori sunt descoperite tarziu => cost crescut
- toate riscurile sunt incluse intr-un singur ciclu de dezvoltare

Modelul spirala:

Avantaje:

- abordare evolutionista
- ajuta la intelegerea riscurilor si la identificarea modalitatilor de tinere sub control a acestora
- prototipizarea este folosita ca mecanism de reducere a riscurilor
- functionalitati aditionale pot fi adaugate mai tarziu
- software-ul este produs devreme in ciclul de dezvoltare

Dezavantaje:

- Analiza riscurilor este o activitate critica
- Poate sa fie costisitor
- Succesul proiectului este dependent de pasul de analiza a riscurilor
- Nu functioneaza foarte bine pentru proiectele mai mici

Modelul V:

Avantaje:

- Este simplu si usor de folosit
- Activitatile de testare se petrec inainte de inceperea scrierii codului
- Functioneaza foarte bine pentru proiecte mici, ale caror cerinte sunt usor de inteles

Dezavantaje:

- Este un model foarte rigid si cel mai putin flexibil
- Software-ul este dezvoltat in timpul pasului de implementare, deci nu avem prototipuri ale softului pe care urmeaza sa-l producem
- Daca se petrec schimbari la jumatatea proiectului, atunci documentele de testare si de cerinte trebuie updatate

Prototipuri:

Avantaje:

- Necesar redus de documentatie
- Costuri reduse de intretinere
- Clientii pot fi mult mai implicati in deciziile legate de designul final al sistemului
- Comunicare imbunatatita

Dezavantaje:

- O aplicatie incompleta poate sa duca la utilizarea acesteia diferit fata de cum a fost proiectata
- Complexitatea sistemului poate sa creasca fata de planurile originale

Agile:

Avantaje:

- Satisfacerea clientului prin livrarea continua de soft
- Soft functional este livrat frecvent
- Metode de comunicare face-to-face
- Adaptabilitate crescuta la schimbari
- Chiar si schimbarile tarzii sunt acceptate

Dezavantaje:

- Nu se pune accent pe design si documentatie
- Doar programatorii seniori sunt capabili sa ia decizii necesare in timpul procesului de dezvoltare
- In cazul unor livrabile, in special cele mari, este greu sa evaluezi efortul necesar la inceputul procesului de dezvoltare

b. La ce gen de proiecte le-ati recomanda ?

Modelul Cascada : Adecvat pentru proiecte in care cerintele sunt bine intelese de la inceput si nu se modifica pe parcursul procesului de dezvoltare. De obicei proiectele bazate pe modelul cascada sunt scurte.
Modelul Spirala: Adecvat pentru proiectele de risc mediu spre ridicat, unde costurile si evaluarea riscurilor sunt importante, unde userii sunt nesiguri de ceea ce doresc, cerintele sunt complexe si sunt asteptate schimbari insemnate.
Modelul in V : Adecvat pentru proiecte mici-medii, unde cerintele sunt clar definite si fixate. Ar trebui ales cand sunt disponibile ample resurse tehnice, cu efectuarea unei expertize tehnice.
Prototipuri : Se folosesc atunci cand trebuie ca sistemul dorit sa aiba multa interactiune cu end-userii. Recomandat pentru sisteme online, interfete web etc.
Agile : Adecvat pentru proiecte in care apar noi schimbari care trebuie implementate. Libertatea de schimbare oferita de „agile” este foarte importanta pentru ca noile schimbari pot fi implementate cu un cost scazut, datorita frecventei noilor incremente produse.

c. La ce gen de echipe le-ati recomanda ?
Cascada: Adecvat pentru echipe in care pot aparea membri noi intrucat documentatia si proiectarea structurii reprezinta un avantaj pentru acestia. Deasemenea este adecvant pentru echipe in care membrii apar si se implica la diferite faze ale dezvoltarii in comparatie

cu Agile cand toti membrii echipei sunt necesari inca de la inceput pentru fiecare iteratie. De exemplu testerul poate fi implicat abia la sfarsitul procesului de dezvoltare.
If you have several project teams located in different geographic locations, co-ordination of work needs to be more detailed and stringent. Work assignments need to be well-defined to avoid confusion and redundancy of work. In such cases, Waterfall is likely more beneficial as it provides clear-cut deliverables and milestones.
Spirala: Adecvat mai mult pentru seniors si architects in prima faza pana la evaluarea obiectivelor, riscurilor si a concretizarii taskurilor. Apoi este adecvat pentru orice tip de echipa dupa stabilirea riscurilor si a obiectivelor. Intrucat lucrurile sunt clare/risk-free si cateodata chiar si atractive intru cat in acest model se asteapta schimbari semnificative dar care sunt risk-free.
V: Adecvat echipelor cu programatori disciplinati si avansati intrucat foloseste mecanisme de TDD care pot deveni foarte costly in timp fara experienta.
Prototipuri: Adecvat echipelor care lucreaza foarte mult cu front-end(web-designers) sau alte tipuri de interfatare directa cu clientul.
Agile: Applying the agile approach on geographically separate teams may introduce new challenges. As noted by Martin Fowler, a well-known agile evangelist, Because agile development works best with close communication and an open culture, agilists working offshore feel the pain much more than those using plan-driven approaches.

Subiect 2

Utilizand o lista simplu inlantuita scrieti o specificatie tehnica pentru detectia existentei unui ciclu in lista (este lista finita sau infinita/ciclica de la un punct incolo?).

Nu este permisa alocarea de structuri de date suplimentare sau a marcarii/alterarii unora din elementele listei.

The idea is to have two references to the list and move them at **different speeds**. Move one forward by 1 node and the other by 2 nodes.

- If the linked list has a loop they will *definitely* meet.
- Else either of the two references(or their next) will become null.

Java function implementing the algorithm:

```
boolean hasLoop(Node first) {
   Node slow = first;
   Node fast = first;

   while(fast != null && fast.next != null) {
      slow = slow.next;          // 1 hop
      fast = fast.next.next;     // 2 hops

      if(slow == fast)  // fast caught up to slow, so there is a loop
         return true;
   }
   return false;  // fast reached null, so the list terminates
}
```

Subiect 3

Se doreste alocarea unui numar de persoane pe o serie de posturi disponibile, dupa cum urmeaza: team leader(i), developer(i), tester(i), researcher(i). Ordonati in ordinea descrescatoare a importantei urmatoarele calitati, pentru fiecare post in parte. Motivati alegerea si puneti la punct un algoritm de mapare a resuselor.

- Spirit inovativ
- Cunostinte solide matematice
- Calitatea codului scris
- Calitatea documentatiei scrise
- Inclinatie spre “despicarea firului in patru”
- Abilitatea de exprimare
- Constanta in activitate
- Abilitatea de a performa la inalta calitate activitati monotone
- Ce alte calitati credeti ca ar trebui luate in considerare si in ce mod?

!Leadershipul înseamnă direcţionare şi motivare. - deci pentru Team Lider foarte mult trebuie sa aiba calitati de motivare, directionare comunicare

Team lideri: abilitatea de exprimare, spirit inovativ, inclinatie spre despicarea firului in 4, constanta in activitate, calitatea codului scris, abilitatea de a performa la inalta calitate activitati monotone, calitatea documentatiei scrise.

Developerii: Inclinatie spre “despicarea firului in patru, calitatea codului, constanta in activitate, abilitatea de exprimare, abilitatea de a performa la inalta calitate activitati monotone, calitatea documentatiei, cunostinte matematice

Testerii: calitatea documentatiei, abilitatea de a performa la inalta calitate activitati monotone, constanta in activitate, abilitatea de exprimare, …

Researcherii: Spirit inovativ, constanta in activitate, abilitatea de exprimare, inclinatie spre descpicarea firului in 4, calitatea documentatiei scrise, performarea la inalta calitatea a abil. monotone, cunostinte solide matematice, calitatea codului

Subiect 4

Descrieti utilizand diagrame UML structura si functionarea cat mai completa a structurii de conducere a statului roman: Presedentie, Guvern, Ministri, Parlament, Constitutie, Legi, etc.

Incercati sa atingeti cat mai multe aspecte pentru ca, in mod teoretic, cel ce primeste specificatia dumneavoastra sa aiba o vedere de ansamblu cat mai cuprinzatoare si detaliata asupra functionarii statului.

Se poate merge pe class diagram(ar merge niste interfete + mosteniri) + use-case diagram.

Subiect 5

Se considera urmatoarea structura de conducere (*N specifica o multiplicitate a nucleelor PM-DEV-TEST), structura functionala in cadrul companiei Microsoft (Redmond):

Sarcinile fiecarui post sunt urmatoarele:

- GM
  - alege proiectul
  - il descrie intr-o intalnire celor 3 lead.
  - se ocupa de impartirea banilor, gasirea de noi resurse, rezolvarea de probleme deosebit de importante (care trec de lead)
- PM-lead
  - imparte proiectul in sub-arii
  - scrie specificatiile high-level
  - supervizeaza pm-ii
  - se sincronizeaza cu dev-lead si test-lead
- Dev-lead
  - supervizeaza dev-ii
  - se sincronizeza cu pm lead si test lead
  - scrie cod la nevoie
  - face code review
- Test-lead
  - supervizeaza testerii
  - se sincronizeaza cu dev lead si pm lead
  - face code review la testeri
- PM
  - scrie specificatiile mid-level
  - imparte aria in “work item”-uri
  - face research
  - aduna resursele de care au nevoie DEV si TEST
  - se sincronizeaza cu ceilalti PM
- DEV
  - implementeaza specificatiile
  - se sincronizeaza cu alti DEV-i
- TEST
  - scrie test planul
  - il implementeaza

▫se sincronizeaza cu ceilalti TEST

▫face code review DEV-ului

Intrebari:

•Care sunt avantajele si dezavantajele structurii de conducere particulare prezentate?

•Cum ati modifica structura pentru a compensa dezavantajele?

•Motivatie completa pentru fiecare aspect

----------subiect2015----------

Subiecte „Managementul Proiectelor Software" - conf. dr. ing. Costin-Anton BOIANGIU

S1. Se consideră preluarea unui proiect ce are ca scop dezvoltarea, predarea cu punerea în funcțiune și mentenenanța unui sistem de comunicație tip Skype sau YahooMessenger.
Descrieți viziunea voastră asupra proiectului din următoarele puncte de vedere:
- Structură departamente înglobate în activitate;
- Structură ierarhică intra și inter-departamentală;
- Fluxul documentației;
- Strategii și variante de planificare;
- Riscuri și precauții specifice proiectului;

Acolo unde este posibil, utilizați diagrame UML.

S2. Un tester are de verificat funcționalitatea a 12 module procesatoare de date de mari dimensiuni. Pentru a ușura testarea, modulele sunt create astfel încât în cazul unei funcționări corecte să ruleze fix 12 minute fiecare iar în cazul în care apare o eroare în oricare din fazele execuției, un mecanism de captare excepții pornește un timer intern și dacă nu se reușește refacerea funcționalității termină modulul după exact 13 minute (marker ghinion :D) de la pornirea sa. Problema este însă că testerul este la finalul programului său de muncă de 8 ore și ați dori în calitate de PM ca mâine la începutul programului să aveți rezultatele testării. Testerul poate măsura în scriptul său de validare doar durata totală a execuției scriptului, durată ce apare imediat după finalizarea execuției acestuia. Presupunând că vă așteptați ca maxim un modul să fie defect, ce îi veți cere testerului să ruleze în scriptul său astfel încât la întoarcerea sa mâine la serviciu să vă trimită diagnosticul: dacă vreun modul este defect și dacă da, care?

S3. Descrieți utilizând diagrame UML structura și funcționarea cât mai completă a structurii facultății de Automatică și Calculatoare: departamente, decan, prodecani, directori de departamente, săli, cursuri, laboratoare, centre de cercetare, studenți, bibliotecă, secretariat, personal auxiliar, etc... Încercați să atingeți cât mai multe aspecte pentru ca, în mod teoretic, cel ce primește specificația dumneavoastră să aibă o vedere de ansamblu cât mai cuprinzătoare și detaliată asupra funcționării facultății noastre.

S4. ***MergeSort*** este un algoritm de sortare cu complexitatea O(n*log(n) inventat de John von Neumann în 1945. Este un exemplu de algoritm de tip divide et impera. Algoritmul merge sort execută următorii pași:
- Dacă lista este de lungime 0 sau 1, atunci este deja sortată. Altfel:
- Împarte lista nesortată în două subliste aproximativ egale.
- Sortează fiecare sublistă recursiv prin reaplicarea algoritmului merge sort.
- Se interclasează cele două liste și se obține lista inițială sortată.

QuickSort este un celebru algoritm de sortare, dezvoltat de C. A. R. Hoare și care, în medie, efectuează O(n*log(n)) comparații pentru a sorta n elemente. În cazul cel mai defavorabil, efectuează O(n^2) comparații. De obicei, în practică, quicksort este mai rapid decât ceilalți algoritmi de sortare de complexitate O(n*log(n)) deoarece bucla sa interioară are implementări eficiente pe majoritatea arhitecturilor și, în plus, în majoritatea implementărilor practice se pot lua, la proiectare, decizii ce ajută la evitarea cazului când complexitatea algoritmului este de O(n^2)
Quicksort efectuează sortarea bazându-se pe o strategie divide et impera. Astfel, el împarte lista de sortat în două subliste mai ușor de sortat. Pașii algoritmului sunt:
- Se alege un element al listei, denumit pivot
- Se reordonează lista astfel încât toate elementele mai mici decât pivotul să fie plasate înaintea pivotului și toate elementele mai mari să fie după pivot. După această partiționare, pivotul se află în poziția sa finală.
- Se sortează recursiv sublista de elemente mai mici decât pivotul și sublista de elemente mai mari decât pivotul.
- O listă de dimensiune 0 sau 1 este considerată sortată.

RadixSort este un algoritm ce nu folosește comparații de chei, ci distribuie cheile în grupe pe baza unei informații particulare referitoare la natura cheilor. Fie tipul cheilor Key alcătuit din k componente: f1, f2, ..., fk cu tipurile t1, t2, ... tk. Se consideră că o cheie (a1, a2, ..., an) este inferioară unei chei (b1, b2, ..., bn) unde ai și bi sunt valori ale câmpului fi cu i=1, 2, ..., k dacă există o valoare j între 0 și k-1 astfel încât: a1=b1, a2=b2, ..., aj=bj și aj+1<bj+1. RadixSort sortează după compartimente (grupe) mai întâi după câmpul fk (considerat câmpul cel mai puțin semnificativ), apoi după câmpul fk-1, și așa mai departe până la câmpul f1 (considerat câmpul cel mai semnificativ). Complexitatea algoritmului este O(k*n).
Cerință: Cei trei algoritmi de sortare prezentați mai sus au fiecare avantaje și dezavantaje. Efectuați o comparație a acestora, evidențiind unde anume diferă, în ce gen de aplicație, pe ce tip de arhitecturi le-ați utiliza pe fiecare?

S5. "Agile" vs "Cascadă" vs „V". Avantaje vs Dezavantaje. La ce gen de proiecte le-ați recomanda? La ce fel de echipe le-ați recomanda? Nu descrieți modelele in sine ci efectuați doar comparația.

Precizări generale:
- Scrieți fiecare subiect pe câte o foaie separată, cu numele si grupa dvs;
- Alegeti 4 dintre cele 5 subiecte;
- Timp de lucru: 80 min

S4 2015:

Quick sort: When you don't need a stable sort and average case performance matters more than worst case performance. A quick sort is O(N log N) on average, O(N^2) in the worst case. A good implementation uses O(log N) auxiliary storage in the form of stack space for recursion.

Advantage: Pretty fast in most cases

Disadvantage: The worst-case complexity of quick sort is O(n^2), which is worse than the O(n log n) worst-case complexity of algorithms like merge sort, heapsort, binary tree sort, etc.

Merge sort: When you need a stable, O(N log N) sort, this is about your only option. The only downsides to it are that it uses O(N) auxiliary space and has a slightly larger constant than a quick sort. There are some in-place merge sorts, but AFAIK they are all either not stable or worse than O(N log N). Even the O(N log N) in place sorts have so much larger a constant than the plain old merge sort that they're more theoretical curiosities than useful algorithms.

Avantaje: stabil, intotdeauna O(n log n), usurinta implementarii

Dezavantaje: foloseste multa memorie

Heap sort: When you don't need a stable sort and you care more about worst case performance than average case performance. It's guaranteed to be O(N log N), and uses O(1) auxiliary space, meaning that you won't unexpectedly run out of heap or stack space on very large inputs.

Avantaje: foarte eficient, consuma putina memorie, simplitate, performanta consistenta

Dezavantaje: instabil, mai incet decat quick si merge

Introsort: This is a quick sort that switches to a heap sort after a certain recursion depth to get around quick sort's O(N^2) worst case. It's almost always better than a plain old quick sort, since you get the average case of a quick sort, with guaranteed O(N log N) performance. Probably the only reason to use a heap sort instead of this is in severely memory constrained systems where O(log N) stack space is practically significant.

Insertion sort: When N is guaranteed to be small, including as the base case of a quick sort or merge sort. While this is O(N^2), it has a very small constant and is a stable sort.

Bubble sort, selection sort: When you're doing something quick and dirty and for some reason you can't just use the standard library's sorting algorithm. The only advantage these have over insertion sort is being slightly easier to implement.

Non-comparison sorts: Under some fairly limited conditions it's possible to break the O(N log N) barrier and sort in O(N). Here are some cases where that's worth a try:

Counting sort: When you are sorting integers with a limited range.

Radix sort: When log(N) is significantly larger than K, where K is the number of radix digits.

Avantaje: foarte rapid daca K e mic

Dezavantaje: consum de memorie, mai incet in cazurile reale, inutil pt K prea mare

Bucket sort: When you can guarantee that your input is approximately uniformly distributed.

Modele ale Procesului de dezvoltare software

Ca orice produs fabricat complex, un program este realizat urmând un anume proces.

Un proces de dezvoltare a programelor se bazează pe o formalizare a activităţilor specifice, la care ne-am referit. Scopul formalizării este obţinerea unui ansamblu de mecanisme care, în cazul în care sunt aplicate sistematic permit obţinerea într-un mod repetitiv şi fiabil de produse software de calitate constantă. Descrierea procesului rămâne generală, pentru că nu este posibil să se definească un standard unic adaptat la toate tipurile de aplicaţii şi la toate persoanele.

Dezvoltarea unui program poate fi văzută ca stabilirea unui şir de descrieri din ce în ce mai precise şi din ce în ce mai apropiate de un program executabil şi de documentaţia sa. Trecerile de la o descriere la alta sunt deseori numite rafinări succesive. Aceasta este o vedere simplificată, pentru ca nu ia în considerare natura iterativă a procesului de dezvoltare.

O dată dezvoltat un program, este pus în exploatare. Se pun atunci probleme de întreţinere, de corectare a defectelor, de ameliorare a anumitor caracteristici sau de urmărire a evoluţiei cerinţelor. Intreţinerea poate impune modificarea funcţiilor sistemului şi deci, un proces de redezvoltare. Din aceasta cauză se vorbeşte despre ciclul de viaţă al unui program, când considerăm exploatarea şi întreţinerea în plus faţă de dezvoltare.

Ce este ciclul de viata al unui program (Software life cycle)?

- O secventa de etape in existenta produsului software. Sunt incluse toate activitatile necesare pentru dezvoltarea produsului si relatiile temporale dintre ele.
- Fiecare etapa din ciclul de viata este caracterizata prin activitati specifice si produsele rezultate din activitatile respective.

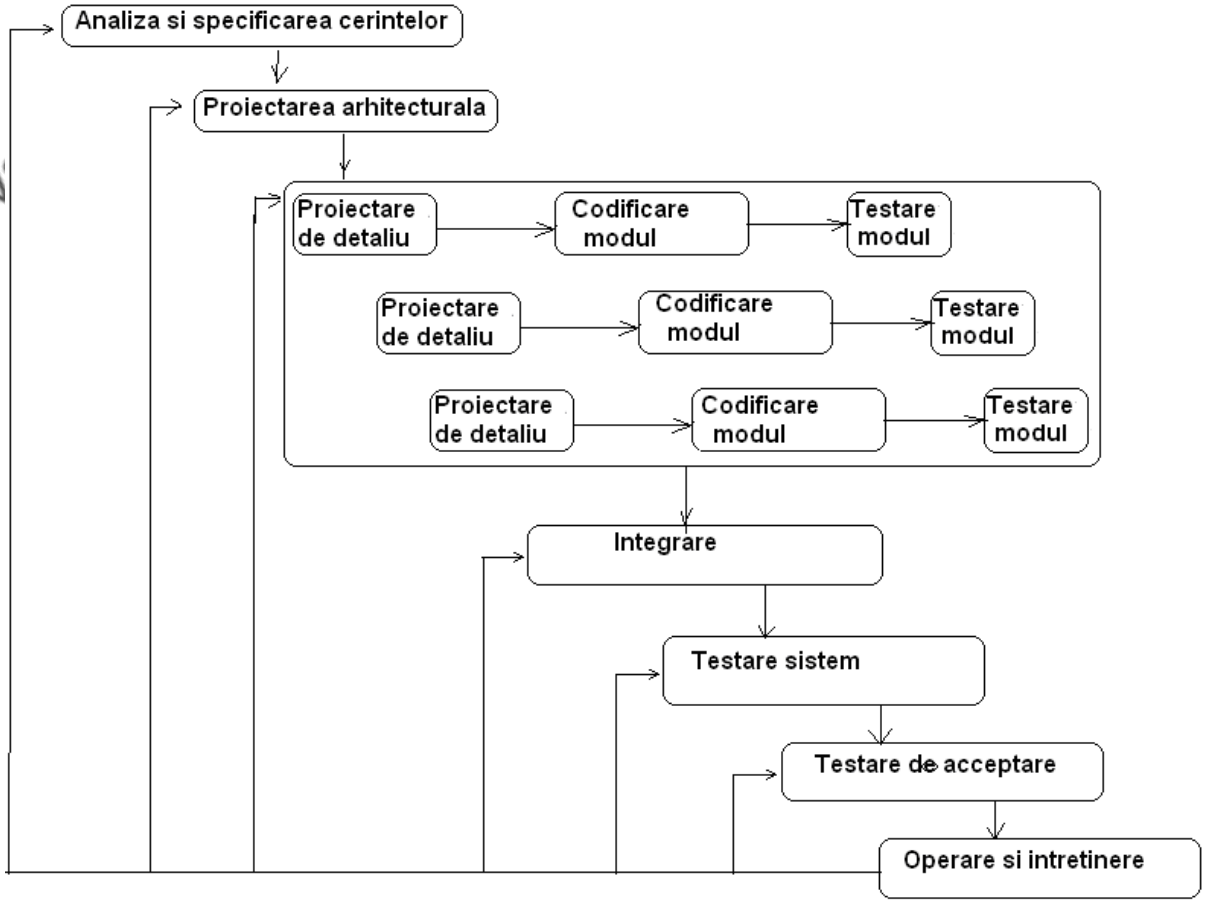
Modele ale ciclului de viata software (Software development life cycle models or process models):

- Waterfall model (Modelul cascada)
- V model (modelul in V)
- ESA (European Space Agency) model
- Incremental model (Modelul iterativ si incremental)
- Dezvoltarea “agila”
- Prototyping (Dezv pe baza de prototip)
- Spiral model (Modelul in spirala)

Modelul cascada

- Numit si **modelul clasic al ciclului de viata** sau **modelul liniar**
- Descris de Royce în 1970, a fost larg utilizat de atunci, pentru descrierea generală a procesului de dezvoltarea programelor.
- Ciclul de viaţă în cascadă prezintă dezvoltarea unui program ca o succesiune de faze ce se înlănţuie într-o derulare liniară, de la analiza cerinţelor şi până la livrarea produsului către client.
- Fiecare fază corespunde unei activităţi şi trebuie să se termine la o anumită dată prin producerea anumitor documente sau programe.
- Rezultatele fazei sunt supuse unei revizii aprofundate şi nu se trece la faza următoare decât atunci când sunt considerate satisfăcătoare.

va urez restanta usoara zgarcitilor



- In prima sa versiune, modelul avea numai săgeţi descendente, care materializează înlănţuirea etapelor; el nu prevedea iteraţii. Săgeţile ascendente au fost adăugate mai târziu pentru a ilustra principiul că o etapă repune în discuţie numai etapa precedentă.

Limitele modelului în cascadă

- Modelul în cascadă se bazează pe o secvenţă de faze bine delimitate. Documentele produse de fiecare fază sunt evaluate în cadrul reviziilor care validează trecerea de la o fază la alta. Din păcate, proba efectivă a bunei sau a proastei funcţionări a sistemului este realizată numai în cadrul fazei de integrare, când este posibilă evaluarea concretă a programului. Inaintea acestei faze au fost produse numai documente.

va urez restanta usoara zgarcitilor

- Abordarea în cascadă dă rezultate satisfăcătoare numai în cazul în care este efectiv posibilă înlănţuirea fazelor fără prea multe probleme. Aceasta presupune ca ansamblul cerinţelor să fie perfect cunoscut şi problema complet înţeleasă de analişti. Trebuie de asemenea ca soluţia să fie uşor de determinat de proiectanţi şi codificarea simplă - redusă ideal la generarea automată a codului plecând de la documentele de proiectare.
- In realitate se constată că partea de necunoscut poate fi însemnată în anumite dezvoltări, în special datorită:
  - neînţelegerii cerinţelor de către client sau analist;
  - instabilităţii cerinţelor;
  -alegerilor tehnologice;
  -schimbărilor de personal.
- Din toate aceste motive sunt necesare reveniri în etape anterioare ale procesului de dezvoltare. Aceste reveniri sunt de fapt o reflectare a realităţii. Dacă aceste reveniri sunt ocazionale şi limitate la faze adiacente, modelul în cascadă este pertinent. In caz contrar, modelul în cascadă nu corespunde realităţii.

Avantaje:

1. Sistemul este bine documentat
2. Permite un bun management al proiectului:
   - planificarea resurselor umane pe etape
   - estimari de cost mai exacte

Dezavantaje:

1. Un produs executabil, care sa demonstreze functionarea sistemului este disponibil destul de tarziu, dupa integrare. Pana atunci s-au produs numai documente.
2. Deoarece modelul este secvential, exista numai uhn feedback local, la tranzitiile intre faze.
3. Multe erori sunt descoperite tarziu ☹cost crescut
4. Toate riscurile sunt incluse intr-un singur ciclu de dezvoltare

va urez restanta usoara zgarcitilor

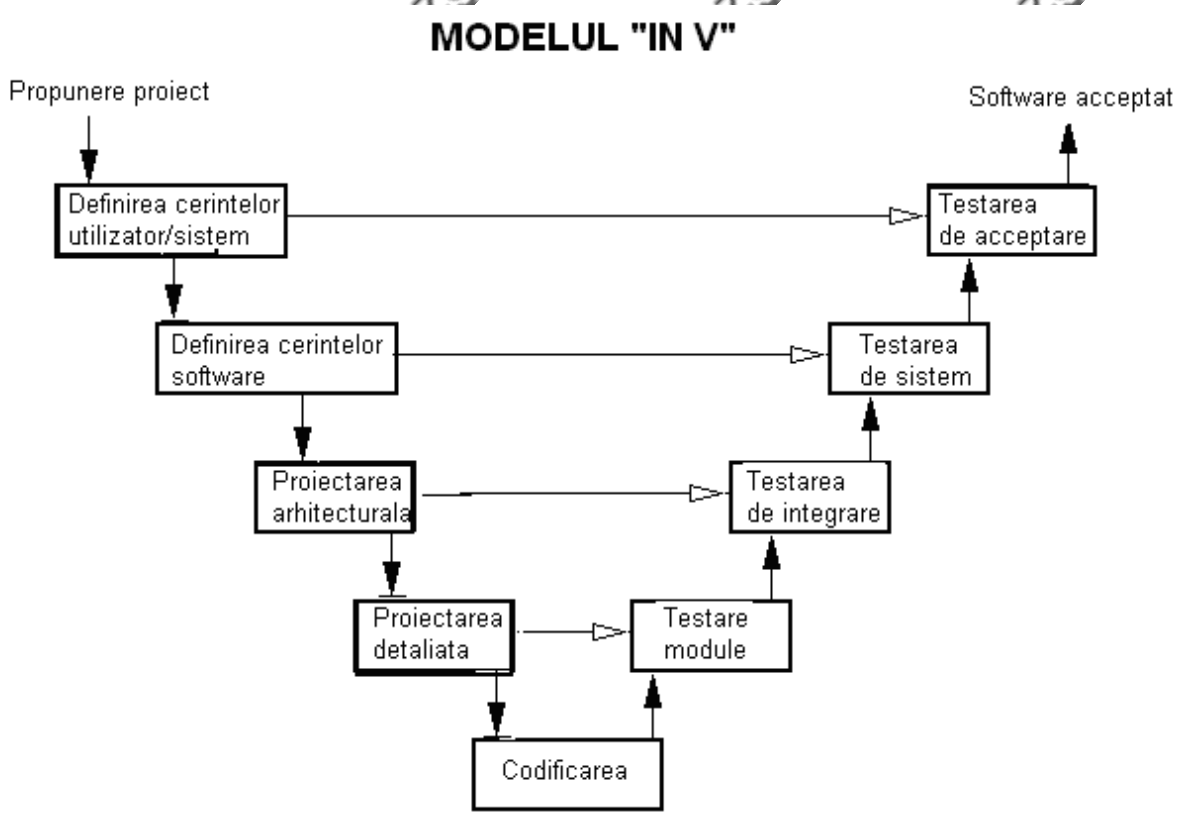
Adecvat pentru proiectele in care cerintele sunt bine intelese de la inceput si nu se modifica pe parcursul procesului de dezvoltare.

Experienta ultimelor decenii a demonstrat ca modelul este valoros.

Este utilizat si in prezent de multe organizatii mari.

Modelul in V

- Este o varianta a modelului cascada, care pune in evidenta corelarea dintre activitatile de specificare si cele de testare, inlantuirea in timp a activitatilor fiind aceeasi.

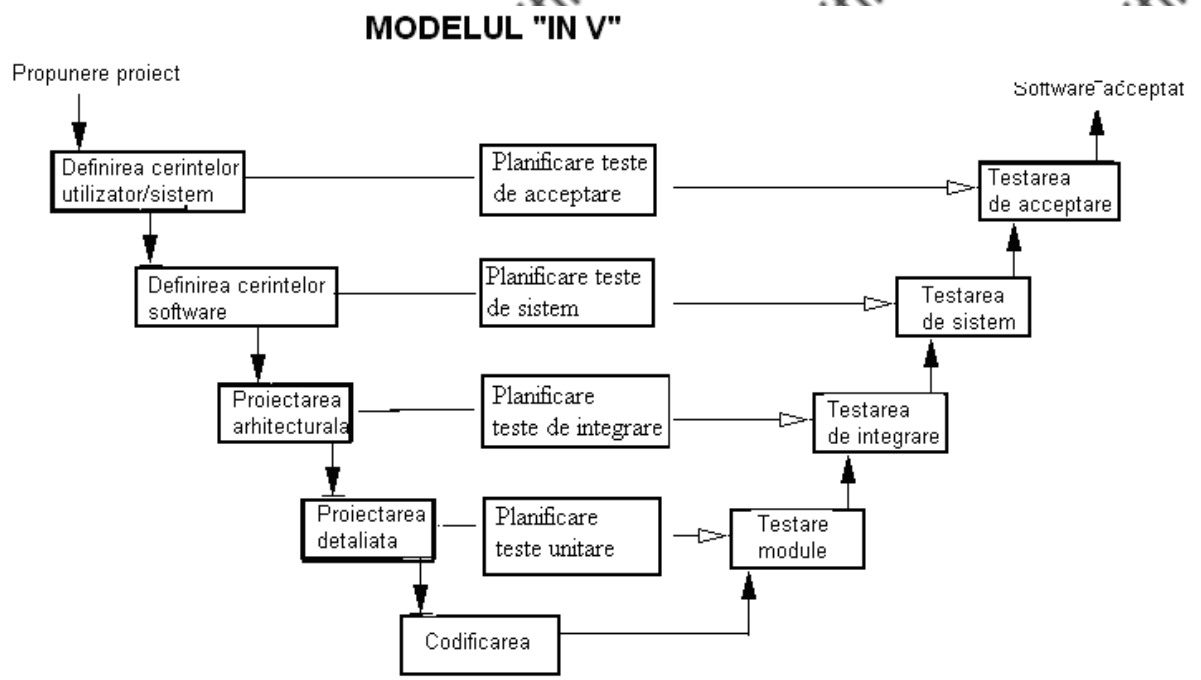


Partea din stanga reprezinta lantul de specificare a sistemului iar cea din dreapta lantul de testare. Partea de jos a v-ului reprezinta implementarea

- In acest model există două tipuri de dependenţe între etape:

-cele reprezentate prin liniile care formează V-ul, care corespund înlanţuirii şi eventualei iteraţii din modelul cascadă; etapele se derulează deci secvenţial, urmând V-ul de la stânga la dreapta;

-cele reprezentate prin linii orizontale, care indică faptul că o parte din rezultatele etapei din care pleacă săgeata sunt utilizate direct în etapa ţintă. De exemplu, la terminarea etapei de proiectare arhitecturală, cazurile de teste de integrare trebuie să fie complet descrise.



Prototiparea

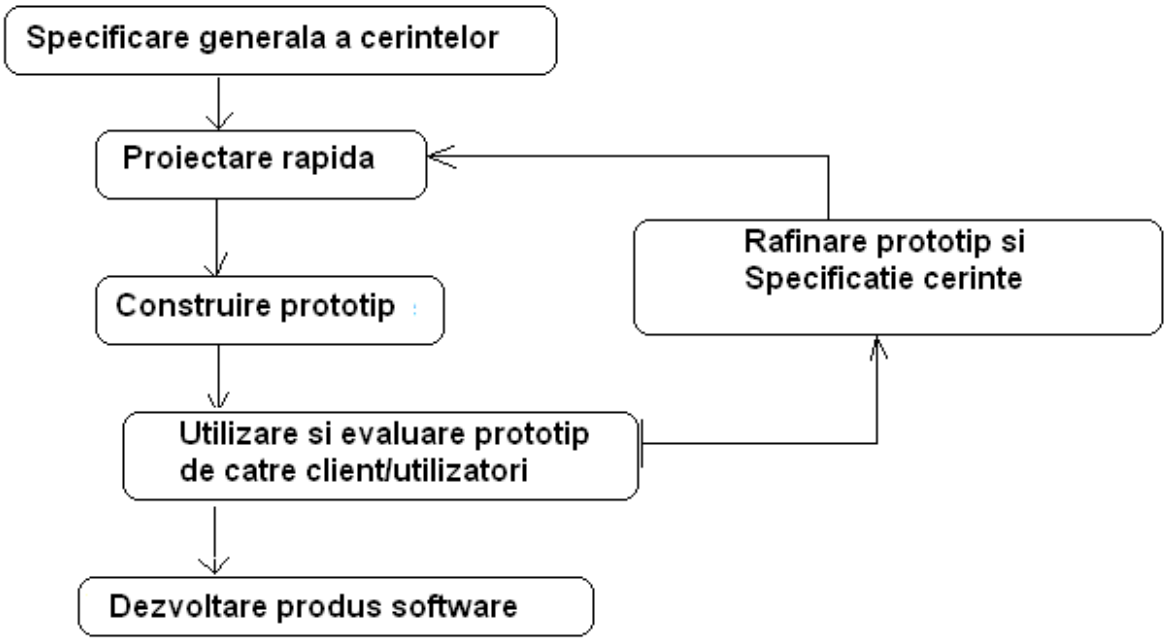
Pentru sisteme noi, în mod special dacă ele sunt mari şi complexe este puţin probabil să se construiască o specificaţie completă, logică şi validă înainte ca sistemul sa fie construit şi utilizat. Acest fapt a condus la ideea prototipării. Ideea este de a permite viitorilor utilizatori să exerseze cu o primă versiune a sistemului, care poate fi obţinută rapid prin tehnici de simulare şi/sau de interpretare a specificaţiilor. In acest ultim caz, limbajele logico-funcţionale sunt în mod sigur chemate să joace un rol important.

Prototipul este o schiţă a viitorului sistem: el nu are performanţele şi nu răspunde exigenţelor de calitate ale unui produs finit. Prototipul oferă clientului functionalităţi (nu în totalitate) ale viitorului sistem şi interfaţa sa utilizator. El este dezvoltat într-o manieră iterativă. Cerinţele sunt extrase şi validate iterativ prin utilizarea prototipului. In fiecare iteraţie specificaţia sistemului este modificată şi detaliată până când prototipul satisface necesităţile utilizatorilor.

Un prototip care este utilizat pentru a desprinde cerinţele viitorilor utilizatori, este o "machetă exploratoare".

Activitatea de prototipare poate interveni de asemenea în etapa de proiectare, pentru a experimenta şi compara diferite variante. Astfel de prototipuri se numesc "machete experimentale".

Figura următoare redă procesul de prototipare dedicat extragerii cerinţelor:



Pentru dezvoltarea prototipului se folosesc limbaje de nivel foarte înalt: Smalltalk, PROLOG, LISP, SETL şi altele. In prezent există limbaje şi medii de

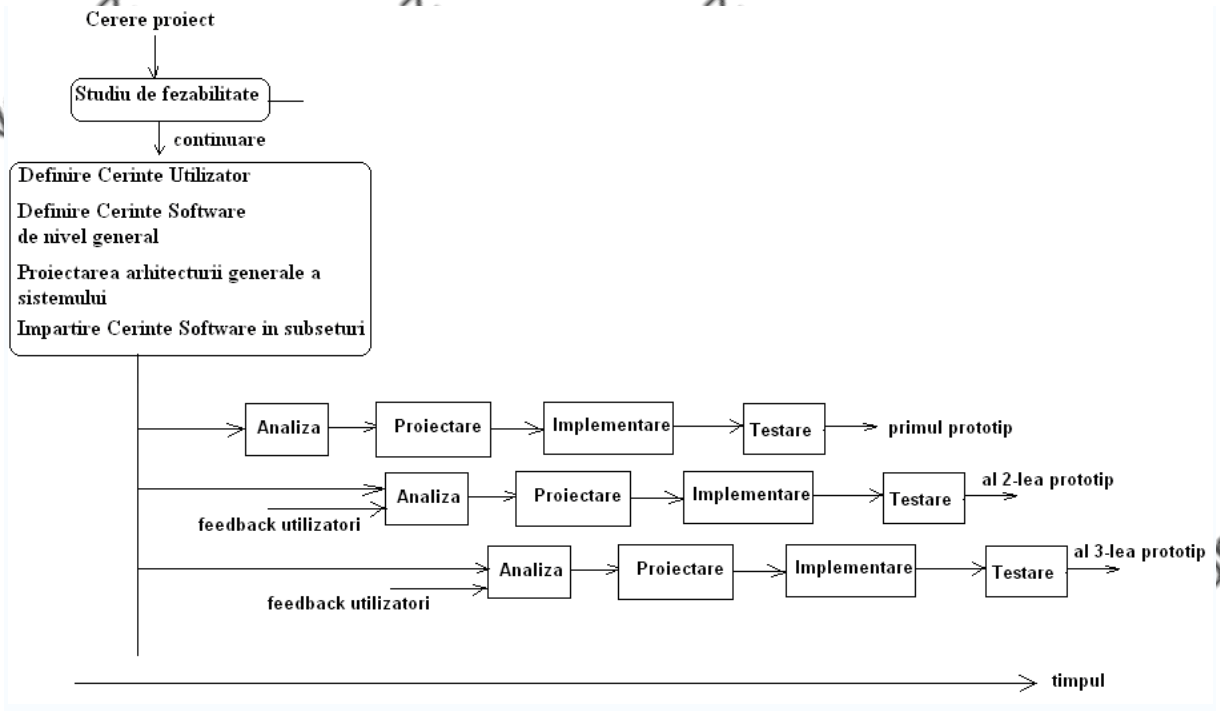
dezvoltare specializate pentru construirea prototipurilor. Astfel de limbaje sunt limbajele de generaţia a 4-a (4GL).

Cu toate că se poate folosi acelaşi limbaj de programare, atât pentru realizarea prototipului cât şi pentru dezvoltarea aplicaţiei, se recomandă ca prototipul să fie considerat un sistem "închis", adică să nu fie folosit ca bază pentru dezvoltarea aplicaţiei. Aceasta deoarece:

- prototipul a fost dezvoltat prin modificări succesive, de aceea arhitectura sa iniţială a fost alterată, ceea ce îngreunează întreţinerea;
- prototipul trebuie să corespundă cerinţelor utilizatorilor numai din punct de vedere funcţional. De aceea, în dezvoltarea sa sunt neglijate aspecte ca: eficienţa, adaptabilitatea, compatibilitatea şi chiar fiabilitatea.

Modelul Incremental

- Modelul ciclului de viaţă "Incremental" este opus modelului « in cascada ».
- El se bazează pe o idee foarte simplă: dacă un sistem este prea complex pentru a fi înţeles, conceput sau realizat intr-o singura fază este mai bine sa fie realizat în mai multe faze, prin evoluţie.



Cum se aplica

- Intr-o faza initiala:
  - Se studiaza scopul proiectului si fezabilitatea sa. In cazul deciziei de continuare a proiectului:
  - Se detaliaza cerintele utilizatorilor si se efectueaza o analiza de nivel inalt, pentru a se determina cerintele software la un nivel general.
  - Se determina o arhitectura generala a sistemului.
  - Se impart cerintele in subseturi care pot fi implemenate in incremente separate. Se stabileste planificarea in timp a incrementelor.
- Fiecare increment implementeaza un subset de cazuri de utilizare de nivel inalt, care exprima cerintele utilizatorilor. El este construit urmand abordarea "cascada": analiza detaliata a cerintelor din subset, proiectarea, implementarea si testarea. Rezultatul este un produs care satisface un subset al cerintelor si este livrat utilizatorilor.
- Un produs tipic consta din 10-50 incremente.

- Se incepe cu o implementare simpla a unui subset al cerintelor software. Rezulta o prima livrare.
- Scopul primei implementari este de a crea un produs la care utilizatorul poate reactiona. El trebuie sa puna in evidenta aspectele cheie ale problemei si sa furnizeze o solutie sufient de simpla pentru a fi inteleasa si usor de implementat.
- Fiecare noua iteratie include analiza ultimei versiuni si adaugarea de noi functionalitati, ceea ce presupune reproiectarea, codificarea si testarea. Se urmareste ca proiectarea sa fie directa, modulara, sa suporte reproiectarea.
- Analiza unei iteratii se bazeaza pe feedback-ul utilizatorului si pe instrumentele de analiza disponibile. Ea se refera la: modularitea produsului, cuplarea modulelor, utilizabilitatea, fiabilitatea, eficienta si realizarea scopurilor.
- Fiecare iteratie este o varianta imbunatatita a celei anterioare, de aceea metoda se mai numeste "imbunatatirea iterativa" (iterative enhancement).
- Se utilizeaza masuri pentru evaluarea evolutiei sistemului. Masurile prin care se evalueaza un software sunt dificil de inteles ca valori absolute, dar schimbarile valorilor lor in evolutia unui sistem sunt o baza de comparatie. Astfel de masuri

sunt: numarul de defecte, efortul de actualizare, dimensiune, complexitatea, cuplarea modulelor. Schimbarile diferitelor aspecte se pot monitoriza si se pot stabili limite pentru anumite masuri, pentru a semnala probleme sau anomalii.

- Utilizarea analizei si a masuratorilor ca ghid in procesul iterativ este o diferenta majora intre aceasta metoda si metodele de “dezvoltare agila” (agile software development). Ele sunt suportul pentru determinarea eficientei procesului si a calitatii produsului.
- Fiecare iteraţie a ciclului de viaţă iterativ si incremental reproduce ciclul de viaţă în cascadă la o scară mai mică. Obiectivele unei iteraţii sunt stabilite pe baza evaluării iteraţiilor precedente. Documentaţia este construită treptat, în timpul fiecărei iteraţii. La sfârşitul dezvoltării, documentele obţinute au aceeaşi formă cu cele obţinute în maniera convenţională.

Avantaje:

- In fiecare etapa este livrat un produs executabil, care satisface o parte din cerintele utilizator. Opus modelului cascada in care se elaboreaza documente.
- Prototipurile sunt livrate clientului/utilizatorilor.
- Feedback-ul utilizatorilor este distribuit pe intreg parcursul dezvoltarii.
- In cazul aparitiei unor schimbari in cerinte acestea pot fi incoporate in urmatorul prototip.

Ciclul de viaţă iterativ se bazează pe evoluţia prototipurilor executabile, măsurabile şi deci pe evoluţia elementelor concrete. El este opus din acest punct de vedere ciclului de viata în cascadă care se bazează pe elaborarea de documente. Livrările forţează echipa să dea rezultate concrete în mod regulat.

Integrarea diferitelor componente ale programului este realizată într-o manieră progresivă în timpul construcţiei.

Progresele se măsoară prin programe demonstrabile mai degrabă decât prin documente sau estimări, ca în ciclul de viaţă în cascadă;

In cursul dezvoltării, clientul poate utiliza prototipurile. Demonstrarea prototipurilor prezintă numeroase avantaje:

- utilizatorul este pus în faţa situaţiilor de utilizare concrete care-i permit să-şi definească mai bine dorinţele şi să le comunice echipei de dezvoltare;
- utilizatorul devine partener al proiectului; el îsi ia partea de responsabilitate în noul sistem şi de fapt îl acceptă mai uşor;

Dezavantaje

- Ciclul de viaţă Incremental se bazează pe evoluţia prototipurilor executabile. Din nefericire, programele nu se pretează în mod natural evoluţiei, din contră, ele sunt deseori foarte "fragile" la modificări. Efectul unei modificări locale se poate propaga în ansamblul aplicaţiei. Fiecare nou increment poate necesita reorganizarea structurii interne, degradand arhitectura initiala a programului, facându-l greu de verificat şi de întreţinut.
- Erorile de proiectare sunt mai greu de eliminat.
- Abordarea obiect bazată pe modularitate şi încapsulare conduce la programe mai robuste şi mai rezistente faţă de schimbări. Din acest punct de vedere, abordarea obiect furnizeaza un cadru confortabil pentru dezvoltarea prin evoluţie, într-o manieră iterativă.
- Clientul poate vedea ce se poate face si poate cere mai mult!
- Abordarea incrementala se poate transforma usor intr-una « codifica si repara « (« build and fix »).

Planificarea

Principalul risc in utilizarea unui model incremental este de a lucra intr-o manierea “ad-hoc”. Determinarea unui plan de actiuni este de prima importanta pt succesul abordarii incrementale. In faza de **analiza preliminara** se determina scopul proiectului si se incearca determinarea riscurilor majore ale proiectului, se determina o lista o cerintelor si constrangerilor mai importante, pt a construi un plan de dezvoltare. Se stabileste natura fiecarui icrement si ordinea de construire a incrementelor.

va urez restanta usoara zgarcitilor

O varianta a modelului este **“Numai implementarea incrementala”:**

- Urmeaza modelul cascada pana in faza de implementare
- In timpul analizei cerintelor si proiectarii de sistem:
  - Se definesc subseturi / subsisteme care pot fi livrate
  - Se definesc interfete care permit adaugari iterative simple, cu un minim de modificari a arhitecturii existente
- Diferitele parti sunt implementate, testate si livrate in functie de diferite prioritati si la diferite momente de timp

Construirea in paralel a incrementelor: risc

Diferite incremente pot fi construite in paralel de diferite echipe.Dupa inceperea fazei de proiectare a primului increment, echipa de specificare incepe specificarea celui de-al 2-le increment. Riscul este ca cele 2 incremente sa nu “se potriveasca”. In mod norma, al 2-lea trebuie sa-l includa pe primul. Fiecare increment are parti comune cu altele. Este necesara o buna comunicare si coordonare intre echipele care construiesc in paralel incrementele care au parti comune (privind implementarea partilor comune). Aceste probleme cresc exponential cu numarul de incremente.

Rational Unified Process (RUP: www.rational.com**)**

Este un proces de dezvoltare iterativ. Cei care l-au definit, Ivar Jacobson, Grady Booch, and James Rumbaugh, il caracterizeaza astfel:

Dirijat de cazurile de utilizare

Modelul cazurilor de utilizare descrie complet functionalitatea sistemului. Cazurile de utilizare dirijeaza procesul de dezvoltare: dezvoltatorii creaza modele de proiectare si implementare pentru a realiza cazurile de utilizare iar testorii testeaza sistemul pentru a verifica daca sistemul implementeaza corect cazurile de utilizare.

va urez restanta usoara zgarcitilor

Centrat pe arhitectura

Arhitectura sistemului este aspectul cel mai important al sistemului. Arhitectul selecteaza cazurile de utilizare care reprezinta functile cheie ale sistemului (5%-10% din cazurile de utilizare), le specifica in detaliu si le realizeaza in termeni de subsisteme, clase, componente.

Inerativ si incremental

Proiectul de dezvoltare software este divizat in mini-proiecte, fiecare fiind o iteratie din care rezulta un increment. Fiecare iteratie are de-a face cu cele mai importante riscuri si realizeaza un grup de cazuri de utilizare care impreuna extind utilizabilitatea produsului. In fazele initiale, un proiect initial poate fi extins cu unul mai detaliat. In fazele urmatoare, incrementele sunt in mod tipic aditive.

Agile software development

- Este un cadru conceptual pentru proiectele software. Exista mai multe metode de “dezvoltare agila”, cum sunt cele expuse de “Agile Alliance”, o organizatie non-profit.

Pentru orice proiect software exista o serie de factori de risc care pot periclita finalizarea cu succes a proiectului, cum ar fi: factori de experienta (conducatorului de proiect, a echipei), factori de planificare, factori tehnologici, factori externi (modificarea cerintelor, modificarea interfetelor externe).

- Cele mai multe metode de dezvoltare agila incearca sa minimizeze riscul dezvoltand software-ul in intervale scurte de timp, numite “timeboxes”, constand din 1-4 saptamani. Data de sfarsit a unui “timebox” nu poate fi modificata. Daca echipa depaseste data, iteratia este anulata si replanificata.
- Fiecare iteratie este un proiect software in miniatura si include toate activitatile necesare livrarii mini-incrementului unei noi functionalitati: planificare, analiza cerintelor, proiectare, codificare, testare, documentare.
- Fiecare noua iteratie trebuie sa livreze un nou produs. La sfarsitul fiecarei iteratii echipa re-evalueaza prioritatile proiectului.

Timebox-urile sunt utilizate in ,metodele de dezvoltare agila ca o forma de management al riscului in dezvoltarea unui software.

Metodele “agile” pun in evidenta comunicarea directa intre participantii la elaborarea unei iteratii, in locul documentelor scrise. Acestia lucreaza in aceeasi locatie, intre ei fiind cel putin programatorii si “clientii” lor (cei care definesc produsul). Echipa poate include testori, proiectanti de interactiune, echipe de documentare tehnica si manageri.

Principala masura a progresului este considerata software-ul functional. Se produce foarte putina documentatie, motiv pentru care metodele sunt criticate.

Pentru mai multe informatii despre metodele de dezvoltare agila: http://www.agilealliance.org/.

Comparatie cu alte metode

Considerand ca metodele de dezvoltare pot fi plasate pe o scara continua de la cele adaptive la cele predictice, metodele agile se situeaza la extremitatea celor adaptive.

<--Agile--> <--Iterative/Incrementale--> <--Cascada-->
<----------|----------------------------------|---------------|->
Adaptive Predictive

Metodele adaptive sunt focalizate pe adaptarea rapida la schimbari. Nu se au in vedere cu exactitate ce se va intampla in viitor. O echipa adaptiva poate raporta exact ce sarcini vor fi realizate saptamana urmatoare si ce este planificat pentru luna urmatoare.

Metodele predictive sunt focalizate pe planificarea in detaliu a activitatilor, in timp. O echipa predictiva poate raporta exact ce este planificat pentru intreagul proces de dezvoltare. Echipa predictiva are dificultati in schimbarea directiei. Planul este optimizat pentru desinatia originala iar schimbarea directiei poate necesita renuntarea la rezultatele curente si replanificarea activitatilor. Numai schimbarile considerate importante sunt luate in considerare.

In comparatie cu metodele incrementale:

- asemanare- creaza produse livrabil in perioade de timp scurte
- deosebiri:
  - perioadele de timp sunt masurate in saptamani si nu luni
  - perioadele de timp sunt stricte (time-box-uri)
  - in metodele incrementale procesul este ghidat de analiza si masurare a caracteristicilor produsului. Ele sunt suportul pentru determinarea eficientei procesului si a calitatii produsului

In comparatie cu modelul cascada

Dezvoltarea agila are foarte putine in comun cu dezvoltarea cascada. Modelul cascada are si in prezent o utilizare larga.. Modelul cascada este cel mai predictiv, activitatile sunt pre-planificate intr-o secventa. Progresul este masurat prin produse intermediare (specificatii, doc de proiectare, planuri de testare, revizii ale codului, etc). Efortul de integrare si testare poate fi foarte mare (cateva luni – cativa anui!), fiind una dintre cauzele de esec ale unei dezvoltari cascada. Prin dezv agila se produc produse executabile intervale de cateva saptamani sau luni. Se pune accentul pe obtinerea de executabile cat mai repede apoi imbunatatirea lor continua!

Extreme Programming, este o metoda de dezvoltare agila care a crescut popularitatea metodelor agile.

Extreme Programming (**XP**) este un model incremental bazat pe iteratii scurte, testare intensiva si simplitate. Este adecvata pentru echipe mici si proiecte caracterizate prin schimbari rapide ale cerintelor.

Informatii suplimantare despre XP pot fi gasite la: www.extremeprogramming.org, www.xprogramming.com.

Modelul spirala

Barry Boehm a definit acest model plecand de la slabiciunile modelului cascada, in special lipsa sa de flexibilitate la schimbari ale cerintelor.

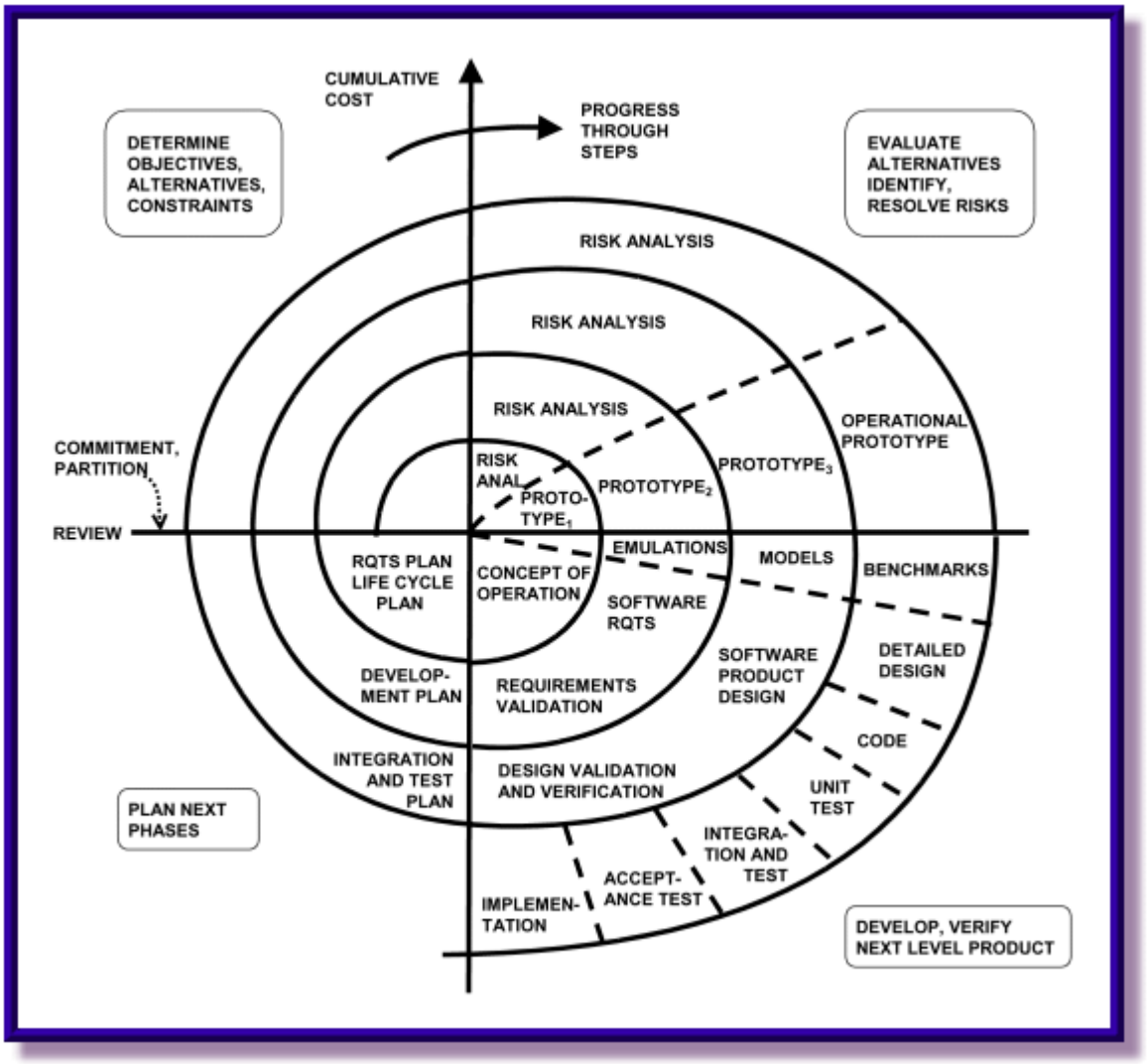
Este focalizat pe analiza riscurilor in mod incremental, repetand modelul cascada intr-o serie de cicluri, ca in figura urmatoare.

Fiecare ciclu consta din 4 faze.

1. **Determinarea obiectivelor:** definirea produsului, determinarea scopurilor si a constrangerilor, generarea alternativelor.
2. **Evaluarea alternativelor**: analiza riscurilor, prototiparea
3. **Dezvoltarea produsului**: proiectarea de detaliu, testarea unitara, integrarea, ….
4. **Planificarea urmatorului ciclu**: evaluarea de catre client, planificarea dezvoltarii si a livrarii catre client.

- Este o imbunatatire a modelului cascada deoarece prevede mai multe livrari si mai multe posibilitati de implicare a clientului.
- Raza spiralei reprezinta costul acumulat.

Informatii suplimentare: http://www.stsc.hill.af.mil/crossTalk/2001/05/boehm.html.



Ulterior Boem a sugerat o versune modificata a modelului – modelul spirala WinWin, care adauga la inceputul fiecarui ciclu activitati de identificare a partilor interesate in proiect (“stakeholders”) si conditiile lor de castig (“win conditions”).

Diagrame de clase

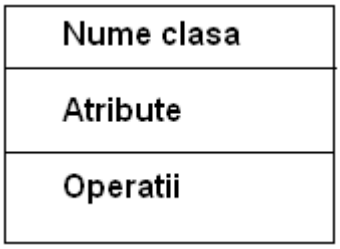
Clase

- O clasă de obiecte reprezintă un grup de obiecte care au:
  - proprietăţi similare (atribute),
  - un comportament comun (operaţii),
  - relaţii comune cu alte obiecte şi
  - o aceeaşi semantică.

De exemplu, "Persoana", "Firma", "proces" sunt clase de obiecte.

- Semantica asociată unei clase corespunde unui punct de vedere. Obiectele din lumea reală pot fi abstractizate în mod diferit. De exemplu, un cal poate fi încadrat în clasa mijloacelor de transport terestre sau în clasa animalelor.

In UML, o clasă este reprezentată printr-un dreptunghi alcatuit din trei compartimente care conţin: numele clasei, atributele, operaţiile. Compartimentul atributelor şi cel al operaţiilor pot fi omise.



Clasele definite in etapa de „Analiza a cerintelor" contin numai numele clasei si atributele (fara specificarea tipului fiecarui atribut), eventual unele operatii.

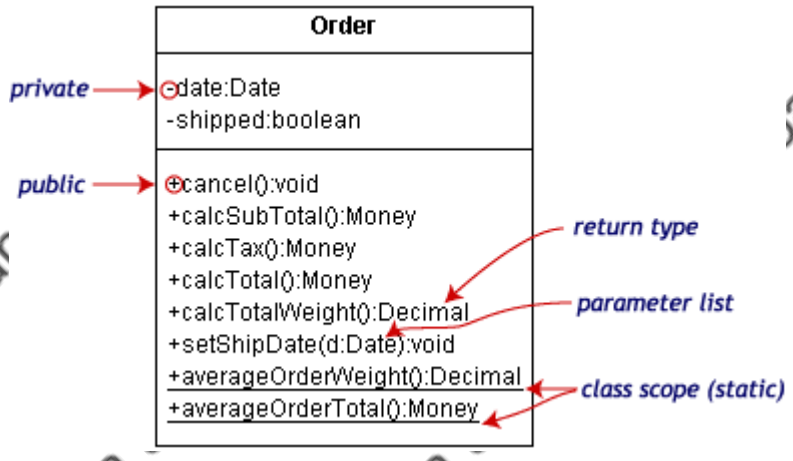
Reprezentarea detaliata a unei clase,construita in etapa de proiectare de detaliu, precizeaza vizibilitatea informatiilor din clasa, lista de parametri a fiecarei operatii si tipul parametrilor.

va urez restanta usoara zgarcitilor

Regulile de vizibilitate se aplica atat atributelor cat si operatiilor din clase si se refera la domeniul de acces permis la un membru al unei clase. Fiecare nivel de vizibilitate este reprezentat printr-un simbol:

- Private (-) : accesibiltate numai din interiorul clasei
- Public (+) : accesibiltate la nivelul intregului sistem
- Protected (#) : accesibiltate in arborele de mostenire
- Package (~) : accesibiltate din interiorul pachetului care contine clasa

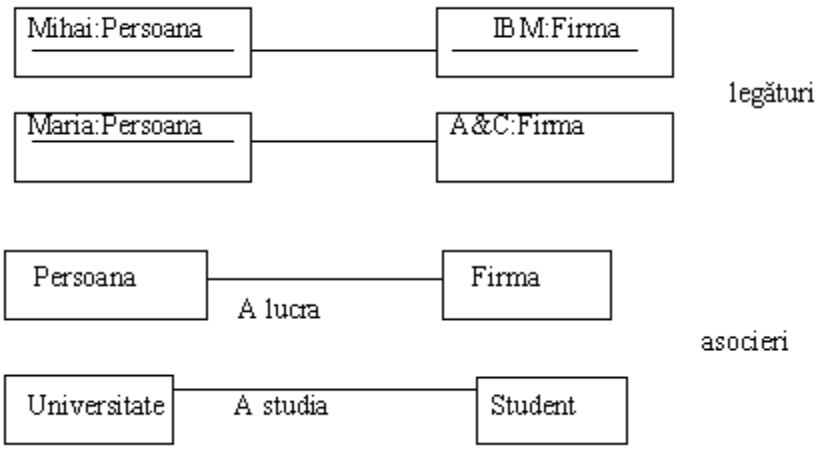
Membrii statici sunt subliniati.



Clasa detaliata

- Relaţiile dintre clase sunt abstracţii ale relaţiilor existente între obiecte. Fiecărei familii de legături între obiecte îi corespunde o relaţie între clasele obiectelor.
- Obiectele sunt instanţe ale claselor,
- Legăturile între obiecte sunt instanţe ale relaţiilor dintre clase.

va urez restanta usoara zgarcitilor

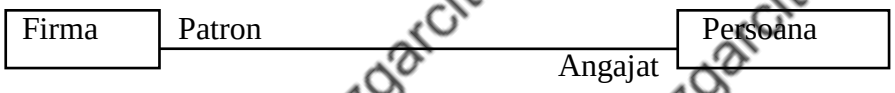


- Diagramele de clase redau structura statica a unui sistem software.
- Exista doua tipuri principale de relatii intre clase:
  - asociere si
  - generalizare

Asocierea

- Asocierea este o abstractie a unui set de legaturi existente intre obiecte. De exemplu, asocierea “A lucra” dintre clasele Persoana şi Firma reprezintă toate legăturile posibile dintre obiecte ale clasei Persoana şi obiecte ale clasei Firma:

- Extremitatea unei asocieri este numita **rol**.
  - Rolul exprimă felul în care o clasă "vede" o altă clasă în cadrul unei asocieri. De exemplu, în asocierea dintre clasele Firma şi Persoana, orice obiect al clasei Persoana este un “Angajat” al unei Firme, care este reprezentată printr-un “Patron” .
  - Numele de rol sunt amplasate la cele două extremităţi ale asocierii:



va urez restanta usoara zgarcitilor

Nume de rol.

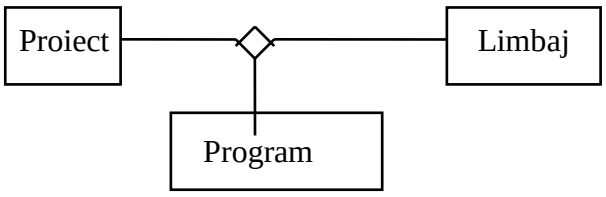
- Numele de rol sunt optionale. Numai numele asocierii este obligatoriu pentru o asociere.

Dacă între două clase există o singură asociere, numele asocierii este suficient pentru a preciza relaţia. Numele de rol se folosesc de regulă atunci când între două clase există mai multe asocieri. In loc de nume de rol se poate scrie un verb sau o fraza care contine un verb, pentru a explica semantica asocierii:

Observatie: *Executable UML* impune specificarea numelor de rol. Ele sunt folosite in codul generat automat pentru implementarea asocierilor (VEZI generarea automata a codului pornind de la diagramele de clase).

Aritatea asocierilor

Cea mai mare parte a asocierilor sunt binare - ele unesc două clase. Asocierile de aritate superioară se reprezintă cu ajutorul unui romb, ca în figura de mai jos. De exemplu, asocierea ilustrată în figura urmatoare exprimă faptul ca “Proiectele sunt implementate prin Programe scrise în Limbaje de programare”.

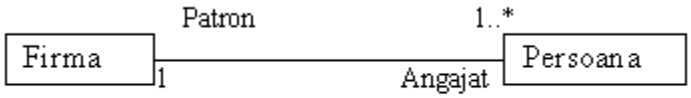


Asociere ternară.

Multiplicitatea asocierilor

va urez restanta usoara zgarcitilor

Fiecare rol al unei asocieri poate purta o indicaţie de multiplicitate care arată câte obiecte ale unei clase pot fi legate la un moment dat unui obiect al celeilalte clase. De exemplu, o firma poate avea unul sau mai mulţi angajaţi. O persoana poate lucra la o singura firma. Multiplicitatea poate fi "unu", "mai multe "(*) sau un subansamblu de întregi pozitivi: 1, 0..1, M..N, * sau 0..* (de la zero la mai mulţi), 1..*.



Multiplicitatea asocierilor.

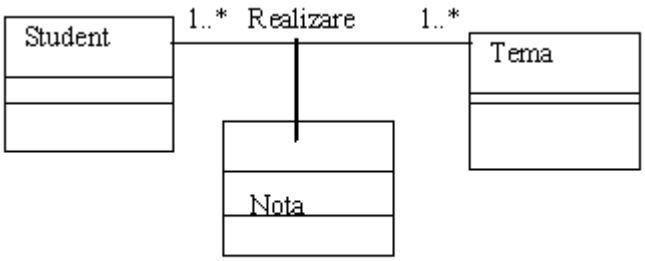
- Multiplicitatea unei asocieri exprimă o constrângere valabilă pe toată durata de existenţă a obiectelor claselor asociate. Multiplicităţile nu trebuie să fie considerate în timpul regimurilor tranzitorii, de creare sau de distrugere a obiectelor.

EXEMPLU de diagrama de clase conceptuale:→ librarie, Student-curs

Asocieri cu atribute

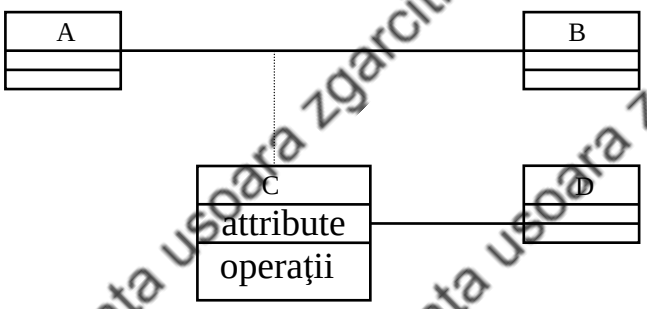
Asocierile pot fi caracterizate prin atribute. In figura urmatoare, “Nota” este un atribut al asocierii existente între clasa “Student” şi clasa “Tema”.
Asocierea dintre clasa Student şi clasa Tema este de tip N la N. Fiecare student realizează individual o tema dată iar nota obţinută nu poate fi reprezentată nici ca atribut al studentului (căci un student efectuează mai multe teme), nici ca atribut al temei (căci fiecare student primeşte o notă pentru aceeaşi tema). Nota este un atribut al relaţiei dintre clasa studenţilor şi clasa temelor.

va urez restanta usoara zgarcitilor



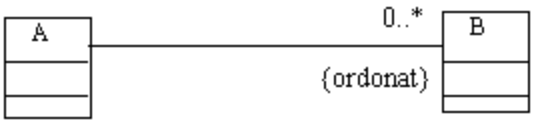
Asocieri cu atribute.

- O asociere cu atribute este numită asociere atributată. Ea poate fi reprezentată printr-o clasă cu atribute şi operaţii proprii. O asemenea clasă, numită uneori clasă-asociere, este ataşată asocierii printr-o linie punctată:



VEZI EXEMPLU CURS (Student-Curs-Nota, Angajat- Departament - Proiect)!

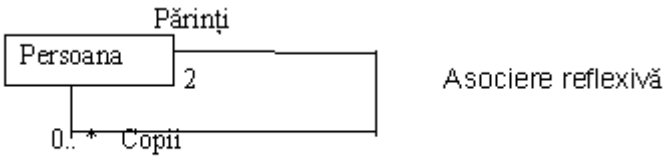
- Asocierile pot fi constrânse. O constrangere este o regula care trebuie sa fie implementata pentru ca asocierea sa fie valida. Constrângerile sunt scrise între acolade. De exemplu, instanţele clasei B asociate unei instanţe a clasei A trebuie să fie ordonate:



Asociere constrânsă.

- O asociere poate lega o clasă de ea însăşi. O asemenea asociere este numită asociere reflexivă. Un exemplu de asociere reflexivă este cel

ilustrat în figura urmatoare. Fiecare persoana are doi părinţi şi zero sau mai mulţi copii. Numirea rolurilor este în acest caz esenţială pentru claritatea diagramei.



- Asocierile 1 la mai mulţi şi mai mulţi la mai mulţi pot fi calificate. Calificarea este specificată printr-o cheie ataşată rolului clasei de plecare.

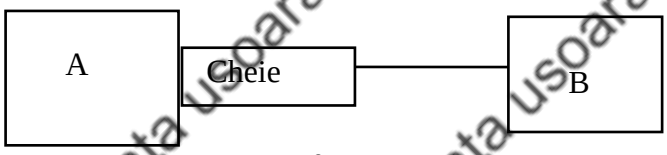


Fig. 2.27. Calificarea asocierilor

Fiecare instanţă a clasei A împreună cu valoarea cheii identifică un sub-ansamblu al instanţelor clasei B, care participă la asociere. Exemplu:

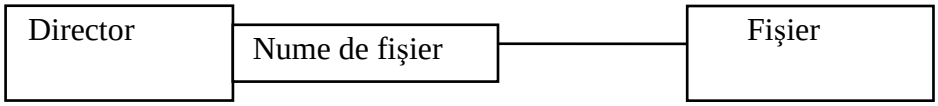


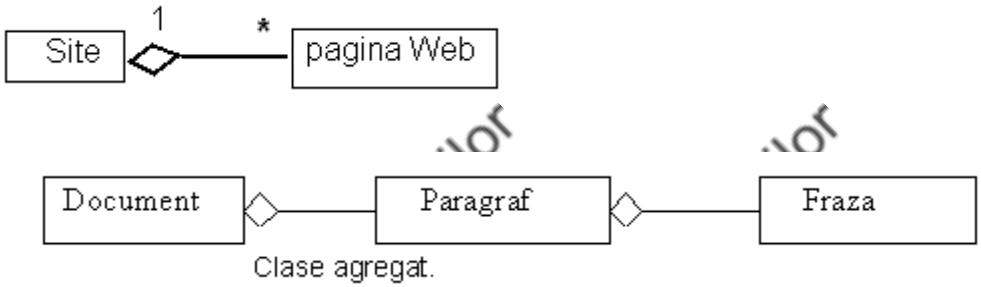
Fig. 2.28. Identificarea unui fişier.

Un director împreună cu un nume de fişier desemnează toate fisierele din director cu acelasi nume si diferite extensii.

Agregarea

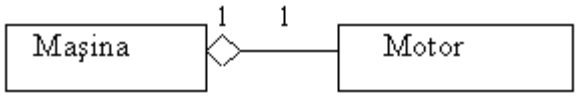
- Agregarea este o formă particulară de asociere care exprimă o relatie "compus - componenți".

- O clasa **are** anumite parti, care insa au o existenta independenta. Agregarea este desemnată printr-un un mic romb amplasat alături de clasa agregat:



Clase agregat.

Un document are mai multe paragrafe şi fiecare paragraf este alcătuit din mai multe fraze.



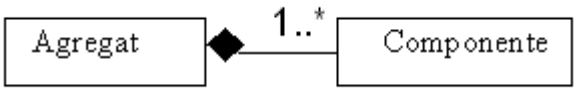
Fiecare maşină are un motor.

- Agragatul nu poate exista fara una dintre componente
- Distrugerea agregatului nu conduce la distrugerea componentelor

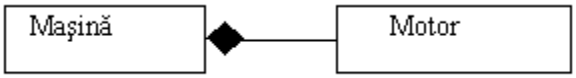
Compunerea

- Compunerea este un caz particular de agregare. Exprima o agregare prin continere fizica.

va urez restanta usoara zgarcitilor

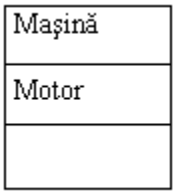


Distrugerea obiectului agregat antreneaza distrugerea componentelor. De exemplu, in agregarea masina- motor definita mai sus, motorul unei masini poate fi inlocuit cu altul. Motorul unei masini poate fi pus in alta masina! Daca insa reprezentam relatia masina – motor ca relatie de compunere-> fiecare masina are un motor unic, care nu poate fi inlocuit si motorul nu poate fi pus in alta masina!



- Relaţia de compunere este semantic echivalentă cu considerarea componenţilor ca atribute ale clasei agregat:

Astfel, relatia de mai sus este echivalenta cu:

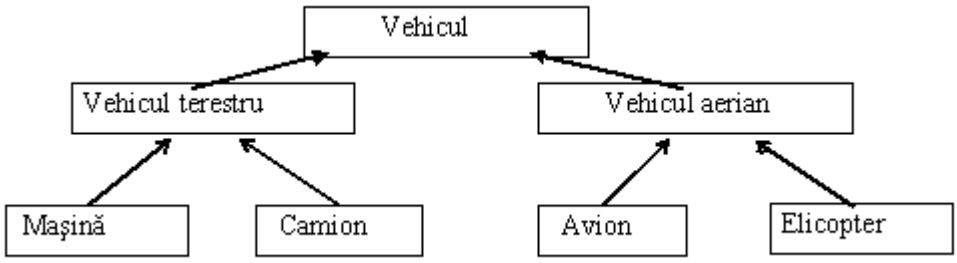


Generalizarea

Ierarhiile de clase sunt bazate pe noţiunile de clasificare, generalizare şi specializare.

- Generalizarea constă în factorizarea elementelor comune (atribute, operaţii şi constrângeri) ale unui ansamblu de clase într-o clasă mai generală, numită superclasă. Clasele sunt ordonate într-o ierarhie. Săgeata care simbolizează generalizarea între două clase punctează către clasa mai generală:

va urez restanta usoara zgarcitilor



In ierarhia din figura de mai sus, fiecare clasă (exceptând clasa rădăcină) are o singură super-clasă. Reprezentarea grafică corespunde unui arbore, numit şi arborele de moştenire. Clasele pot avea mai multe super-clase. In acest caz, generalizarea este numită multiplă iar ierarhia de clase se reprezintă printr-un graf , numit şi graful de moştenire.

- Specializarea permite capturarea particularităţilor unui ansamblu de obiecte, nereprezentate prin clasele existente. Noile caracteristici sunt definite într-o clasă nouă, sub-clasă a uneia sau mai multor clase existente. Specializarea este o tehnică foarte eficientă de extensie coerentă a unui ansamblu de clase existente. Noile cerinţe sunt încapsulate în sub-clase care extind funcţiile existente. De exemplu, dacă într-o aplicaţie apare necesar să se reprezinte "bicicleta" ca vehicul de transport, în plus faţă de cele reprezentate în ierarhia de mai sus, atunci se va defini o clasă nouă, sub-clasă a clasei "Vehicul terestru", în care vor fi definite particularităţile bicicletei ca vehicul terestru. Fiecare sub-clasă a unei ierarhii moşteneşte atributele şi operaţiile definite în clasele aflate pe calea de la clasa radacină la subclasa respectiva, fiind cu atât mai specializată cu cât se află mai departe de radacină.

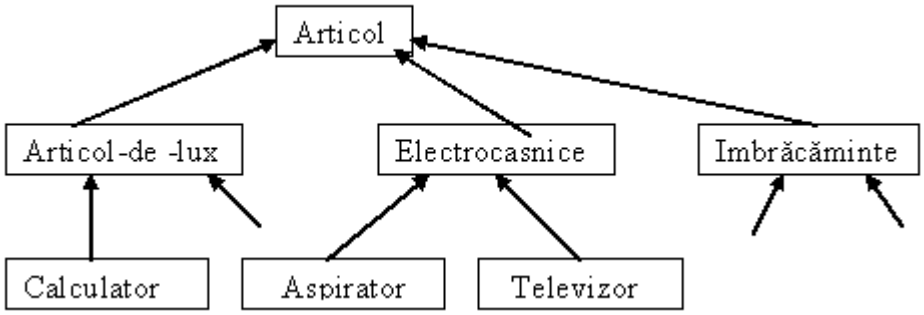
- **Moştenirea** este o tehnică oferită de limbajele de programare pentru a construi o clasă plecând de la una sau mai multe clase, partajând atribute, operaţii şi uneori constrangeri în interiorul unei ierarhii de clase. Clasele copil moştenesc caracteristicile claselor părinte. Atributele şi operaţiile

va urez restanta usoara zgarcitilor

declarate în clasele părinte sunt accesibile în clasele copil, ca şi cum ar fi declarate local. Moştenirea este o modalitate de a realiza clasificarea.

- Relaţia de generalizare definită în UML este mai abstractă decât relaţia de moştenire care există în limbajele de programare obiect, cum ar fi C++. Ea este mai adecvată etapei de analiză (exista si intre cazuri de utilizare!). Decizia asupra modalităţii de a realiza generalizarea se ia mai târziu, în etapa de proiectare.

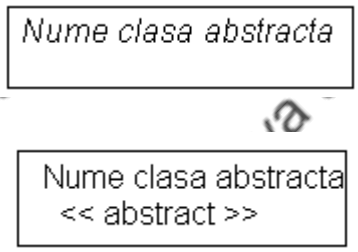
Prin clasificare şi generalizare, universul problemei este divizat in parţi independente care grupează obiectele prin afinitate. Modificarea unei părţi antrenează un minimum de modificări ale celorlalte, fapt pus în evidenţă de arborele de moştenire: fiecare subarbore grupează obiectele care împart caracteristicile rădăcinii sale. De exemplu, adăugarea de noi caracteristici la clasa Articol-de-lux (figura)nu afectează clasa îmbrăcăminte şi nici subclasele acesteia, dar extinde automat caracteristicile subclaselor clasei Articol-de-lux.



Uneori, anumite clase sunt create doar ca surse de moştenire pentru alte clase; ele sunt clase abstracte. De exemplu, clasa Articol este o clasa abstractă daca nu caracterizează complet nici un obiect din universul problemei. Clasa Articole-electrice a fost introdusă pentru a factoriza proprietăţile electrice (de exemplu, tensiunea de alimentare, consumul şi altele), comune calculatorului şi articolelor electrocasnice.

va urez restanta usoara zgarcitilor

Notatie:



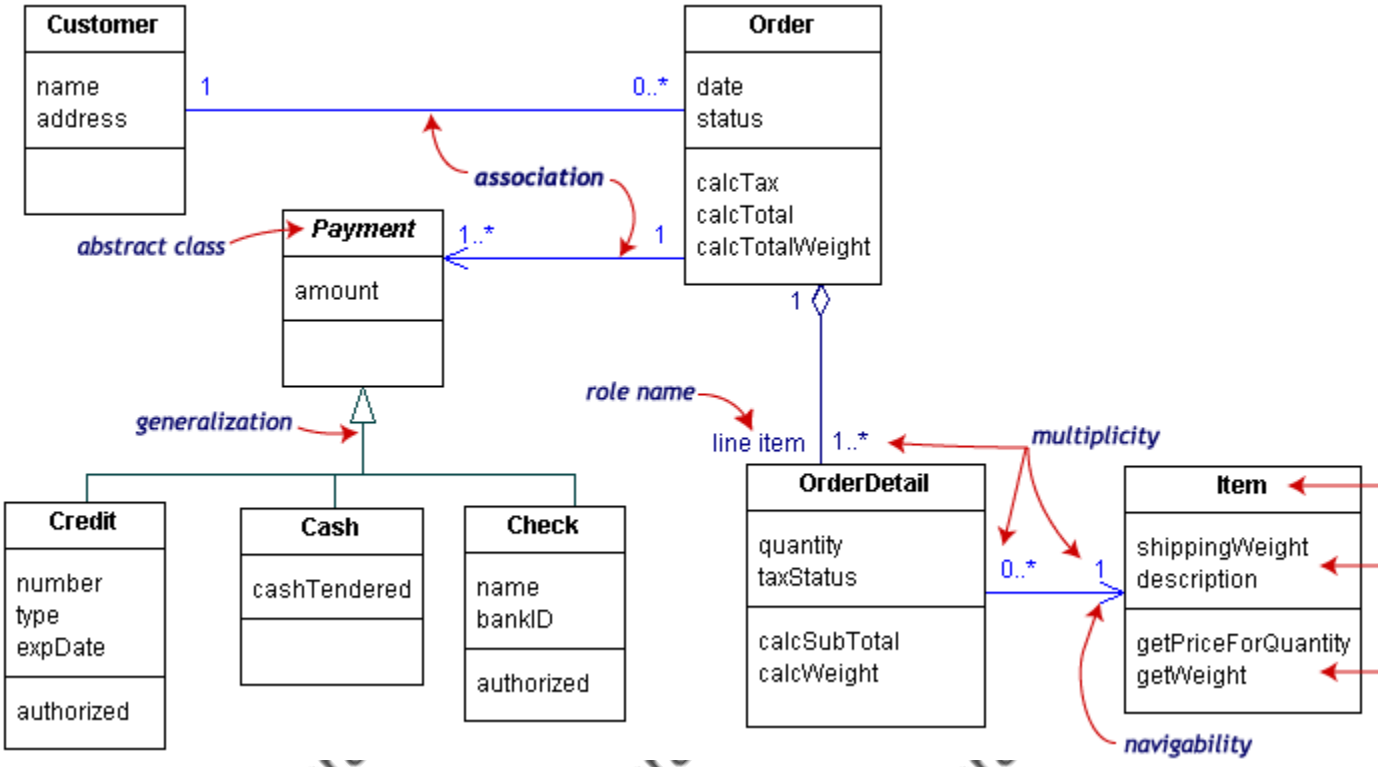
Numele operatiilor abstracte (fara implementare) se scriu tot in italic.

Navigabilitate

Navigabilitatea desemneaza necesitatea ca un obiect al unei clase sa acceseze un alt obiect “navigand” de-a lungul unei legaturi. Navigabilitatea se reprezinta printr-o sageata la capatul “navigabil” al asocierii. Obiectul de la capatul navigabil este accesibil unui obiect de la cealalata extremitate. Invers, nu.

Urmatoarea diagrama de clase modeleaza o aplicatie care permite plati pe baza de ordine ale clientilor.

va urez restanta usoara zgarcitilor



Sageata asociata asocierii se numeste navigabilitate. Ea indica directia in care trebuie sa fie traversata sau interogata asocierea. De exemplu, un obiect OrderDetail poate interoga obiectul Item asociat. Un obiect Item poate fi asociat mai multor obiecte OrderDetail dar pentru el nu este necesar sa le cunoasca. Un obiect Order este asociat cu mai multe obecte Payment, pe care trebuie sa le cunoasca. Pentru un obiect Payment nu este necesar sa cunoasca obiectul Order asociat.

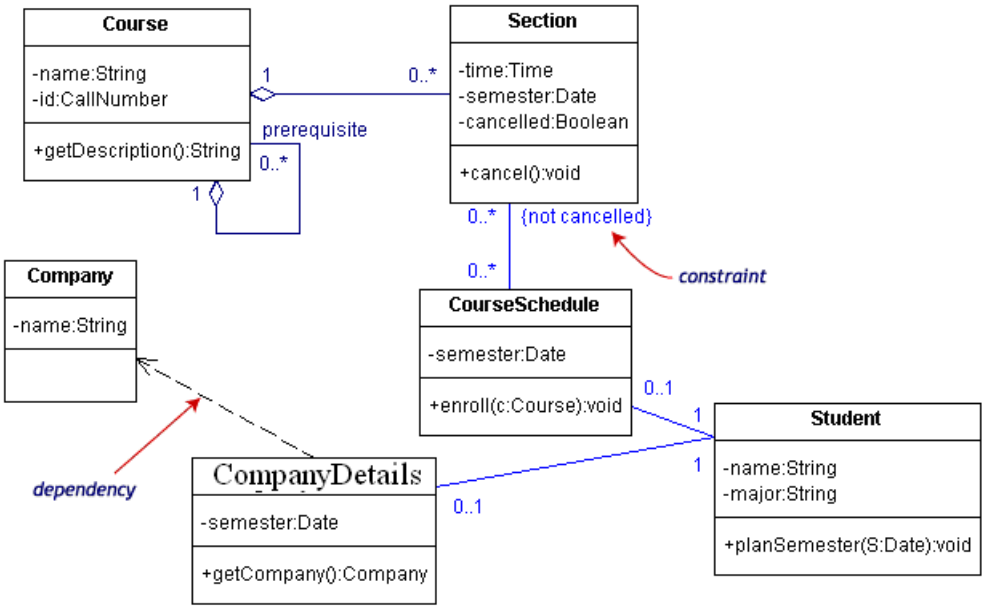
Asocierile fara navigabilitate sunt considerate bi-directionale. Navigabilitatea este un element optional. Se adauga pentru a imbunatati claritatea diagramei.

Dependente si constrangeri

O dependenta este o relatie intre 2 clase in care modificarea uneia poate forta modificarea celeilalte. Exemplu: CompanyDetails depinde de Company. Daca se modifica Company trebuie sa se modifice si CompanyDetails.

va urez restanta usoara zgarcitilor

O constrangere este o conditie pe care orice implementare a diagramei trebuie sa o satisfaca. Este scrisa intre paranteze {}. Exemplu: Un obiect Section poate fi parte dintr-un obiect CourseSchedule numai daca nu a fost anulat.



Folosirea diagramelor de clase:

1) In modelarea conceptuala (analiza oriectata obiect)

- Clasele corespund conceptelor / obiectelor (entitatilor) din domeniul aplicatiei
- Nu exista neaparat o legatura directa cu clasele de obiecte utilizate in implementare si deci diagrama de clase nu face parte din modelul structural al sistemului

- De regula, nu sunt definite operatiile din clase prin tipurile parametrilor si nici tipul atributelor.

va urez restanta usoara zgarcitilor

- Diagrama de clase poate fi folosita in modelarea conceptuala a unei baze de date. In modelul fizic al BD clasele se implementeaza prin tabele ale bazei de date.

2) Pentru specificarea software

- Se pune accent pe interfata si nu pe implementare
- Adesea se foloseste cuvantul “tip” in legatura cu interfata unei clase: un tip poate fi implementat de mai multe clase si o clasa poate implementa mai multe tipuri

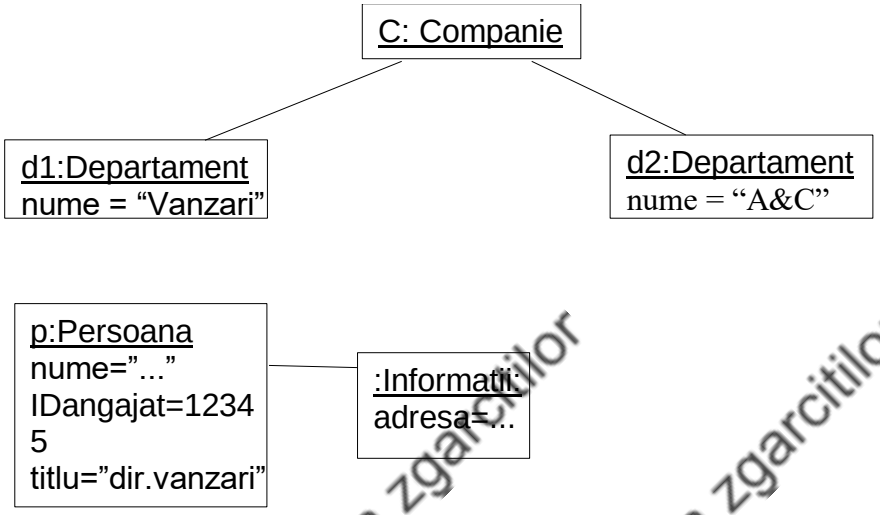
3) In proiectarea de detaliu si implementare

- Diagramele contin clase de obiecte intr-un anumit limbaj de programare
- Diagramele fac parte din modelul structural al sistemului

va urez restanta usoara zgarcitilor

Diagrame de obiecte

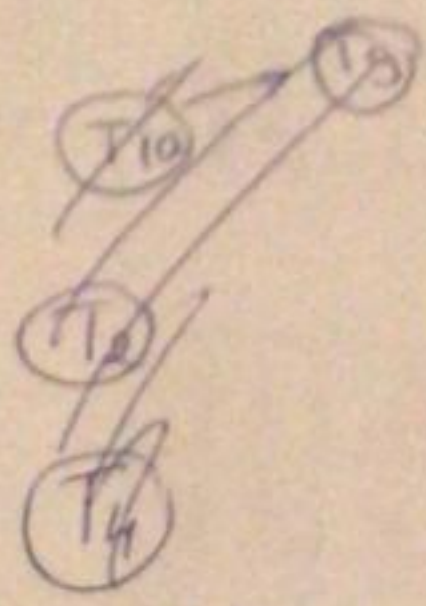
O diagrama de obiecte reda un set de obiecte si legaturile dintre ele la un moment dat.

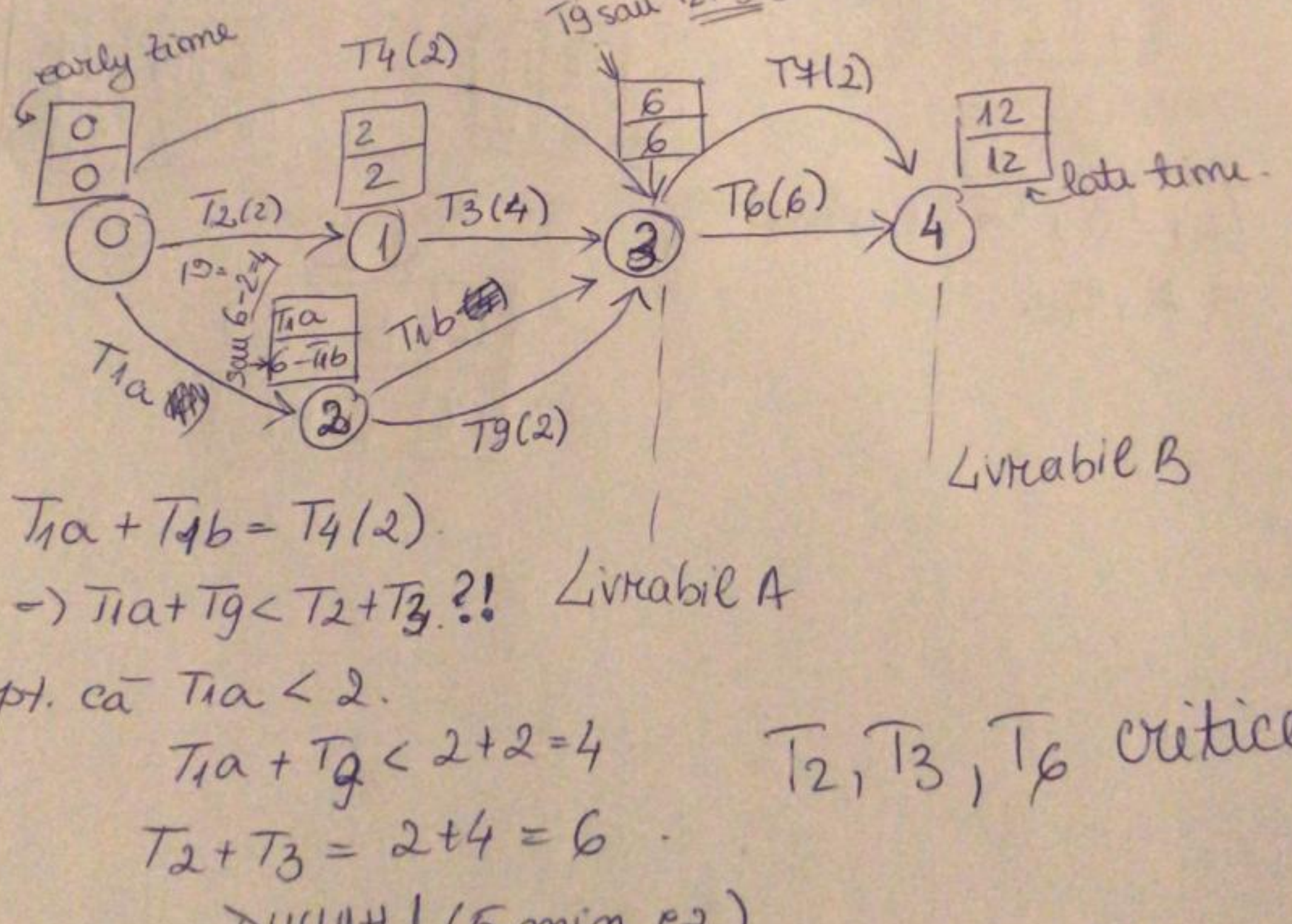


- O diagrama de obiecte este o instanta a unei diagrame de clase sau partea statica a unei diagrame de interactiune.
- O diagrama de interactiune adauga la o diagrama de obiecte mesajele care sunt schimbate intre obiecte.

va urez restanta usoara zgarcitilor

Task	Durată	Depend
T_1	4 z	-
T_2	2 z	-
T_3	4 z	2
T_4	2 z	-
LivrabilA	-	1, 4, 3
Task6	6 z	5 / LA
T7	2 z	5 / LA
LB	-	6, 7
T9	2 z	1 SS





$T_{1a} + T_{1b} = T_4(2)$.

$\Rightarrow T_{1a} + T_9 < T_2 + T_3$. ?!

pt. că $T_{1a} < 2$.

$T_{1a} + T_9 < 2 + 2 = 4$

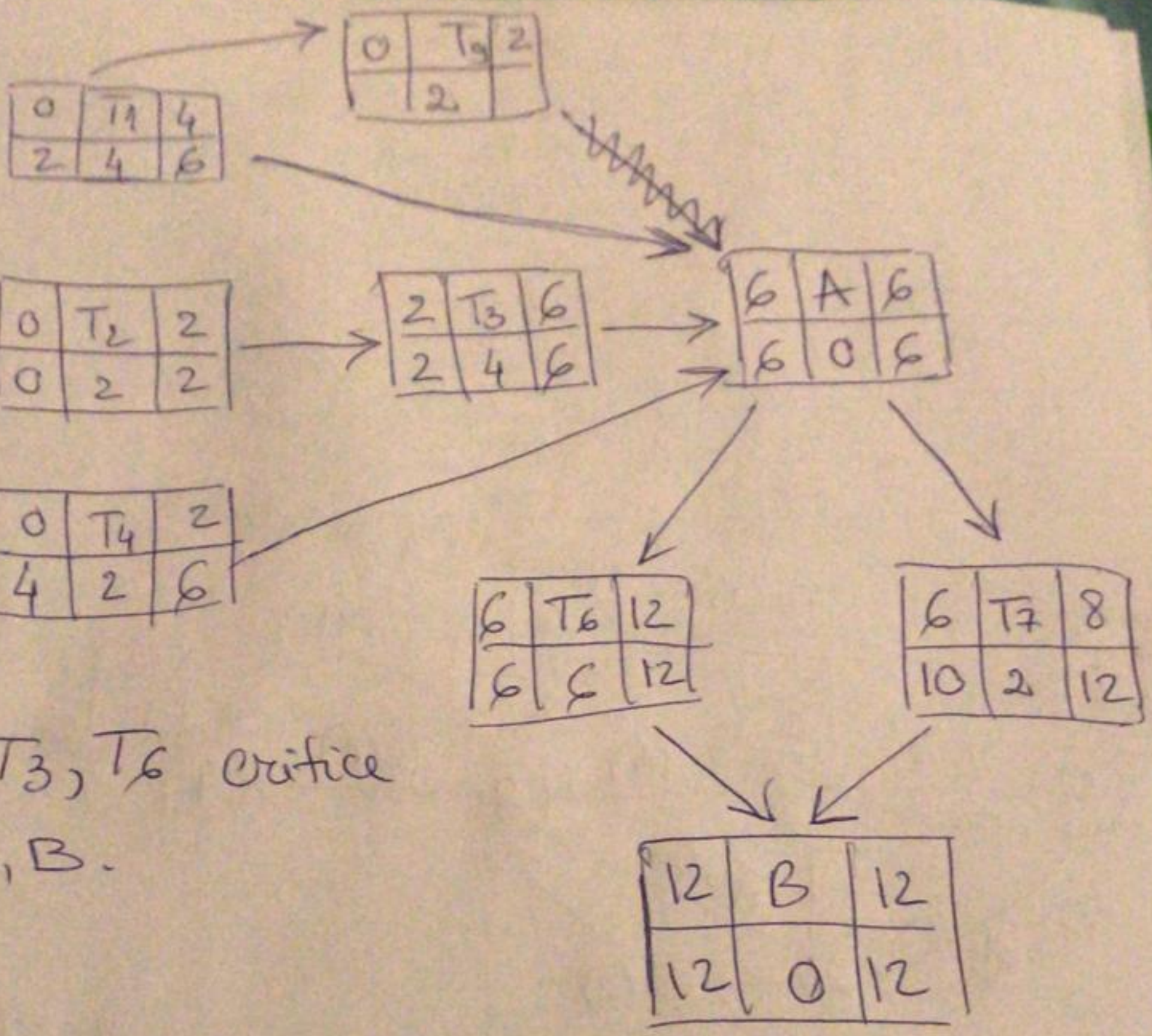
$T_2 + T_3 = 2 + 4 = 6$.

T_2, T_3, T_6 critice.

DUUUH.! (5 min. e2)

float = late time (end of arrow) − duration − early time (beg. of arrow)

8.



T_2, T_3, T_6 critice

+ A, B.

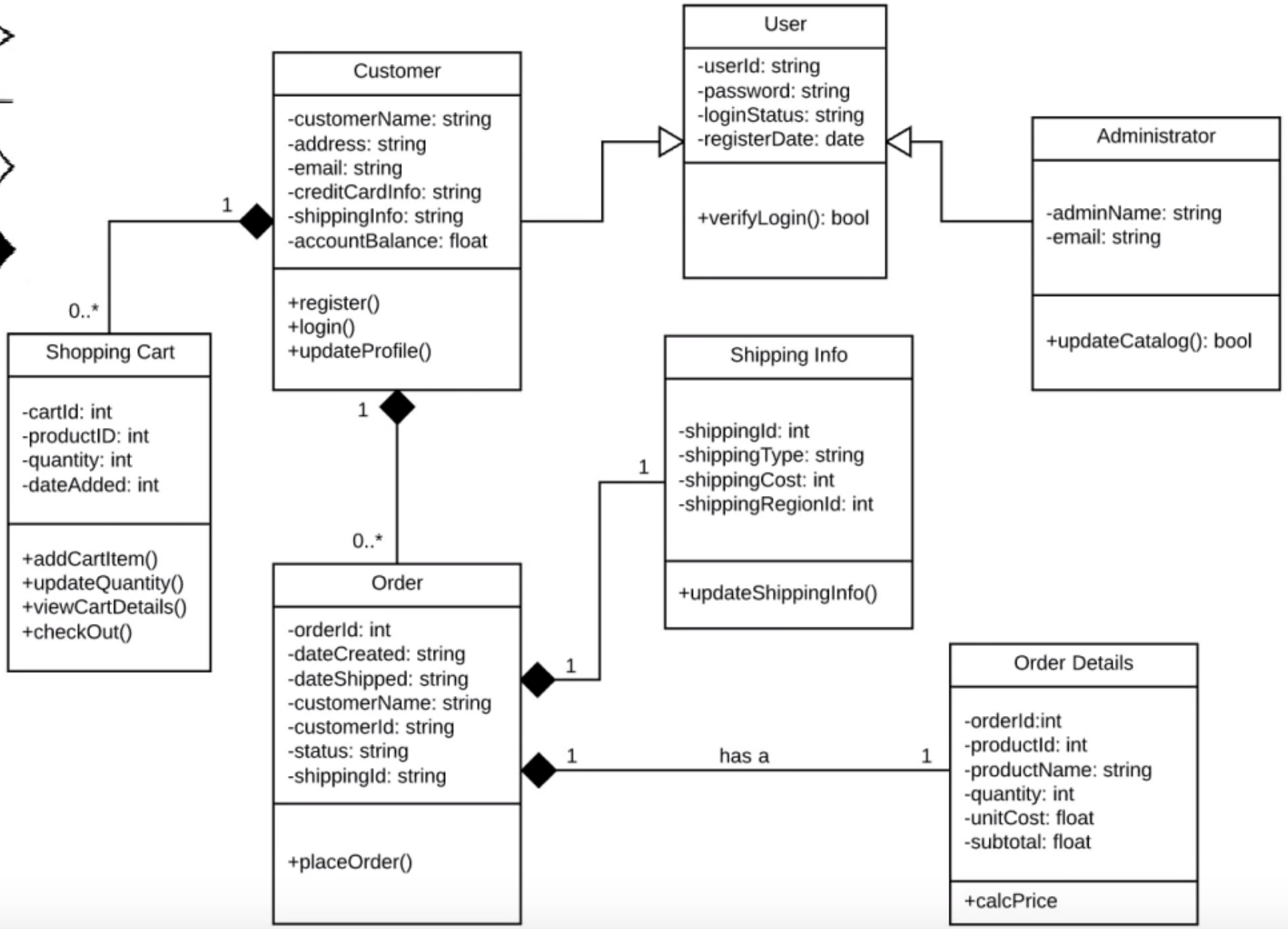
Relationships

Inheritance

Association

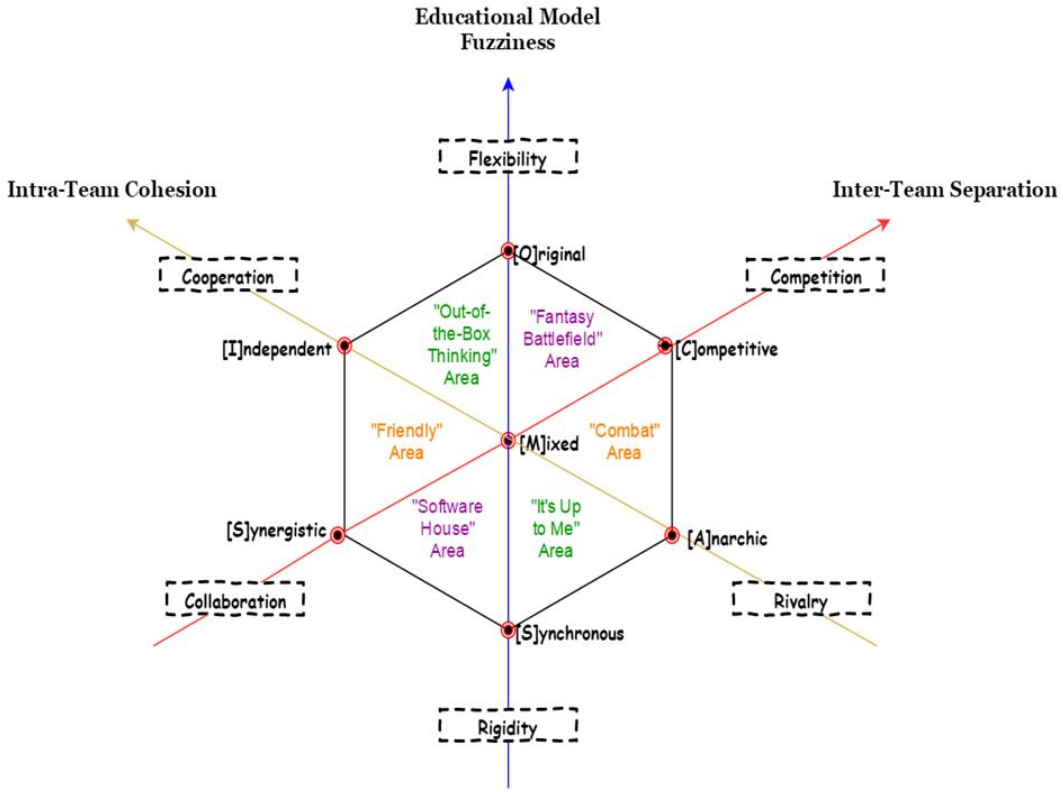
Aggregation

Composition



Cursul 0
Introducere

Mosaic



Abordarea Independenta (“Independent”)

- Solicitare Proiect: Scopul si continutul documentului; Descrierea sumară a produsului software; Tehnologii şi tooluri de dezvoltare; Estimări resurse necesare; Prezentare calendar livrări; Motivare echipă. etc.

Abordarea Concurentiala (“Competitive”)

- Realizarea unui sistem de simulare a competiţiilor între maşini de Formula 1.

Abordarea Colaborativa (“Synergistic”)

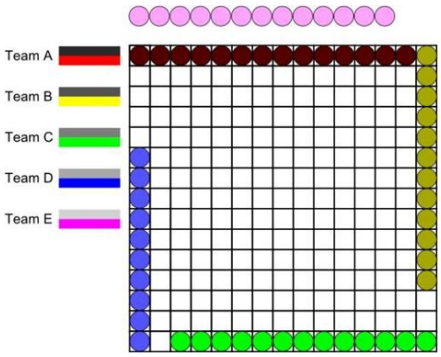
- Sistem Automat de Analiză şi Extragere a Informației din Documente

Abordarea Mixta (“Mixed”)

- Construirea unui sistem de prelucrare de imagini bazat pe o strategie de votare: fiecare echipa era impartita in patru celule: 3 celule care ofereau trei algoritmi diferiti de conversie si o celula care combina cele trei rezultate intr-unul mai bun decat oricare din cele trei rezultate initiale.

Abordarea Anarhica (“Anarchy”)

- Scopul jocului este ca un pion sa ajunga pe partea opusa a tablei, moment in care echipa din care face parte pionul (cea la care a fost mutat) castiga un punct (creste nota echipei respective). Toti pionii care fac parte din echipa castigatoare mai castiga un punct si la nota individuala iar jocul se reia, cu jucatorii realocati in echipe diferite.



Abordarea Artistica (“Originality”)

- Scopul proiectului este de a obtine o imagine cu o valoare “artistica” de un cat mai ridicat nivel de exprimare utilizand doar generare procedurala, geometrie fractalica, si algoritmica simpla.

Abordarea Sincrona (“Synchronous”)

- Primul proiect este in saptamanile 3-6, iar al doilea in saptamanile 7-10

Cursul 1
Introducere

Cuprins:

1. Dimensiunea unui proiect software
2. Planificarea proiectului
3. Execuţia proiectului
4. Închiderea proiectului
5. Procesul de dezvoltare
6. Particularităţile proiectelor software

1. Dimensiunea unui proiect software

Un proiect software are două dimensiuni principale:

- Ingineria proiectului (se ocupă cu dezvoltarea efectivă a proiectului si se concentrează pe aspecte precum design, cod, testare)
- Managementul proiectului -> planificarea şi controlul activităţilor de inginerie în scopul atingerii obiectivelor proiectului (costuri, timpi de execuţie, calitate)

Proiecte mici:

- Echipe formate dintr-un număr redus de persoane, durată de câteva săptămâni

- Metode informale de management şi dezvoltare: email-uri, câteva termene limită, comunicare verbală

Proiecte mari:

- Echipe mari; durată câteva luni
- Taskuri efectuate cu atenţie, planificate şi urmărite pas cu pas; metode bine cunoscute
- Fiecare produs intermediar este documentat riguros şi verificat

Procesul de management al proiectului are trei etape principale: planificarea, executia, inchiderea proiectului.

2. Planificarea proiectului

- Activităţi administrative şi de pornire
- Planificarea şi orarul proiectului
    - Definirea obiectivelor proiectului
    - Estimarea costurilor şi a efortului
    - Definirea unui plan de masurare a proiectului
    - Identificarea riscurilor şi a modului de evitare/recuperare
- Obţinerea acordului de la managementul superior
- Definirea şi revizuirea planului de management al configuraţiilor
- Realizarea unei echipe şi stabilirea responsabilităţilor fiecăruia

3. Execuţia proiectului

- Execuția proiectului după planul propus
- Monitorizarea conformităţii cu procesele definite
- Analiza defectelor şi efectuarea de activităţi de prevenire a acestora
- Monitorizarea performaţelor la nivel de program si a progresului proiectului
- Efectuarea de review-uri la anumite etape critice şi replanificarea unor etape dacă este necesar

4. Închiderea proiectului

- Etapa are loc dupa ce clientul şi-a dat acceptul pentru produsul final
- Se urmăreşte stabilirea unor concluzii ca urmare a experienţei acumulate, pentru a îmbunătăți procesele folosite în viitor

5. Procesul de dezvoltare

Principiile fundamentale în MPS:

- Procesul de dezvoltare bazat pe arhitectură
    - Componentele arhitecturale - înţelese foarte bine înainte de a lua în considerare amănuntele de detaliu
    - Gradul de refacere/abandon a unor componente – ar trebui să scadă sau să rămână constant în timpul desfăşurării unui proiect
- Modul de dezvoltare iterativ
    - Framework de planificare cât mai dinamic
    - Rezolvarea problemelor critice foarte devreme => Dezvoltare mai predictibilă & mai puține surprize => Management al riscului mult mai bun

- Principalele riscuri confruntate cat mai devreme.
    - La fel ca la modul de dezvoltare iterativ???
- Dezvoltarea bazată pe componente
    - Complexitatea dezvoltării de software ~ numărul de elemente generate de către membrii echipei
    - Diminuarea numărului acestora si a complexităţii procesului de management
- Plan de management al schimbărilor
    - Dinamica dezvoltării iterative => fluxurile de lucru concurente ale diferitelor echipe de dezvoltare care folosesc aceleaşi componente
    - Necesită linii de referinţă controlate foarte riguros
- Model de evaluare bazat pe demonstraţii
    - Integrarea apare foarte devreme în viaţa unui proiect şi se continuă pe parcursul întregului proces de dezvoltare.
    - Rezultatele intermediare sunt elemente esenţiale, deoarece sunt tangibile şi obiective
- Evaluare obiectivă a calităţii şi corectă a progresului
    - Indicatorii de progres şi calitate derivă direct din componentele dezvoltate şi conferă informaţii importate în legatura cu trendul proiectului şi gradul de corelare al produsului cu cerinţele iniţiale
- Notaţii bazate pe modele
    - Utilizarea unor notaţii inginereşti în faza de design va conduce la un control mai bun al complexităţii, evaluări intermediare mai obiective şi mai corecte, precum şi analize ce pot fi automatizate
- Procesul de dezvoltare configurabil şi scalabil economic
    - Experienta, metodele, uneltele si tehnicile trebuie folosite împreună pentru a lărgi segmentul de piaţă ţintă => o întoarcere a investiţiei mult mai mare
- Versiunile intermediare având nivele de detaliu din ce în ce mai mari

6. Particularităţile proiectelor software

- Invizibilitate - spre deosebire de un pod sau un drum care sunt construite şi progresul este vizibil imediat, în cazul unui produs software progresul nu este evident foarte repede
- Complexitate - Produsele software sunt unele dintre produsele cu cea mai mare complexitate per euro/dolar/lei investiţi
- Flexibilitate - Uşurinţa cu care un produs software poate fi modificat este unul dintre cele mai importante atu-uri ale acestui tip de proiecte

Cursul 2
Vedere de ansamblu asupra managementului de proiect

Cuprins:

1. Definiţii
2. PMI – Organizarea Proiectului
3. Procese de Management al Proiectelor
4. Noţiuni de etică profesională

1. Definiţii

- Operaţiune – activitate continuă şi repetitivă
- Program – grup de proiecte gestionate într-un mod coordonat
- Faza a unui proiect – o colectie de activitati interconectate din punct de vedere logic, care de obicei conduc la realizarea unui produs intermediar/final important
- Tehnica Delphi – tehnica de previziune folosită pentru a aduna informaţii despre evenimentele viitoare din viaţa proiectului; se bazează pe părerile unor experţi
- Managementul prin obiective – sistem de conducere managerială care defineşte responsabilităţile unui manager pe baza obiectivelor urmărite de către organizaţie
- Ciclul de viaţă al unui proiect – o serie de faze ale unui proiect, care de obicei urmează una după alta, alea căror nume şi număr depinde de cerinţele tuturor organizaţiilor implicate în proiect

2. PMI – Organizarea Proiectului

PMI – Project Management Institute

Structuri organizatorice (din punctul de vedere al nivelului de autoritate al managerului de proiect):

- Funcţională:
    - Organizarea se bazează pe diferite arii de expertiză (marketing, producţie etc.)
    - Fiecare angajat are un superior bine determinat
    - Managerul de proiect are putere scăzută; responsabilitatea revine managerilor funcţionali
- Proiectizată (“Projectized”)
    - Întreaga organizare se bazează pe proiecte
    - Managerul de proiect deţine controlul total asupra proiectelor
    - Fiecare angajat este repartizat la un proiect şi răspunde direct managerului de proiect
- Matriceală
    - Slabă
        - Deciziile sunt luate în principal de managerul funcţional
        - Un manager de proiect poate îndeplini unul din următoarele două roluri:
            - Mesager: coordoneaza comunicarea din echipa; nu poate lua decizii
            - Coordonator: are puterea de a lua anumite decizii;
    - Puternică: deciziile sunt luate în principal de managerul de proiect
    - Echilibrată

3. Procese de Management al Proiectelor

Procesele se impart in:

- Procese de management - asigură execuţia eficientă a proiectului
- Procese orientate pe produs - specifică şi determină crearea produsului final

Grupuri de procese:

- Procese de iniţializare
    - Reprezinta faza în care se obţin autorizaţiile necesare începerii unui nou proiect sau a unei noi faze în cadrul unui proiect
- Procese de planificare
    - Grupul de procese ce ţin de formularea obiectivelor proiectului
    - În această etapă se creează planul de proiect – se stabileşte în detaliu modul în care vor fi atinse obiectivele propuse
- Procese de execuţie
    - Managerul de proiect are responsabilitatea de a coordona personalul şi resursele disponibile astfel încât să respecte planul de proiect stabilit
- Procese de monitorizare şi control
    - Grup de procese responsabile cu măsurarea şi analiza performanţelor proiectului
- Procese de încheiere
    - Grupul de procese necesare pentru a termina formal un proiect
    - Produsul este livrat şi acceptat de către beneficiar, iar proiectul se încheie

4. Noţiuni de etică profesională

Responsabilităţile faţă de profesie

- Să fie sincer tot timpul, indiferent de situaţie
- Să nu ascundă eventuale conflictele de interese
- Să respecte legislatia în vigoare
- Să respecte drepturile intelectuale ale altora

Responsabilităţile faţă de clienţi şi faţă de public

- Să îşi păstreze integritatea din punct de vedere profesional
- Să respecte caracterul confidenţial al datelor personale
- Să evite primirea unor cadouri/compensaţii în situaţiile în care acest lucru este inadecvat
- Să se asigure că niciun conflict de interese nu mijloceşte interesele clientului sau judecata profesională

Cursul 3
Faza de planificare

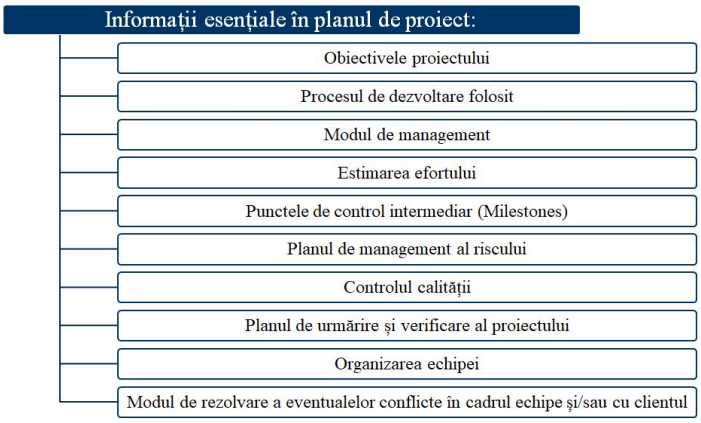
Cuprins:

1. Planul de Proiect
2. Planificarea pas cu pas
3. Metode de dezvoltare
4. Prototipuri
5. Metode ale generaţiei a 4-a
6. WBS
7. RBS

1. Planul de Proiect

Structura:

- Sumarul proiectului
- Secţiunea de planificare
    - Modul de execuţie al diferitelor proceduri de planificare
    - Modul de dezvoltare ce va fi folosit, estimarea timpilor de execuţie etc.
- Secţiunea de urmărire (tracking)
    - Măsurătorile ce vor fi făcute în timpul proiectului
- Secţiunea destinata echipei
    - Structura şi membrii echipei, rolurile lor



2. Planificarea pas cu pas

Pasul 0 – Alegerea proiectului

- În aceasta etapă au loc activităţi ce duc la luarea unei decizii în legatura cu proiectele ce vor fi începute – această decizie poate fi luată individual sau poate să facă parte dintr-o strategie pe termen lung a companiei

Pasul 1 – Identificarea domeniului si a obiectivelor proiectului

- Modificarea obiectivelor în lumina analizei asupra persoanelor interesate în proiect
- Stabilirea metodelor de comunicare cu toate părţile interesate

Pasul 2 – Identificarea infrastructurii

- Identificarea modului în care va fi organizată echipa de dezvoltare
- Deciziile strategice sunt de obicei documentate fie într-un plan de strategie business, fie într-un plan tehnologic dezvoltat pe baza planului business

Pasul 3 – Analiza caracteristicilor

- Identificarea celor mai importante riscuri

- Analiza modului de implementare, selectarea ciclului de viaţă folosit pentru dezvoltare
- Revizuirea estimărilor asupra resurselor

Pasul 4 – Identificarea produselor şi a activităţilor

- Documentarea eventualelor probleme ale produsului
- Modificarea reţelei de activităţi, luând în considerare nevoia pentru etape intermediare şi puncte de verificare

Pasul 5 – Estimări ale efortului pentru fiecare activitate

- Efectuarea de estimari folosind o abordare de jos in sus
- Estimări de personal, timp, resurse

Pasul 6 – Analiza riscurilor

- Identificarea şi cuantificarea riscurilor datorate activităţilor
- Ajustarea planurilor şi a estimărilor astfel încât să ia în considerare riscurile identificare anterior

Pasul 7 – Alocarea resurselor

- Identificarea şi alocarea resurselor
- Revizuirea planurilor şi a estimarilor, astfel încat să ia în considerare constrângerile datorate resurselor

Pasul 8 – Revizuirea/Publicarea planului

Pasul 9 – Execuţia planului

Pasul 10 – Nivele inferioare de planificare

3. Metode de dezvoltare

Metode structurate (inclusiv metodele OO)

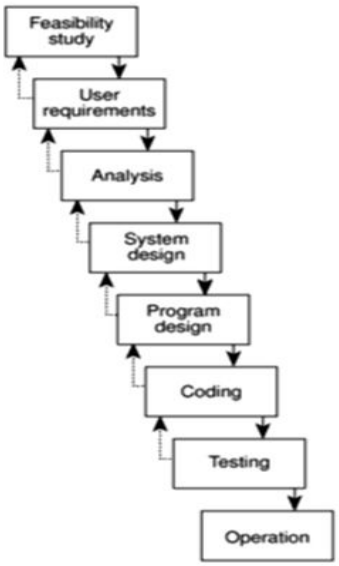
- O mulţime de paşi şi reguli care genereaza prodiagramele de flux, de date, etc.
- De cele mai multe ori sunt mult mai consumatoare de timp decat metodele intuitive, acest lucru ducând şi la o creştere a costurilor proiectului
- Avantaje: sistemul este mult mai puţin sensibil la erori si mult mai uşor de întreţinut la sfârşit
- Recomandate în cazul proiectelor mari, care implică mulţi dezvoltatori şi mulţi utilizatori

Metode de dezvoltare rapidă

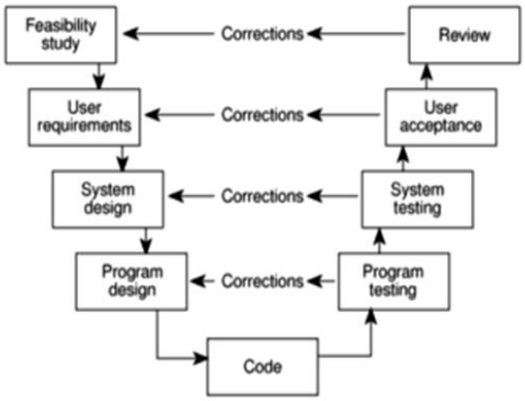
- Workshop-uri de trei-cinci zile în care dezvoltatorii lucreaza intensiv împreună cu clienţii pentru a identifica şi pentru a cădea de acord asupra cerinţelor business ale proiectului
- Time-box – întinderea fiecarei etape a proiectului este constrânsă de un deadline predeterminat, foarte scurt şi inflexibil
- Cerinţele ce nu pot fi satisfacute într-un anumit time-box, sunt mutate în etapele urmatoare

Modelul în cascadă

- Considerat metoda “clasica” de dezvoltare a sistemelor
- Permite controlul eficient al proiectelor şi estimarea foarte exactă a timpilor de execuţie

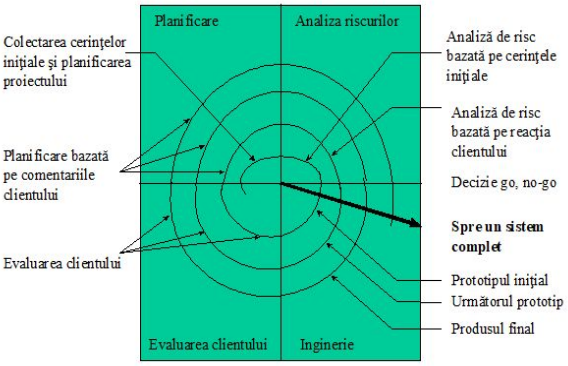


Modelul procesului în V



- Extinde activităţile de testare din modelul în cascadă
- Fiecare pas are un proces de validare corespunzator.
- În cazul în care apar defecte, procesul de validare întoarce dezvoltarea la pasul de dezvoltare corespunzător; toţi paşii următori trebuiesc apoi refăcuţi.
- Ideal, acest tip de feed-back ar trebui să apară numai în cazul unei discrepanţe mari între specificaţiile unei anumite activitaţi şi ceea ce a fost de fapt implementat

Modelul în spirală



- Poate fi considerat ca o alta vedere a modelului în cascadă
- Un mai mare grad de detaliu este necesar la fiecare etapă a proiectului, acest fapt justificând şi un mai mare grad de încredere în probabilitate de succes a proiectului
- Acest model poate fi văzut ca o spirala în care sistemul dezvoltat este văzut din ce în ce mai în detaliu la fiecare rotaţie
- Un proces de evaluare a etapei precedente are loc înaintea începerii unei noi iteraţii
- Dezvoltare iterativă
    - Bazată pe ideea de ciclu de producţie
    - Procesul de dezvoltare cuprinde mai multe cicluri de producţie
- Dezvoltare incrementală
    - Fiecare ciclu are o complexitate (un nivel de detaliere) mai mare decât precedentul
- Activităţile unui ciclu de producţie
    - (1) Planificare
        - Stabilirea obiectivelor, alternativelor de rezolvare şi a restricţiilor pentru ciclul curent
    - (2) Analiza riscurilor
        - Analizează alternativele de rezolvare şi restricţiile din (1)
        - Identifică factorii de risc
        - Decizia GO/NO GO (continuă/renunţă)
    - (3) Inginerie - începutul unui ciclu nou
        - Dezvoltarea produsului pe următorul nivel de detaliere
        - Se pot folosi
            - Modelul clasic
            - Prototipizarea - pentru clarificarea unor cerinţe
    - (4) Evaluarea clientului

- Avantaje
    - Abordare evoluționistă
    - Ajută la înțelegerea riscurilor şi la identificarea modalităților de ținere sub control a acestora
    - Prototipizarea este folosită ca mecanism de reducere a riscurilor
    - Ciclul clasic de viață este încorporat într-un cadru iterativ, care reflectă mai bine lumea reală
- Dezavantaje
    - Analiza riscurilor este o activitatea critică
    - Atenție acordată riscurilor tehnice în toate etapele proiectului

4. Prototipuri

Tipuri de prototipuri:

- Throw-away
    - Folosit doar pentru a testa unele idei; se renunță la el în momentul în care începe dezvoltarea sistemului operaţional
- Evoluţionar
    - Este dezvoltat şi modificat în continuu până în momentul în care poate deveni un sistem operaţional
- Incremental
    - Sistemul operaţional este dezvoltat şi implementat în etape mici; feed-back-ul de la etapele anterioare este folosit şi influenţează dezvoltarea etapelor următoare

Activități:

- (1) Colectarea cerințelor
    - Dezvoltatorul şi utilizatorul stabilesc obiectivele generale, cerințele cunoscute, domeniile în care cerințele vor fi definite ulterior
- (2) Producerea rapidă a unui proiect
    - Se reprezintă acele elemente care sunt percepute de utilizator
        - Formatul datelor de intrare
        - Formatul rezultatelor
- (3) Construirea prototipului
- (4) Reevaluarea prototipului de către utilizator
- (5) Rafinarea prototipului
- (6) Realizarea produsului final

Activitățile (3) - (5) se repetă până când sunt satisfăcute toate cerințele clientului

Avantajele utilizării prototipurilor:

- Comunicarea este îmbunătăţită:
- Când nu există un sistem care poate fi imitat, clienții pot testa diferite prototipuri pentru a îşi da seama care dintre ele le este cel mai util
- Necesarul de documentaţie este redus datorită faptului că prototipul poate fi examinat în practică
- Costurile de întreținere sunt reduse; dacă clientul nu cere multe schimbări ale prototipului, este foarte probabil ca acesta să nu ceară nici multe schimbări ale produsului final
- Clienții pot fi mult mai implicaţi în deciziile legate de design-ul final al sistemului

5. Metode ale generaţiei a 4-a

4GL - Fourth Generation Languages

Instrumente CASE Computer-Aided Software Engineering

- Specificarea cerințelor se face folosind limbaje de specificare apropiate de limbajul natural SAU folosind notații matematice (algebrice)
- Sprijin pentru modelare, inclusiv teste de consistență şi validitate
- Traducerea automată a specificațiilor în cod sursă (forward engineering), trecându-se prin nivele de
    - Analiză - modele de analiză
    - Proiectare - modele de proiectare

Activități:

- (1) colectarea cerințelor
    - Ideal: clientul descrie cerințele folosind limbajul de specificare al instrumentului CASE
    - În realitate: dialog între client şi specialistul în specificarea cerințelor
- (2) Proiectare
    - Include elaborarea de modele pentru analiză şi proiectare
    - Este nevoie de o reprezentare a modelelor care să permită generarea automată de cod
- (3) Implementare folosind generarea automată de cod sursă
- (4) Testarea

Avantaje

- Productivitate ridicată
- Întreținere uşoară a programelor DACĂ
- Cerințele sunt formulate corect
- Activitatea de proiectare este bine structurată

Dezavantaje

- Curba de învățare a folosirii instrumentelor este lungă
- Codul generat nu este întotdeauna şi eficient
- Costuri de achiziție/întreținere foarte mari
- Probleme de migrare, comunicare cu alte instrumente similare
- Problemă deschisă: întreținerea sistemelor mari

6. WBS – Work Breakdown Structures

- Liste de task-uri detaliate
- O decompoziţie a muncii necesare dezvoltării unui proiect în bucaţi din ce în ce mai mici până la nivelul la care există suportul pentru o urmărire detaliată a progresului la care se afla proiectul
- Fiecare pas elementar din decompoziţie va avea
    - un cost
    - o estimare muncii individuală
- Eforturile aferente şi costurile activităţilor de la nivelele superioare sunt calculate pur şi simplu prin însumarea eforturilor şi costurilor activităţilor din care sunt compuse
- În momentul în care lista de task-uri, împreuna cu estimările aferente, este aprobată, estimările de cost devin bugetul proiectului
- Exemple: pentru proiectele ce durează între 4 şi 12 luni, activităţile trebuiesc descompuse în general până când nivelul cel mai de jos reprezintă aproximativ 1 sau 2 saptămâni de muncă; pentru proiectele mai lungi se poate ajunge şi până la 4 şi chiar 8 săptămâni

7. RBS - Resource Breakdown Structures

- Liste de resurse detaliate
- Sunt similare listelor de task-uri detaliate, dar se referă la organizație, echipa de dezvoltatori şi personalul implicat în realizarea produsului
- Relaţia cea mai importantă ce este documentată prin intermediul RBS este aceea a autorității: cine răspunde în fața cui şi cine spune cui ce să facă
- Nu este importantă poziţia în cadrul organizaţiei a fiecărui individ în parte (se urmăreşte documentarea faptului că angajatul X răspunde în faţa managerului Y, indiferent dacă X este secretară, unul dintre programatori sau un alt manager)

Cursul 4
Managementul bugetului

Cuprins:

1. Estimarea Costurilor
2. Tipuri de estimări
3. Riscuri
4. Clasificarea costurilor
5. Optimizarea costurilor
6. Bugetul proiectului
7. Managemetnul custurilor
8. Subprocese PCM

1. Estimarea Costurilor

Estimarea costurilor unui proiect reprezintă în mare bugetul acelui proiect

2. Tipuri de estimări

Estimarea de bază

- Costurile pe activități şi posturi de execuție
- Precizie de 10%
- Riscurile sunt evaluate => un buget al obiectivelor care va fundamenta decizia de investiție în proiect

Estimarea detaliată

- Precizie de 5%
- Permite evaluarea unei oferte de prestație care se identifică cu oferta clientului
- Este riguroasă, analitică; consumă timp şi bani

Estimări exacte

- Estimare corecta a costurilor <= luate in considerare urmatoarele elemente:
    - Domeniul produsului - cerintele si caracteristicile produsului software
    - Domeniul proiectului - cerintele proiectului, constrangerile, modalitatile de control, etc.
    - Diverse presupuneri - disponibilitatea tuturor resurselor necesare, intarzieri, etc.
    - Contrangeri: de timp, referitoare la resurse, de buget, ale mediului de dezvoltare, etc.

3. Riscuri

- Depăşirea bugetului prevăzut: activități suplimentare
- Stagnarea activităților: probleme în aprovizionar, subcontractări neonorate la timp
- Neîndeplinirea obiectivului propus la nivelul performanțelor angajate
- Repartiția necorespunzătoare a bugetelor alocate
- Necorelarea bugetelor acestora cu termenele sau problemele tehnologice
- Estimarea necorespunzătoare a volumului şi distribuției bugetului necesar|

4. Clasificarea costurilor

Criterii de clasificare:

- Câmpul de aplicație al costurilor
    - Costuri pentru funcția economică : cercetare, aprovizionare, producție, vânzare, etc.
    - Costuri în funcție de zona de exploatare: costul pentru locul de muncă, de uzină etc.
    - Costuri pe activitatea de exploatare : produse, familii de produse
    - Costuri pe centre de responsabilitate: studii de piață, dezvoltare, industrializare
    - Alte costuri (pe departamente, pe clienți, etc.)
- Conținutul costurilor
    - Costul manoperei directe: valoarea dată direct personal
    - Costul manoperei indirecte: valoare prin care se recompensează personalul auxiliar
    - Costul managementului proiectului
    - Costul componentelor necesare pentru realizarea proiectului (de la prototip până la produs final)
    - Cost primar - constituit ca suma tuturor costurilor directe (manoperă şi materiale) utilizate pentru realizarea proiectului
    - Costuri de amortisment al echipamentelor utilizate în proiect
    - Costul de achiziție a echipamentelor necesare proiectului
    - Costuri variabile (proporțional cu volumul producției): salarizarea directă, materiile prime, materialele, energia
    - Costuri indirecte: costuri de regie, sumele prevăzute pentru administrarea proiectului, asigurarea dotărilor, întreținere, iluminat, etc.
    - Costuri fixe (costuri care rămân neschimbate şi se efectuează indiferent de volumul producției realizate): chirii, taxe, asigurări
    - Administrație
    - Costuri de contract - costuri care regrupează o serie de angajamente efectuate în cadrul proiectului de o serie de factori externi.
- Momentul de calcul al costurilor
    - Costul constatat, un cost istoric şi real calculat aposteori
    - Costul prestabilit, calculat apriori, înainte de a demara activitățile, un cost previzionat, care susține decizia.

5. Optimizarea costurilor

Optimizarea costurilor pe relația client/furnizor, comportă patru etape de negociere:

- Compromisul nevoie/funcțiune:
    - utilizatorul îşi exprimă dorințele în termeni de performanțe, termene, cost de achiziție (şi utilizare)
    - furnizorul formulează o propunere cu cel mai bun cost global
- Compromisul funcțiunii de serviciu/concepția produsului:
    - proiectantul caută soluții tehnice conceptuale mai puțin costisitoare pentru performanțele dorite de utilizator
- Compromisul proiectant-executant/furnizori:
    - se obține din partea furnizorilor: subcontractori, materiale, echipamente
    - prețul cel mai competitiv posibil
- Compromisul concepția produsului/realizare:
    - executantul se străduieşte să optimizeze costurile de producție prin planificarea sarcinilor si gestionarea economică a resurselor

6. Bugetul proiectului

Bugetul proiectului reprezintă suma pusă la dispoziția echipei de proiect pentru a realiza conform exigențelor caietului de sarcini respectând calitatea şi termenele impuse

Bugetul inițial conține:

- Sarcinile, costurile şi eventualele rezerve
- Liniile de bugetare
- Modul în care se urmăresc costurile
- Abaterile tehnice permise şi suporturile bugetare suplimentare aferente acestora

7. Managementul costurilor de proiect MCP

MCP va include

- procese adiționale
- tehnici numeroase de management general cum ar fi contul de profit și pierderi, elementele de fezabilitate, rata profitului, rata dobânzii, perioada de amortizare

MCP ar trebui să considere de asemenea și informațiile care vin din partea participanților (stakeholders)

- De exemplu, costul unui articol poate fi măsurat în diferite momente, când este sesizat ca necesar în cadrul proiectului, când este comandat furnizorului, când este livrat, când este exploatat și în funcțiune sau/și cqnd este trecut în evidențele contabile

8. Subprocese MCP

Principalele subprocese ale MCP

- Planificarea resurselor (Resource Planning)
    - Determinind care resurse (oameni,echipament,materiale) si ce cantități în parte ar trebui utilizate pentru a operaționaliza activitățile de proiect
- Estimarea Costului (Cost Estimating)

    - dezvoltarea unei aproximaţii (estimări) a costului resurselor necesare finalizării activităţilor proiectului
- Bugetarea Costului (Cost Budgeting)
    - Alocarea unei estimari de cost generale a activitatilor de munca depusa indiviidual
- Controlul Costului (Cost Control)
    - Controlul tuturor schimbarilor bugetului de proiect

Cursul 5
Managementul dezvoltarii

Cuprins:

1. Fazele ciclului de viata
2. Componente
3. Fundamentele proiectarii
4. Pasii proiectarii
5. Principiile proiectarii
6. Etapele proiectarii

1. Fazele ciclului de viata

- Definitia
    - începe când este formulată problema de rezolvat
    - accentul pe CE face programul (CE informație se prelucrează, CE funcții sau performanțe trebuie să aibă sistemul, CE interfețe cu alte sisteme etc.)
- Dezvoltarea
    - pune accentul pe CUM trebuie realizată aplicaţia
    - se definesc structurile de date si arhitectura programului
    - detalii de implementare a procedurilor şi datelor
    - testare
- Exploatarea
    - Instalare, exploatare, întreținere

2. Componente

- metode
    - informații despre cum se construieşte aplicaţia
    - metode pentru:
        - planificarea şi estimarea proiectului
        - analiza de sistem şi analiza cerințelor
        - proiectarea structurilor de date, arhitecturii programului şi a algoritmilor
        - coding, testare şi întreținere
- instrumente
    - oferă sprijin automat şi semiautomat pentru metode

    - specifice pentru fiecare clasă de metode
    - instrumente integrate (CASE)
- procedee
    - liantul ce uneşte metodele şi instrumentele
    - definesc
        - secvența în care se aplică metodele
        - documentele (documentații, rapoarte, formulare) necesare
        - verificările pentru asigurarea calității
        - punctele de verificare (milestones) pentru evaluarea progreselor realizate

3. Fundamentele proiectarii

Scopul proiectării este producerea specificațiilor de proiectare, formate din

(i) proiectul de arhitectură a sistemului

(ii) modelele logice şi fizice de date

(iii) specificațiile de proiectare a procedurilor

(iv) proiectul interfețelor

4. Pasii proiectarii

- Selectia
    - Obiective
        - căutarea şi identificarea soluțiilor alternative (manuale şi informatice) pentru sistemul studiat (țintă)
        - evaluarea fezabilității fiecărei soluții alternative
    - Activități
        - identificarea soluțiilor posibile
        - consultarea utilizatorilor, managerilor, personalului tehnic
        - start: specificarea cerințelor
        - analiza fezabilității fiecărei variante (soluții)
        - stabilirea soluției alese
        - conducerea ia decizia: cumpără ŞI/SAU dezvoltă
- Achizitia
    - Obiectivele achiziției:
        - căutarea şi identificarea produselor specifice care pot ajuta soluția recomandată pentru sistemul țintă
        - solicitarea, evaluarea şi clasificarea propunerilor (ofertelor) furnizorilor
        - selectarea şi recomandarea celei mai bune oferte
        - stabilirea cerințelor pentru integrarea produselor ce se vor achiziționa în soluție
    - Activitățile achiziției:
        - (i) stabilirea criteriilor tehnice
        - (ii) solicitarea de oferte
        - (iii) validarea ofertelor
        - (iv) evaluarea ofertelor
        - (v) stabilirea ofertei câştigătoare
        - (vi) stabilirea cerințelor de integrare a produselor achiziționate în soluția propusă
- Proiectarea propriu-zisa

- Schiță a proiectului general pentru sistemul țintă
  - proiect de ansamblu (proiect preliminar)
- Activități
  - (1) Proiectarea arhitecturii programului
  - (2) Analiza şi distribuirea datelor (proiectarea logică a datelor)
  - (3) Proiectarea logică a prelucrărilor

5. Principiile proiectarii

- Abstractizarea
  - Abstractizarea funcțională (procedurală) - se realizează prin
    - specificare: nume, parametri, pre- şi postcondiții
    - parametrizare: clase de probleme
  - Abstractizarea datelor - caracteristici:
    - încapsularea: reprezentarea şi operațiile sunt puse împreună (clase)
    - ascunderea informației: accesul la reprezentare se face numai prin intermediul operațiilor (metode get/set)
  - Abstractizarea controlului
- Ascunderea informației
- Descompunerea
  - Instrumente folosite:
    - diagrame ierarhice
    - rețele de procese (DFD – “Data Flow Diagram” numai cu procese şi fluxuri de date)
  - Avantaje
    - gestionarea complexității
    - implementare şi testare separată a subsistemelor
    - activități paralele, muncă în echipă
- Modularizarea
  - Permite proiectantului să
    - (i) descompună un sistem în unități funcționale
    - (ii) impună o ordine ierarhică a folosirii acestora
    - (iii) implementeze abstractizarea datelor
    - (iv) dezvolte subsisteme independente
  - Gradul de independență a unui modul
    - cuplarea (măsoară interdependența relativă)
    - coeziunea (măsoară puterea funcțională)

6. Etapele proiectarii

(1) structurarea sistemului

- descompunerea sistemului în subsisteme
- identificarea interfețelor dintre subsisteme - fluxuri de date

(2) modelarea controlului

- arhitectura centralizată - un subsistem central care asigură controlul execuției tuturor celorlalte subsisteme
- sistemele dirijate de evenimente - subsistemele răspund la evenimente generate în exteriorul lor

(3) descompunerea în module

- descompunerea se face la nivelul fiecărui subsistem
    - diagramele de blocuri de arhitectură (conțin subsistemele şi fluxurile de date)
    - modelele de control

(4) optimizarea proiectului

- (1) la proiectarea arhitecturii - se descompune sistemul în module şi se specifică fiecare modul
- (2) la proiectarea de detaliu - se elaborează specificațiile de programare pentru fiecare modul
- (3) la implementare - se implementează (codificare + testare) modulele proiectate
- (4) la testarea de sistem - se măsoară performanțele sistemului şi se detectează locurile înguste (servicii şi module critice)
- (5) se reconfigurează şi se recombină modulele critice şi se reia de la (3), până când se obțin rezultatele aşteptate

(5) revizuirea proiectului

- obiectul revizuirii
    - (a) caracteristicile funcționale
    - (b) atributele de performanță
    - (c) interfețele cu mediul extern
    - (d) dialogurile cu utilizatorul
    - (e) formatul rapoartelor
    - (f) condițiile ce generează excepții şi gestiunea acestora

Cursul 6
Managementul estimarilor

Cuprins

1. Estimari
2. Metodologii de estimare
3. Estimarea efortului
4. Evaluarea costurilor – COCOMO
5. Estimarea iterativa
6. Analiza financiara a proiectelor

1. Estimari

- Realizate, utilizate si modificate in timpul etapelor de
    - Planificare strategica
    - Studiu de fezabilitate si/sau SOW (Statement of Work)
    - Propuneri
    - Evaluarea dezvoltatorului sau a sub-contractantilor
    - Planificarea proiectului (iterativa)
- Procesul de estimare
    - Estimarea dimensiunii produsului
    - Estimarea efortului necesar (oameni-luni)
    - Estimarea programului proiectului

2. Metodologii de estimare

- Estimari “top-down”
    - Avantaje
        - Usor de calculat
        - Foarte eficiente la inceput (ex. Estimarea initiala a costurilor)
    - Dezavantaje
        - Modele discutabile
        - Risc de potrivire redusă
        - Precizie redusă – nu iau in calcul detaliile proiectului
    - Exemple: Analogia, Judecata Expertului si Metodele algoritmice
- Estimari “bottom-up”
    - Genereaza WBSs (Work-Breakdown Structures)
    - Avantaje
        - Foarte eficiente in cazul activitatilor care sunt foarte bine intelese
    - Dezavantaje
        - Anumite activitati nu sunt cunoscute intotdeauna
        - Consumatoare de timp

3. Estimarea efortului

- Tabele de orar
    - Convertirea estimarilor de dimensiune in estimari de effort
    - Folosesc date istorice
- Combinare între estimarile de dimensiune şi de effort
- Programarea bazată pe angajament
    - Un dezvoltator isi ia un angajament pe baza unei estimari proprii

4. Evaluarea costurilor – COCOMO

- COCOMO – COnstructive COst Model
    - cel mai bine documentat şi transparent model de evaluare a costurilor
- Influența a 15 factori de cost în determinarea efortului de dezvoltare a entităților software

5. Estimarea iterativa

- Estimarile sunt rafinate gradual
- La fiecare etapa de planificare – cea mai buna estimare posibila
- Estimarile sunt revizuite iterativ in timp ce planurile sunt re-ajustate
- Planurile si deciziile sunt revizuite in functie de noile estimari
- Pastrarea unui echilibru: prea mule revizuiri vs. prea putine

6. Analiza financiara a proiectelor

Exista trei metode principale de determinare a valorii financiare a unui proiect:

- Analiza NPV (Net Present Value) - Valoarea Actualizata a Investitiei
    - NPV reprezinta valoarea neta actualizata- acea valoare prezenta a beneficiilor obtinute printr-o investitie, dupa ce s-a tinut cont de orizontul de timp specific proiectului pentru care se calculeaza si luand in considerare valoarea in timp a investitiei.
    - Daca valoarea NPV este negativa, ea indica cu certitudine faptul ca proiectul nu ar mai trebui realizat. Daca insa este pozitiva, atunci nu mai ofera indicii referitoare la ceea ce ar trebui facut
    - O valoare pozitiva nu poate fi comparata cu alt proiect decat daca orizontul de timp si dimensiunile investitiei sunt aceleasi.
- Randamentul investitiei (Return Of Investment)
    - Printre factorii secundari trebuie luati in considerare urmatorii:
        - Costurile- cu cat costa mai mult o anumita rutina de activitate, cu atat mai mare va fi beneficiul dedus din automatizare sau din suportul tehnologic specific
        - Cunoasterea- cu cat este mai mare potentialul de reutilizare a informatiei in sistem, cu atat este mai mare si ROI
        - Colaborarea- comunicarea intre angajati este costisitoare, astfel ca, cu cat va fi mai extinsa componenta de colaborare, cu atat va fi mai mare ROI potential
    - Daca ROI este mai mic decat este nevoie pentru a initia proiectul, este foarte posibil sa poata fi corectat astfel:
        - Modificarea calendarului costurilor: schimbarea costurile din anul initial prin distribuirea investitiilor de instruire si consultant ape parcursul celorlalti ani
        - Negocierea preturilor: o mica scadere procentuala a preturilor poate determina o crestere dramatica a ROI, in functie de dimensiunile proiectului
        - Cresterea graduala a costurilor cu angajatii pentru instruire si alte scopuri, pe masura ce utilizarea tehnologiei devine mai eficienta si determina cresterea ROI
- Perioada de amortizare
    - Perioada de amortizare este intervalul de timp necesar obtinerii beneficiilor pentru a stinge costul initial al proiectului
    - Acesta este un indicator cheie al riscului - intr-un mediu tehnologic in schimbare
    - Majoritatea companiilor doresc proiecte IT care sa aiba o perioada de amortizare relativ scurta

Cursul 7
Controlul proiectelor

Cuprins

1. Acțiuni pentru controlul proiectului
2. Monitorizarea costurilor
3. Analiza riscurilor
4. Identificarea riscurilor

1. Acţiuni pentru controlul proiectului

- Anticiparea (acțiune proactivă)
  - buclă de control cu feedback ↔ control feed-forward
  - contracararea din timp a perturbațiilor din proiect
- Modificările de plan (acţiune reactivă)
  - cererea clientului
  - greşeli în planurile inițiale
  - dificultăți neprevăzute în planul iniţial

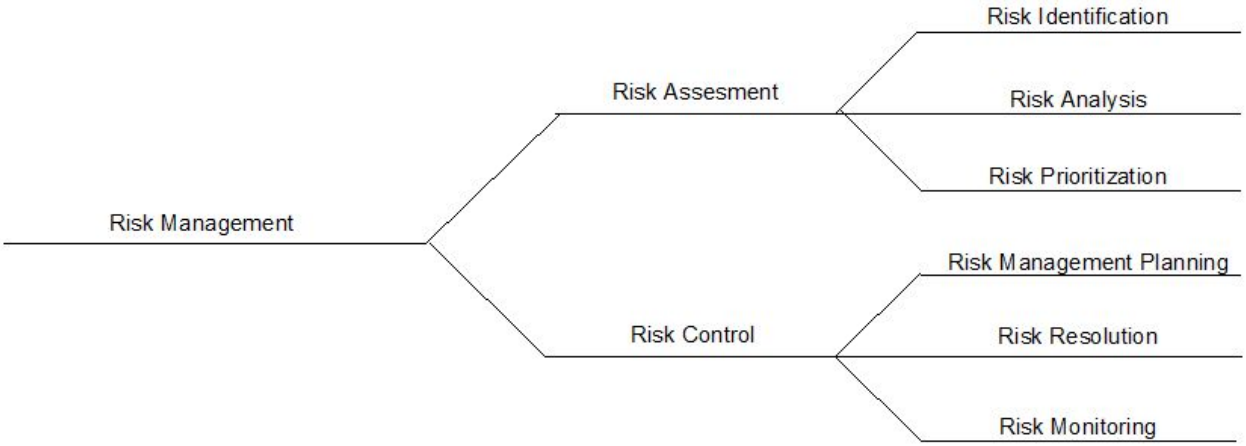
2. Monitorizarea costurilor

- Faza de concepţie – cel mai potrivit moment
- Proiectul avansează → influenţa costurilor scade
- În fazele finale, în mod practic, lipsă de influenţă a costurilor

3. Analiza riscurilor

- În faza de elaborare a proiectului analiza de risc se realizează cu titlu preventiv asupra mijloacelor şi ipotezelor de orientare a proiectului
  - riscul de concurență (bariere de intrare, concurența, etc.)
  - riscul de piață (conjunctura comercială, obiceiuri)
  - riscuri comerciale (fabricația produsului, la termen, raportul cost / calitate)
  - riscuri tehnologice (norme, competențe)
- În curs de execuție, prin mijloace tehnice de pilotaj se detectează şi se măsoară abaterile realizând o analiză aposteori a cauzelor evenimentului într-o viziune corectivă ce va putea fi utilizată pentru proiectele viitoare
  - Conform metodei AMDEC (Analiza modurilor de defectare şi efectele acestora) o percepție rațională a riscului se realizează după următoarea tipologie:
    - detectarea slăbiciunilor înainte ca ele să se producă poate fi mai mult sau mai puțin precisă şi mai mult sau mai puțin tardivă, după caz. O procedură controlată, într-un context organizațional favorabil, poate da rezultate bune. Concret o informație bună circulă la un moment favorabil către un actor responsabil
    - diagnosticul cauzei – există metode statistice care pot furniza date care pot decide măsuri asiguratorii contra erorilor de diagnostic

- analiza prognosticată a efectului – acest tip de analiză intervine când efectul încă nu este evident sau va fi realizat în termen lung care nu poate fi precizat



4. Identificarea riscurilor

- Experienta cu proiecte similare in trecut
- Stabilirea profilului riscurilor
- Brainstorming in cadrul echipei
- Analiza planurilor initiale – incertitudinile si presupunerile initiale reprezinta de cele mai multe ori riscuri

Cursul 8
Managementul contractelor si achizitiilor

Cuprins

1. Reguli generale
2. Tipuri de contractari
3. Procesul de achiziţie
4. Finalizarea contractului

1. Reguli generale

- Contractele reclama derularea unor formalitati contractuale
- Toate cerintele/specificatiile de produs si de proiect management trebuie mentionate specific
- Daca nu se metioneaza limpede in contract, modificarile se fac numai cand apar in discutie
- Orice schimbare trebuie intreprinsa prin acordul semnat de parti si conform unui derulator
- Modificarile contractuale trebuie consemnate in scris
- Contractele trebuie sa respecte cadrul legal

- Contractele trebuie si au rolul de a diminua din riscul de proiect
- Majoritatea guvernelor sustin contractarile prin rezolvarea disputelor in fata unor curti speciale

2. Tipuri de contractari:

- Contractarea centralizata
    - Firma dispune departament de contractari si relatii comerciale
    - Avantaje
        - Creşterea experienţei în contractări
        - Feedback membrilor departamentului
        - Standardizarea practicilor companiei
        - Profesionistii in contractare au drumul batatorit in ascensiunea profesionala
    - Dezavantaje
        - Supraîncărcarea angajaţilor departamentului
        - Dificultatea asigurării unei forme de asistență la nevoie
- Contractarea descentralizata
    - Procesul achizitiilor e delegat unui manager insarcinat cu contractarile
    - Avantaje:
        - Accesul facil la cunoasterea tehnicilor contractuale
        - Solicita mai multa concentrare pe experienta contractuala
    - Dezavantaje:
        - Dificultate in mentinerea experientei contractuale la un nivel inalt
        - Standardizare redusă a practicilor contractuale de la un proiect la altul
        - Tendinte de imprecizie in definirea carierei legata de profesia in contractare

3. Procesul de achiziţie

Exista 6 activităţi consecutive de achizitie

- Planul de achiziţii
- Planul de Contractări
- Solicitarile de Oferta catre furnizori prin Cererile de Oferta adresate acestora
- Selectarea furnizorilor
- Administrarea contractelor
    - Atentie la:
        - analiza performantei furnizorului(buyer conducted performance review)
        - administrarea petitiilor(claims administration)
        - sistemul de mentinerea evidentelor (records managemnt system)
        - interpretarea contractului
- Finalizarea contractelor de achizitii

4. Finalizarea contractului

Finalizarea contractului presupune:

- Verificarea produsului - Verificarea produsului consta in controlul tuturor operatiunilor si elementelor care trebuie finalizate corect si satisfacator.
- Finalizarea financiara a contractului - Realizarea platilor finale si mentinerea evidentelor
- Actualizarea evidentelor in sistemul de management al evidentelor

- Raportarea Finala a activitatii ca urmarea a inchiderii contractului-Analiza si evidenta asupra gradului de performanta decurs si a eficientei activitatilor ca urmare a finalizarii contractului
- Arhivarea contractului - Organizarea documentelor contractului dupa sistemul dosarelor, mentinerea unui sistem de evidenta si inregistrare, folosirea unui sistem de codare, cu evidente istorice si elemenete anexa utile in cazul nefinalziarii contractului sau a unor intimpinari de natura juridica si financiara
- Auditul de Achizitie - Este o trecere in revista structurata a procesului de achizitie. E posibili ca chiar furnizorul sa fie implicat in realizarea unor astfel de auditari
- Lectii invatate - vor reflecta modul de lucru cu furnizorul

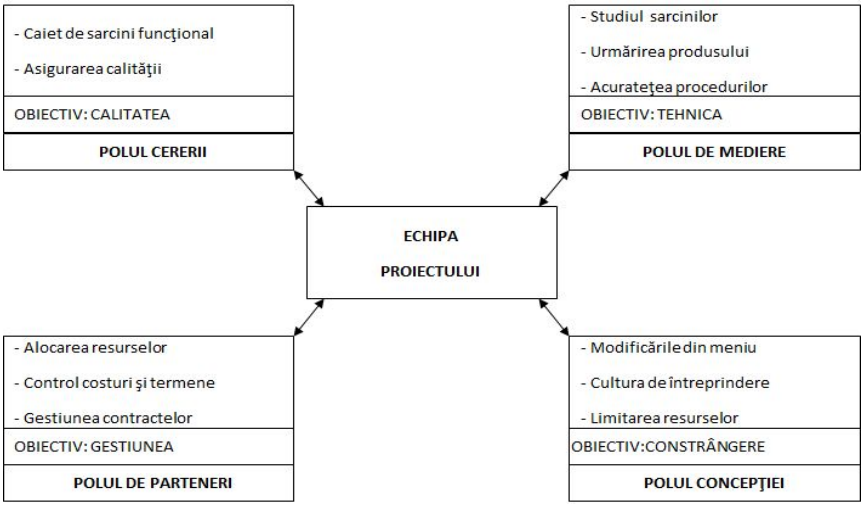
Cursul 9
Managementul oamenilor

Cuprins:

1. Echipa de lucru a proiectului
2. Programatorul ca membru al unei echipe
3. Structura echipei proiectului
4. Conducerea proiectului
5. Cultura organizațională
6. Planificarea detaliată a activităților pentru un şef de proiect

1. Echipa de lucru a proiectului

- Polul cererii – regrupează clienții şi utilizatorii potențiali ce pot fi consultați într-o manieră informațională de exemplu cu ocazia unui târg de prezentare sau într-o metodă formală, într-un cadru oferit de panoul de utilizatori
- Polul de mediere – are contribuții în sprijinirea inovației : consilii regionale, organisme interesate în dezvoltarea tehnologiei, etc. În general aceste organisme reuşesc să promoveze diferite inovații materiale şi competențe ştiințifice, punând la dispoziție suporturi de proiecte care pot transforma o idee în produs comerciabil
- Polul de concepție – format din diferite centre de competență susceptibile a furniza sfaturi specializate asupra fezabilității tehnice a ideii
- Polul de parteneriat – sunt cuprinşi ansamble de furnizori sau subantreprenori care participă la realizarea unor subansambluri ale proiectului



Caracteristici principale ale echipei de lucru în managementul proiectelor sunt:

- mărime: conține maxim 10, optim 6 – 7 persoane
- regulă de joc în care trebuie stabilite obiectivele şi misiunile, calendarul, condițiile de buget, riscurile, nivelul de calitate, durata de viață a echipei
- puternică motivație la nivelul tuturor membrilor
- un sistem de comunicare eficient
- solidaritate reală bazată pe ascultare, întrajutorare, cooperare, utilizarea unui limbaj comun
- creativitate stimulată pentru cercetare şi găsire de soluții rapide şi eficiente

2. Programatorul ca membru al unei echipe

Tipuri de personalități de grup:

(I) orientată pe lucru - motivată de munca pe care o face

(II) orientată pe interacțiune - motivată de prezența şi acțiunile colaboratorilor

(III) auto-orientată - motivată de dorința de succes personal

Personalități predominante în echipă:

- tipul I: orientată pe lucru
  - EŞEC echipa se sparge într-o sumă de indivizi, fiecare cu idei proprii asupra modului de realizare a proiectului
- tipul II: orientată pe interacțiune
  - SUCCESUL se obține prin participarea tuturor membrilor echipei
- tipul III: (III) auto-orientată
  - primează succesul personal nu succesul echipei

Recomandări:

- personalități de toate tipurile, în special I şi II
- tipul III să aibă puțini reprezentanți
- şeful echipei să aibă o personalitate de tipul I

Stiluri de conducere a echipei

- autoritar
- democratic

3. Structura echipei proiectului

Structurarea proiectului

- stabileşte câte echipe participă la realizarea acestuia
- alternative
    - o singură echipă
        - echipa este responsabilă cu produsul soft pe tot ciclul de viață a acestuia
        - avantaje
            - tranziție uşoară între activități
        - dezavantaje
            - nevoie de specializare mai largă a membrilor echipei
            - lucrează concomitent la mai multe proiecte
    - structură funcțională - fiecare activitate este realizată de o altă echipă
        - echipe diferite pentru: planificare, analiză, proiectare, implementare, testare de sistem, asigurarea calității, întreținere, documentare, etc.
        - în practică, se recomandă trei echipe (analiză, proiectare + implementare, testare + întreținere) şi grupuri de sprijin specializate (documentare, instalare, instruire)
        - avantaje
            - specializare mai îngustă a personalului
            - atenție acordată activităților auxiliare (documentare, instruirea utilizatorilor, asistență tehnică)
        - dezavantaje
            - comunicarea între echipe
    - structură matriceală - fiecare activitate are o echipă de conducere şi o echipă de specialişti
        - similară organizării matriciale a serviciului informatic
        - fiecare activitate are:
            - o echipă de conducere proprie
            - o echipă de specialişti proprie
            - o persoană face parte concomitent din mai multe echipe
        - avantaje
            - concentrarea competenței în echipe mici şi stabile
            - flexibilitate în asigurarea de resurse umane pentru proiecte
            - dă o deosebită flexibilitate întreprinderii
            - stimulează cooperarea interdisciplinară
            - implică şi provoacă angajații în activități inovatoare
        - dezavantaje
            - o persoană are doi şefi: unul pe linie tehnică, unul pe linie ierarhică
            - încurajează lupta pentru putere
            - poate conduce la discuții inutile în argumentarea acțiunilor
            - solicită pentru conducători mari aptitudini interpersonale
            - de multe ori dublează efortul intelectual
            - afectează moral personalul care este continuu rearanjat în alte grupe de lucru

Organizarea internă a echipei - de la descentralizat spre centralizat

- echipe informale (nestructurate)
    - Altruistă
        - echipa stabileşte prin consens obiectivele de atins şi deciziile de urmat
        - conducerea este asigurată prin rotație
    - Democratică
        - la fel cu echipa altruistă cu deosebirea că şeful nu se roteşte
    - Avantaje
        - toți membrii participă la luarea deciziilor
        - membrii învață unii de la alții
        - satisfacții profesionale sporite
    - Sezavantaje
        - volum mare de comunicare
        - cerință: compatibilitate ridicată între membri
        - slabă responsabilitate: a greşit echipa (CINE?), nu individul
        - dezastru: echipe fără experiență şi formate numai din incompetenți
- echipe structurate
    - echipa ierarhică (impartita in subechipe)
        - Avantaje: limitează numărul de canale de comunicare (la echipe mari)
        - Dezavantaje: de regulă devin şefi de subechipe specialiştii cei mai buni
    - echipa programatorului şef
        - Avantaje
            - deciziile se iau centralizat
            - canale de comunicare puține - productivitate sporită
            - dublarea productivității față de echipele nestructurate
        - Dezavantaje
            - programatorul-şef este de neînlocuit
            - programatorul şef culege toate laudele
            - este greu de evidențiat contribuția unui membru de rând

4. Conducerea proiectului

Se disting patru opțiuni de configurații:

- proiect cu facilitator – un personaj detaşat care se ocupă de un proiect antrenând în discuții şi acțiuni minore definite
- proiecte cu coordonator – coordonatorul proiectului nu are autoritate ierarhică asupra membrilor grupului. El are doar o autoritate funcțională
- proiecte structurate în matrice – matricea are două logici : logica de funcțiuni şi logica de proiecte.
- structuri ad-hoc

5. Cultura organizațională

Cultura organizațională cuprinde un ansamblu de valori morale, principii, norme, simboluri şi rituri care-şi transmit şi imprimă mesajul asupra comportamentului angajaților, cu implicații directe asupra eficienței, eficacității şi imaginii întreprinderii.

Cultura organizațională este determinată de trei categorii de factori:

- factori centrali, sunt determinați de resursele umane ale organizației şi ei sunt reprezentați de:
    - profilul şi personalitatea conducătorilor

    - riturile şi simbolurile specifice instituției
    - comunicația
- factori manageriali care sunt generați de metodele şi instrumentele de lucru manageriale:
    - strategia
    - structurile şi procedurile
    - sistemele de conducere
- factori de mediu, generați de starea organizației şi a mediului exterior

6. Planificarea detaliată a activităților pentru un şef de proiect

Pasul 1 – Identifică sarcinile

Pasul 2 – Pune sarcinile într-o ordine logică

Pasul 3 – Studiază implicațiile

Pasul 4 – Evaluarea resurselor cerute

Pasul 5 – Identifică ierarhia proiectului

Pasul 6 – Clarifică cine poate să adopte decizii

Pasul 7 – Monitorizează şi controlează

Pasul 8 – Respectă anumite reguli de grup

Cursul 10
Fazele finale ale proiectelor

Cuprins:

1. Fazele finale ale proiectelor
2. Criteriile de baza pentru evaluarea membrilor echipei
3. Factori de succes a proiectelor

1. Fazele finale ale proiectelor:

- Confirmarea indeplinirii tuturor cerintelor de proiect
- Verificarea finalizarii fiecarei faze de proiect
- Obtinerea formala a aprobarii de la beneficiar
- Evidenta finalizarilor timpurii
- Plati finale
- Documenarea experientei/lectii invatate
- Actualizari de date
- Asigurarea finalizarii proceselor
- Actualizare proceduri de proiect
- Adaugarea noilor abilitati dobindite in baza de cunostinte
- Auditarea achizitiilor
- Elaborarea procedurii de finalizare

- Finalizarea contractuala si administrativa
- Analiza factorilor de succes si eficienta
- Intocmirea si distribuirea raportului final
- Arhivarea generala a documentelor de proiect
- Evaluarea satisfatiei clientului
- Finalizarea operatiunilor si lucrari intretinere
- Disponibilizarea resurselor
- Celebrarea

2. Criteriile de baza pentru evaluarea membrilor echipei

- Calitatea muncii depuse
- Costul
- Timpul consumat
- Creativitatea
- Performanta de natura administrativa
- Abilitatea de a lucra in echipa
- Atitudinea
- Abilitatile de comunicare
- Abilitatea tehnica
- Recomandarile de imbunatatire
- Consistenta in respectarea termenelor

3. Factori de succes a proiectelor

In cadrul managementului de proiect exista numai 3 factori de succes:

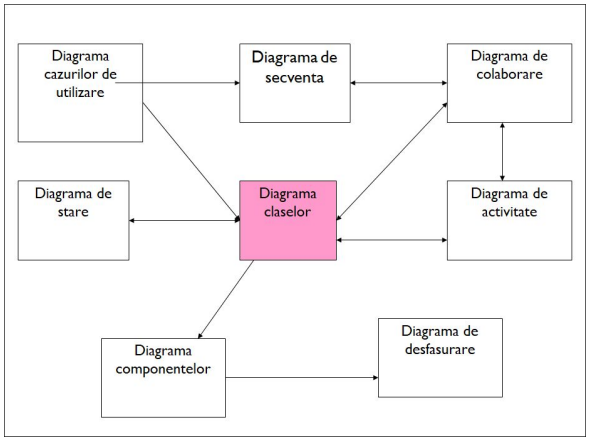
- Predarea la timp a lucrarii - Predarea la timp se face conform agendei stabilite preliminar
- Incadrarea in buget - Proiectul se va incadra in estimarile de buget previzionate
- Predarea la nivel de calitate ridicata

Cursurile 11 & 12
UML

Cuprins

1. UML - Unified Modeling Language
2. Clase de obiecte
3. Mostenirea
4. Relații de asociere
5. Actorul
6. Diagrama de secventa
7. Diagrama de stari
8. Diagrama de colaborare
9. Diagrama de activitati
10. Diagrama de pachete
11. Diagrame de componente
12. RUP

1. UML - Unified Modeling Language

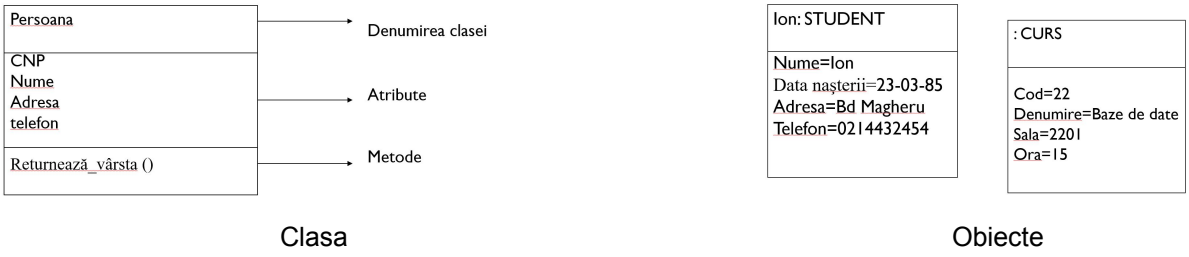


2. Clase de obiecte

- un set de obiecte cu structură şi comportament similar.
- obiectele sunt instanțe ale claselor

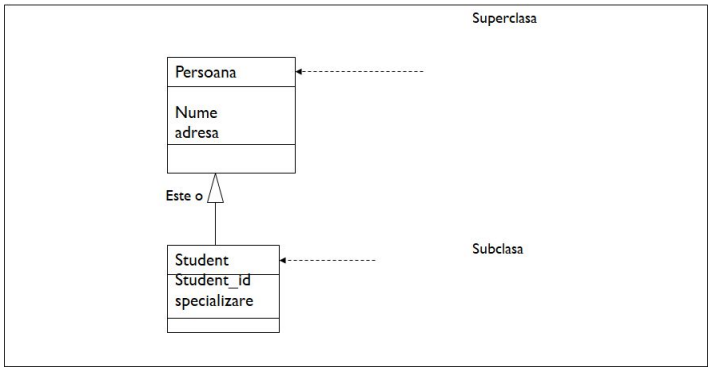
Exista trei niveluri de vizibilitate în UML:

- public (+) - are acces orice alt clasificator;
- protected (#) - are acces orice descendent;
- private (-) - numai clasificatorul însuşi poate folosi această caracteristică.



Clasa Obiecte

3. Mostenirea



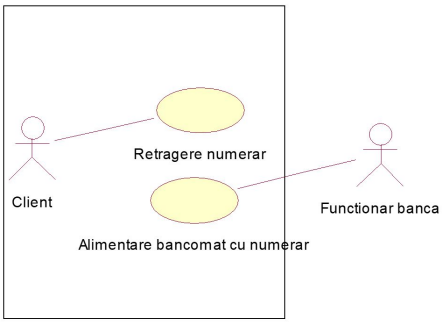
4. Relații de asociere

- Asocierea = corespondenta abstracta intre doua clase
- Legatura = corespondenta abstracta intre doua instante ale claselor
- Asocierea = abstractizare a legaturilor

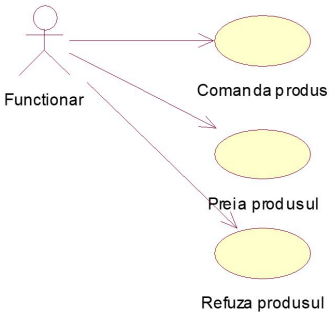
ASOCIERE	
ASOCIERE DIRECŢIONALĂ	
GENERALIZARE	
REALIZARE	
DEPENDENŢĂ	

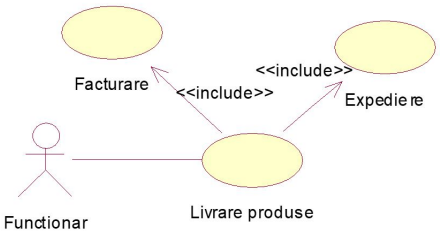
5. Actorul

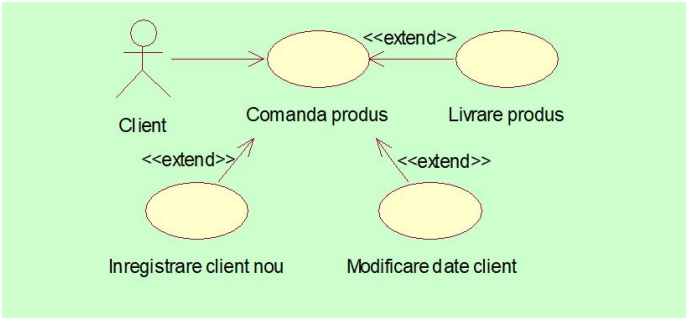
- Entitate exterioara sistemului informatic (SI) care beneficiaza de servicii, rol jucat de o persoana/sistem care interactioneaza cu SI
- Persoana, dispozitiv fizic, alt sistem
- O persoana-mai multe roluri; un rol – mai multe persoane



- CINE- doreste/e interesat de informatiile din sistem
- CINE modifica date in SI
- CINE interactioneaza cu sistemul







6. Diagrama de secventa

- Înțelegerea ordinii evenimentelor pentru a parcurge întregul scenariu vizualizare a intercomunicării claselor
- Fiecare eveniment are ca rezultat un mesaj trimis unui obiect cu perspectiva că acel obiect va realiza o operație
- Recomandate pentru realizarea de specificații în timp real şi pentru scenarii complexe

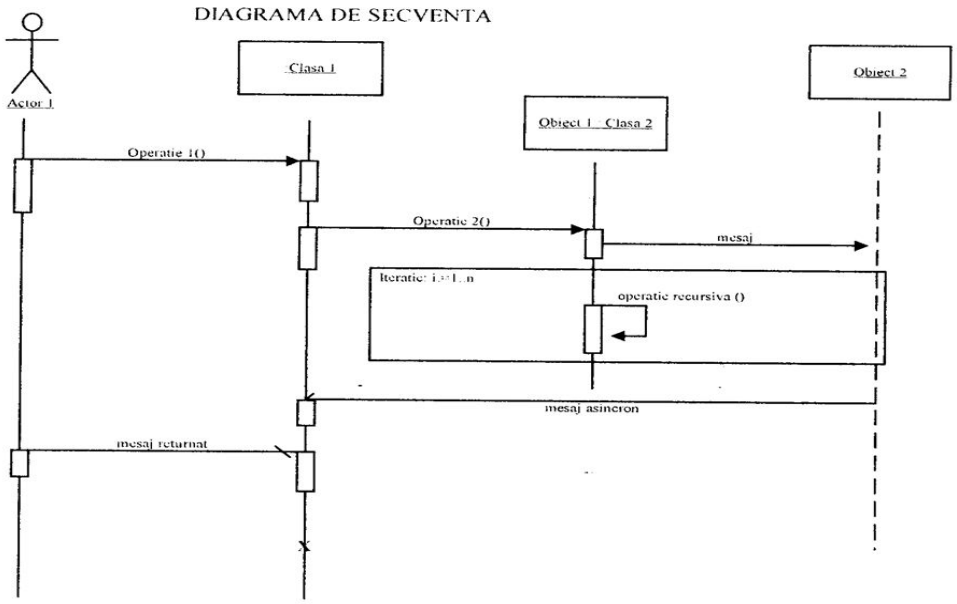


Figura 7.21. Diagrama de secvenţă (Sequence Diagram)

7. Diagrama de stari

- Comportament dinamic al obiectelor unei clase
- constă din stări, acțiuni, activități şi tranziții
- o stare = „o condiție sau o situație din momentul existenței unui obiect care satisface în acel moment anumite condiții, efectuează anumite activități sau aşteaptă anumite evenimente”.

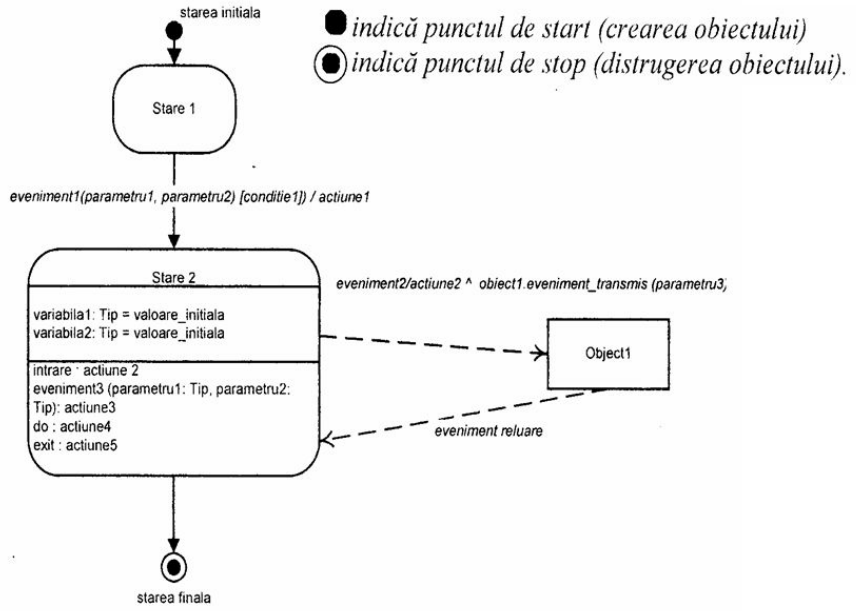
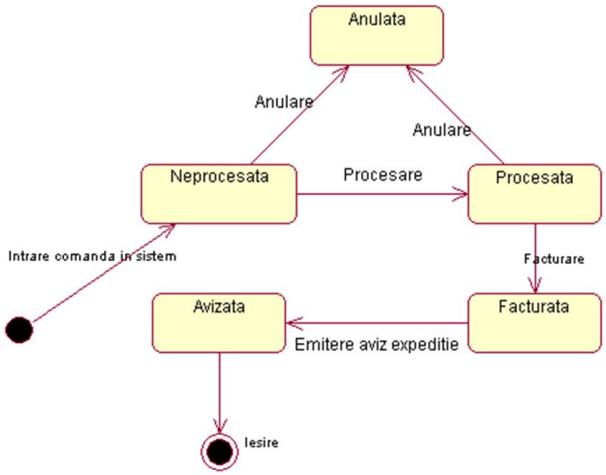


Figura 7.22. Diagrama de stare (Statechart Diagram)



8. Diagrama de colaborare

- descrie o examinare non-secvențială a modului în care interacționează obiectul.
- arată modul în care obiectele colaborează în cadrul unui singur scenariu al cazurilor de utilizare similar cu diagrama de secvență

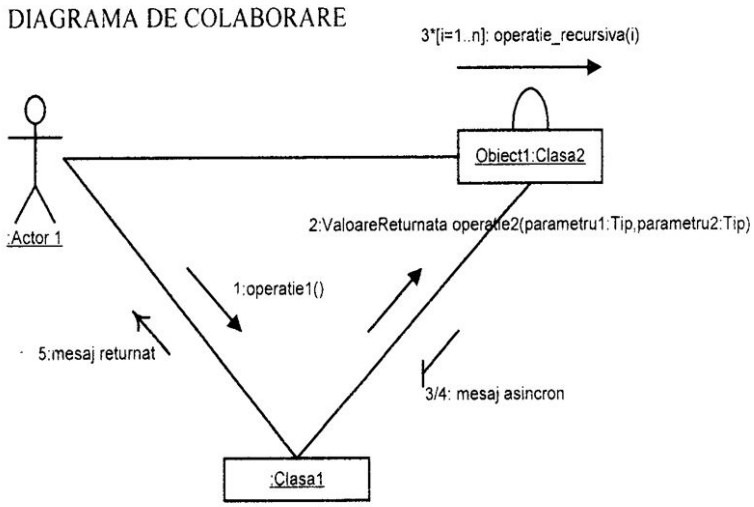


Figura 7.23. Diagrama de colaborare (Collaboration Diagram)

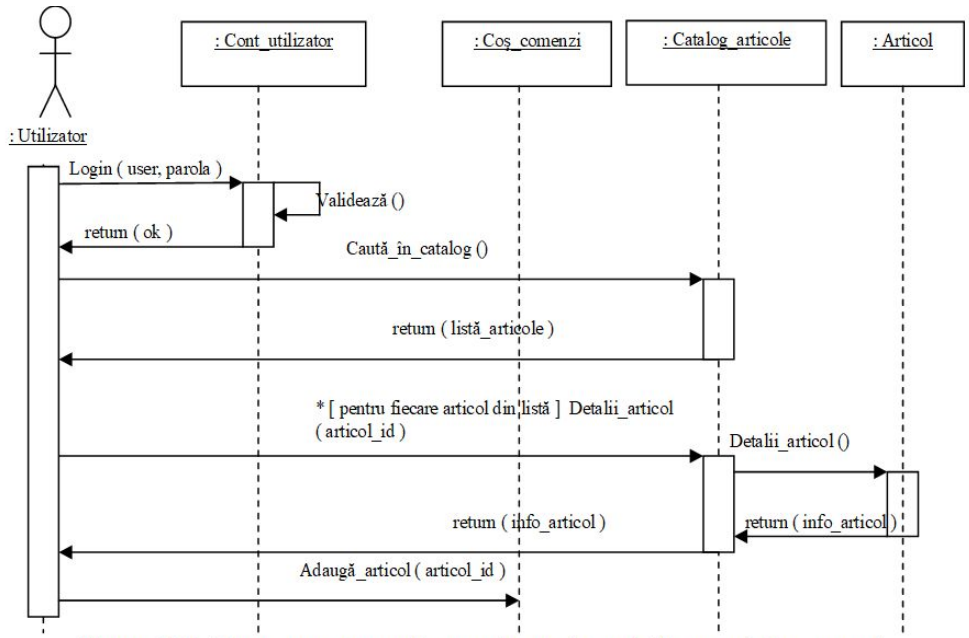


Figura 7.34. Reprezentarea unui scenariu cu ajutorul diagramei de secvenţă

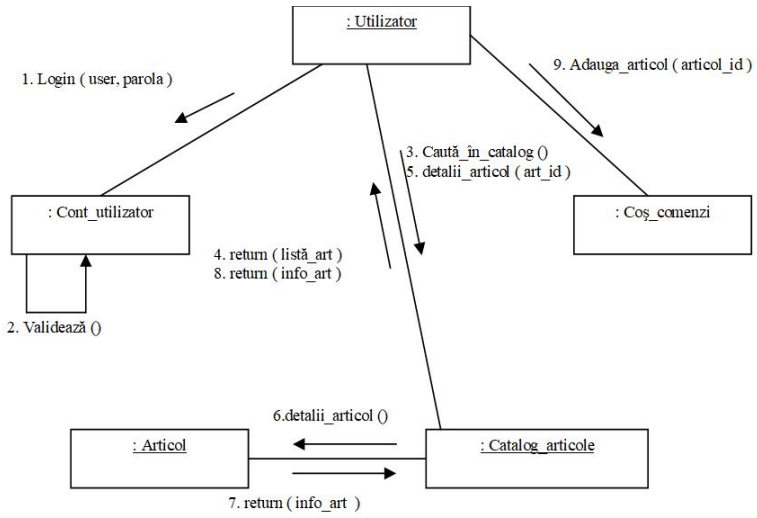


Figura 7.35. Reprezentarea unui scenariu cu ajutorul diagramei de colaborare

9. Diagrama de activitati

- o variantă a diagramei de stare, cu un scop puțin diferit, acela de a evidenția acțiuni şi rezultate ale acestor acțiuni
- permite mai buna înțelegere a detaliilor din cadrul unei operații a unei clase
- ilustreaza stările sub-acțiunilor şi sub-tranzițiile.

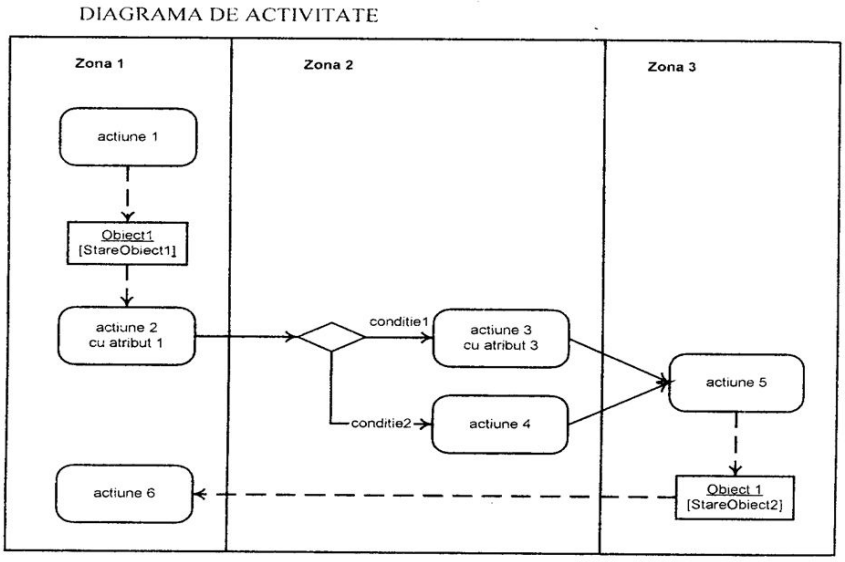


Figura 7.24. Diagrama de activitate (Activity Diagram)

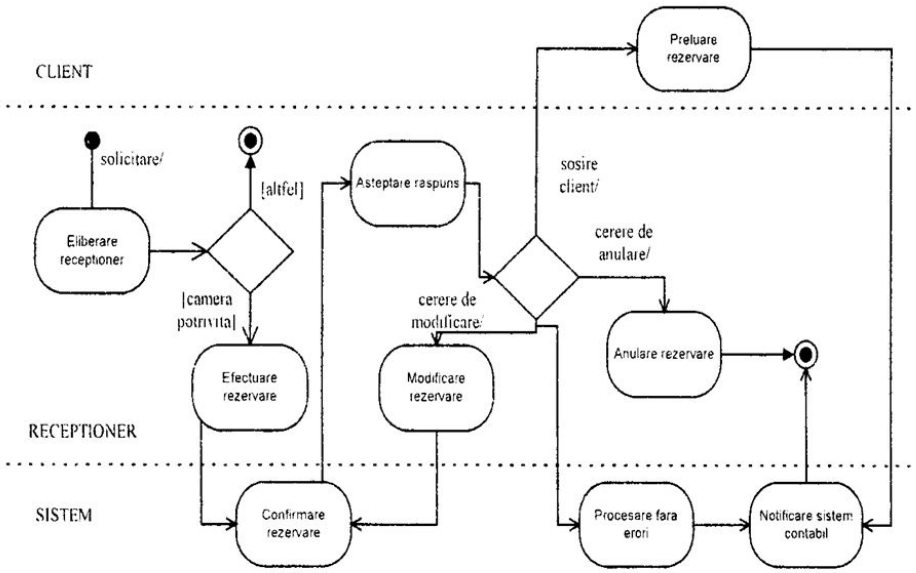


Figura 7.37. Diagrama de activitate pentru recepţia unui hotel

10. Diagrama de pachete

- grupare a elementelor din cadrul diagramelor, numite pachete.
- pot fi ambalate alte pachete, clase, cazuri de utilizare, colaborări etc.
- Un element de modelare aparține unui singur pachet, dar alte pachete pot consulta acest element.
- Dacă se arată explicit conținutul pachetului, atunci numele pachetului se trece pe etichetă.

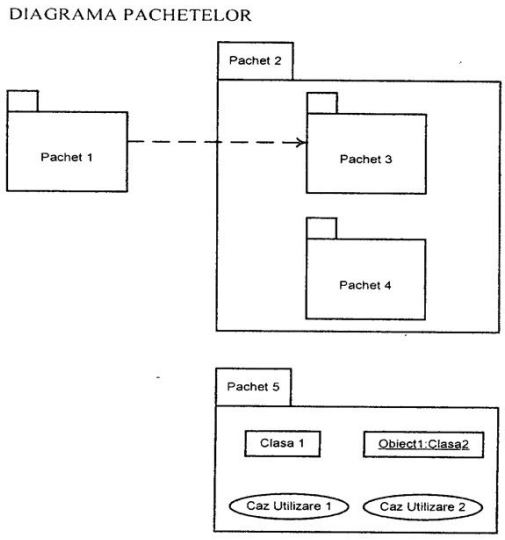
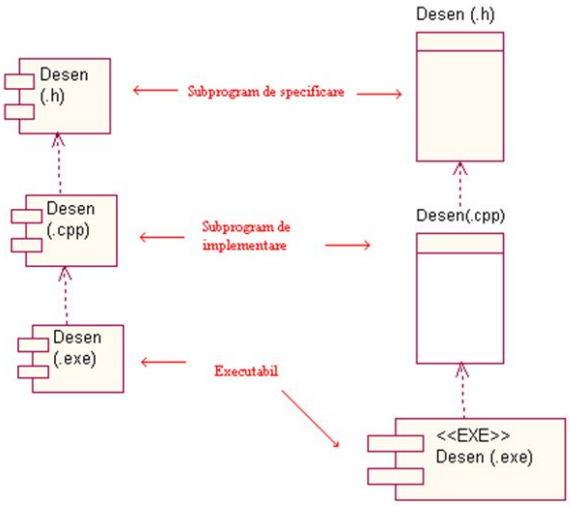
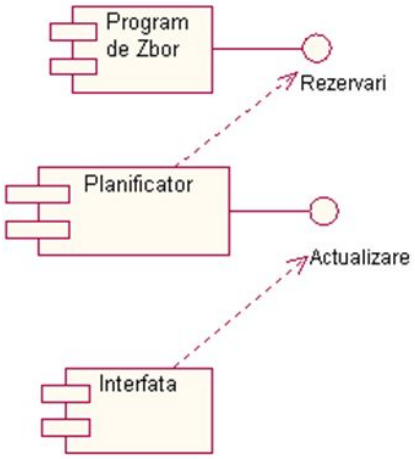


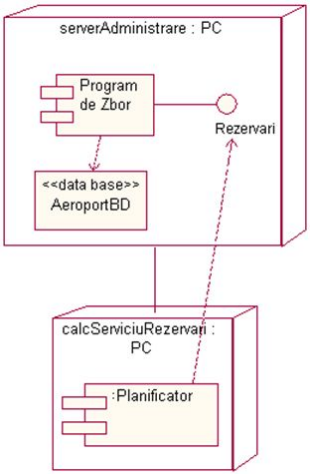
Figura 7.27. Diagrama pachet (Package Diagram)

11. Diagrame de componente

- prezintă dependențele existente între diverse componente software
- O componenta este un modul soft (cod sursa, cod binar, dll, executabil etc) cu o interfata bine definita.







12. RUP - Rational Unified Process

- Fazele descrise de UP sunt:
  - Explorarea inițială
    - definirea scopului proiectului şi la pregătirea mediului de dezvoltare
  - Elaborarea
    - capturarea şi organizarea cerințelor,
    - rafinarea viziunii asupra sistemului,
    - definirea şi validarea arhitecturii şi
    - elaborarea planului pentru faza de construcție.
  - construcția
    - Fiecare iterație - trei activități de baza:
      - managementul resurselor şi controlul procesului,
      - dezvoltarea şi testarea componentelor şi
      - evaluarea la sfârşitul iterației.
  - Tranziția
    - finalizarea documentației
    - testarea produsului la client
    - modificări minore dictate de client
    - lansarea sau instalarea produsului final
- Fiecare faza are asociat un rezultat final
- Fazele reprezintă perioada de timp scursa intre doua rezultate finale
- La sfârşitul fiecărei faze este efectuata o analiza

Dezvoltarea agila - 1

Prof. univ. dr. ing. Florica Moldoveanu

Ingineria programelor

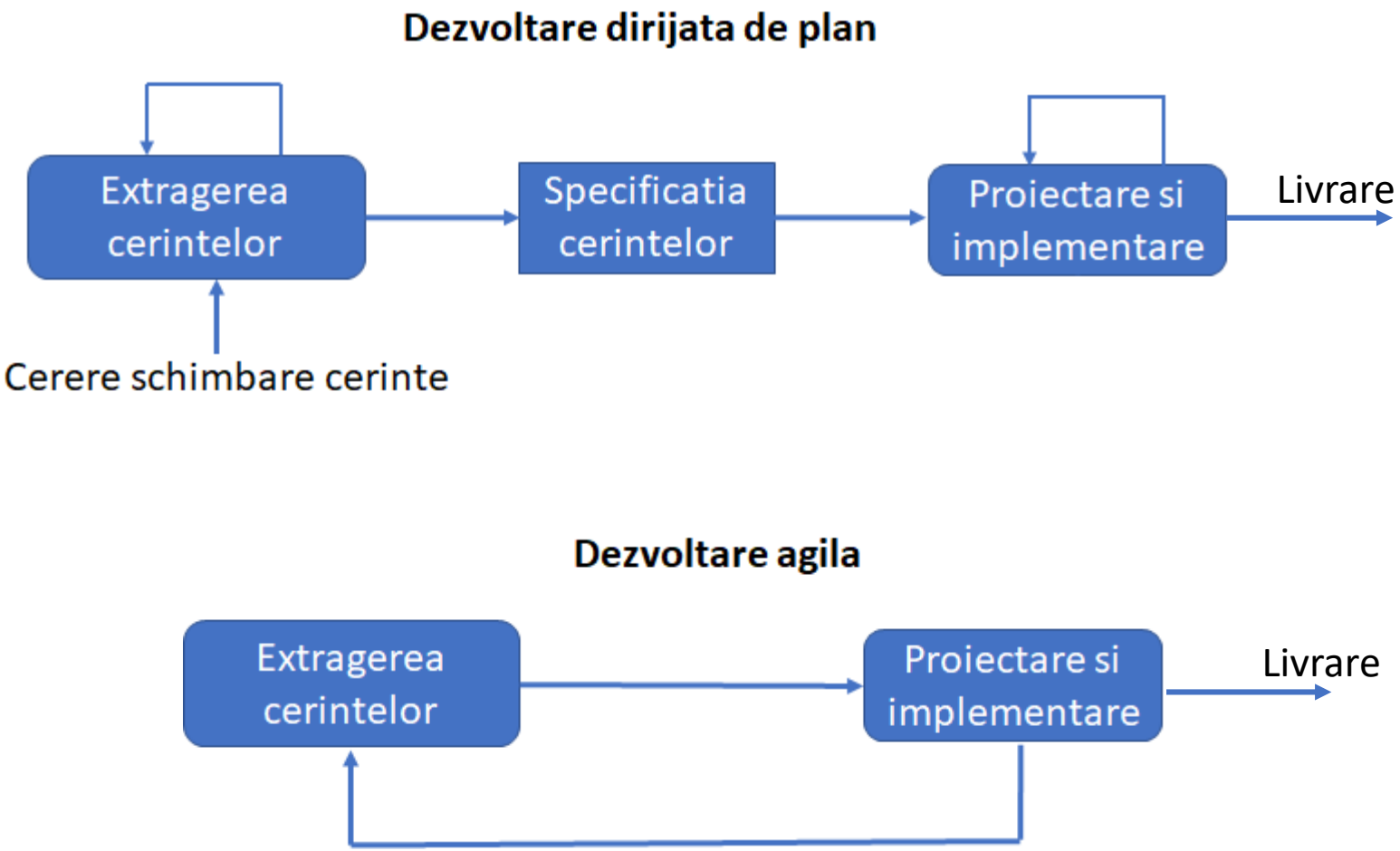
UPB, Automatică şi Calculatoare

2018-2019

Dezvoltarea agila (1)

- Dezvoltarea si livrarea rapida → aspecte importante ale proceselor software in prezent
  - ✓ Cerintele software se schimba rapid si este dificil de produs un set de cerinte stabile
  - ✓ Produsul software trebuie sa evolueze rapid pentru a reflecta schimbarile in cerinte
- **Dezvoltarea dirijata de plan** este esentiala pentru anumite tipuri de sisteme dar nu satisface cerintele unei dezvoltari rapide, adaptate la schimbarea frecventa a cerintelor
- Metodele agile au aparut la finalul anilor 1990, din necesitatea de a reduce drastic timpul de livrare pentru produsele software.
- **Dezvoltare agila**:
  - ✓ Specificarea, proiectarea si implementarea sunt intercalate
  - ✓ Sistemul este dezvoltat printr-o serie de incremente cu partile interesate implicate in specificarea si evaluarea versiunilor
  - ✓ Livrari frecvente
  - ✓ Utilizare intensiva de instrumente de dezvoltare (ex. Instrumente de testare automata)
  - ✓ Documentatie minimala – focalizare pe cod functional

Dezvoltarea agila (2)



Dezvoltarea agila (3)

❖ **Dezvoltarea dirijata de plan**

- Dezvoltare in etape separate, cu rezultatul fiecarei etape planificat in avans
- Nu neaparat dezvoltarea cascada – dezvoltarea iterativa dirijata de plan este posibila (ex. RUP)
- Iteratiile sunt planificate la inceputul procesului de dezvoltare

❖ **Dezvoltarea agila**

- Specificarea, proiectarea, implementarea si testarea sunt intrepatrunse
- Rezultatul procesului de dezvoltare este decis printr-un proces de negociere in timpul procesului de dezvoltare.

Metodologii de dezvoltare agile

PRINCIPII GENERALE

Implicarea clientului	**Clientul trebuie sa fie puternic implicat in procesul de dezvoltare. Rolul lui este de a furniza si prioritiza noi cerinte si de a evalua iteratiile dezvoltarii.**
Livrarea incrementala	**Produsul software este dezvoltat in incremente, clientul specificand cerintele care trebuie sa fie implementate in fiecare increment.**
Oameni, nu procese	**Abilitatile echipei de dezvoltare trebuie sa fie recunoscute si exploatate. Membrii echipei trebuie lasati sa-si dezvolte propriul stil de lucru, fara sa li se impuna procese prescriptive.**
Inglobeaza schimbarea	**Se asteapta schimbarea cerintelor, de aceea sistemul trebuie proiectat pentru a fi usor adaptat la schimbari.**
Mentine simplitatea	**Focalizare pe simplitate atat in produsul software dezvoltat cat si in procesul de dezvoltare. Elimina complexitatea oricand este posibil.**

Agile Manifesto – cele 4 valori fundamentale

- Metodologiile agile au aparut din necesitatea de a elimina din dezvoltarea de software activitatile care conduc la intarzierea livrarii de software operational.
- Termenul “dezvoltare agila” - derivat din **Agile Manifesto**, publicat in 2001 de un grup din care au facut parte creatorii unor metodologii existente la acea vreme: Scrum, Extreme Programming (XP), Dynamic Systems Development Method (DSDM) si Crystal.
- Agile Manifesto a stabilit un set de valori și principii generale comune metodologiilor agile existente la momentul respectiv. Sunt definite **4 valori fundamentale** care permit echipelor de dezvoltare software sa fie performante:
  - **Indivizi si interactiuni** in loc de procese si instrumente
  - **Software functional** in loc de documentatie cuprinzatoare
  - **Colaborarea cu clientul** in loc de negociere contractuala
  - **Raspunsul la schimbari** in loc de urmarirea unui plan

Agile Manifesto – cele 12 principii (1)

1. Prioritatea maxima o are satisfacerea clientului prin furnizarea rapida si continua de software functional.
2. Schimbarile in cerinte sunt binevenite, chiar si tarziu in dezvoltare, in avantajul clientului.
3. Livrarea frecventa de software functional, la cateva saptamani sau luni, preferabil la intervale mai scurte.
4. Dezvoltatorii si expertii din domeniul aplicatiei trebuie sa lucreze impreuna zilnic pe toata durata proiectului.
5. Construieste proiecte în jurul unor persoane motivate. Dă-le mediul și sprijinul de care au nevoie, și ai încredere în ei pentru a obtine rezultatul.

   Echipele agile sunt alcatuite din oameni care doresc sa lucreze impreuna colaborativ si sa invete unul de la altul. Oamenii sunt principalul factor de succes in dezvoltarea de software.

6. Cea mai eficace metoda de transmitere a informatiilor este comunicarea directa in echipa.
7. Software-ul functional este principala masura a progresului.

Agile Manifesto – cele 12 principii (2)

8. Procesele agile promoveaza dezvoltarea durabila. Sponsorii, dezvoltatorii si utilizatorii trebuie sa fie capabili sa mentina un ritm constant pe timp nelimitat.

   Pentru a obtine rezultate de calitate, oamenii nu trebuie fortati sa munceasca 12 ore/zi in permanenta.

9. Atenția continua la excelenta tehnica si o buna proiectare maresc agilitatea.

   Practici: refactoring - mentinerea unui cod de calitate si test-driven development - software-ul este functional in permanenta. Se folosesc ghiduri de codare si uneori ghiduri de modelare.

10. Simplitatea – arta de a maximiza cantitatea de munca nefacuta – este esentiala.

    Dezvoltatorii agili se concentreaza pe activitatile cu valoare mare, care maximizeaza ROI in IT, automatizand sau renuntand la munca nefolositoare.

11. Cele mai bune arhitecturi, cerinte si proiectari rezulta din echipe care se auto-organizeaza.

    Agile Model Driven Development (AMDD) și Test Driven Design (TDD) sunt abordările principale din cadrul comunității agile, pentru producerea de arhitecturi eficiente, cerințe, și modele de calitate.

12. La intervale regulate echipa analizeaza cum poate sa devina mai eficienta apoi isi ajusteaza corespunzator comportamentul.

Metode agile

❖ **Cand sunt aplicate metodele agile in dezvoltarea de software?**

- Atunci cand se doreste dezvoltarea rapida a unui produs software, fara clarificarea tuturor cerintelor de la inceput, acestea urmand sa fie extrase si implementate pe baza feedback-ului utilizatorilor la versiunile livrate pe parcurs.
- Atunci cand se doreste dezvoltarea personalizata a produsului software, cu angajarea clara a clientului de a deveni implicat in procesul de dezvoltare si exista putine reguli si reglementari externe care sa afecteze produsul software.

Care metodologie este mai potrivita pentru un proiect, dezvoltarea liniara dirijata de plan (Waterfall) sau dezvoltarea agila: https://www.seguetech.com/waterfall-vs-agile-methodology/

Metodologii agile

- Extreme programming
- Scrum
- Crystal
- Dynamic Systems Development Method (DSDM)
- Feature Driven Development (FDD)
- Kanban, s.a.

Extreme programming (XP)

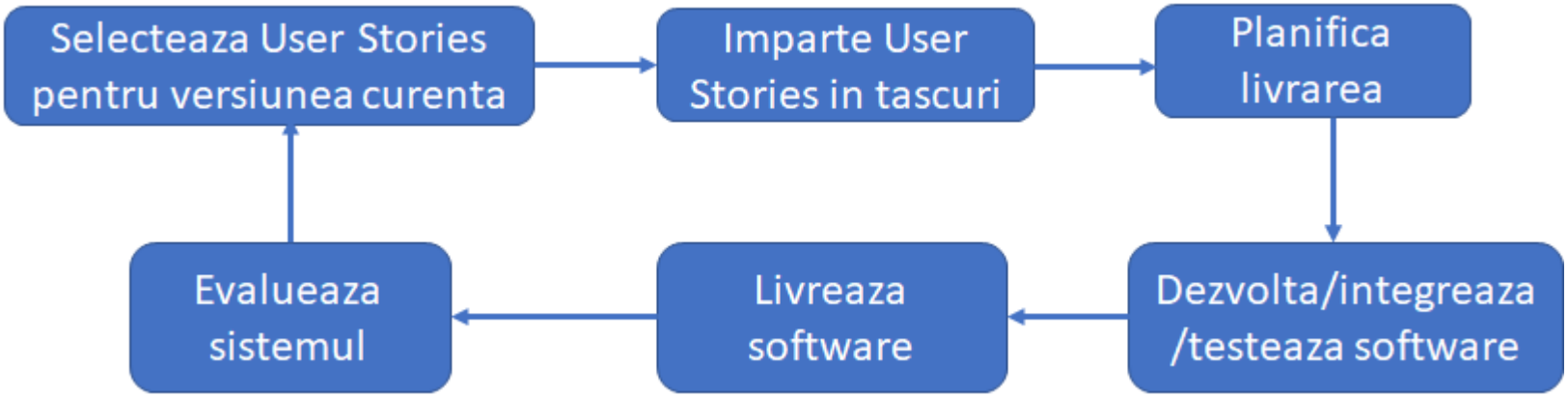
- Aparuta la sfarsitul anilor ’90
- **Abordare “extrema” in dezvoltarea iterativa**
  - Versiuni noi pot fi construite de mai multe ori pe zi
  - Incrementele sunt livrate clientilor la intervale de 2 saptamani
  - Pentru fiecare versiune executabila (build) trebuie rulate toate testele si versiunea este acceptata numai daca testele au trecut cu succes.

Practici cheie

- “User Stories” pentru specificare
- Test driven development: Test first development + Refactorizare
- Pair programming

XP – User stories

- In XP, un reprezentant al utilizatorilor finali/clientului face parte din echipa de dezvoltare si este responsabil cu cerintele.
- Cerintele utilizator sunt exprimate sub forma de scenarii, numite “user stories”.
- User story trebuie sa fie o descriere scurta, fiind limitata la o pagina de hartie de dimensiuni mici (de obicei 3x5 inches), impiedicand astfel cresterea descrierii.
- Reprezinta o modalitate simpla si rapida de a capta cerintele clientului.
- Pornind de la scenarii dezvoltatorii definesc tascuri de implementat.
- Pe baza tascurilor se fac estimarile de cost si timp.
- Clientul alege cerintele de implementat in urmatorul livrabil pe baza prioritatii lor si a estimarilor de timp.



User story

Kate is a doctor who wishes to prescribe medication for a patient attending a clinic. The patient record is already displayed on her computer so she clicks on the medication field and can select current medication', 'new medication' or 'formulary'.

If she selects 'current medication', the system asks her to check the dose. If she wants to change the dose, she enters the dose and then confirms the prescription.

If she chooses 'new medication', the system assumes that she knows which medication to prescribe. She types the first few letters of the drug name. The system displays a list of possible drugs starting with these letters. She chooses the required medication and the system responds by asking her to check that the medication selected is correct. She enters the dose and then confirms the prescription.

If she chooses 'formulary', the system displays a search box for the approved formulary. She can then search for the drug required. She selects a drug and is asked to check that the medication is correct. She enters the dose and then confirms the prescription.

The system always checks that the dose is within the approved range. If it isn't, Kate is asked to change the dose.

After Kate has confirmed the prescription, it will be displayed for checking. She either clicks 'OK' or 'Change'. If she clicks 'OK', the prescription is recorded on the audit database. If she clicks on 'Change', she reenters the 'Prescribing medication' process.

http://iansommerville.com/software-engineering-book/

XP - Test driven development (1)

Test driven development: Test first development + Refactorizare

- **Test-first development**: programatorul scrie testele de nivel coborat inainte de a scrie codul
- Testele sunt dezvoltate incremental pornind de la scenarii
- Reprezentantului utilizatorului/clientului este implicat in dezvoltarea si validarea testelor
- Sunt folosite instrumente de testare automata
- Atunci cand este adaugata o functionalitate sunt rulate toate testele anterioare si cele noi – se verifica astfel ca noua functionalitate nu a introdus erori.

Refactorizare (imbunatatirea codului)

- Rescrierea codului, fara a modifica functionalitatea sa externa, pentru imbunatatirea unor atribute ale codului, cum ar fi: usurinta de intelegere, complexitatea, usurinta de intretinere
- Codul este mai usor de inteles, reduce necesitatea documentatiei
- Modificarile se fac mai usor deoarece codul este bine structurat si clar

XP - Test driven development (2)

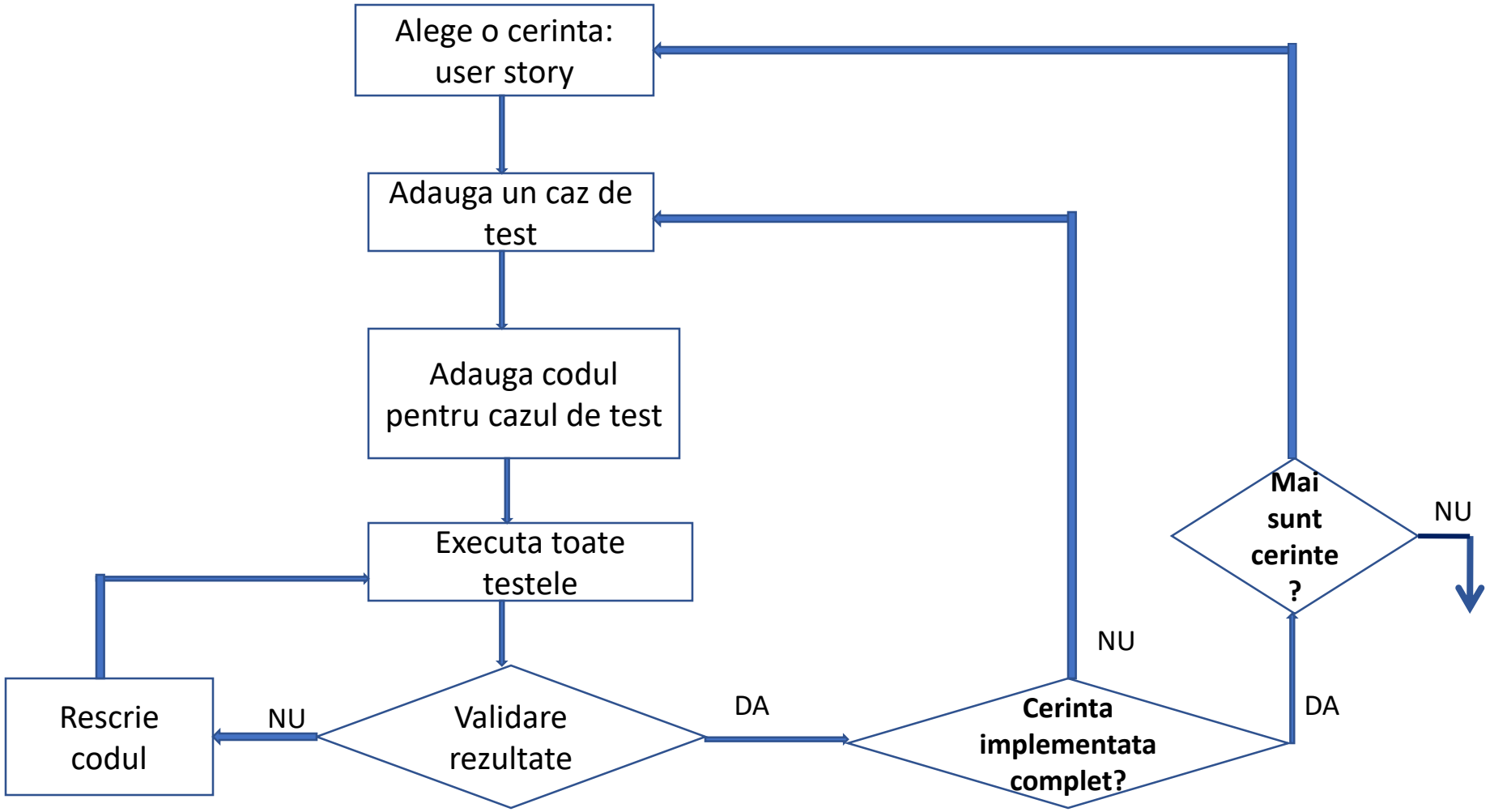
Procesul de testare si codificare in XP poate fi descris astfel:

Repeta cat timp mai exista cerinte neimplementate:

1. Se alege o cerinta functionala exprimata printr-o descriere a utilizatorului (user story)
2. Se scrie un caz de test care va verifica o parte a cerintei.
3. Se scrie codul care implementeaza partea din cerinta verificata prin cazul de test.
4. Se executa toate cazurile de test.
5. Daca rezultatele sunt validate, se reia procesul din pasul 2, pana la implementarea completa a cerintei,

   altfel, se rescrie codul si se revine in pasul 4.

XP - Test driven development (3)



XP – Pair programming

- Dezvoltatorii lucreaza in perechi, dezvoltad codul impreuna
- Este o revizie informala a codului – fiecare linie de cod este vazuta de 2 persoane
- Perechile sunt create dinamic, astfel incat toti membrii echipei lucreaza impreuna pe parcursul dezvoltarii

  → ajuta la cunoasterea codului de toata echipa

  → toti dezvoltatorii sunt responsabili pentru intregul cod (proprietate colectiva)
- Incurajeaza refactorizarea
- Dezvoltarea in echipa reduce riscul proiectului atunci cand un membru paraseste echipa.
- Dezvoltarea in perechi nu este ineficienta: exista dovezi care sugereaza ca este mai eficienta daca cei 2 ar lucra separat.

XP si principiile agile

- Dezvoltarea incrementala: incremente mici livrate frecvent
- Implicarea clientului: angajarea permanenta a clientul in echipa.
- Oameni nu procese: pair programming, proprietate colectiva, evitarea programului de lucru prelungit.
- Schimbarile acceptate prin livrari regulate.
- Mentinerea simplitatii prin refactorizarea continua a codului.

XP - limitari

- Fara teste/proceduri de acceptare specificate de client “user stories” sunt supuse interpretarii, ceea ce le face dificil de utilizat ca baza contractuala.
- Solicita contactul permanent cu clientul pe tot parcursul proiectului, dificil in unele cazuri, sau timp suplimentar consumat.
- Dificil de scalat la proiecte mari.
- Presupune dezvoltatori competenti.
- XP este greu de integrat cu practicile de management din cele mai multe companii
- Desi in dezvoltarea agila se folosesc practici din XP, metoda asa cum a fost definita initial nu este larg folosita.

Dezvoltarea agila - 2

Prof. univ. dr. ing. Florica Moldoveanu

Ingineria programelor

UPB, Automatică şi Calculatoare

2018-2019

Scrum

Metodologie de dezvoltare agila cu urmatoarele caracteristici esentiale:

❑ Echipa Scrum

- Se auto-organizeaza: nu exista un conducator al echipei care decide ce va face fiecare membru al echipei si cum se rezolva o anumita problema; acestea sunt hotarate de echipă ca un intreg.
- Este multifunctionala: este necesar ca fiecare membru al echipei să fie capabil sa trateze o problema/sarcina de la idee la implementare.
- Este sprijinită de 2 roluri specifice:

- **ScrumMaster**, care poate fi considerat ca un antrenor al echipei, ajutand-o sa foloseasca procesul Scrum pentru a performa la nivelul cel mai inalt.

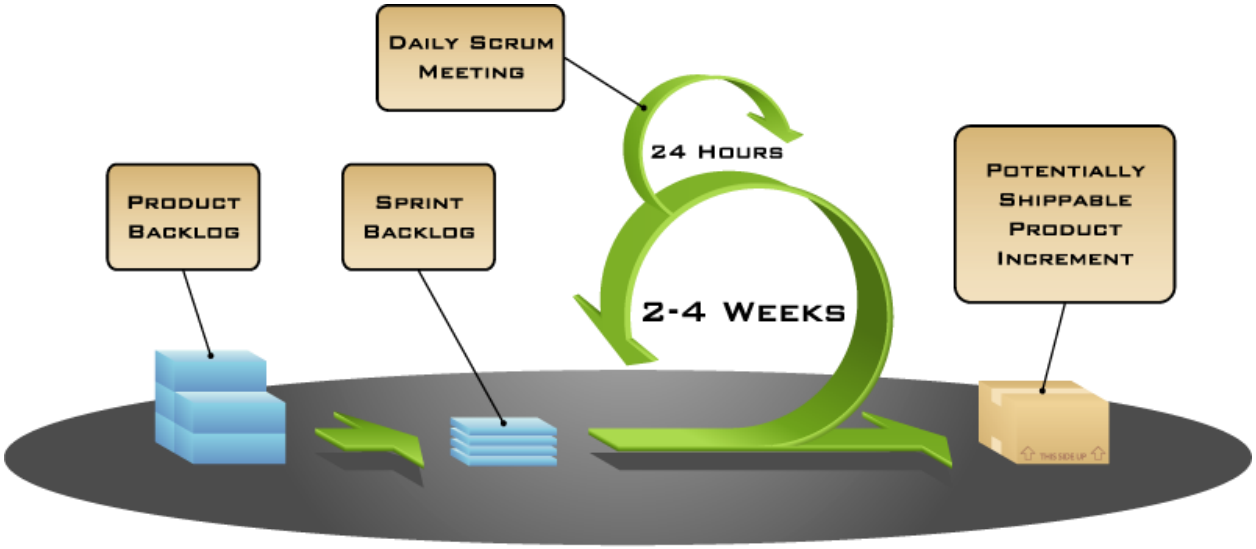
- **Product Owner**, care reprezinta clientul/utilizatorii şi ghideaza echipa pentru construirea produsului aşteptat de acestia.

Caracteristici Scrum (2)

❑ Procesul de dezvoltare

- **Product owner** creaza lista de cerinte prioritizata, numita "**product backlog**"
- Dezvoltarea este organizata in iteratii scurte, de 2-4 saptamani, numite "sprint-uri".
- In cadrul unui sprint este proiectat, codat și testat un increment al produsului final, potential livrabil.
- In fiecare sprint sunt implementate o parte dintre cerintele din "product backlog", selectate de echipă, care formeaza "**sprint backlog**".
- In timpul unui sprint echipa se intalneste zilnic pentru a evalua progresul său (**Daily Scrum**)
- Nu se accepta schimbari in timpul unui sprint, de aceea durata sprint-urilor este planificata de echipă astfel încat sa nu fie necesare modificari in timpul sprint-urilor.
- Sprintul se termina cu un "**sprint review**" si o "**retrospectiva**" a sprint-ului

Procesul Scrum (2)



Sursa: [1]

Scrum framework

Roluri

- Product owner
- ScrumMaster
- Echipa de dezvoltare

Activităţi

- Planificare sprint
- Daily Scrum
- Sprint review
- Retrospectiva sprint-ului
- Rafinare product backlog

Artefacte

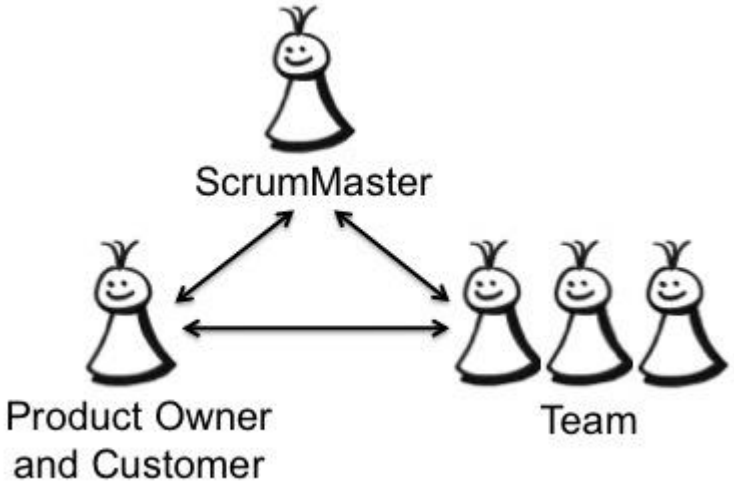
- Product backlog
- Scrum backlog
- Burndown charts
- Versiuni executabile ale produsului

Roluri - Product owner

- Este membrul echipei Scrum care are sarcina de a maximiza valoarea muncii echipei:
  - este responsabil pentru profitabilitatea produsului (ROI – Return On Investment)
- Detine viziunea asupra produsului, colaboreaza strans cu partile interesate in proiect (utilizatori, clienti), cultiva şi alimenteaza o comunitate in jurul produsului.
- Faciliteaza comunicarea dintre echipă şi partile interesate, asigura construirea produsului potrivit (asteptat de clienti, utilizatori).
- Decide continutul listei de cerinte (product backlog) şi ordoneaza articolele sale după prioritate astfel incat sa se obtina valoarea maxima.
- Decide când va fi lansat produsul și ce funcționalitați va avea.
- Modifică lista de cerințe înaintea fiecarei iterații, după cum e necesar.
- Acceptă sau respinge ceea ce s-a produs.
- Decide care articole din lista de cerințe vor trebui considerate pentru sprintul curent.

Roluri - ScrumMaster

- Antreneaza echipa: ajuta intreaga echipa sa obtina performanțe mai bune, sa aplice corect metodologia Scrum, sa aplice practici tehnice care o ajuta sa finalizeze sarcinile fiecarui sprint.
- Responsabil pentru aplicarea valorilor și practicilor din Scrum.
- Incurajeaza auto-organizarea echipei.
- Îndepărtează impedimentele.
- Se asigură că echipa este complet funcțională și productivă.
- Încurajează cooperarea strânsă între toate rolurile.
- Protejează echipa de interferențele externe.



Roluri – Echipa

- De obicei 5-9 persoane.
- **Multi-funcțională**:
  - Nu include roluri traditionale ca programator, tester, arhitect
  - Toti din echipa lucreaza împreuna pentru realizarea sarcinilor sprint-ului, pe care si le-au asumat in mod colectiv
- Membrii trebuie sa fie disponibili echipei si proiectului tot timpul.
- **Se auto-organizează**:
  - Product owner creaza o lista ordonata cu ceea ce este necesar sa fie facut
  - Membrii echipei evalueaza cât de mult se poate face într-un singur sprint şi se auto-organizeaza, hotărând ei singuri cine ce face astfel încât la sfarsitul sprintului să se obtina un nou increment al produsului.
- Membrii echipei ar trebui schimbați doar între sprinturi.

Artefacte - Product backlog

- O lista ordonata de idei si cerințe pentru produs, care pot proveni de la product owner, membrii echipei sau partile interesate.
- Fiecare articol din lista este insotit de o estimare a efortului de implementare.
- Lista este ordonata (prioritizata) de către product owner, astfel incat valoarea muncii echipei sa fie maximizata.
- Lista este reordonata la începutul fiecărui sprint.
- De obicei, la inceputul proiectului lista este scurta şi vagă, devine apoi din ce in ce mai lunga şi mai bine definita.
- Product owner este responsabil pentru crearea şi rafinarea listei dar echipa îl ajuta în producerea si actualizarea listei.

Un exemplu de product backlog

Articol in product backlog	Estimare
Permite unui oaspete să facă o rezervare	3
În calitate de oaspete, vreau să anulez o rezervare.	5
În calitate de oaspete, vreau să schimb datele rezervării.	3
Ca angajat al hotelului, pot să execut rapoartele RevPAR (revenue-per-available-room)	8
Îmbunătățește gestionarea excepțiilor	8
...	30

Sursa: [1]

Planificarea sprint-ului

- Are loc in cadrul unei intalniri cu durata limitata la care participa intreaga echipa.
- Product owner:
  - prezinta ce anume trebuie facut, utilizand articole din product backlog
  - raspunde la întrebarile echipei pentru a lamuri neînțelegerile asupra articolelor din product backlog
- Echipa:
  - decide cate articole sa ia din product backlog şi cum sa le realizeze
  - identifica taskurile şi estimeaza durata pentru fiecare (1-16 ore)
  - creaza sprint backlog-ul
- Durata recomandata pentru o intalnire de planificare sprint este de 2 ore (sau mai putin) x numarul de saptamani ale sprint-ului.
- Succesul intalnirii depinde de calitatea product backlog-ului, de aceea este importanta rafinarea sa.
- Adesea se defineste un scop al sprintului, care ajuta echipa sa se focalizeze pe realizarea sa.
- Se discuta design-ul de nivel înalt.

Artefacte - sprint backlog

- Este lista articolelor din product backlog detaliate, alese pentru implementare in sprintul curent, impreuna cu taskurile planuite de echipa pentru realizarea lor.
- Odata stabilit sprint backlog, echipa incepe sa lucreze la noul increment al produsului.

Exemplu de sprint backlog

Task -uri	L	Ma	Mi	J	V
Impl. interf. cu utilizatorul	8	4	8		
Impl. middle tier-ul	16	12	10	4	
Testează middle tier-ul	8	16	16	11	8
Scrie help-ul online	12				
Impl. clasa foo	8	8	8	8	8
Adaugă logarea erorilor			8	4	

sursa:[1]

Meeting-ul SCRUM zilnic

- Are loc la aceeasi ora si in acelasi loc.
- Dureaza maxim 15 minute. Se stă în picioare.
- Nu se rezolvă probleme
  - Este invitata toată lumea
  - Doar membrii echipei, ScrumMaster-ul si Product owner-ul pot vorbi
- Fiecare membru al echipei raspunde la 3 intrebari:
  - Ce ai facut ieri?
  - Ce vei face azi?
  - Exista ceva care te împiedică?
- Acestea NU reprezinta un raport pentru ScrumMaster, sunt angajamente in faţa unor egali.
- In functie de ceea ce s-a discutat la intalnirea zilnica echipa se poate reorganiza, pentru indeplinirea scopului sprintului si finalizarea incrementului.

Sprint review

- Intalnire la finalul sprintului, cu durata limitata:
  recomandabil, o ora x nr. de spatamani ale sprintului.
- Informal
  - De regulă, 2 ore pentru pregătire
  - Fără slide-uri
- Participanti: product owner, echipa, ScrumMaster, conducerea companiei, clienti, dezvoltatori din alte proiecte.
- Echipa prezintă ce a realizat în timpul sprintului
- De obicei are forma unei demonstrații a noilor funcționalități
- Se discuta ce au observat membrii echipei in timpul sprintului si eventual idei noi
- Se actualizeaza product backlog cu cerintele ramase

Retrospectiva sprint-ului

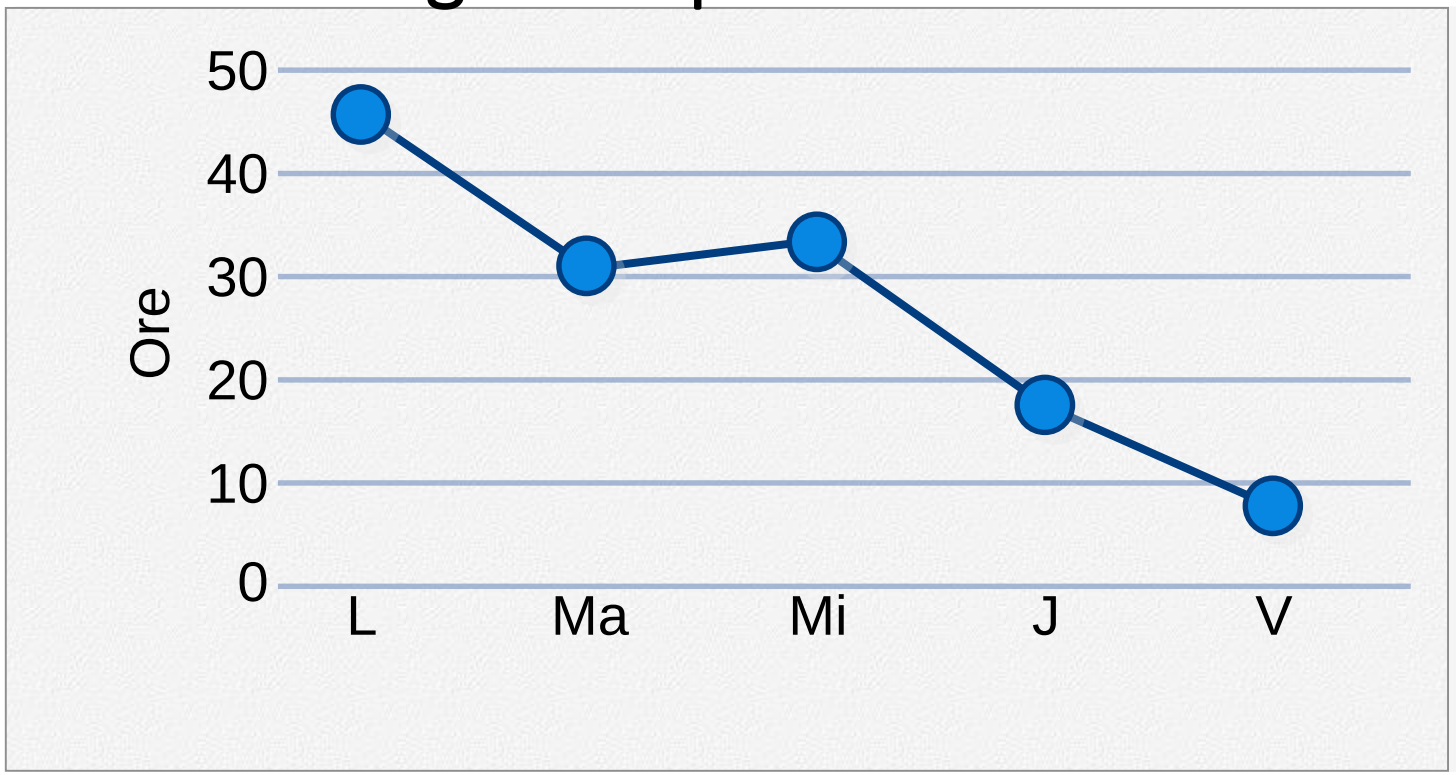
- Intalnire scurta (maxim 3 ore), dupa Sprint review.
- Se discuta despre “ce merge și ce nu merge”.
- Se identifica potentiale imbunatatiri si se planifica imbunatatirea procesului.
- Participa:
  - toti membrii echipei
  - eventual clienți și alte persoane
- Scrum master trebuie sa asigure ca retrospectiva are loc si participanții înțeleg scopul ei.

Gestionarea sprint backlog-ului

- Fiecare persoană își alege ce va lucra după propria dorință.
  - Taskurile nu sunt niciodată asignate de altcineva
- Estimările din sprint backlog sunt actualizate zilnic.
- Orice membru al echipei poate sa adauge, sa șteargă sau sa modifice articole din sprint backlog.
- Taskurile din cadrul sprint-ului sunt descoperite în mod natural.
- Durata lor este ajustata pe parcursul sprintului.
- Se actualizează volumul de muncă rămas pe măsură ce se obțin mai multe informații.

Task-uri	L	Ma	Mi	J	V
Impl. interf. cu utilizatorul	8	4	8		
Impl. middle tier-ul	16	12	10	7	
Testează middle tier-ul	8	16	16	11	8
Scrie help-ul online	12				

Diagrama sprint burndown



Sursa: [1]

Rafinarea product backlog-ului

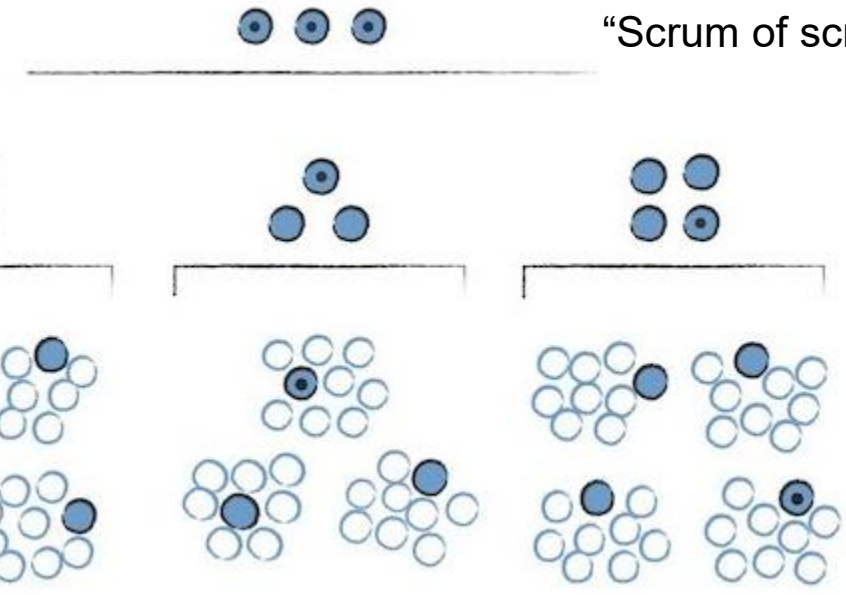
- Activitate care se desfasoara pe toata durata unui proiect Scrum.
- Este responsabilitatea intregii echipe.
- Consta din:

- Adaugarea de articole noi sau care au devenit mai importante
- Eliminarea articolelor care nu mai sunt importante
- Ordonarea articolelor
- Impartirea articolelor mari in articole mai mici, astfel incat sa poata fi implementate pe durata unui singur sprint
- Gruparea mai multor articole intr-unul singur
- Identificarea articolelor care ar trebui sa fie selectate in urmatorul sprint

Scalabilitate

- Echipa Scrum tipică e formată din 7 ± 2 persoane
  - Scalabilitatea se obține prin echipe formate la rândul lor din echipe: **scrums of scrums**
- Factori care pot influența scalarea
  - Tipul aplicației
  - Dimensiunea echipei
  - Împrăștierea echipei
  - Durata proiectului
- Scrum-ul a fost folosit in mai multe proiecte la care au participat peste 500 persoane
- **Meeting-ul "Scrum of scrums"**
  - Are rolul de a coordona activitatea unui grup de echipe
  - Fiecare echipa din grup, la finalul intalnirii zilnice, desemneaza o persoana care va participa la meeting-ul "Scrum of scrums".
  - Meeting-ul "Scrum of scrums" este analog meeting-ului Scrum zilnic, dar nu are loc in mod necesar in fiecare zi; in multe organizatii are loc de 2-3 ori pe saptamana.

Scrums of scrums



Sursa: [3]

- Meeting-ul “Scrum of scrums” se desfasoara la fel ca o intalnire Scrum zilnica, fiecare participant raportand ce s-a facut, ce se va face si impedimentele, in numele echipei pe care o reprezinta.
- Impedimentele sunt focalizate pe problemele de coordonare intre echipe.
- Rezolvarea impedimentelor poate consta in: acordul asupra interfețelor dintre echipe, negocierea asupra responsabilitatilor, etc.

Metodologia Scrum a fost folosita de:

- Microsoft
- Yahoo
- Google
- Electronic Arts
- High Moon Studios
- Lockheed Martin
- Philips
- Siemens
- Nokia
- Capital One
- BBC
- Intuit
- Intuit
- Nielsen Media
- First American Real Estate
- BMC Software
- Ipswitch
- John Deere
- Lexis Nexis
- Sabre
- Salesforce.com
- Time Warner
- Turner Broadcasting
- Oce

Sursa: [1]

Scrum a fost folosita pentru:

- Software comercial
- Aplicații in-house
- Aplicații la comandă
- Proiecte cu preț fix
- Aplicații financiare
- Aplicații certificate ISO 9001
- Sisteme încorporate (embedded)
- Sisteme cu cerințe de disponibilitate 99.999%, 24x7
- Programul Joint Strike Fighter

- Dezvoltarea de jocuri video
- Sisteme aprobate de FDA, life-critical
- Software de control al sateliților
- Site-uri web
- Software pentru dispozitive mobile
- Aplicații pentru rețele de calculatoare
- Unele din cele mai mari aplicații in uz

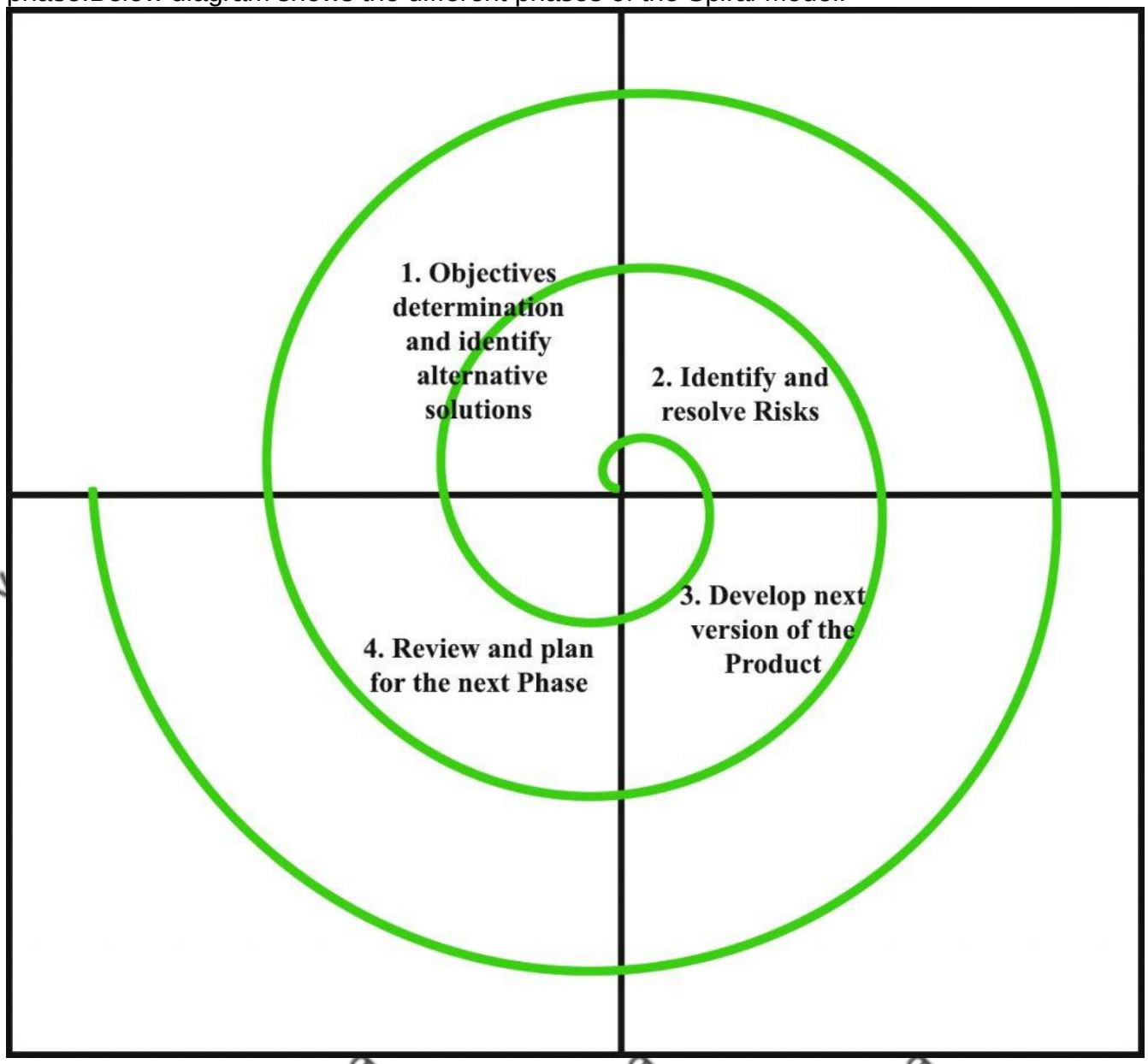
Sursa: [1]

Mai multe informații

1. “O introducere in Scrum”, autor Mike Cohn: www.mountaingoatsoftware.com/agile/scrum/a-reusable-scrum-presentation
2. www.mountaingoatsoftware.com/scrum
3. https://www.mountaingoatsoftware.com/agile/scrum/team
3. www.scrumalliance.org
4. www.controlchaos.com
5. Craig Larman, *Agile and Iterative Development: A Manager's Guide,* 2004, Pearson Eucation.
6. Mike Cohn, *Agile Estimating and Planning*, 2006, Prentice Hall.
7. Ken Schwaber, *Agile Project Management with Scrum,* 2004, Microsoft Press.

Software Engineering | Spiral Model

Spiral model is one of the most important Software Development Life Cycle models, which provides support for **Risk Handling**. In its diagrammatic representation, it looks like a spiral with many loops. The exact number of loops of the spiral is unknown and can vary from project to project. **Each loop of the spiral is called a Phase of the software development process.** The exact number of phases needed to develop the product can be varied by the project manager depending upon the project risks. As the project manager dynamically determines the number of phases, so the project manager has an important role to develop a product using spiral model.
The Radius of the spiral at any point represents the expenses(cost) of the project so far, and the angular dimension represents the progress made so far in the current phase.Below diagram shows the different phases of the Spiral Model:



Each phase of Spiral Model is divided into four quadrants as shown in the above figure. The functions of these four quadrants are discussed below-

1. **Objectives determination and identify alternative solutions:** Requirements are gathered from the customers and the objectives are identified, elaborated and analyzed at the start of every phase. Then alternative solutions possible for the phase are proposed in this quadrant.
2. **Identify and resolve Risks:** During the second quadrant all the possible solutions are evaluated to select the best possible solution. Then the risks associated with that solution is identified and the risks are resolved using the best possible strategy. At the end of this quadrant, Prototype is built for the best possible solution.
3. **Develop next version of the Product:** During the third quadrant, the identified features are developed and verified through testing. At the end of the third quadrant, the next version of the software is available.
4. **Review and plan for the next Phase:** In the fourth quadrant, the Customers evaluate the so far developed version of the software. In the end, planning for the next phase is started.

Risk Handling in Spiral Model

A risk is any adverse situation that might affect the successful completion of a software project. The most important feature of the spiral model is handling these unknown risks after the project has started. Such risk resolutions are easier done by developing a prototype. The spiral model supports coping up with risks by providing the scope to build a prototype at every phase of the software development.

Prototyping Model also support risk handling, but the risks must be identified completely before the start of the development work of the project. But in real life project risk may occur after the development work starts, in that case, we cannot use Prototyping Model. In each phase of the Spiral Model, the features of the product dated and analyzed and the risks at that point of time are identified and are resolved through prototyping. Thus, this model is much more flexible compared to other SDLC models.

Why Spiral Model is called Meta Model ?

The Spiral model is called as a Meta Model because it subsumes all the other SDLC models. For example, a single loop spiral actually represents the Iterative Waterfall Model. The spiral model incorporates the stepwise approach of the Classical Waterfall Model. The spiral model uses the approach of **Prototyping Model** by building a prototype at the start of each phase as a risk handling technique. Also, the spiral model can be considered as supporting the evolutionary model – the iterations along the spiral can be considered as evolutionary levels through which the complete system is built.

Advantages of Spiral Model: Below are some of the advantages of the Spiral Model.

- **Risk Handling:** The projects with many unknown risks that occur as the development proceeds, in that case, Spiral Model is the best development model to follow due to the risk analysis and risk handling at every phase.
- **Good for large projects:** It is recommended to use the Spiral Model in large and complex projects.

va urez restanta usoara zgarcitilor

- **Flexibility in Requirements:** Change requests in the Requirements at later phase can be incorporated accurately by using this model.
- **Customer Satisfaction:** Customer can see the development of the product at the early phase of the software development and thus, they habituated with the system by using it before completion of the total product.

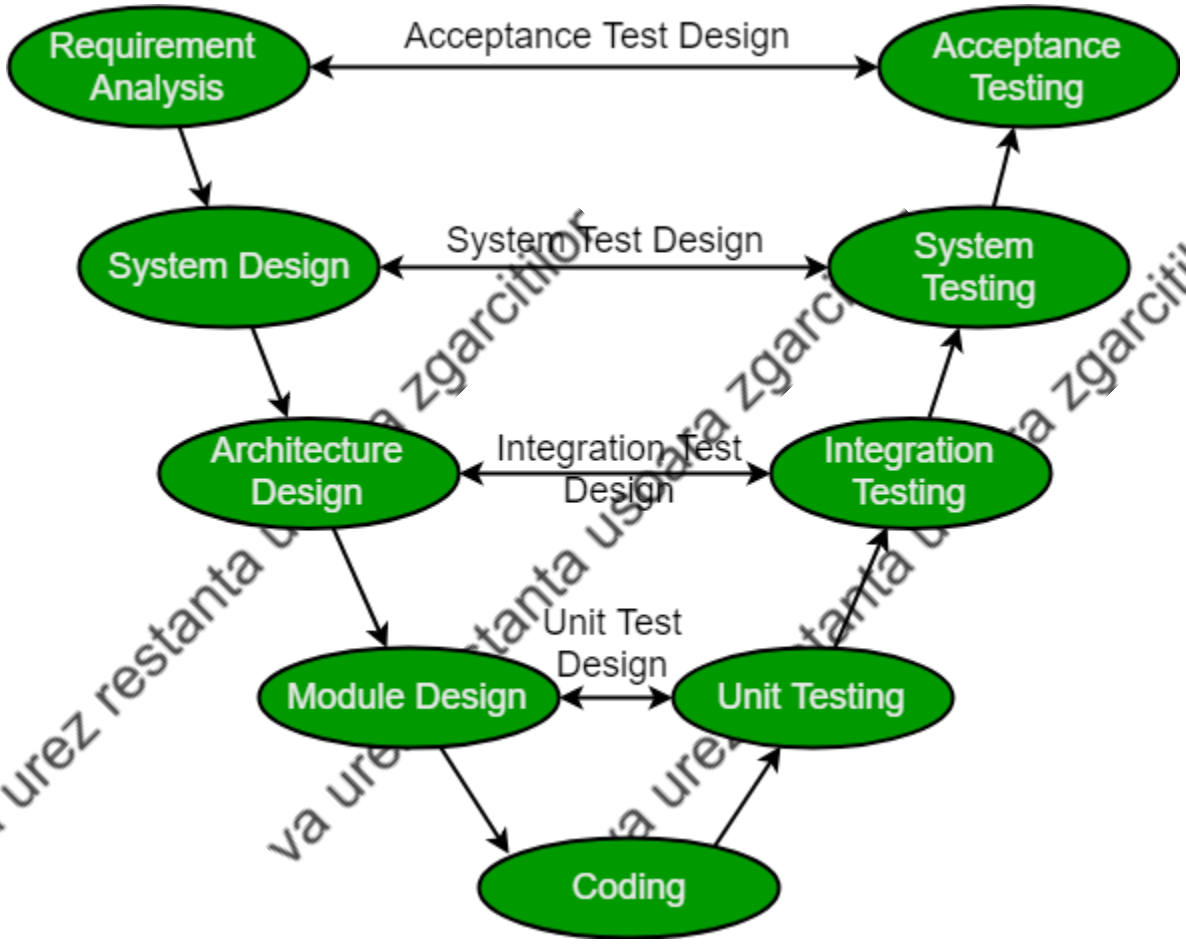
Disdvantages of Spiral Model: Below are some of the main disadvantages of the spiral model.

- **Complex:** The Spiral Model is much more complex than other SDLC models.
- **Expensive:** Spiral Model is not suitable for small projects as it is expensive.
- **Too much dependable on Risk Analysis:** The successful completion of the project is very much dependent on Risk Analysis. Without very highly experienced expertise, it is going to be a failure to develop a project using this model.
- **Difficulty in time management:** As the number of phases is unknown at the start of the project, so time estimation is very difficult.

va urez restanta usoara zgarcitilor

Software Engineering | SDLC V-Model

The V-model is a type of SDLC model where process executes in a sequential manner in V-shape. It is also known as Verification and Validation model. It is based on the association of a testing phase for each corresponding development stage. Development of each step directly associated with the testing phase. The next phase starts only after completion of the previous phase i.e. for each development activity, there is a testing activity corresponding to it.



Verification: It involves static analysis technique (review) done without executing code. It is the process of evaluation of the product development phase to find whether specified requirements meet.

Validation: It involves dynamic analysis technique (functional, non-functional), testing done by executing code. Validation is the process to evaluate the software after the completion of the development phase to determine whether software meets the customer expectations and requirements.

va urez restanta usoara zgarcitilor

So V-Model contains Verification phases on one side of the Validation phases on the other side. Verification and Validation phases are joined by coding phase in V-shape. Thus it is called V-Model.

Design Phase:

- **Requirement Analysis:** This phase contains detailed communication with the customer to understand their requirements and expectations. This stage is known as Requirement Gathering.
- **System Design:** This phase contains the system design and the complete hardware and communication setup for developing product.
- **Architectural Design:** System design is broken down further into modules taking up different functionalities. The data transfer and communication between the internal modules and with the outside world (other systems) is clearly understood.
- **Module Design:** In this phase the system breaks dowm into small modules. The detailed design of modules is specified, also known as Low-Level Design (LLD).

Testing Phases:

- **Unit Testing:** Unit Test Plans are developed during module design phase. These Unit Test Plans are executed to eliminate bugs at code or unit level.
- **Integration testing:** After completion of unit testing Integration testing is performed. In integration testing, the modules are integrated and the system is tested. Integration testing is performed on the Architecture design phase. This test verifies the communication of modules among themselves.
- **System Testing:** System testing test the complete application with its functionality, inter dependency, and communication.It tests the functional and non-functional requirements of the developed application.
- **User Acceptance Testing (UAT):** UAT is performed in a user environment that resembles the production environment. UAT verifies that the delivered system meets user's requirement and system is ready for use in real world.

Industrial Challange: As the industry has evolved, the technologies have become more complex, increasingly faster, and forever changing, however, there remains a set of basic principles and concepts that are as applicable today as when IT was in its infancy.

- Accurately define and refine user requirements.
- Design and build an application according to the authorized user requirements.
- Validate that the application they had built adhered to the authorized business requirements.

Principles of V-Model:

- **Large to Small:** In V-Model, testing is done in a hierarchical perspective, For example, requirements identified by the project team, create High-Level Design, and Detailed Design phases of the project. As each of these phases is completed the requirements, they are defining become more and more refined and detailed.
- **Data/Process Integrity:** This principle states that the successful design of any project requires the incorporation and cohesion of both data and processes. Process elements must be identified at each and every requirements.
- **Scalability:** This principle states that the V-Model concept has the flexibility to accommodate any IT project irrespective of its size, complexity or duration.
- **Cross Referencing:** Direct correlation between requirements and corresponding testing activity is known as cross-referencing.

va urez restanta usoara zgarcitilor

- **Tangible Documentation:** This principle states that every project needs to create a document. This documentation is required and applied by both the project development team and the support team. Documentation is used to maintaining the application once it is available in a production environment.

Why preferred?

- It is easy to manage due to the rigidity of the model. Each phase of V-Model has specific deliverables and a review process.
- Proactive defect tracking – that is defects are found at early stage.

When to use?

- Where requirements are clearly defined and fixed.
- The V-Model is used when ample technical resources are available with technical expertise.

Advantages:

- This is a highly disciplined model and Phases are completed one at a time.
- V-Model is used for small projects where project requirements are clear.
- Simple and easy to understand and use.
- This model focuses on verification and validation activities early in the life cycle thereby enhancing the probability of building an error-free and good quality product.
- It enables project management to track progress accurately.

Disadvantages:

- High risk and uncertainty.
- It is not a good for complex and object-oriented projects.
- It is not suitable for projects where requirements are not clear and contains high risk of changing.
- This model does not support iteration of phases.
- It does not easily handle concurrent events.

va urez restanta usoara zgarcitilor

What is Prototype model- advantages, disadvantages and when to use it?

The basic idea in **Prototype model** is that instead of freezing the requirements before a design or coding can proceed, a throwaway prototype is built to understand the requirements. This prototype is developed based on the currently known requirements. Prototype model is a **software development model**. By using this prototype, the client can get an "actual feel" of the system, since the interactions with prototype can enable the client to better understand the requirements of the desired system. Prototyping is an attractive idea for complicated and large systems for which there is no manual process or existing system to help determining the requirements.

When to use Prototype model:

- Prototype model should be used when the desired system needs to have a lot of interaction with the end users.
- Typically, online systems, web interfaces have a very high amount of interaction with end users, are best suited for Prototype model. It might take a while for a system to be built that allows ease of use and needs minimal training for the end user.
- Prototyping ensures that the end users constantly work with the system and provide a feedback which is incorporated in the prototype to result in a useable system. They are excellent for designing good human computer interface systems

Advantages of Prototype model:

1) Clients are actively involved in the development.

2) Since in this methodology a working model of the system is provided, the users get a better understanding of the system being developed.

3) Errors can be detected much earlier.

4) Quicker user feedback is available leading to better solutions.

5) Missing functionality can be identified easily.

6) Confusing or difficult functions can be identified.

Disadvantages of Prototype model:

1) Leads to implementing and then repairing way of building systems.

2) Practically, this methodology may increase the complexity of the system as scope of the system may expand beyond original plans.

3) Incomplete application may cause application not to be used as the full system was designed.

4) Incomplete or inadequate problem analysis.

va urez restanta usoara zgarcitilor

Scrum

Here Is an Overview of the Scrum Framework

- The product owner creates a product backlog (essentially, a wishlist of tasks that need to be prioritized in a project)
- The Scrum team conducts a sprint planning session where the tasks necessary to complete items on the wishlist is broken down into small, more easily manageable chunks
- The team creates a sprint backlog and plans its implementation
- The team decides a time duration for every sprint (the most common intervals is probably two weeks)
- The team gets together every day for a brief Scrum meeting (often referred to as a Daily Standup) where each member of the team shares daily updates, helping the team and the project manager assess the progress of the project
- The certified Scrum master guides the team and keeps them focused and motivated
- The stakeholders and the product owner conduct a review at the end of each sprint

This is the cycle followed by a Scrum team in a product development project. The three roles mentioned above - the Product Owner, the Scrum Team, and the ScrumMaster together play a major role in exercising this framework.

Pros and Cons of Scrum Project Management

Pros of Scrum

Here's why the framework is so popular today:

Scrum can help teams complete project deliverables quickly and efficiently.

- Scrum can help teams complete project deliverables quickly and efficiently
- Scrum ensures effective use of time and money
- Large projects are divided into easily manageable sprints
- Developments are coded and tested during the sprint review
- Works well for fast-moving development projects
- The team gets clear visibility through scrum meetings
- Scrum, being agile, adopts feedback from customers and stakeholders
- Short sprints enable changes based on feedback a lot more easily
- The individual effort of each team member is visible during daily scrum meetings

Cons of Scrum

But like every framework, scrum also has few disadvantages.

Nothing is perfect, and the Scrum methodology is no exception. In some cases, Scrum is combined with other project management techniques that can help resolve some of these drawbacks:

- Scrum often leads to scope creep, due to the lack of a definite end-date
- The chances of project failure are high if individuals aren't very committed or cooperative
- Adopting the Scrum framework in large teams is challenging
- The framework can be successful only with experienced team members
- Daily meetings sometimes frustrate team members
- If any team member leaves in the middle of a project, it can have a huge negative impact on the project
- Quality is hard to implement until the team goes through an aggressive testing process

Conclusion

Needless to say, proper planning and smart decision making can help you get past these disadvantages with the Scrum methodology. For example, in larger teams, each member needs to have defined roles and responsibilities with definite goals, so that there is no compromise on quality and no excuse for failure. This will keep the team focused to project goals. Plus, the ScrumMaster needs to guide the team effectively to avoid pitfalls and ensure 100% project success.

va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor

Waterfall vs. Agile: Which is the Right Development Methodology for Your Project?

One of the first decisions we face for each of our project implementations at Segue is “Which development methodology should we use?” This is a topic that gets a lot of discussion (and often heated debate). If this is not something you’ve worked with before, a definition of development methodology is in order; put very simply, it’s a way of organizing the work of software development. This is **NOT** about a style of project management or a specific technical approach, although you will often hear these terms all thrown together or used interchangeably.

The two basic, most popular methodologies are:

1. *Waterfall:* (ugh, terrible name!), which might be more properly called the “traditional” approach, and
2. *Agile:* a specific type of Rapid Application Development and newer than Waterfall, but not that new, which is often implemented using Scrum.

Both of these are usable, mature methodologies. Having been involved in software development projects for a long time, here are my thoughts on the strengths and weaknesses of each.

The Waterfall Methodology

Waterfall is a linear approach to software development. In this methodology, the sequence of events is something like:

1. Gather and document requirements
2. Design
3. Code and unit test
4. Perform system testing
5. Perform user acceptance testing (UAT)
6. Fix any issues
7. Deliver the finished product

In a **true** Waterfall development project, each of these represents a distinct stage of software development, and each stage generally finishes before the next one can begin. There is also typically a stage gate between each; for example, requirements must be reviewed and approved by the customer before design can begin.

There are good things and bad about the Waterfall approach. On the **positive** side:

- Developers and customers agree on what will be delivered early in the development lifecycle. This makes planning and designing more straightforward.
- Progress is more easily measured, as the full scope of the work is known in advance.
- Throughout the development effort, it's possible for various members of the team to be involved or to continue with other work, depending on the active phase of the project. For example, business analysts can learn about and document what needs to be done, while the developers are working on other projects. Testers can prepare test scripts from requirements documentation while coding is underway.
- Except for reviews, approvals, status meetings, etc., a customer presence is not strictly required after the requirements phase.
- Because design is completed early in the development lifecycle, this approach lends itself to projects where multiple software components must be designed (sometimes in parallel) for integration with external systems.
- Finally, the software can be designed completely and more carefully, based upon a more complete understanding of **all** software deliverables. This provides a better software design with less likelihood of the "piecemeal effect," a development phenomenon that can occur as pieces of code are defined and subsequently added to an application where they may or may not fit well.

Here are some **issues** we have encountered using a pure Waterfall approach:

- One area which almost always falls short is the effectiveness of requirements. Gathering and documenting requirements in a way that is meaningful to a customer is often the most difficult part of software development, in my opinion. Customers are sometimes intimidated by details, and specific details, provided early in the project, are required with this approach. In addition, customers are not always able to visualize an application from a requirements document. Wireframes and mockups can help, but there's no question that most end users have some difficulty putting these elements together with written requirements to arrive at a good picture of what they will be getting.
- Another potential drawback of pure Waterfall development is the possibility that the customer will be dissatisfied with their delivered software product. As all deliverables are based upon documented requirements, a customer may not see what will be delivered until it's almost finished. By that time, changes can be difficult (and costly) to implement.

nta usoara zgarcitilor

The Agile Methodology

Agile is an iterative, team-based approach to development. This approach emphasizes the rapid delivery of an application in complete functional components. Rather than creating tasks and schedules, all time is “time-boxed” into phases called “sprints.” Each sprint has a defined duration (usually in weeks) with a running list of deliverables, planned at the start of the sprint. Deliverables are prioritized by business value as determined by the customer. If all planned work for the sprint cannot be completed, work is reprioritized and the information is used for future sprint planning.

As work is completed, it can be reviewed and evaluated by the project team and customer, through daily builds and end-of-sprint demos. Agile relies on a very high level of customer involvement throughout the project, but especially during these reviews.

Some **advantages of the Agile approach** are easy to see:

- The customer has frequent and early opportunities to see the work being delivered, and to make decisions and changes throughout the development project.
- The customer gains a strong sense of ownership by working extensively and directly with the project team throughout the project.
- If time to market for a specific application is a greater concern than releasing a full feature set at initial launch, Agile can more quickly produce a basic version of working software which can be built upon in successive iterations.
- Development is often more user-focused, likely a result of more and frequent direction from the customer.
- For more Agile Development benefits, please see 8 Benefits of Agile Software Development

And, of course, there are **some disadvantages**:

- The very high degree of customer involvement, while great for the project, may present problems for some customers who simply may not have the time or interest for this type of participation.
- Agile works best when members of the development team are completely dedicated to the project.
- Because Agile focuses on time-boxed delivery and frequent reprioritization, it’s possible that some items set for delivery will not be completed within the allotted timeframe. Additional sprints (beyond those initially planned) may be needed, adding to the project cost. In addition, customer involvement often leads to additional features requested throughout the project. Again, this can add to the overall time and cost of the implementation.
- The close working relationships in an Agile project are easiest to manage when the team members are located in the same physical space, which is not always possible. However, there are a variety of ways to handle this issue, such as webcams, collaboration tools, etc.

- The iterative nature of Agile development may lead to a frequent refactoring if the full scope of the system is not considered in the intial architecture and design. Without this refactoring, the system can suffer from a reduction in overall quality. This becomes more pronounced in larger-scale implementations, or with systems that include a high level of integration.

Making the Choice Between Agile and Waterfall

So, how do we choose? First, we change the game a little (which is what most software development organizations do) by defining our own process. At Segue, it's called our ***Process Framework***, and it's a variation on the traditional Waterfall methodology. Our modifications include use of prototyping where possible to provide the customer a better view of their finished product early in the design/development cycle. This helps to improve the team's understanding of requirements and communication with the customer. After the primary framework of the application is completed per high level requirements, we continue to develop and also to reach out to the customer for refinement of requirements. In this way, we strive to be as iterative as possible without compromising our overall system architecture.

We consider the following factors when considering which methodology to use:

va urez restanta usoara zgarcitilor

ALIGNING PROJECT TRAITS *with* DEVELOPMENT METHODOLOGIES

PROJECT TRAIT/FACTOR	AGILE	PLAN - DRIVEN (WATERFALL)	COMMENTS
CUSTOMER AVAILABILITY	Prefers customer available throughout project.	Requires customer involvement only at milestones.	Customer involvement reduces risk in either model.
SCOPE/ FEATURES	Welcomes changes, but changes come at the expensive of Cost, Schedule, or other Features. Works well when scope is not known in advance.	Works well when scope is known in advance, or when contract terms limit changes.	Change is a reality so we should prefer adaptability where possible. Contract terms sometimes restrict it.
FEATURE PRIORITIZATION	Prioritization by value ensures the most valuable features are implemented first, thus reducing risk of having an unusable product once funding runs out. Funding efficiency is maximized. Decreases risk of complete failure by allowing "partial" success.	"Do everything we agreed on" approach ensures the customer gets everything they asked for; "all or nothing" approach increases risk of failure.	Contract terms may not permit partial success and may require "do everything".
TEAM	Prefers smaller, dedicated teams with a high degree of coordination and synchronization.	Team coordionation/ synchronization is limited to handoff points	Teams that work together work better, but when contracts are issued to different vendors for different aspects of the project, high degrees of synchronization may not work.
FUNDING	Works extremely well with Time & Materials or other non-fixed funding, may increase stress in fixed-price scenarios.	Reduces risk in Firm Fixed Price contracts by getting agreement up-front.	Fixed price is tough when scope is not known in advance, but many government contracts require it.
SUMMARY	Agile is better, where it is feasible.	Plan-Driven may reduce risk in the face of certain constraints in a contract between a vendor and external customer such as the government.	Through educating our customers about the strengths and weaknesses of each model, we hope to steer them towards a more Agile approach. This may require changes to how our customers, particualarly the government, approach software development projects.

The factors above are not equally weighted; each is assessed depending on the individual project and circumstances.

Although we are starting to see mass adoption of various Agile methodologies in the Enterprise (even DoD and Federal agencies), there are still many organizations that are slow to make the change. It is also very common for organization to transition into more of a hybrid Agile approach that combines aspect of both Agile and Waterfall. The Agile Practice Guide was developed specifically to help organization understand and evaluate the use of Agile and hybrid Agile approaches. The Project Management Institute (PMI) that developed the Project Management Body of Knowledge (PMBOK) Guide collaborated with the Agile Alliance to bundle the two guides in one offering to help organizations, managers and leadership increase agility in the development process.

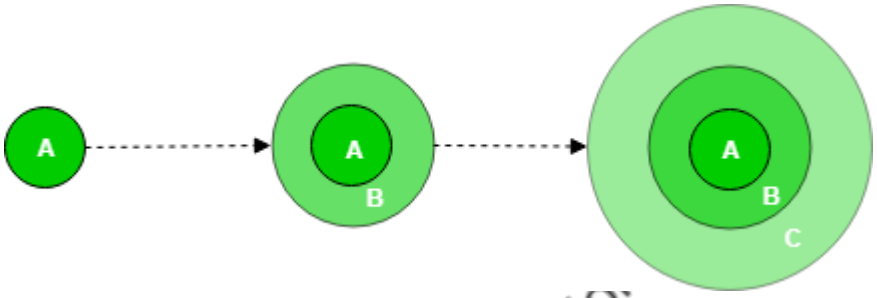
Once we've decided which basic methodology to utilize, we can further refine the process to best fit our project goals. Ultimately, although the way in which we do our work is important, delivering a solid and maintainable product that satisfies our customer is what really counts.

va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor
va urez restanta usoara zgarcitilor

Software Engineering | Incremental process model

Incremental process model is also know as Successive version model.

First, a simple working system implementing only a few basic features is built and then that is delivered to the customer. Then thereafter many successive iterations/ versions are implemented and delivered to the customer until the desired system is released.

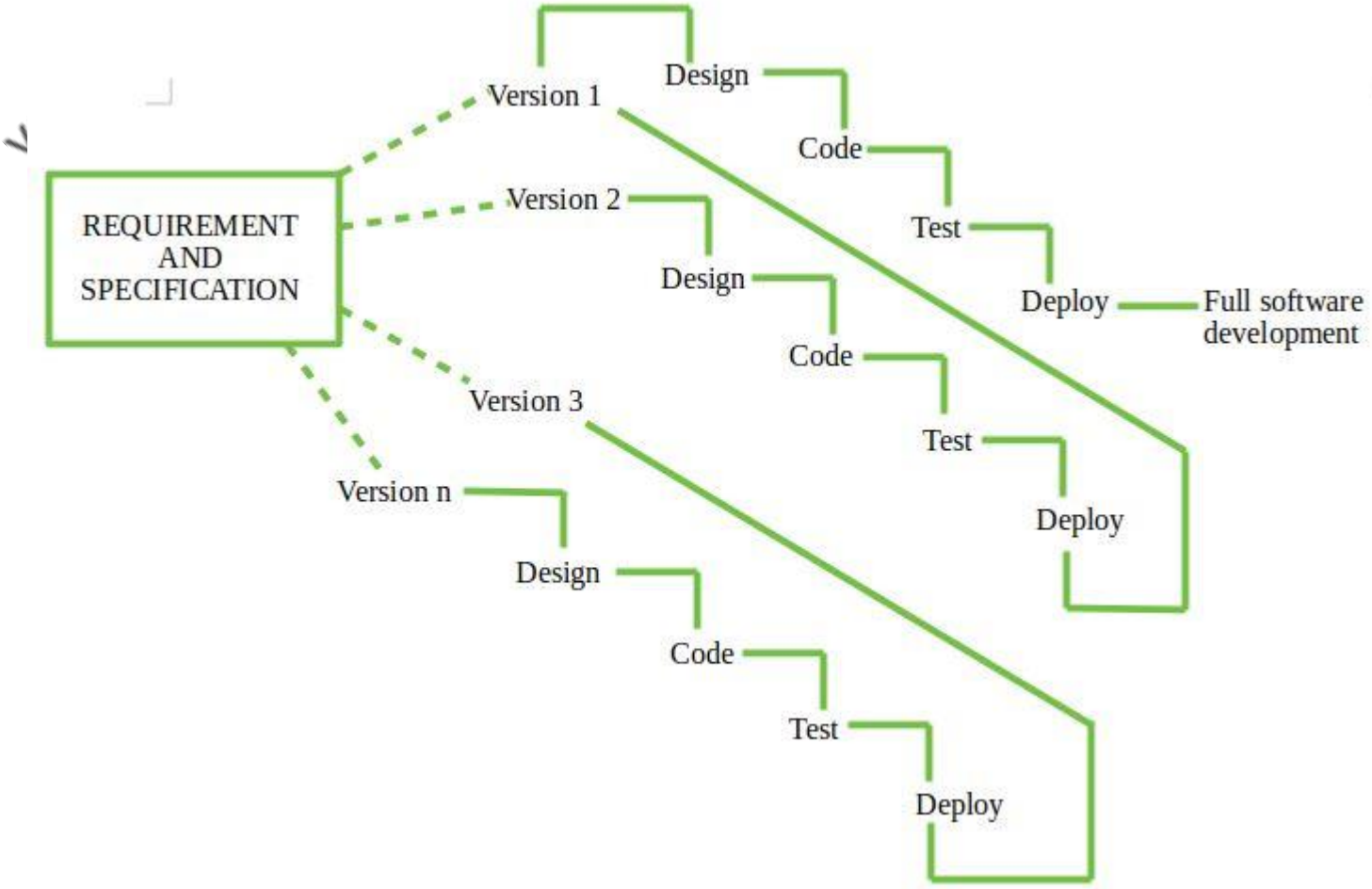


A, B, C are modules of Software Product that are incrementally developed and delivered.

Life cycle activities –
Requirements of Software are first broken down into several modules that can be incrementally constructed and delivered. At any time, the plan is made just for the next increment and not for any kind of long term plans. Therefore, it is easier to modify the version as per the need of the customer. Development Team first undertakes to develop core features (these do not need services from other features) of the system.
Once the core features are fully developed, then these are refined to increase levels of capabilities by adding new functions in Successive versions. Each incremental version is usually developed using an iterative waterfall model of development.

As each successive version of the software is constructed and delivered, now the feedback of the Customer is to be taken and these were then incorporated in the next version. Each version of the software have more additional features over the previous ones.

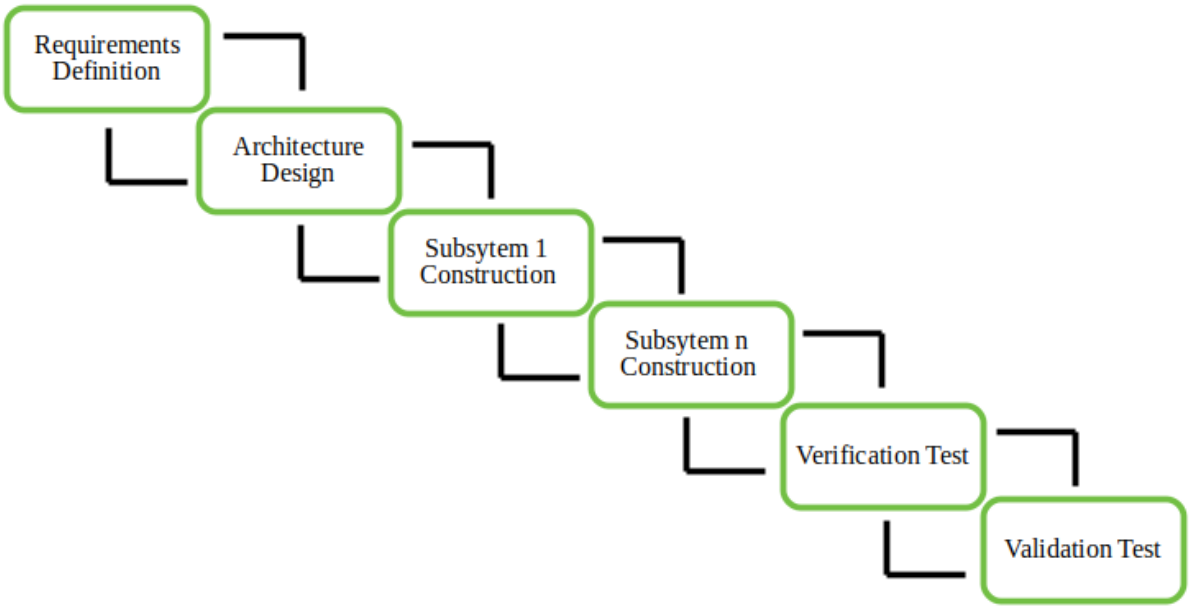
va urez restanta usoara zgarcitilor



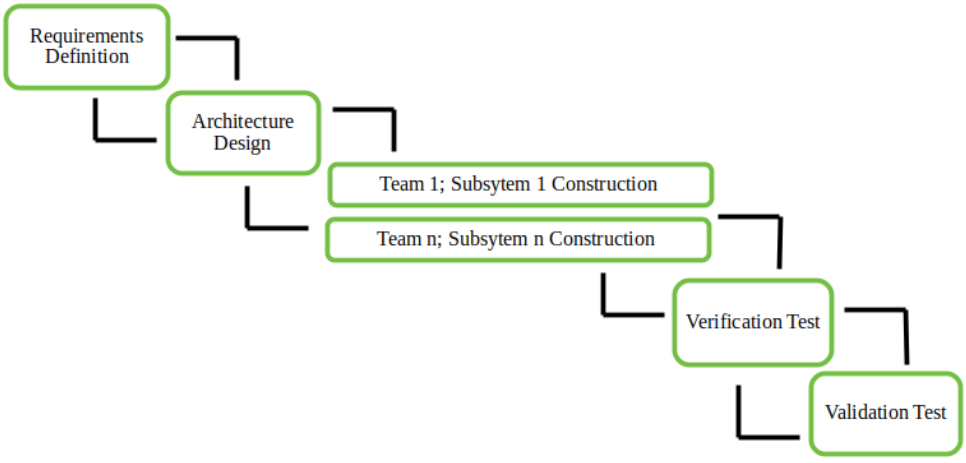
After Requirements gathering and specification, requirements are then spitted into several different versions starting with version-1, in each successive increment, next version is constructed and then deployed at the customer site. After the last version (version n), it is now deployed at the client site.

Types of Incremental model –

1. **Staged Delivery Model –** Construction of only one part of the project at a time.



2. **Parallel Development Model –** Different subsystems are developed at the same time. It can decrease the calendar time needed for the development, i.e. TTM (Time to Market), if enough Resources are available.



When to use this –

1. Funding Schedule, Risk, Program Complexity, or need for early realization of benefits.
2. When Requirements are known up-front.
3. When Projects having lengthy developments schedules.
4. Projects with new Technology.

Advantages –

- Error Reduction (core modules are used by the customer from the beginning of the phase and then these are tested thoroughly)
- Uses divide and conquer for breakdown of tasks.
- Lowers initial delivery cost.
- Incremental Resource Deployment.

Disadvantages –

- Requires good planning and design.
- Total cost is not lower.
- Well defined module interfaces are required.

va urez restanta usoara zgarcitilor

Laborator 1 - Introducere

Înainte de toate

- Prezentare asistent
- Prezentare/parcurgere pagină MPS Need to Know
- Prezentare/parcurgere pagină Reguli generale și notare

Componență echipă

Există mai multe modele de lucru în echipă

- **model descentralizat/anarhic** - nu există o demarcare clară a rolurilor; funcționează în proiectele free/open-source bazate pe comunități cu o motivație internă
- **modele ierarhice** - există responsabilități clare pentru fiecare membru al echipei; în funcție de ierarhie se distribuie responsabilități și se transmit rapoarte
  - în general există un *Project Manager* (**PM**) al cărui rol este de coordonare, distribuție task-uri, motivare, feedback etc.
  - pot exista mai mulți *Lead Developeri* (*Team Leaderi* - **TL**) ale căror responsabilități sunt coordonarea, training-ul, oferirea de sfaturi tehnice, revizia codului și proiectarea unei anumite componente
  - dezvoltatorii (*Developeri* - **D**) participă la implementarea componentelor; în general, fac parte din echipe distincte dedicate unei anumite componente a proiectului
  - testerii (**T**) (pot fi tot dezvoltatori) verifică respectarea cerințelor, funcționarea în regim normal și în regim de stres a aplicației
- **combinaţie** - încearcă rezolvarea problemei: "conducătorii nu stiu implicaţiile deciziilor lor, iar lucrătorii sunt demotivaţi de decizii luate aiurea", dar fără să elimine ierarhia the alternative to hierarchy or anarchy
  - În proiectele mari pot apărea mai multe roluri. Vom avea o abordare mai detaliată în laboratorul 2.

va urez rest... usoara zgarcitilor

Laborator 2 - Rolurile în echipă. Redactarea specificațiilor de proiect.

Rolurile în cadrul echipei de proiect

Cu cât proiectul este mai mare, cu atât rolurile din echipă sunt mai diversificate (mai specializate pe o anume arie de activități).

Într-un ***proiect mic***, principalele roluri din echipă sunt:

1. **Project Manager** / **Manager-ul Proiectului**
   - responsabil de succesul/eșecul proiectului
   - planifică & monitorizează evoluția proiectului
   - ia decizii
   - adesea, are background tehnic (pentru a înțelege ciclul de viață al proiectului)
   - întotdeauna, are abilități soft (de conducere, de motivare și stimulare, de conștientizare a cauzelor, de decizie, de negociere, de previziune, de inovație)
   - certificări PMI (Project Management Institute): PMP (Project Management Professional)
2. **Team Leader** / **Liderul echipei** (Dezvoltator Lider, QA Lider)
   - responsabil de execuția conform standardelor a activităților planificate
   - monitorizează activitatea echipei sale
   - intervine pentru a corecta, pentru a îndruma (coaching/support)
   - raportează managerului statusul real al proiectului
3. **Software Developer** / **Dezvoltator software**
   - responsabil de implementarea software a cerințelor
   - în metode de dezvolare agile contribuie și la design, arhitectură, specificații
4. **Tester** (Inginerul Calității)
   - responsabil de verificarea funcționalităților și a performanțelor produsului
   - scrie scenarii de test, le execută, analizează rezultatele ⇒ rapoarte de testare

Într-un ***proiect mare***, pe lângă rolurile de mai sus, apar și altele, precum:

1. **System Architect** / **Arhitectul Sistemului**
   - responsabil de arhitectura produsului software
   - captează interesele partenerilor de business și le transpune alegând arhitectura potrivită

2. **Technical Writer** / **Scriitorul tehnic**
   - responsabil de documentația tehnică a proiectului
   - primește documente tehnice de la ingineri, pe care le editează spre a fi corecte, clare și conforme cu standardele în vigoare
3. **Analyst** / **Analist** (Analist de Business, Analist de Sisteme Business, Analist de Sistem, Analist de Cerințe)
   - responsabil de înțelegerea corectă a cerințelor de business
   - studiază specificațiile clienților, clarifică toate detaliile proiectului cu clientul și cu partenerii (stakeholders)
   - redactează specificațiile aplicației software
4. **Consultant**
   - analizează fezabilitatea ofertelor de proiecte
   - propune soluții pentru diverse probleme legate de ciclul de viață al proiectelor
   - evaluează gradul de atingere a obiectivelor proiectelor
5. **Experți business**: SME (Subject Matters Expert), Manager-ul Produsului, Manager-ul de program

Gestiunea timpului

Orice membru al echipei și project manager-ul în special trebuie să aibă skill-uri de organizare a timpului pentru a obține maximum de rezultate. Aspecte precum planificarea task-urilor, stabilirea obiectivelor, evitarea întreruperilor, prioritizarea activităților trebuie avute constant în vedere pentru eficientizarea muncii.

Redactarea specificațiilor de proiect

După primirea specificațiilor clientului și după analiza de profunzime a acestora, contractorul redactează *documentele de specificație* ale soluției oferite, de obicei sub formă de:

- **Software Requirements Specification** / **Specificarea Cerințelor Software** (SRS/SCS)
- **Software Design Description** / **Descrierea Design-ului Software** (SDD/DDS)

Este **esențial** pentru reușita proiectului ca aceste documente de specificații să fie complete și absolut clare deoarece:

- acestea reprezintă parte a contractului dintre client și contractor;
- pe baza acestora se pot determina clar cerințele proiectului, rezolvându-se astfel eventualele conflicte dintre parteneri
- pe baza acestora se vor realiza estimările de timp și buget și planificările de proiect;

- pe baza lor se va realiza dezvoltarea soluției și testarea funcționalităților și performanțelor ei;
- pe baza lor se va realiza evaluarea finală a proiectului (a gradului de atingere a obiectivelor).

Documentul SRS cuprinde descrierea completă a comportamentului sistemului software, adică:

- a interacțiunilor dintre utilizator și sistem (**cerințe funcționale**). Acestea se descriu cu ajutorul *use-cases* (cazurilor de utilizare).
- a constrângerilor de proiectare și dezvoltare (**cerințe non-funcționale**). Acestea se referă la restricții de performanță, de securitate, de fiabilitate etc.

Conținutul unui SRS/SCS

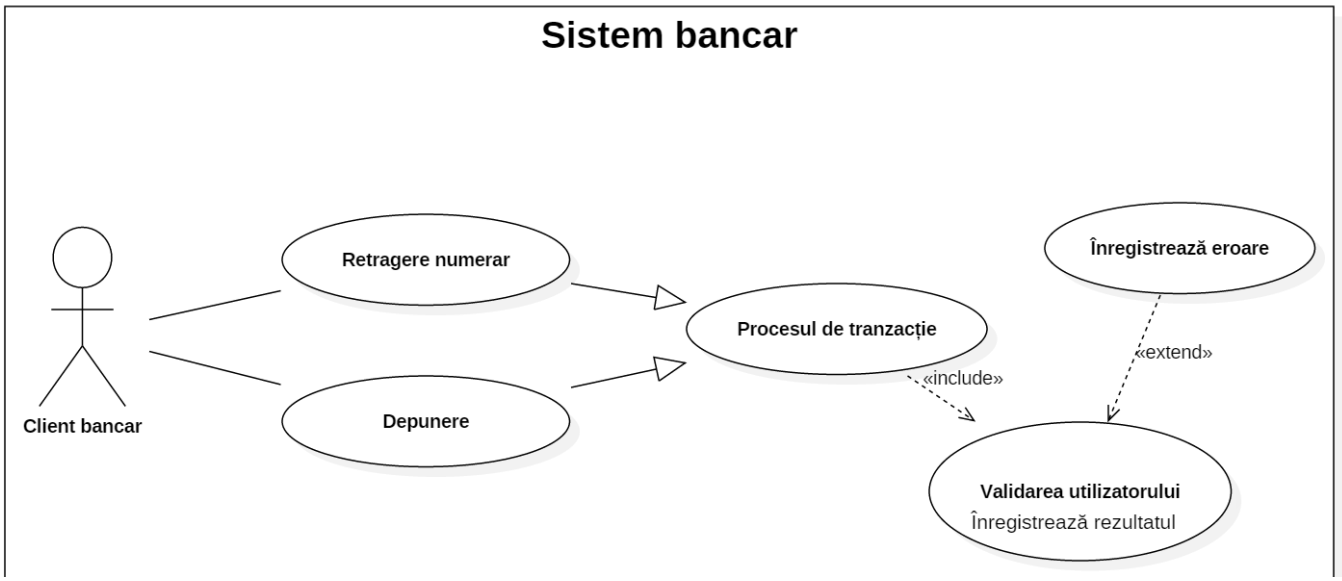
- Scopul documentului (*Document purpose*)
- Conținutul documentului (*Document overview*)
- Descrierea generală a produsului (***General description of the product***)
- Situația curentă (*The current situation*)
- Misiunea proiectului (*Purpose of the product*)
- Contextul proiectului (*Product context*)
- Beneficii (*Benefit*)
- Cerințe funcționale (***Functional requirements***)
- Actori (***Actors***)
- Diagrama de sistem (***System boundary***)
- Descrierea cazurilor de utilizare (***Use cases description***)
- Cerințe nefuncționale (***Non-functional requirements***)
- Cerințe de interfață (*User Interface Requirements*)
- Cerințe de performanță (*Performance Requirements*)
- Cerințe de fiabilitate (*Availability & Reliability*)
- Cerințe de securitate (*Security Requirements*)

System Boundary

Se mai numește și *Use-case diagram / Diagrama de cazuri de utilizare*.
Este:

- o diagramă UML obținută în urma analizei cerințelor utilizatorului
- schema funcționalităților oferite de sistemul software, în termeni de actori, cazuri de utilizare și relații între acestea.

Mai multe detalii, poți afla aici și aici.



Astfel de diagrame se pot realiza în Visio, Visual Paradigm sau Dia.

Exemplu de document SRS

Exemplu de Document SRS - Alumni database
Exemplu de Document SRS - Simulator cursa formula 1

Arhitectura proiectului

Stabilirea arhitecturii unui sistem software implică:

1. *alegerea șabloanelor arhitecturale* potrivite (se pot combina mai multe, se pot personaliza)
2. în funcție de șabloanele arhitecturale alese, *proiectarea structurii* sistemului software la nivel de:
   - **componente** specializate pe o gamă de servicii (module și interfețe),
   - **conexiuni** dintre componente.

Există mai multe tipuri șabloane arhitecturale (**architectural patterns**), precum:

- Tehnică de calcul distributivă
  - sistemul software e alcătuit dintr-o serie de componente software ce rulează pe mașini de calcul diferite, ce comunică prin rețea
  - mașinile de calcul interacționează pentru a oferi cât mai rapid un răspuns corect la cererea utilizatorului
- Client-server
  - arhitectură distribuită

- conține componente furnizoare de servicii = componente *server* și componente ce solicită servicii = componente *client*
- Arhitectură bazată pe evenimente
  - se bazează pe producerea, detectarea, consumarea și reacția la evenimente
- Front-end și back-end
  - front-end = interfață de colectare a datelor de la utilizator și de procesare a acestora pentru a fi aduse la formatul prevăzut de componenta back-end
  - back-end = componenta software de procesare a datelor furnizate de front-end
- Modelul pe trei nivele
  - arhitectură de tip client-server
  - conține: nivel de prezentare (browser web), nivel de logică business (middleware - server de application: "motor" de rulare a paginilor web dinamice scrise în ASP.NET, JSP/Java, PHP, Perl, Python, Ruby), nivel de bază de date (server de bază de date)
  - browserul trimite cererile către serverul de aplicație, care le servește interogând și modificând conținutul bazei de date și, de asemenea, generează pagina de răspuns care va fi trimisă înapoi browserului.

Exemplu de descriere a arhitecturii unei aplicații: Platformă de gestiune a datelor medicale electronice

Software Design Document (SDD)

- document de **specificație** a soluției pentru sistemul software descris în **SRS**
- răspunde la întrebarea: *cum va fi construit sistemul software pentru a avea comportamentul descris în SRS?*
- prezinta metodologii, tehnologii, participanți și resurse implicate în proiect
- se poate redacta numai după finalizarea SRS-ului fiind un răspuns la cerințele prezentate în SRS
- este redactat de o echipă de software designers (proiectanți de sistem) și analiști business, pe baza SRS-ului și a experienței acestora
- reprezintă ghidul de construire a soluției folosit de echipa de dezvoltare a proiectului
- reprezintă un tool de analiză a întregului proiect în faza de început cât și de monitorizare pe parcurs

Secțiuni SDD

Un SDD are următoarele secțiuni:

1. **Scopul documentului (Document purpose)**
2. **Obiective (Objectives)**
3. **Conținutul documentului (Document overview)**
4. **Modelul datelor** (*Data Design*)
   - prezintă *structurile de date* importante, *formatele fișierelor* folosite în cadrul soluției și *schema bazei de date*.
   - Structurile de date pot fi:
     - **globale** (structuri de date disponibile tuturor componentelor arhitecturii)
     - **de legătură** (structuri de date trimise ca argumente între componente pentru a asigura fluxul informației la nivel de aplicație)
     - **temporare** (structuri de date folosite temporar).
   - Schema bazei de date este descrisă prin:
     - **diagrama schemei** bazei de date
     - **descrierea semnificației tabelelor** și, pentru fiecare tabelă, descrierea semnificației coloanelor și indicarea cheilor primare și referențiale.
5. **Modelul arhitectural** (*Architectural Design*)
   - este o structură ierarhică alcătuită din componente interconectate
   - prezintă arhitectura sistemului – atât descriptiv, cât și sub forma unei diagrame de arhitectură
   - descrie:
     - fiecare componentă a arhitecturii,
     - restricțiile de implementare ale componentelor,
     - interacțiunea dintre componentele sistemului.
   - poate fi reprezentat prin :
     - **diagrame de componente** (pentru proiecte mari) - le-am numit "generic" în laboratorul 2: diagrame de arhitectură
     - **diagrame de clase**, în care relațiile ierarhice se bazează pe generalizare și specializare (pentru proiecte mici).
6. **Modelul interfeței cu utilizatorul** (*User Interface Design*)
   - prezintă *ferestrele* aplicației și *succesiunea* lor în cadrul sistemului.
7. **Elementele de testare** (*Testing Issues*)
   - identifică *componentele critice* ale aplicației (componente a căror performanță influențează decisiv performanța globală a aplicației)

- propune *alternative* de proiectare a componentelor critice (pentru a fi folosite în cazul insuccesului soluției propuse inițial).

Exemplu de Document SDD

Exemplu de document SDD mai vechi: Exemplu de Document SDD
Exemplele de mai jos sunt mai noi. Unul se remarcă prin organizarea clară a informaţiilor şi structură/organizare, dar şi prin concizie, însă Scopul şi Obiectivele nu sunt în totalitate corecte. Cel de-al doilea se remarcă prin forma simplă şi schematică, dar care include toate secţiunile importante (şi chiar extra) într-un mod concis.

Exemple mai noi

va urez restanta usoara zgarcitilor

Laborator 3 - Mijloace de organizare a proiectului. Controlul versiunii

Cum să ne organizăm pentru a atinge obiectivele definite în SRS conform planurilor din SDD?

În managementul de proiect, există o serie de utilitare de organizare a lucrului în proiecte, precum:

- **WBS**
- **Grafic Gantt**

WBS (Work Breakdown Structure)

- structură arborescentă cu rolul de a determina și de a grupa **sarcinile proiectului** în funcție de categoria din care fac parte
- ajută la organizarea proiectului prin clarificarea obiectivelor, a **elementelor de lucru** mai generale (*work packages*) și mai specifice (*work elements*)
- pe baza WBS, se pot face estimări de durată, de resurse umane și de buget → *suport pentru construirea diagramei Gantt*
- ajută la identificarea relațiilor de precedență dintre elementele de lucru (care elemente de lucru sunt independente, care sunt restricționate să aibă loc înainte de altele, etc.) - elemente vizibile în diagrama Gantt.
- WBS reprezinta graful acțiunilor necesare unui proiect grupând acțiunile în categorii de interes
- oferă suport pentru identificarea tuturor acțiunilor necesare și prioritizarea lor în funcție de efort, cost, durată etc.

Modul de construire a WBS

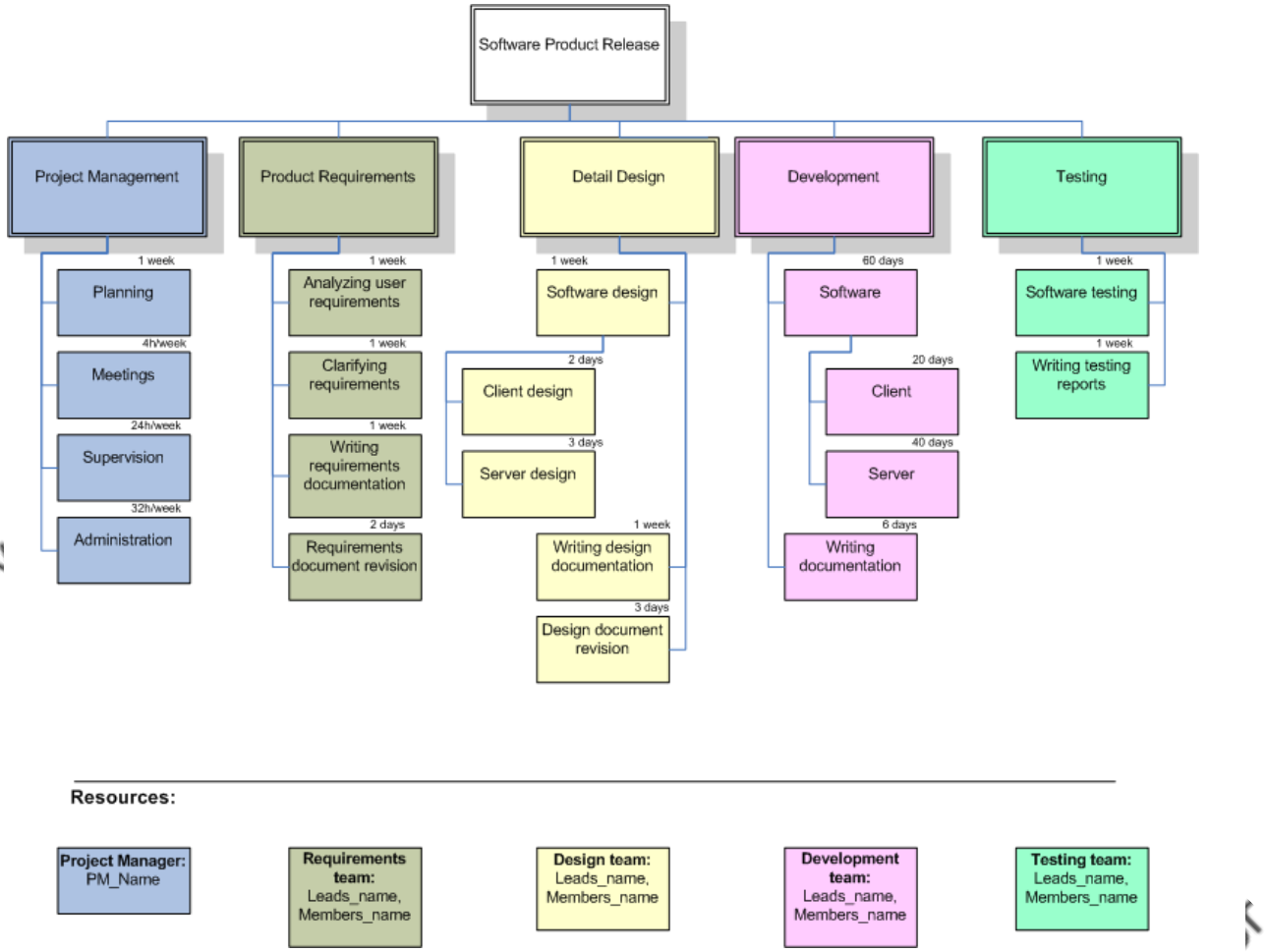
- nodurile arborelui conțin elementele de lucru, care pot fi produse, date, servicii
- rădăcina arborelui coincide cu obiectivul proiectului: sistemul software de dezvoltat
- nivelul al doilea al arborelui: subsistemele componente ale sistemului
- tot așa în adâncimea arborelui, în sensul că orice nod părinte înseamnă un element de lucru general, iar copiii reprezintă elemente de lucru componente ale lui

- deci, se detaliază efortul de dezvoltare a proiectului, de la obiectivul general la elemente de lucru și sarcini practice.
- nivelul de detaliere al unui WBS nu este foarte ridicat; astfel, adâncimea arborelui nu depășește, adesea, mai mult de *4-5 niveluri*.

Reguli

- un WBS trebuie să cuprindă prin elementele sale **tot** efortul necesar dezvoltării proiectului (100% din muncă)
- elementele de muncă de pe același nivel din arbore nu se acoperă total sau parțial (nu sunt *overlapping*)
- frunzele din arbore au adesea asociate resurse, durate și costuri.

Tool-uri de creare a WBS-urilor: WBS Schedule Pro, Online Visual Paradigm, Planhammer.io
Exemplu de diagramă WBS a unui proiect software:



Grafic Gantt

- grafic pentru **planificarea temporală** a sarcinilor proiectului

- construit pe baza activităților identificate în **WBS**
- indică data de început și de sfârșit a activităților din WBS și dependențele dintre activități:
  - start-to-start
  - finish-to-start
  - finish-to-finish
- întotdeauna se va ține cont de potențialele întârzieri și vor incluse în planificare sub o formă sau alta (review, delay, etc.)
- graficul Gantt este unul orientativ și starea sa se poate altera în timpul proiectului în funcție de situație
- acțiunile în grafic sunt trecute ca începand cel mai curând posibil, urmând să se asigure o marjă de eroare până la momentul cel mai târziu posibil
- este un tool bun pentru monitorizarea stării proiectului în timp, în funcție de gradul de realizare a acțiunilor până la un moment dat
- adesea, conține și următoarele elemente:
  - o linie verticală ce marchează timpul prezent
  - alte linii verticale cu milestone-urile proiectului
  - starea activităților exprimată prin procente (100% înseamnă că activitatea a fost realizată complet).

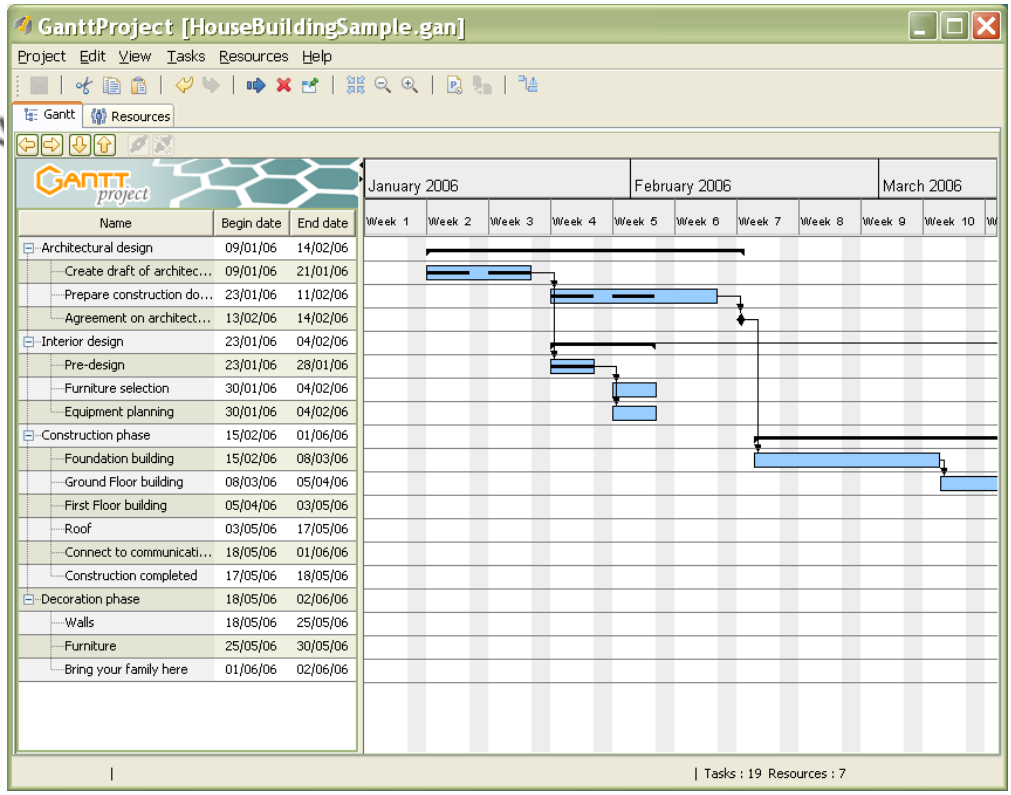
Organizarea unui proiect înseamnă parcurgerea etapelor:

1. Determinarea activităților proiectului (*WBS*)
2. Estimarea duratelor activităților și a consumului de resurse (astfel încât costul total să fie mai mic decât bugetul alocat)
3. Planificarea activităților în funcție de dependențe și de eventualele restricții legate de resurse (*Gantt*)
4. Stabilirea de *milestone-uri*.

Tool-uri de creare a diagramelor Gantt: Gantt Project, Team Gantt, Gantter, Gantt Excel, Microsoft Project.

va ure-tanta usoara zgarcitilor

Exemplu de diagramă Gantt a unui proiect de construcție de casă:



Milestone

- deadline de finalizare a unei etape importante și decisive a proiectului
- adesea presupune predarea unor livrabile către beneficiar
- în vederea respectării milestone-urilor fixate, adesea se impune luarea unor decizii cu impact major asupra evoluției proiectului.
- deși reprezintă puncte cheie în cadrul unui timeline, din varii motive acestea pot fi decalate dacă acest lucru ajută și nu au un impact negativ asupra proiectului.

Controlul versiunii (Git)

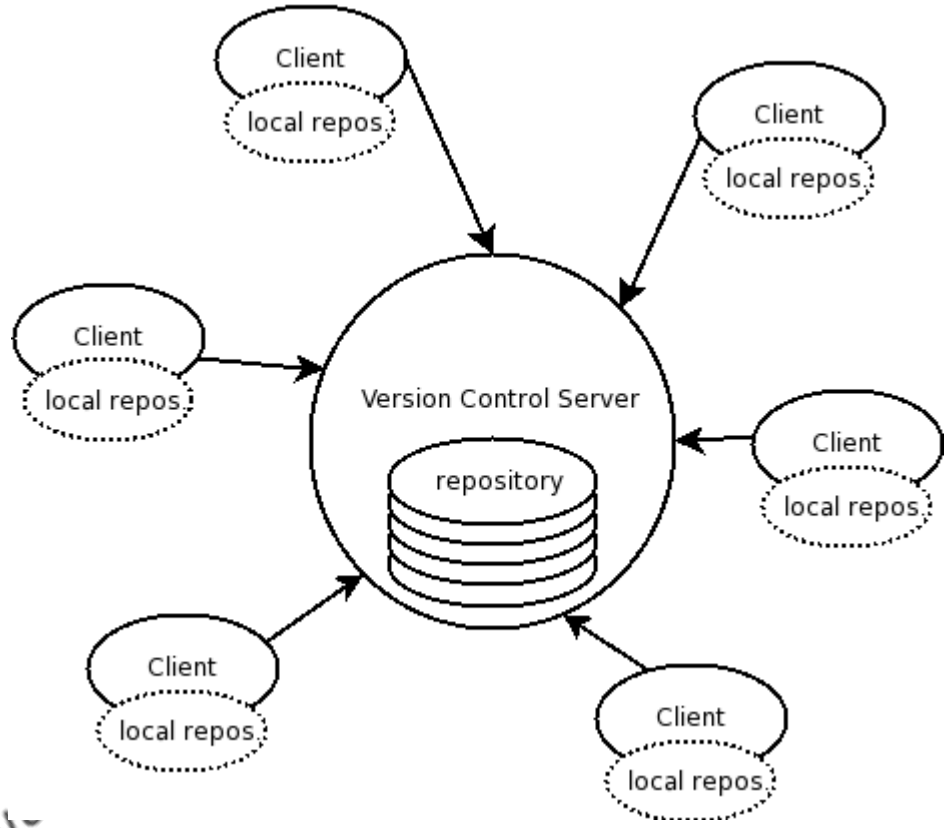
Sisteme pentru contolul versiunii (*Version Control Systems* - **VCS** - sau *Source Code Management* - **SCM**) sunt aplicații care permit lucrul colaborativ pe diverse fișiere, în special fișiere cod sursă. Sistemele pentru controlul versiunii sunt practic obligatorii în cadrul unui proiect cu dezvoltatori multipli. Astfel de sisteme rețin istoricul modificărilor efectuate de fiecare dezvoltator și folosesc comenzi specializate care să faciliteze transmiterea acestor modificări între dezvoltatori.
Exceptând sistemele de gestiune a surselor, prezentate mai detaliat în continuare, și alte aplicații folosesc versiuni:

- Wiki Engines - pentru fiecare modificare se rețin doar schimbările de la versiunea anterioară; orice modificare poate fi anulată;

- Google Docs - versiuni pentru fiecare modificare;
- MS Office, OpenOffice - permit atașarea unor numere de identificare pe documentele editate.

Principiul de funcționare a sistemelor de gestiune a codului este comun:

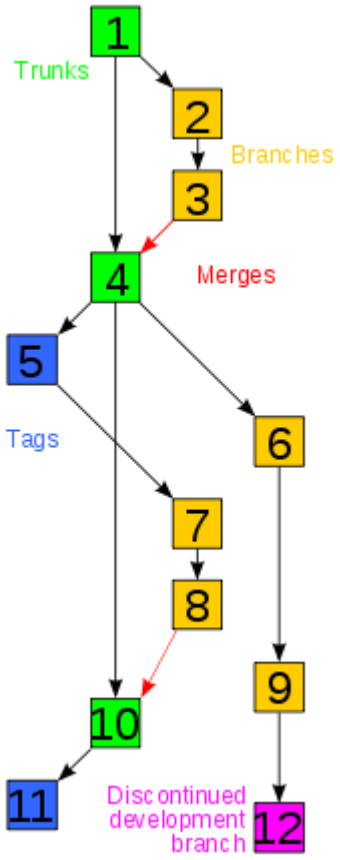
- Sursele sunt păstrate, de obicei, într-un **repository** (**depozit**) aflat pe un server accesibil tuturor dezvoltatorilor. Acest repository constituie mecanismul principal de sincronizare a surselor dezvoltatorilor.



- Fiecare dezvoltator obține o copie a repository-ului, denumită **copie locală**. Copia rezidă pe sistemul dezvoltatorului. Dezvoltarea se va realiza în acest director.
- Există două operații care permit comunicația cu repository-ul:
  - **update** sau **pull** înseamnă actualizarea copiei locale cu informațiile din repository; este posibil ca alți dezvoltatori să fi comis schimbări în repository;
  - **commit** sau **push** înseamnă transmiterea/comiterea modificărilor locale în repository; ceilalți dezvoltatori vor folosi comanda **update** pentru actualizarea copiei locale cu acele informații.

Câteva cuvinte cheie utile în lucrul cu sisteme de control al versiunii sunt (denumirile sunt în engleză pentru că aceasta este forma uzuală de utilizare):

- **repository** locul (centralizat) în care se găsesc sursele și informațiile de modificare;
- **working copy** copia locală a repository-ului folosită de un dezvoltator;
- **check-out** operația de creare a unei copii locale pornind de la repository (clonarea repository-ului);
- **commit** (vezi mai sus)
- **update** (vezi mai sus)
- **branch** o ramură de dezvoltare (un fork) care poate fi dezvoltată în paralel față de ramura principală de dezvoltare (vezi imaginea de mai jos);
- **trunk** ramura principală de dezvoltare (vezi imaginea de mai jos);
- **tag** o referință la un snapshot al surselor la un moment dat în timp;
- **merge** o operație de unificare a două seturi de schimbări petrecute distinct;
- **conflict** situație în care două modificări sunt efectuate din surse diferite și sistemul nu poate găsi o soluție de unificare (merge) a acestor schimbări; dezvoltatorul va trebui să rezolve conflictul fie prin unificarea schimbărilor, fie prin selectarea unei singure variante



Există două tipuri de sisteme pentru gestiunea codului sursă:

- sisteme centralizate (Subversion, CVS, perforce);
- sisteme distribuite (Git, Darcs, Mercurial, Bazaar).

Detalii despre diferențele dintre acestea (mai degrabă între doi dintre cei mai cunoscuți reprezentanți, Subversion și Git) găsiți aici.

Git

Git este unul dintre cele mai folosite sisteme de versionare distribuite. Cele mai importante concepte din Git sunt următoarele:

- repository - există un repository remote și oricâte repository-uri locale
- commit - conținut modificat
- branch - lanț de commit-uri

va urez restanta usoara zgarcitilor

Laborator 4 - Utilitare pentru managementul și dezvoltarea proiectelor software

Aplicații web pentru MPS - GitHub

Aplicațiile web pentru managementul proiectelor software integrează mai multe aplicații folosite pentru a asigura o bună comunicare între membrii echipei și monitorizare a evoluției proiectului: wiki, source-code browser, tichete (buguri și solicitări de implementare), calendar (roadmap) etc. Trac, Redmine, Google Code și GitHub reprezintă cele mai cunoscute platforme ale acestei categorii de aplicații.
GitHub oferă următoarele funcționalități:

- crearea de organizații: de pe Select button-ul cu numele utilizatorului se selectează opțiunea "Create Organization"
- crearea de echipe: din interfața pentru organizație se accesează meniul "Teams" și se crează o echipă folosind butonul "New Team"
- crearea de proiecte publice sau private: din meniul din dreapta "Your repositories" se accesează butonul "New Repository" si se selectează tipul acestuia
- crearea unei pagini pe wiki: de pe pagina unui proiect se selectează din meniu "Wiki" și se apasă butonul "New Page"
- crearea de milestone-uri: de pe pagina unui proiect se selectează din meniu "Issues", se comută pe tab-ul "Milestones" și se apasă butonul "Create a new milestone"
- vizualizarea repository-ului și a commit-urilor
- realizarea de fork-uri: se apasă butonul "Fork"
- realizarea și acceptarea de pull request-uri
- crearea de branch-uri
- operația de "merge" între 2 branch-uri
- realizarea de code review
- bugtracking
- crearea și vizualizarea rapoartelor (issues)
- realizarea unor grafice despre evoluția proiectelor
- vizualizarea activităților din cadrul proiectului (creare de rapoarte (issues), știri, upload de document, commit-uri în repository, editări de wiki etc.)
- posibilitatea de a urmări evoluția altor proiecte: se apasă butonul "Watch" de pe pagina proiectului respectiv

Pentru a vă familiariza cu funcționalitățile sale de bază, accesați pagina de ajutor a platformei GitHub.

În caz de hazarde naturale, instrucțiunile sunt disponibile aici.

Modelare - Netbeans, Eclipse

Netbeans și Eclipse reprezintă un set de IDE-uri cu funcționalități complexe ce îi ajută, în special, pe dezvoltatorii de Java. Una din funcționalitățile importante ale IDE-urilor o reprezintă posibilitea creării de diagrame UML. În urma creării acestor diagrame se poate genera cod automat, reducând astfel efortul de dezvoltare a aplicației. Pentru C++/C# cel mai adesea se folosește Visual Studio.

Diagrame - MS Visio, Dia

În activitatea de proiectare se realizează numeroase grafice pentru a descrie componentele și modulele aplicației, legăturile între acestea și cazuri de utilizare.

Una dintre cele mai cunoscute aplicații pentru realizarea de diagrame este Microsoft Visio. Alternativa lightweight a acestuia este Dia. Alternativa din familia OpenOffice este OpenOffice Draw. Toate aceste aplicații conțin un set de imagini/module predefinite (utilizatori, sisteme de calcul, elemente de rețelistică, forme predefinite de imagini de componente). Recomandăm folosirea acestor aplicații pentru realizarea diagramelor și graficelor în realizarea documentației pentru toate proiectele unde este necesar. Aplicațiile de mai sus oferă o interfață grafică user-friendly și permit exportarea într-un număr vast de formate.
Pentru modelarea arhitecturii (în special pentru SDD) amintim StarUML (cu suport pentruMacOSX, Linux, Windows) și DrawIO, soluție online care poate exporta rezultatul atât ca imagine cât și ca pdf. Schema este salvată sub forma unui XML, ceea ce face posibilă salvarea și încărcarea cu ușurință a fișierelor de pe device-ul local direct în aplicația online si invers, pentru editare ulterioară. Alte aplicatii: Visual Paradigm sau Astah

Limbaje de programare, biblioteci, framework-uri, coding style

În cadrul procesului de proiectare și dezvoltare a unui proiect software, trebuie să se aleagă limbajul/limbajele de programare și bibliotecile folosite, framework-urile utilizate și alte aplicații care vor înlesni procesul de dezvoltare, testare și mentenanță.

Una din ideile importante în alegerea unui limbaj sau biblioteci este că nu există un super-limbaj sau un super-framework care să rezolve toate problemele, indiferent de natura acestora. Domeniul de software engineering s-a dezvoltat tocmai pentru a găsi cele mai bune opțiuni și proceduri pentru o situație dată.

Alegerea unui limbaj, a unui framework sau a unor utilitare pentru dezvoltare/testare se realizează, în general, ținând cont de două aspecte:

- cât de potrivit este acel limbaj sau framework pentru proiectul dat;
- cât de acomodați sunt viitorii dezvoltatori cu acel limbaj/framework.

Folosirea unui limbaj/framework adecvat dar pe care utilizatorii nu-l cunosc și pentru care partea de training durează mult, poate produce frustrări în rândul viitorilor dezvoltatori și poate genera ineficiență și, în consecință, un produs necompetitiv.

Ca recomandare generală, limbajele de programare se pretează la anumite situații:

- aplicații low-level, care necesită eficiență, se vor realiza, de obicei în C
- aplicații care au nevoie de dezvoltare rapidă vor fi realizate, de obicei, într-un limbaj cu facilități object-oriented de forma Java sau Python
- aplicații middleware vor folosi biblioteci și API-uri specializate (de forma OpenMP, MPI, RMI, Corba) și limbaje specializate
- aplicațiile web vor folosi framework-uri și limbaje specializate (Ruby on Rails, PHP, JSP/Servlets)

Un alt aspect important în proiectarea și dezvoltarea unei aplicații îl constituie formatul fișierelor de configurare. Acestea pot fi:

- fișiere binare - ocupă puțin spațiu, sunt relativ ușor de parsat, dar nu sunt portabile
- fișiere text format propriu - pot fi editate de utilizator, nu reprezinta un format standard
- fișiere .ini - reprezintă un fișier text în format standard
- fișiere .xml - format standard răspândit cu dezavantajul că ocupă mult spațiu și parsarea fișierului consumă memorie

Indiferent de limbajul, bibliotecile și framework-urile folosite, pentru facilitarea parcurgerii, înțelegerii și îmbunătățirii codului se recomandă folosirea unui set de reguli, încadrate de obicei, într-un document (de obicei fișier text) de tip "Coding Style" sau "Coding Standards". Exemple de astfel de documente sunt:

- Linux Coding Style
- GNU Coding Standards
- Kernel Normal Form
- PEP 8 -- Style Guide for Python Code
- pear Coding Standards
- Code Conventions for the Java Programming Language

În general, convențiile de codare diferă dar urmăresc aspecte comune:

- indentarea codului
- cod lizibil prin folosirea de spații și linii libere
- cod comentat și comentarii relevante
- nume relevante pentru variabile
- funcții cu dimensiune "umană" (nu kilometrice)
- consecvența stilului de codare (dacă se indentează codul cu TAB, se va indenta cu TAB peste tot; dacă se indentează cu 4 spații, se vor folosi 4 spații peste tot)

Documente de raportare

Folosirea unor documente de raportare periodice reprezintă o acțiune de bază în coordonarea și evaluarea activităților din cadrul unui proiect. Astfel de documente de raportare se vor referi la sarcinile planificate și realizate în perioada anterioară. Vor descrie, de asemenea, probleme apărute și soluții la acestea.

Documentele de raportare pot fi furnizate la nivel de săptămâna, lună, semestru sau an, funcție de care variază dimensiunea documentului și detaliile cuprinse. Exemple de șabloane de documente de raportare săptămânală se găsesc aici, aici și aici. Documentul de raportare săptămânală este realizat de project manager prin analiza activităților planificate/efecuate în cadrul săptămânii anterioare.
Documentele de raportare a activității au o structură tabelară în care sunt descrise principalele activități care s-au desfășurat pe parcursul perioadei raportate. Fiecare activitate este descrisă printr-un nume, data planificată pentru încheiere, responsabilul/responsabilii activității, starea curentă (în lucru, suspendată, dependentă de un rezultat, încheiată) și procentul realizat. Exemple de șabloane de documente de raportare a activității se găsesc aici și aici. Documentul de raportare a activității este completat de project manager pe baza informațiilor furnizate de membrii echipei și responsabilii fiecărei activități.
Evaluarea membrilor echipei este importantă în cadrul unui proiect. Evaluarea are rolul de a determina gradul de implicare și competență a membrilor echipei și de a lua decizii de actualizare a distribuției sarcinilor pentru eficientizarea realizării acestora. Un exemplu de formular de implicare a membrilor echipei este acesta. Evaluarea este realizată de project manager.

va urez restanta usoara zgarcitilor

Comunicarea

Ca în orice domeniu, comunicarea poate să aibă loc, fie pe cale verbală, fie pe cale scrisă.

Comunicarea verbală este interactivă, rapidă, dar nu ajunge la nivelul de detaliere posibil prin comunicarea scrisă. Comunicarea verbală transmite sensuri și simboluri mai curând decât informație exactă. De exemplu: daca toata lumea este de acord ca la forma "patrat" sa ii zica cerc atunci acesta este sensul general acceptat de catre toti. Capcana tipică este de a crede că toată lumea a înțeles același lucru. Detalii esențiale pot dispărea rapid din memorie. Fiecare persoana umple acele goluri existente intr-un mesaj cu elemente proprii din alte experiente.
Comunicarea trebuie sa fie exacta,specifica fara generalizari,omisiuni,ambiguitati. Tipic, o persoană poate reține circa 50% din ce s-a discutat în primele 20 de minute, iar după aceea procentul scade rapid (http://www.mtholyoke.edu/acad/intrel/speech/differences.htm). Comunicarea verbală este utilă în special în situații decizionale, când interlocutorii sunt deja familiari cu subiectul, și este recomandat să fie urmată de un rezumat scris.
Comunicarea scrisa este mult mai susceptibilă la erori de emisie și/sau recepție. Problemele pot apărea în funcție de formularea aleasă și de interpretarea dată de cititor. În lipsa posibilitătii de interacțiune în vederea lămuririi unui aspect, confuziile și erorile de comunicare pot apărea ușor.(vezi interpretari ale legilor). Ca avantaje, informația poate fi foarte detaliată, exactă și mai ales e stabilă în timp. Comunicarea scrisă este utilă pentru transmiterea efectivă de informație și fixarea informației schimbate în comunicările verbale.

Comunicarea verbală

Următoarele reguli sunt utile în cadrul comunicării verbale:

- **Nu întrerupeți pe cel care vorbește la un moment dat.**
  - Există 2 posibilități: aveți ceva valoros de zis sau doar o observație care nu ar aduce valoare discuției. În ambele cazuri, în afară de lipsa de politețe, și potențialele tensiuni create din partea persoanei respective, o să aduceți și un minus în termen de eficiență a discuției.
  - Soluția recomandată, pentru a nu întrerupe discuția, dar şi a păstra ideile bune care v-au venit, este să aveți un carnețel / laptop în care să notați aceste idei, pe care le puteți expune când vorbitorul a terminat ce avea de spus.
- **Nu vorbiți mai mult de 5 minute fără pauză.**
- **Separați clar ideile distincte.**
  - Spre deosebire de un material scris, unde ideile distincte pot fi separate ușor în mod vizual, folosind diverși despărțitori, în comunicarea verbală, trecerea bruscă de la un subiect la altul poate reprezenta o problemă.
- **Nu stați cu spatele la cei cărora vă adresați.**
- În cadrul grupului de lucru puteţi stabili un set de **reguli suplimentare de comunicare**.

- O bună practică este să existe un set de semne stabilit pentru a facilita transmiterea de mesaje vorbitorului fără o întrerupere verbală. Exemple de mesaje pentru care pot fi transmise folosind anumite semne:
  - off-topic - pentru a avertiza vorbitorul ca a divagat de la subiect;
  - scurtează - dacă vorbitorul discută prea mult pe o singură temă;
  - vreau să iau cuvântul - pentru a cere o întrerupere, pentru a aduce un comentariu important;
  - subiect închis - dacă se consideră că subiectul este clar și se poate trece la următorul punct în discuție.
- **Fiti asertivi.**
- În cadrul discuției nu încercați să vă impuneți punctul de vedere și ascultați și părerile celorlalți încercând să înțelegeți argumentele lor.

Comunicarea în scris

Deși se preferă, în general, comunicarea verbală în cadrul dezvoltării unui proiect software (în special în cadrul metodologiilor Agile), există numeroase situații în care este necesară transmiterea unor informații în formă scrisă.

Întotdeauna când comunicați, trebuie să aveți în vedere respectul pentru timpul celorlalți. Acest lucru înseamnă că trebuie să comunicați ce aveți de transmis scurt și clar.

Înainte de a trimite un mesaj, fie că este vorba despre un e-mail sau despre un document cu un raport, puneți-vă următoarele întrebări:

- Ce vreau să transmit?
- Cui vreau să transmit? (persoanele de la HR probabil nu vor fi interesate de ce bug ați descoperit în kernel)
- Ce rezultat / ce răspuns aștept? (dacă PM-ul trimite un mail în care anunță că există un mare bug în aplicație, dar nu numește un responsabil / un deadline / nu propune o soluție, mesajul nu va fi eficient).

Liste de discuții & e-mailuri

Reguli pentru listele de discuții:

- Folosiți subiecte clare, nu subiecte de forma “Urgent”, “Important”, “Ceva”, “Pentru mâine”, “Reply”. Nu transmiteți mesaje fără subiect.
- Folosiți etichete la început de subiect pentru a încadra domeniul mesajului: (Exemplu: [Testare] / [Debug] / [Intalnire Joi]
- Evitați subiecte de tipul “Re: Re: Re: Re: Re: […]”.
  - Exemplu de Subject prost: “Intalnire”

- Exemplu de Subject bun: ”[Intalnire] Intrevedere luni, 15 noiembrie 2010, 16:00 pentru stabilire plan de testare”
- Scrieți mesaje scurte și la obiect; nu trimiteți mesaje gigant (nu le va citi nimeni) sau diluate (se va căuta ideea dorită în cadrul paragrafelor).
- Dacă sunt mai multe idei într-un e-mail, separați-le vizual cât mai bine (prin paragrafe noi, prin gruparea in liste).
- Dacă vreți să transmiteți două topic-uri diferite, trimiteți două mesaje.
- Dacă aveți un mail mai lung, puteți scrie un rezumat în prima parte, pentru ca cel care citește să înțeleagă contextul și ce urmează să citească.
- Dacă un e-mail are mai mult de 500 de cuvinte, cel mai probabil era nevoie de un fișier.
- Dacă răspundeți la un mail dintr-un lung șir de răspunsuri, ștergeți din când în când conținutul mesajelor vechi, pentru a nu ajunge la un mail de 2000 de rânduri din care doar 20 mai sunt de actualitate.
- Dacă răspundeți la un mail cu mai multe puncte, răspundeți între acele puncte.
- Dacă aveți link-uri lungi de inclus în mail, le puteți indexa cu [1] [2] [3] și include la sfârșitul mailului, pentru a nu intercala în mesaj linkuri lungi.
- Nu folositi ALL CAPS. ALL CAPS denotă un ton răstit, și nimeni nu apreciază asta.
- Evitați transmiterea de atașamente pe o listă. Încarcă inutil căsuța poștală a tuturor membrilor. Publicați-le pe un site și trimiteți link-ul aferent în e-mail.
- Dacă un mesaj se adresează numai unei minorități a membrilor unei de liste de discuție, trimiteți mesajul privat acelor membri. Nu trimiteți un mesaj unor persoane neinteresate de subiect (nu trimiteți spam 😀)

Change logs

Este recomandat să existe o evidență a derulării unui proiect. Deși în general, există o motivație scăzută pentru evidența schimbărilor, este util să existe un set de fișiere de tip jurnal/log care să conțină date despre:

- bug-fix-uri;
- modificări efectuate;
- decizii de proiectare/alegere de componente.

Majoritatea aplicațiilor de (software) project management (inclusiv GitHub) conțin facilități de înregistrare a activității utilizatorilor, precum: adăugarea unui issue, rezolvarea unei probleme, posting-ul unui mesaj, editarea unei pagini wiki.

va urez restanta usoara

Comentarii cod

Există două moduri de a comenta cod: un mod "asistat" și unul "liber".

Modul de comentare "asistat" este acela în care programatorul urmărește o sintaxă ce poate fi citită și interpretată de un sistem automat de generare de documentație.

- Pentru Java, un astfel de sistem de documentare a codului este http://java.sun.com/j2se/javadoc/
- Pentru .NET există o soluție similară: http://msdn.microsoft.com/en-us/library/b2s063f7.aspx
- Acest mod de comentare are totuși limitările sale, întrucât este conceput mai mult pentru a documenta clasele, interfețele, metodele, legăturile dintre acestea, parametrii care se transmit, în general informații care nu explică implementarea.

Pentru documentarea implementării, modul "liber", nu există un sistem clar definit, dar următoarele reguli sunt, în general, bine de urmărit:

- Când vreți să explicați o linie, puneți comentariile înaintea liniei respective, nu în dreapta

```
// un select simplu pentru a prelua din BD toate campurile din tabelul primit ca parametru
ResultSet rs = st.executeQuery("select * from " + tableName);
```

```
ResultSetMetaData metadata = rs.getMetaData();
```

- Dacă o bucată de cod este incompletă/neoptimizată și există șanse ca ulterior să se mai lucreze la ea, folosiți cuvinte cheie:
  - **BUG/FIX**: Pentru a marca un bug care nu a fost încă eliminat, sau un fix. De multe ori un fix introduce bug-uri noi, astfel este bine să documentați locurile în care ați făcut modificări și ați schimbat comportamentul programului.
  - **TODO**: Puteți folosi acest cuvânt cheie pentru a marca un punct din program unde se pot aduce îmbunătățiri/ unde mai trebuie completat ceva. Există medii de dezvoltare care pot chiar recunoște astfel de cuvinte cheie și pot crea o listă de TODO-uri.

```
// TODO: check data input
```

- Comentați și explicați cât mai clar; nu mergeți pe ideea "lasă că știu eu ce am vrut să fac aici".
- Nu exagerați cu diverse comentarii inutile. Exemplu:

```
// increment i
i++;
```

- La această adresă găsiți sfaturi legate de comentarea corespunzătoare a codului.

Pentru amuzament, consultați pagina What is the best comment in source code you have ever encountered?

Întâlnirile

Întâlnirile reprezintă un mod de comunicare vital pentru realizarea unui proiect, fiind folosite în special pentru a lua decizii în anumite situații, pentru a găsi soluții și pentru a analiza progresul actual al proiectului precum și planul de lucru pentru perioada următoare.

Ca regulă generală, pentru ca o întâlnire să fie eficientă, aceasta trebuie să fie cât mai scurtă și să implice doar oamenii direct interesați / în cunoștință de cauză. De exemplu, in metodologia SCRUM (metodologie de tip Agile), se recomandă întâlniri cu timp fix de 15 minute. Nu se permite depășirea timpului. Astfel, participanții au fiecare un timp limitat pentru a își expune ideile, și sunt astfel obligați să își structureze și rezume ideile înainte de întâlnire.

Întâlnirile SCRUM au doar caracter informativ. Astfel, fiecare participant prezintă statusul acțiunilor de care este responsabil fără să se dezbată eventualele probleme. Problemele ce nu pot fi rezolvate la sursă se scalează către superior.

Programarea întâlnirilor

Planificarea întâlnirilor trebuie să fie efectuată de comun acord, pentru a nu interfera cu programul celorlalți / pentru a nu apărea suprapuneri. Doodle este unul din cele mai cunoscute site-uri create pentru acest scop.
Se recomandă, de asemenea, folosirea aplicațiilor de tip calendar pentru stabilirea întâlnirilor și pentru notificarea participanților. Există aplicații Desktop precum Microsoft Outlook, Evolution, Mozilla Sunbird sau aplicații web precum Google Calendar. Mai multe informații despre aplicații de calendar sunt prezentate pe Wikipedia.

Planificarea și organizarea unei întâlniri

Pentru ca o întâlnire să fie eficientă, trebuie ca cel care organizează întâlnirea să înțeleagă răspunsul la următoarele întrebări, în funcție de care să iau deciziile aferente. *Care este scopul întâlnirii?* - Este important ca o întâlnire să aibă la bază un scop precis. *Care este rezultatul scontat în urma întâlnirii și cum poate fi el cuantificat?* - Este bine să existe un target clar stabilit, sub forma unui output ce trebuie să rezulte din întâlnire. La sfârșitul întâlnirii, situația proiectului trebuie să fie diferită decât la începutul întâlnirii: o decize luată, un set de probleme clarificate, un impas depășit, etc. *Cine trebuie să participe la întâlnire?* - Este vital pentru succesul unei întâlniri ca la aceasta să participe persoanele interesante /

persoanele care au răspunsuri / persoanele care cunosc proiectul/aspectul care va fi discutat.
De asemenea, atât din respect pentru timpul celor implicați, dar și pentru a fi o întâlnire eficientă, este recomandat ca înainte de întâlnire, invitații să fie informați de:

- scopul întâlnirii
- aspectele ce vor fi discutate
- rezultatele așteptate
- durata estimată a întâlnirii

În plus, organizatorul întâlnirii ar trebui să:

- planifice o locație adecvată
- creeze o agendă de discuție

Înainte de începerea discuţiilor:

- O persoană trebuie să fie desemnată să noteze **o minută** cu cele discutate.
- Coordonatorul întâlnirii va enumera pe scurt ce urmează a fi discutat.

Pe parcursul întâlnirii:

- Coordonatorul întâlnirii trebuie să încurajeze discuția.
- Pentru a fi eficientă, discuția trebuie să fie axată pe argumente.
- Coordonatorul este cel ce va "arbitra discuția".
- El trebuie să aibă și rol de "focus-man", și anume să aibă grijă ca discuția să se concentreze pe temele importante, să nu se divage de la scop.
- Dacă se ajunge la un impas din lipsă de informații, se poate programa o întâlnire ulterioară, până la care anumiți membri ai echipei vor avea ca task obținerea informațiilor necesare.

La sfârșitul întâlnirii:

- Se trece prin agenda întâlnirii, pentru a verifica dacă au fost acoperite toate aspectele.
- Se trag concluziile.
- Minuta redactată pe parcursul întâlnirii este distribuită factorilor de interes / echipei.

Se recomandă existența unui document sau pagini wiki care să fie completată înainte, în timpul și după întâlnire. O posibilă structură a acestei pagini este:

```
* Topic întâlnire

* Dată/timp întâlnire
```

```
===== Agendă =====


-- se completează la planificarea întâlnirii

* item1

* item2


===== Participanţi =====


-- se completează înainte de începerea discuţiilor

* participant1

* participant2


===== Discuţii =====


-- minuta întâlnirii pe parcursul discuţiei

* Iniţiale1: ...

* Iniţiale2: ...


===== Decizii/concluzii =====


-- imediat după încheierea discuţiei

* ...

* ...
```

Link-ul la acea pagină sau documentul aferent este transmis participanților după încheierea întrevederii.

Brainstorming

Brainstorming este o tehnică de grup creativă proiectată pentru a genera un număr mare de idei pentru soluția unei probleme.

Un set de utilitare precum FreeMind și Mind Manager sunt folosite pentru brainstorming/mind-mapping.

Decizii

În orice proiect există momente de impas în care o decizie trebuie luată, dar există mai multe opțiuni, și membrii echipei au păreri distincte.

În aceste situații, cel mai important lucru de avut în vedere este faptul că în multe situații, o decizie greșită poate fi mai puțin dăunătoare decât un impas pe termen lung. Cu alte cuvinte, leader-ul trebuie să își asume uneori o decizie pentru a scoate echipa din impas. Chiar dacă decizia este greșită, o decizie luată în timp util poate fi corectată în momentul în care lucrurile merg prost; pe de altă parte, dacă echipa rămâne în impas, nu se ajunge nicăieri și proiectul nu înaintează. Mai rău, se poate ajunge în situația în care o decizie este impusă în momentul în care echipa nu mai are opțiuni din lipsă de timp.

Decizia în grup

Există mai multe metode prin care un grup poate să ajungă la o decizie, fiecare având avantajele și dezavantajele ei, în funcție de situație:

* **Competiția ideilor** - fiecare opțiune este combătută până când toată lumea ajunge la aceeași părere * **Compromisul** - este adoptată o idee de mijloc, care echilibrează avantajele și dezavantajele * **Votul** - utilă mai ales pentru decizii obiective (Exemplu: decizii de design) * **Impunerea** - utilă în caz de impas. Dacă proiectul stagnează, conducătorul trebuie să își asume decizia pentru a merge mai departe * **Încrederea** în persoana care cunoaște cel mai bine domeniul / are cea mai multă **experiență**

Riscul

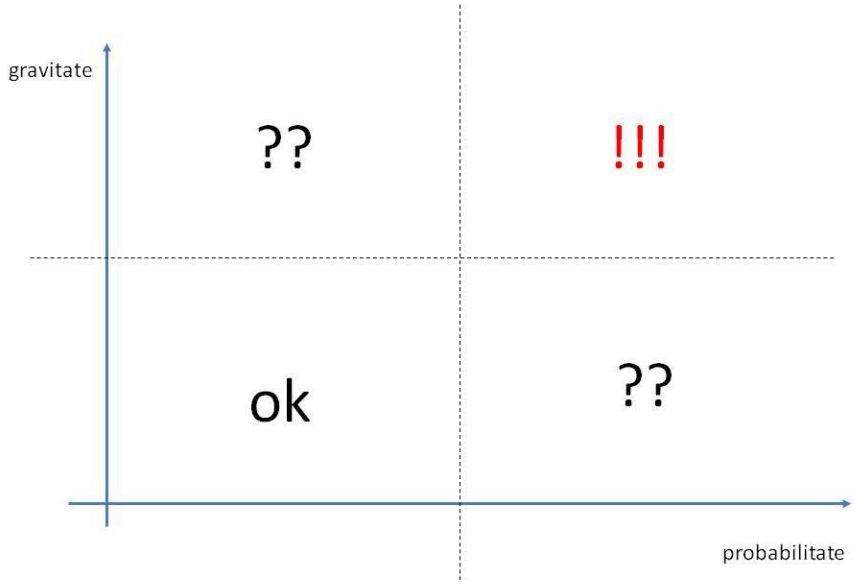
Uneori, o decizie greu de luat depinde de evaluarea unui risc. În cadrul unui proiect care "merge în necunoscut" apar o serie de riscuri. Unele variante prezintă anumite riscuri, alte variante de decizie prezintă alte riscuri. Problema este cum decidem ce variantă să alegem, pentru a minimiza riscul.

Riscul are în general 2 aspecte:

- probabilitatea - Care sunt șansele să aibă loc evenimentul?
- gravitatea - Cât de grav este dacă are loc?

O metodă general recomandată de analiză a riscului este de a aloca fiecărui risc identificat 2 note, una pentru probabilitate și una pentru gravitate. Riscul poate fi analizat fie grafic, fie ca o sumă a celor 2 note. Dacă folosim o scară de la 1 la 10, și un eveniment primește nota 16, în mod cert acesta trebuie considerat. In cazul in care se cunosc probabilitatea riscului si impactul pe care il are acesta in cadrul proiectului (numar de zile sau costuri), riscul va fi calculat ca produsul dintre probabilitatea de aparitie a riscului si efectul produs.

usoara zgarcitilor



CheckList

- Cât de importantă este decizia? Care sunt consecințele deciziei? Riscurile?
- Cât de clară este imaginea pe care o avem acum asupra problemei? Avem toate datele problemei?
- Au fost cercetate toate opțiunile?
- Au fost implicați / consultați membrii echipei cu cea mai mare expertiză în problema de față?

Rezolvarea conflictelor

Deși, în general, se dorește evitarea conflictelor, uneori pot apărea divergențe între membrii unei echipe. Pentru bunul mers al lucrurilor, este recomandat ca aceste divergențe să fie discutate și aplanate cât mai din timp. Orice problemă de tip conflictual neabordată la timpul ei poate escalada și va avea tendința de a se manifesta în momentele critice ale proiectului, când un conflict este ultimul lucru de care are nevoie echipa.

În cazul în care apar divergențe între doi membri ai echipei, este responsabilitatea team leader-ului/PM-ului, eventual cu ajutor din partea unei persoane de la HR să aplaneze conflictul înainte ca acesta să afecteze echipa.

În cazul unui conflict manager - subordonat, lucrurile se complică, și un factor determinant în rezolvarea unui astfel de conflict este dat de maturitatea managerului.

Există mai multe modalități de abordarea a unui conflict

Abordarea directă/dictatorială

Deși este cea mai "incomodă" poate fi și cea mai eficientă. Presupune determinare din partea liderului și este necesar ca orice critică adusă în cadrul confruntării să fie constructivă. Această abordare se bazează pe un stil de abordare logic, ingineresc, de rezolvare a unei probleme. Problema este expusă direct, sincer, și liderul încearcă să prezinte soluția logică. Dezavantajul metodei este dat de nivelul de sinceritate și de exprimarea directă necesară, care uneori nu va fi privită bine de toți cei implicați.

Negociere

Avantajul metodei este dat de faptul ca teoretic, prin negociere se poate ajunge la un compromis acceptat de toți factorii implicați. În realitate, de multe ori la final toți cei implicați pot ajunge nemulțumiți de ce au trebuit să cedeze.

Aplicarea regulilor

Sunt cazuri în care un membru al echipei generează un conflict prin nerespectarea regulilor / standardelor. Uneori, nu este oportună regenocierea / schimbarea regulilor, și deși cu un cost mare de antipatie din partea liderului, trebuie impuse regulile echipei / companiei.

Ocolire

În cazul în care miza este prea mică / nu este ceva urgent, liderul poate alege fie să amâne rezolvarea, fie să accepte o soluție chiar dacă aceasta nu este optimă. Deși această abordare are dezavantaje evidente, este foarte utilă totuși în momentul în care echipa stagnează blocată pe o discuție pe un detaliu nesemnificativ.

Empatizarea

Uneori, oamenii pot avea o idee similară, dar sunt blocați într-o discuție contradictorie pe detalii. În astfel de situații, liderul este cel care poate sublinia similitudinile între soluțiile propuse și le poate exploata pentru a sublinia că ideile nu sunt atât de diferite și, în final, poate negocia un consens.

Laborator 6 - Monitorizarea, evaluarea și controlul evoluției proiectului

Introducere

Premisele unui **proiect de succes** din perspectiva contribuției **PM**-ului sunt:

- *estimările corecte* ale duratelor și resurselor necesare îndeplinirii sarcinilor (planificări potrivite)
- *monitorizarea* și *evaluarea* consecventă a proiectului având drept ghid planul proiectului (Obiectivitate, Informare, Prevedere)
- *controlul* permanent al evoluției proiectului (Direcția și sensul către succes)
- *comunicare* clară și motivantă
- *coeziune* cu echipa.

Monitorizarea, evaluarea și controlul proiectului alcătuiesc **instrumentul de bază** utilizat de **PM** în mod **iterativ** pentru a asigura **evoluția** proiectului în consonanță cu planurile de **succes**.

Monitorizarea proiectului (5 min)

- este realizată de PM
- cuprinde activități precum:
  1. **colectarea de informații** privind evoluția proiectului și a factorilor de influență (echipă, context)
  2. **diseminarea** concluziilor evaluării și a schimbărilor decise către echipă.

Ce monitorizează PM-ul (colectează informații despre):

- statusul proiectului în raport cu planul proiectului
- costurile înregistrate în raport cu planul
- evoluția factorilor de risc
- evoluția contextului proiectului
- cantitatea și calitatea muncii îndeplinite de fiecare membru al echipei
- coeziunea și comunicarea în cadrul echipei.

Cum monitorizează PM-ul:

- prin întâlniri de proiect
- cu ajutorul tool-urilor ajutătoare de Software Project Management (Redmine, Trac): sistem de Ticketing, News, forumuri
- cu ajutorul listelor de discuție
- cu ajutorul rapoartelor de testare
- prin comunicare directă cu membrii echipei.

În urma monitorizării, PM-ul deține informațiile necesare evaluării proiectului, urmând ca după evaluarea propriu-zisă, să facă publice echipei concluziile evaluării și deciziile luate.

Recomandări pentru etapa de diseminare a concluziilor evaluării și a deciziilor:

1. PM-ul trebuie să evite să facă comparații între membrii echipei sau să propună ca modele sau anti-modele diverse persoane. Acest lucru conduce la distrugerea coeziunii echipei și la generarea de tensiuni, invidii, antipatii în cadrul echipei (NU "*de ce nu poți fi ca …*", "*X este cel mai grozav / slab*")
2. PM-ul trebuie să dea mereu ca model sau anti-model tipul generic de membru al echipei, conform standardelor de lucru
3. PM-ul trebuie să trateze toți membrii implicați în proiect obiectiv, egal și nediscriminatoriu – altfel, membrii echipei se pot simți nerespectați, defavorizați, nedoriți
4. se recomandă ca PM-ul să acorde feedback constant membrilor echipei pentru ca aceștia să fie conștienți de situație, să poată acționa rapid fiind în cunoștință de cauză și să nu fie tensionați la auditurile periodice

Important: Monitorizarea presupune colectarea periodică de informații astfel încât PM-ul să fie la curent cu starea proiectului și a echipei. Monitorizarea nu presupune urmărirea permanentă și insistentă a activităților; acest lucru poate deranja echipa și transforma PM-ul într-un "spion" sau o "moară stricată". O monitorizare bună presupune colectarea de informații cu deranjare cât mai mică a echipei, cât mai puțin intruzivă; echipa are ca motivație să lucreze la proiect nu să răspundă la toate întrebările de detaliu ale PM-ului.

Evaluarea proiectului

- este realizată în principal de *PM*. Contribuie membrii echipei, managementul superior, clienții și subcontractorii
- are loc după etapa de colectare de informații a monitorizării
- evaluarea proiectului se referă la:
  1. **determinarea statusului proiectului** în raport cu planul proiectului (respectarea deadline-urilor, calitatea proiectului și încadrarea în planul de buget)
  2. **evaluarea echipei**.

Planul proiectului cuprinde următoarele:

1. *WBS*-ul detaliat al proiectului (asocierea activități – resurse - durate)
2. planificarea temporală a activităților din WBS (*diagrama Gantt*)
3. *planul de risc* al proiectului (ce riscuri sunt, cum pot fi preîntâmpinate, planuri de acțiune în cazul apariției lor).

Determinarea statusului proiectului (5 min)

Pentru a evalua obiectiv și acurat statusul proiectului, se folosește **metrica SPI**.

SPI (Schedule Performance Index):

- este o măsură a eficienței evoluției proiectului

- se calculează ca raport între **valoarea obținută** (EV – earned value) și **valoarea planificată** (PV – planned value): **SPI = EV / PV**
  - EV răspunde la întrebarea "*Cât de mult s-a realizat până în acest punct din ce s-a planificat*?"
  - PV răspunde la întrebarea "*Cât de mult s-a planificat pentru a se realiza până în acest punct*?".

Interpretarea SPI-ului se face astfel:

- dacă SPI are o valoare *egală cu 1*, atunci starea proiectului este *conform planului* ⇒ proiectul are șanse favorabile de a se încheia cu succes
- dacă SPI are o valoare *supraunitară*, atunci starea proiectului este *înfloritoare*, mai bună chiar decât cea planificată ⇒ proiectul are șanse mari de a se încheia cu succes. În acest caz, este nevoie de analiza situației, determinarea factorilor de succes, stimularea și diseminarea lor și actualizarea planului proiectului.
- dacă SPI are o valoare *subunitară*, atunci proiectul este *în urmă față de planificări* ⇒ proiectul poate deveni un eșec dacă nu se acționează corectiv prompt. În acest caz, este nevoie de analiza situației, determinarea factorilor de insucces, a acțiunilor corective și actualizarea planului proiectului.

Exemplu de calcul SPI

Pentru proiectul de dezvoltare a unor biblioteci Java ce oferă servicii auxiliare față de cele oferite de Sun, pentru etapa dezvoltării bibliotecii de servicii auxiliare de email, s-au planificat activitățile:

- proiectare bibliotecă 100% 4
- dezvoltare bibliotecă 50% 4
- testare servicii oferite de bibliotecă 30% 1
- redactare rapoarte de testare 30% 1.

(activitățile au notate alături informațiile: procentul cât s-a realizat până la deadline și o pondere egală cu efortul necesar de împlinire a activității raportat la efortul total).

În acest caz, calculul SPI este defapt calculul unei medii ponderate: SPI = (4*100 + 4*50 + 1*30 + 1*30) / (10*100) = 0.66 ⇒ proiectul este în întârziere.

Întârzieri ale proiectului (5 min)

SPI-ul poate avea **valori subunitare** în următoarele cazuri:

- *estimările* PM-ului de timp și resurse necesare au fost *departe de realitate*
- *echipa nu și-a îndeplinit sarcinile* planificate (când un membru al echipei conștientizează faptul că nu poate termina la timp sarcina atribuită, are

datoria de a anunța imediat PM-ul pentru ca acesta să poată încerca evitarea întârzierii prin plasarea de noi resurse sau prin alte metode)

- a intervenit un *context nefavorabil* (poate s-au dezlănțuit factori de risc ori s-au creat alte circumstanțe nefavorabile proiectului).

Premisele unei estimări greșite sunt următoarele:

- neînțelegerea scopului propus
- necunoașterea activităților de desfășurat pentru a împlini sarcina
- estimarea pe baza unei alte păreri (îndoielnice)
- estimarea necoordonată între activități
- subestimarea următoarelor probleme:
  - timpul de pregătire / învățare
  - timpul consumat pentru comunicare
  - întâlnirile de proiect
  - control
- Optimism nefondat - nimic nu va merge prost; totul va fi conform planului
- Ignorarea realității – totul merge bine; nimic nu se va schimba în cursul proiectului.

```
Do you know what Murphy's Law is?

"If it can go wrong it will".


Do you know what Sod's Law is?

"It will go wrong in the worst possible way".
```

Recomandări pentru estimări corecte:

- pentru a estima cât mai corect timpul și numărul de resurse necesare unei sarcini, este recomandat să se determine clar și detaliat *ce activități* presupune acea sarcină și *ce efort* este asociat cu fiecare activitate.
- O regulă destul de răspândită în estimarea duratei unui proiect este **calculul om/lună**:
  1. se estimează cât i-ar lua în luni unui singur om să realizeze proiectul (X luni)
  2. se scoate rădăcina pătrată din X și se obține Y, adică Y = mărimea aproximativă a echipei, precum și Y = durata proiectului (în luni).

Se recomandă ca evaluarea proiectului să fie realizată **săptămânal**, pentru proiectele mici și pentru cele medii/mari, **lunar**. Astfel, PM-ul va fi în deplină cunoștință de cauză și va putea acționa **corectiv** la timp.

Metode de abordare a întârzierilor proiectelor (5 min)

Pentru proiectele **întârziate**, se poate alege una din următoarele **abordări**:

1. a fost o *întâmplare* și se va munci mai mult în perioada imediat următoare pentru a recupera, deci proiectul nu va fi afectat
2. *nu se poate face nimic*, așa că terminarea proiectului se amână (în caz că este posibil) cu timpul necesar terminării task-urilor
3. se *asignează* încă niște *persoane* care sa ajute la terminarea taskurilor în timp util.

Fiecare dintre cele trei metode are avantaje si dezavantaje:

1. pentru prima abordare, dacă nu se reușește să se reintre în program, proiectul va acumula întârzieri, crescându-i șansele de eșec
2. a doua abordare nu este tot timpul disponibilă deoarece nu se poate intârzia oricât cu terminarea proiectului, acesta putând sa ajungă irelevant. De exemplu, dacă firma X lucrează în paralel cu firma Y la un produs, chiar dacă firma X face produsul bine, dacă îl termină cu întârzieri față de firma Y, s-ar putea să nu mai aibă clienți.
3. pentru ultima abordare, urmează o dezbatere mai detaliată.

- **Legea lui Brook** spune că dacă se adaugă persoane la un proiect care este în întârziere, proiectul va fi și mai mult întârziat.
- alți autori (*Abel-Hamid și Madnick* în cartea "*Software Project Dynamics: An Integrated Approach*") au protestat față de această lege:
  - ei consideră că adăugarea de persoane nu implică neapărat că proiectul va fi și mai mult întârziat
  - ei susțin că adăugarea de persoane într-un stadiu incipient, cînd persoanele au mai mult timp să cunoască proiectul și să se acomodeze cu echipa ar putea fi benefică, pe când adăugarea de persoane când proiectul este deja în întârziere nu va aduce niciun beneficiu.
- o *problemă* în adăugarea de persoane este *redistribuirea de sarcini*. Trebuie ca tot proiectul sa fie redistribuit pentru noua echipă. Acest proces ar putea să fie foarte complicat și să se piardă mult timp astfel.
- altă *problemă* este faptul că *efortul de comunicare și sincronizare* a unui număr crescut de membri ai echipei ar putea deveni în anumite proiecte inutil de mare, întârziind clar avansul proiectului.

Înainte de a adăuga noi resurse umane pentru un proiect, **PM**-ul trebuie să determine din **ce categorie** face parte proiectul dezvoltat:

1. **Proiect cu etape nepartiționabile** (indiferent de câte persoane lucrează la acest proiect, el se va termina tot în același timp)
2. **Proiect perfect partiționabil** (cu cât se adaugă mai multe persoane, cu atât proiectul este terminat mai repede)
3. **Proiect care necesită comunicare** (se poate observa o îmbunătățire cu adăugarea de persoane, dar comunicarea implicată face ca efectul să fie mai scăzut ca în cazul proiectelor perfect partiționabile)
4. **Proiect cu interconexiuni complexe** (există un minim în acest graf – care corespunde duratei minime a proiectului și numărului optim de membri ai echipei; dacă se adaugă mai mulți membri, defapt, se îngreunează munca și timpul de terminare a proiectului crește)

Evaluarea echipei (5 min)

- este o activitate esențială pentru că echipa *influențează major evoluția proiectului*
- se referă la **evaluarea întregii echipe**, dar și la **evaluarea individuală** a membrilor echipei
- astfel de evaluări se realizează periodic pentru a determina eventualele probleme spre a le soluționa și preveni, pentru a recompensa și stimula membrii echipei
- în companii, evaluările individuale detaliate se realizează la terminarea proiectelor și în cadrul auditurilor anuale; ele sunt incluse în arhivele individuale de performanță și constituie baza deciziilor de promovare, asignare de sarcini și bonusuri.
- evaluarea *individuală* se începe prin a îi cere persoanei să se *autoevalueze* (sunt șanse mari să se descopere informații noi despre persoana respectivă – se poate evalua foarte corect, încearcă să mușamalizeze, etc.)
- evaluarea individuală devine un proces mai complicat și laborios cu cât poziția persoanei este mai înaltă în companie: un membru al echipei va fi evaluat de ceilalți membrii ai echipei și de PM; PM-ul va fi evaluat de către membrii echipei, de clienți, de subcontractori, de managementul superior.

În faza de evaluare, PM-ul realizează:

- **concluzionarea** asupra statusului proiectului

- **evidențierea** *problemelor* și a potențialelor surse de probleme
- evidențierea activităților împlinite cu un *real succes*, deci a resurselor umane implicate și a contextului de lucru

Important: Nu există forme de evaluare perfectă. Orice formă de evaluare este contestabilă. Orice PM va adăuga propriile informații și intuiții în cadrul evaluării, fără a le prioritiza însă. Evaluarea și feedback-ul sunt necesare pentru buna evoluție a proiectului și trebuie tratate cu seriozitate. Evaluarea presupune atât determinarea părților bune cât și a celor slabe. Ulterior evaluării, concluziile sunt de forma: aici stai bine, aici mai ai de lucrat.

Control (5 min)

- este realizat în principal de PM. Contribuie clienții, managementul superior.
- are loc după finalizarea evaluării proiectului
- este o acțiune constantă

În faza de control, PM-ul realizează:

- **raportare** (starea proiectului, planuri de acțiune/schimbare) către nivelul superior de management și către client
- **găsirea de soluții** pentru probleme și de măsuri de evitare a potențialelor probleme și de contracarare a acestora în cazul în care nu pot fi prevenite
- găsirea de modalități de **stimulare a factorilor de succes** (recompensarea celor mai buni din echipă, îmbunătățirea contextului general de lucru în funcție de modelul observat)
- **clarificarea planurilor** viitoare de acțiune
- **actualizarea planului** proiectului, deci implementarea deciziilor de schimbare (corective sau stimulente)

Urmează etapa de diseminare a concluziilor evaluării și a deciziilor de schimbare către echipă.

Important: Controlul este o buclă de feedback pentru acțiunile de monitorizare și de evaluare, așa cum reiese și din schema de mai jos. Controlul presupune "ținerea trenului pe șine". Atunci când apar probleme, PM-ul trebuie să decidă modul de soluționare; de dorit este să prevină apariția acestora, având din timp obținute informațiile necesare. PM-ul trebuie să evite să se transforme într-un "control freak" dar trebuie să evite să fie "devil may care"; echipa trebuie lăsată să lucreze, iar PM-ul să intervină doar când este cazul; eventual poate decide să lase pe cineva să greșească ca proces de învățare. Un PM care vrea să controleze prea mult va demotiva echipa, membrii considerând că sunt tratați ca niște copii; un PM care are un nivel de control redus va fi considerat lipsit de interes și de autoritate și echipa va fi, la fel, demotivată.

7 secrets to master timeboxing

Timeboxing is a simple technique, but it has its challenges when implemented in practice. This article lists some typical challenges when timeboxing and how to master them. It's a destillation of many years of experience in applying the technique in practice.

What is timeboxing?

Timeboxing is a very simple technique to manage time and become more productive. The idea is to allocate a certain amount of time to an activity in advance and then complete the activity within that time frame. One example is to plan a meeting agenda and determine up front, how long you want to work on each topic. During the meeting you then set a timer and when time is up, you move on to the next topic. In theory it sounds simple, but in practice it can be hard. Let's look at some typical challenges and how to master them…

1. Hard and soft time boxes

> A deadline is negative inspiration. Still, it's better than no inspiration at all.
>
> –Rita Mae Brown

There are two types of time boxes: „hard time boxes" and „soft time boxes". The terms *hard* and *soft* refer to how you handle the end of a time box:

va urez restanta usoara zgarcitilor

- When a *soft* time box ends, you allow the current point in discussion or the current task at work to finish.
- When a *hard* time box ends, you drop everything, stop doing what you do and take a break or move on to the next point on the agenda.

Both types are useful.

The alarm at the end of a soft time box gives you a hint that you should come to a conclusion, a result or the end of a task. It brings the focus of the group back to the point in questions and reminds everyone, that the time scheduled for this point or task is up. The drawback of a soft ending is the danger of continuing the discussion or work far longer than what is sensible — not seldom multiple times longer than the original time box had been. The soft way works well when first establishing the idea of time boxing with a team, because it allows them to adjust to this new tool without being forced to cut discussions short in a rude way.

A hard time box often leads to agitated utterances like „but we're not finished yet!" or „we need more time!". This reaction is intentional! Teams who don't have much experience with timeboxing have to learn it and get a feel for time. They will run into „time's up" events without having produced any results for the agenda point in discussion or the task at hand. But you will see, that after a few of these events, they will gradually become more efficient within the time box. They will remind each other „oh, we only have x minutes left — let's come to a conclusion now". And they will arrive at results before time is up.

Be clear whether you want to use a soft or hard time box and why.

2. Visual time

va urez restanta usoara zgarcitilor

Almost nothing need be said when you have eyes.
–Tarjei Vesaas

Make the time box visible! Use a kitchen timer or an iPad app so everyone in the room can see it. Use countdown timers instead of a regular clock to make the time remaining easy to grasp. The best timers use visualizations instead of raw numbers to clearly communicate the time left.

Another big benefit of a visual time indicator is that everyone in the room becomes a time keeper. The moderator is not left alone with reminding people to focus and to come to conclusions. You will see different people from the team pointing towards the timer and reminding the group to return to the topic. The timer gives everyone the authority to cut disucssions short, while without the timer, many people would hesitate to remind their boss about focussing.

Make the time box visible to everyone.

3. Plan in advance

I would like to allocate more time to dating, though. I need to find a girlfriend. That's why I need to carve out just a little more time. I think maybe even another five to ten – how much

time does a woman want a week? Maybe ten hours? That's kind of the minimum? I don't know.
— Ashlee Vance (Elon Musk: Inventing the Future)

Announce the time you give each topic in advance. For a workshop or meeting, plan the time for each topic before it starts. Think about how much time you're willing to give a topic, so that you won't get caught up in distorted priorities; we all know these long and interesting discussions about a topic that noone wants to cut short, yet later everyone realizes that they spent half a day on a rather unimportant topic. Avoid this by setting the time — and with this the time/value ratio — for each topic in advance.

If you don't have an agenda in a spontaneous meeting, take a minute to think about and decide how much time you're willing to give the next topic. Also communicate the planned time box clearly to everyone. People need to know how much time they have.

Plan the time you want to give a topic in advance and announce it.

4. Maximum length

It is not the length of life, but the depth.
— Ralph Waldo Emerson

va urez restanta usoara zgarcitilor

Is there a maximum length for a time box? Yes. I believe, a time box should not be longer than 45 minutes. This interval fits with our biological rhythm and is also supported by psychology. There is a reason why many schools around the world have kids take breaks after 45min. A longer stint would drain the team's energy in the long run. Go for 45min max and take short breaks of 5–15min in between. If you need more than 45min for a topic, simply break the topic down into two or more time boxes.

Is there a minimum length as well? Of course you can use any time below the 45min mark for topics that don't need as much time: 30min, 20min, 12min, … . But is there a lower limit? No, I don't see one. Sometimes it makes perfect sense to break a task down into small intervals of merely 10 seconds, like „take 10 seconds to write down your first thoughts on topic 1 of 6, then move on to topic 2 and repeat". Get creative with the use of time boxes. It does not always make sense to break down a topic into small time boxes, but sometimes it just makes perfect sense.

Stick to a maximum length of 45 minutes.

5. Breaks

Sometimes I do need to go to karaoke, sometimes I need to relax.
– Jackie Chan

Breaks are important! As I already mentioned, take short breaks every 45min. I would also advice you to take slightly longer breaks (e.g. 15min) every 90min. We often structure workshops by giving

va urez restanta usoara zgarcitilor

each agenda point 60 minutes: 45min work + 15min break. This also makes planing with full hours very easy.

If possible, move around or get outside every 90min. Don't just sit around and check your emails during breaks. Moving, fresh air and seeing something different (even if it's just the bathroom) keeps the brain fresh and ensures enough energy for a full workshop day.

Make it clear to everyone when the break will end! Do this *before* everyone leaves the room. You don't want to wait another 10min after a 5min break until everyone is back and you can start working again. State the end time of the break, not its duration! Instead of announcing „5min break", phrase it as „break until 11:48". This works, because many people don't look at their watch once the break starts. When they look at their watch after some time, they don't know when the break started. Then one of two things happens: they either fall back to „feeling" when the break is over or they measure the length of the break from the point they first looked at their watch (this is why such breaks often take exactly double the time you've given them). So make it easy for everyone to be back on time and announce the end time, not the duration of a break.

Take breaks and announce the end time of the break.

6. What can be time boxed?

Pizza makes me think that anything is possible.

– Henry Rollins

A time box is not only meant for workshops and meetings. Time boxing is also used sucessfully on a larger scale of days and weeks,

like in a Scrum sprint (1–4 weeks) or Google's Design Sprint (5 days). Milestones and deadlines in projects or product development are also forms of large time boxes. They all help you make decisions and stay focused on what's most important.

Meetings are seldomly time boxed correctly. Most organizers will tell you the start time of a meeting, but not the end time. If there is an end time, it is often not set on purpose and there is no intention to honor this end time either. But if you really plan a meeting's duration up front, you can dramatically improve its usefulness. You won't spend more time than is worth in a meeting, people will actually look forward to meetings and you will be able to plan appointments after the meeting without having to calculate considerable „buffer" time.

Some argue that there is one even bigger and very effective time box given to us all: life and death. But the bigger the time box, the less it serves to help us focus on the important stuff. Steve Jobs really felt the pressure to focus when his time box shrank dramatically due to his illness (Steve Job's commencement speech at Stanford). Some people benefit from visualizing this large but nontheless endless timebox in a life calendar. It's certainly interesting to look at life as a huge time box.

I also use time boxes in my daily solo work. The pomodoro technique is a great way for me to stay productive throughout the day. I work for 25min straight and then take a short 5min break to get up and relax. I can manage to stay focused for 25 minutes without taking calls, going to the bathroom or checking twitter — almost everything can be postponed for 25 minutes. My brain has enough energy to really be productive during these 25min. The days when I use the pomodoro time boxes are the ones I am most productive overall, still feel fresh in the evening and look forward

to tackling the next work day. I even measure the time my work took me in „pomodori“.

Pecha kucha presentations or the famous TED talks also benefit from timeboxing. They help the presenter to stay focused and they allow an audience to sensibly digest several talks during a single day. If you do presentations, aim to keep them under the 45min mark. Even if it’s not a pecha kucha or a TED talk, make it a challenge to turn it into one of these even shorter presentation formats. You’ll see that it greatly benefits your final presentation.

Time boxes are universally aplicable. I’m sure there’re even more things that can benefit from time boxing out there.

7. Time is up

Never interrupt your enemy when he is making a mistake.

— Napoléon Bonaparte

I already talked about the end of a soft or a hard time box before. Especially with hard time boxes, people will be frustrated and sometimes almost panic about not being able to „finish“ the discussion or task. As a facilitator or moderator, you need to stand your ground and make clear that time boxes are not a mere embellishment. People will learn and thank you over time, but it can be tough in the beginning.

A good practice is to give a short warning before time is up. You can either verbally announce a „2min warning“ to the team or even

va urez restanta usoara zgarcitilor

better use a timer that does this for you (I currently use the TimeBoxing app on an iPad).

When time is up, you have two options: move on to the next topic or start a new time box for the current topic. What you do is solely at your discretion. The important thing to remember is that you don't let yourself get caught up in the moment and what „seems important right now". Rather trust your preparation when you allocated time for each topic depending on its importance. At the very least, take some time when the alarm rings to quickly think about or discuss, whether it will be worth to give this topic another x minutes. Pull your head out of the work and stick it up onto a strategic level for a minute and don't let the work inadvertantly alter your strategic goals.

Moving on to the next topic even when there is no result or conclusion for the current topic can be a tool to become more productive. If a team has been frustrated a few times with not having any results for a point on the agenda, they will quickly adapt and come to results earlier in the time box. Especially if you introduce time boxes as a coach, you should make use of this educational effect.

Use the end of a time box to remember your strategic goals.

Benefits of timeboxing

Timeboxing has many benefits:

- Focus: stay focused and do the important stuff
- Stay fresh: stay innovative and productive, even on the next day

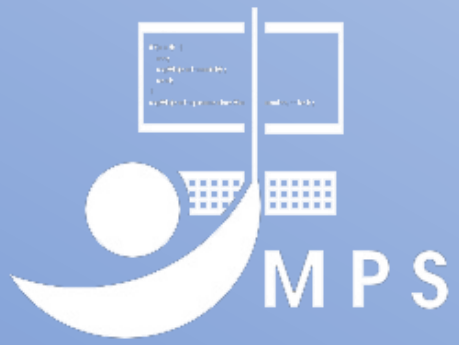
va urez restanta usoara zgarcitilor

- Breaks: time boxes allow to have regular breaks
- Measurable progress: have clear results
- Predictability: ability to have appointments right after a meeting, because you *know* when it ends
- Everyone is a leader: time boxes are an „excuse“ for anyone on the team to cut a fruitless discussion short and get back to the point
- Results: time boxes force results
- Liberating: when you got used to it, limiting time feels liberating instead of limiting

If you apply time boxes in the right way, they’re an invaluable tool for productivity and fun. As a coach and facilitator I use this technique all the time! And I hope you also learned some new things about time boxing through this article. Cheers!

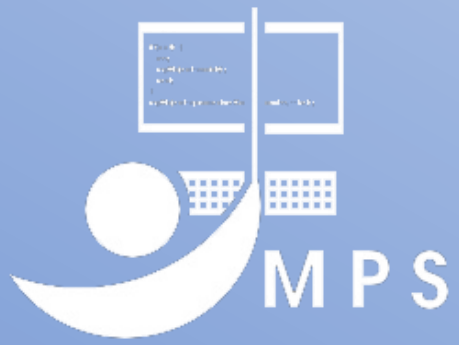
va urez restanta usoara zgarcitilor

Managementul Proiectelor Software



Curs #0

Introducere



The conventional definition of management is getting work done through people, but real management is developing people through work.

Obiective

- Activitatea unui **Manager de Proiect** în domeniul dezvoltării de software
- Vedere de ansamblu asupra rolului, metodelor de management și a responsabilităților
- Nu este necesară cunoașterea prealabilă a tehnicilor de management
- Dedicat studenților care doresc să își dezvolte aptitudini, stiluri și abordări specifice în această arie
- Necesită cunoștințe de bază referitoare la analiza programelor și algoritmilor

Componente

- Curs
  - Noțiuni teoretice
  - Elemente de bază de management de proiect
- Laborator
  - Noțiuni aplicate de management de proiect
  - Lucru în echipă
  - Doua proiecte pe semestru în echipă

Resurse

- Wiki-ul cursului
  - http://ocw.cs.pub.ro/courses/mps
- Portalul http://cs.curs.pub.ro
- Sala de laborator EG-405A

Echipa MPS

Nume	E-mail
Costin BOIANGIU	**costin.anton.boiangiu@gmail.com**
Răzvan DEACONESCU	**razvand@gmail.com**
Giorgiana VLĂSCEANU	giorgiana.vlasceanu@gmail.com
Iulia STĂNICĂ	iulia.stanica@gmail.com
Costin SPECIAC	costin.speciac@gmail.com
Laura ANTONACHE	lauraantonache@gmail.com
Mihai SOARE	soare.mihai1992@gmail.com
Alexandru BREBU	alex.brebu13@gmail.com
Marius BÎNĂ	bina.marius@gmail.com
George OPREA	georgeo4567@gmail.com
Diana NIȚĂ	diana.nita.luiza@gmail.com

Evaluare

- Pentru absolvirea materiei trebuie să obțineți cel puțin 50% din punctajul de curs (lucrări de curs și examen final) și cel puțin 50% din punctajul de laborator (laborator și teme).
- Notare
  - Curs – 4 puncte
  - Activitate parcurs (laborator + proiect) – 6 puncte

Evaluare

- Curs (4 puncte)
  - **3.5p**: examen final (se poate lua 10 fără bonusuri, prin rotunjire superioară, doar dacă a fost obținut 10 la fiecare activitate în parte)
- Alternative/Extra/Bonusuri:
  - Articol de cercetare referitor la abordări de proiecte din domeniul MPS
  - Prezentarea (pe baza unei programări) a unui articol științific relevant preluat de pe internet
  - Prezentarea (pe baza unei programări) a unui curs din cadrul materiei
  - Participarea la o dezbatere pe o temă dată
  - Diverse intervenții, răspunsuri la întrebări, interacțiuni în cadrul “cafenelelor publice” sau discuții la filme si prezentările colegilor.
  - Prezenta la curs

Detalii Alternative/Extra/Bonusuri:

A. **4p**: printr-un articol de cercetare referitor la abordări de proiecte din domeniul MPS (se pot înscrie echipe de 2-3 studenți, se oferă material suport și detalii la cerere), se prezintă articolele în ultimele 2 săptămâni (cu **echivalarea** examenului final)

B. **1.5p**: Prezentarea (pe baza unei programări) a unui articol științific relevant preluat de pe internet (se pot înscrie studenți individual, sunt oferite câteva exemple de articole, orice subiect din domeniu este însă valid, încurajăm împărtășirea experiențelor personale)

C. **1p**: Prezentarea (pe baza unei programări) a unui curs din cadrul materiei (se pot înscrie echipe de 2 studenți, se oferă prezentări PPT ce pot fi ajustate pentru o interactivitate cât mai ridicată)

D. **0.75p:** Participarea la o dezbatere pe o temă dată: (4-6 studenți împărțiti în 2 tabere)

E. **0.5p**: intervenții, răspunsuri la întrebări, interacțiuni în cadrul “cafenelelor publice” sau discuții la prezentările colegilor.

F. **1p:** prezenta la curs

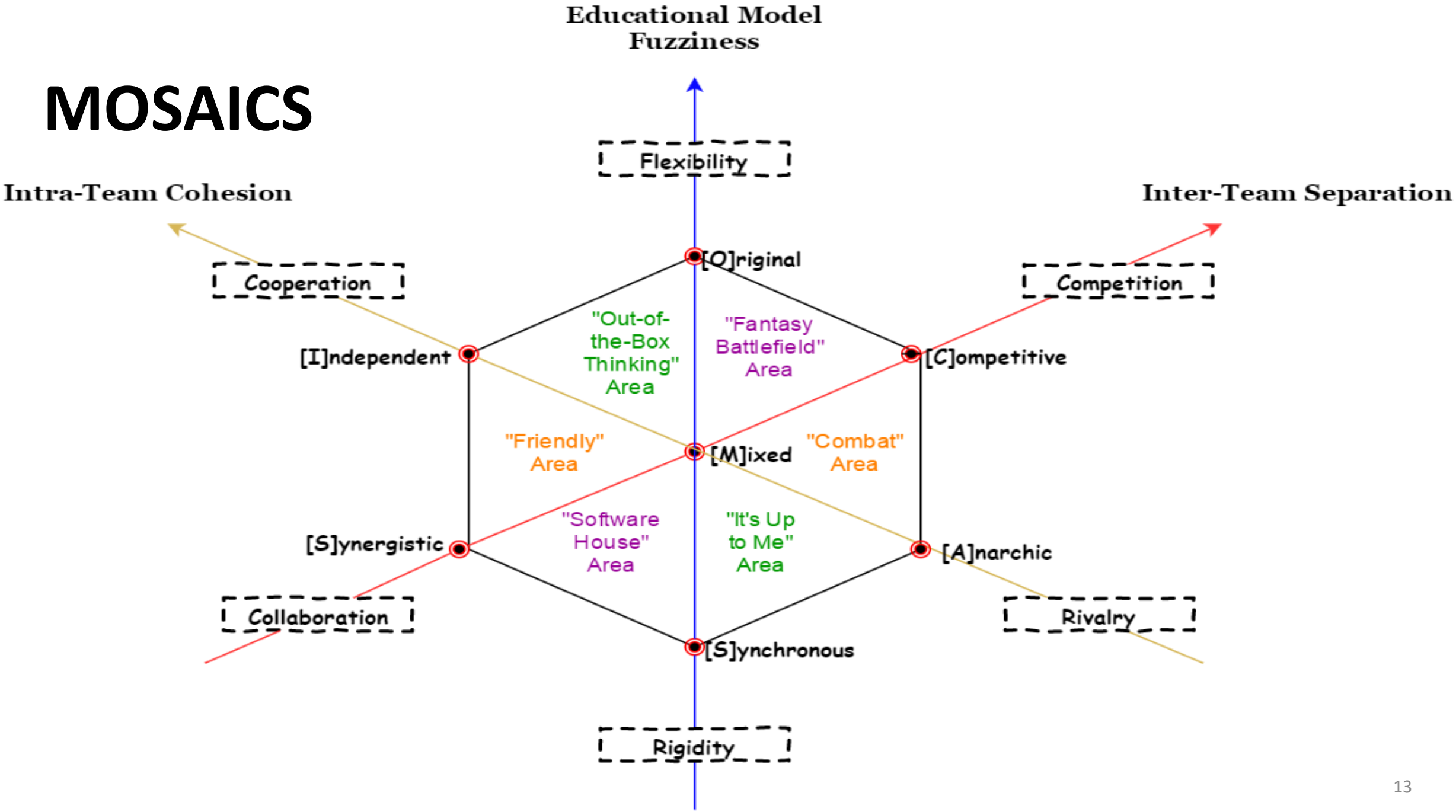
Evaluare

- Laborator + Proiect (6 puncte)
  - **3p** Proiect 1
    - **0.5p** activitate individuală în cadrul proiectului
    - **1p** respectarea deadline-urilor individual
    - **1p** rezultat final (respectarea cerințelor și specificațiilor, funcționalitate minimală a proiectului, notă per echipă)
    - **0.5p** impresia finală, nota colegilor și prezentarea.
  - **3p** Proiect 2 (repartizate identic)
  - **0.5p** bonus pentru intervenții și activitate remarcabilă în cadrul laboratoarelor.
- În total ar fi 6.5 puncte pe laborator care se trunchiază la 6p
- Se evaluează atât activitatea inidividuală cât și cea în echipă.
- Detalii pot fi găsite la: http://ocw.cs.pub.ro/courses/mps/reguli

Abordari educationale variate

- Curs
  - Studii de caz / Probleme de tip examen
  - Prezentarea părții teoretice a cursului
  - Prezentări articole de cercetare
  - Redactarea și prezentarea unui articol de cercetare
  - Dezbateri
  - Cafenele publice
  - Filme comentate
- Proiect
  - Încercăm să găsim cea mai bună modalitate de a învăța studentii aspecte practice legate de lucrul în echipă și management de proiect software

MOSAICS


Educational Model
Fuzziness
Flexibility
Intra-Team Cohesion
Inter-Team Separation
Cooperation
Competition
[O]riginal
[I]ndependent
[C]ompetitive
"Out-of-the-Box Thinking" Area
"Fantasy Battlefield" Area
"Friendly" Area
[M]ixed
"Combat" Area
[S]ynergistic
[A]narchic
"Software House" Area
"It's Up to Me" Area
Collaboration
Rivalry
[S]ynchronous
Rigidity

Abordarea Independentă (“Independent”)

Propuneri proiect

- Mortar Mayhem
- Fastest and Brightest
- Battleship
- E-commerce Lite
- SimTraf
- Stonemason
- TTSched
- Analizor de imagini
- Circuit Designer
- Aplicație pentru teste grilă

Planul abordării independente

- Solicitare Proiect → scopul documentului, conţinutul documentului
- Descrierea sumară a produsului software
- Descrierea soluţiei → arhitectura soluţiei, tehnologii şi tooluri de dezvoltare
- Experienţa echipei în domeniul produsului
  - Cunoştinţele tehnice ale echipei (Advanced, Medium, Beginner)
  - Experienţa echipei
- Estimări resurse necesare
  - Determinarea rolurilor din cadrul echipei (ex: Developer, Project Manager, etc.)
  - Determinarea distribuţiei echipei per roluri (câte persoane au acelaşi rol?)
  - Determimarea efortului depus de fiecare rol într-o săptămână (x ore/săpt)
- Prezentare calendar livrări propuse
- Motivare echipă.

Abordarea Concurențială (“Competitive”)

- Realizarea unui sistem de simulare a competițiilor între mașini de Formula 1.
- Echipele vor alinia la startul unei curse câte două mașini fiecare. Pentru implementare se menționează o serie de caracteristici și parametrii. Exemplu de parametrii:
  - Culoarea (specifică echipei)
  - Acceleratie (m/s2) = ACC
  - Frana (m/s2) = BRK
  - Viteza maxima (m/s) = SPD
  - Manevrabilitate (rad/s) = STR
  - Constrângere (default): 10*ACC+10*BRK+SPD+200*STR = 400
  - Modul de strategie conducere vehicul (AI)

Desfășurarea competiției

- Vor fi desfășurate un număr de curse egal cu numărul de echipe. Fiecare echipă va alinia în cursă două mașini cu setări (posibil) diferite și strategii (posibil) diferite;
- Competiția cu un mecanism bazat pe AI. Punctaje acordate: loc 1 – 7 p, loc 2 – 4p, loc 3 – 3p, loc 4 – 2p, loc 5 – 1 p, ultimul loc - -1p, descalificare: -2p;
- Între masina cu cel mai bun rezultat al unei echipe (MAX) și cea cu rezultatul mai slab (MIN) punctajul acumulat de echipa va fi: (MAX * 7 + MIN * 3) / 10
- După terminarea curselor se efectuează clasamentul prin însumarea punctelor obținute
- Primele două echipe vor participa într-o nouă cursă, pentru departajare, fiecare cu câte o singură mașină

Componente. Permisiuni la nivelul interfeței – aplicației

- Încărcarea hărților
- Alocarea punctelor și orientărilor de start, a numărului de ture, a timpului de referință, a sensului de parcurgere, etc.
- Alegerea culorilor participanților
- Conexiunea cu serverului și actualizarea continuă a situației în funcție de pozițiile venite de la server pentru fiecare mașină
- Nu efectuează calcule ci doar citește poziții specificate prin coordonate în sistem metric și afisează mașinile în pozițiile specificate utilizând culoarea selectată pentru fiecare echipă
- Afișează rezultatele cursei, punctejele obtinute și clasamentul provizoriu
- Afisează harta, vehiculele, timpul scurs de la start, parametrii fiecărui vehicul și ordinea în cursă. Afișarea se face în timp real, în mod grafic, full-screen

Componente. Server

- Inițializează conexiuni cu interfață cât și cu cele 2**X**N mașini ale celor N echipe în cazul unei confruntări normale sau ale celor 2 mașini pentru finală. Conexiunea celor 2**X**N procese-mașini se pot iniția de la mașini către server (fizica aplicației) sau invers. Probabil că este mai ușor conexiuni de la procese-mașini către.
- Așteaptă la inițializarea modulelor AI, după transmiterea hărții către acestea, un interval de 30 de secunde pentru calculul parametrilor mașinilor
- Comunică aplicației starea în cursă: ordine, timpi, poziții, parametri de rulare pentru fiecare vehicul în parte
- Citește periodic de 10 ori pe secunda de la fiecare modul de AI deciziile de conducere ale vehiculului asociat, calculează noile poziții ale acestuia, noul unghi de orientare al volanului și confirmă AI noile poziții ale tuturor participanților, viteza curentă și noile direcții de deplasare (X, Y, speed, steer)
- Limitează viteza de deplasare a unui vehicul la SPD
- Calculează noile poziții ale vehiculelor, unghiurile de orientare pe baza unei aproximări discrete corespunzătoare împărțirii unei cuante de timp de 1ms

Componente. Strategia de deplasare (AI)

- Comunica server-ului de 10 ori pe secunda decizia curentă de conducere a vehiculului si anume:
  - Franare în procent (0-100] din BRK sau accelerație în procent [0-100] din ACC
  - Cerere traiectorie de deplasare în unghiul [0-2*PI) (măsurat în valoare absolută față de **OX**), volanul va fi mișcat către poziția cerută cu viteza STR , în sensul ce asigură o rotație minimă

Evaluarea echipei

- Evaluarea se va realiza în cadrul a 4 milestone-uri:
  - S04 → Specificațiile proiectului (SRS – “Software Requirement Specifications”, SDD – “Software Design Document”, Arhitectură) (0.5 puncte)
  - S06 → Aplicația de vizualizare a mașinilor (0.5 puncte)
  - S10 → Serverul (fizica aplicației) (1 punct)
  - S14 → AI engine-ul și competiție; primele două echipe au echivalat examenul final cu punctaj maxim (1 punct)

Abordarea Colaborativă (“Synergistic”)

Sistem Automat de Analiză şi Extragere a Informaţiei din Documente

- Transformarea documentelor scrise (ex: cărţi, ziare, reviste, facturi, hărţi) în documente electronice (ex: PDF, RTF) utilizabile pe o gamă largă de dispozitive, inclusiv mobile
- Conservarea meta-informaţiei şi a aspectului original (arhitectura multi-layer a documentului)
- Generarea conţinutului paginilor, layout-ului şi a ierarhiei documentului
- Generarea informaţiilor relevante pentru efectuarea unor căutări inteligente

Intrare

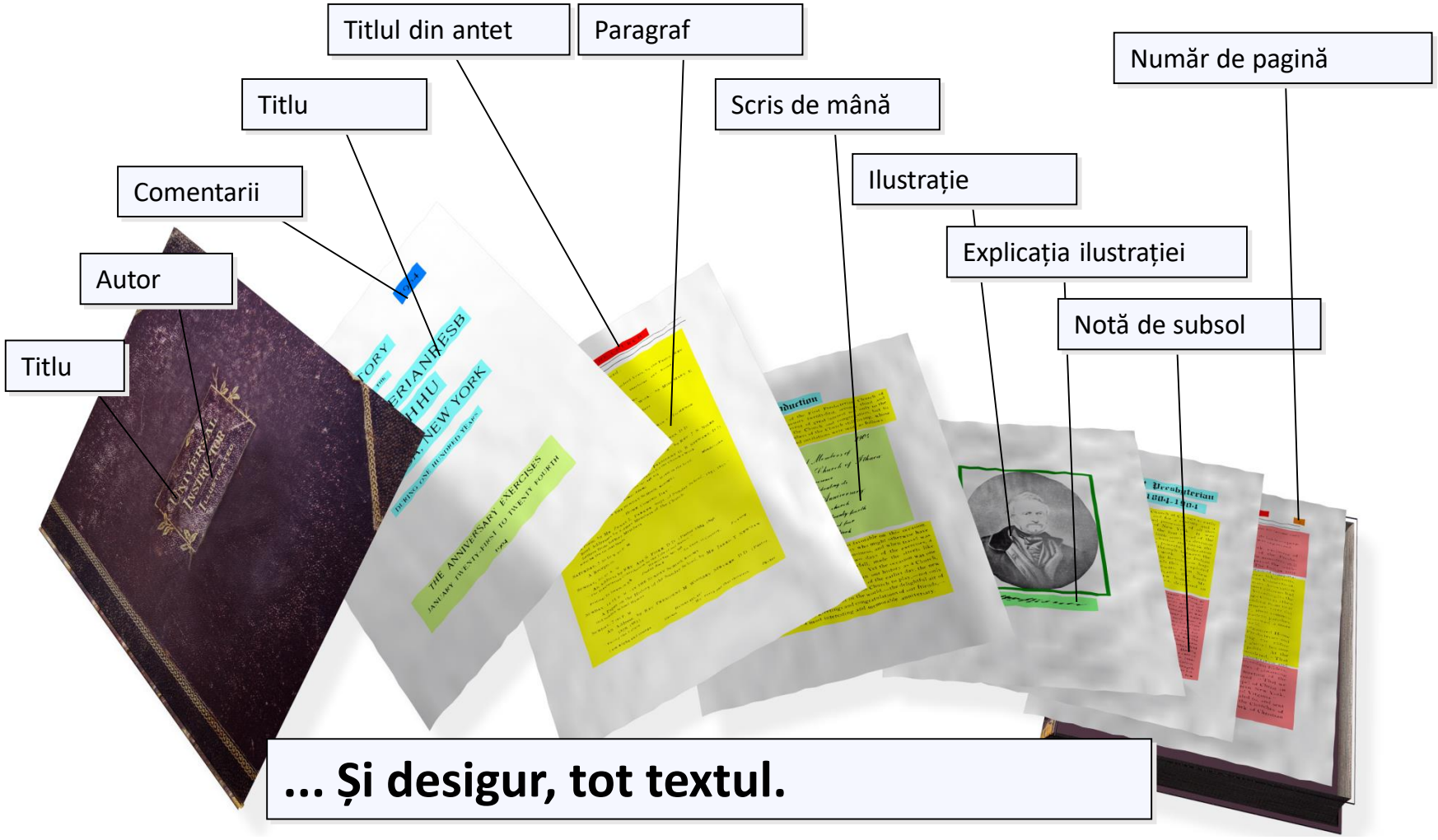
Ieșire

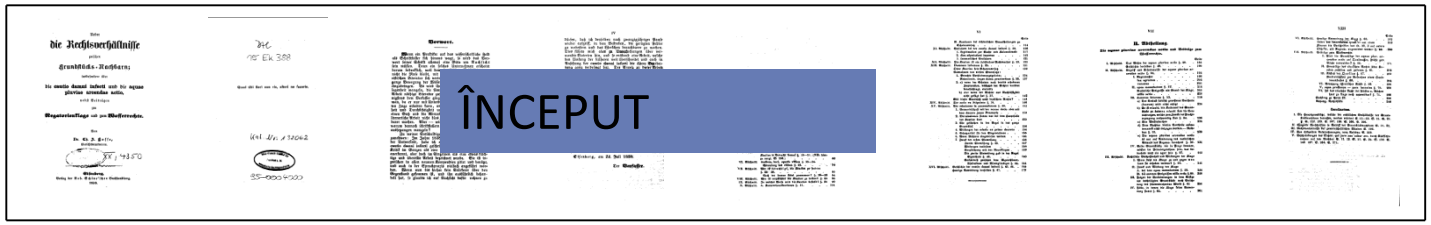
document

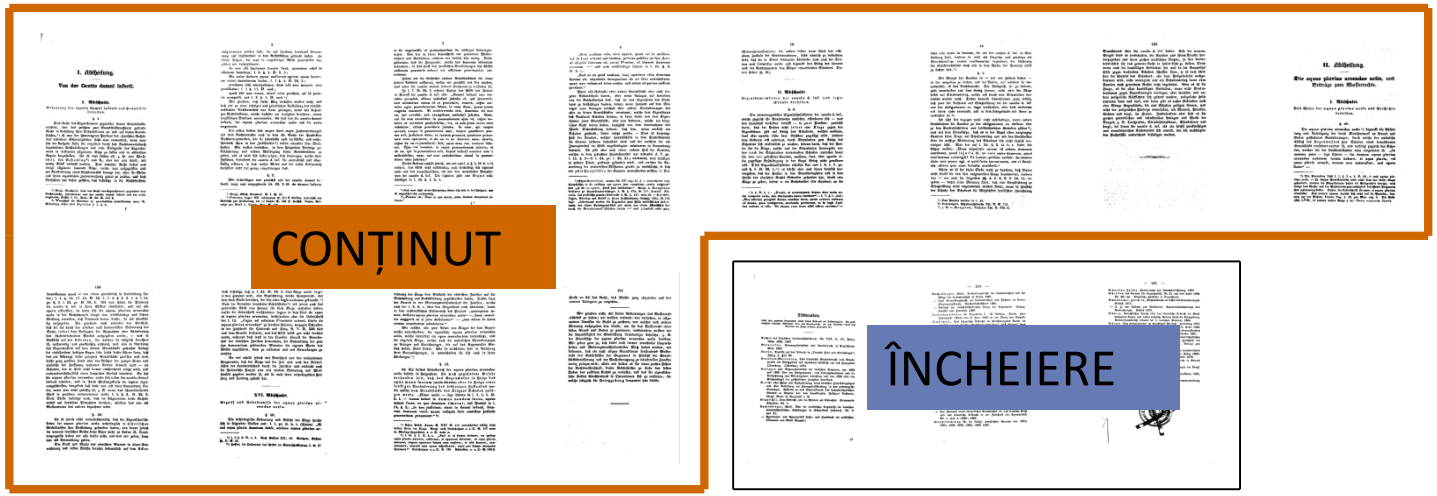
Scanare	Pre-procesarea imaginii	Corecție
	Analiza elementelor	
Import	Recunoașterea optică a caracterelor (OCR)	Export
	Analiza structurală	

Reguli
DB

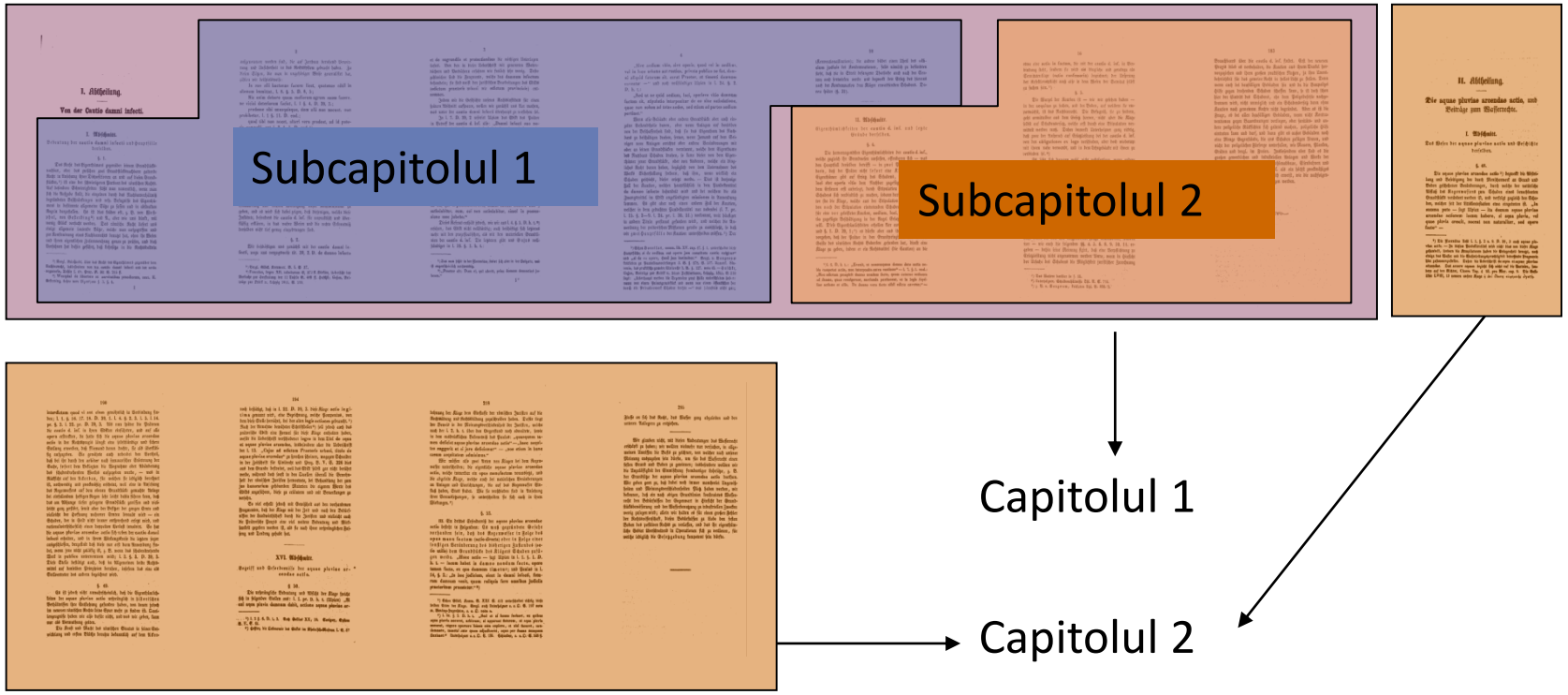
PDF
METS/ALTO
(TIFF+JPEG)
XML (MARC)


Titlul din antet
Paragraf
Titlu
Scris de mână
Număr de pagină
Comentarii
Ilustrație
Autor
Explicația ilustrației
Titlu
Notă de subsol
... Și desigur, tot textul.

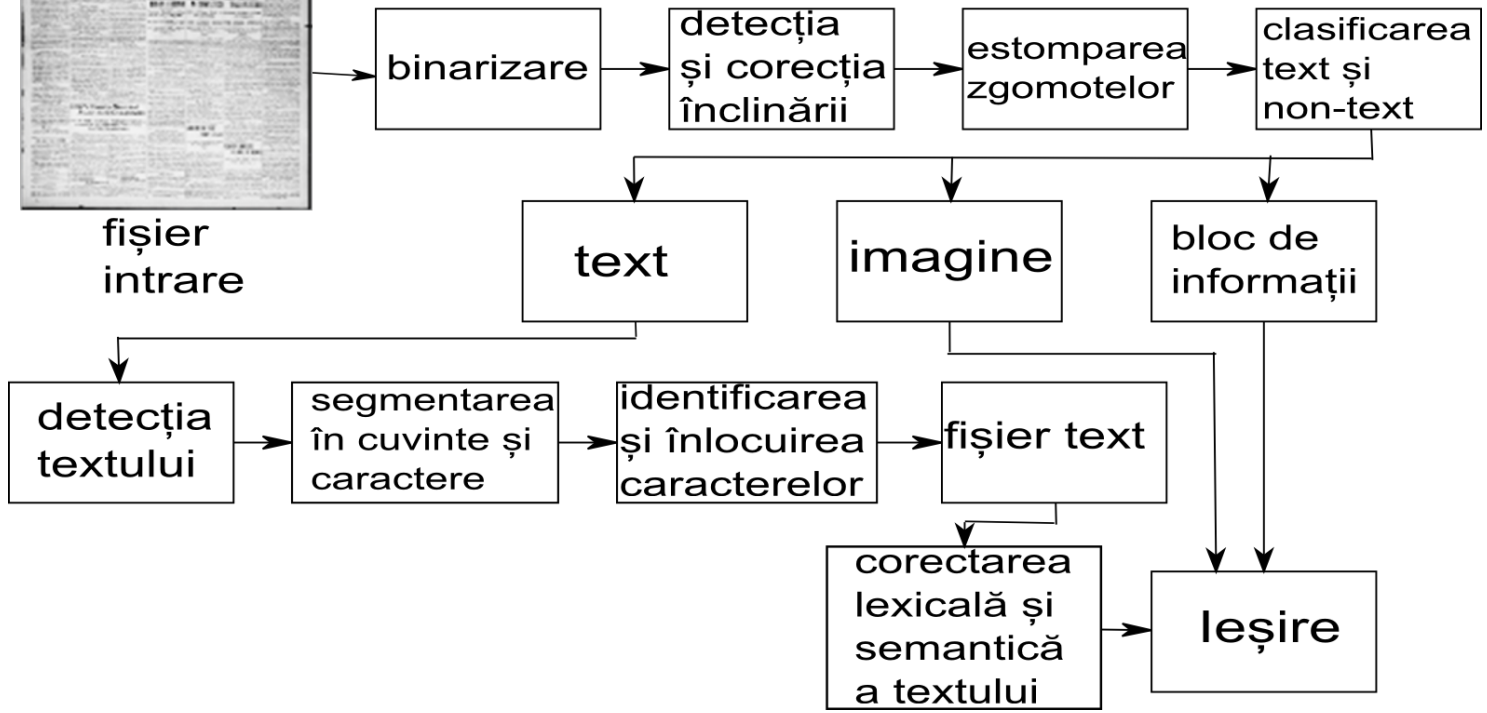

ÎNCEPUT


CONȚINUT

ÎNCHEIERE


Subcapitolul 1
Subcapitolul 2
Capitolul 1
Capitolul 2

fișier intrare



Abordarea Mixtă (“Mixed”)

- Construirea unui sistem de prelucrare de imagini bazat pe o strategie de votare
- Prelucrarea constă în transformarea unei imagini color (o pagină scanată de ziar) într-o imagine bitonală, astfel încât să fie evidențiat cât mai clar textul. Imaginea bitonală era trimisă spre OCR pentru a fi convertită în text digital. Cu cât erau recunoscute mai multe caractere, cu atât rezultatul era considerat mai bun.
- Echipa împărțită în patru celule: 3 celule care ofereau trei algoritmi diferiți de conversie și o celulă care combină cele trei rezultate într-unul mai bun decât oricare din cele trei rezultate inițiale.

Exemplu

(a) Imagini reprezentative din setul de dat cu:

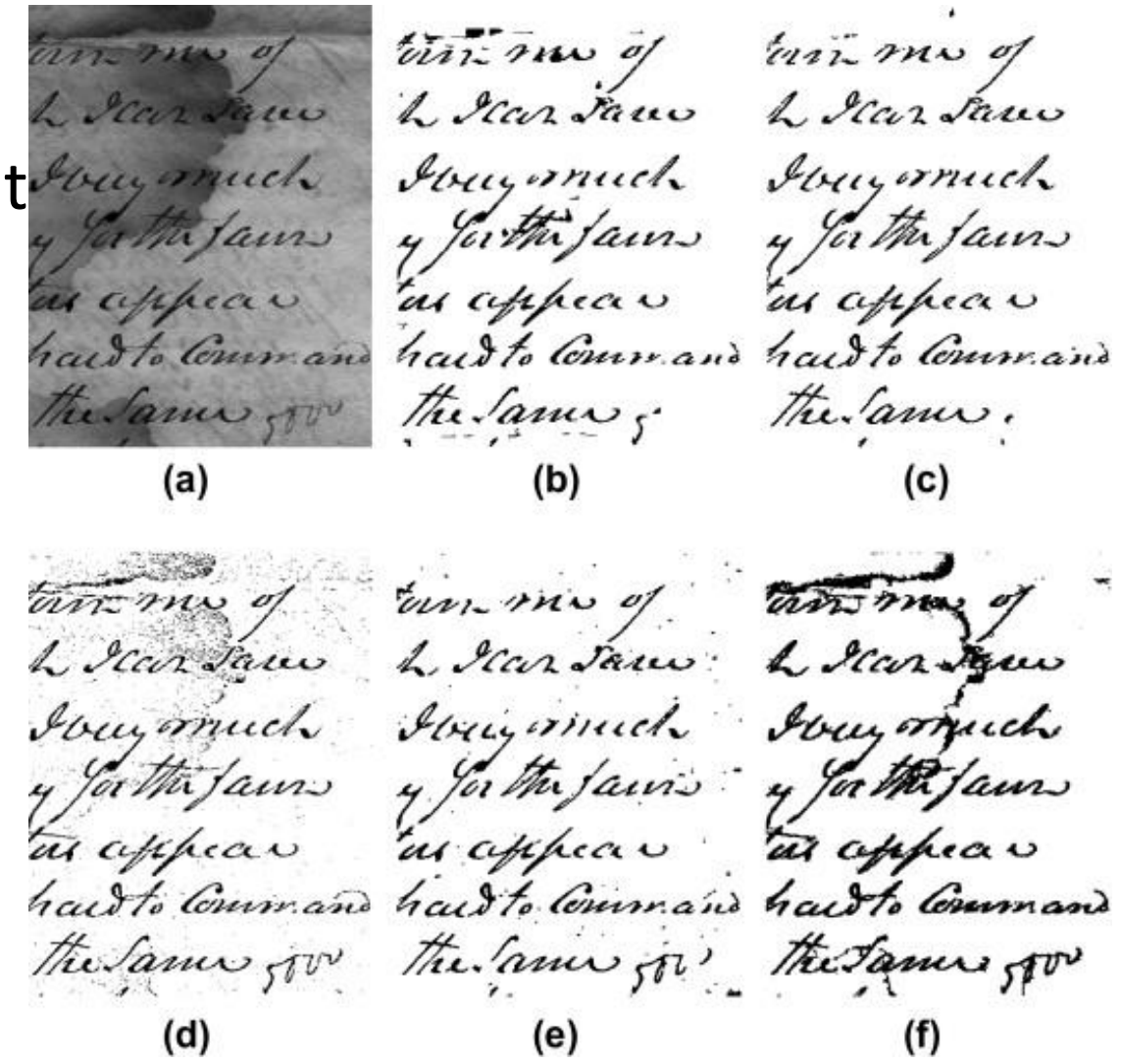
(b)Messaoud et al. (2011);

(c) Su et al. (2010);

(d) Ntirogiannis et al. (2009);

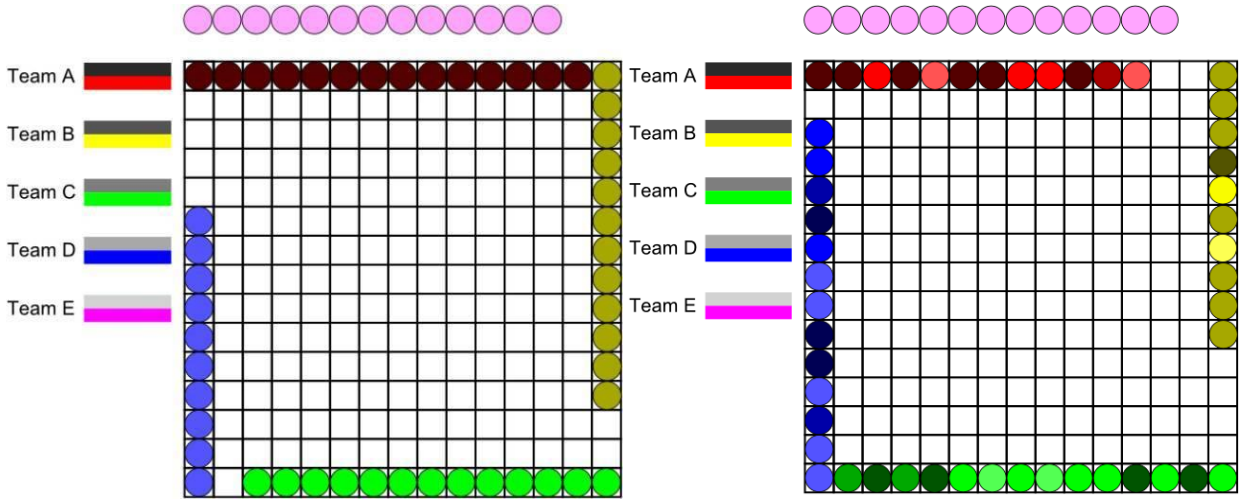
(e) Howe (2011);

(f) Gatos et al. (2008).



Abordarea Anarhică (“Anarchy”)

- Board game, în care un pion este executabilul unui singur student, un cod care conține euristici pentru jucare și mecanisme de comunicare cu serverul și ceilalti pioni.
- Studenții dintr-o grupa vor avea o luminozitate comună și o culoare, care se schimbă în funcție de echipa la care joacă.



Abordarea Artistica (“Originality”)

- Scopul proiectului este de a obține o imagine cu o valoare “artistică” de un cât mai ridicat nivel de exprimare utilizând doar generare procedurală, geometrie fractalică, și algoritmică simplă.
- Durata de rulare este setată la **60 s** și utilizarea a maximum **1KB date seed**.
- Evaluarea subiectivă de către o comisie stabilită.
- Trofeu simbolic **Van'Gogu**

Abordarea Sincrona (“Synchronous”)

Proiect 1

- Echipa: 7 oameni
- Primul proiect la Managementul proiectelor software se desfășoară în perioada laboratoarelor 3, 4, 5, 6. Începutul laboratorului 7 este termenul limită pentru definitivat proiectul. Proiectul va fi prezentat în forma sa finală în laboratorul 7

Abordarea Sincrona (“Synchronous”)

Proiect 2

- Echipa: 5 oameni
- Al doilea proiect la Managementul proiectelor software se desfășoară în perioada laboratoarelor 7, 8, 9, 10. Începutul laboratorului 11 este termenul limită pentru definitivat proiectul. Proiectul va fi prezentat în forma sa finală în laboratorul 11

Bibliografie

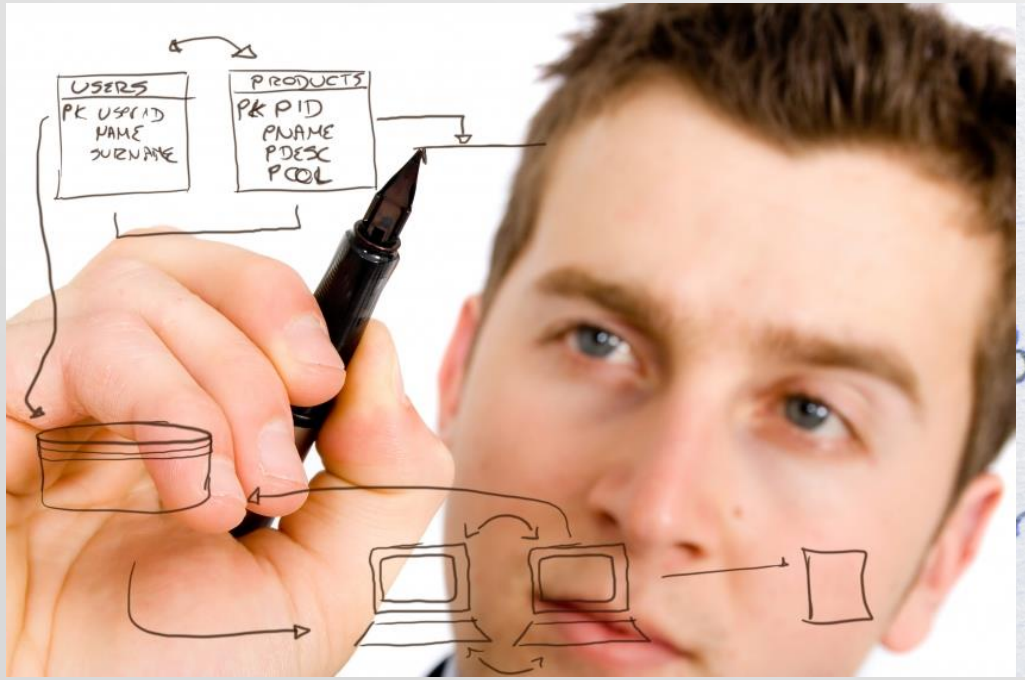
- TEXTBOOK OFICIAL : “*Introduction to Software Project Management*”, Adolfo Villaflorita, CRC Press, 2014, ISBN: 9781466559530

- “*Rapid Development*”, McConnell, Steve, Microsoft Press, 1996, ISBN 1-55615-900-5.

- “*Information Technology Project Management*”, Schwalbe, Kathy, 2nd ed., Course Technology, 2002, ISBN 0-619-03528-5 (include copie de evaluare 120-zile pentru MS-Project 2000).

- “*UML Distilled: A Brief Guide to the Standard Object Modeling Language*”, Fowler, Martin. 3rd ed., Addison-Wesley. ISBN 0-321-19368-7.

If you're 6 months late on a milestone due next week but nevertheless really believe you can make it, you're a project manager.



Obiective specifice

- Înțelegerea clară a problemelor, factorilor de succes și a riscurilor asociate cu dezvoltarea proiectelor în domeniul software
- Rolul și scopul managementului de proiect
- Asimilarea etapele și procesele din cadrul ciclului de viață al unui proiect
- Cunoașterea tehnicilor variate de planificare și de management al unui proiect software
- Evaluarea metodologiilor de bază în proiectarea, dezvoltarea, testarea și implementarea software-ului

Obiective specifice (2)

- Examinarea metodologiilor de bază în proiectarea, dezvoltarea, testarea și implementarea software-ului
- Examinarea tehnicilor variate de management a unei echipe de dezvoltare software
- Necesitatea și tehnicile corespunzătoare de management al utilizatorilor și ale așteptărilor acestora
- Utilizarea tehnicilor de planificare și dezvoltarea arhitecturii software și a ciclului de viață a produselor

« De ce » management software?

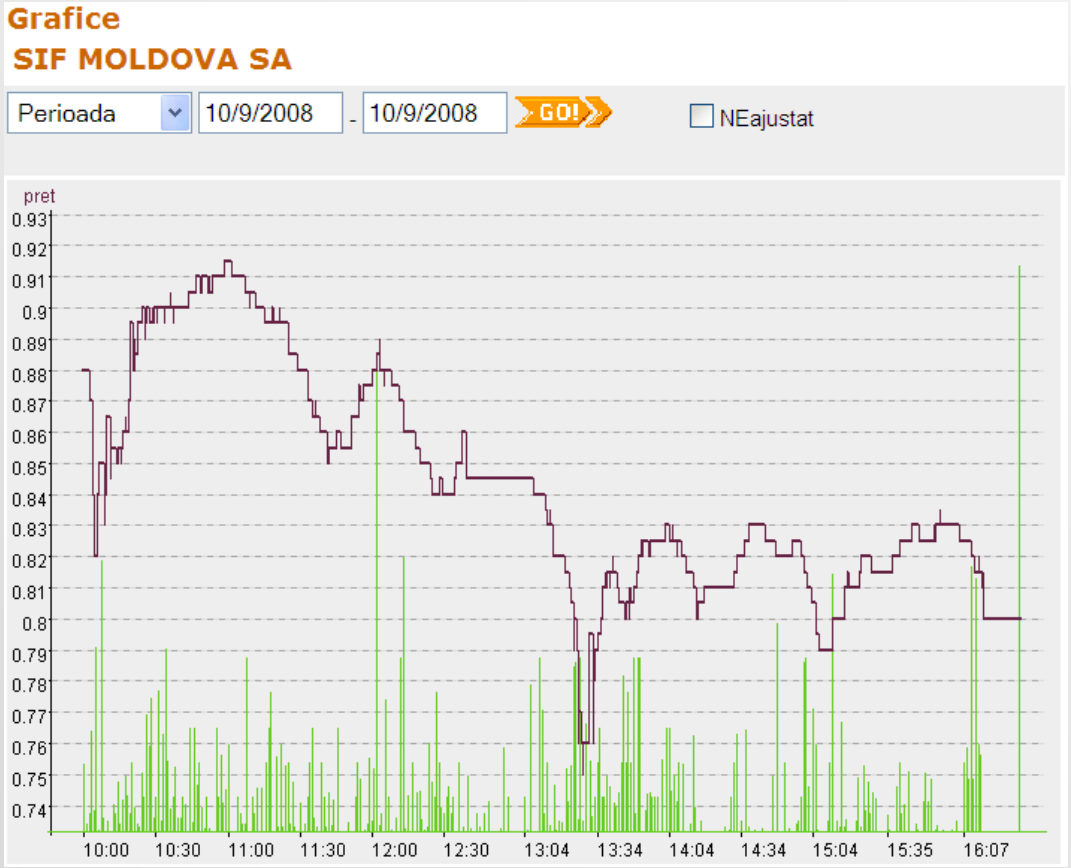
- Motivație
- Necesitate
- Obiective
- Impact

If you don't know where you're going, any road will take you there.

Criza dezvoltării software

- Erori grave
  - Sonde spațiale pierdute (Venus ’60, Marte 99)
  - Criza rachetelor – Cuba 1979
  - Rachetele Patriot 1991
  - Primul zbor Ariane 5 1996, artificii de 7 miliarde $
  - Aeroportul Denver 1994-1996
  - Anul 2000
  - Incidente în fiecare lună
    - bursa din Tokyo
    - accidente de circulație
- Proiectarea software
  - Livrarea în întârziere a tuturor proiectelor
  - Cost mult ridicat față de cel prevăzut
  - Livrarea unui produs de proastă calitate
  - Eșuarea în majoritatea cazurilor

Studiu de caz: Lucrare de doctorat



Sistem expert pentru prognoza acțiunilor bursiere

Studiu de caz: Lucrare de doctorat

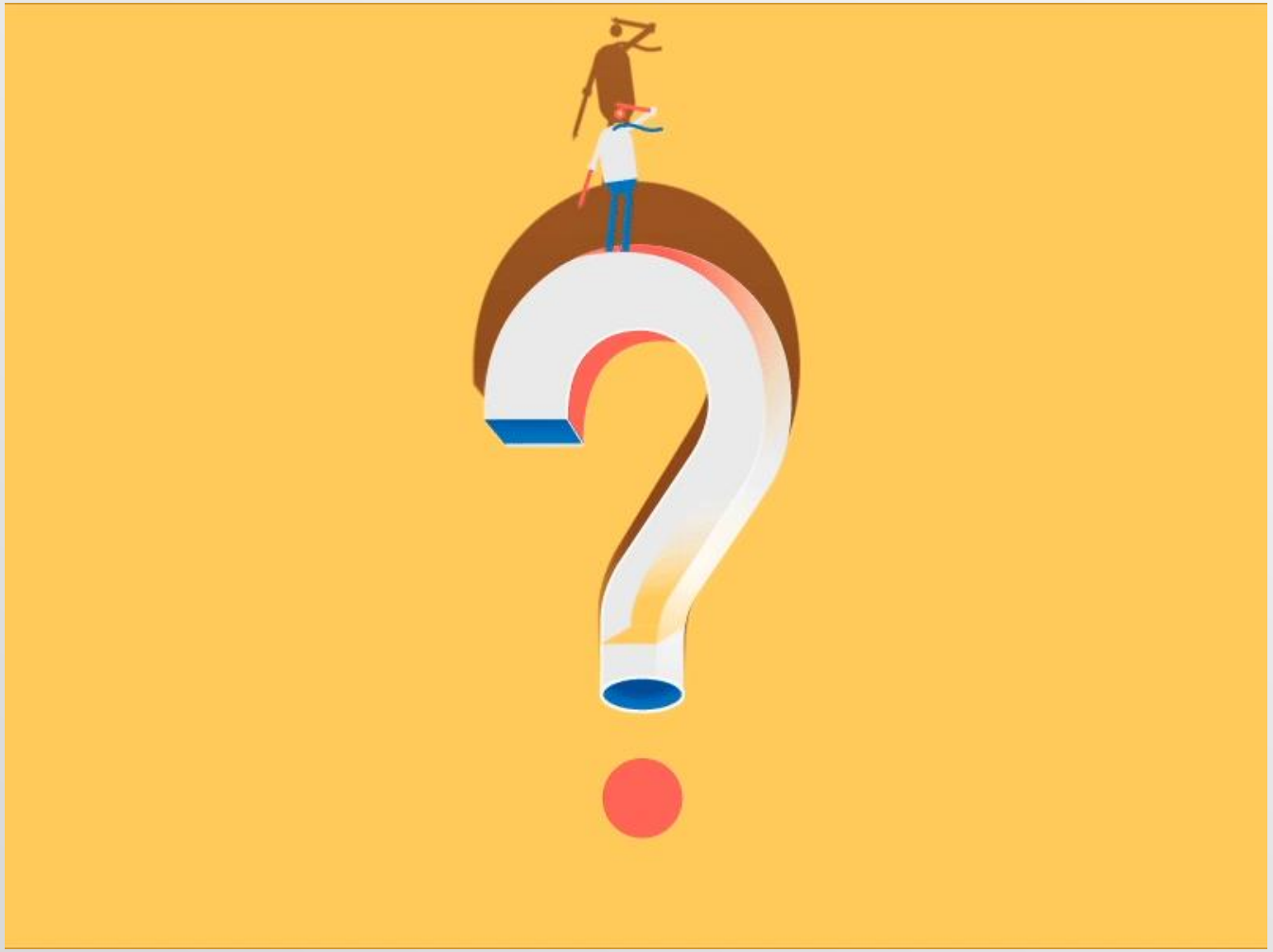
- Interval de estimare: 10 secunde
- Evaluare proces stochastic: Lanțuri Markov
- 70% predictie corectă. E suficient?
- 99% știință, 1% rutină

Concluzii

- Evaluare comision
- Omul de știință nu este neapărat înclinat practic
- Un bun manager asigură un produs complet și viabil

Good project managers know when not to manage a project.

Mulțumesc



Curs #1

Introducere

Motivație (1)

- Multe cursuri de Computer Science sau Electronică se concentrează doar pe aspectele tehnice și pe notații pentru buna dezvoltare a software-ului (ex: cum are loc procesul de testare; cum se scriu cerințele funcționale; cum se modelează sistemele; UML; Java)
- În schimb, pentru a obține software de calitate, este necesar un process bine definit și gestionat, ce organizează activitățile într-o manieră eficientă și controlată

Motivație (2)

- Să considerăm următoarele:
  - Redactarea corectă a cerințelor funcționale nu este de ajutor în cazul în care nu există un proces controlat pentru acceptarea și urmărirea modificărilor
  - Tehnicile de testare sunt inutile dacă nu mai există timp rămas pentru testare (ex: în cazul în care timpul necesar pentru dezvoltare a fost subestimat, iar lansarea software-ului a fost întârziată)
  - Dezvoltarea unui sistem software necesită execuția, monitorizarea și controlul diferitelor activități puțin sau deloc conectate cu scrisul codului (ex: instruirea utilizatorilor, prezentarea produsului software, gestionarea publicității si a comunicării, redactarea documentației)

Motivație (3)

- Dacă se dorește livrarea unui produs la timp și respectând bugetul, având de asemenea proprietățile de calitate dorite, sunt necesari:
  - Un proces pentru definirea planificării, a bugetului și punerea de comun acord a caracteristicilor de calitate a produsului
  - O listă de tehnici pentru definirea, stabilirea, planificarea, execuția și monitorizarea: obiectivelor, calității, timpului și costurilor

Competențe și obiective ale cursului

- Gestionarea unui proiect (software) necesită competențe, aptitudini și tehnici specifice
- O parte dintre întrebările ce vor putea fi răspunse la sfârșitul cursului sunt:
  - Cum pot estima cât de mult va dura completarea unui task?
  - Care este suma de bani pe care o voi solicita pentru un proiect?
  - Cum pot să mențin echipa motivată și să mă asigur că proiectele reprezintă pentru angajați o ocazie de a învăța, crește și avansa în carieră?
  - Cum pot gestiona riscurile unui proiect?
  - Cum evaluez dacă proiectul respectă timpul, bugetul, deadline-ul?
  - Cum controlez calitatea produsului final?

Managementul Proiectelor Software (MPS)

- Tehnicile de **Project Management** sunt multidisciplinare

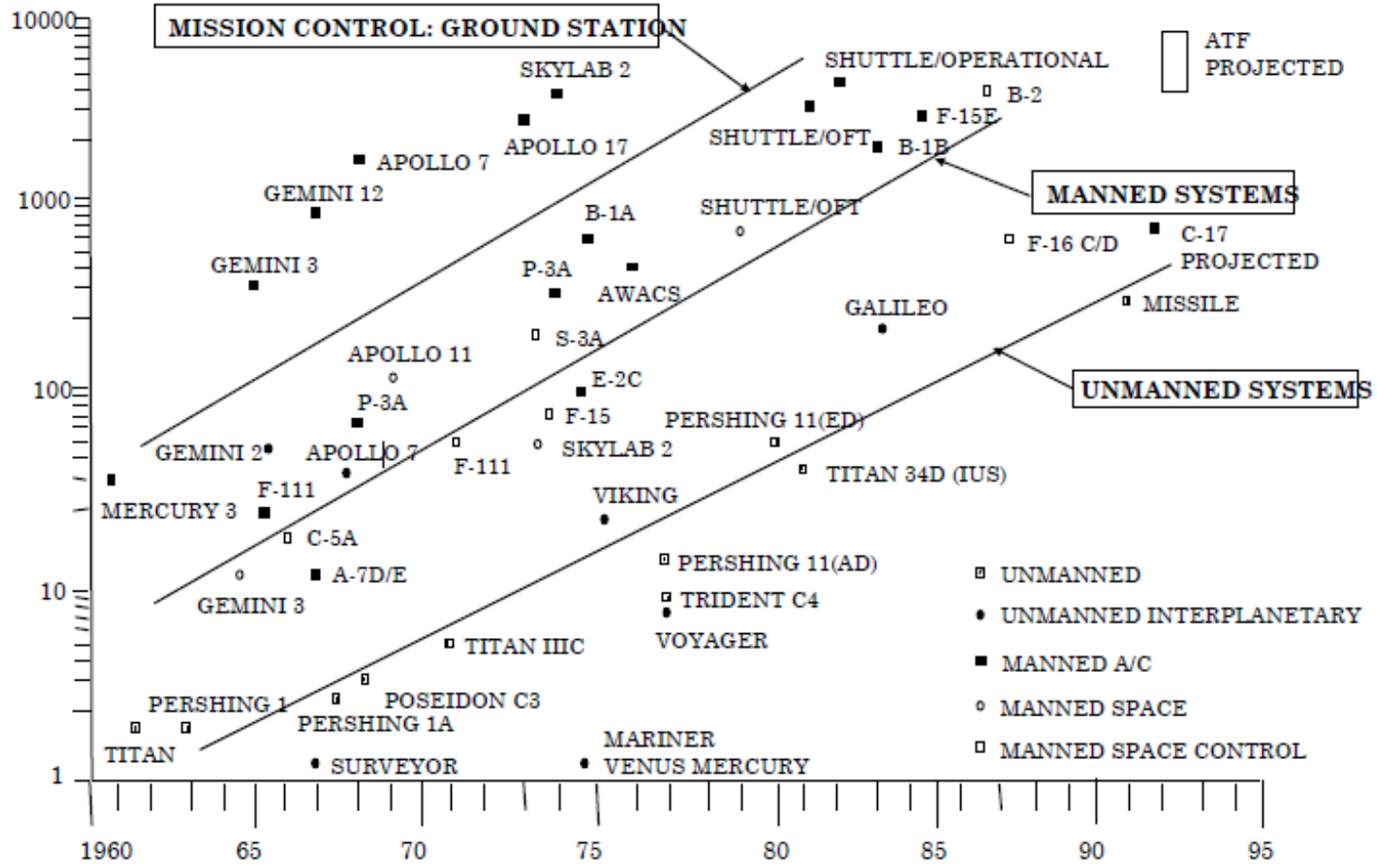
  - ceea ce veți învăța în acest curs este aplicabil pentru orice alt domeniu (al ingineriei).

- Există unele caracteristici care fac managementul proiectelor software interesant.

Managementul Proiectelor Software (MPS)

- Managementul proiectelor software este interesant și provocator pentru că:
  - Produsul este intangibil
  - Produsul este flexibil (ex: dimensiuni diferite, constrângeri diferite)
  - Multe proiecte software sunt irepetabile (pot fi realizate doar o data)
  - Procesul de dezvoltare este flexibil
  - Dimensiunea și complexitatea cresc exponențial
  - Viețile oamenilor pot depinde de buna funcționare a produselor software (ex: sistemul de control al unei aeronave) – sisteme critice de siguranță

Complexitate (1)



Sursă: “Mars and Beyond: NASA’s Software Challenges in the 21st Century”

Dr. Michael R. Lowry. NASA Ames Research Center. 5 decembrie 2003
web.cecs.pdx.edu/~black/S3S/MichaelLowry.ppt

Complexitate (2)

- Întregul ansamblu al rachetei Saturn V (propulsoarele, modulul de comanda și modulul lunar) avea o capacitate de calcul mai mică decât un telefon mobil standard din ziua de azi.

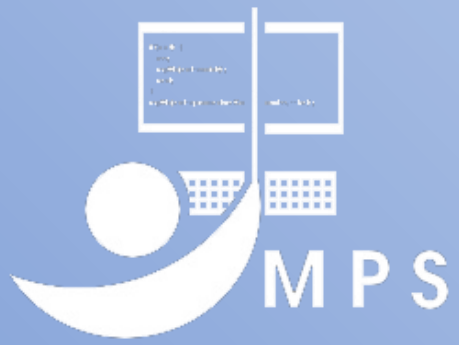
Sursa: **Apollo**

by Charles Murray and

Catherine Bly Cox

O scurtă istorie

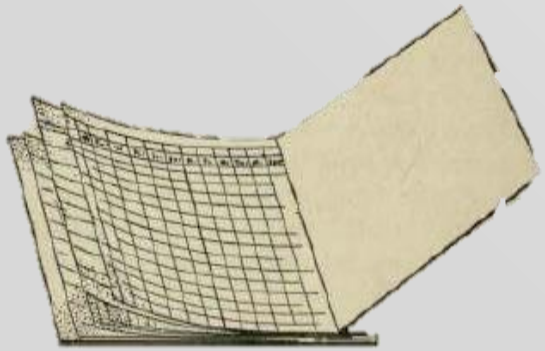
a Managementul Proiectelor (Software)



Frederick Winslow Taylor

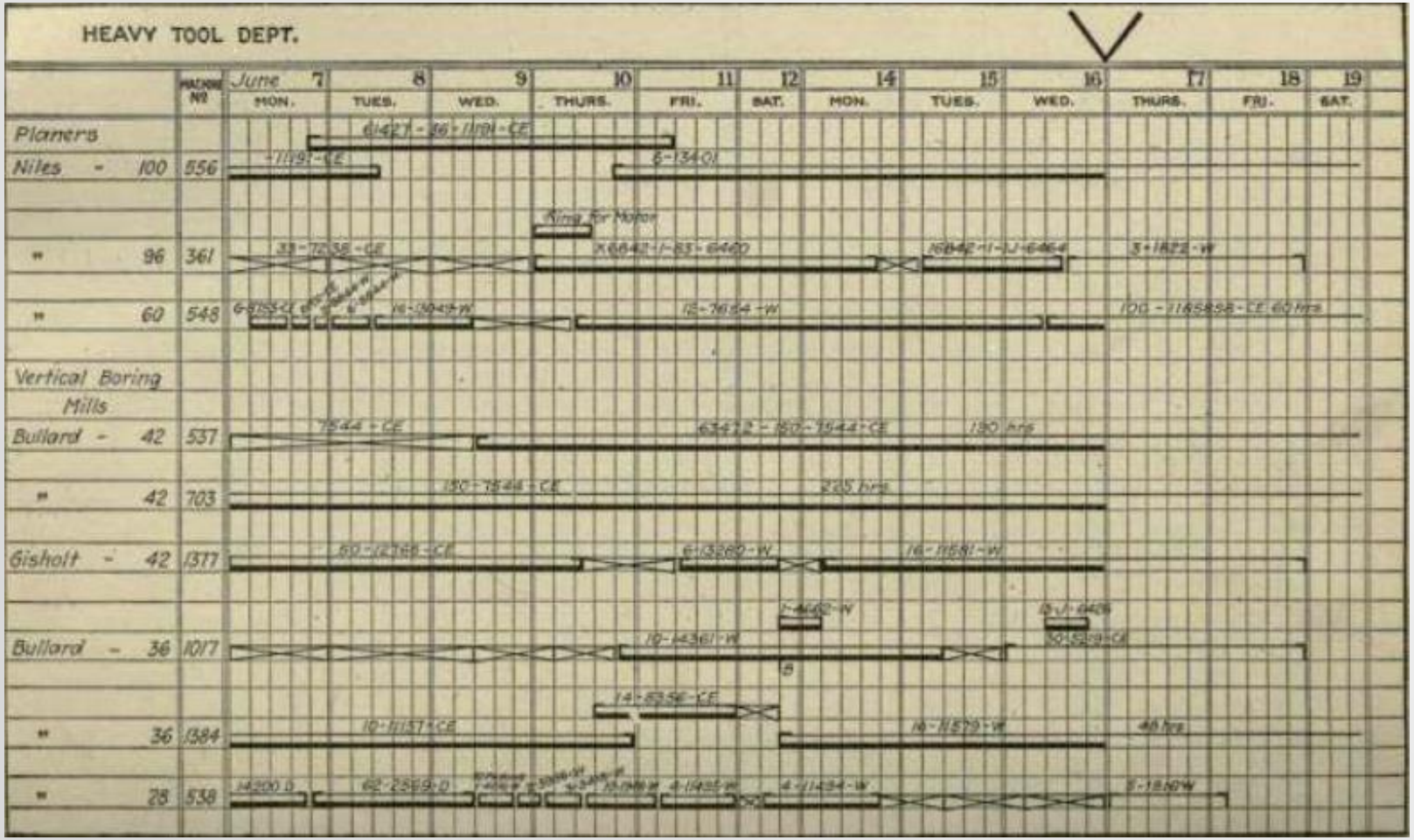
- “Scientific Management” al lui Taylor este prima teorie despre muncă și management
- Începutul anilor 1900
- Viziune negativă despre muncitori (atunci când fac task-uri repetitive, ei lucrează la acea capacitate minimă necesară pentru a scăpa nepedepsiți)
- Alte caracteristici interesante:
  - Definirea științifică a muncii
  - Selecția științifică a personalului
  - Împărțirea responsabilităților între muncitori și conducere
  - Stimulente și perioade de odihnă (pentru a crește eficiența muncitorilor)

Henry Gantt



- Diagramele Gantt încă sunt folosite în ziua de azi pentru planificarea proiectelor
- Definite în timpul Primului Război Mondial
- Folosite prima oară în planificarea și monitorizarea lucrului și progresului în construcția vapoarelor: distinctive între lucru și progress
- Cartea sa este disponibilă pentru descărcare pe archive.org

Exemplu de Diagrama Gantt



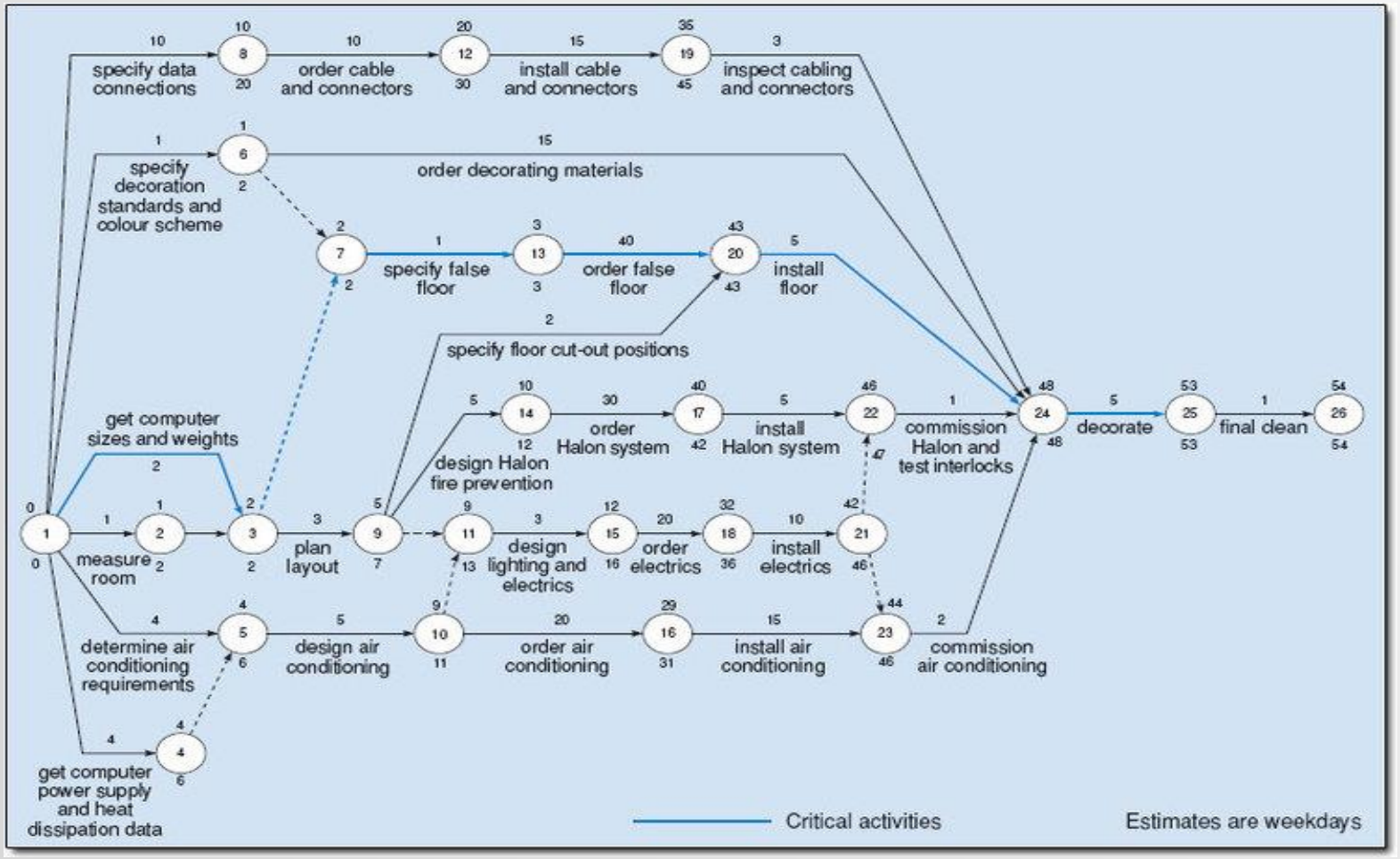
Sursă: “The Gantt chart, a working tool of management” – Clark, Wallace si Gantt

Anii ‘50 și ‘60

- Al Doilea Război Mondial:
  - **Proiectul Manhattan**: diagrame de flux de proces
- 1957 **CPM** (Critical Path Method – metoda drumului critic)
  - Algoritm matematic pentru planificarea unui set de activități ale unui proiect, folosit pentru planificarea activităților de mentenanță în fabrici
  - Dupont + Remington Rand => echipa UNIVAC
  - Nu a suferit modificări fundamentale până în prezent
- 1958 **PERT** (Program Evaluation and Review Technique)
  - Programul de rachete Polaris al marinei SUA (Booz Allen & Hamilton – firma de consultanță managerială – lucrând ca echipa ORSA pentru Lockheed Missile System)

Motivații pentru CPM si PERT

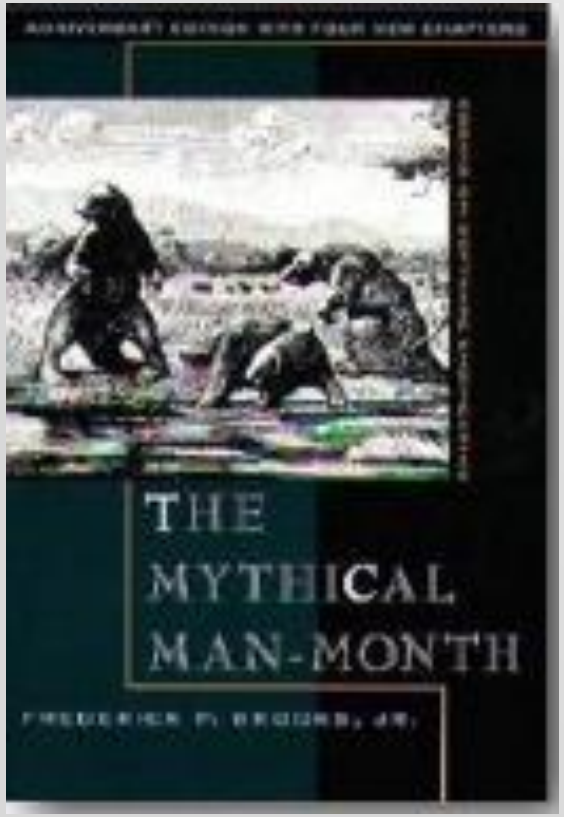
Ce activități pot întârzia finalizarea proiectului din următorul plan?



Anii ‘60 și ‘70

- 1960: contracte importante ale Guvernului SUA (Vietnam, centrale nucleare, NASA Apollo): standardizare și automatizare
  - PERT/COST și WBS devin obligatorii în proiectele sponsorizate de Guvern
  - Metoda Analizei Valorii (Earned Value Analysis - EVA) este definită
  - Managementul configurațiilor
  - Organizații de project management (PMI, IPMA) promovează profesia și tehnicile
  - (1961) IBM folosește PM în mod comercial

Anii ‘60 și ‘70



- 1970: Dezvoltarea Software intră în ecuație
  - EVA dezvoltat pentru monitorizarea programului și a costului
  - Modelul cascadă pentru devoltarea software
  - “The mythical man-month” subliniază numeroase capcane ale dezvoltării software

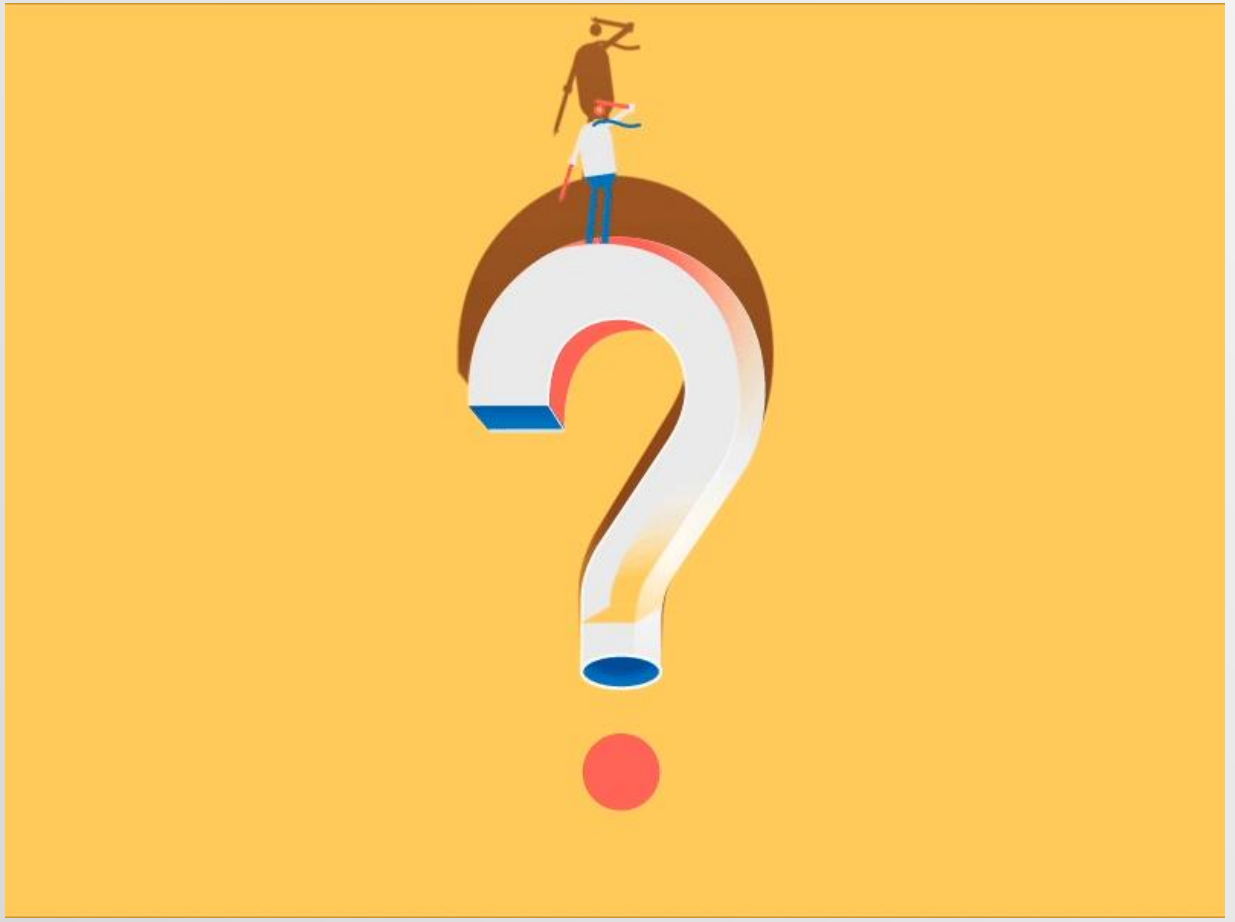
Revoluția Computerelor

- 1980
  - Proliferarea hardware-ului și a software-ului fac tool-urile de PM accesibile și pentru firmele mai mici
  - Capacitatea hardware crește exponential (Legea lui Moore) și la fel și software-ul
  - Modele de estimare (FP și COCOMO) sunt introduce pentru a prezice complexitatea software-ului

Revoluția Computerelor

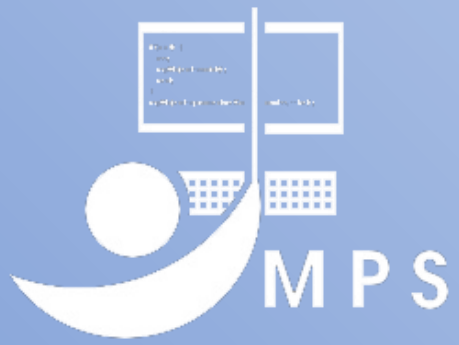
- 1990
  - Calitate totală
  - Organizații mai flexibile, mai rapide și mai receptive
- Astăzi
  - Aplicații Web și noi modele de distribuție a aplicațiilor
  - Dezvoltare prin componente și framework-uri
  - Agilitate, interacțiune rapidă, feedback constant

Mulțumesc



Proiecte și Stakeholderi

Numele jocului, jucătorii și (unele) reguli

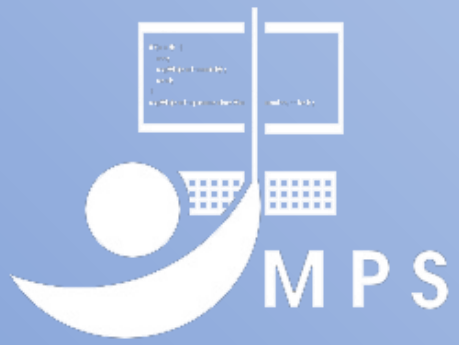


Obiective

- Înțelegerea noțiunii de proiect, a ciclului de viață al unui proiect
- Înțelegerea părților implicate (stakeholders) și a relațiilor dintre acestea
- Înțelegerea influenței organizațiilor asupra proiectelor

Ce este un proiect?

Numele jocului

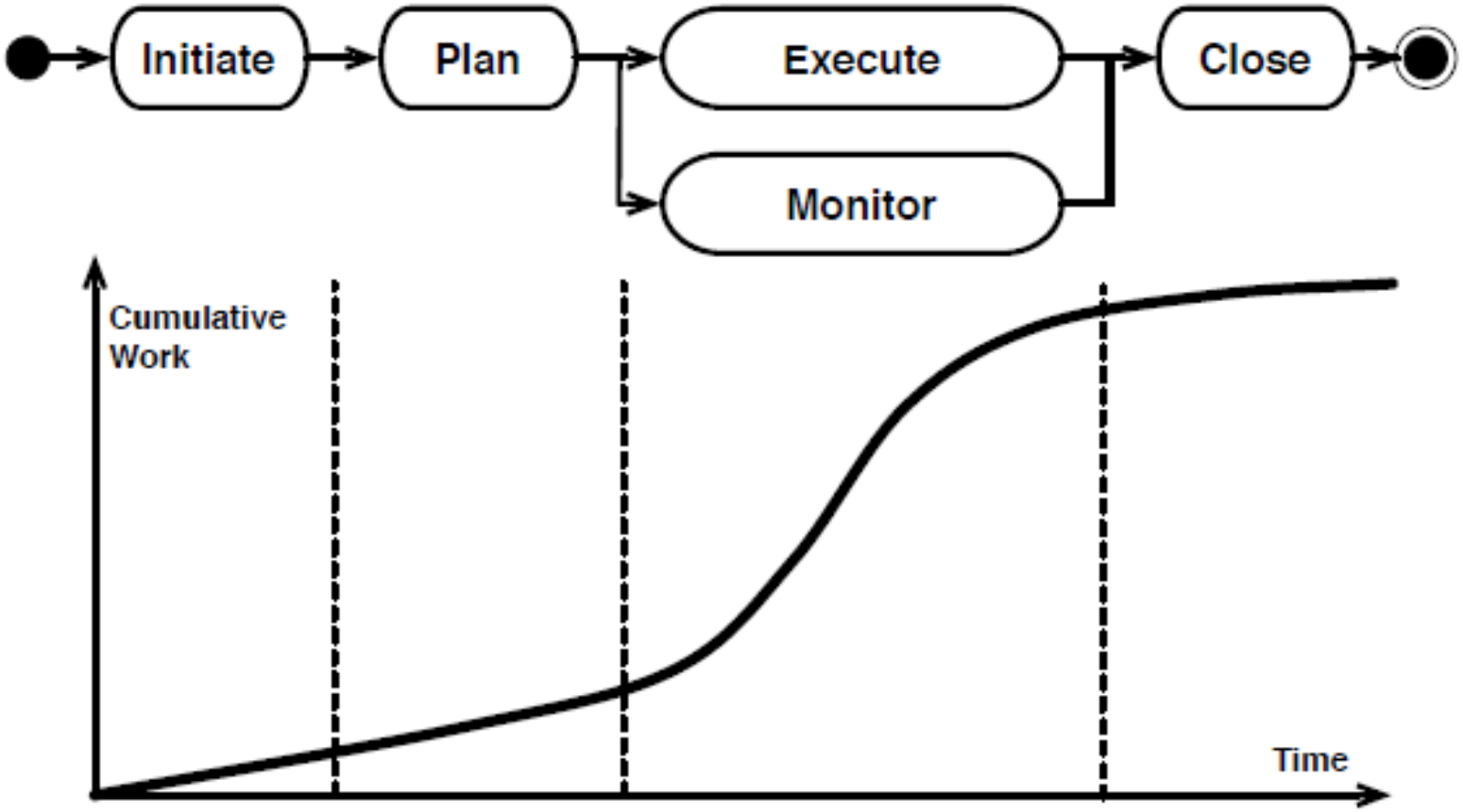


Un proiect este un efort **temporar** realizat pentru crearea unui produs, serviciu sau rezultat **unic**.

Caracteristicile unui proiect

- Temporar
  - Un început și un sfârșit definite (obiectivele sunt îndeplinite sau proiectul a fost finalizat – caz în care obiectivele nu sunt îndeplinite)
  - Rezultatele proiectelor nu sunt neapărat temporare (ciclul de viata al proiectelor)
- Produse, servicii sau rezultate unice
  - Un produs cuantificabil (ex: o componentă, ...)
  - Capacitatea de a realiza un serviciu (ex. printr-o funcție de business)
  - Un rezultat, de exemplu cunoștințe (strânse în documente, prezentări...)
- Elaborare progresivă
  - Dezvoltare prin pași și incremente (necesară pentru a păstra în vedere scopul proiectului)
- Constrângeri de resurse

Elaborare progresivă



Contextul managementului proiectelor

- Subproiecte
  - Proiectele pot fi împărțite în subproiecte (uneori numite și ele tot “proiecte” și administrate ca atare)
- Managementul proiectelor și programelor
  - Set de proiecte înrudite și manageriate într-o manieră coordonată pentru obținerea unui anumit tip de beneficiu
- Managementul portofoliului
  - Colecție de proiecte / programe fără legătură, dar grupate împreună pentru facilitarea managementului și îndeplinirea obiectivelor strategice

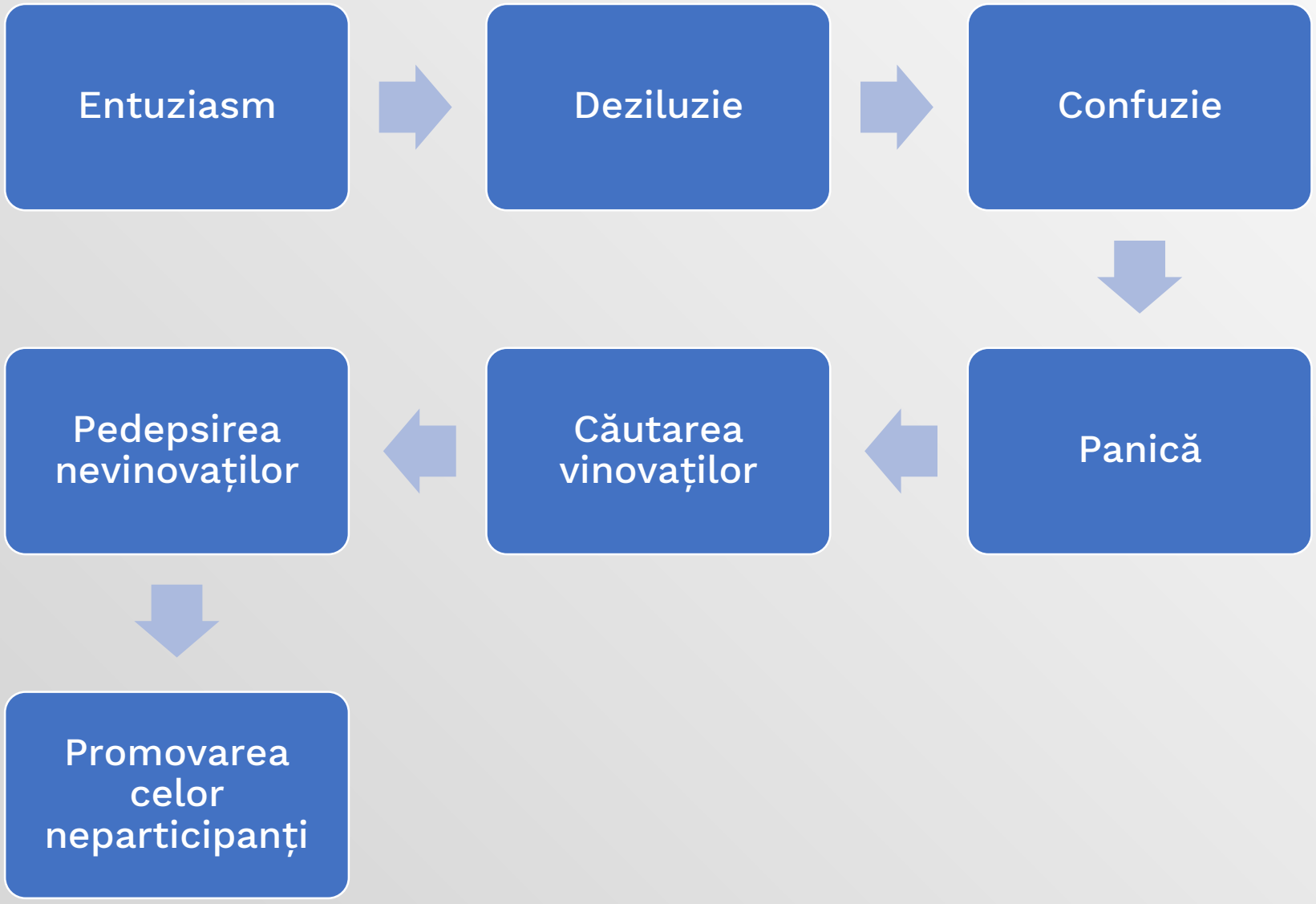
Proiecte și proceduri operaționale

- Lucrul poate fi pe bază de proiect sau operațional
- Caracteristici comune:
  - Realizat de oameni
  - Resurse limitate
  - Planificat, executat și controlat
- Diferențe
  - **Proiect**: obținerea obiectivelor și finalizare
  - **Operațional**: susținerea business-ului

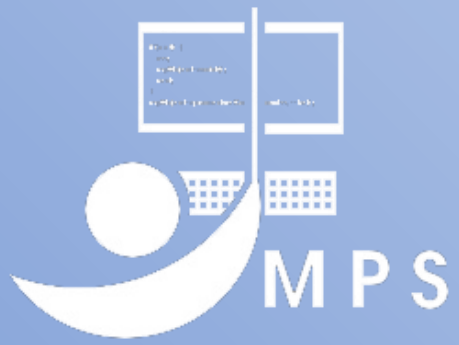
Exemple (și contraexemple)

- Pregătirea cinei
- Fabricarea unui mașini
- Design-ul unei mașini
- Redactarea unui articol
- Dezvoltarea unui sistem software
- Menținerea unui sistem software
- Managementul personalului

Cele 7 faze ale unui proiet



Proiecte de Dezvoltare Software



Tipuri de Proiecte de Dezvoltare Software

În cariera de project manager, o persoană poate fi implicată în diferite tipuri de proiecte de dezvoltare software, precum:

- Dezvoltarea de aplicații
- Reingineria proceselor și a sistemelor
- Integrarea sistemului
- Servicii de consultanță
- Instalare și training

A. Dezvoltarea de aplicații

- Scop: dezvoltarea unei aplicații (desktop, web, mobil, incorporate)
- Tipul cel mai distractiv 😂
- Tipuri de dezvoltare de aplicații:
  - **One-off-uri**: sisteme create special pentru client
  - **Off-the-shelf**: pentru satisfacerea nevoilor unei game largi de utilizatori
  - **Off-the-shelf customizat**: sisteme standardizate ce necesită o cantitate semnificativă de customizare pentru a putea fi folosite într-o organizație. Ex: sisteme de tip ERP (Enterprise Resource Planning)

B. Reingineria proceselor și a sistemelor

- Scop: schimbarea modalității de desfășurare a procedurilor operaționale din cadrul unei organizații în vederea realizării unui scop strategic (ex: îmbunătățirea calității, eficienței)
- De obicei proiecte ample, care necesită o analiză precisă a situației existente din punct de vedere al procedurilor, sistemelor, infrastructurii
- Necesită adesea introducerea unui sistem ERP și a activităților de integrare a sistemului și a datelor

C. Servicii de integrare a sistemului

- Scop: automatizarea fluxului informațional în cadrul sistemelor unei organizații
- Tipuri de integrare:
  - **Orizontal**: integrarea sistemelor ce realizează operațiuni similare
  - **Vertical**: integrarea sistemelor care automatizează diferite etape ale unei proceduri

Alte tipuri de proiecte

- D. Servicii de consultanță
  - Cerută de cele mai multe ori pentru a aduce expertiză din exterior echipei
- E. Servicii de instalare și training
  - Servicii legate de instalare și training pentru sisteme software specific
  - Remarcă: constituie de asemenea sursă de venit în dezvoltarea open source

Proiectele și mediul lor

Stakeholder

Un **stakeholder** reprezintă un individ sau o organizație ce este implicat(ă) activ într-un proiect, sau al cărui interes ar putea fi influențat (pozitiv sau negativ) în urma realizării sau finalizării proiectului.
(PMBOK)

Stakeholder

- Câteva caracteristici:
  - Pot avea influențe diferite și diverse niveluri de responsabilitate în timpul proiectului
  - Pot juca roluri diferite
  - Pot avea o influență pozitivă sau negativă asupra proiectului
  - Pot fi dificil de identificat
  - Lipsa intervenției lor poate influența în mod negativ proiectul (nevoie de identificare și implicare)
- Remarcă: managerul de proiect și echipa de proiect **sunt** și ei stakeholderi în cadrul proiectului, deși termenul este adesea folosit pentru a desemna “ceilalți” stakeholderi

Tipuri de stakeholderi

- Project managerul
- Echipa de proiect
- Sponsorul proiectului
- Organizațiile participante
- Partenerii
- Clientul
- Restul: oricine ar putea fi afectat de rezultatele proiectului

Stakeholderi Cheie

- Interni:
  - Membrii echipei de proiect: grupul ce implementează
  - Echipa de management a proiectului: membrii echipei direct implicați în activitatea de project management

- De mijloc:
  - Client / Utilizator: persoană sau organizație ce va utiliza rezultatele proiectului. Pot fi mai multe niveluri de utilizatori.
  - Sponsor: persoană sau grup ce furnizează resursele financiare
  - Organizație participant: organizația implicată cel mai mult în realizarea proiectului

- Externi:
  - Influenceri: persoane sau grupuri neimplicate direct în proiect, dar care i-ar putea influența cursul

Exercițiu – Identificare Stakeholderi

Identificați stakeholderii următoarelor proiecte:

- Construirea unui pod ce conectează o insulă de continent;
- Construirea unui depozit de deșeuri (groapă de gunoi)
- Construirea unei foi de calcul (spreadsheet) open source
- Dezvoltarea unei aplicații web pentru monitorizarea masei corporale
- Portarea OpenOffice (suita Office gratuita) pe Android

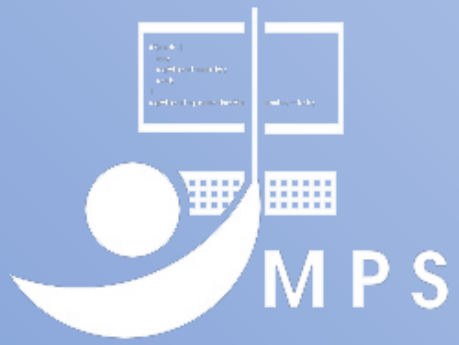
Managerul de Proiect

Persoana responsabilă cu gestiunea proiectului și a așteptărilor stakeholderilor.

Câteva competențe necesare:

- Comunicare și negociere
- (Oarecare) Predispoziție către riscuri
- Orientare către scop
- Abilități de leader
- Gândire creativă, în afara tiparelor
- Cunoștințe tehnice solide
- Corectitudine profesională
- Bun simț
- Stil

Organizarea Dezvoltării Proiectelor Software



Managementul Proiectelor Software

- **Managementul proiectelor software (Software project management)** reprezintă integrarea tehnicilor de management în dezvoltarea software

- Nevoia unei asemenea integrări își are rădăcinile în anii ‘60, în zilele “crizei software”, când specialiștii recunoșteau complexitatea crescătoare a livrării produselor software ce îndeplinesc specificațiile

Ce face un Software să fie considerat Produs?

- O aplicație pentru gestionarea filmelor vizionate
- Un sistem de contabilitate cu intrare dublă pentru acasa sau pentru mici intreprinderi
- Un serviciu web pentru gestionarea rezervărilor terenurilor de squash într-un club de squash
- Un sistem de planificare a resurselor într-o corporație
- Un sistem de frânare automat pentru trenuri

- Din ce componente este compus?
- Care ar fi efectele în cazul unui eșec?
- Ce competente sunt necesare pentru dezvoltare?
- Care este modelul de business asociat?

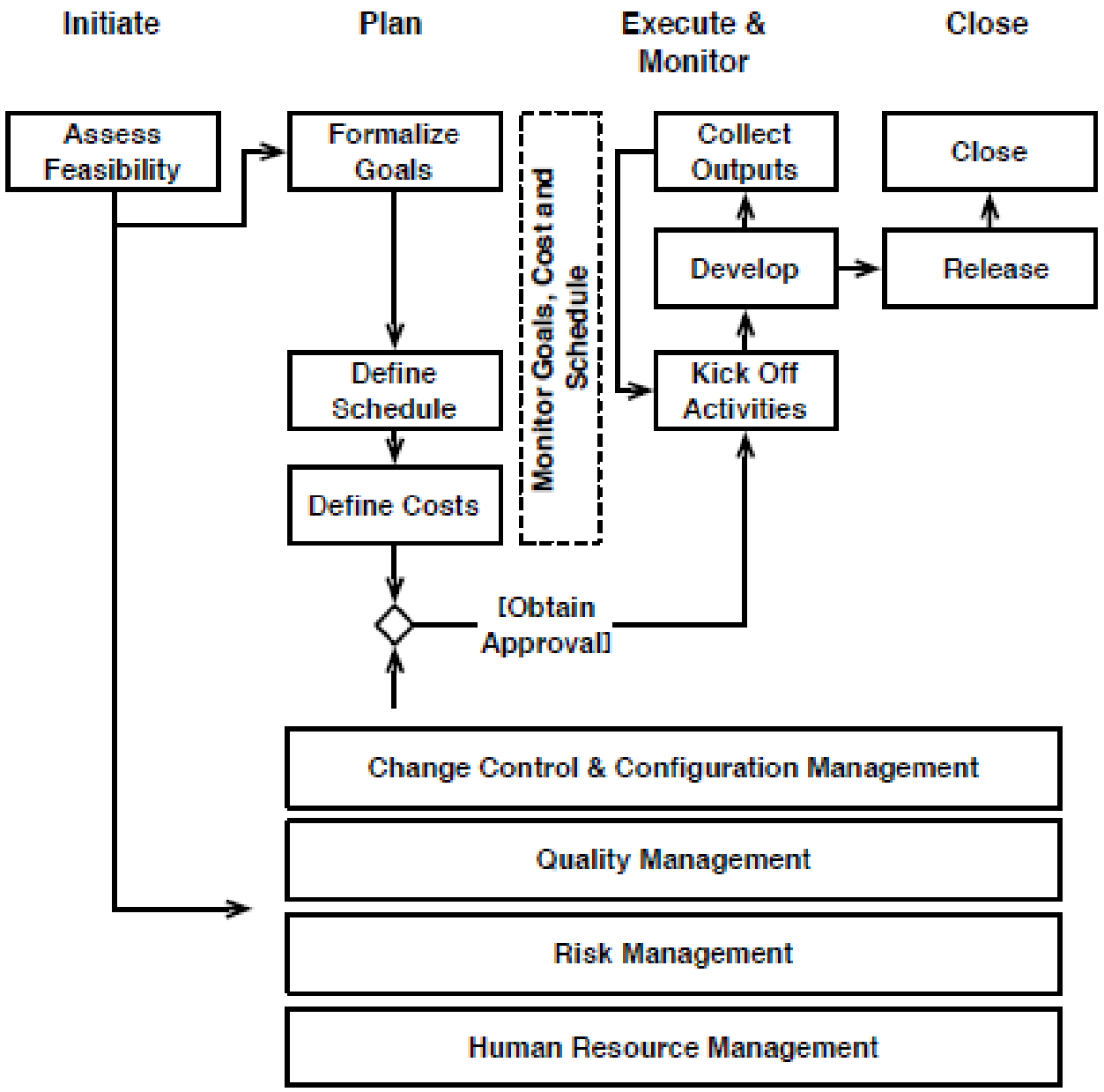
Framework de Dezvoltare Software

Un framework general trebuie să:

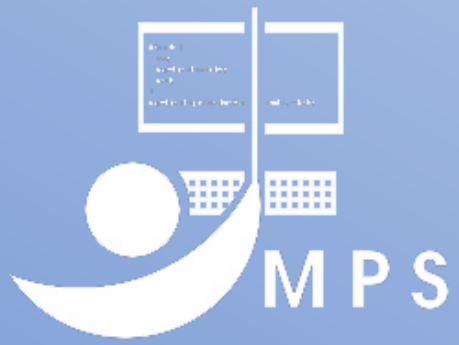
- Formeze o viziune comună despre scopurile ce trebuie realizate, caracteristicile rezultatelor proiectului și caracteristicile procesului de dezvoltare
- Structureze lucrul ca o perfecționare progresivă, de la specificații la scopuri
- Reducă impactul nesiguranțelor și a necunoscutelor
- Evidențieze orice deviere de la plan (scopuri, costuri, calitate)
- Asigure coerență și calitatea componentelor proiectului de-a lungul timpului în pofida necunoscutelor și a schimbărilor (sau a cererilor de schimbare)
- Motiveze echipa

Câteva preocupări

- Evaluarea fezabilității
- Managementul scopurilor
- Managementul timpului
- Managementul costurilor
- Controlul schimbărilor și managementul configurației
- Managementul calității
- Managementul riscurilor
- Managementul resurselor umane


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

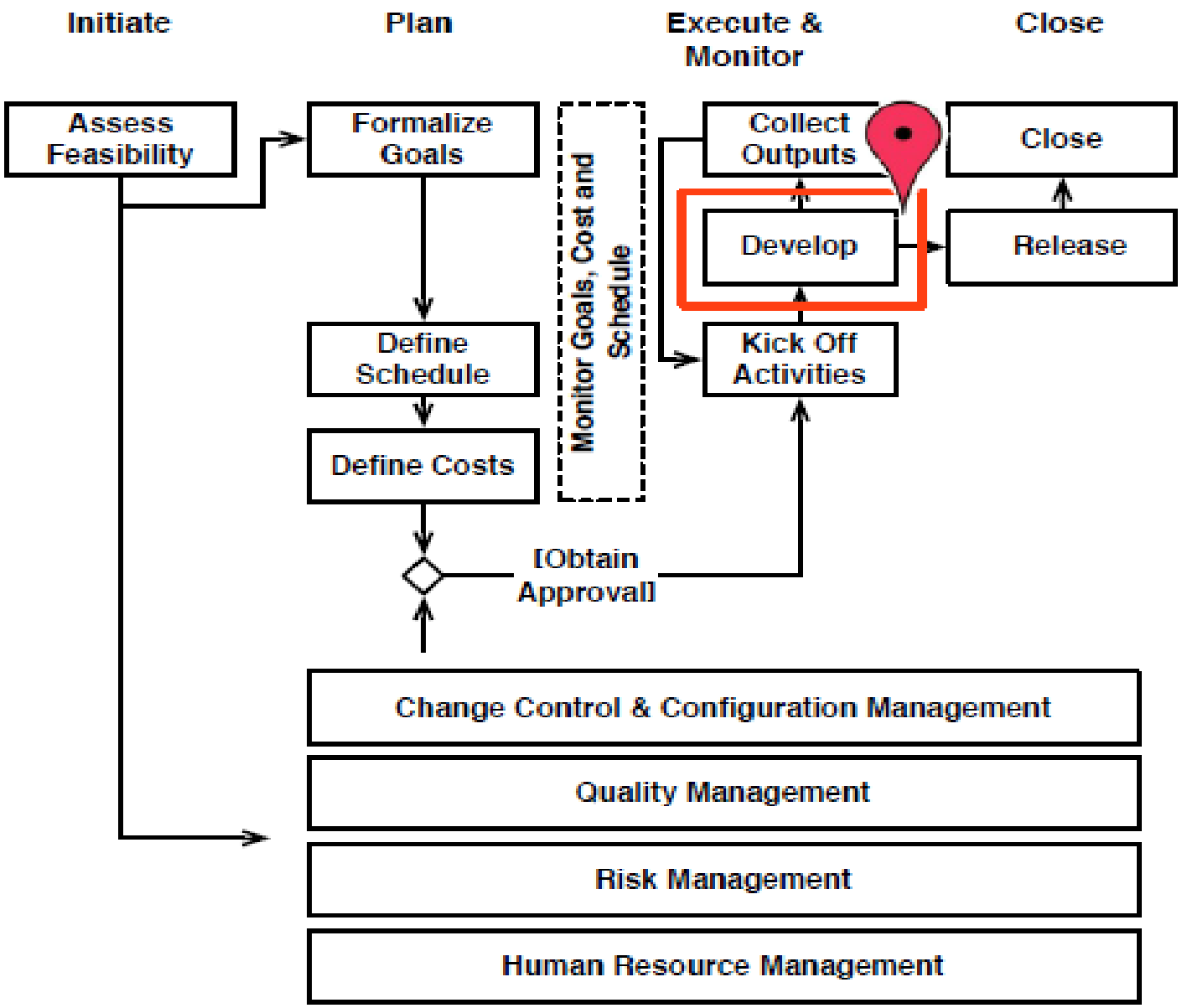
Procese (tradiționale) de dezvoltare a Software-ului



Good project management is not so much knowing what to do and when, as knowing what excuses to give and when.

Obiective

- O introducere subtilă și la nivel înalt în activitățile de dezvoltare a software-ului
- Descoperirea pietrelor de temelie necesare pentru producerea de software
- Remarci:
  - Acest curs nu îl inlocuiește pe cel de software engineering
  - Activitățile trebuie integrate într-un process coerent pentru a avea sens
  - Proiectele de dezvoltare a software-ului variază de la extrem de reduse la extrem de extinse … nu toate activitățile sunt la fel de utile sau relevante în orice context


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
[Obtain Approval]
Collect Outputs
Develop
Kick Off Activities
Close
Release
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Prezentare generală

- Dezvoltarea de software necesită o perfecționare progresivă, ce transforma conceptul în operații prin următoarele faze:
  - Stabilirea cerințelor si a experienței utilizatorilor
  - Design
  - Implementare
  - Verificare și validare
  - Deployment
  - Operații și mentenanță
- O dată cu parcurgerea acestor faze, se iau anumite alegeri specifice; costul schimbărilor crește în consecință
- Procese diferite pun accentul pe fiecare activitate sau definesc ordinea în care aceste activități pot fi efectuate.

Managementul Cerințelor

Cerințe

- Scop: formarea unei viziuni comune despre caracteristicile sistemului ce trebuie construit

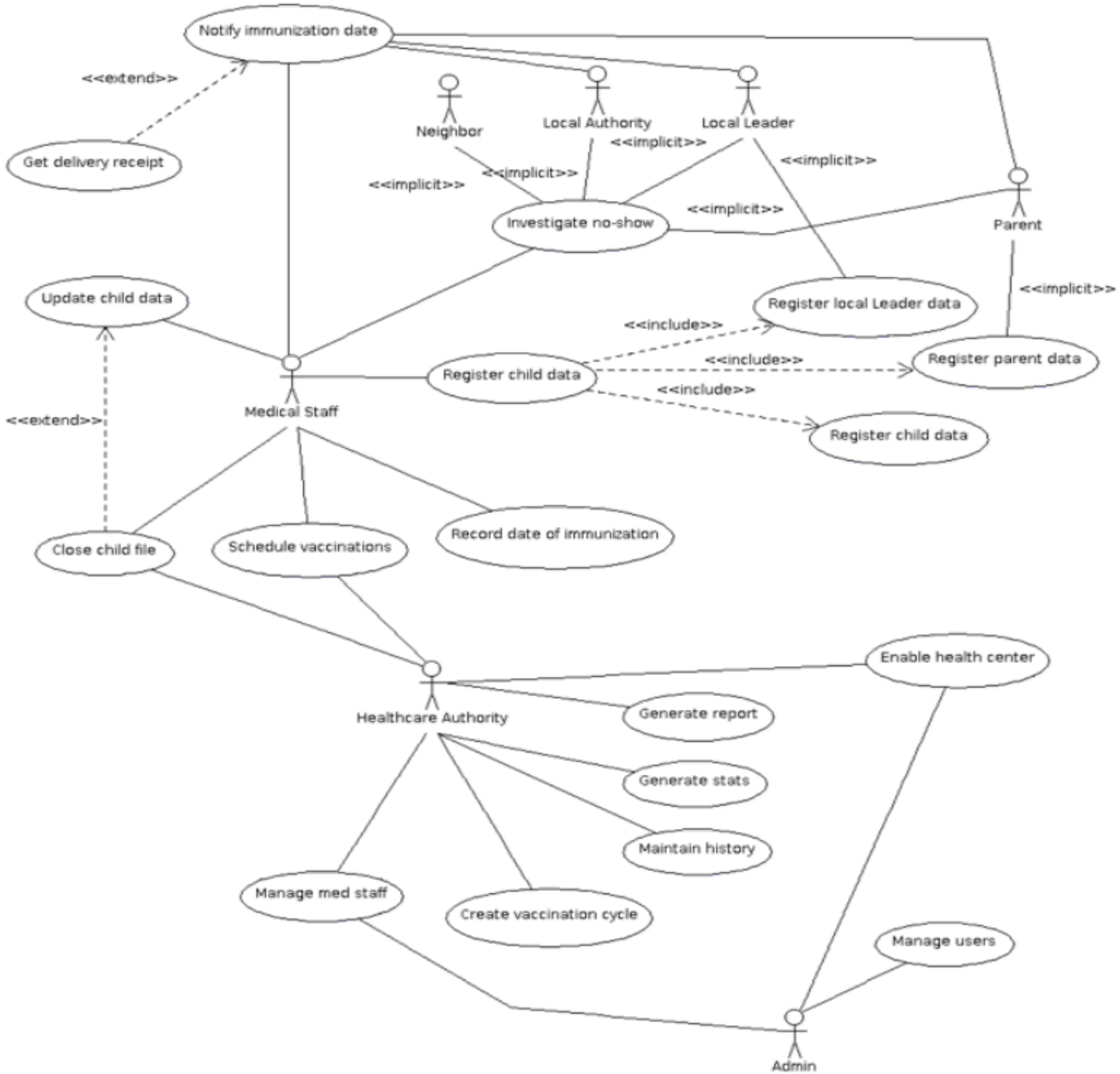
- Rezultat:
  - Lista de cerințe:
    - un document text
    - o listă de scenarii ale utilizatorilor
    - un set de diagrame (e.g. diagrame de cazuri de utilizare) și descrierile textuale corespunzătoare

Lista de cerințe

- Format:
  - Text liber sau structurat descriind funcționalitățile și celelalte proprietăți ale sistemului
- Avantaje:
  - Simplu de schițat și distribuit
  - Formatul poate fi utilizat pentru a păstra evidența schimbărilor (versionare)
- Dezavantaje:
  - Nu se concentrează pe interacțiunea cu utilizatorul: poate fi ușor de înțeles de către client
  - Ambiguități și incoerențe; interacțiuni între cerințe

Diagramele cazuri de utilizare (use case)

- Format:
  - Diagrame ce descriu interacțiunea dintre utilizatori și sistem
  - Descriere textuală a interacțiunii ca o secvență de pași
- Avantaje:
  - Intuitive, mai ușor de înțeles de către client
  - Se concentrează pe ceea ce face sistemul (functionalități)
- Dezavantaje:
  - Dificil de reprezentat și de păstrat evidența cerințelor non-funcționale
  - Gestionarea diagramelor necesită puțin mai multă muncă decât cea a textului


Notify immunization date
<<extend>>
Get delivery receipt
Neighbor
Local Authority
Local Leader
<<implicit>>
<<implicit>>
<<implicit>>
<<implicit>>
<<implicit>>
Investigate no-show
Parent
Update child data
Register local Leader data
<<include>>
<<implicit>>
Register child data
<<include>>
Register parent data
<<include>>
Medical Staff
<<extend>>
Register child data
Close child file
Schedule vaccinations
Record date of immunization
Enable health center
Healthcare Authority
Generate report
Generate stats
Maintain history
Manage med staff
Create vaccination cycle
Manage users
Admin

Povești de utilizator (de planificare)

- Format:
  - Descrieri textuale structurate ale funcționalităților: Din postura de [utilizator], vreau să fac [asta] pentru că [motiv]
- Avantaje:
  - Intuitiv, compact, mai ușor de înțeles de către client
  - Se concentrează pe ceea ce face sistemul (functionalități)
- Dezavantaje:
  - Dificil de reprezentat și de păstrat evidența cerințelor non-funcționale
  - Reprezintă o specificație partială (multe detalii trebuie stabilite în timpul implementării) – folosită în metodologiile Agile

Ingineria cerințelor

- Scop:
  - Definirea și menținerea cerințelor de-a lungul timpului
- Activități:
  - Elaborarea cerințelor (workshop-uri, brainstorming, grupuri țintă)
  - Structurarea cerințelor
  - Design-ul experienței utilizatorului
  - Validarea cerințelor

Structurarea cerințelor

- Scop: Imbunătățirea mentenanței cerințelor de-a lungul timpului
- Tool-uri:
  - Izolare și identificare: (gestionarea și manipularea mai ușoară a fiecărei cerințe)
  - Organizare și clasificare (e.g. FURPS)
  - Adnotări (prioritate, importanță, trasabilitate)

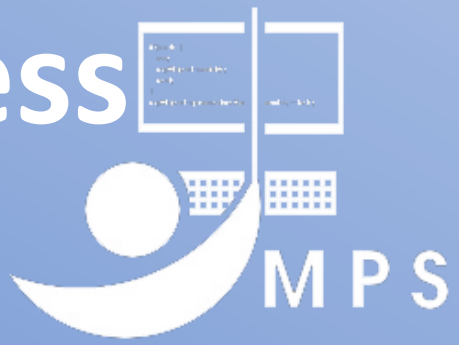
Design-ul experienței utilizatorului

- Scop: Asigurarea unei experiențe coerente și satisfăcătoare asupra diferitelor componente ale unui sistem software, incluzând design-ul, interfața, interacțiunea și manualele
- Tool-uri:
  - Analiza centrată pe utilizator: înțelegerea felului în care utilizatorii interacționează cu sistemul (grupuri țintă, experimente)
  - Design centrat pe utilizator: specificarea felului în care utilizatorii vor interacționa cu sistemul (mock-ups)

Validarea cerințelor

- Găsirea (și soluționarea):
  - Inconsistențelor
    - scenariul 1: R1. A; …; Rn: diferit de A
    - scenariul 2: R1. oricare ar fi x. A(x); …; Rn: A(c) fals
  - Cerințelor incomplete
    - comportamentul nu este specificat pentru anumite cazuri și situații (adesea situații non-nominale)
  - Duplicatelor
    - aceleași cerințe sunt descrise de două ori (posibil în feluri diferite)

Modelarea procesului de business si reingineria



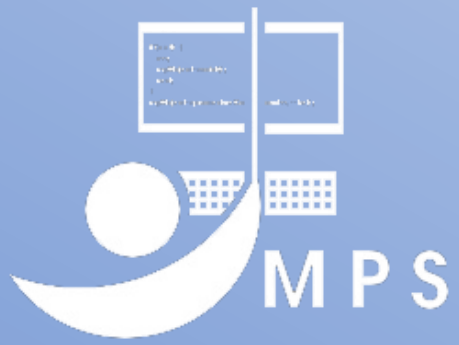
Organizații și Software

- Software-ul trebuie creat în așa fel încât să se potrivească în structura operațională a unei organizații
- Totusi: software-ul poate de asemenea schimba felul în care funcționează o organizație
- Modelarea procesului de business determină felul în care functionează o organizație
- Reingineria procesului de business planifică felul în care funcționează o organizație, în vederea eficientizării operațiunilor sale.

Modelarea procesului de business

- Proces complex, este uneori planificat și organizat ca un proiect independent
- Efectuat prin interviuri, analiza documentelor, shadowing
- Informații de colectat:
  - Structura organizațională: lanțul de responsabilitate
  - Procesele de business
  - Infrastructura IT existentă: hardware, sisteme, baze de date
  - Entități de business: datele produse și procesate de către organizație

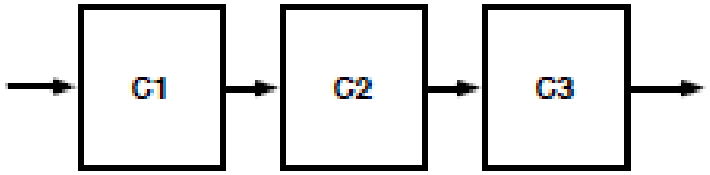
Design-ul sistemului



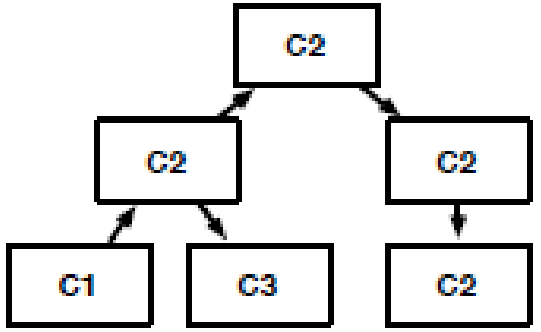
Design-ul sistemului

- Scop: Definirea structurii software-ului ce trebuie construit (= arhitectura sistemului)
- Ieșiri:
  - componentele ce constituie sistemul
  - funcțiile implementate de către fiecare componentă
  - cum sunt interconectate componentele
- Activitatea este relevanta si din motive manageriale: arhitectura sistemului produce descompunerea naturală a muncii

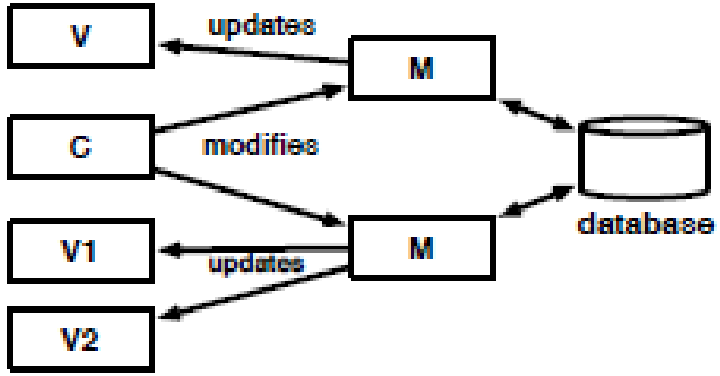
Modele Arhitecturale



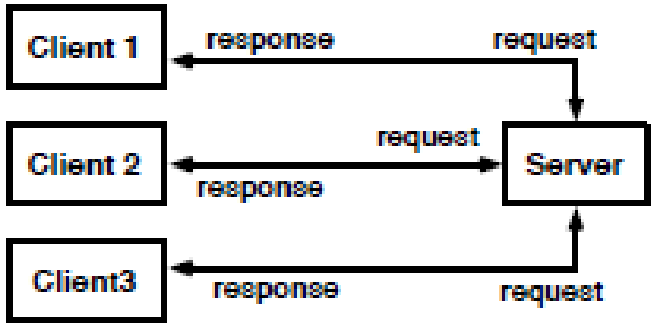
A Pipeline Architecture



A Layered Architecture



A Data-Centric applicaton with two MVCs



A client-server Architecture

Modele Arhitecturale

- “Pipe and filter”
  - Alcătuirea unităților de procesare a datelor
  - Important: specificațiile I/O
- Pe niveluri / ierarhic
  - Ierarhia componentelor
  - Important: controlul și fluxul informației; resposabilitățile blocului
- Data-Centric
  - MVC: date, prezentare, logică
  - Important: modelul de date, operațiuni
  - Multe aplicații web sau desktop folosesc acest stil architectural
- Client-server
  - Server (principalele funcții) și client (servicii de tip request)
  - Important: protocolul de comunicație / specificațiile serviciului

Implementare

Implementare

- Scop: Scriere de cod
- Câteva activități relevante pentru project management ce au loc în timpul implementării:
  - Colectarea metricilor de productivitate și mărime
  - Colectarea metricilor de calitate
  - Utilizarea standardelor de documentatie și scriere de cod
  - Practici de managementul codului (versionare; standard de lansare a codului)

Verificare și validare

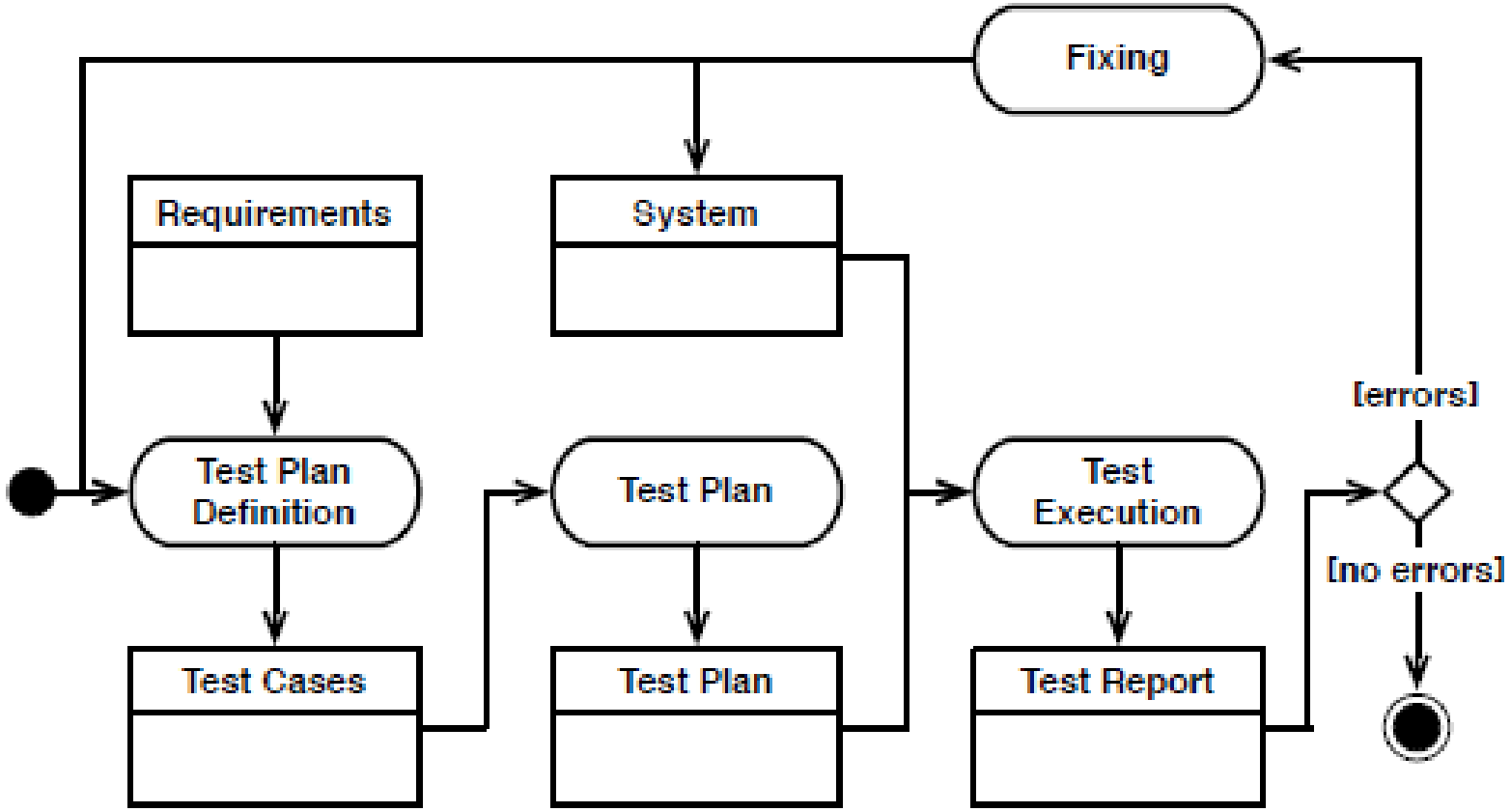
Verificare și validare

- Validare = construim sistemul care trebuie?
- Verificare = am construit sistemul cum trebuie?
- Cunoscute colectiv sub acronimul V & V
- Parte a managementului calității
- Principala (dar nu singura) metodă de a realiza V & V pentru sistemele software este testarea

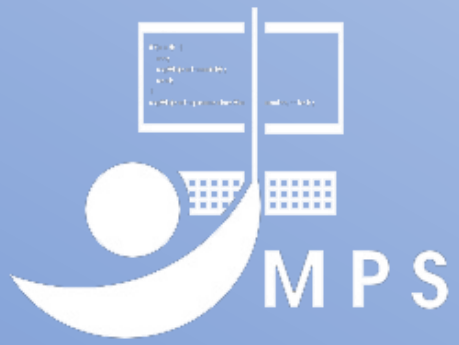
Tipuri de testare

- Unit testing
  - Scop: o secvență de cod, ca de exemplu o clasă
- Testare de integrare
  - Scop: interacțiunea dintre două componente
  - Bug-ul Mars Climate Orbiter: două componente au folosit diferite unități de măsura (metric și imperial) => ~ 400 milioane dolari pierduți
- Testarea sistemului
  - Scop: sistemul se comportă precum se aștepta și implementează în mod correct toate cerințele
  - Cazuri de test
- Testarea utilizabilității
  - Scop: verifică dacă experiența utilizatorului și interacțiunea sunt intuitive, eficiente și satisfăcătoare
  - Folosită pentru a reduce probabilitatea erorilor umane (sisteme de siguranță critică)

Procesul de testare a sistemului



Deployment



Deployment

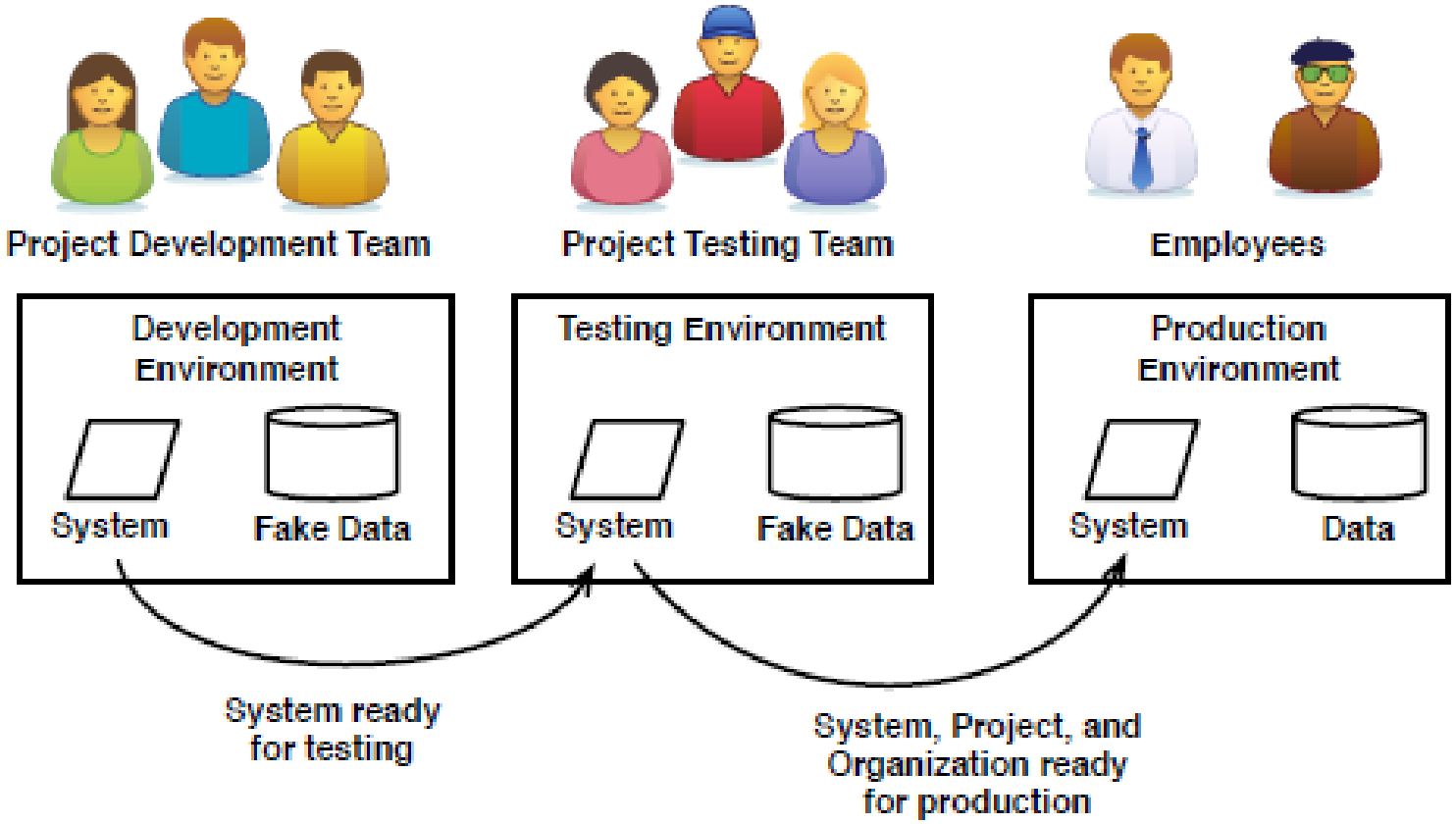
- Scop:
  - A instala noul sistem și a-l face operațional
- Câteva roluri:
  - Asigurarea continuității operațiunilor de business
  - Migrarea datelor
  - Tranziția către operațiuni și mentenanță

- Factori de luat în considerare
  - Factorul uman: oamenii sunt gata să utilizeze sistemul?
  - Factorul datelor: sunt disponibile pentru noul software toate datele necesare pentru ca sistemul să ruleze?
  - Factorul hardware: sunt toate interfețele gata și funcționale?

Abordări

- Cut-over: noul system il inlocuieste pe cel vechi
- Abordare paralelă: vechiul și noul sistem operează simultan pentru o perioadă
- Pilotare: noul sistem este instalat pentru un număr limitat de utilizatori sau pentru o unitate de business specifică
- Abordarea în faze: funcțiile sunt lansate incremental

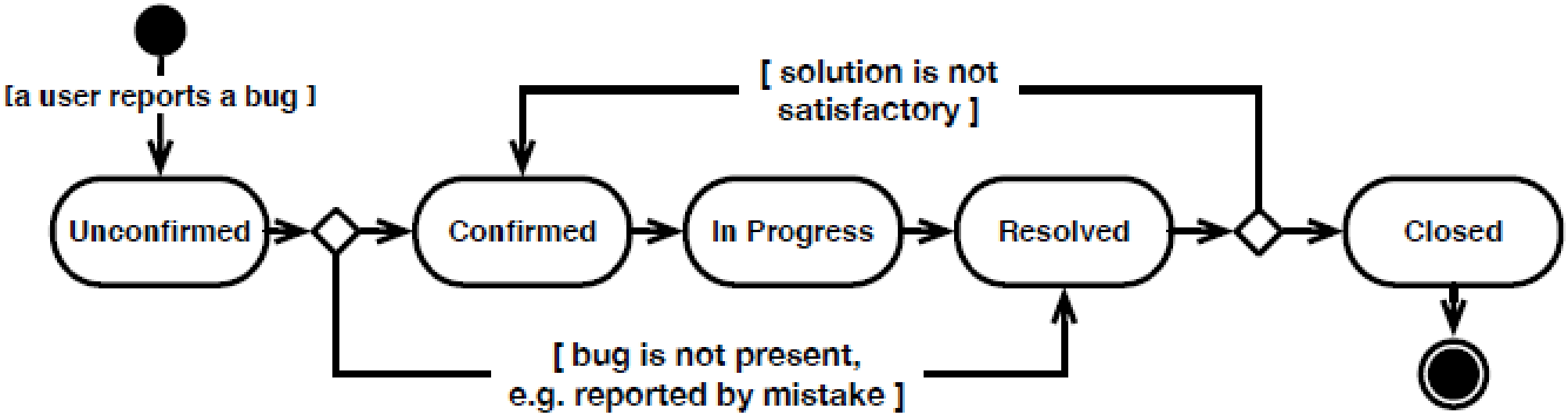
Gestionarea evoluției Software-ului



Operațiuni și mentenanță

Operațiuni și mentenanta

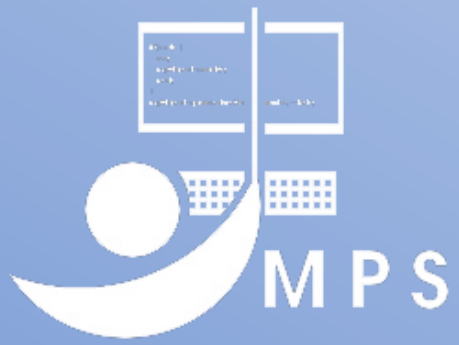
- Scop: Asigurarea bunei funcționări a sistemului
- Activități:
  - Oferirea de Suport Tehnic
  - Monitorizarea performanței sistemului
  - Colectarea și gestionarea tichetelor (clarificări, bug-uri, cereri de îmbunătățire)
  - Declansarea activităților de mentenanță


[a user reports a bug]
[solution is not satisfactory]
Unconfirmed
Confirmed
In Progress
Resolved
Closed
[bug is not present, e.g. reported by mistake]

Tipuri de mentenanță

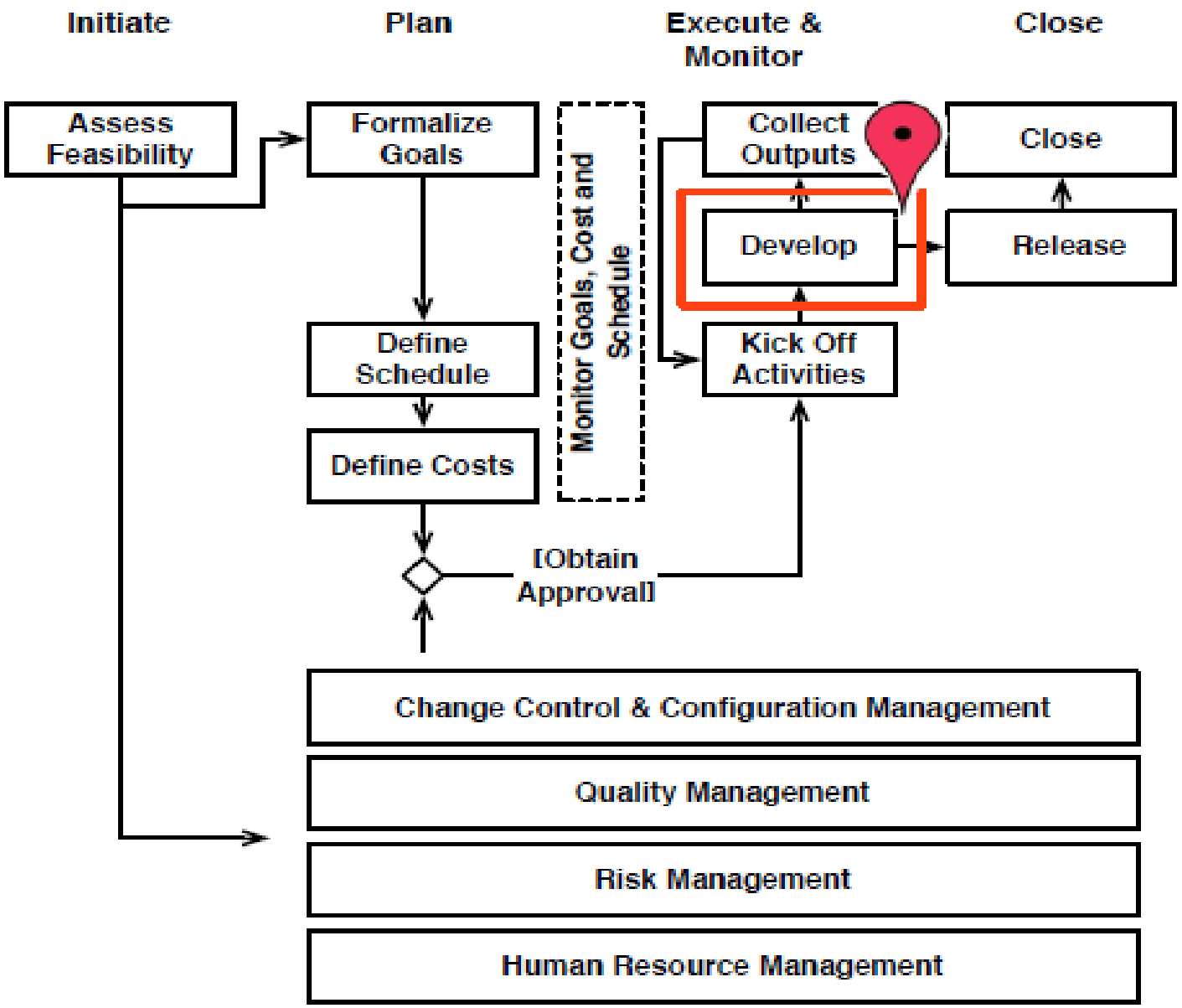
- Corectivă, referitoare la repararea unei probleme apărute după lansarea sistemului
- Preventivă, referitoare la repararea unei probleme descoperite, dar neproduse încă (sau, cel puțin, nesemnalizate de către utilizatori)
- Adaptivă, referitoare la adaptarea sistemului la condiții externe schimbate
- Perfectivă, referitoare la îmbunătățirea unor caracteristici ale sistemului, ca de exemplu, a performanțelor

Activități (tradiționale) de dezvoltare a Software-ului



Obiective

- O introducere subtilă și la nivel înalt în activitățile de dezvoltare a software-ului
- Descoperirea pietrelor de temelie necesare pentru producerea de software
- Remarci:
  - Acest curs nu il inlocuieste pe cel de software engineering
  - Activitățile trebuie integrate într-un process coerent pentru a avea sens
  - Proiectele de dezvoltare a software-ului variază de la extrem de reduse la extrem de extinse … nu toate activitățile sunt la fel de utile sau relevante în orice context


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
[Obtain Approval]
Collect Outputs
Develop
Kick Off Activities
Close
Release
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Prezentare generală

- Dezvoltarea de software necesită o perfecționare progresivă, ce transformă conceptul în operații prin urmatoarele faze:
  - Stabilirea cerințelor și a experienței utilizatorilor
  - Design
  - Implementare
  - Verificare și validare
  - Deployment
  - Operații și mentenanță
- O dată cu parcurgerea acestor faze, luăm anumite alegeri specifice; costul schimbărilor crește în consecință
- Procese diferite pun accentul pe fiecare activitate sau definesc ordinea în care aceste activități pot fi efectuate.

Procesul Software

- Procesul Software: **un set coherent de activități pentru specificarea, design-ul, implementarea și testarea sistemelor software.**
- Procesul software gestionează tranziția de la concept la produs
- Un model de process software reprezintă **reprezentare abstractă** a unui proces. Acesta prezintă o descriere a procesului dintr-o perspectivă particulară.
- Caracteristicile unui proces:
  - "Building blocks" (blocurile de construcție)
  - Ordinea în care blocurile sunt executate
  - Intrări / ieșiri ale fiecărei activități
  - Nivelul de formalitate (când și cum se realizează deplasarea către următoarea activitate; ce trebuie urmărit)

Building Blocks: ce

Modelarea mediului / business-ului	Descrie mediul (oamenii, organizația, celelalte sisteme) în care software-ul va rula	Descriere textuală, diagrame (BPMN, UML)
Cerinte (“ce” – ul)	Descrie calitățile externe ale software-ului (ce văd utilizatorii) și constrangerile	Descriere textuală, layouts GUI, cazuri de utilizare, povești de utilizator etc.

Exemplu

Modelarea Business-ului

1. Conducatorul unității primește cererea de cheltuieli

2. Conducatorul unității autorizează și trimite mai departe către administrație

3. Administrația verifică disponibilitatea bugetului și trimite mai departe către achiziții

4. Achizitiile continuă procesul

5. …

Cerințe

1. Sistemul trebuie să îi permită conducatorului unității să autorizeze cererea de cheltuieli

2. …

FURPS

- Funcționalitate
- Uzabilitate
- Fiabilitate (Reliability)
- Performanță
- Suportabilitate

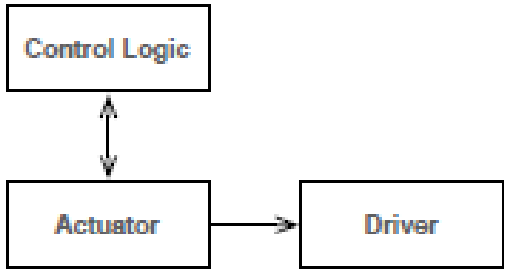
Building Blocks: cum

Design architectural **cum**	Descrie cum este sistemul structurat pe componente	Diagrame (clase, componente, …)
Implementare	Implementarea cerințelor conform structurii definite în design-ul arhitectural	Cod

Exemplu

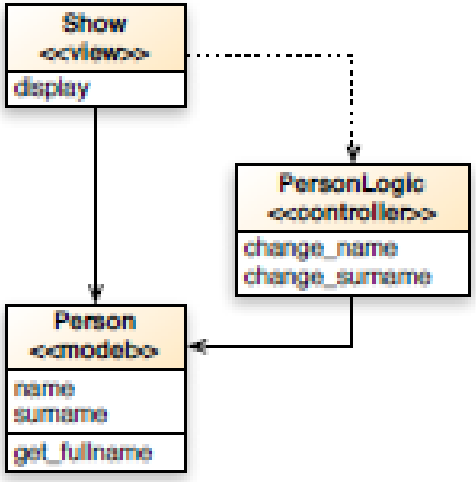
Câteva notații

- Diagrame de flux de date (data flow)
- Descompunere ierarhică
- Diagrame de stare
- Diagrame de clase
- …



Obiective

- Din ce componente este alcătuit un sistem (logic / fizic)
- Cum interacționează
- Cum se comportă
- …



Building Blocks: v&v

Verificare (“functioneaza cum trebuie?”)	Verifică dacă implementarea funcționeaza conform așteptărilor	Raport (și specificațiile testelor – cod sau text)
Validare (“este ceea ce trebuie?”)	Verifica dacă sistemul face ceea ce era prevăzut în cerințe	Raport (și specificațiile testelor – de obicei text)

Diferite scopuri / reprezentari:

- unitate (testul unei secvențe de cod)
- componenta / system (testarea unei componente): specificații textuale / scripturi executabile

Diferite tipuri:

- testare white box (se cunoaște codul sursă)
- testare black box (nu se cunoaște structura internă)

Exemple

- Liste textuale / executabile de teste
- Adesea sub formă:
  - cu sistemul într-o anumită stare
  - dacă unele evenimente se întâmplă (e.g. interacțiune cu utilizatorul)
  - când un anume output este așteptat
- Exemplu:
  - După logarea cu succes ca utilizator simplu
  - Dacă utilizatorul apasă butonul admin, este trimis către o pagină de eroare ce afișează utilizatorul nu este autorizat
- Poate fi automatizat (chiar și cu GUI)

Building Blocks: going live

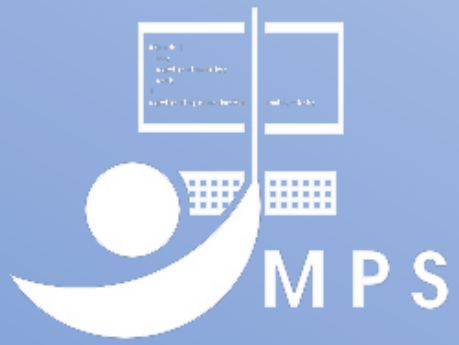
Documentație pentru utilizator	Manualul de utilizare și alte informatii pentru a ajuta utilizatorii (functionalități și utilizarea sistemului)	Manual
Distribuire și Deployment	Activități pentru a se lansa live și a ajunge la clienti	Pachetul de instalare a software-ului, Appstore, Website, Aplicații Web
Mentenanță	Ce se face după lansarea software-ului (menționat uneori; abordările moderne de software development consideră mentenanță ca un nou proiect)	

(Unele) Preocupări transversale

Definiția Planului de Business	Cum veți face bani din sistem	
Managementul Configurației	Activități legate de menținerea coerenței sistemului atunci când evoluează	Lista de obiecte, repository
Managementul Proiectului	Activități necesare pentru a menține proiectul sub control	Acest curs ☺
Imbunatatirea Procesului	Măsurarea performanțelor și asigurarea îmbunătățirii personale în ceea ce privește producerea de software	

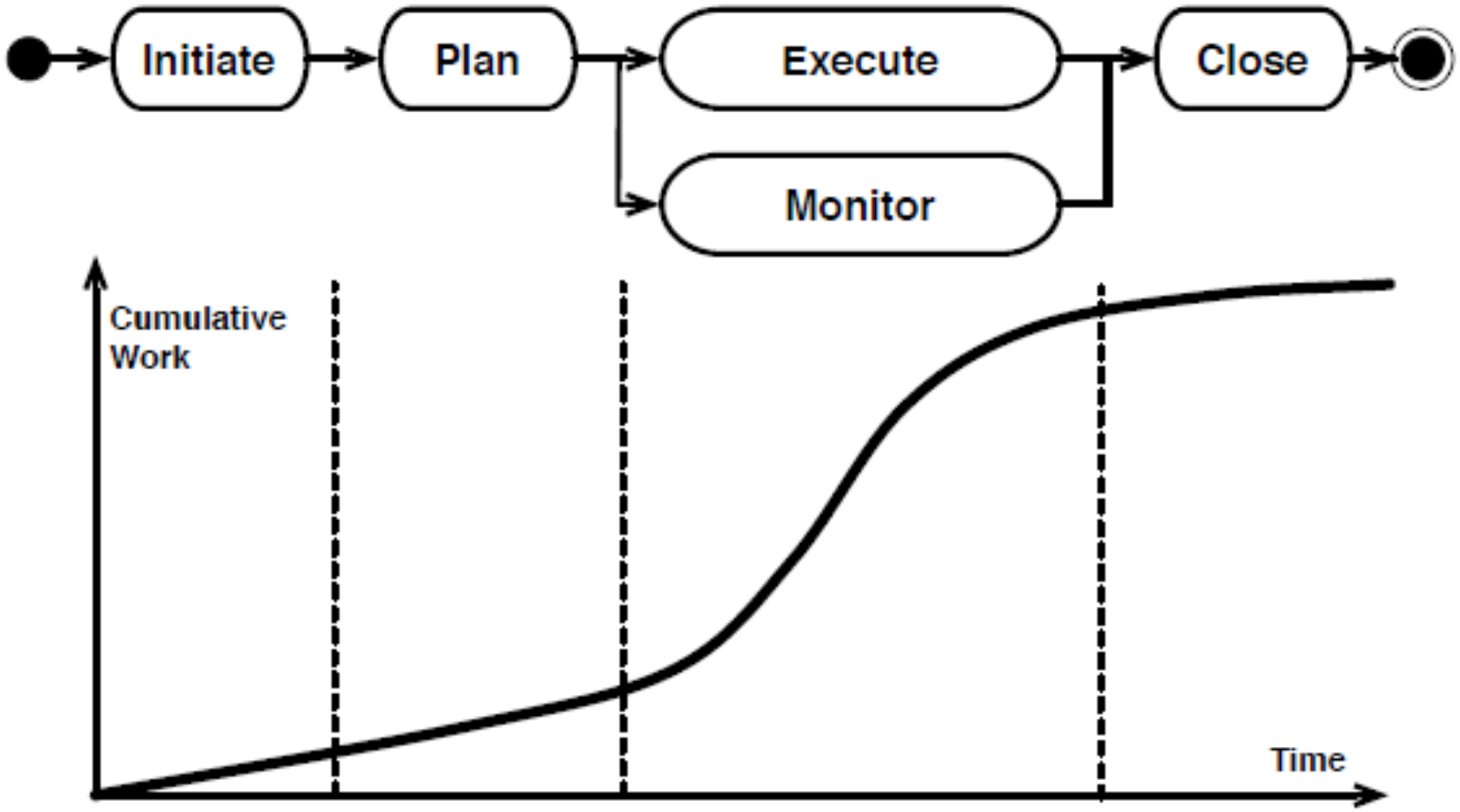
Ce este un proiect?

Numele jocului



Un proiect este un efort temporar **realizat pentru crearea unui produs, serviciu sau rezultat** unic**.**

Elaborare progresivă



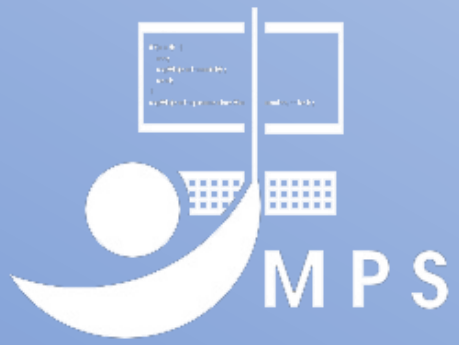
Proiecte și proceduri operaționale

- Lucrul poate fi pe bază de proiect sau operațional
- Caracteristici comune:
  - Realizat de oameni
  - Resurse limitate
  - Planificat, executat și controlat
- Diferențe
  - Proiect: obținerea obiectivelor și finalizare
  - Operațional: susținerea business-ului

Exemple (și contraexemple)

- Pregătirea cinei
- Fabricarea unui mașini
- Design-ul unei mașini
- Redactarea unui articol
- Dezvoltarea unui sistem software
- Menținerea unui sistem software
- Managementul personalului

Proiecte de Dezvoltare Software



Tipuri de Proiecte de Dezvoltare Software

În cariera de project manager, o persoană poate fi implicată în diferite tipuri de proiecte de dezvoltare de software, precum:

- Dezvoltarea de aplicații
- Reingineria proceselor și a sistemelor
- Integrarea sistemului
- Servicii de consultanță
- Instalare și training

A. Dezvoltarea de aplicații

- Scop → dezvoltarea unei aplicații (desktop, web, mobil, incorporate)
- Tipul cel mai distractiv ☺
- Tipuri de dezvoltare de aplicatii:
  - **One-off-uri** → sisteme create special pentru client
  - **Off-the-shelf** → pentru satisfacerea nevoilor unei game largi de utilizatori
  - **Off-the-shelf customizat** → sisteme standardizate ce necesita o cantitate semnificativa de customizare pentru a putea fi folosite intr-o organizatie. Exemplu: sisteme de tip ERP (**Enterprise Resource Planning**)

B. Reingineria proceselor și a sistemelor

- Scop: schimbarea modalității de desfășurare a procedurilor operaționale din cadrul unei organizații în vederea realizarii unui scop strategic (ex: îmbunătățirea calității, a eficienței)
- De obicei proiecte ample, care necesită o analiză precisă a situației existente din punct de vedere al procedurilor, sistemelor, infrastructurii
- Necesită adesea introducerea unui sistem ERP și a activităților de integrare a sistemului și a datelor

C. Servicii de integrare a sistemului

- Scop: automatizarea fluxului informational in cadrul sistemelor unei organizatii
- Tipuri de integrare:
  - **Orizontal** → integrarea sistemelor ce realizeaza operațiuni similare
  - **Vertical** → integrarea sistemelor care automatizeaza diferite etape ale unei proceduri

Alte tipuri de proiecte

D. Servicii de consultanță

- Cerute de obicei pentru a îmbunătăți comunicarea cu clienții (know-how despre competențele companiei)

E. Servicii de instalare si training

- Servicii legate de instalare și training pentru sisteme software specific
- Remarcă: constituie de asemenea sursă de venit în dezvoltarea open source

Proiectele și mediul lor

Jucatorii

Un stakeholder reprezintă un individ sau o organizație ce este implicat(ă) activ într-un proiect, sau al cărui interes ar putea fi influențat (pozitiv sau negativ) în urma realizarii sau finalizării proiectului.
(PMBOK)

Jucătorii

- Câteva caracteristici:
  - Pot avea influențe diferite și diverse niveluri de responsabilitate în timpul proiectului
  - Pot juca roluri diferite
  - Pot avea o influență pozitivă sau negativă asupra proiectului
  - Pot fi dificil de identificat
  - Lipsa intervenției lor poate influenta in mod negativ proiectul (nevoie de identificare si implicare)
- Remarcă: managerul de proiect si echipa de proiect **sunt** și ei stakeholderi în cadrul proiectului, desi termenul este adesea folosit pentru a desemna “ceilalți” stakeholderi

Tipuri de Stakeholderi

- Project managerul
- Echipa de proiect
- Sponsorul proiectului
- Organizațiile participante
- Partenerii
- Clientul
- Restul: oricine ar putea fi afectat de rezultatele proiectului

Stakeholderi Cheie

- Interni:
  - Membrii echipei de proiect: grupul ce realizează munca
  - Echipa de management a proiectului: membrii echipei direct implicați în activitatea de project management

- De mijloc:
  - Client / Utilizator: persoana sau organizație ce va utiliza rezultatele proiectului. Pot fi mai multe niveluri de utilizatori.
  - Sponsor: persoană sau grup ce furnizează resursele financiare
  - Organizatie participant: organizația implicată cel mai mult în realizarea proiectului

- Externi:
  - Influenceri: persoane sau grupuri neimplicate direct în proiect, dar care i-ar putea influența cursul

Exercițiu – Identificare Stakeholderi

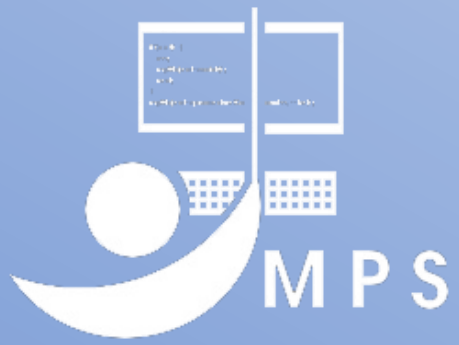
- Identificati stakeholderii următoarelor proiecte
  - Un proiect pentru construirea unui pod ce conectează o insulă de continent
  - Un proiect pentru construirea unui deposit de deșeuri (groapă de gunoi)
  - Un proiect pentru construirea unei foi de calcul (spreadsheet) open source
  - Un proiect pentru dezvoltarea unei aplicații web pentru monitorizarea masei corporale
  - Un proiect pentru a porta OpenOffice (suita Office gratuita) pe Android

Managerul de Proiect

- **Managerul de Proiect**
  - Persoana responsabilă cu gestiunea proiectului și a așteptărilor stakeholderilor

- **Câteva competențe necesare:**
  - Comunicare și negociere
  - (Oarecare) Predispoziție către riscuri
  - Orientare către scop
  - Abilități de leader
  - Gândire creativă, în afara tiparelor
  - Cunoștinte tehnice solide
  - Corectitudine profesională
  - Bun simț
  - Stil

Organizarea dezvoltării proiectelor software



Managementul Proiectelor Software

- **Managementul proiectelor software (Software project management)** reprezintă integrarea tehnicilor de management în dezvoltarea software
- Nevoia unei asemenea integrari își are rădăcinile în anii ‘60, în zilele “crizei software”, când specialiștii recunosteau complexitatea crescătoare a livrării produselor software ce îndeplinesc specificațiile

Ce face un Software să fie considerat Produs?

- O aplicație pentru gestionarea filmelor vizionate
- Un sistem de contabilitate cu intrare dublă pentru acasă sau pentru mici întreprinderi
- Un serviciu web pentru gestionarea rezervărilor terenurilor de squash într-un club de squash
- Un sistem de planificare a resurselor într-o corporație
- Un sistem de frânare automată pentru trenuri

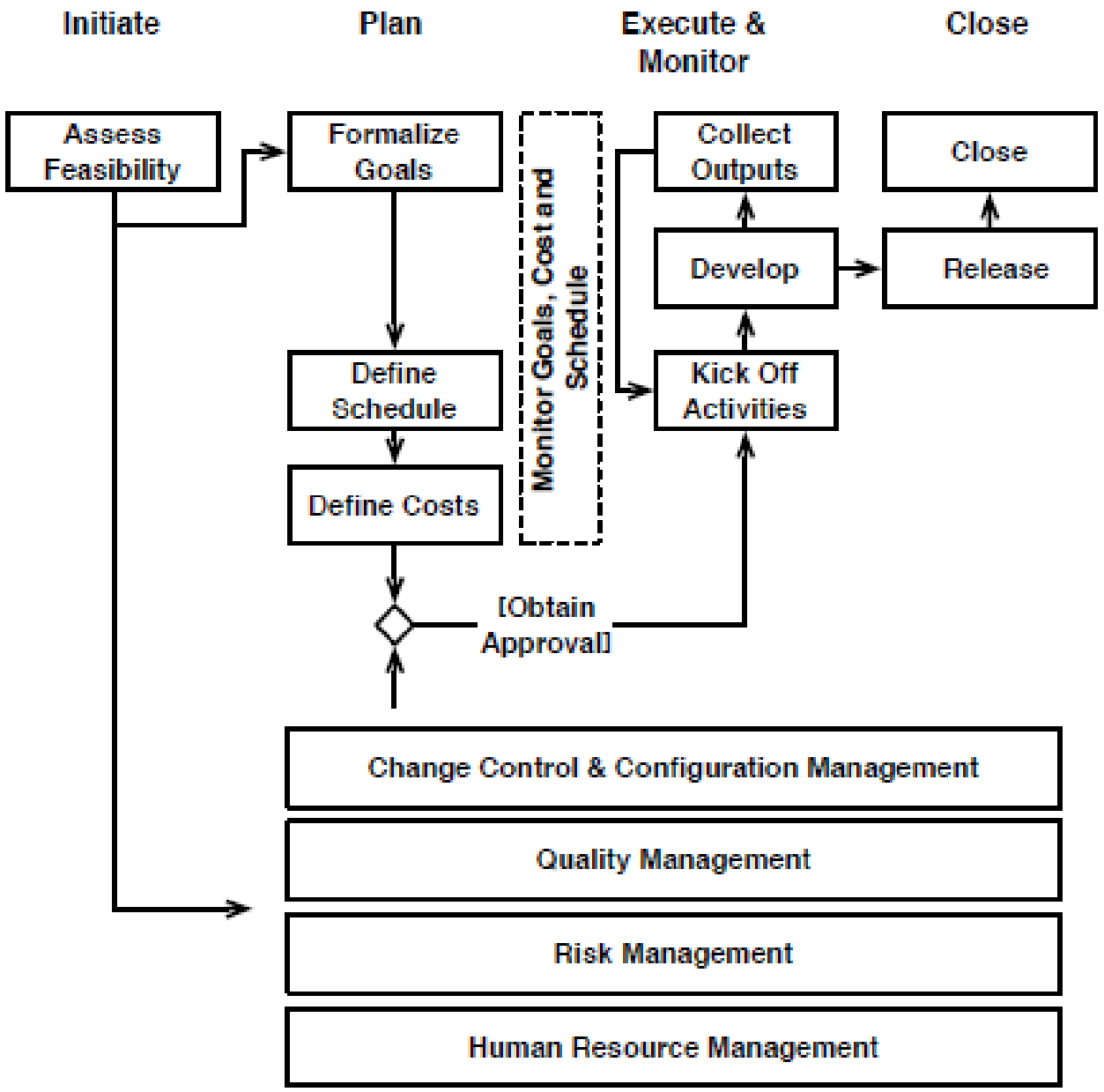
- Din ce componente este compus?
- Care ar fi efectele în cazul unui eșec?
- Ce competente sunt necesare pentru dezvoltare?
- Care este modelul de business asociat?

Framework de Dezvoltare Software

- Un framework general pentru MPS trebuie să:
  - Formeze o viziune comună despre scopurile ce trebuie realizate, caracteristicile rezultatelor proiectului și caracteristicile procesului de dezvoltare
  - Structureze lucrul ca o perfectionare progresivă, de la specificații la scopuri
  - Reducă impactul nesiguranțelor și a necunoscutelor
  - Evidențieze orice deviere de la plan (scopuri, costuri, calitate)
  - Asigure coerență și calitatea componentelor proiectului de-a lungul timpului în pofida necunoscutelor și a schimbărilor (sau a cererilor de schimbare)
  - Motiveze echipa

Câteva preocupări

- Evaluarea fezabilității
- Managementul scopurilor
- Managementul timpului
- Managementul costurilor
- Controlul schimbărilor și managementul configurației
- Managementul calității
- Managementul riscurilor
- Managementul resurselor umane


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Inițierea unui proiect

Fezabilitatea și autorizarea unui proiect

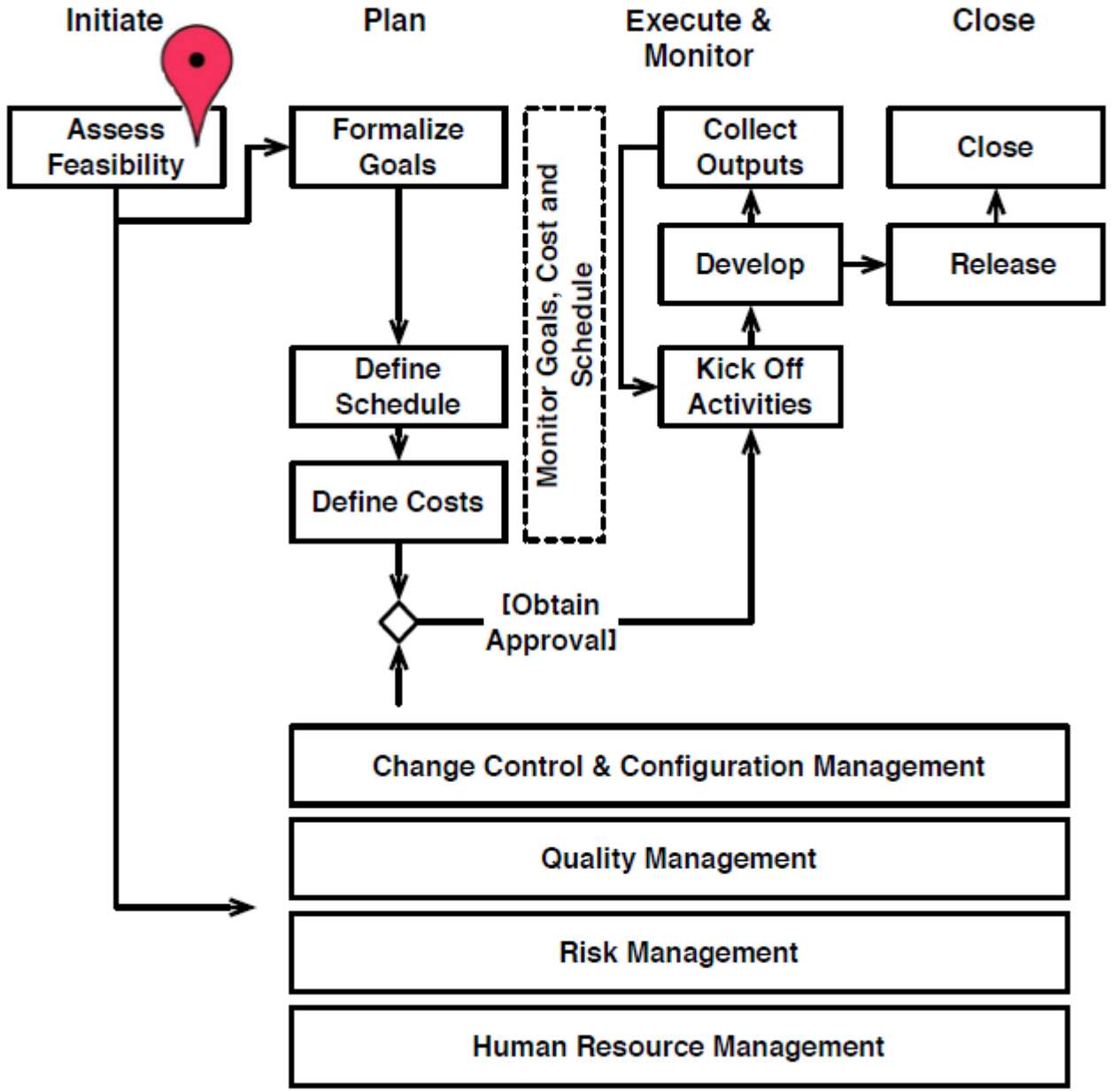
Obiective

- Înțelegerea tehnicilor calitative și cantitative pentru a selecta dintre anumite proiecte
- Înțelegerea tehnicilor calitative și cantitative pentru a alege cea mai bună alternativă dintre diferite implementări ale aceluiași proiect
- Înțelegerea redactării unui Studiu de Fezabilitate
- Alegerea între dezvoltare internă sau externă (a face sau a cumpăra)

A badly planned project will take three times longer than expected - a well planned project only twice as long as expected.

Cum începe un proiect?

- Inițierea de către un stakeholder (o companie, un potențial client) condus de o anumită nevoie (piață, socială, legală, tehnologică etc.)
- Limitele și procesele nu sunt întotdeauna clare sau formalizate
- Primele activitati realizate pentru a:
  - Decide obiectivele
  - Înțelege valorile și riscurile (pentru organizație și pentru celelalte părți implicate)
  - Alege o abordare a proiectului


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Valoarea proiectului și riscurile

PERFORMANCE REVIEW
YOU NEED TO GET BETTER AT ANTICIPATING PROBLEMS.
IF I COULD ANTICIPATE PROBLEMS, I WOULDN'T HAVE AGREED TO WORK FOR YOU.
YOU SEEM ANGRY. I DID NOT SEE THAT COMING.
GET US SOME RISK MANAGEMENT SOFTWARE.
WHAT CAN RISK MANAGEMENT SOFTWARE TELL YOU THAT MY COMMON SENSE AND EXPERIENCE CAN'T?
DATA.
STOP FAILING THE TURING TEST!
www.dilbert.com
scottadams@aol.com
© 2001 Scott Adams, Inc./Dist. by UFS, Inc.

Valoarea proiectului și riscurile

- Două caracteristici principale determină dacă un proiect merită să fie pornit sau nu:
  - Valoarea generată de proiect
  - Riscurile asociate proiectului
- Semnificația valorii și a riscului depind de o mulțime de factori
- Valoarea și riscurile pot fi evaluate calitativ sau cantitativ
- Proiectele (și oamenii) sunt imprevizibile
- Garbage in = Garbage out

Valoarea proiectului și riscurile

- Valoarea Proiectului
  - Valoarea directă și indirectă generate de proiect
  - Durabilitatea rezultatelor proiectului
  - Alinierea la obiectivele strategice ale unei organizații
- Riscurile proiectului
  - Disponibilitatea resurselor
  - Sincronizare
  - Dificultăți tehnice și incertitudini

Valoare - Directă, Indirectă

- Măsoară ieșirile (outputs) și urmările pozitive sau negative ale unui proiect
- Unele metrici de luat în considerare includ:
  - Venituri, directe și indirecte
  - Impact social și asupra mediului
  - Imagine și publicitate
  - Cunoștintele dobândite
- Sunt strict legate de modelul de business și de sustenabilitatea rezultatelor proiectului.

Sustenabilitate

- Capacitatea de a susține proiectul și rezultatele sale după ce acesta a luat sfârșit
- Luarea în considerare a costurilor operaționale ale rezultatelor unui proiect, precum și a felului în care aceste rezultate supraviețuiesc după sfârșitul proiectului reprezintă considerații importante pentru a afla dacă un proiect merită demarat sau nu
- Sustenabilitatea este adesea trecuta cu vederea, mai ales când executarea proiectului generează venituri

Alinierea cu obiectivele strategice

- Măsoară cât de important sau relevant este un proiect pentru organizație
- Prioritatea, resursele alocate, suportul intern, oportunitățile pentru echipa de proiect după finalizarea proiectului sunt toate afectate de cât de strategic este proiectul pentru organizație.

Riscuri - Disponibilitatea Resurselor

- Proiectele necesită **disponibilitatea de resurse umane, financiare și tehnice** în anumite intervale de timp
- Deși poate fi dificil de preîntâmpinat nevoia exactă de resurse, o verificare a nevoilor proiectului este esențială
- Aspecte ce trebuie luate în calcul: resursa necesară, disponibilitatea curentă, proiecții despre nevoia și disponibilitatea viitoare, prioritatea și importanța proiectului

Riscuri - Timing

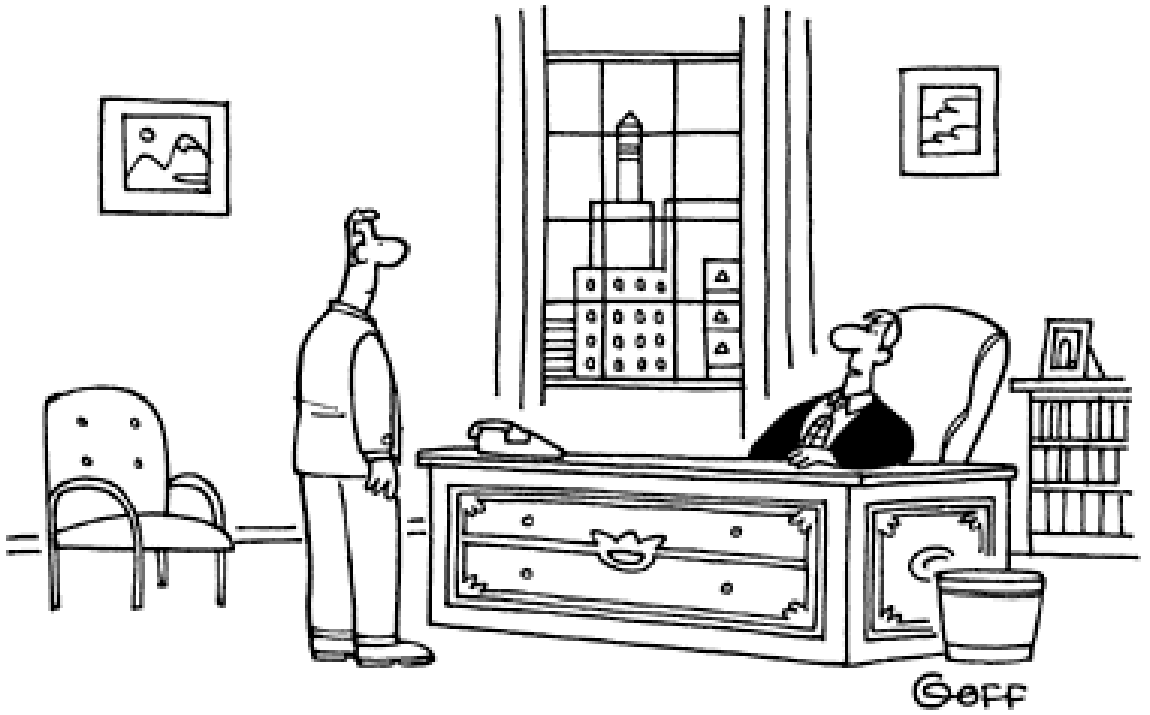
- Multe proiecte au **ferestre de timp specifice pentru livrarea rezultatelor**
- Livrarea prea devreme sau prea târzie poate face ca rezultatele proiectului să fie inutile
- De exemplu, să luăm în considerare cursa dintre firmele rivale ce livrează produse similare

Riscuri - Dificultati Tehnice si Incertitudini

- Succesul multor proiecte se bazează pe **capacitatea reală de a rezolva anumite provocări tehnice**, atunci când este cazul
- A înțelege care sunt aceste provocări reprezintă un factor important în determinarea riscurilor proiectului

Tehnici pentru evaluarea valorii și a riscurilor

© 2000 Ted Goff www.tedgoff.com



"I had to spend the money I budgeted for your raise on a therapist after listening to your endless complaining."

Perioada de rambursare

Perioada de rambursare reprezintă timpul necesar pentru a dobândi un câștig financiar egal cu investiția inițială.

- Măsurat în luni sau ani
- Atunci când se utilizează aceasta metodă, sunt alese proiectele / opțiunile care minimizează perioada de rambursare

Exemplu

	Project A	Project B	Project C
Year 0	€ (50,000.00)	€ (20,000.00)	€ (15,000.00)
Year 1	€ 30,000.00	€ (10,000.00)	€ 15,000.00
Year 2	€ 30,000.00	€ 10,000.00	€ 1,000.00
Year 3	€ 1,000.00	€ 60,000.00	
Year 4	€ 1,000.00	€ 50,000.00	
Expenses	€ (50,000.00)	€ (30,000.00)	€ (15,000.00)
Gains	€ 62,000.00	€ 120,000.00	€ 16,000.00
Profit	€ 12,000.00	€ 90,000.00	€ 1,000.00
Payback	2 years	3 years	1 year

Remarcă: notații în stil contabil, numerele negative cu roșu, între paranteze

Discuție

Avantaje

- Date simple, ușor accesibile
- Reduce expunerea la risc
- Eficiență în special în proiecte de tehnologie înaltă
- Favorizează beneficiile pe termen scurt

Dezavantaje

- Dificil de utilizat pentru proiecte pe termen lung
- Bazate doar pe fluxul numerar
- Nu cuantifică expunerea la risc
- Nu ține cont de câștigurile totale

Slăbiciunile rambursării

- Proiecte diferite pot avea aceeași perioada de rambursare, dar profiluri diferite în returnarea investițiilor
- Aceste profiluri nu sunt luate în considerare de către tehnică, dar ar putea face diferența dintre două proiecte

Slăbiciunile rambursării

Aceeași perioadă de rambursare, dar **Project A** aduce mai mulți bani mai repede (și reduce riscurile)

Year	Project A	Project B
Year 0	€ (10,000.00)	€ (10,000.00)
Year 1	€ (5,000.00)	€ (5,000.00)
Year 2	€ 10,000.00	€ 5,000.00
Year 3	€ 5,000.00	€ 10,000.00

Slăbiciunile rambursării

Perioade de rambursare diferite, **Project B** are perioada de rambursare mai scurtă, dar aduce mai puțini bani

Year	Project A	Project B
Year 0	€ (10,000.00)	€ (10,000.00)
Year 1	€ (5,000.00)	€ (5,000.00)
Year 2	€ 5,000.00	€ 5,000.00
Year 3	€ 5,000.00	€ 11,000.00
Year 4	€ 20,000.00	

Randamentul investitiei
(Return on Investment – ROI)

ROI calculează profitul anual mediu și îl transformă în procent al investițiilor totale

- Profit = Câștiguri – Investiții
- Profit Anual = Profit / Durată
- ROI = Profit Anual / Investiții
- Când se utilizează ROI, se alege proiectul cu cel mai mare ROI

Exemplu

Să presupunem că avem următoarele date pentru un proiect și trebuie să decidem dacă îl demarăm sau nu

	Project A	Project B	Project C
Year 0	€ *(50,000.00)*	€ *(20,000.00)*	€ *(15,000.00)*
Year 1	€ 30,000.00	€ *(10,000.00)*	€ 15,000.00
Year 2	€ 30,000.00	€ 10,000.00	€ 1,000.00
Year 3	€ 1,000.00	€ 60,000.00	
Year 4	€ 1,000.00	€ 50,000.00	

Exemplu

Soluție: Project B (cel mai mare ROI)

Project A

- Profit = 62000 – 50000 = 12000
- Profit anual = 12000/4 = 3000
- ROI = 3000/50000 = 6%

Project B

- Profit = 120000 – 30000 = 90000
- Profit anual = 90000/4 = 22500
- ROI = 22500/30000 = 75%

Project C

- Profit = 16000 – 15000 = 1000
- Profit anual = 1000/2 = 500
- ROI = 500/15000 = 3%

Fluxurile de numerar actualizate
Inflație

- Valoarea banilor scade de-a lungul anilor (inflație!) conform formulei:

$$\text{Discount Factor} = \frac{1}{(1+i)^n}$$

- Astfel, banii investiți acu ere peste 5 ani
- DCF (Discounted Cash Flow – Fluxurile de Numerar Actualizate) reprezintă tehnici care iau în calcul inflația
- De unde vine inflația? Din devalorizare.

http://en.wikipedia.org/wiki/Inflation

Valoarea actualizată netă (Net Present Value – NPV)

Valoarea Actualizata Neta folosește discount-uri asupra sumelor din viitor pentru a realiza o comparație mai realistă între investițiile prezente și câștigurile viitoare.

Valoarea actualizată netă - Exemplu

Rata de discount ipotetică: 10% (i-ul din formula)

$$\text{Discount Factor} = \frac{1}{(1+i)^n}$$

Year (n)	Cash Flow	Discount Factor	Present Value
0	€ (35,000.00)	1.00	€ (35,000.00)
1	€ 10,000.00	0.91	€ 9,090.91
2	€ 15,000.00	0.83	€ 12,396.69
3	€ 20,000.00	0.75	€ 15,026.30
Expenditure	**€ (35,000.00)**		**€ (35,000.00)**
Gains	**€ 45,000.00**		**€ 36,513.90**
Profit	**€ 10,000.00**		**€ 1,513.90**

Valoarea actualizata netă – Discuție

Avantaje

- Date mai exacte despre profit și pierderi

Dezavantaje

- Folosește o rată de discount fixă (poate fi nerealistă)
- Favorizează proiectele pe termen scurt

Matrice de scor

- Metodele financiare (rambursare, ROI, NPV) țin cont doar de datele financiare
- Matricele de scor permit luarea în considerare și a altor factori
- Se bazează pe un set standardizat de criterii și ponderi, care evidențiază caracteristicile relevante ale unui proiect
- O evaluare calitativă a scorurilor proiectului în funcție de fiecare criteriu permite poziționarea criteriului pe o scară și comparația cu proiecte trecute sau rivale

Matrice de Scor - Exemplu

Factor	Value	Weight	SUM	Comment
The project aligns with the strategic objectives	YES	2	2	
The project has a profit > 20%	NO	4	0	
Payback period < 2 years	YES	5	5	
Enlarges the customer base	YES	2	2	
The project requires a standard technology	NO	3	0	
The quality constraints are simple to meet	YES	1	1	
The timing is not too tight	NO	4	0	
We have skilled personnel to do the work	YES	5	5	
			15	

- Valorile pot fi binare (Da / Nu) sau numere (e.g. de la 1 la 5) și măsoara cât de bine îndeplineste proiectul cerința
- Ponderea (weight) măsoară cât de important este un factor pentru decizia finală

Discuție

Avantaje

- Simplă
- Încurajează standardizarea și luarea obiectivă a deciziilor
- Ajută la discutarea și evaluarea caracteristicilor proiectului
- Lărgește gama evaluării
- Nu favorizează proiectele pe termen scurt

Dezavantaje

- Un model simplu poate încuraja dezvoltarea unor liste lungi și inutile
- Factori diferiți au aceeași importanță (cu excepția cazului în care matricea de ponderi este folosită)

Avertisment

- Nu toate matricele de scor sunt la fel de bune
- Mai jos aveți un exemplu de matrice nepotrivită. De ce?

Factor	Value	Weight	SUM	Comment
The project has a profit > 20%	YES	3	3	
The project is highly risky	NO	3	0	
			3	

Avertisment

Asigurați-vă că toate aspectele influențează decizia fie pozitiv, fie negativ;
sau puteți folosi scoruri cu semn diferite!

Analiza SWOT

- Tehnica lui Albert Humphrey
- Analiza sistematica a:
  - Punctelor tari (Strengths)
  - Punctelor slabe (Weaknesses)
  - Oportunităților (Opportunities)
  - Amenințărilor (Threats)

  … pentru a înțelege fezabilitatea unui proiect și/sau a crea scopuri realizabile ale proiectului
- Adesea prezentată ca o matrice 2x2, cu fiecare celula listând toate elementele de un anume tip

Analiza SWOT

Sursă: http://en.wikipedia.org/wiki/File:SWOT_en.svg (cc license)

SWOT - Factori de luat în calcul

Puncte tari

- Competențe
- Puncte de vânzare

Puncte slabe:

- Dezavantaje
- Metodologie
- Timing
- Deficiențe de capabilitate

Oportunități

- Trenduri pe piață și în industrie
- Slăbiciunile competitorilor

Amenințări

- Trenduri pe piață și în industrie
- Tehnologiile competitorilor
- Sustenabilitate

Analiza părților implicate (stakeholders)

- Scop: a înțelege care sunt părțile implicate într-un proiect și care este influența lor asupra proiectului
- Diferite tehnici disponibile
- Una dintre tehnici organizează părțile implicate într-o matrice 2x2 în care:
  - o dimensiune măsoară puterea pe care o exercită un stakeholder (slabă sau ridicată)
  - cealaltă dimensiune măsoara interesul pe care îl are un stakeholder în proiect (negativ sau pozitiv)
- Aceasta permite definirea politicilor de management specifice pentru diferitele părți implicate

Evaluarea sustenabilității

- Această analiză are scopul de a permite înțelegerea costurilor operaționale ale rezultatelor unui proiect
- Reprezintă uneori o activitate specifică a proiectului
- O analiză preliminară a sustenabilității poate ajuta decizia de a alege una dintre implementarile proiectului
- Aspecte ce trebuie luate în calcul include **modelul de business** și **pragul de rentabilitate**

Studiul de fezabilitate

Studiul de fezabilitate

Reprezintă un document ce permite autorizarea formală a unui proiect și legarea lui la scopurile organizației

- Gama largă de rezultate: de la puține la câteva sute de pagini (în funcție de complexitate și formalitate)
- Studiul de fezabilitate poate fi văzut ca un proiect în miniatură, conturând informația principal ce va fi detaliată de-a lungul proiectului
- Baza pentru selecția proiectului: conducerea trebuie să aleagă ce proiecte vor fi activate

Scopurile studiului de fezabilitate

- Identificarea:
  - obiectivele proiectului
  - constrângerilor proiectului
- Evaluarea valorii și a riscurilor
- Asigurarea alinierii proiectului la:
  - obiectivele clientului
  - obiectivele organizației principale
- A demonstra că obiectivele proiectului
  - pot fi indeplinite respectând contrângerile de calitate, costuri și timp

Document de fezabilitate - Structură

- O declarație a muncii, ce descrie ce va realiza proiectul
- Obiectivele de business (valoarea) ale proiectului sau rezultatele și informațiile despre modelul de business, dacă sunt relevante
- Un sumar al bugetului proiectului, care prognozează cheltuielile și câștigurile
- Un sumar al milestone-urilor, mai exact o planificare grosieră ce include cele mai importante evenimente
- O analiză a părților implicate
- Riscurile proiectului
- Posibile alternative ale proiectului, de exemplu decizii de tipul a face sau a cumpăra
- O evaluare a proiectului și a alternativelor sale

Fezabilitate – considerații aditionale

Documentul de fezabilitate are valoare pentru:

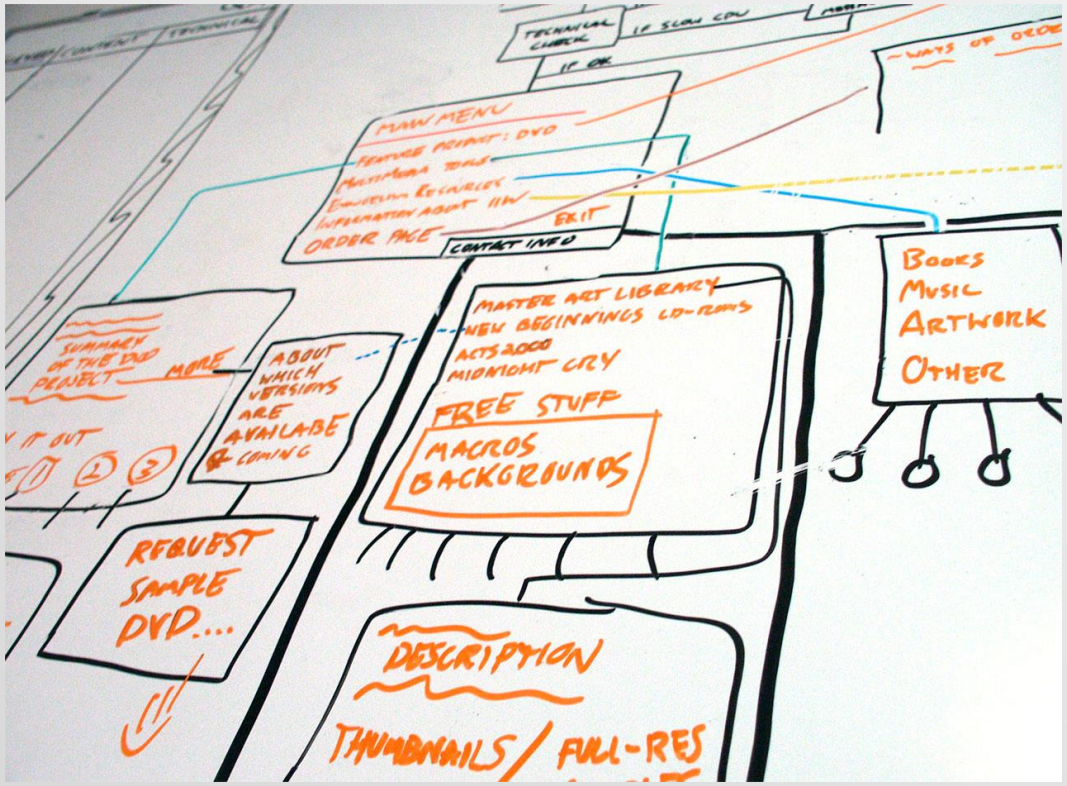
- **Client** – îl ajută să înțeleagă ceea ce urmează să fie realizat, precum și perspectivele pe termen scurt sau lung
- **Organizația principală** – o ajută să înțeleagă dacă are sens să se continue un proiect
- **Managerul de proiect** – îl ajută să înțeleagă dacă proiectul va fi sau nu în zona sa de confort (și să ia astfel o decizie obiectivă cu privire la a realiza sau nu proiectul)

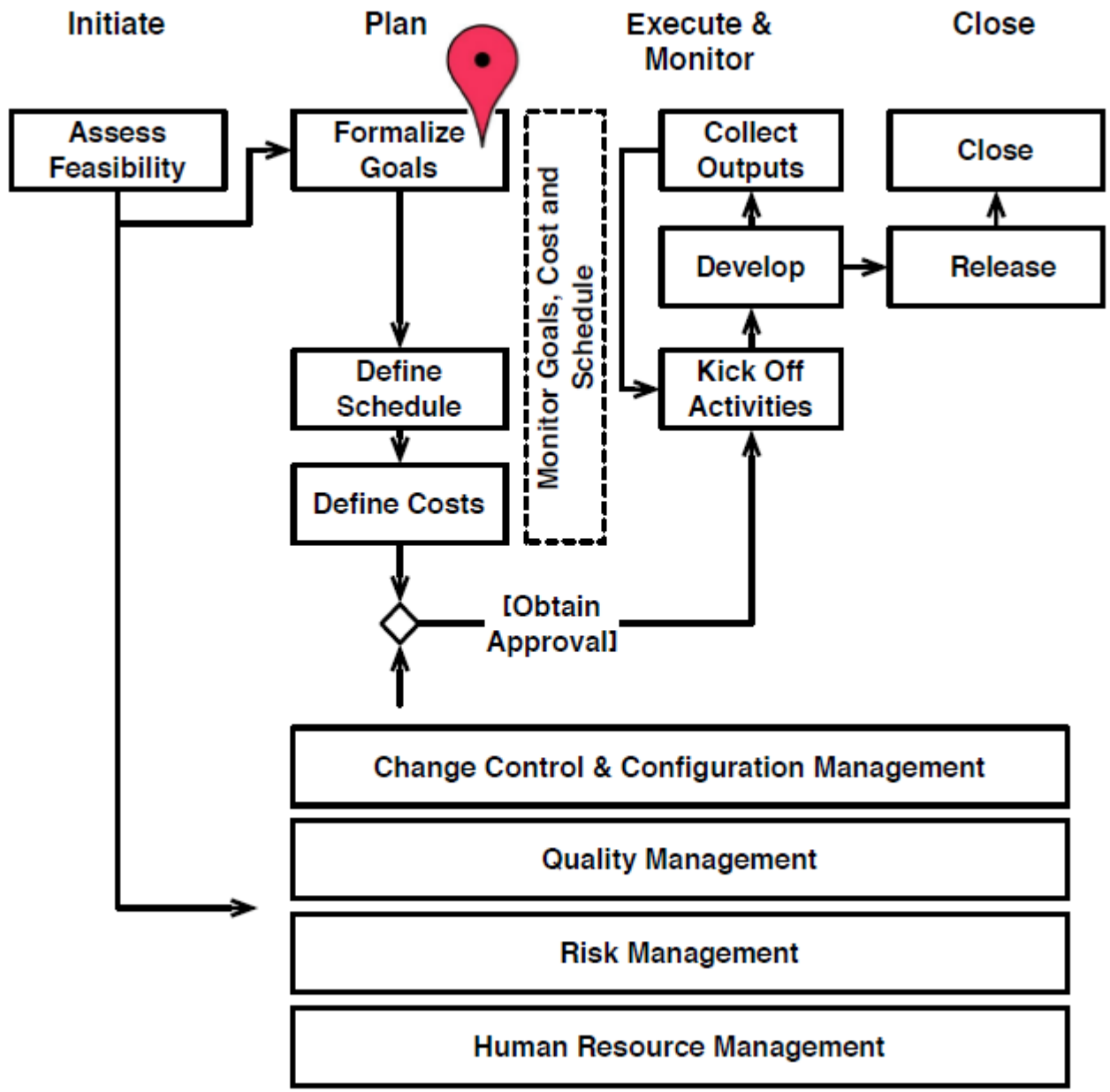
Procesul de aprobare al proiectului

- Procesul ce duce la aprobarea proiectului este mai mult sau mai puțin structurat conform practicilor organizației principale
- Este organizat după următorii pași:
  - Dupa primirea unei cereri, identificarea unui manager de proiect (preliminar)
  - Managerul de proiect pregătește un studio de fezabilitate care este aprobat de către client și principalele părți implicate
  - Managerul de proiect trimite documentele pentru autorizare
  - Documentul este analizat și o decizie formală este luată
  - Managerul de proiect definitiv este ales și proiectul trece în faza de planificare

Formalizarea obiectivelor proiectului

The first myth of management is that it exists.




Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Formalizarea obiectivelor proiectului

- Definirea obiectivelor proiectului (domeniul de aplicare al proiectului) reprezintă una dintre primele și cele mai importante activități din cadrul unui proiect
- Domeniul de aplicare al unui proiect:
  - Asigură că proiectul include doar (și toată) munca necesară
  - Stabilește baza muncii ce va fi efectuată
  - Definește un document de referință pentru acceptarea proiectului
- Definirea domeniului de aplicare al proiectului începe în timpul studiului de fezabilitate
- Obiectivele proiectului și alinierea proiectului la scopul său continuă pe tot parcursul ciclului de viață al unui proiect

Documentul de aplicare a proiectului

Acest document conține:

- Obiectivele si cerintele proiectului - descriu ceea ce vrem să realizăm prin proiect, principalele sale caracteristici și rezultate
- Presupuneri și constrângeri - ce descriu condițiile ce trebuie îndeplinite pentru a asigura succesul proiectului
- Rezultatele proiectului și punctele de control - ce descriu rezultatele proiectului și, în unele cazuri, o aproximare a timpului de livrare

Obiectivele și cerințele proiectului

- Obiectivele și cerințele proiectului sunt baza de definire a:
  - Bazei muncii de efectuat (comparație între Work Breakdown Structure si Change and Configuration Management)
  - Criteriile de acceptare a proiectului (comparație între închiderea proiectului și managementul calității)
- Strict legate de cerințele software
- Uneori este util să includă și ceea ce este în afara domeniului de aplicare a proiectului

Obiectivele proiectului

Make the objectives **SMART!**

- **Specifice** - Clare și concise
- **Masurabile** – Ușor de utilizat, măsuratori pentru a înțelege dacă scopul a fost atins. O dată, un număr, o formula (keep it simple!)
- **De comun acord** - Obiectivele trebuie sa fie suficient de specifice pentru ca echipa să le îndeplinească
- **Realiste** - Obiectivele trebuie să fie realiste. Obiectivele nerealiste stabilesc așteptări nerealiste și fac echipa apatică
- **Legate de timp** - Trebuie să aibă un început și un sfârșit. Dacă nu poate fi stabilit un sfârșit, este probabil vorba despre muncă operațională.

Obiectivele proiectului

Make the objectives **russian (MoSCoW)!**

- **M:** Must Have
  - Esențiale
- **S:** Should Have
  - Importante, dar nu esențiale
- **C:** Could Have
  - De dorit
- **W:** Won't Have
  - Nu vor fi realizate

Obiectivele proiectului - MoSCoW

- Trebuie să ne asigurăm că **acele criterii pentru succesul clasei M depind** de factori ce ne sunt sub control.
- Exemple negative (M nu e sub controlul managerului de proiect):
  - Sistemul va avea 1000 de utilizatori în prima luna (De ce tool-uri dispune managerul de proiect pentru a asigura indeplinirea acestui obiectiv?)
  - Viteza de introducere a datelor de către utilizatori va crește de zece ori (Cum se poate asigura managerul de proiect ca această creștere va fi atinsă?)
- Uneori contează foarte mult cuvintele alese pentru descrierea obiectivelor. Consecințele pot fi costisitoare.

Presupuneri

- **Presupunerile** sunt condiții considerate ca fiind adevărate, dar care în realitate pot fi false
- Presupunerile nu se afla sub controlul managerului de proiect, dar **pot fi sub controlul unora dintre părțile implicate în proiect**
- În acest caz, presupunerile pot fi folosite pentru a defini datoriile și obligațiile părților implicate
- **Constrângerile** sunt limitări cunoscute. Ele explică de ce alegem unele obiective în locul altora și de ce munca este structurată într-un anume fel.

Ieșirile proiectului - Milestone, Livrabile

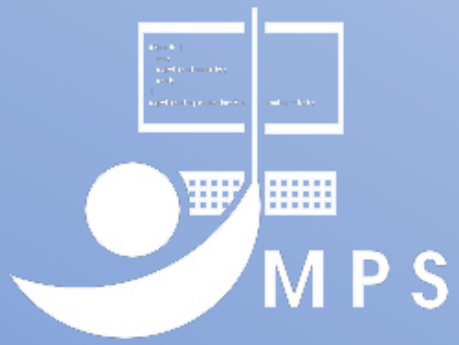
- Ieșirile Proiectului (Outputs) definesc ceea ce proiectul va realiza și când
- **Milestone** - un eveniment semnificativ din cadrul proiectului
  - Identifică puncte critice în proiect și în planificare
  - Adesea folosit în momente de review sau de livrare
  - Poate fi legat de termeni contractuali, constrângeri de calendar, **livrabile**
- **Livrabil** - un produs unic, măsurabil și verificabil al muncii
  - Poate fi intern sau extern
  - Poate avea diferite diseminări și niveluri de formalitate
  - In diagramele Gantt, ele interconectează adesea sarcini (output-ul unei sarcini este un livrabil ce constituie input-ul sarcinii următoare)

Ieșirile proiectului - Milestone, Livrabile

În practicile curente, adesea milestone-urile și livrabilele sunt folosite în mod interschimbabil (amândouă sunt folosite pentru a identifica produse; milestone-urile pot reprezenta produse cheie)

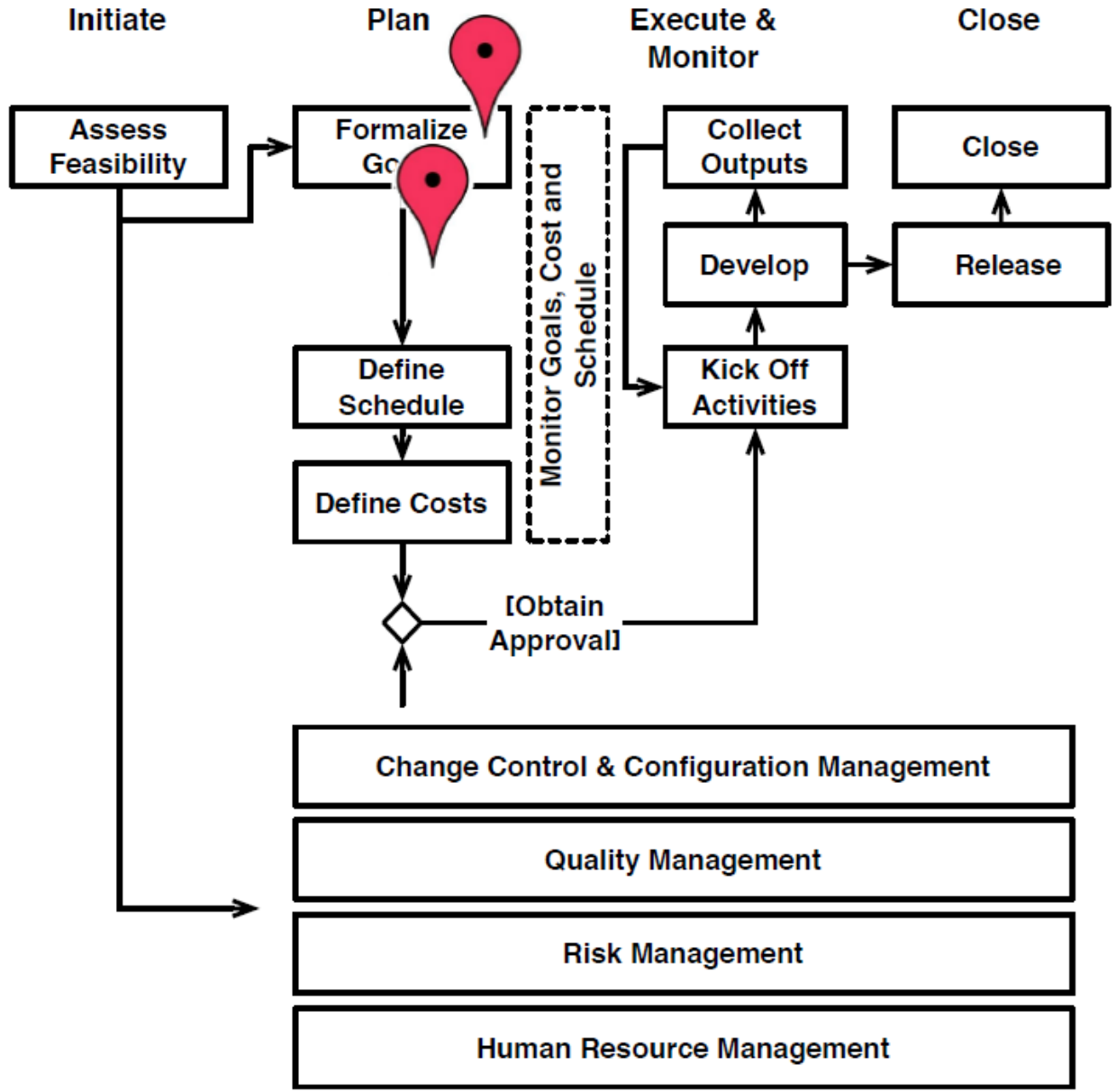
Ambele au durata **zero** in planificare

Work Breakdown Structure (WBS)



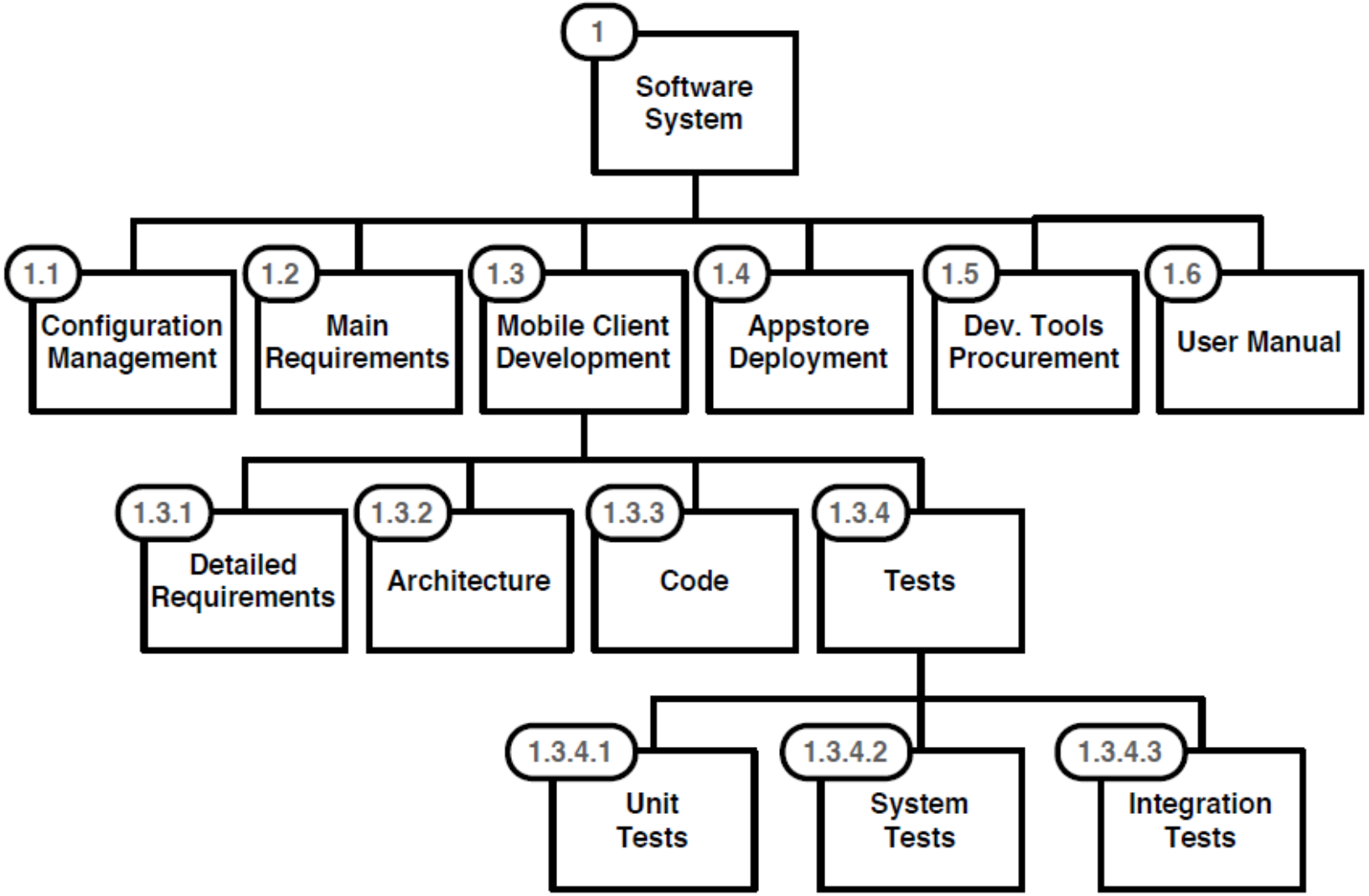
Obiective

- Înțelegerea tranziției de la obiectivele proiectului la planificarea proiectului
- Introducere în notația specifică pentru Work Breakdown Structure


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Go
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
Collect Outputs
Develop
Kick Off Activities
Close
Release
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Work Breakdown Structure (WBS) reprezintă o descompunere ierarhică (orientată spre livrabile) a muncii ce trebuie efectuate de către echipa de proiect pentru realizarea obiectivelor proiectelor și crearea livrabilelor dorite.

Exemplu WBS


1
Software System
1.1
Configuration Management
1.2
Main Requirements
1.3
Mobile Client Development
1.4
Appstore Deployment
1.5
Dev. Tools Procurement
1.6
User Manual
1.3.1
Detailed Requirements
1.3.2
Architecture
1.3.3
Code
1.3.4
Tests
1.3.4.1
Unit Tests
1.3.4.2
System Tests
1.3.4.3
Integration Tests

WBS – Observații

- Două formate:
  - Arbore graphic (Vision, Graffle, LibreOffice, …)
  - Schemă textuală (Microsoft Word, Text Editor, Outliner, …)
- Folosește un sistem numeric decimal pentru identificarea elementelor (Ex: 3.1.5)
- Arată legăturile de tipul este inclus în
- Nu include dependențe sau durate

De ce este util?

Un WBS stabilește baza pentru:

- Definirea lucrului ce va fi efectuat într-un proiect
- Stabilirea felului în care diferite activități sunt legate de obiectivele proiectului
- Stabilirea unui cadru pentru definirea, alocarea și monitorizarea lucrului și a costurilor
- Identificarea elementelor organizatorice necesare pentru finalizarea lucrului

WBS – Reguli generale

- Totul este la locul lui:
  - Regula 100% - asigurarea ca toate elementele lucrului sunt incluse (WBS-urile orientate catre produs sunt mai potrivite pentru aceasta regula)
  - Regula ER (Excludere Reciprocă): asigurarea că nu există suprapuneri în definirea elementelor
  - Nu este nevoie de echilibrare: nu este necesar ca toate drumurile să meargă până la același nivel
- Calitatea WBS-ului este ridicată
  - Coerenta: task-urile din acelasi pachet (work package) trebuie să aibă același scop
  - Cuplare: dependențele dintre pachete trebuie minimizate, astfel încât membrii echipei să poată lucra independent
  - Continuitate: producerea de pachete ar trebui să fie full-time pentru maximizarea eficienței
  - Cost: pachetele de la nivelul inferior ar trebui să necesite munca unui singur om, pe o perioadă între o săptămână și o lună

Cand ne oprim?

- Raspuns simplu: la nivel de pachet (care poate fi compus din mai multe activități elementare pe care nu este necesar sa le urmărim)
- Totusi: cat de mare este un pachet?
  - Conform “DOD and NASA Guide to PERT COST”: frunzele dintr-un WBS nu ar trebui să necesite mai mult de 3 luni de muncă sau 100.000 dolari cheltuiți
  - Conform altor standarde: 1-2 săptămâni pentru 1-2 persoane
- Nivelul de detalii depinde în schimb de dimensiunea proiectului

Tipuri de WBS (1/3)

- WBS de Produs
  - Evoluează conform structurii ieșirilor ce trebuie produse
  - Poate porni de la un Product Breakdown Structure
- WBS de Proces
  - Evoluează conform fazelor de organizare ale proiectului
  - De exemplu: Cerințe, Analiză, Design, Testare
- WBS Hibrid: ambele de mai sus
  - Îmbină procesul și produsul
  - De exemplu: fazele ciclului de viata la niveluri mai înalte; componente la niveluri joase

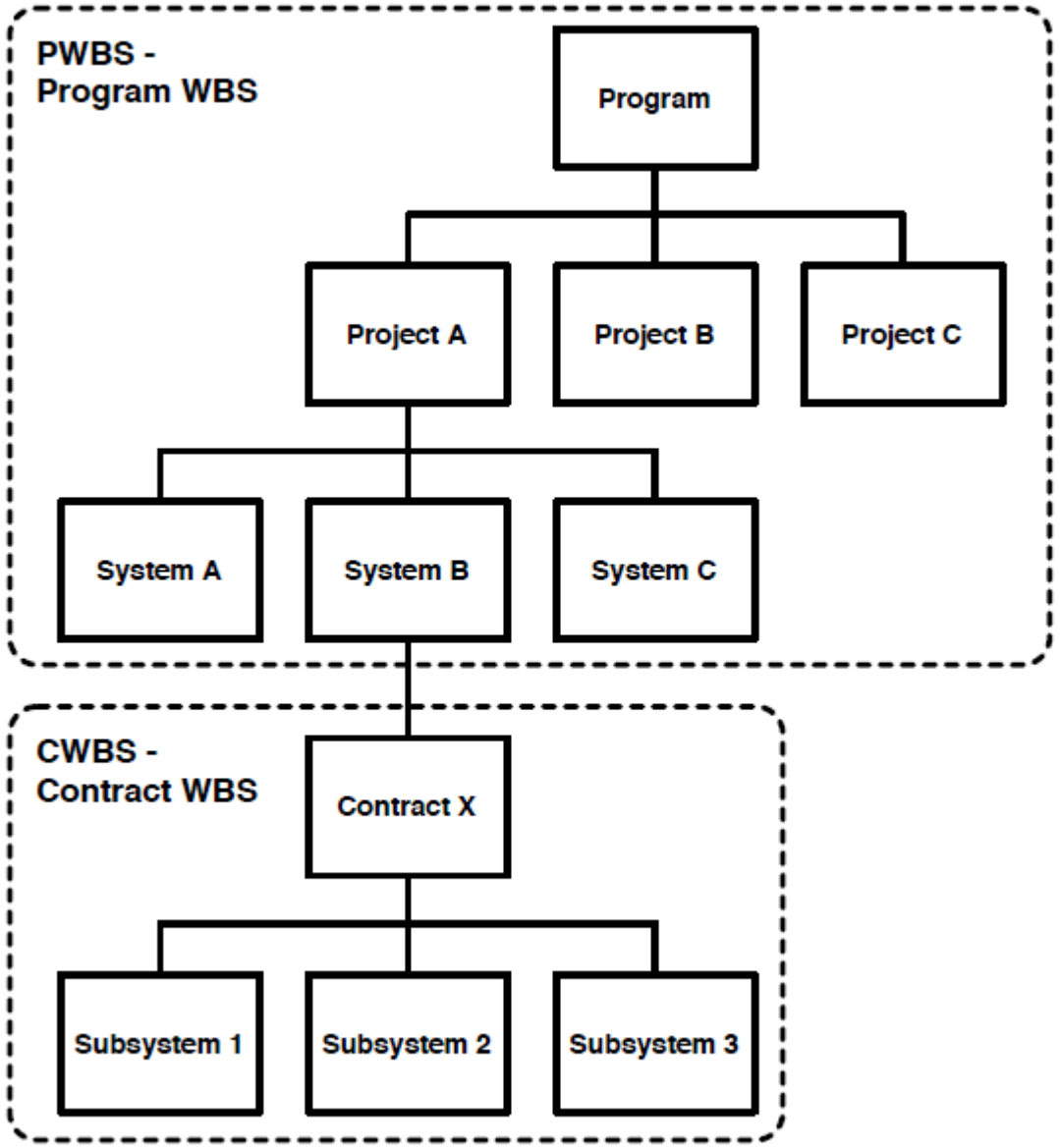
Tipuri de WBS (2/3)

- WBS de Organizare și WBS Geografic
  - Nivelurile înalte sunt unități organizatorice
  - Nivelurile joase reunesc munca ce se află sub responsabilitatea unei unități
  - Pot fi utile pentru proiectele multidisciplinare
- WBS Geografic
  - Nivelurile înalte sunt reprezentate de echipe distribuite geographic (e.g. echipa din NY, echipa din Roma)
  - Nivelurile joase reunesc munca ce se afla sub responsabilitatea unei echipe

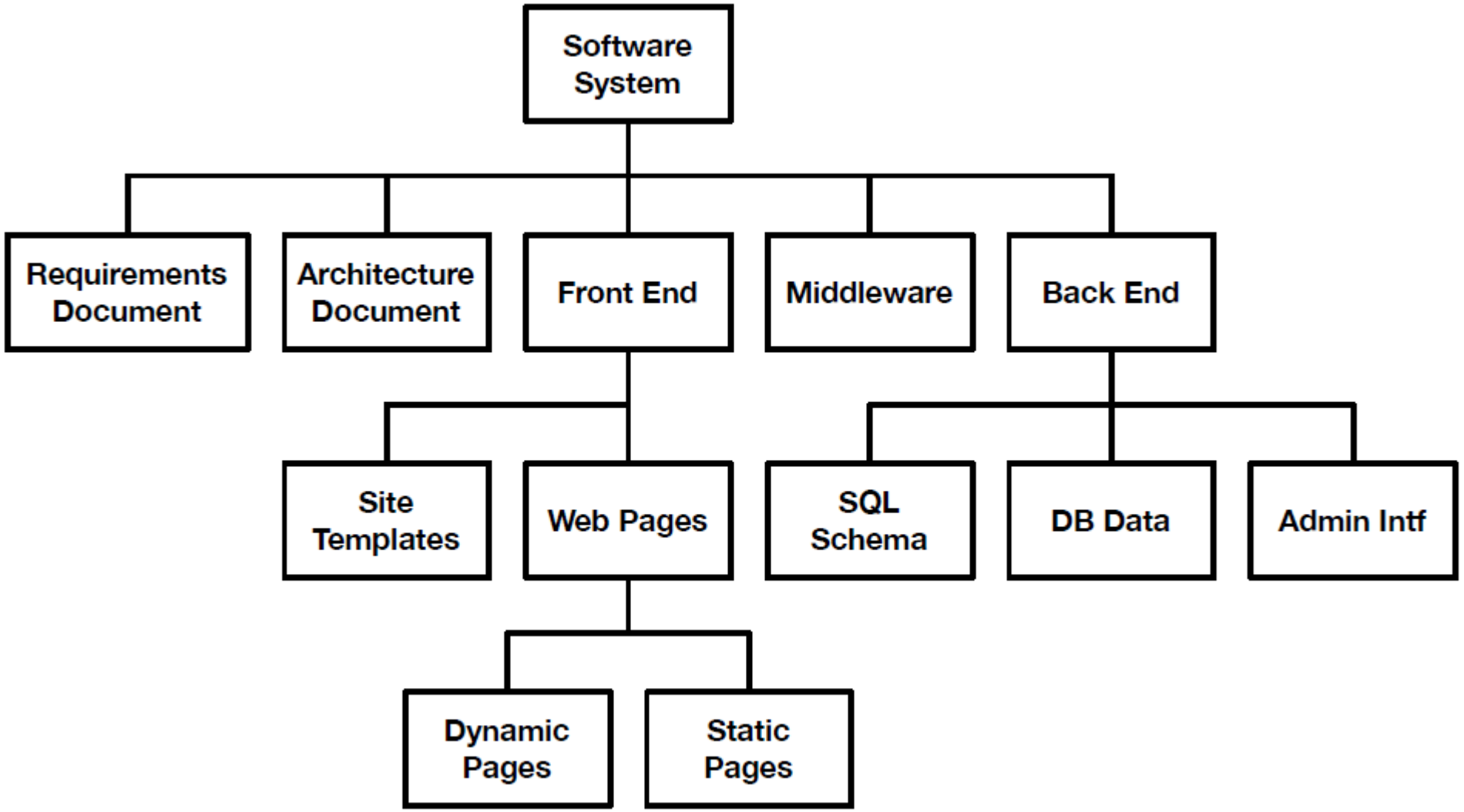
Remarcă: conform PMBOK, acestea nu sunt considerate WBS-uri. Sunt într-adevăr mai rar întâlnite.

Tipuri de WBS (3/3)

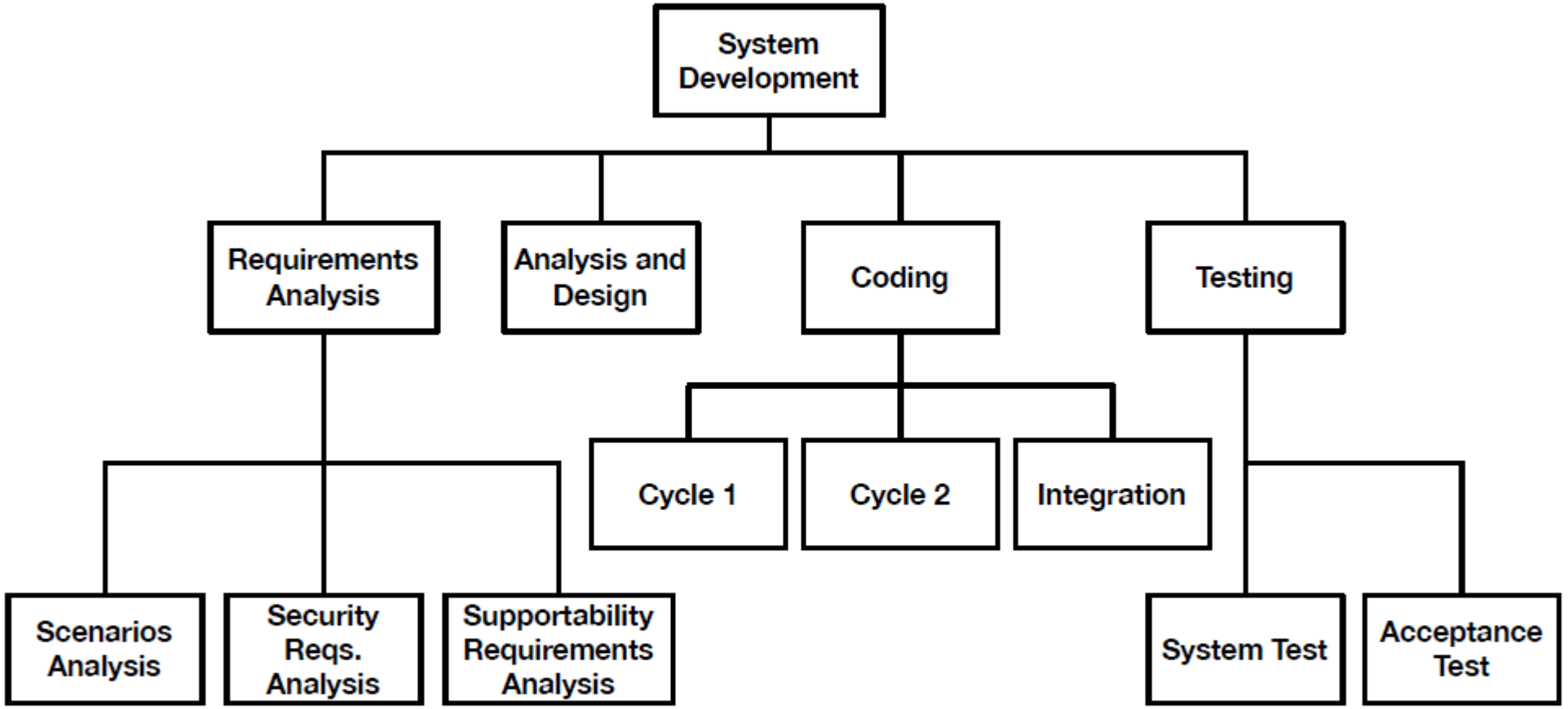
- PWBS - WBS de Program, folosit pentru coordonarea tuturor proiectelor (sistemelor)
- CWBS - WBS de Contract, baza pentru subcontractarea dezvoltării sistemului



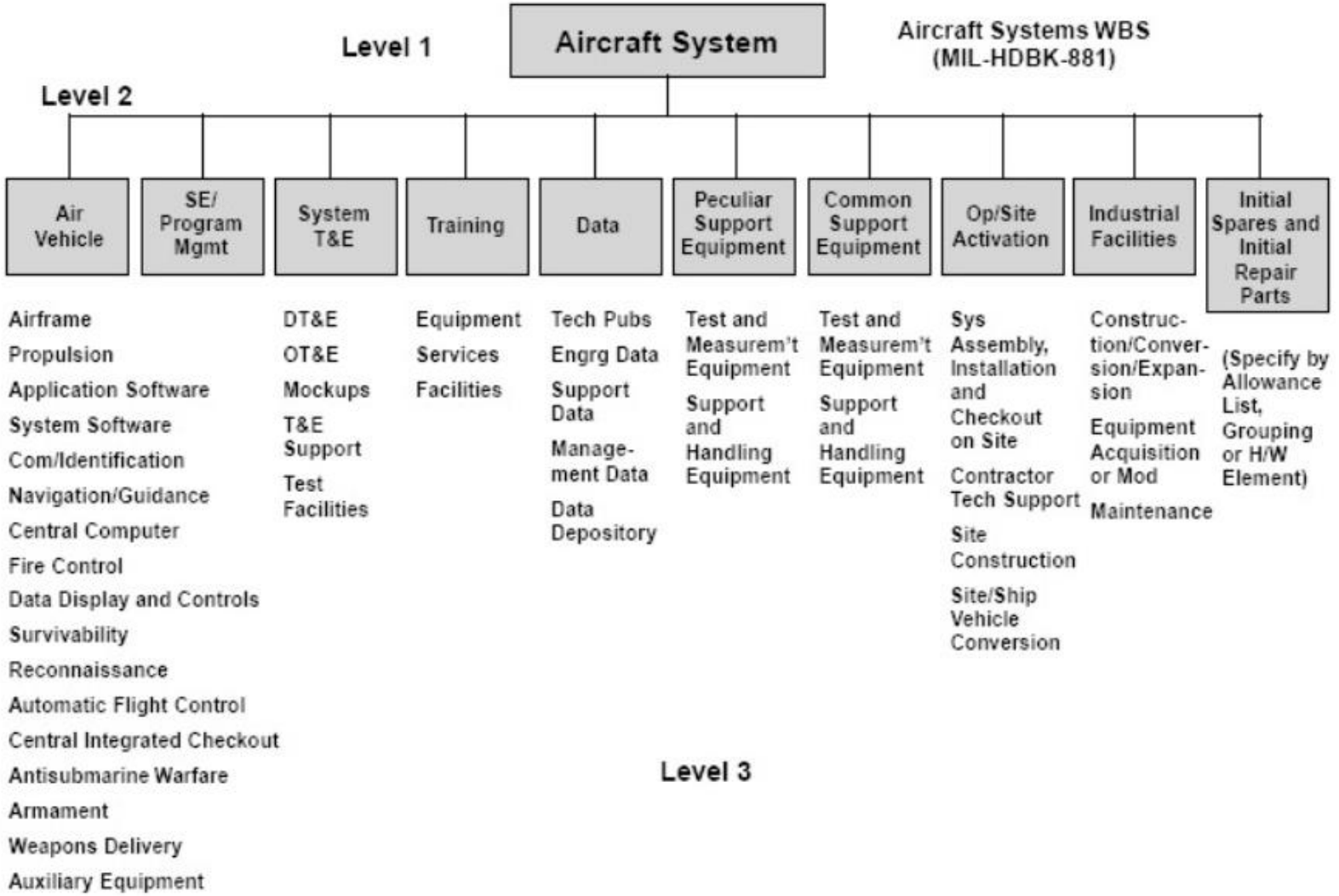
WBS de Produs - Exemplu



WBS de Proces - Exemplu



Exemplu WBS (MIL-HDBK-881)



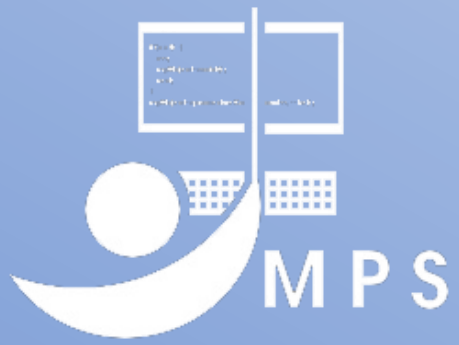
Dicționarul WBS

- Dicționarul WBS ajută la specificarea amanunțită a intrarilor unui WBS
- Poate conține titlu, număr, descriere detaliată a elementului, cantități, muncă asociată, elemente contractuale
- Reguli generale:
  - poate fi realizat pentru fiecare intrare a arborelui
  - urmariți definiția: creșteți cantitatea de detalii în timp ce coborâți în arbore
  - o practică bună este realizarea dicționarului pentru frunze (work packages)

Dicționarul WBS

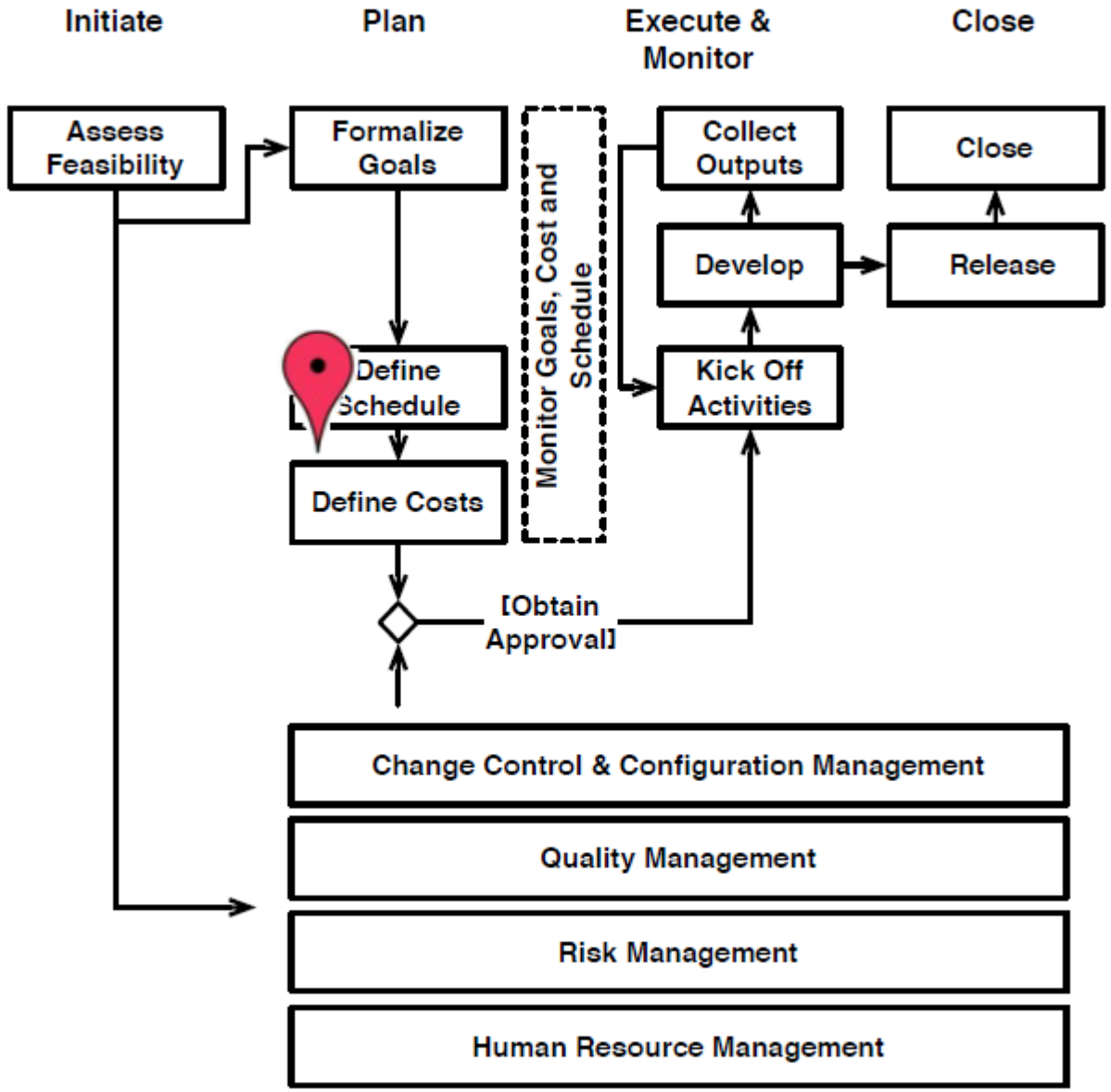
Work package number	1	**Start date or starting event:**	Month 1								
Work package title	Case Study Requirements and Experimentation Site Assessment										
Activity type	RTD										
Participant number	1	2	3	4	5	6	7	8			
Participant short name	P1	P2	P3	P3	P4	P5	P6	P7			
Person-months per participant	1	0	0	9	6	0	2	0			
Objectives											
Description of work	Task 1. Task 2. ...										
Deliverables	D1.1. D1.2.										
Milestones	M1.1. M1.2.										

Tehnici Algoritmice pentru Estimare: Function Points



Predictions are hard, especially about the future.

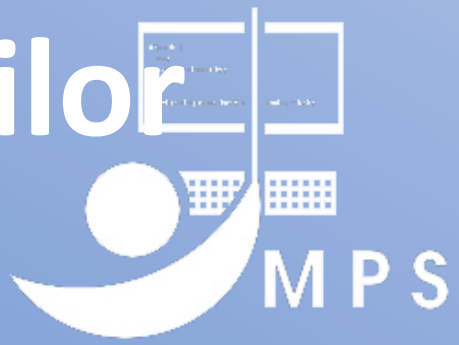
Yogi Berra


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
Collect Outputs
Develop
Kick Off Activities
Close
Release
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Obiective

- Înțelegerea principiilor de bază ale tehnicilor algoritmice și, în mod detaliat, a metodei de estimare Function Point

Recapitulare: Principiile Tehnicilor Algoritmice



Introducere

- Scop: găsirea unei metode pentru a determina sistematic efortul (durata) necesar pentru un task/proiect arbitrar
- Ideal:
  - Identificarea unui set de caracteristici **măsurabile** ale unui proiect ce determină efortul/durata acestuia
  - Definirea unei funcții ce calculează **efortul/durata** pe baza caracteristicilor menționate mai sus

$$f(x_1, \ldots, x_n) = e$$

Problemă: cum calculăm f, x_1, …, x_n?

Soluție

- Analizarea proiectelor/seturilor de date existente; fiecare proiect este reprezentat de un vector:

$$< a_1, \ldots, a_n, \mathit{efort} >$$

- Găsirea corelărilor dintre (unele) variabile din seturile de date:

$$f(a_1, \ldots, a_k) \propto \mathit{efort}$$

- Găsirea mijloacelor adecvate de măsurare pentru variabile la începutul proiectului (astfel încât putem aplica funcția unui nou proiect)

Metrici de productivitate

Există două clase principale de metrici folosite pentru a măsura efortul necesar pentru dezvoltarea unei aplicații:

- Metrici bazate pe funcții - măsoară numărul și complexitatea funcțiilor de dezvoltat
- Metrici bazate pe dimensiune - măsoară dimensiunea unei aplicații (în materie de număr de linii de cod, de exemplu)

Metrici bazate pe funcții

Avantaje

- Complexitatea funcțională poate fi estimată atunci când cerințele sunt finalizate
- Nu depind de limbajul de programare

Dezavantaje și critici

- Dificil de legat de productivitate
- Mai dificil de măsurat decât metricile bazate pe dimensiune

Metrici bazate pe dimensiune

Avantaje

- Pot fi măsurate cu precizie (o dată ce avem acces la codul sursă și am adoptat standardele de codare)
- Instrumente de măsurare disponibile
- Mai ușor de legat de productivitate

Dezavantaje și critici

- Sunt cunoscute cu exactitate atunci când sistemul este construit (… un pic prea târziu pentru a le putea folosi pentru estimare)
- Sensibile la limbajul de programare și la standardele de codare
- Nu măsoara complexitatea codului

Ce este o linie de cod?

```
System.out.println("Hello world");
```

```
if (a == 1) {
return true;
}
else {
return false;
}
```

```
return a == 1 ? true : false
```

```
if (a == 1) { return true; } else { return false; }
```

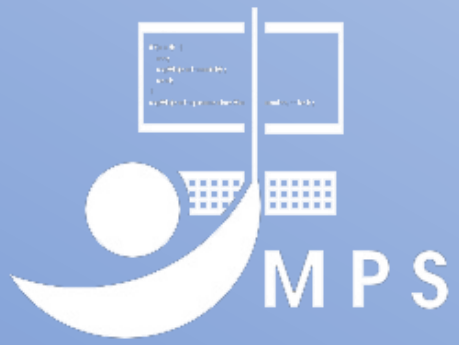
Exemple de productivitate

- Din Software Engineering, Ian Sommerville:
  - **Sisteme integrate in timp real**
    - 40-160 LOC/P-month (linii de cod / lună-persoană)
  - **Programe de sistem**
    - 150-400 LOC/P-month (linii de cod / lună-persoană)
  - **Aplicatii comerciale**
    - 200-9s00 LOC/P-month (linii de cod / lună-persoană)

Exemplu - https://en.wikipedia.org/wiki/Source_lines_of_code

- Red Hat Linux 7.1 contine aprox. 30 milioane SLOC (source lines of code), ar fi avut nevoie de 8000 de ani-persoană dacă ar fi fost dezvoltat conform practicilor de proprietate convenționale

Function Points



Function Points

- Bazate pe o combinație de caracteristici de program derivate din cerințe (metrici bazate pe funcții)
- Intâi propuse de Albrecht în anii ‘70
- Prima aplicabilitate în sistemele de business
- Evoluate pentru a include o gamă largă de sisteme și aplicații
- Baza de utilizatori largă și un IFPUG (International Function Point Users Group) pentru instruire în aplicarea tehnicii

Procesul pe scurt

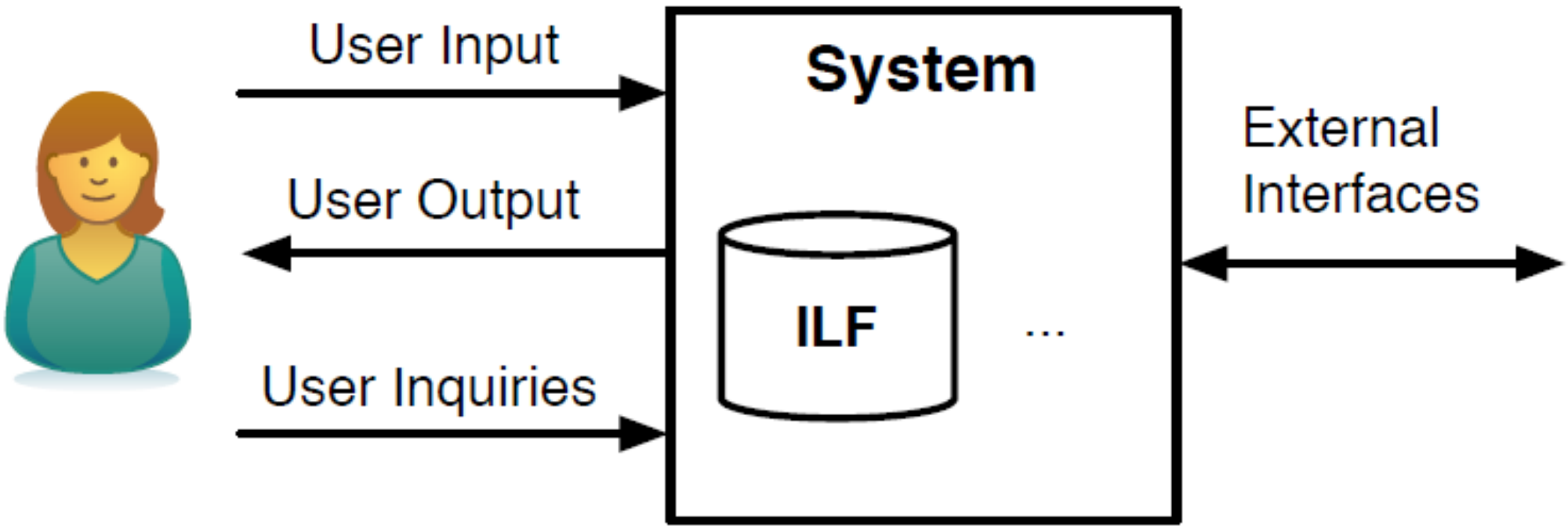
1. Definirea graniței analizei
2. Măsurarea unui set de caracteristici legate de cerințele funcționale ale sistemului și folosirea unei formule pentru determinarea Punctelor Funcționale Neajustate (Unadjusted Function Points – UFP)
3. Măsurarea unui set de caracteristici legate de cerințele non-funcționale: calcularea valorii factorului de ajustare (value adjustment factor – VAF)
4. Calcularea Punctelor Functionale (Ajustate) – FP:

   FP = f(VAF, UFP)

UFP - Intrări

- Intrările utilizatorului - numărul total de input-uri ale utilizatorului și complexitatea lor
- Ieșirile utilizatorului - numărul total de output-uri ale utilizatorilor și complexitatea lor
- Cererile utilizatorului - numărul total de intrări ale utilizatorului care generează un răspuns din partea software-ului, ca de exemplu numărul de cuvinte, rezultatele căutarii sau statusul software-ului
- Fișiere Logice Interne - numărul total de fișiere create și utilizate în mod dinamic de către sistem
- Interfețe externe - numărul total de fișiere externe care conecteaza software-ul cu un sistem extern. De exemplu, dacă software-ul și un dispozitiv comunică între ele, avem 1 interfață externă

Numărul total al fiecărei categorii este clasificat ca fiind ușor, mediu sau complex.


User Input
User Output
User Inquiries
System
ILF
...
External
Interfaces

VAF - Intrări

- Sistemul are nevoie de salvări și backup-uri de încredere?
- Sistemul are nevoie să comunice date?
- Elaborarea este distribuită?
- Performanța este un factor critic?
- Sistemul va funcționa într-un mediu bine cunoscut?
- Introducerea datelor este inclusă în tranzacții complexe?
- Datele sunt complexe?
- Elaborarea internă este complexă?
- Codul trebuie să fie reutilizabil?
- Proiectul include de asemenea instalarea și conversia datelor?
- Sistemul trebuie să fie utilizat în diferite organizații?
- Sistemul trebuie să fie ușor de menținut si de utilizat?

Raspunsuri de la 0 (irelevant) la 5 (extrem de relevant)

Formule pentru Function Points

$$UFP = \sum_{i=1}^{5} \begin{bmatrix} k_i^E & k_i^A & k_i^C \end{bmatrix} \cdot \begin{bmatrix} n_i^E \\ n_i^A \\ n_i^C \end{bmatrix}$$

$$FP = UFP \cdot (0.65 + 0.01 \cdot \sum_{i=1}^{14} C_i)$$

Comentarii despre formule

- **UFP** reprezintă suma ponderată a input-urilor externe, output-urilor externe, …
  - Valoarea constantelor (k^E, k^A, k^C) este dată de către metodă
- **FP (ajustate)** iau în considerare caracteristicile generale ale sistemului (cele 14 intrebări)
  - FP în intervalul **[0.65 UFP, 1.35 UFP]**

UFP
Caracteristici de program

- Numărarea UFP-urilor se face privind cerințele
- Analiștii se uită după urmatoarele elemente:
  - DET: tipuri de elemente de date (data element types)
  - FTR: tipul de fișier referit (file type referenced)
  - RET: tipul elementului înregistrat (record element type)
- Tabelele convertesc numerele de mai sus în 5 caracteristici
- Trainingul este necesar pentru aplicarea corectă a tehnicii

ILF/EIF		DET	
RET	1..19	20..50	> 50
0..1	Low	Low	Average
2..5	Low	Average	High
>5	Average	High	High

UI		DET	
FTR	1..4	5..15	> 15
0..1	Low	Low	Average
2	Low	Average	High
> 2	Average	High	High

UO/EQ		DET	
FTR	1..5	6..19	> 19
0..1	Low	Low	Average
2..3	Low	Average	High
>3	Average	High	High

FP: Caracteristici de program

- Joel Henry (“Introduction to SPM”) propune o abordare practică de numărare:
  - UI: numărul de casuțe de dialog, clasificat drept simplu, mediu și complex
  - UO: numărul de rapoarte, clasificat drept simplu, mediu și complex
  - UI: numărul total de intrări ale utilizatorului ce generează un răspuns din partea software-ului
  - ILF: numărul de fișiere și structuri de date interne complexe, clasificat drept simplu, mediu și complex
  - EIF: numărul de fișiere externe și dispozitive conectate, clasificat drept simplu, mediu și complex
- Regula de baza: realizează o estimare grosieră

Ce se întamplă o dată de avem FP-urile?

- O estimare FP măsoara printr-un număr dimensiunea sistemului ce trebuie construit
- Este posibil acum:
  - Să se lege FP direct de efort, prin măsuri de productivitate, dacă organizația păstrează acest gen de date:
    - FP pe persoană, pe lună
    - FP per lună calendaristică
  - Să se convertească FP-ul în SLOC și folosirea unei alte metode de estimare (e.g. COCOMO)
  - Să se utilizeze FP-ul pentru a verifica estimările furnizate de alte tipuri de estimare

Metrici de productivitate ale FP-urilor

- Fiecare companie ar trebui să pornească propriul program de măsurare a productivității
- Câteva date și studii “generale” sunt disponibile (e.g. http://www.softwaremetrics.com/Articles/estimatingdata.htm)
- Productivitate se schimbă cu dimensiunea sistemului (http://www.softwaremetrics.com/Articles/estimatingdata.htm):
  - 1.3 ore/FP pentru un sistem de 50 FP
  - 12.1 ore/FP pentru un sistem de 7000 FP
  - 133.6 ore/FP pentru un sistem de 15000 FP

Relația dintre FP și SLOC

- FP pot fi de asemenea legate de SLOC prin metrici FP/SLOC
- Aceasta permite:
  - Aplicarea altor tehnici algoritmice (e.g. COCOMO)
  - Măsurarea progresului folosind SLOC-uri

Language	Avg	Median	Low	High
ABAP (SAP)	18	18	16	20
Access *	36	38	15	47
Ada	154	-	104	205
Advantage	38	38	38	38
Assembler *	209	203	91	320
C *	148	107	22	704
C++ *	59	53	20	178
C# *	58	59	51	66
Clipper *	40	39	26	53
COBOL *	80	78	8	400
ColdFusion	68	56	52	105
Cool:Gen/IEF *	37	35	10	180
Culprit	51	-	-	-
Datastage	67	79	16	85
DBase IV	52	-	-	-
Easytrieve+	33	34	25	41
Excel	47	46	31	63
FORTRAN	90	118	35	-
J2EE *	57	50	50	67
Java *	55	53	9	214

Sursă: http://www.qsm.com/resources/function-point-languages-table

FP - Concluzii

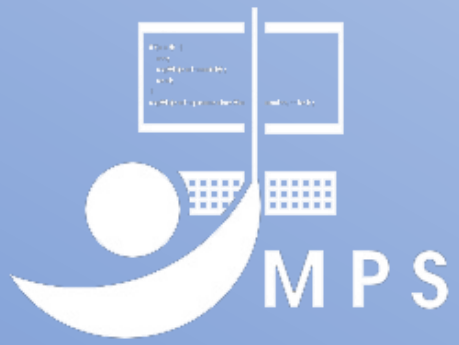
Dezavantaje

- necesită training specific pentru a fi aplicat corect
- poate introduce rigiditate în procesul de dezvoltare (mai ales în cazul în care este folosită această tehnică pentru stabilirea prețurilor, trebuie luată în considerare o schimbare a cerințelor)

Avantaje

- aduce o măsura **obiectivă** în complexitatea sistemului și poate fi folosită pentru punerea de acord în stabilirea prețurilor
- poate fi calculate după cerințe! (efortul de dezvoltare al unui sistem este mai dificil de estimat decât redactarea cerințelor)
- analiza, chiar și atunci când este derulată naiv, permite o mai bună înțelegere a sistemului de construit

COCOMO – Constructive Cost Modeling



Modelul COCOMO

- O familie de modele empirice bazate pe analiza proiectelor diferitelor companii
- Lungă istorie de la COCOMO-81 (1981) până la COCOMO-II (1999, 2000)
- Extins pentru a acoperi diferite procese de dezvoltare și alte aspecte, precum calitatea (COQUALMO)

Modelul COCOMO

- COCOMO este bazat pe o măsură fizică (linii de cod sursă)
- Estimarile devin mai precise atunci când trecem la faza de dezvoltare
- Erori de estimare:
  - Estimările inițiale pot fi eronate cu un factor de 4x
  - Atunci când avansăm în procesul de dezvoltare, estimările devin mai precise (și modelul ia în calcul parametri mai detaliați)

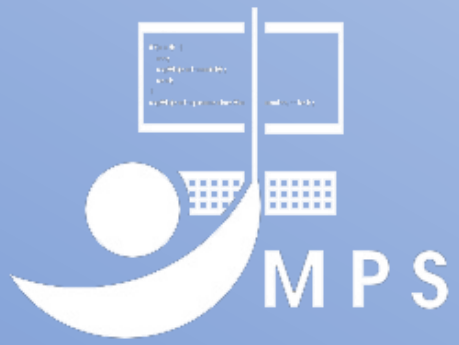
COCOMO – Structura Generala

- Toate modelele COCOMO au aceeași structură de bază

$$\mathrm{OUTPUT} = A \cdot (\mathrm{size})^B \cdot M$$

- OUTPUT-ul poate fi efort sau timp
- Măsură fundamentală este dimensiunea codului (exprimată în linii de cod sursă)
- Dimensiunea codului are un efect exponențial asupra efortului și dimensiunii (dar foarte aproape de 1)
- Diferiți factori de ajustare sunt folosiți pentru a face modelul mai precis

COCOMO 81



COCOMO 81 - Introducere

- Combinație de 3 modele cu diferite niveluri de detaliu și complexitate:
  - **de bază:** estimare rapidă, stadiu de dezvoltare incipient
  - **intermediar:** mai precis, necesită mai multe caracteristici ale produsului, stadiu de dezvoltare mai avansat
  - **avansat:** cel mai detaliat, necesită mai multă informație
- În toate modelele COCOMO:
  - 1 persoană-lună = 152 ore de lucru
  - SLOC este DSI (delivered source instructions)

  (doar codul este livrat către client. ex. testarea unitară, utilitățile nu sunt luate în considerare)

COCOMO 81 - Tipuri de Proiecte

COCOMO 81 face distincție între 3 tipuri de proiecte:

- **organic**: echipe mici, mediu familiar, aplicații bine înțelese, cerințe non-funcționale simple → ușor
- **semi-detașat**: echipa de proiect poate avea experiență mixtă, sistemul poate avea constrângeri non-funcționale semnificative, organizația poate avea mai puțină familiaritate cu aplicația → mai dificil
- **incorporat (embedded)**: constrângeri puternice, incluzând regularizări locale și proceduri operaționale; neobișnuit pentru echipe să aibă o experiență puternică asupra aplicației → hard

COCOMO 81 - Model de Bază

$$PM = A_{PM} \cdot (KSLOC)^{B_{PM}}$$

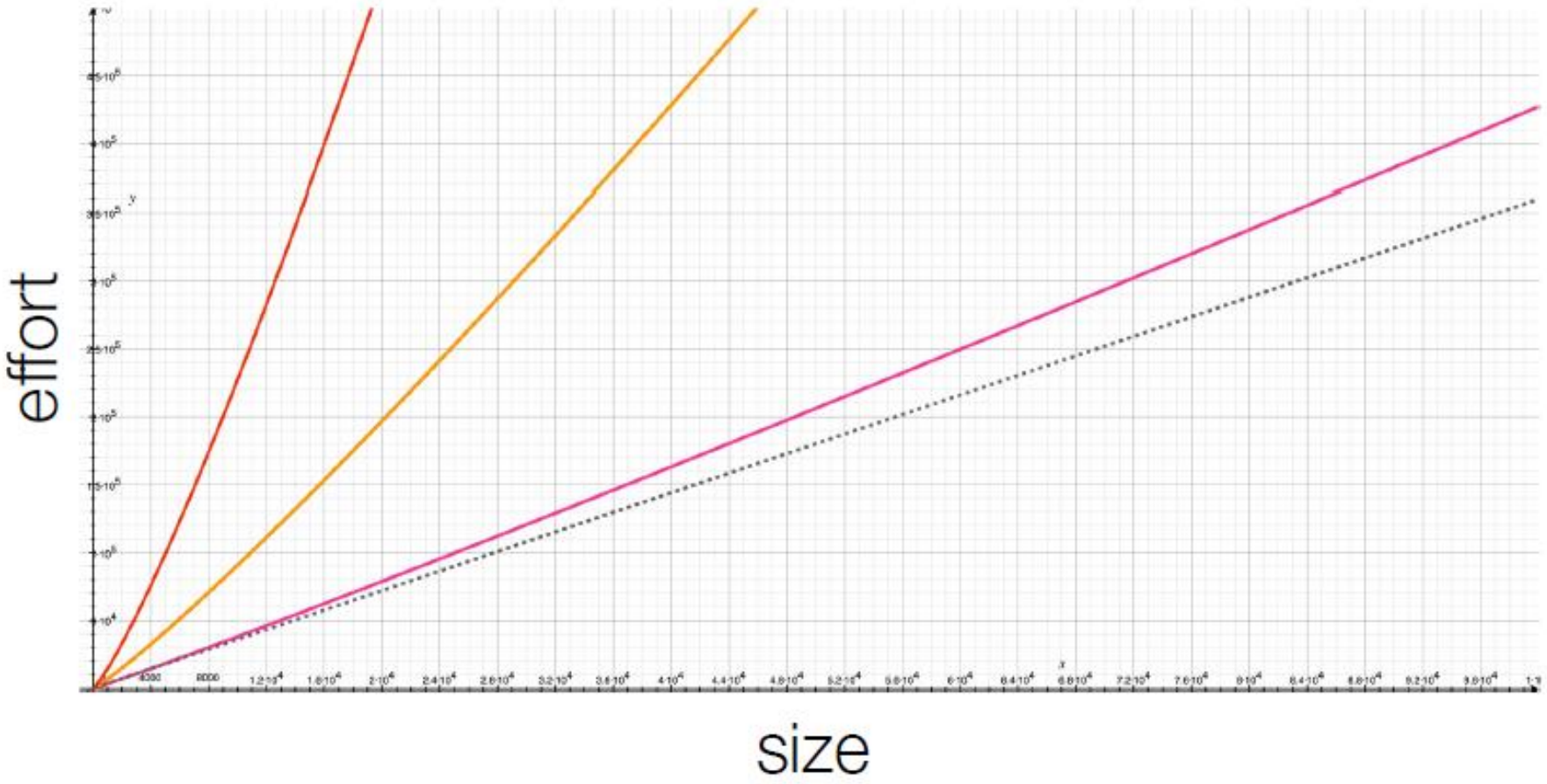
$$TDEV = A_{TDEV}(PM)^{B_{TDEV}}$$

unde:

- KSLOC: mii de linii de cod sursă livrate
- M este egal cu 1 (astfel nu apare în formule)

	A	B	A	B
Organic	2.4	1.05	2.5	0.38
Semi-detached	3	1.12	2.5	0.35
Embedded	3.6	1.2	2.5	0.32

Efectul Exponential COCOMO vs. liniar



Exemplu de aplicație

- Estimarea de 50 KDSI pentru un proiect organic
  - PM = 2.4 (50)^1.05 ~= 146 mm
  - TDEV = 2.5 (371.54)^0.38 ~= 16 luni
  - Echipă = 371.54 / 23.69 ~= 9 persoane
- Efectul diferiților parametri ai proiectului:

	A	B	A	B	PM	TDEV	Team
Organic	2.4	1.05	2.5	0.38	146	16.6	8.8
Semi-detached	3	1.12	2.5	0.35	240	17.0	14.1
Embedded	3.6	1.2	2.5	0.32	394	16.9	23.3

COCOMO - Intermediar

- Folosește o caracterizare mai fină, incluzând **atribute** (multiplicatori de efort) pentru a lua în calcul:
  - cerințe funcționale și non-funcționale
  - atribute ale proiectului
- **Multiplicatorii de efort** sunt organizați în 4 clase și 15 sub-elemente
- Importanța fiecărui atribut este **evaluată calitativ** între 1 (foarte scăzută) și 6 (foarte ridicată)
- Fiecare valoare corespunde unui multiplicator, în intervalul [0.7, 1.66] (multiplicator < 1 implică reducerea costului)
- Toate valorile **sunt multiplicate împreună** pentru a ajusta efortul

COCOMO 81 - Model Intermediar

$$PM_{nominal} = A_{PM} \cdot (KSLOC)^{B_{PM}}$$

$$PM = PM_{nominal} \cdot \Pi_{i=i}^{15} EM_i$$

$$TDEV = A_{TDEV}(PM)^{B_{TDEV}}$$

	A	B	A	B
Organic	3.2	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	2.8	1.2	2.5	0.32

Model Intermediar - Parametri

Effort Adjustment Factors		*Very_Low*	*Low*	***Nominal***	*High*	*Very_High*	*Extr_High*
Product Attributes							
Required Software Reliability	RELY	0.75	0.88	**1.00**	1.15	1.40	
Database Size	DATA		0.94	**1.00**	1.08	1.16	
Product Complexity	CPLX	0.70	0.85	**1.00**	1.15	1.30	1.65
Computer Attributes							
Execution Time Constraints	TIME			**1.00**	1.11	1.30	1.66
Main Storage Constraints	STOR			**1.00**	1.06	1.21	1.56
Virtual Machine Volatility	VIRT		0.87	**1.00**	1.15	1.30	
Computer Turnaround Time	TURN		0.87	**1.00**	1.07	1.15	
Personnel Attributes							
Analyst Capability	ACAP	1.46	1.19	**1.00**	0.86	0.71	
Applications Experience	AEXP	1.29	1.13	**1.00**	0.91	0.82	
Programmer Capability	PCAP	1.42	1.17	**1.00**	0.86	0.70	
Virtual Machine Experience	VEXP	1.21	1.10	**1.00**	0.90		
Programming Language Experience	LEXP	1.14	1.07	**1.00**	0.95		
Project Attributes							
Use of Modern Programming Practices	MODP	1.24	1.10	**1.00**	0.91	0.82	
Use of Software Tools	TOOL	1.24	1.10	**1.00**	0.91	0.83	
Required Development Schedule	SCED	1.23	1.08	**1.00**	1.04	1.10	

Atribute

- Atribute:
  - PRODUCT = RELY * DATA * CPLX
  - COMPUTER = TIME * STOR * VIRT * TURN
  - PERSONNEL = ACAP * AEXP * PCAP * VEXP * LEXP
  - PROJECT = MODP * TOOL * SCED
- Impactul parametrilor este între [0.09, 73.28]
- Managerul de proiect (sau echipa) estimează valorile parametrilor pentru a prezice efortul
- Exemplu:
  - Dacă “fiabilitatea necesară a software-ului” este scăzută, efortul prezis este 0.88 din cel calculat cu formula de bază

COCOMO 81 - Modelul Avansat detaliat

- Modelul detaliat
  - are multiplicatori mai detaliați pentru fiecare fază de dezvoltare
  - organizează parametrii în mod ierarhic, pentru simplificarea calculului în cazul sistemelor alcătuite din mai multe module
- Proiectele sunt organizate în 4 faze:
  - Planificarea Cerințelor și Design de Produs (RPD)
  - Design Detaliat (DD)
  - Cod și Testare Unitară (CUT)
  - Teste de Integrare (IT)
- Estimările sunt realizate în fiecare fază
- Datele astfel obținute sunt agregate pentru a da estimarea totală

COCOMO 81 - Modelul Avansat detaliat

Exemplu de parametru:

Cost Driver	Rating	RPD	DD	CUT	IT
ACAP	Very Low	1.8	1.35	1.35	1.5
	Low	0.85	0.85	0.85	1.2
	Nominal	1	1	1	1
	High	0.75	0.9	0.9	0.85
	Very High	0.55	0.75	0.75	0.7

COCOMO - Faza de mentenanță

- Modelul COCOMO poate fi de asemenea aplicat pentru a prezice efortul în timpul mentenanței sistemului.

mentenanța sistemului → mici *actualizări* și *reparații în* timpul vieții operaționale ale unui sistem

- Mare parte dintre parametrii de dezvoltare se aplică atât la dezvoltare cât și la mentenanță (câteva excepții: SCED, RELY, MODP)
- Un input essential este reprezentat de estimarea ACT-ului

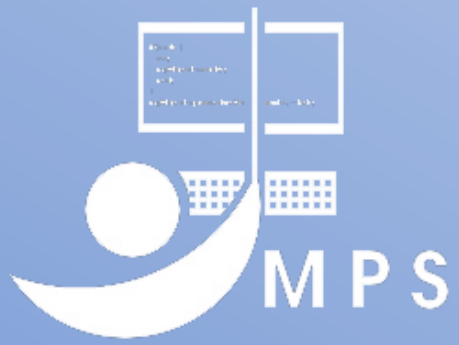
ACT - annual change traffic – porțiunea de cod sursă ce se schimbă în decursul unui an

COCOMO 81 – Mentenanță

$$ACT = \frac{\%Added + \%Modified}{100}$$

$$PM = ACT \cdot PM_{nom} \cdot EAF_{maint}$$

COCOMO II



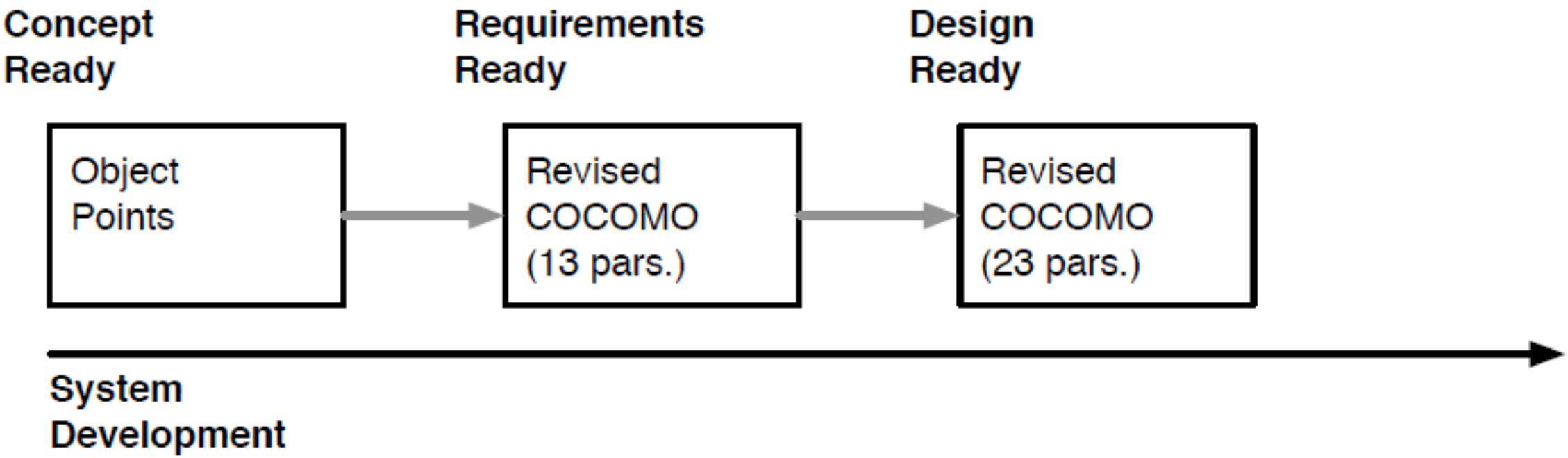
COCOMO II

- COCOMO este bazat pe COCOMO 81 și ia în considerare:
  - Noi procese de dezvoltare (e.g. spirală)
  - Flexibilitate crescută în dezvoltarea software (e.g. reutilizarea codului, generarea automată de cod)
  - Nevoia de a lua decizii în condiții de informații incomplete
  - Date noi cu privire la proiecte (nu neapărat o nevoie, cât o oportunitate (161 proiecte vs. 61)

COCOMO II - Modele

- COCOMO II include o serie de submodele ce produc estimări din ce în ce mai detaliate ale software-ului
- Submodelele COCOMO II sunt:
  - **Application Composition Model**: pentru prototipuri
  - **Early Design Model**: folosit atunci când cerințele sunt disponibile dar design-ul nu a început încă
  - **Post-architecture Model**: folosit o dată ce arhitectura sistemului a fost creată și avem la dispoziție mai multe informații cu privire la sistem
- Mai mult:
  - **Reutilizarea modelului**: folosit pentru a calcula efortul necesar pentru integrarea componentelor reutilizabile

COCOMO II - Stadiile modelelor



COCOMO II - Modelele ED și PA

$$PM_{NS} = 2.94 \cdot (SIZE)^{E} \cdot \Pi_{i=1}^{n} EM_i$$

$$E = 0.91 + 0.01 * \sum_{j=1}^{5} \mathrm{SF}_i$$

$$TDEV_{NS} = 3.67 * (PM_{NS})^{F}$$

$$F = 0.28 + 0.01 * \sum_{j=1}^{5} \mathrm{SF}_i$$

- Toate constantele pot (și trebuie) să fie ajustate ce valori dependente de organizație.

  Recomandat: 2.94 multiplicator de efort și 3.67 multiplicator de planificare

- Diferența între ED și PA este reprezentată de numărul de parametri.

- Exponentul depinde de factorii de ajustare. (față de COCOMO 81 în care era o simplă constantă)

COCOMO II - Multiplicatori de efort

Între 7 (Early Design) și 17 (Post Architecture) în funcție de nivelul de detalii necesar

Exemplu:

	Early Design cost drivers	Post-Architecture cost drivers (Counterpart combined)
Product reliability and complexity	RCPX	RELY, DATA, CPLX, DOCU
Required reuse	RUSE	RUSE
Platform difficulty	PDIF	TIME, STOR, PVOL
Personnel capability	PERS	ACAP, PCAP, PCON
Personnel experience	PREX	AEXP, PEXP, LTEX
Facilities	FCIL	TOOL, SITE
Required Development Schedule	SCED	SCED

Sursă: http://www.ifi.uzh.ch/req/courses/seminar_ws02/reports/Seminar_4.pdf

COCOMO II - Factori de scalare

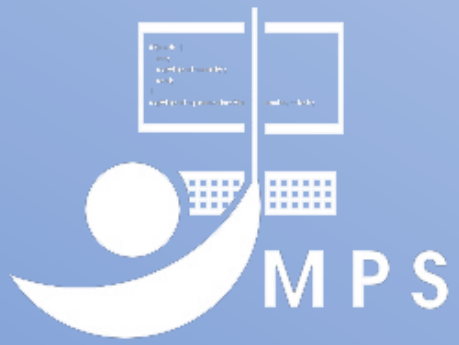
Exponentul este calculat prin furnizarea de răspunsuri calitative la următorii factori:

- **Precedentă**: cat de inedit este proiectul pentru organizatie
- **Flexibilitate**: flexibilitatea dezvoltarii (rigiditatea conformitatii cu cerintele)
- **Coeziunea echipei**
- **Maturitatea Procesului**: maturitatea conform chestionarului CMMI

Scale Factors (W_i)	Very Low	Low	Nominal	High	Very High	Extra High
PREC	thoroughly unprecedented	largely unprecedented	somewhat unprecedented	generally familiar	largely familiar	throughly familiar
FLEX	rigorous	occasional relaxation	some relaxation	general conformity	some conformity	general goals
RESL	little (20%)	some (40%)	often (60%)	generally (75%)	mostly (90%)	full (100%)
TEAM	very difficult interactions	some difficult interactions	basically cooperative interactions	largely cooperative	highly cooperative	seamless interactions
PMAT	Weighted average of "Yes" answers to CMM Maturity Questionnaire					

Sursă: http://www.ifi.uzh.ch/req/courses/seminar_ws02/reports/Seminar_4.pdf

Tehnici Algoritmice Concluzii



COCOMO – Considerație

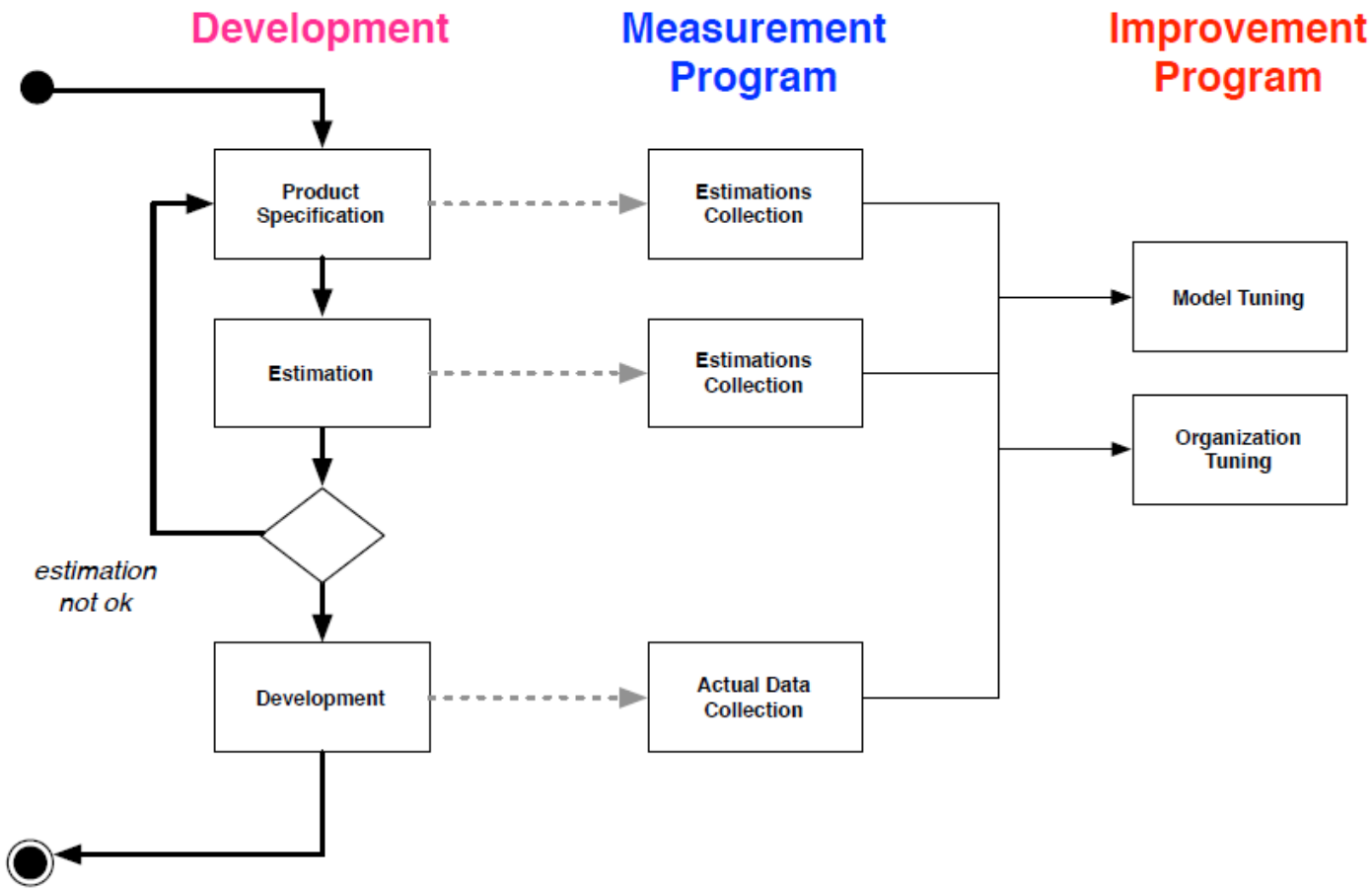
- O serie de modele din ce în ce mai complexe
- COCOMO calculează ambele D și E, iar forța de munca este derivata din D și E (adesea estimăm E, decidem M și calculam doar D)
- Estimările costurilor proiectului pot fi auto-îndeplinite: estimarea definește bugetul, iar produsul este ajustat pentru a respecta bugetul
- O aplicație precisă necesită ca organizațiile să își seteze propriile programe de măsurare (pentru rafinarea parametrilor)
- Modelele trebuie să fie adaptate la tehnologiile schimbatoare, ținând cont ca tehnologia se schimbă rapid… Poate fi dificil să le păstrăm actualizate la zi.

Tehnici Algoritmice - Recapitulare

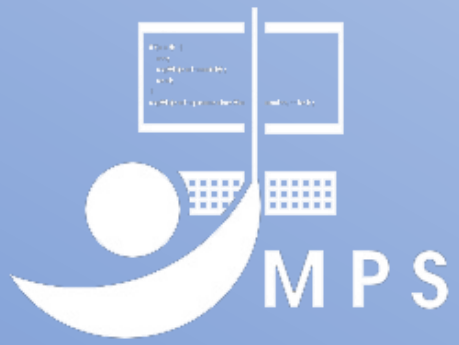
- Bazate pe caracteristicile sistemului și metricile de productivitate colectate în proiectele trecute
- Modele diferite:
  - Function Points:
    Req ☞ UFP ☞ FP, MM/FP

    Req ☞ UFP ☞ SLOC/UFP and COCOMO
  - Object Points
    Screens, Reports, Modules ☞ NOP, NOP/Month
  - COCOMO
    SLOC ☞ PDEV, TDEV ☞ Team Size = PDEV / TDEV

Fabrica de îmbunătățiri



Object Points



Limbaje de generatia a 4-a

- Un set de limbaje de nivel înalt (fiecare “generație” se îndepărteaza de codul mașină)
- Ele pot fi distinse între:
  - Tool-uri de gestionarea datelor: ele presupun utilizarea de funcții sofisticate pentru manipularea datelor (ex: SAS, SPSS, Mathematica, Matlab, ABAP)
  - Limbaje de generare de rapoarte și formulare, ce specifică formatul datelor și output-ul ce va fi generat (ex: SQL, Oracle Forms)
  - Generatoare automate de cod din tool-uri de tip CASE - Computer Aided Software Engineering (ex: Rails)

Object Points

- Metrici pe baza de funcții pentru limbaje de generatia a 4-a
- Calcule mai simple față de metoda Function Points

Referințe

Kauffman, Wright, Zweig, Automating Output Size and Reuse Metrics in a Repository-Based Computer Aided Software Engineering (CASE) Environment IEEE Trans. Software Engineering, 20(3), p. 169-186, 1994

Calculul în metoda Object Points

- Similar cu Function Points
- Calculează numărul de ecrane și le clasifică în 3 categorii → simplu, mediu, complex
- Calculează numărul de rapoarte și le clasifică în 3 categorii → simplu, mediu, complex
- Determină numărul de module ce trebuie dezvoltate
- Folosește matrice de ponderi pentru a însuma valorile de mai sus, luând în considerare codul reutilizat
- O formula transformă OP-urile în măsuri de productivitate

Clasificarea ecranelor și a rapoartelor

- Niveluri de complexitate ale OP pentru ecrane

	Number and sources of data tables		
Number of Views Contained	Total < 4	Total < 8	Total 8+
<3	simple	simple	medium
3-7	simple	medium	difficult
8+	medium	difficult	difficult

- Niveluri de complexitate ale OP pentru rapoarte

	Number and source of data tables		
Number of Sections Contained	Total < 4	Total < 8	Total 8+
0-1	simple	simple	medium
2-3	simple	medium	difficult
4+	medium	difficult	difficult

Sursa: http://yunus.hacettepe.edu.tr/~sencer/objectp.html

Metoda de Calcul pentru Object Points

	Weight		
Type	Simple	Medium	Difficult
Screen	1	2	3
Report	2	5	7
Modules	10	10	10

$$NOP = \sum_{i=1}^{3} \begin{bmatrix} k_i^s & k_i^m & k_i^d \end{bmatrix} \cdot \begin{bmatrix} n_i^s \\ n_i^m \\ n_i^d \end{bmatrix} \cdot \frac{(100 - r)}{100}$$

$$EFFORT = \frac{NOP}{PROD}$$

Unde:
r = procentul de componente reutilizate
PROD→ productivitatea

Estimări ale productivității

- Productivitatea **între 4 și 50 object points / lună,** depinzând de mentenanța tool-ului și capabilitatea dezvoltatorului:

Developer Experience	Very Low	Low	Nominal	High	Very High
PROD	4	7	13	25	50

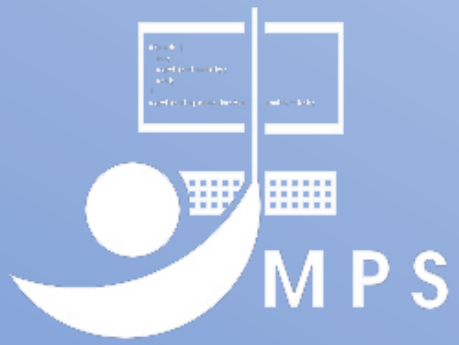
- Alte surse indică până la **75 OP/luna**

Sursă: http://yunus.hacettepe.edu.tr/~sencer/objectp.html

Observații

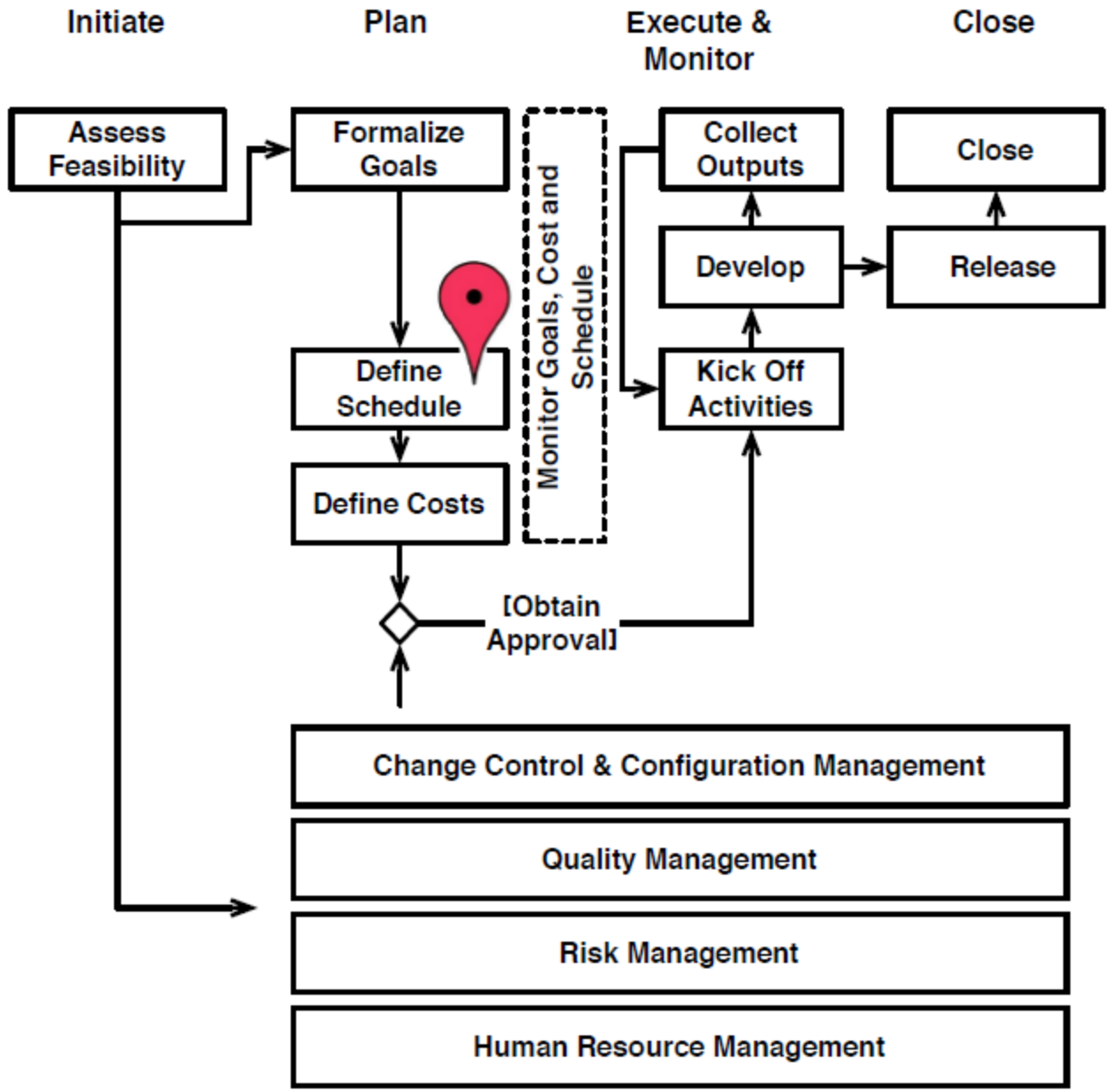
- Simplu de estimat → să ne amintim de abordarea lui Joel Henry pentru calculul de Function Points
- OP ia în calcul și codul reutilizat → fapt apărut și în unele elaborări ale modelului COCOMO

Planificarea proiectului



Obiective

- Transformarea WBS-ului într-o planificare
- Înțelegerea dependențelor dintre activități
- Învățarea tehnicii Drumului Critic (Critical Path)
- Învățarea uniformizării resurselor


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Prezentare generală

- Avem:
  - Un WBS (activități)
  - Estimarile de efort (durată) pentru fiecare element din WBS
- Vrem să planificăm activități astfel încât să putem cunoaște când începe și când se termină fiecare activitate, când avem nevoie de resurse, când livram
- Proces:
  - Identificarea constrângerilor (dependențelor)
  - Alocarea și uniformizarea resurselor
  - Găsirea drumului critic și iterarea până cand planul este satisfăcător
- Ieșire: diagrama Gantt

Diagrama Gantt modernă

- Outline textual + graf al calendarului
- Activitățile drept bare (posibil adnotate cu nume și resurse)
- Livrabile (diamante)
- Activitățile pot fi grupate (informația grupului este derivată din activități de nivel mai jos)
- Dependențe între sarcini

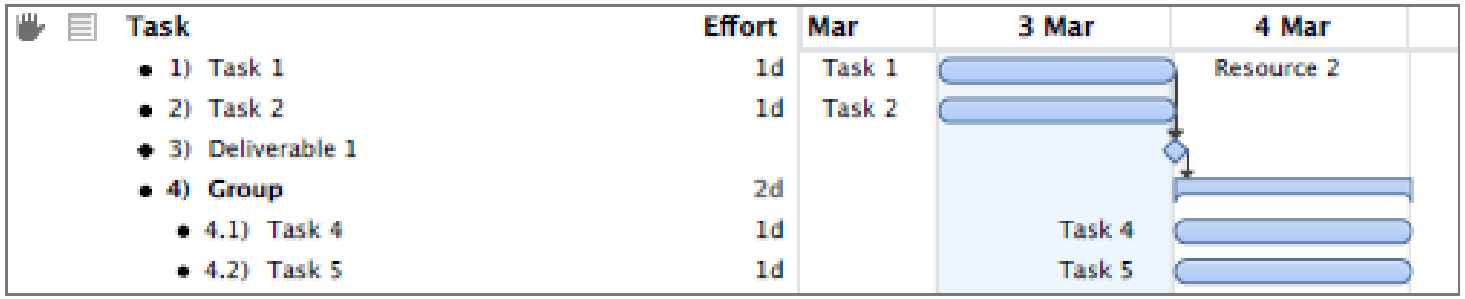
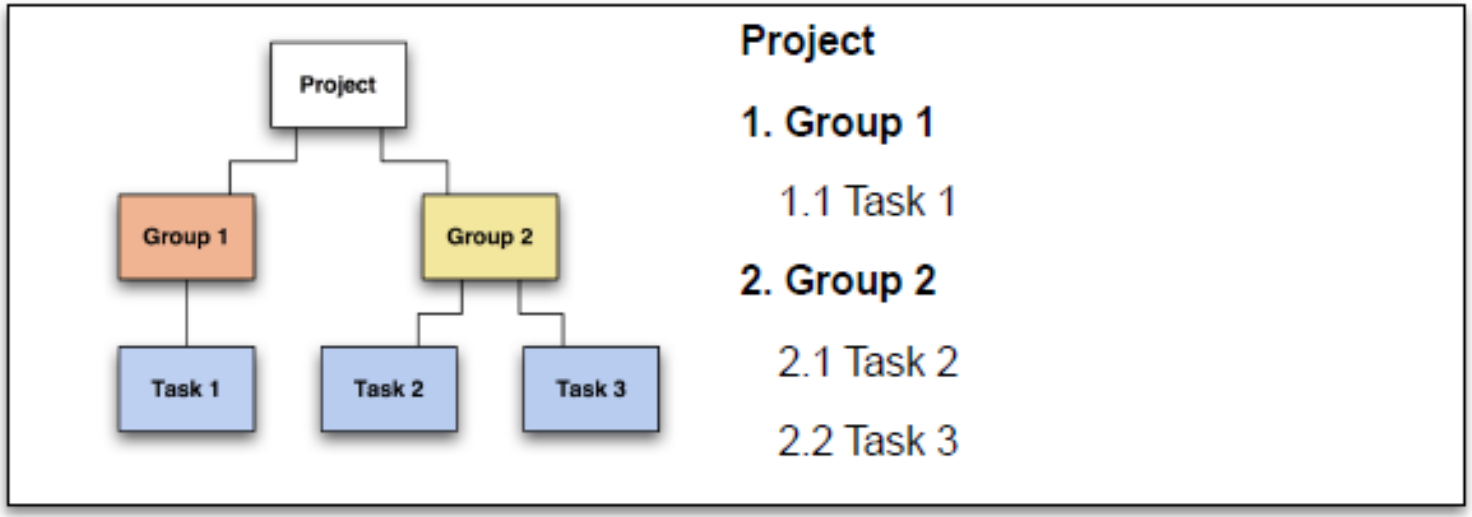
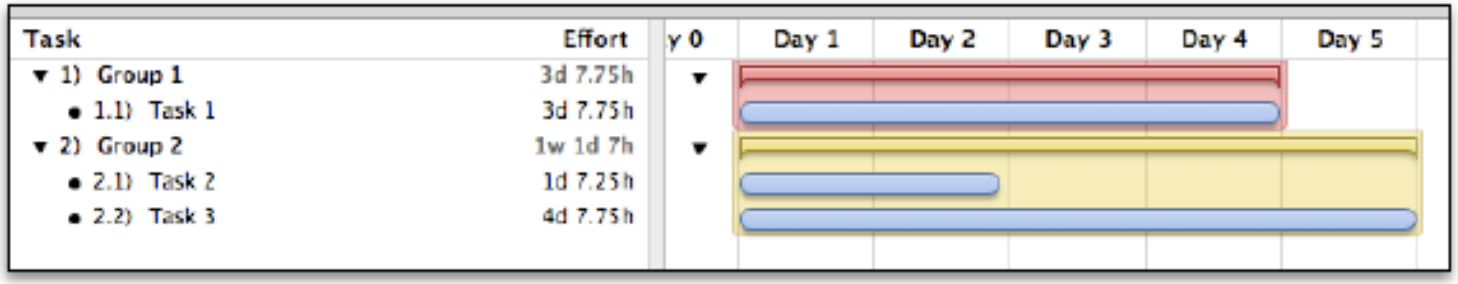


Diagrama Gantt modernă și WBS





Identificarea constrângerilor
/dependențelor

Identificarea dependențelor

- Executarea activităților este constrânsă de logica planului → nu construim acoperișul înainte ca fundația și structura casei să fie gata
- Dependente puternice (hard) și slabe (soft)
- Când folosim tool-uri de planificare:
  - Se specifică doar dependențele puternice
  - Cele slabe sunt de obicei inserate de tool-ul de planificare

Dependențele puternice

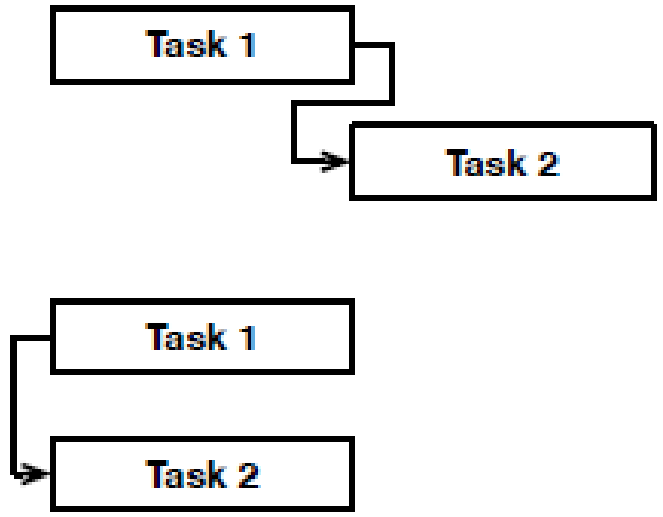
- Nu se poate face mare lucru în ceea ce le privește
- Pot deriva:
  - Fie din logica proiectului (ex. testarea trebuie să vină după scrierea codului)
  - Fie din dependențe externe (ex. semnarea unui contract; o aliniere particulară a planetelor este necesară pentru lansarea unei nave spațiale)
- Eliminarea dependențelor puternice poate fi realizată cu un anumit cost (ex. creșterea riscului, refacerea muncii)

Dependențele slabe

- Apar din cauza unei anumite alegeri făcute dintre toate planurile alternative posibile
- Pot deriva:
  - Fie din alegeri arbitrare (ex. managerul de proiect alege ordinea în care modulele vor fi dezvoltate)
  - Fie din disponibilitatea resurselor și uniformizare (ex. managerul de proiect sau tool-ul de planificare pun în secvență două sarcini ce se bazează pe aceeași resursă)
- De remarcat că odată cu trecerea timpului, poate deveni dificil sau chiar imposibil să anulăm dependențe slabe (ex. o resursă împărțită între proiecte diferite)

Relații de dependență între task-uri

- Finish-to-Start (FS)
  - B nu poate începe înainte ca A să se termine
  - Cea mai des utilizată
- Start-to-Start (SS)
  - B nu poate începe înainte ca A să înceapă
  - Realizarea unui experiment; monitorizarea unui experiment
- Finish-to-Finish (FF)
  - B nu se poate termina înainte ca A să se termine
- Start-to-Finish (SF)
  - B nu se poate termina înainte ca A să înceapa (rar)



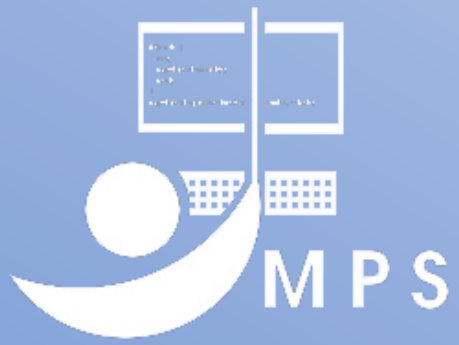
Timpi de lead și lag

- Dependențele dintre activități pot avea durată zero
- **Timp de lag** → întârzierea introdusă de dependența pozitivă (trece ceva timp între cele două sarcini)
- **Timp de lead** → durata dependenței este negativă (activitățile se suprapun parțial)

Reguli de bază

- Folosirea de milestones (și livrabile) pentru a marca în mod clar tranzițiile de fază (sau unele tranziții importante de la o activitate la alta)
- Incercarea de minimizare a dependențelor dintre task-uri (pentru a minimiza întârzierile cauzate de unele activități care le asteaptă pe altele să se încheie)
- Evaluarea alternativelor
- Unele activități ar putea depinde doar de calendar (și să fie constrânse de date)
- Luarea în considerare a tuturor dimensiunilor (cost, calitate, timp): reducerea timpului ar putea crește costurile, riscurile și compromite calitatea

Metoda drumului critic



Drumul critic

- Nu toate activitățile sunt la fel de importante sau critice într-un plan
- Metoda drumului critic alege acele activități care determină durata planului
- Aceste activități determină drumul critic
- Orice mică întârziere arbitrară într-o activitate din drumul critic poate întârzia data de finalizare a proiectului
- Calculul este bazat pe diagrame de rețea (reprezentare pe bază de graf a planului)

Diagrame de rețea

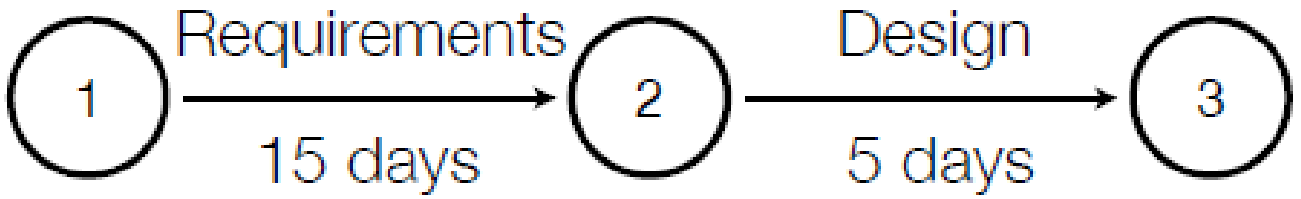
- Aparute în anii ‘50
- O reprezentare grafică a task-urilor este necesară pentru completarea proiectului (planul sub forma de graf)
- Vizualizarea fluxului sarcinilor și a relațiilor
- Două formate clasice:
  - AOA: Activity on Arc (sau Activity on Arrow)
  - AON: Activity on Node
- Convenții:
  - Fiecare task este etichetat cu un identificator și o durată (în unități standard, de ex zile)
  - Există un eveniment de început și unul de sfârșit
  - Timpul se deruleaza de la stânga la dreapta

Diagrame de rețea

- AOA (Activity on Arrow) sau ADM (Activity Diagramming Method):
  - Cercurile reprezintă Evenimente (ex. “start” sau “sfârșit” pentru un anumit task)
  - Liniile reprezinta Task-urile, ca de exemplu “Design”
- AON (Activity on Node) sau PDM (Precedence Diagramming Method):
  - Task-urile sunt în Noduri
  - Arcele reprezintă dependențele dintre task-uri

Formate Grafice

AOA (Activity on Arrow)



AON (Activity on Node)

…care este mai bun?

Comparații AOA/AON

- AOA folosit initial de Walker și Kelly pentru PERT
- AON mai flexibil și mai ușor de reprezentat
- AOA mai simplu de folosit pentru anumiți algoritmi

… ne vom rezuma in general la AON

Exemplu Comparație AOA/AON

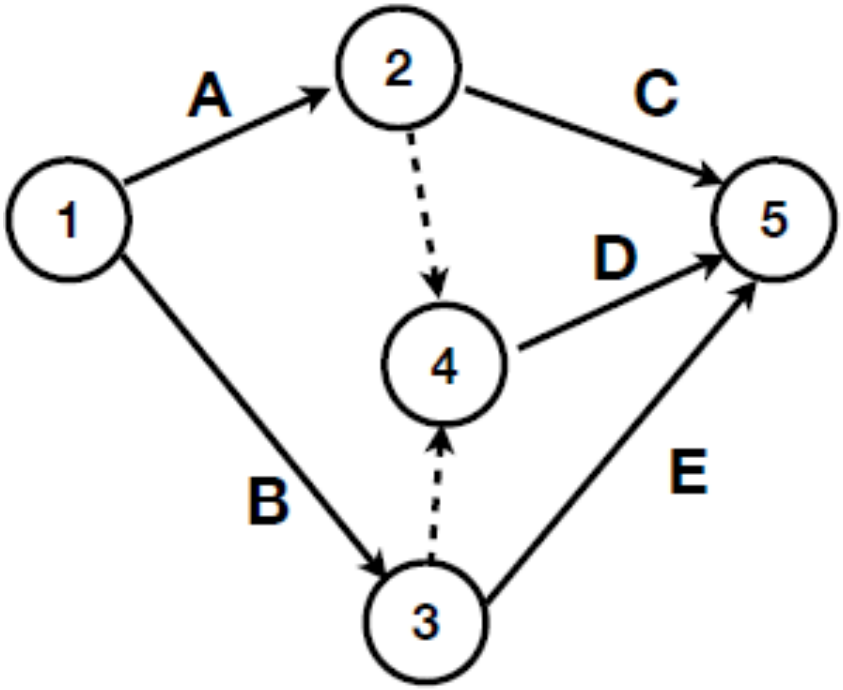
Să considerăm următorul plan:

Activity	Predecessors	Duration
A	None	3 months
B	None	4 months
C	A	3 months
D	A, B	1 month
E	B	2 months

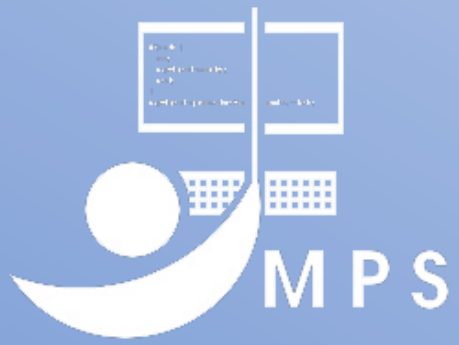
Exemplu Comparație AOA/AON

În notația AOA, unele dependențe pot necesita introducerea de arce sau noduri suplimentare (“dummy”) (*)

(*) De notat că un plan nu are un AOA unic asociat, din moment ce adăugam arce / noduri.

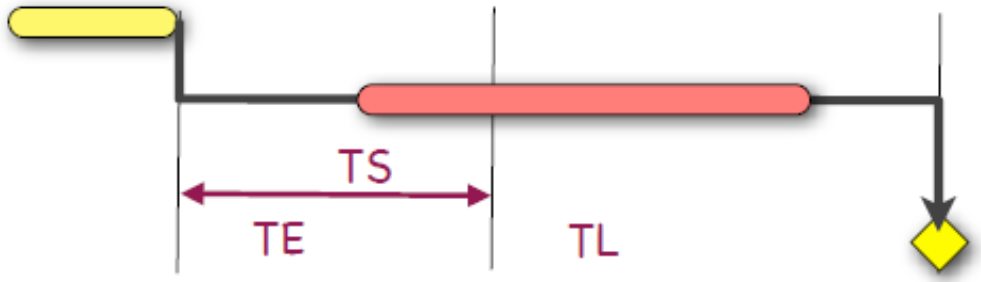


Calculul drumului critic



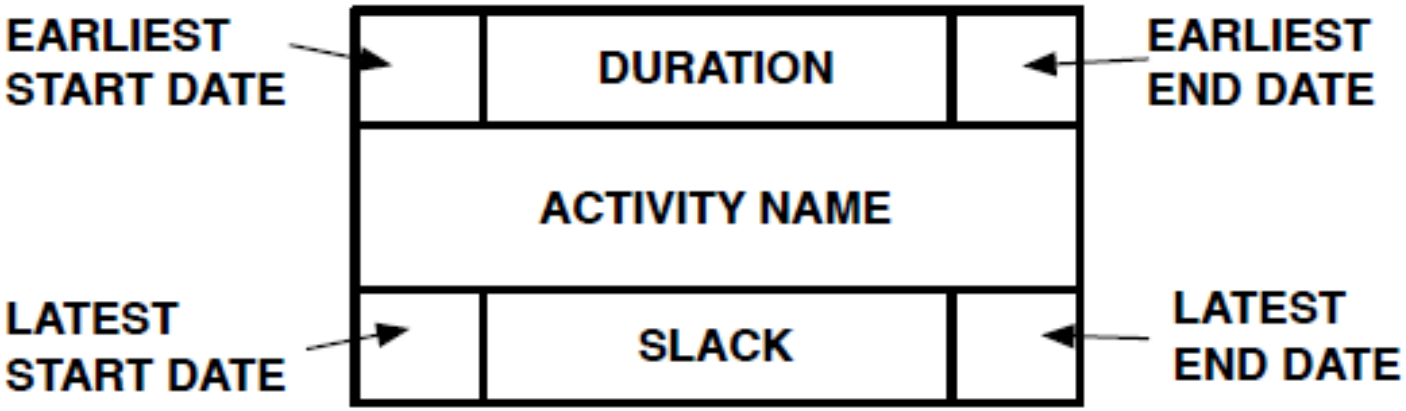
Stagnarea - Slack / Float

- Stagnare liberă
  - Stagnarea pe care o poate avea o activitate până să amâne urmatorul task
- Stagnare totală
  - Stagnarea pe care o poate avea o activitate până să întârzie între proiectul
- Timpul de Stagnare TS = TL – TE
  - TE = momentul de timp cel mai rapid în care poate avea loc un eveniment
  - TL = momentul de timp cel mai târziu în care se poate produce fără a extinde data de completare a proiectului sau a următoarei activități



Calcularea drumului critic

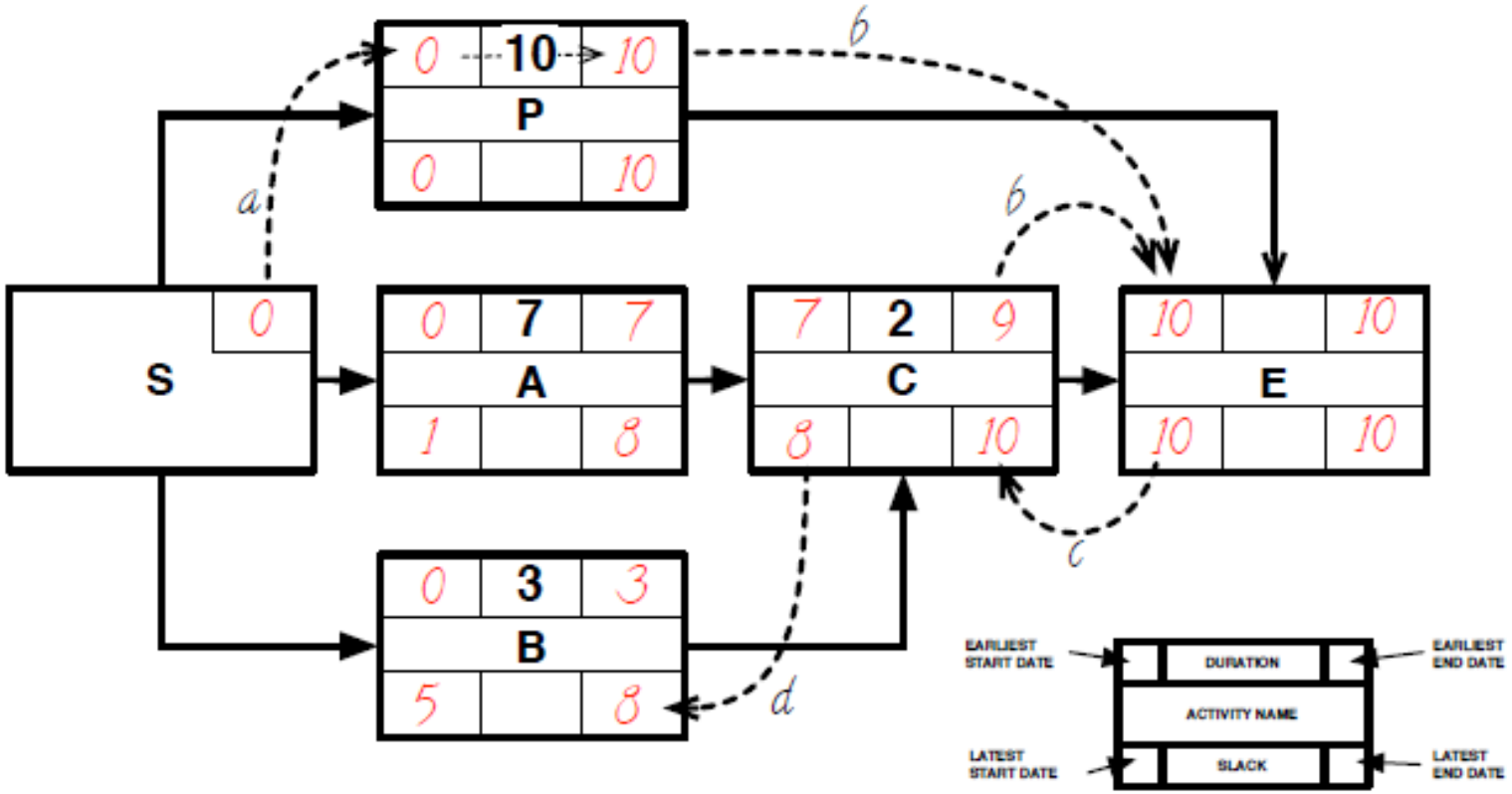
- Scop având un anumit plan (activități, durata, dependențe), să se determine stagnarea, data cea mai rapidă și cea mai târzie a fiecărei activități
- Notație → AON cu noduri reprezentate cum urmează:



Calcularea drumului critic

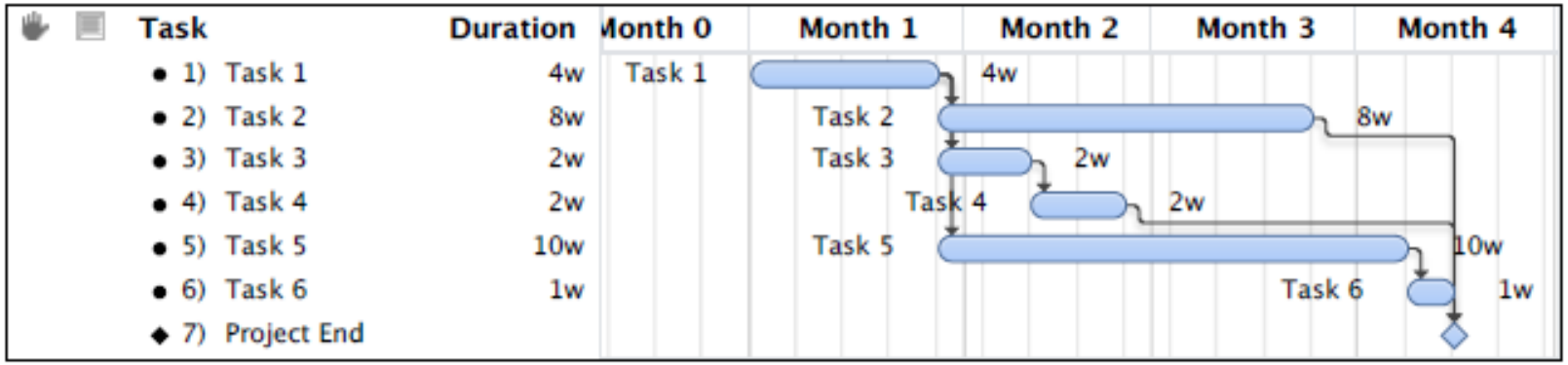
- O **trecere înainte** determină datele cele mai rapide de început și de sfârșit pentru fiecare activitate din plan
- O **trecere înapoi** determină datele cele mai târzii de început și de sfârșit pentru fiecare activitate din plan
- Diferența dintre cel mai rapid început (sfârșit) și cel mai târziu început (sfârșit) reprezintă stagnarea activității
- **Drumul critic** reprezintă drumul în care toate activitățile au stagnare zero
- Un plan are întotdeauna un drum critic → schimbarea planului schimbă activitățile care se găsesc în drumul critic

Exemplul 1


0
10
10
P
0
10
a
b
b
0
S
0
7
7
A
1
8
7
2
9
C
8
10
10
10
10
E
10
10
c
0
3
3
B
5
8
d
EARLIEST START DATE
DURATION
EARLIEST END DATE
ACTIVITY NAME
LATEST START DATE
SLACK
LATEST END DATE

Exemplul 2

- Abordare informală → descoperirea activităților care se pot deplasa într-un plan fără a schimba data de sfârsit a proiectului (ex. Task 3 nu este în drumul critic)
- Metoda drumului critic este subliniata automat de multe tool-uri de creare a diagramelor Gantt



Remarci privind metoda drumului critic

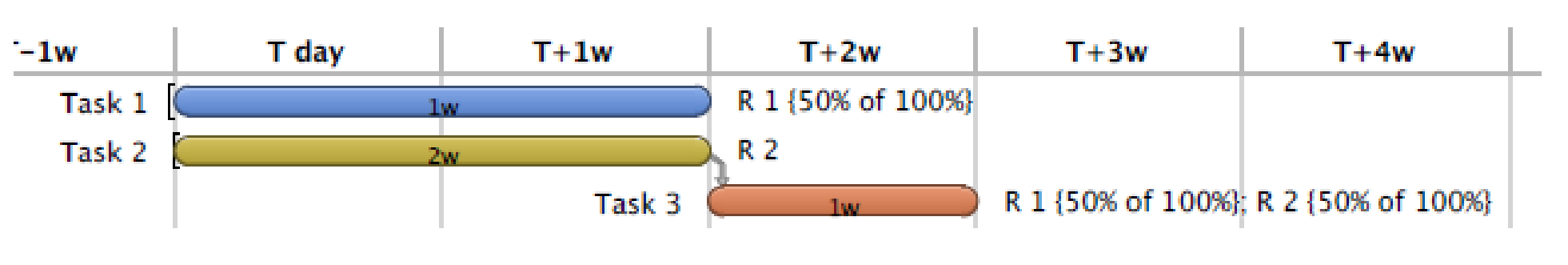
- Drumul critic se referă doar la durata și nu la alte caracteristici precum riscul sau dificultatea
- Activitățile ce nu se află în drumul critic **pot** întârzia planul, dacă întârzierea este suficient de lungă
- Atenție la drumurile (aproape) critice: o întârziere într-o activitate dintr-un drum non-critic poate crea un nou drum critic

Alocarea resurselor și uniformizarea resurselor

Un proces simplificat

- Intrări:
  - planul: activități, constrângeri, efort pentru fiecare activitate
  - echipa de proiect (număr, tipuri și disponibilitatea resurselor)
- Alocarea resurselor:
  - procesul prin care o resursă este alocată unui task, adică este însârcinată cu realizarea unei părți de munca (efort) definite în task
- Constrângeri:
  - conform disponibilității și nevoilor (ex. tipul de resurse necesare pentru o anumită activitate): fără supra-alocari (mai mult față disponibilitatea maximă) (uniformizarea resurselor)
- Daca nu se găseste nici o soluție, se pot varia unele ipoteze (ex. creșterea dimensiunii echipei)

Alocarea resurselor - Exemple



- Legenda:
  - fiecare slot: o săptămână
  - R1 alocat la Task 1 pentru 50% din timpul său
  - **R2 alocat la Task 2 full time**
  - R1 și R2 allocate la Task 3 pentru 50% din timpul lor
- Semnificație:
  - R1 va lucra 20 de ore în săptămânile 1 și 2 și 20 de ore în săptămâna 3
  - R2 va lucra **40 de ore în săptămânile 1 și 2** și 20 de ore in săptămâna 3

Utilizarea resurselor

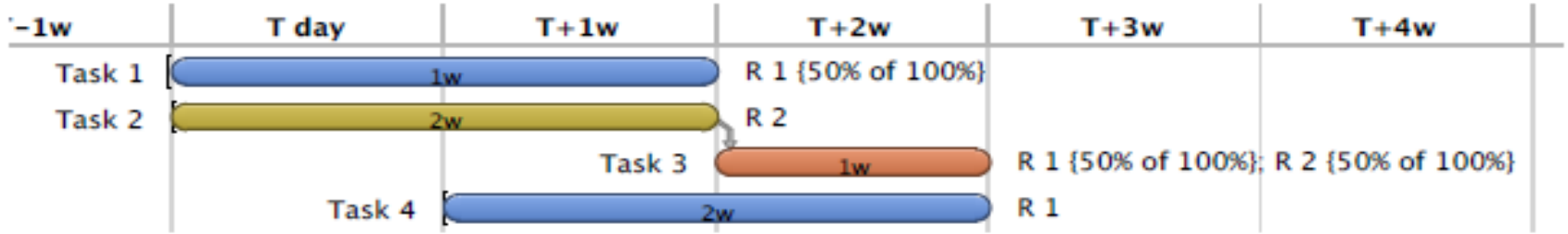
- Puterea de muncă → perioada de timp pentru care este necesară fiecare resursa la un moment dat
- Echipament → numărul de elemente necesare la un moment dat
- Materiale→ cantitatea de materiale necesare (consumate) la un moment dat

Cum se calculează?

- Utilizarea resurselor este calculată adunând cantitatea de muncă necesară pentru o perioadă dată
- Aceasta reprezintă o suma verticală cuprinzând toate sarcinile de muncă

Supra-alocare → o situație în care o resursă este folosită peste capacitatea sa maximă

Exemplu

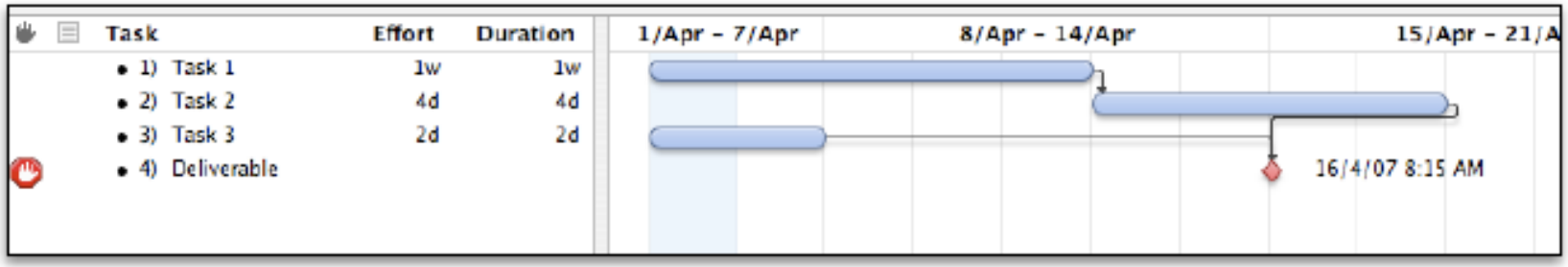


	hours	hours	hours	hours	hours
R1	20	20		T1	
			20	T3	
		40	40	T4	
Total R1	**20**	60	60		
R2	40	40		T2	
			20	T3	
Total R2	**40**	**40**	**20**		

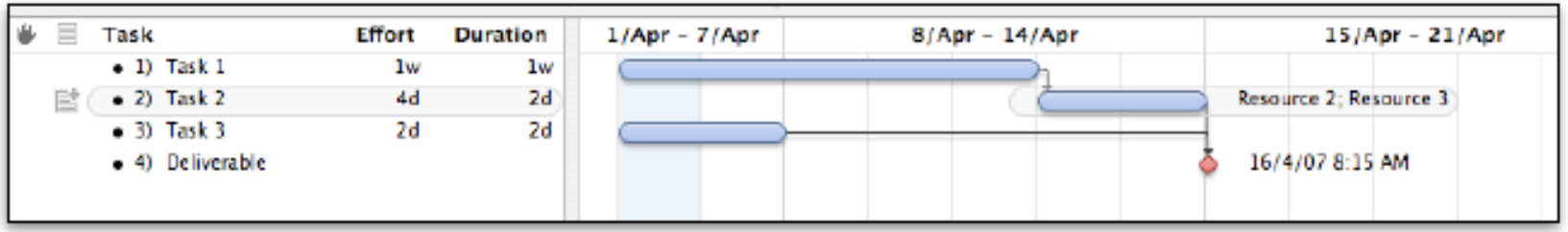
R1 este supra-alocată în W2 (T+1w) și W3 (T+2w)

Exemplu

- Reprezentăm planul subliniind constrângerile puternice. Livrabilele au datele de livrare fixe

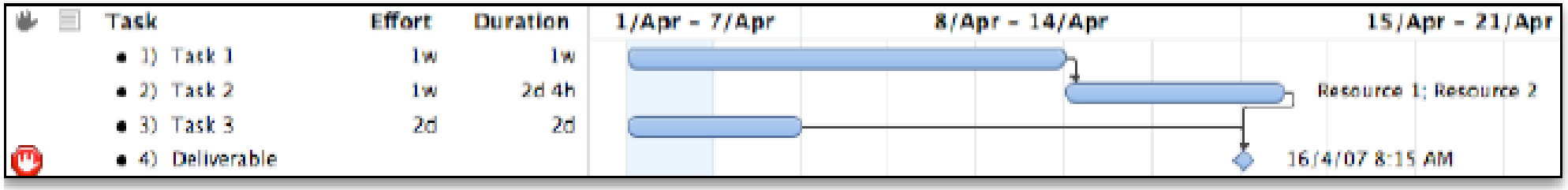


- Alocarea a două resurse pentru Task 2 permite satisfacerea constrângerilor

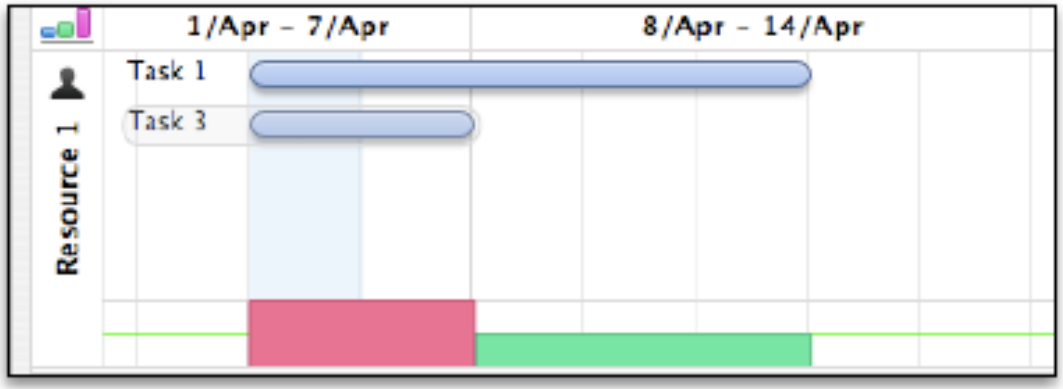


Exemplu

Problemă → Task 1 și Task 3 au nevoie de aceeași resursă



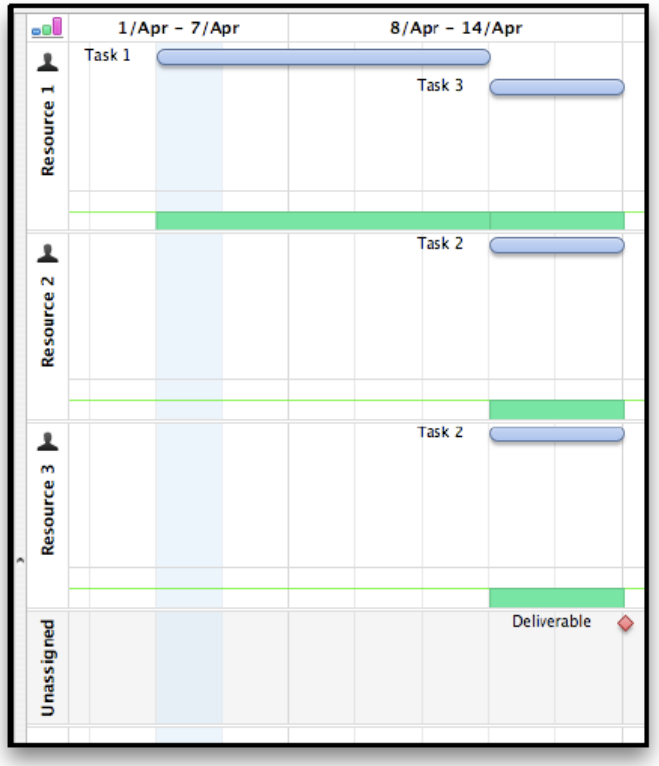
...supra-alocam Resursa 1



Exemplu

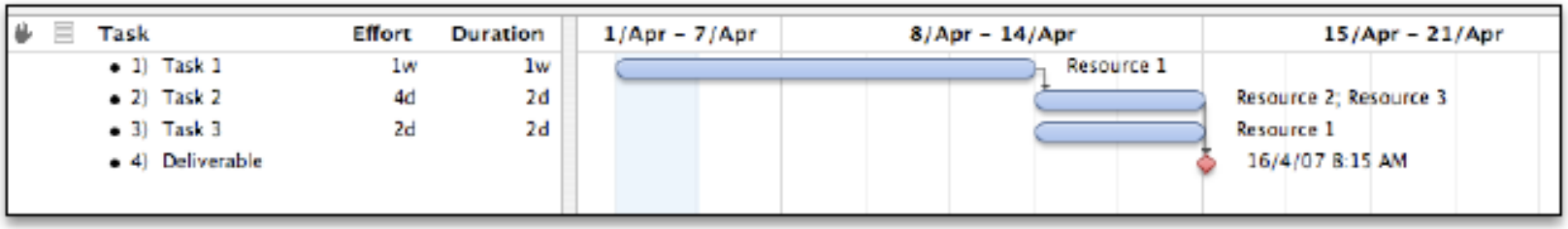
Soluția 1 → Uniformizarea resurselor, inserarea de constrângeri slabe în plan astfel încât nici o resursă nu este supra-alocata (nu muncește mai mult de 100%)

Soluția 2 → Tehnici de compresie (revenim pe acest subiect)



Câteva considerații:

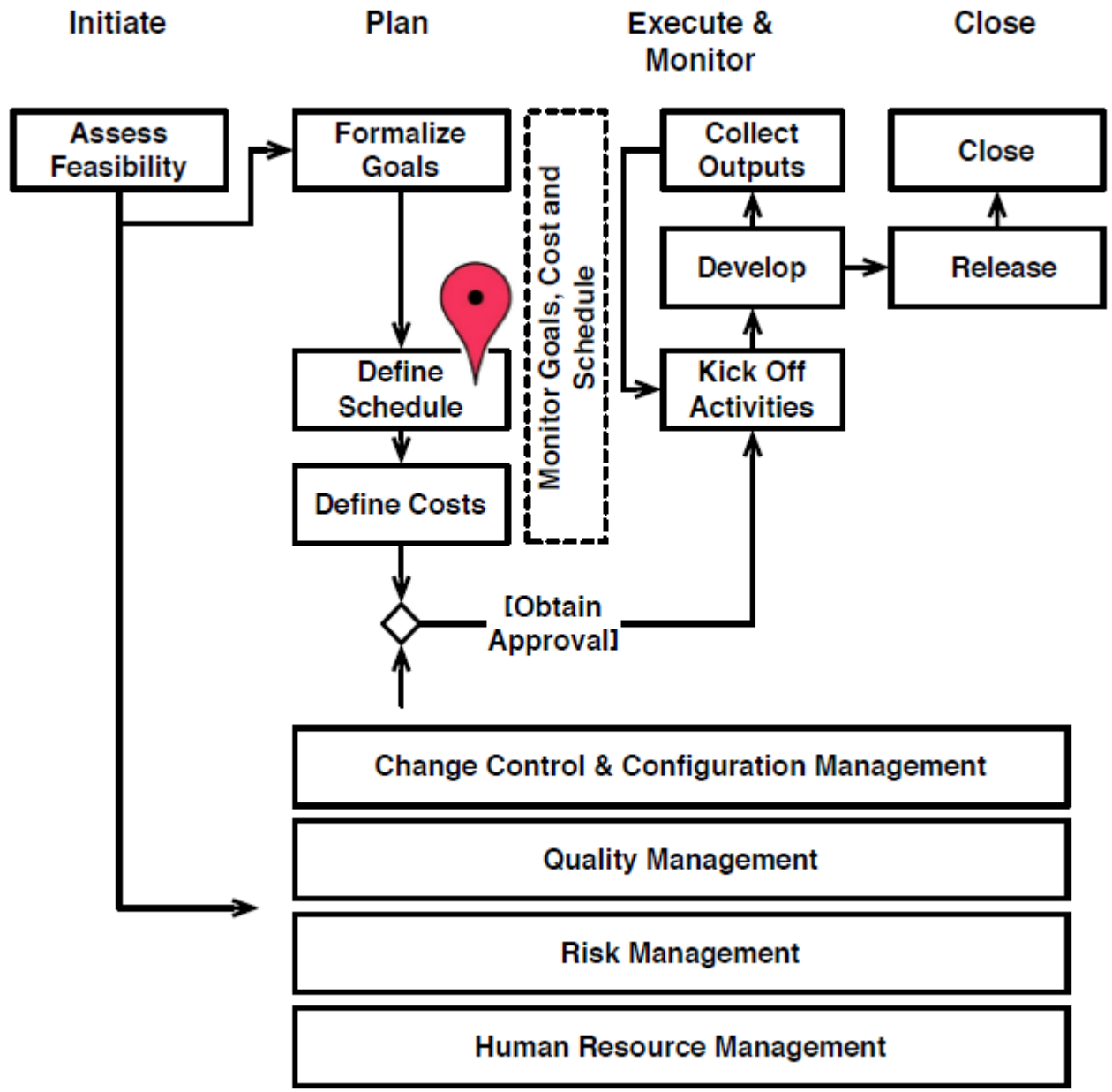
- Resursa 1 va lucra în proiect full time
- Resursa 2 și Resursa 3 sunt necesare doar spre sfârșitul proiectului (pentru Task 2)



Optimizarea planului

Obiective

- Învățarea tehnicilor pentru scurtarea unui plan
- Înțelegerea riscurilor apărute după scurtarea unui plan


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Considerație preliminară

- Prima versiune a planului va arăta cel mai probabil că nu vom fi capabili să livrăm conform deadline-ului stabilit de către sponsor
- În ipoteza ca planul este precis, există două opțiuni:
  - Proiectul nu se începe (în cazul în care nu este fezabil, ținând cont de constrângeri)
  - Proiectul este scurtat, folosind una dintre tehnicile descrise în acest curs
- A treia opțiune (revizuirea estimărilor) nu reprezintă o idee bună.

Realizarea unui plan fezabil

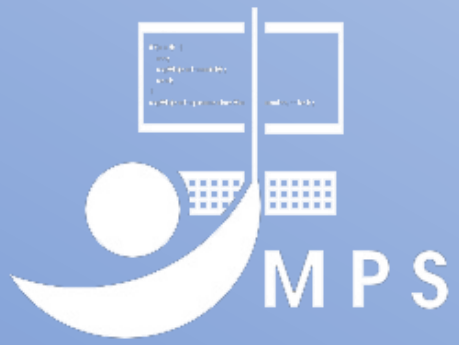
Schimbarea unor ipoteze pe care se bazează planul. De exemplu:

1. Reducerea scopului (transformă unele activități în mai scurte sau inutile)
2. Reducerea calității (transformă unele activități în mai scurte sau inutile)
3. Externalizarea unor activități (crește riscul și, posibil, costurile)

Acționarea asupra logicii planului:

4. Creșterea resurselor (poate face proiectul mai scump) [PROJECT CRASHING]
5. Evaluarea cursului alternativ al acțiunii (ex. schimbarea procesului de dezvoltare). Totuși: poate fi dificil în cazul în care echipa este obișnuită cu un anumit proces de dezvoltare
6. Substituirea activităților
7. Încălcarea regulilor: eliminarea constrângerilor puternice din plan (crește riscurile și, posibil, costurile) [FAST TRACKING - URMARIRE RAPIDA]
8. Lucrul asupra probabilității livrarii la timp **[**CRITICAL CHAIN MANAGEMENT - GESTIONAREA CRITICA A LANTULUI]

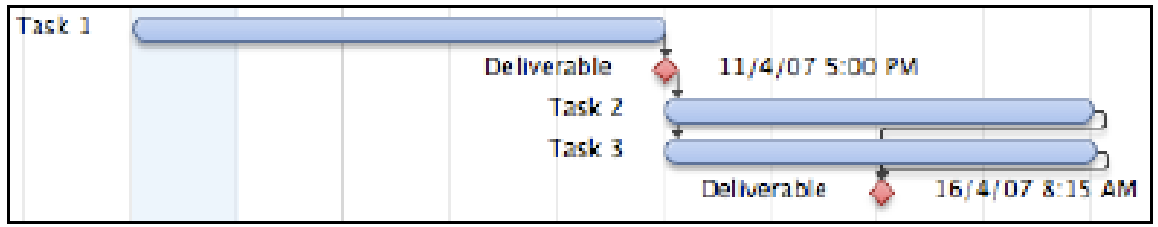
Metoda Project Crashing



Project Crashing

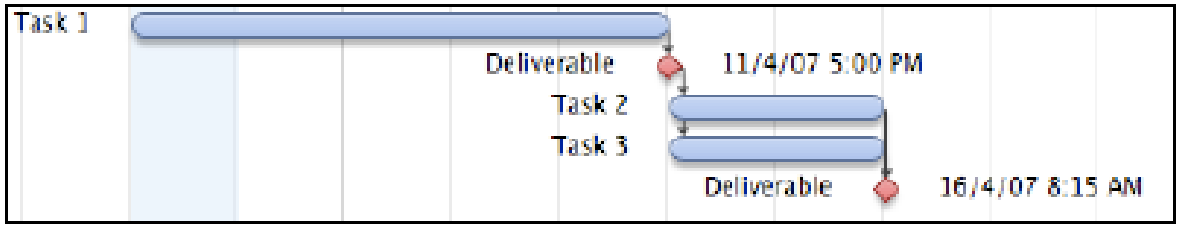
- Durata proiectului poate fi (adesea) redusă prin alocarea de mai multă muncă pentru activitățile proiectului, sub forma de lucru peste program, și prin alocarea de mai multe resurse, precum materiale, echipamente etc.
- Totuși, munca și resursele adiționale cresc costurile proiectului
- Astfel, decizia de reducere a duratei proiectului trebuie să fie bazată pe o analiză și pe compromisul dintre timp și cost
- Project Crashing reprezintă o metoda de scurtare a duratei proiectului prin reducerea timpului uneia sau a mai multor activități critice de proiect, obținând astfel un timp mai scurt decât timpul normal de activitate. Scopul este de a reduce durata proiectului și minimizarea costului crashing-ului

Project Crashing - Exemplu



cost = $ 3000
cost = $ 4000

- Project Crashing: alocăm mai multi oameni pentru Task-urile 2 și 3

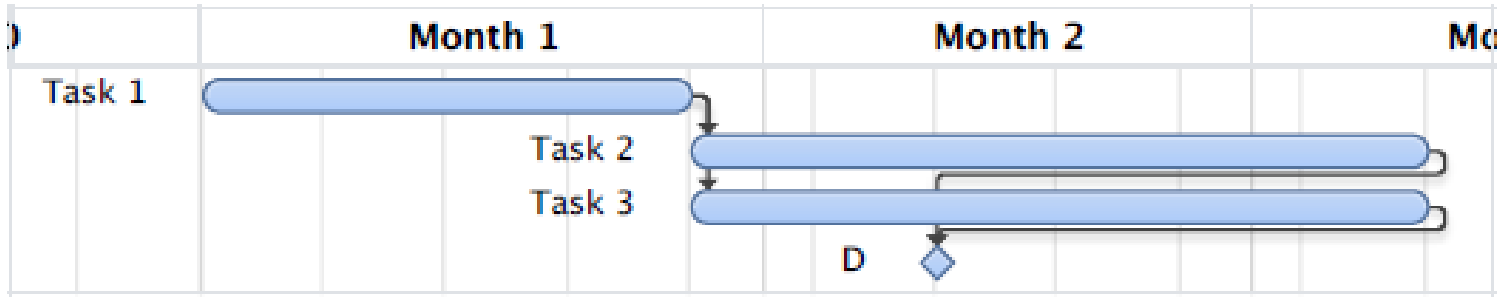


cost = $ 6000
cost = $ 8000

- Timp de Crash = 2 unități de calendar (ex. luni)
- Costul de Crash = (6000 + 8000) - (3000 + 4000) = 14000 - 7000 = 7000
- Crashing convenabil conform costurilor livrării întârziate

Project Crashing simplu - Exemplu

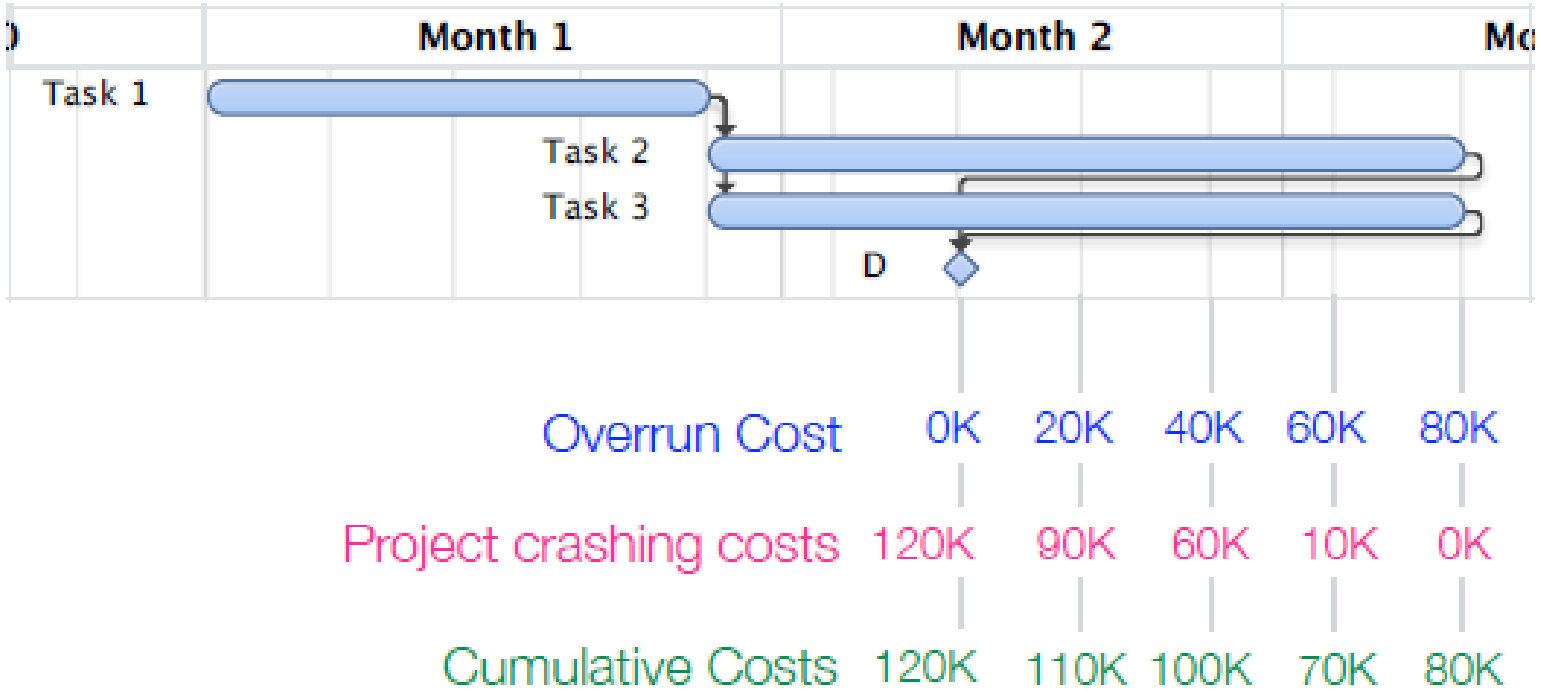
Proiectul este întârziat: se prespune că trebuie să plătim 20 de mii de euro pentru fiecare săptămână de întârziere



Putem reduce Task 2 și Task 3 prin alocarea de mai multe resurse: 10 mii pentru o săptamână, 60 de mii pentru două săptămâni, 90 de mii pentru trei săptămâni, 120 de mii pentru patru săptămâni.

Project Crashing simplu - Exemplu

Proiectul este întârziat: se presupune că trebuie să plătim 10 mii de euro pentru fiecare săptămână de întârziere



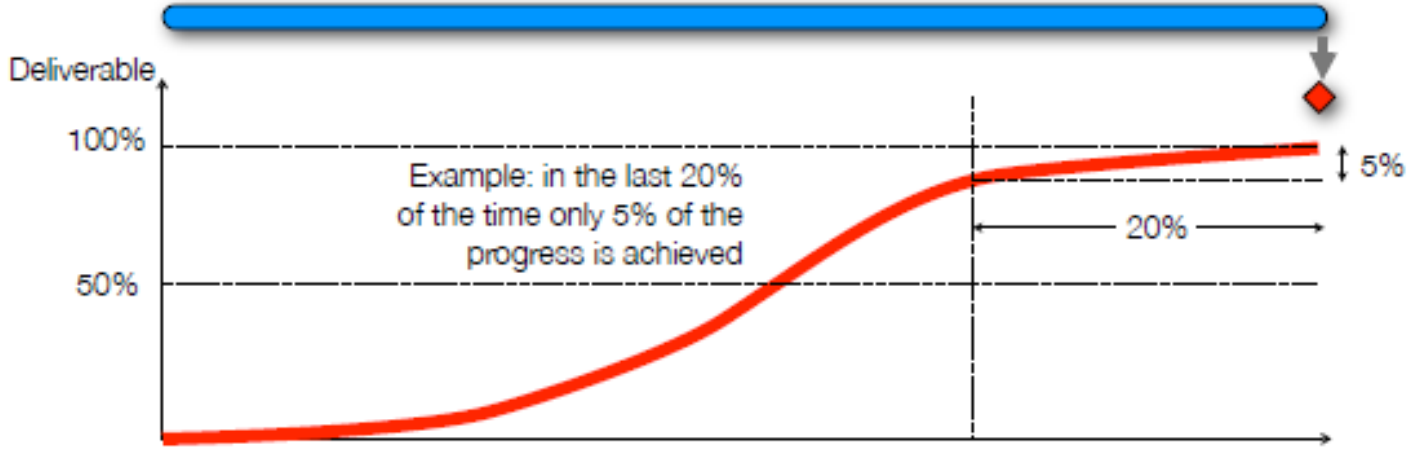
Project Crashing simplu - Exemplu

- Cea mai bună soluție (doar în cazul în care costurile contează) → scurtarea cu o săptămână (costurile cumulative au cea mai mică valoare)
- Alte considerente pot favoriza alte alegeri. De exemplu:
  - Livrarea la timp ar putea fi esențială pentru a păstra un client. Într-un asemenea caz, putem decide să suportăm cheltuieli mai mari
  - Putem alege să livrăm târziu (și să păstrăm activitățile așa cum sunt) pentru a evita lucrul peste program al echipei sau pentru ca riscurile asociate diminuării calității pot crește odată cu scurtarea activităților

Metoda Fast Tracking

Fast Tracking

- Metoda Fast Tracking este bazată pe faptul că livrabilele de activități sunt produse (și rafinate) incremental în timpul executării activității
- Exemplu: un document cu cerințe nu este scris în ultima zi a activității “redactarea cerințelor”. El este mai degrabă scris puțin câte puțin, în timp ce activitatea este executată

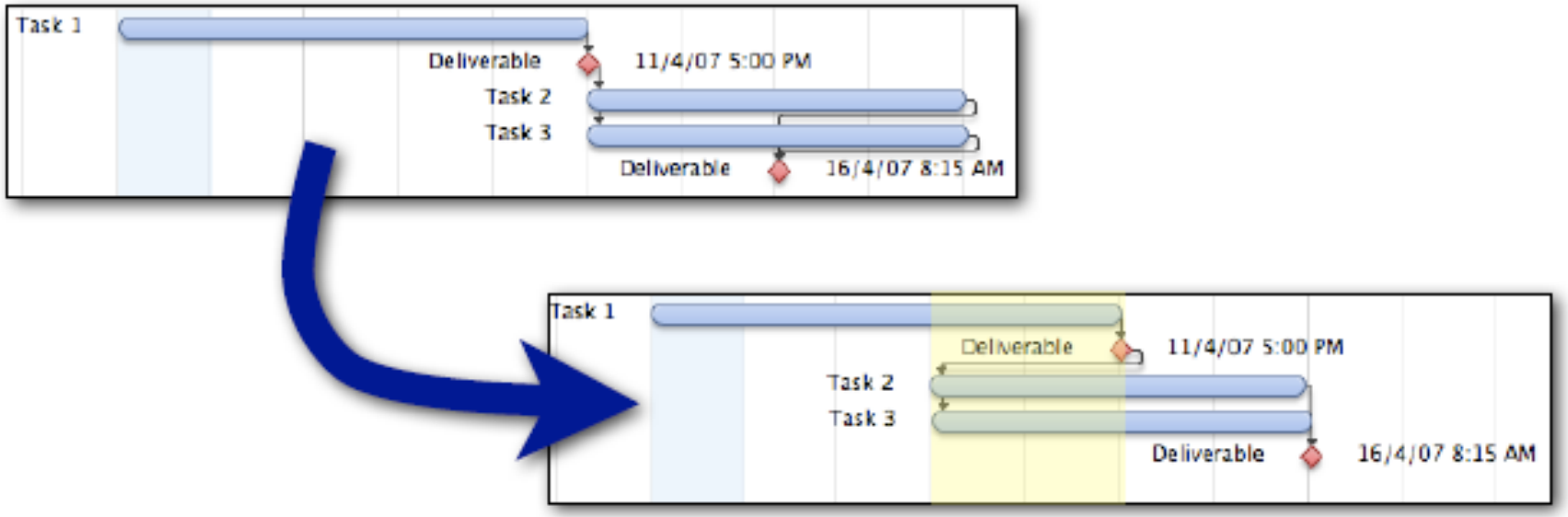


Fast Tracking

- Să considerăm exemplul din slide-ul precedent: dacă ultimele 5% nu reprezintă o parte critică a livrabilului, atunci putem începe mai devreme activitățile ce depind de documentul de cerințe.

Fast Tracking

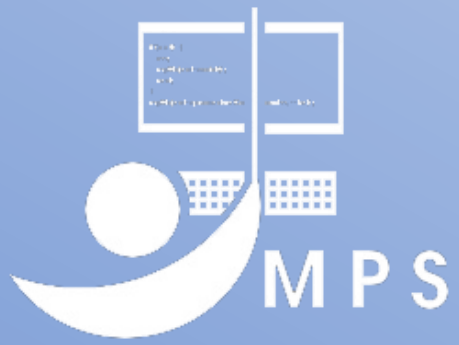
- Metoda Fast Tracking funcționează prin suprapunerea de activități ce în mod normal ar fi secvențiale



Fast Tracking - Probleme și reguli de bază

- Metoda Fast Tracking este riscantă și poate produce refaceri ale activităților
- Când decidem ce dependențe ar trebui sparte, trebuie să luăm în calcul:
  - Cum va progresa producerea de livrabile în timpul activității (va produce ieșiri intermediare?) și cum se va consolida (va fi stabilă ieșirea intermediară?)
  - Riscul implicat în schimbările aduse rezultatului (care va fi consecința refacerii activității? Cum va afecta restul planului?)

Metoda Critical Chain Management (CCM)



Critical Chain Management

- Metoda Critical Chain Management pornește de la ideea că estimările sunt simple presupuneri
- A spune că o activitate durează X (sau necesită efortul Y) înseamnă că **de cele mai multe ori**, activitatea va avea nevoie de X pentru a fi completată. **Vor fi însă și cazuri** în care activitatea poate dura mai mult sau mai puțin.
- Pentru a gestiona evenimentele neprevăzute (și a contrasta cu optimismul), managerii de proiect “umfla” estimările, aducându-le în zona pesimistă.

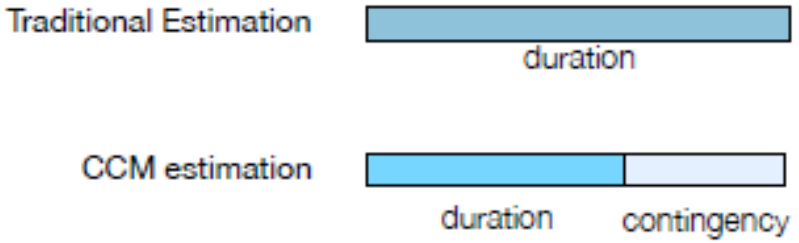
Considerente Statistice

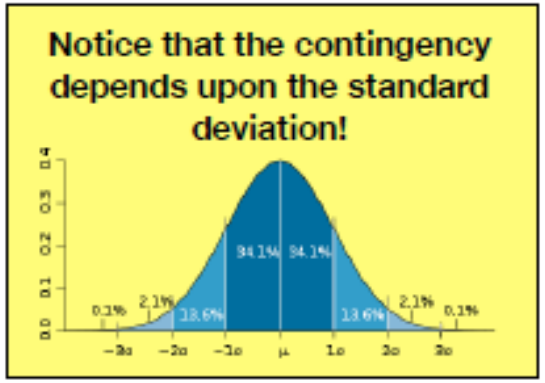
- Eficiența provine din:
  - Item 1 → **Discutăm cele mai probabile estimări, nu pe cele pesimiste:** în particular, numim eveniment neprevăzut diferența de durată dintre o estimare 50% probabilă și una 90% probabilă
  - Item 2 → Discutăm despre **lanțuri de activități. Deviația standard** a unui lanț de activități este mai mică față de suma deviațiilor standard ale activităților din lanț

$$\sigma_{a+b} = \sqrt{\sigma_a^2 + \sigma_b^2}$$

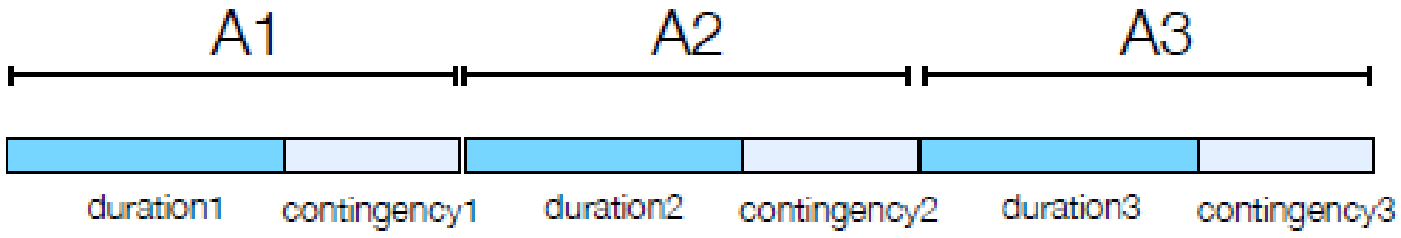
$$\sigma_{a+b} < \sigma_a + \sigma_b$$

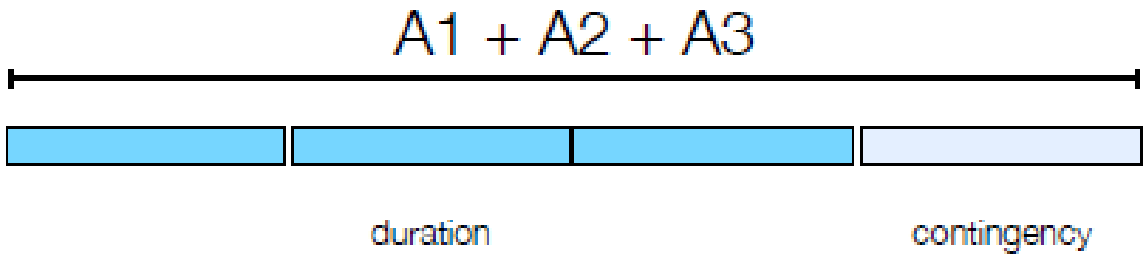
CCM - Item1 estimări





CCM - Item2 suma variantelor





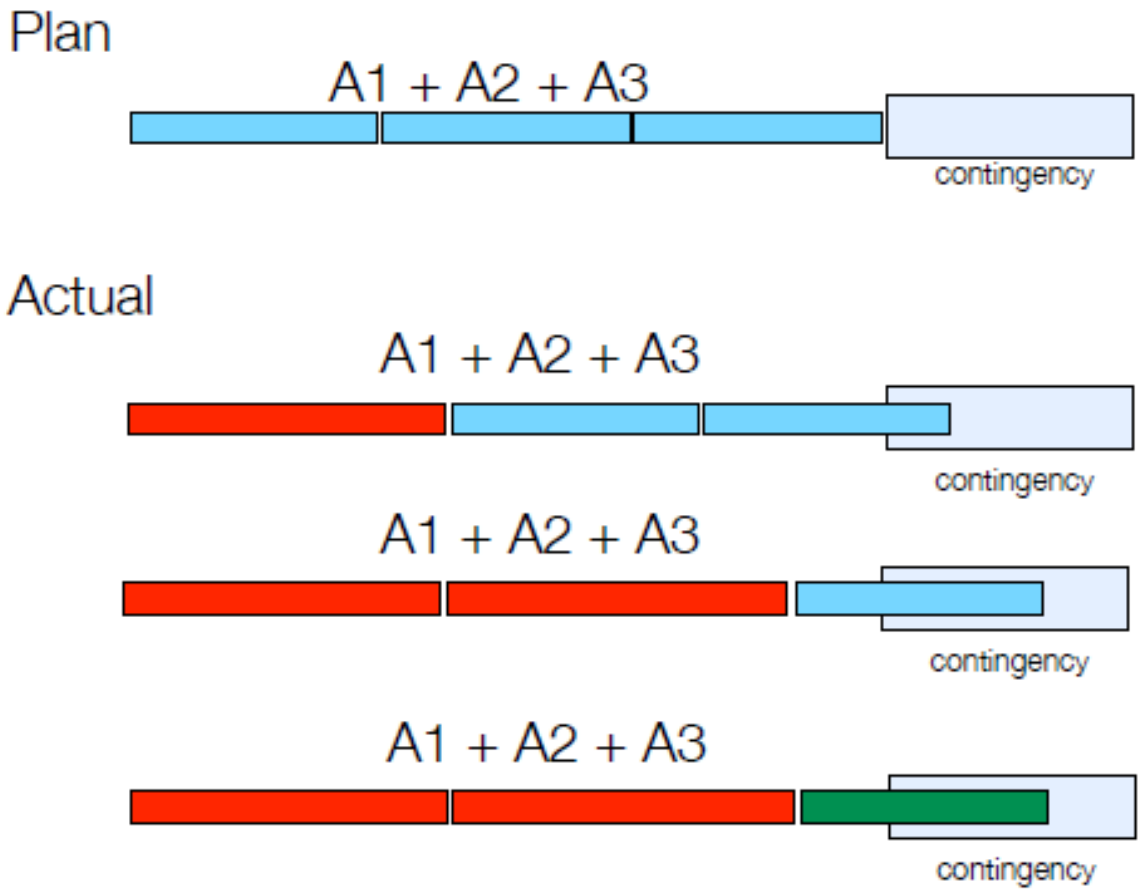
duration = duration1 + duration2 + duration3

contingency < contingency1 + contingency2 + contingency3

CCM - Ideea de bază

- Atunci când planificăm, noi discutăm și monitorizăm lanțurile și facem un buffer explicit de evenimente neprevăzute
- Item1 (durată + neprevăzute) + Item2 (suma variantelor) asigură faptul că planul este mai scurt față de cel obținut în abordarea standard
- În timpul executării planului, variația statistică va face ca unele activități să dureze mai mult decât era planificat. Aceste activități vor face ca lanțul să se reverse peste buffer-ul de evenimente neprevăzute, ceea ce este în regulă până într-un punct
- Managerul gestionează lanțurile, nu activitățile

Exemplu



- A1 durează mai mult față de cât era planificat și lanțul intră în buffer.
- A2 durează mai mult decât era planificat și lantul continuă să intre în buffer.
- Totuși A3 durează cât era planificat și o parte din buffer este economisită.

CCM - Gestionarea lanțurilor

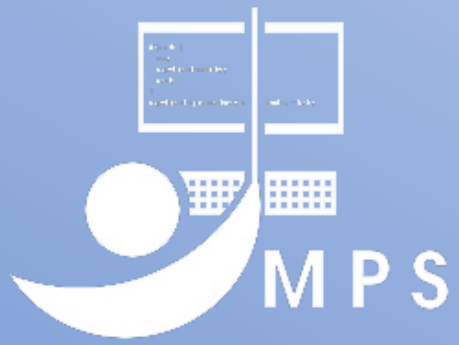
- Atunci când folosim CCM, două noțiuni importante apar:
  - **Definirea buffer-ului** → care lanțuri sunt cele mai bune de considerat sau, altfel spus, unde plasăm bufferele
  - **Gestionarea buffer-ului** → când trebuie să începem să ne îngrijorăm din cauza revărsării buffer-ului de evenimente neprevăzute

Gestionarea buffer-ului

Task Execution Status	Buffer Penetration: First Third	Buffer Penetration: Second Third	Buffer Penetration: Final Third
First Third	NO ACTION	Serious problem	**Very serious problem**
Second Third	NO ACTION	Could be a problem: identify and formulate solution	Serious problem
Final Third	**Very good! We are early**	NO ACTION	Monitor the situation

Fixarea costurilor și bugetarea

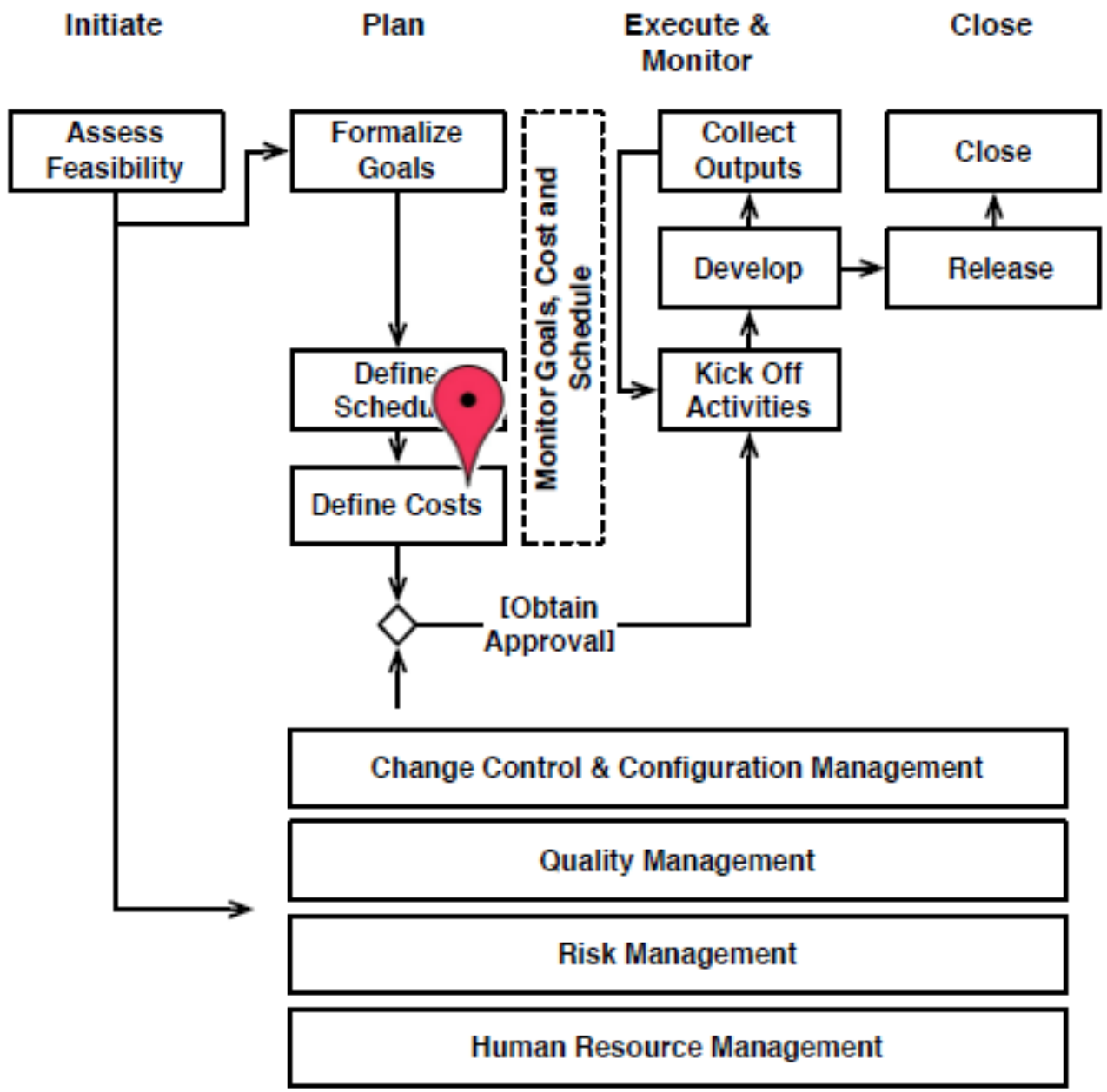
De la cost la valoare: metode și tehnici de stabilire a costului potrivit pentru software



ST COST COST CO
OST COST COST COS
ST COST COST COS
COST COST COS
WE'RE FANTASTICALLY EXCITED ABOUT THIS NEW PROJECT...WE THINK IT OPENS UP HUGE POTENTIAL MARKETS...
SHARE YOUR ENTHUSIASM FOR THE FUTURE
search ID: forn1496
© Original Artist
Reproduction rights obtainable from
www.CartoonStock.com

Obiective

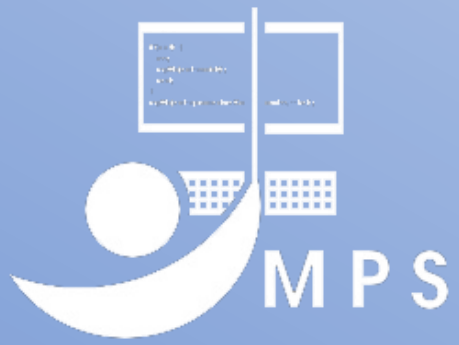
- Intrebări ce pot apărea:
  - Cât de mult costă dezvoltarea software-ului?
  - Respectă proiectul bugetul?
- Obiectivele cursului:
  - Bugetarea
  - Gestionarea costurilor proiectului


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedu
Define Costs
Monitor Goals, Cost and Schedule
[Obtain Approval]
Collect Outputs
Develop
Kick Off Activities
Close
Release
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Câteva definiții

- **Fixarea costurilor**: determinarea costurilor simple pentru livrarea proiectului
- **Bugetarea (și controlul costurilor):** determinarea nevoilor financiare ale unui proiect și pregătirea celor necesare pentru monitorizarea cheltuielilor (și a câștigurilor)
- **Fixarea prețurilor:** determinarea sumei de bani cerute pentru proiect
- **Costul Ciclului de Viata (Life Cycle Cost – LCC, uneori numit și Total Cost of Ownership):** costurile ce trebuie susținute pentru a opera (utiliza) sistemul în întregul său ciclu de viață

Fixarea costurilor proiectului software



Fixarea costurilor proiectului (software)

- **Costurile proiectului**
  - Cheltuielile pe care le vom suporta pentru a termina proiectul
  - Nu iau în calcul profitul
  - Constituite din costurile directe și indirecte
- **Structura elementului de cost**
  - Structura ierarhică ce definește obiectele de cost ce vor determina bugetul proiectului
  - Ajută structurarea fixarii costurilor și monitorizarea proceselor, reducând de asemenea riscul de a contabiliza de două ori aceleași cheltuieli

Costuri directe

- Costurile directe → costuri legate de realizarea proiectului
- Costuri directe pentru proiectele software:
  - **Personal:**
    - salariile persoanelor direct implicate în proiect (brute, nu nete)
  - **Materiale și stocuri:**
    - costurile materialelor necesare pentru a produce rezultatele proiectului
    - de obicei luate în calcul dacă proiectul are nevoi speciale
  - **Hardware și software:**
    - Sistemele necesare pentru a dezvolta produsul
    - de obicei luate în calcul dacă proiectul are nevoi speciale
  - **Transport, ședințe, evenimente**
  - **Alte costuri:**
    - cărți, training, închiriere echipament etc.

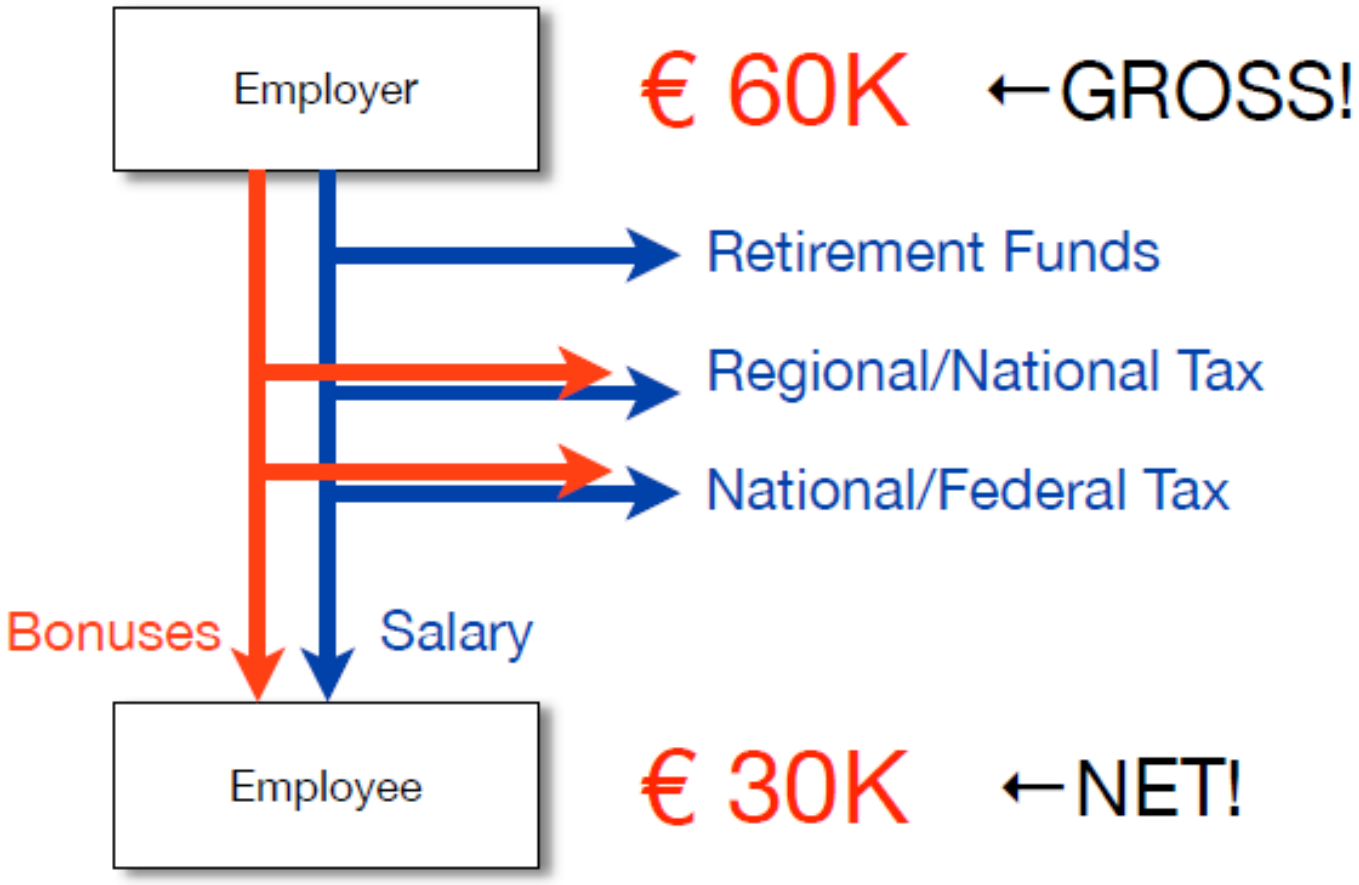
Fixarea costurilor proiectului (software)

- **Costurile indirecte** → cheltuieli necesare pentru a asigura funcționarea întreprinderii și a face munca posibilă
- Principalele elemente de cost în dezvoltarea de software:
  - Generale:
    - ❖ Costurile spațiului de lucru (închiriere, încălzire)
    - ❖ Consumabile
    - ❖ Echipament standard
    - ❖ Personal administrativ
  - De proiect:
    - ❖ Pentru proiectele mai mari, există elemente direct contabilizate în proiect

Calcularea costurilor indirecte

- Identificarea cheltuielilor ce contribuie la costurile indirecte
- Identificarea unei strategii pentru alocarea costurilor indirecte ale unui proiect
  - În funcție de efort (mai mult efort = mai multe costuri indirecte) – rata uniformă sau proporțional
  - În funcție de bugetul proiectului (drept procentaj din bugetul proiectului)
- În mod regulat
  - Evaluarea cheltuielilor apărute în anii anteriori
  - Estimarea cheltuielilor indirecte pentru anii ce urmează
  - Estimarea efortului pe care îl putem produce
  - Determinarea ratei forfetare

Costurile de personal - brut vs net



Costul proiectului reprezintă

$$\sum_{j=1}^{n} Hours_j * (PC_j + O_j) + \sum_{i=1}^{n} C_i$$

H	Numărul total de ore pentru profilul j
PC	Costul profilului j
O	Costurile generale pentru profilul j (Overhead)
C	Costul elementului I (ex. hardware)

Costul proiectului

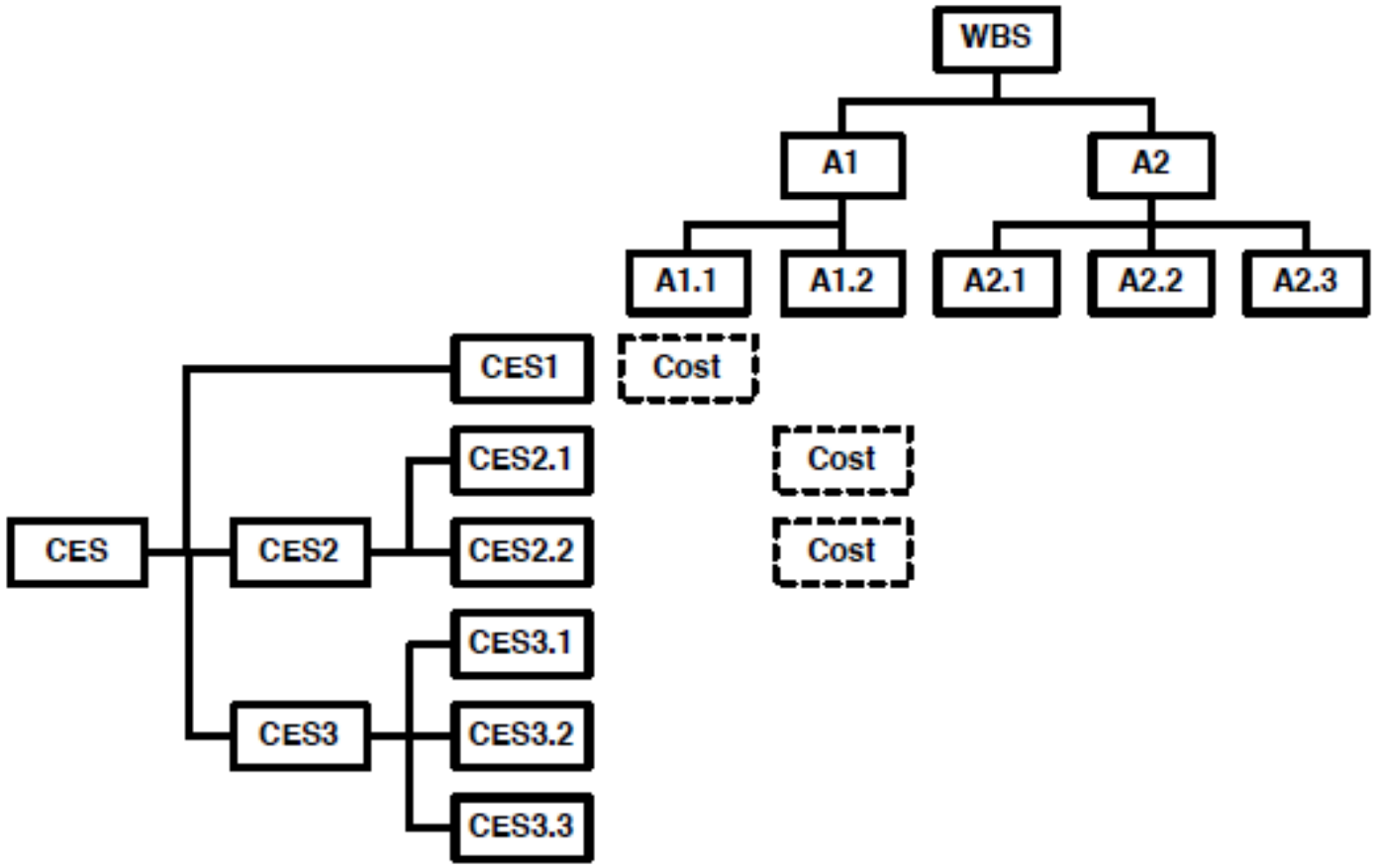
- Costurile proiectului pot fi privite din mai multe puncte de vedere:
  - Costul per element (de cât avem nevoie pentru hardware?)
  - Structura proiectului (de cât avem nevoie pentru redactarea cerintelor?)
  - Cheltuieli de-a lungul timpului (de cât avem nevoie în 2018?)

Gestionarea costurilor proiectului

Obiective și metode

- Obiective
  - Asigurarea disponibilității banilor atunci când trebuie cheltuiți
  - Monitorizarea cheltuielilor proiectului astfel încât el să rămână în buget sau pentru a permite luarea de măsuri corespunzătoare atunci când este nevoie
- Metode
  - Definirea unui flux numerar de bază
  - Autorizarea cheltuielilor
  - Păstrarea evidenței cheltuielilor

Costurile și structura proiectului



Costurile și timpul proiectului

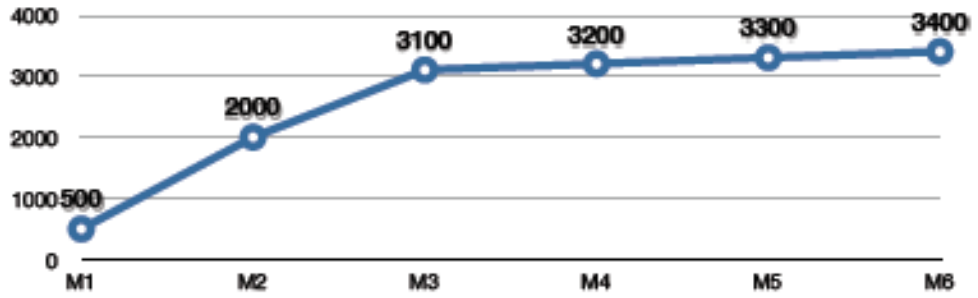
	Q1	Q2	Q3	Q4	Total
Expenses					
Expense 1	€ 10,000	€ 30,000	€ 50,000	€ 10,000	**€ 100,000**
Expense 2	€ 20,000	€ 40,000	€ 60,000		**€ 120,000**
Total Expenses	**€ 30,000**	**€ 70,000**	**€ 110,000**	**€ 10,000**	**€ 220,000**
Incomes					
Payment	€ 50,000			€ 200,000	**€ 250,000**
Total Incomes	**€ 50,000**	**€ 0**	**€ 0**	**€ 200,000**	**€ 250,000**
Balance	€ 20,000	-€ 70,000	-€ 110,000	€ 190,000	**€ 30,000**
Financial Need		-€ 50,000	-€ 180,000		

Costurile și timpul proiectului

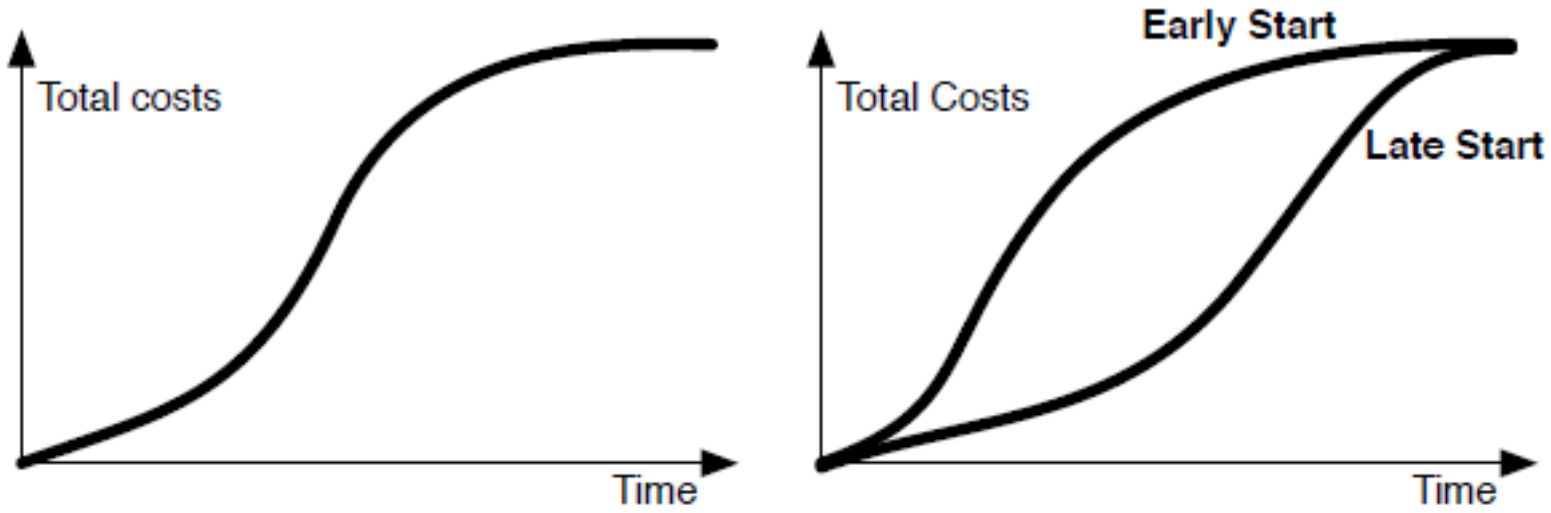
Name	Planned Cost	Time
A	1000	A
B	2000	B
C	400	C

A	1000	500	500			
B	2000		1000	1000		
C	400			100	100	100	100

Total	500	1500	1100	100	100	100
Cumulative	500	2000	3100	3200	3300	3400



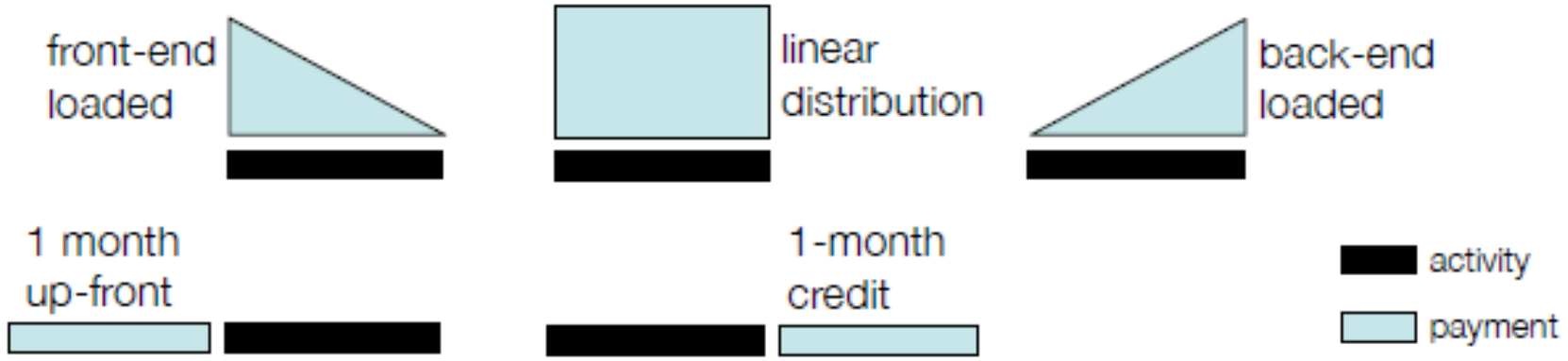
Costurile și timpul proiectului



Banana Shape

Profilul cheltuielilor

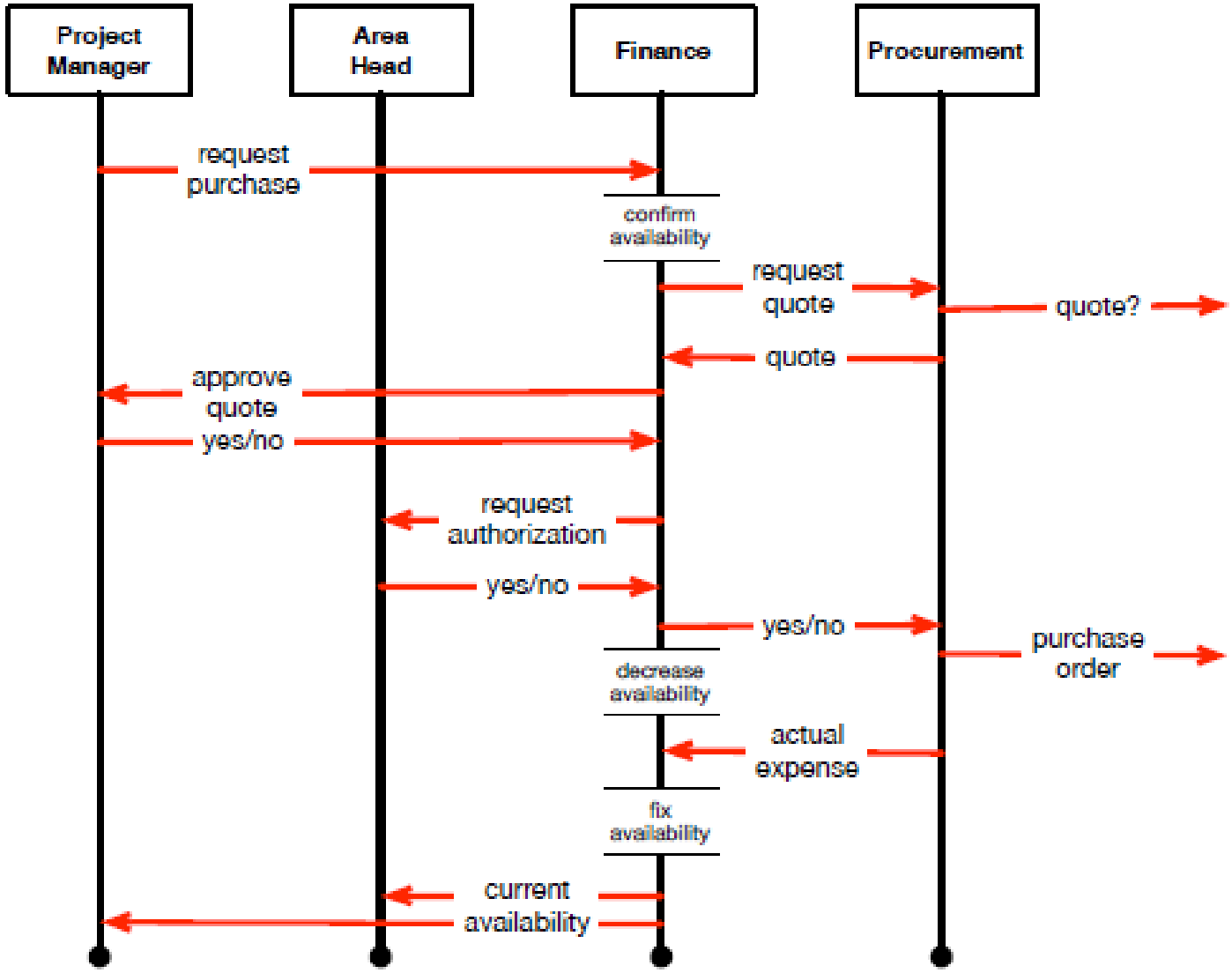
- În general, presupunem că volumul de muncă este distribuit uniform în timpul unei activități:
  - O resursă implicată într-o activitate ce necesită 40 de ore de efort și o durată de o săptâmană va lucra 8 ore pe zi
- Totuși, nu este întotdeauna acesta cazul, întrucât profile diferite pot fi definite pentru efort și cheltuieli



Autorizarea cheltuielilor

- Managementul proiectului și managementul financiar sunt de obicei atribuite unor structuri diferite
- Conform structurii organizaționale, puterea de a autoriza cheltuielile și plățile poate să țină doar de managerul de proiect sau să necesite un flux de lucru mai complex
- Regulile iau în calcul aspecte precum:
  - disponibilitatea fondurilor
  - daca acea cheltuială e bugetată sau nu
  - suma de bani (cheltuieli mai mari față de un anumit prag pot necesita o autorizare specială)

Autorizarea cheltuielilor
Exemplu



Raportul final

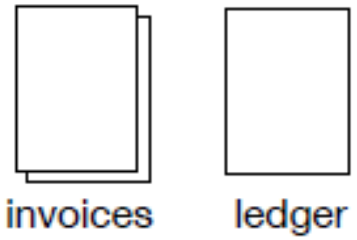
- La sfârșitul fiecărei perioade de raportare, se redactează documente despre statusul financiar al proiectului
- Două informații sunt disponibile:
  - Cheltuielile bugetate vs cheltuielile reale
  - Contabilizarea cheltuielilor
- Informația este folosită în moduri diferite:
  - Pentru a analiza **deviările** (ce diferențe au existat)
  - Pentru a evalua starea proiectului și a lua acțiunile corespunzătoare

Fluxul de lucru

Cheltuieli așteptate

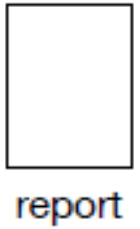
Budget	Y1	Y2	Y3
Personnel	4000	5000	3000
Hardware	1000	2000	
Subcontracting		1000	6000
…			

Cheltuieli reale



Budget	Y1	Y2	Y3
Personnel	**3000**	5000	3000
Hardware	500	2000	
Subcontracting		1000	6000
…			

Raport de final & Revizuire



Budget	Y1	Y2	Y3
Personnel	5000	4000	3000
Hardware	500	1000	
Subcontracting		1000	6000
…			

Informațiile de interes

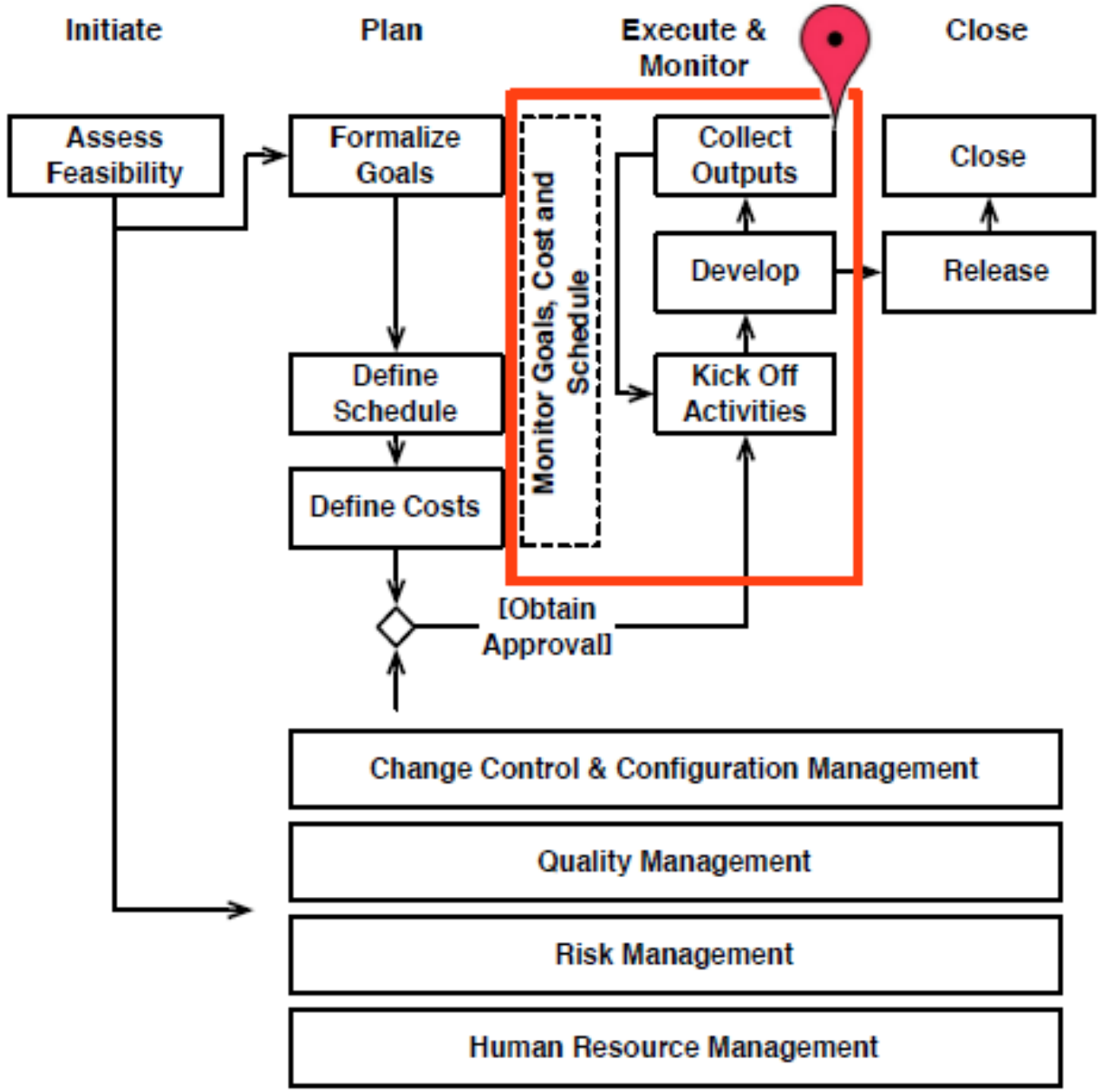
- Budget → cantitatea inițial bugetată
- Transferuri → variațiile realizate asupra bugetului
- Actualitate → cantitatea cheltuită în realitate

Budget	Budget	Variations	New	Spent	Available
Personnel	4000	+2000	6000	5000	1000
Hardware	3000	-2000	1000	0	1000
Subcontracting	1000		1000	400	600

Execuția, monitorizarea și controlul proiectului

Obiective

- Planurile sunt inutile dacă nu sunt puse în practică și, în timpul executării, monitorizate și actualizate pentru a reflecta situația reală
- Acest capitol prezintă:
  - Activitățile necesare pentru a pune un plan în practică
  - Tehnicile de monitorizare și controlare a planurilor (progres și costuri vs planificare și buget)
  - Earned Value Analysis, o tehnică ce îi permite managerului de proiect să monitorizeze un proiect într-o maniera integrată


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
Collect Outputs
Develop
Kick Off Activities
Close
Release
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Executarea proiectului

Executarea proiectului

- Executarea proiectului reprezintă adevarata realizare a muncii
- Exista trei principale activități de management:
  - Demararea activităților
  - Strângerea rezultatelor activităților
  - Strângerea informațiilor despre starea proiectului

Demararea activităților

- Scop:
  - Asigurarea unui început formal pentru o parte semnificativă a proiectului
  - Asigurarea alinierii echipei la obiectivele și modalitățile de începere a activităților
- Principala modalitate este o ședinta de început
- În general, orice modalitate de comunicare poate fi folosită (dar poate fi mai puțin eficientă decât o ședință de început)
- Trebuie ales un nivel potrivit de granularitate

Strângerea rezultatelor activităților

- Scop:
  - Colectarea sistematică a rezultatelor proiectului (livrabile)
  - Ocazie de a evalua lecțiile învățate
- Pentru proiectele software, principala modalitate de colectare a rezultatelor proiectului este reprezentată de un repository + versionare
- O ședință de evaluare a lecțiilor învățate poate fi de asemenea folosită pentru a “formaliza” colectarea rezultatelor

Strângerea informațiilor despre starea proiectului

- Scop → colectarea sistematică a datelor pentru evaluarea statusului proiectului
- Poate fi realizată regulat (frecvența aleasă în funcție de dimensiunea proiectului)
- Poate fi realizată în funcție de nevoie (pentru evenimente exceptionale, ex. riscuri)
- Datele cantitative pot fi strânse pe baza metodelor de monitorizare
- Datele calitative (ex. moralul echipei, feeling-ul despre statusul sau dificultatea unui task) trebuie de asemenea colectate

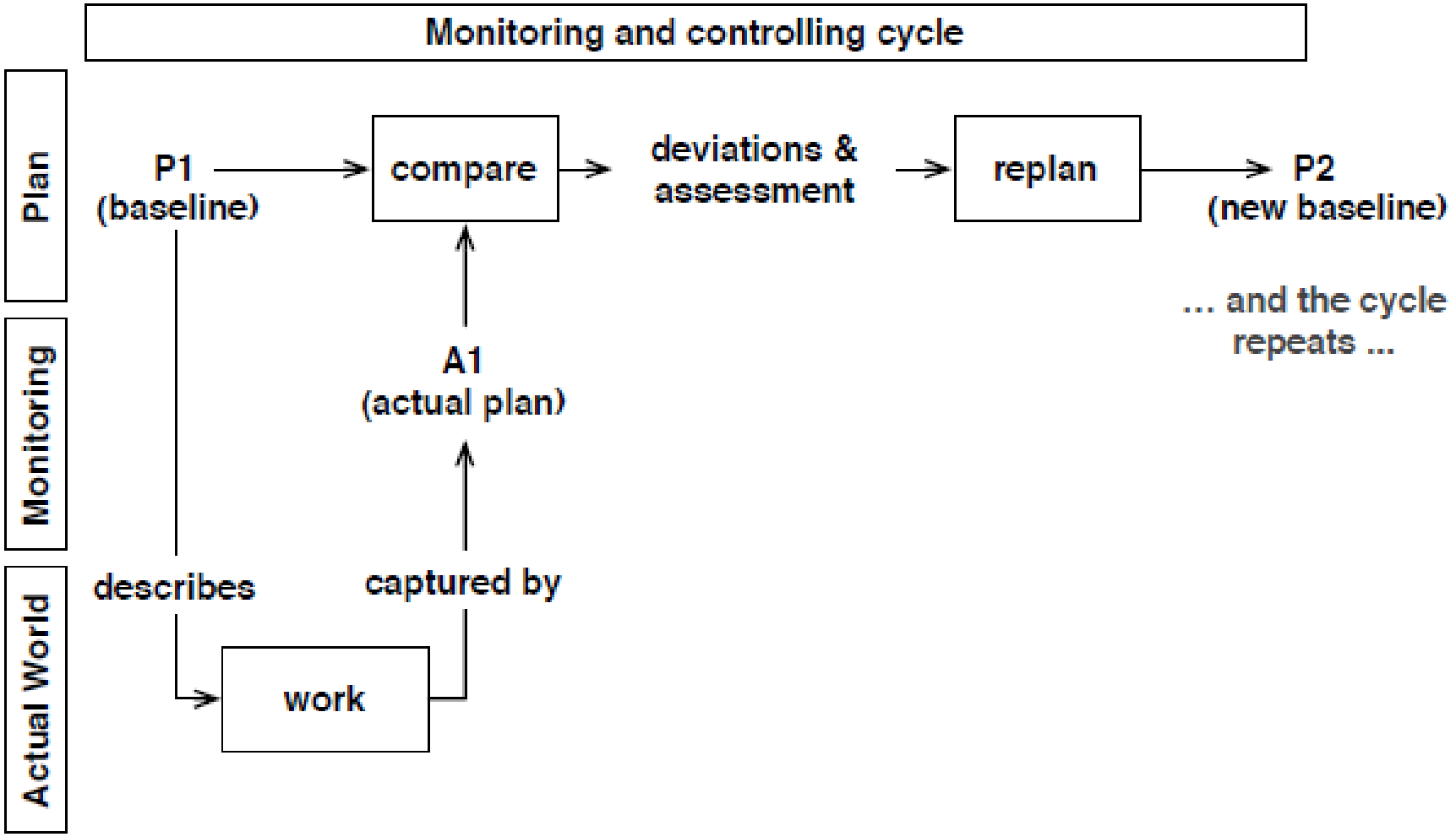
Monitorizarea și controlul proiectului

All stable processes we shall predict. All unstable processes we shall control.

John von Neumann

Introducere

- Obiective:
  - **Pentru proiect** → evaluarea statusului proiectului (scop, timp, cost, calitate etc.), analizarea deviațiilor, luarea de decizii corective, dacă este nevoie
  - **Pentru organizație** → colectarea datelor ajuta la construirea unui plan mai bun și mai exact pentru proiectele viitoare
- Proces (în mod regulat):
  - **Colectează** → Strânge datele despre statusul curent al proiectului
  - **Masoară și compară** → Compară cu un plan de baza, subliniază orice deviație, realizează o proiecție pe baza datelor curente
  - **Evaluează și replanifică** → Decide dacă sunt necesare acțiuni corective. Dacă da, planifică, documentează și ia acele acțiuni


Monitoring and controlling cycle
Plan
Monitoring
Actual World
P1
(baseline)
compare
deviations &
assessment
replan
P2
(new baseline)
... and the cycle
repeats ...
A1
(actual plan)
describes
captured by
work

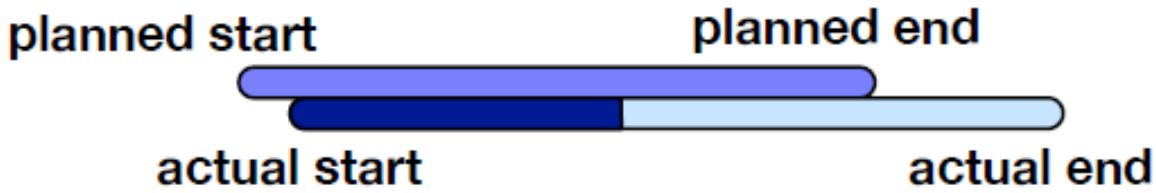
Abordări

- Concentrare:
  - Aici ne concentram pe planificare, costuri și progres
- Abordare neintegrată
  - Monitorizarea planificării → a înțelege dacă suntem înainte sau în urmă
  - Monitorizarea costurilor → a înțelege dacă suntem sub sau peste buget
  - Simple, dar viziuni parțiale
- Abordare integrata
  - Earned Value Analysis → metodă ce măsoară împreună planificarea, costurile și progresul
  - Mai complexă, dar viziune mai cuprinzătoare

Monitorizarea planificării

Concepte de bază

- Baza (valori planificate):
  - Un instantaneu al planului la un moment dat (planul la t1, planul la t2, …)
  - Se pot produce multe astfel de instantanee
- Valori reale
  - Statusul real al planificării
  - Începutul real, sfârșitul real, efortul/progresul real



Procesul

1. Construirea planului
2. Stabilirea unei baze
3. Evaluarea planului în mod regulat:
    1. Începutul real și sfârșitul real al unei activități
    2. Efortul real consumat pe activitate
    3. Progresul tehnic (poate fi dificil de evaluat)
4. Replanificare:
    1. Estimarea efortului și a duratei până la sfârșit
    2. Tehnica 1 → eficiență prin care efortul real a fost exprimat cu privire la efortul planificat
    3. Tehnica 2 → eficiență prin care progresul tehnic a fost exprimat cu privire la efortul planificat
    4. Împărtășirea noului plan și avansare la punctul 2

Colectarea datelor de efort

- Depinde de nivelul de formalitate; oamenii pot fi nevoiți să furnizeze date despre efortul consumat pe activități
- De obicei indicat să fie realizat saptămânal
- Activitățile planului trebuie sa aibă referințe
- Va conține și zgomot

John Doe	W1	W2	W3	W4
Requirements M1	30		6	
Requirements M2		30	6	
Meeting	2	2	6	2
Research	4	4	4	4
Indirect activities				2

Monitorizarea costurilor

Controlul costului abordarea simplă

- Tabelul de buget definește baza
- Costurile reale definesc statusul curent
- Poate fi împărțit de-a lungul anilor (sau a perioadelor de raportare)

CBS Item	Budgeted	Actual	Status	New Budget
Hardware	€10,000.00	€5,000.00	€5,000.00	€5,000.00
Software	€4,000.00	€2,000.00	€2,000.00	€2,000.00
Travel	€5,000.00	€6,000.00	-€1,000.00	**€1,000.00**
Project Bfr	€3,000.00		€3,000.00	**€1,000.00**
Total	€22,000.00	€13,000.00	€9,000.00	€9,000.00

Depășirile sunt rezolvate din alte fonduri sau din alte proiecte

Remarci

Avantaje

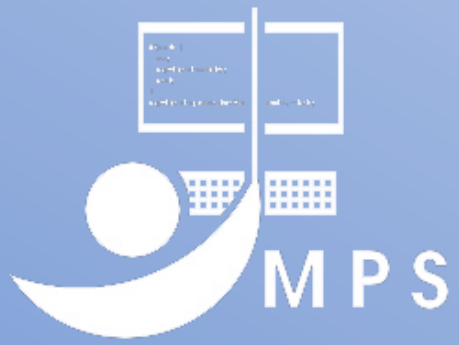
- Relativ simplu (totuși, există întârzieri între realizarea cheltuielilor și fluxurile de bani)
- Pentru anumite elemente, probabil cea mai bună metodă de monitorizare (ex. hardware, software etc.)

Dezavantaje

- Nu este suficient pentru a avea o idee despre statusul global al proiectului (va fi dus la bun sfârșit cu banii rămași?)

Earned Value Analysis

Analiza Valorii Câștigate



Earned Value Analysis

- **Earned Value Analysis** este o metodă ce furnizează o viziune de ansamblu asupra proiectului, măsurând **efortul planificat (costuri), progresul real (valoarea câștigată) și efortul (costurile actuale)** în materie de **valori monetare**
- Măsurarea planului, a muncii și a progresului folosind aceeasi unitate le face comparabile
- Util pentru că:
  - Progresul devine comparabil cu efortul
  - Bugetul și costurile actuale sunt puse în context (a fi sub buget nu este întotdeauna bine, dacă progresul tehnic este și el scăzut)

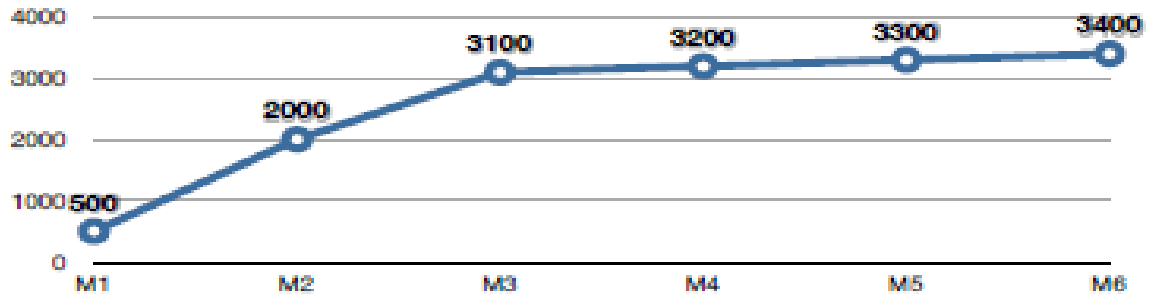
Presupuneri și definiții

- Presupuneri:
  - **Forță de muncă = Cost** → reprezentările grafice ale efortului sau costului sunt echivalente
  - Corolar: Forța de muncă reală = Costul real
  - **Progres = Bani**
- Definitii:
  - **Valoarea Planificată** → costurile cumulate planificate pentru proiect; numite de asemenea: **Costuri Bugetate ale Muncii Planificate**
  - Costuri Reale → costurile cumulate ce sunt suportate în realitate; numite de asemenea Costurile Reale ale Muncii Efectuate
  - **Valoarea Câștigată** → progresul real, exprimat drept cantitatea din valoarea planificată ce a produs rezultate

Calculul costului planificat

Name	Planned Cost	Time
A	1000	A
B	2000	B
C	400	C

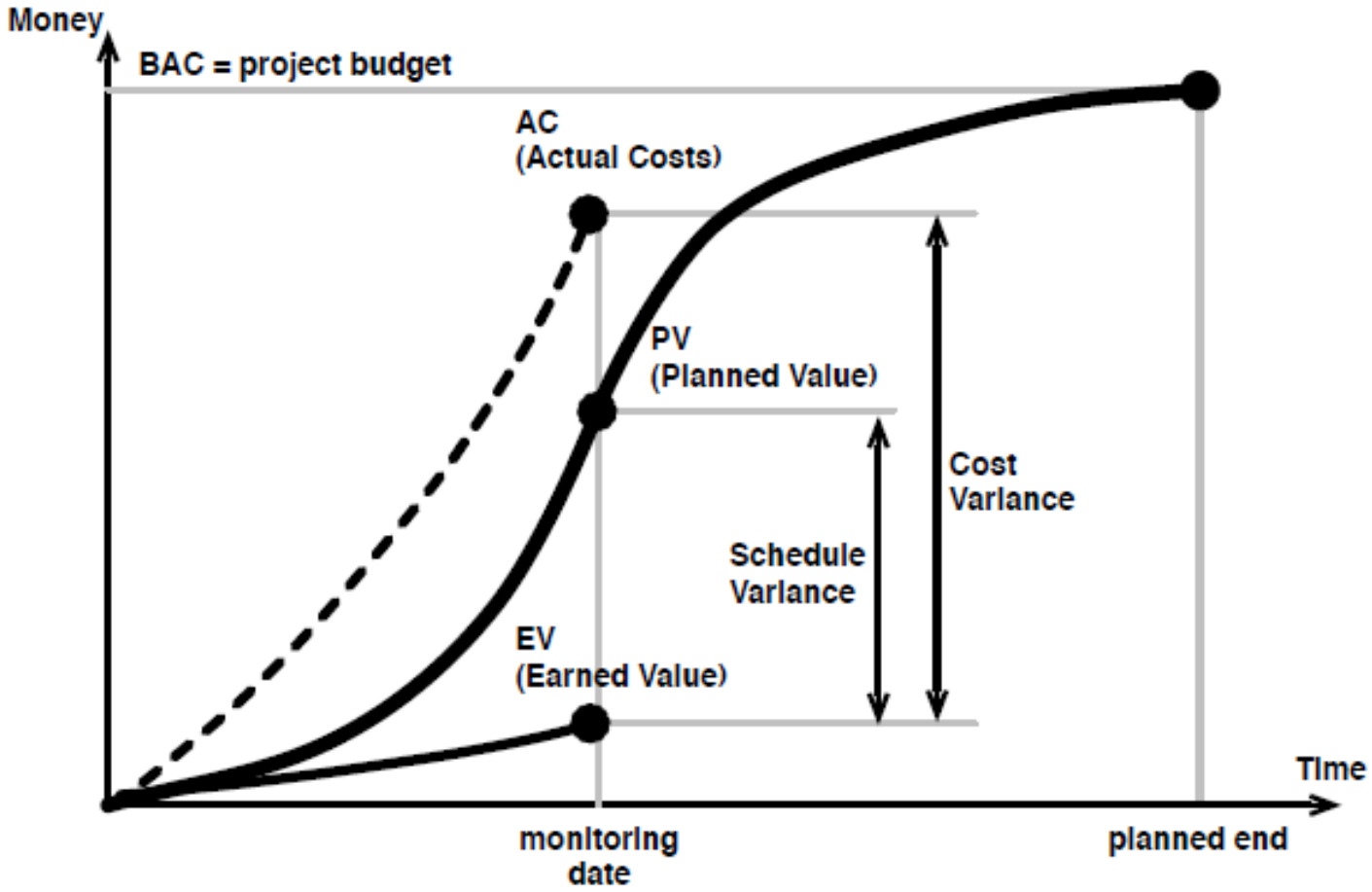
A	1000	500	500			
B	2000		1000	1000		
C	400			100	100	100	100
Total		500	1500	1100	100	100	100
Cumulative		500	2000	3100	3200	3300	3400



Calculul valorii câștigate

- Regula 1:
  - Valoarea câștigată ar trebui determinată prin examinarea produselor
- Regula 2:
  - **Regula 50/50** (50% din valoarea planificată la început, 50% la sfârșit)
  - **Regula 20/80** (20% la început și 80% la sfârșit)
  - **Regula 0/100** (0% la început și 100% la sfârșit)

Earned Value Analysis



Puncte și metrici interesante

- BAC = Budget at Completion (Bugetul la Completarea proiectului)
- SV: Schedule Variance (Varianța Programului)
  - O comparație între cantitatea de muncă realizată într-o anumită perioadă de timp și ce era planificat să fie realizat
  - O variantă negativă înseamnă că proiectul este în urma programului
- CV: Cost Variance (Varianta Costului)
  - O comparație între costul bugetat și cel real folosit pentru munca produsă
  - O variantă negativă înseamnă că proiectul depășește bugetul

Cost Performance Index (CPI)

- CPI (Cost Performance Index) compară valoarea câștigată cu totalul costurilor reale
- Cât câștigăm pentru fiecare Euro pe care îl cheltuim?

$$CPI_t = \frac{EV_t}{AC_t}$$

CPI > 1	Proiectul este eficient
CPI < 1	Proiectul este ineficient

Schedule Performance Index (SPI)

- SPI (Schedule Performance Index) compară munca realizată cu cea planificată
- Cât de repede progresează proiectul raportat la cât de repede ne-am fi așteptat noi?

$$SPI_t = \frac{EV_t}{PV_t}$$

SPI > 1	Proiectul este în fața planificării
SPI < 1	Proiectul este în urma planificării

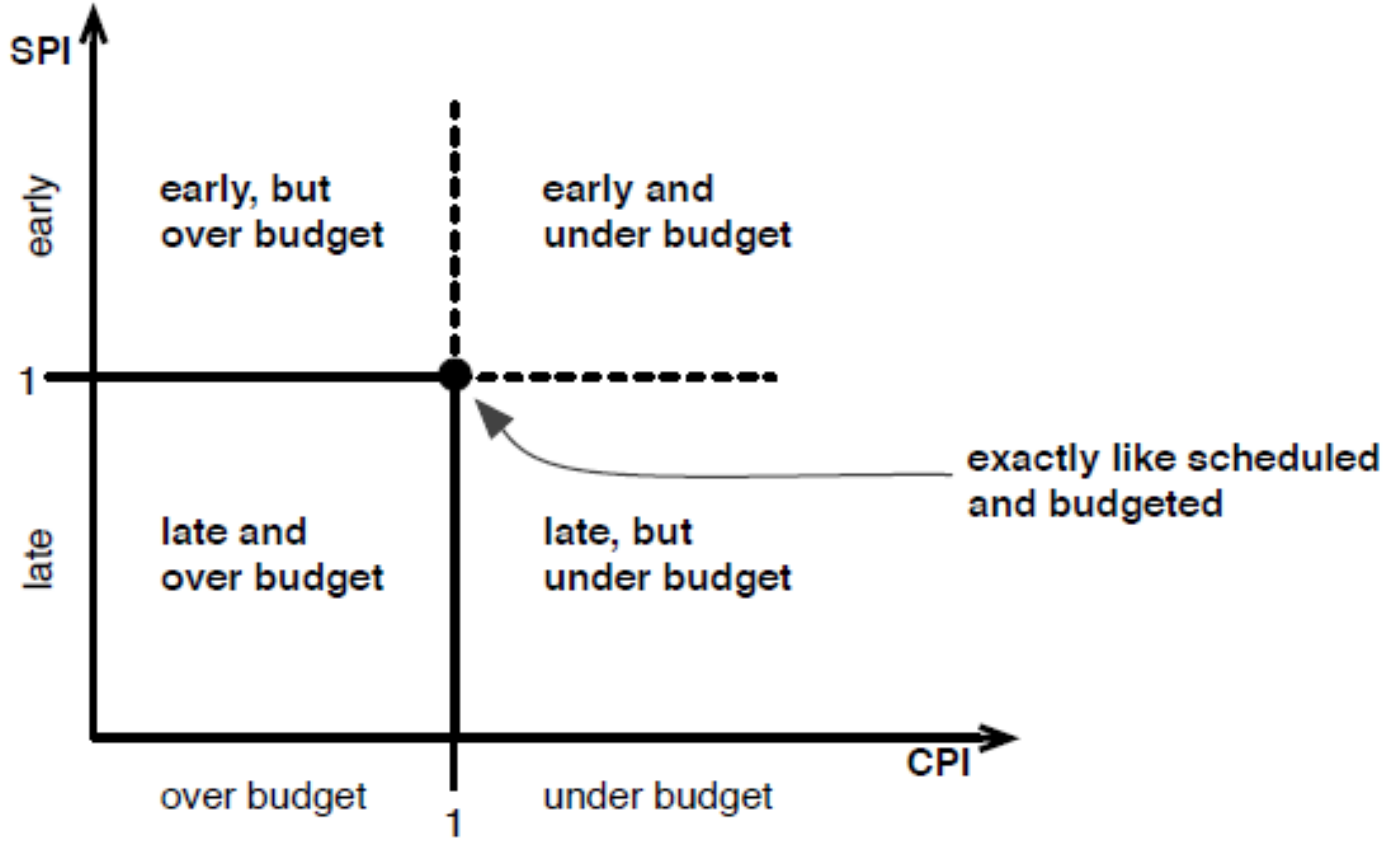
Cost Schedule Index (CSI)

- CSI (Cost Schedule Index) → CSI = CPI x SPI
- Cu cât este mai departe CSI de 1.0, cu atât devine mai puțin probabilă recuperarea proiectului

$$CSI_t = CPI_t * SPI_t$$

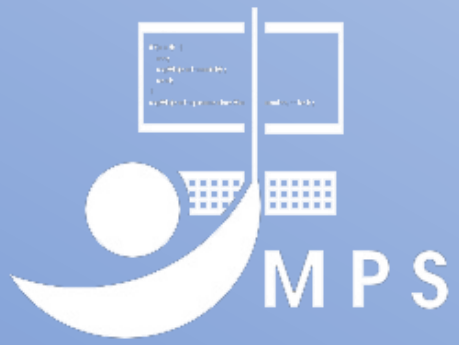
$$CSI_t = \frac{EV_t}{AC_t} * \frac{EV_t}{PV_t}$$

Masurarea indicatorilor SPI și CPI



Earned Value Analysis

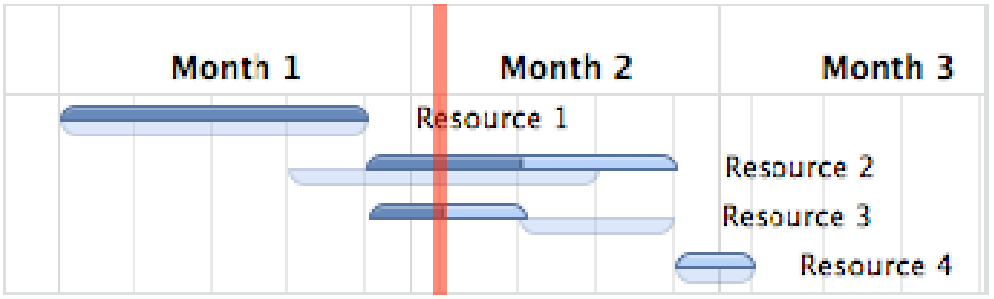
Exemplu



Exemplu

- Remarci:
  - Partea de jos → baza; partea de sus → real & progres
- Întrebări:
  - Suntem în întârziere?
  - Am depășit bugetul?

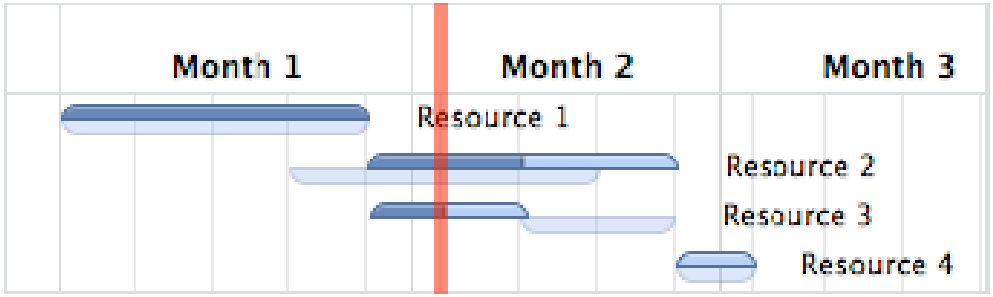
Task	Effort	Resources Cost	Task Cost
1) Paint Wall	4w	€3,200.00	€3,600.00
2) Paint Ceiling	4w	€3,200.00	€2,000.00
3) Refurnish	2w	€800.00	€400.00
4) Clean	1w	€400.00	



Exemplu

- Activitate 1: conform planificării (la timp)
- Activitate 2: începută târziu; în fața planificării
- Activitate 3: începuta devreme; progresul acelașii cu timpul trecut
- Activitate 4: neînceputa încă

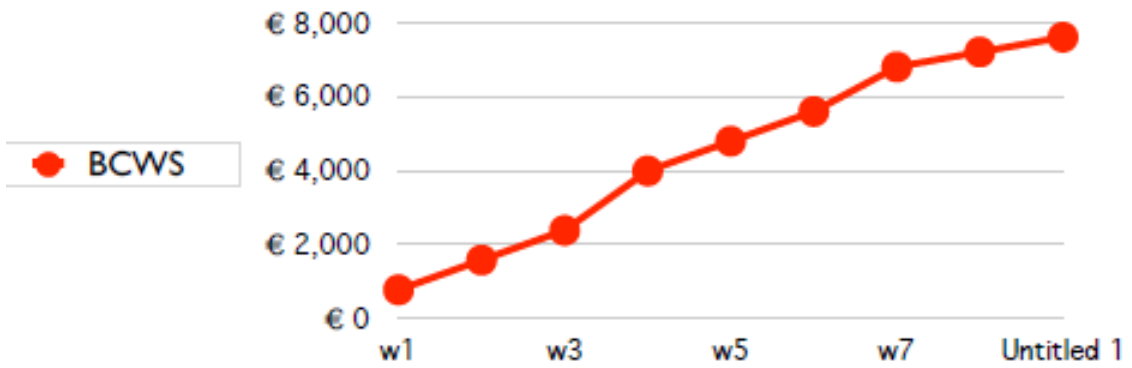
Task	Effort	Resources Cost	Task Cost
1) Paint Wall	4w	€3,200.00	€3,600.00
2) Paint Ceiling	4w	€3,200.00	€2,000.00
3) Refurnish	2w	€800.00	€400.00
4) Clean	1w	€400.00	



Exemplu - Valoarea Planificată (BCWS)

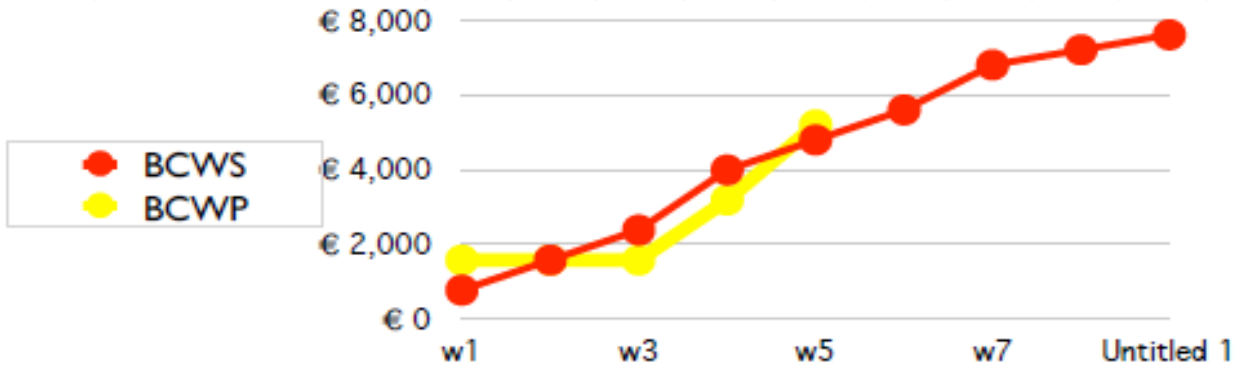
Task	Effort	Resources Cost	Task Cost
• 1) Paint Wall	4w	€3,200.00	€3,600.00
• 2) Paint Ceiling	4w	€3,200.00	€2,000.00
• 3) Refurnish	2w	€800.00	€400.00
• 4) Clean	1w	€400.00	

Paint Wall	800	800	800	800					
Paint Ceiling				800	800	800	800		
Refurnish							400	400	
Clean									400
Budgeted Cost of Work Scheduled (BCWS)	800	1600	2400	4000	4800	5600	6800	7200	7600



Exemplu - Valoarea Câștigată (BCWP)

Paint Wall	1600			1600					
Paint Ceiling				0	1600				
Refurnish					400				
Clean									
Budgeted Cost of Work Scheduled (BCWS)	1600	1600	1600	3200	5200				

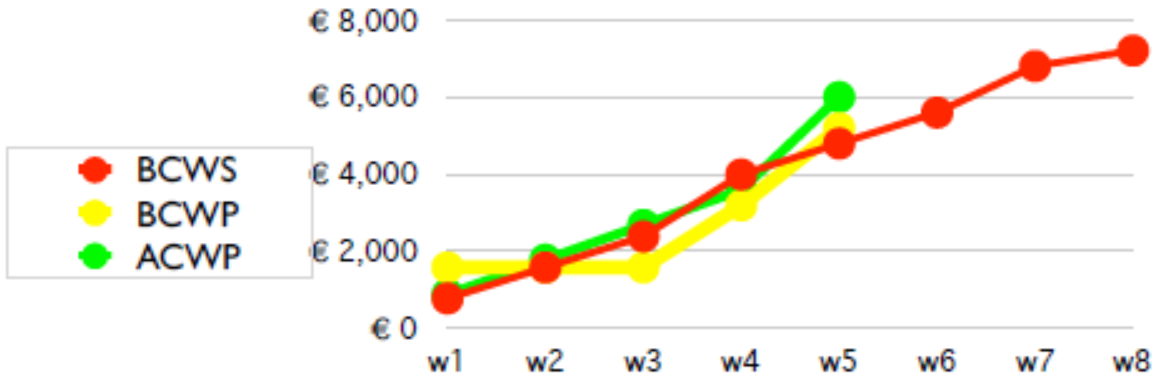


Comentarii

- In fața planificării în săptămâna 1, din cauza zgomotului regulii 50%-50% (analog pentru întârzierile din săptămânile 2 și 3)
- Săptămâna 4: suntem în urma planificării (activitatea 2 nu a început așa cum ne așteptam)
- Săptămâna 5: suntem din nou în fața planificării, din cauza activității 3
- Intrucât regula 50%-50% ia în calcul doar începutul și sfârșitul activităților, faptul că progresul din activitatea 2 este mai bun decât ne asteptam nu este luat în calcul în grafic.

Exemplu - Costurile Reale (ACWP)

Paint Wall	900	900	900	900					
Paint Ceiling				0	2000				
Refurnish					400				
Clean									
Actual Cost of Work Performed (ACWP)	900	1800	2700	3600	6000				



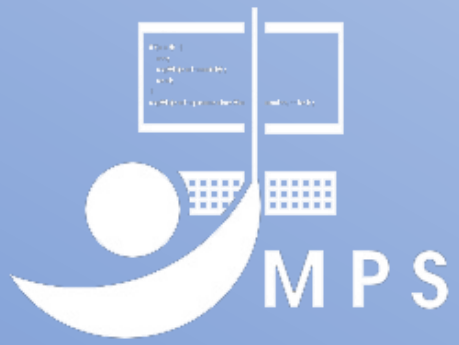
Recapitulare

Paint Wall	€800.00	€800.00	€800.00	€800.00	
Paint Ceiling				€800.00	€800.00
Refurnish					
Clean					
Budgeted Cost of Work Scheduled (BCWS)	€800	€1,600	€2,400	€4,000	€4,800
Budgeted Cost of Work Performed (BCWP)	€1,600	€1,600	€1,600	€3,200	€5,200
Actual Cost of Work Performed (ACWS)	€900	€1,800	€2,700	€3,600	€6,000
CPI (BCWP/ACWP)	178%	89%	59%	89%	87%
SPI (BCWP/BCWS)	200%	100%	67%	80%	108%
Cost Variance	€700	-€200	-€1,100	-€400	-€800
Schedule Variance	€800	€0	-€800	-€800	€400

Comentarii

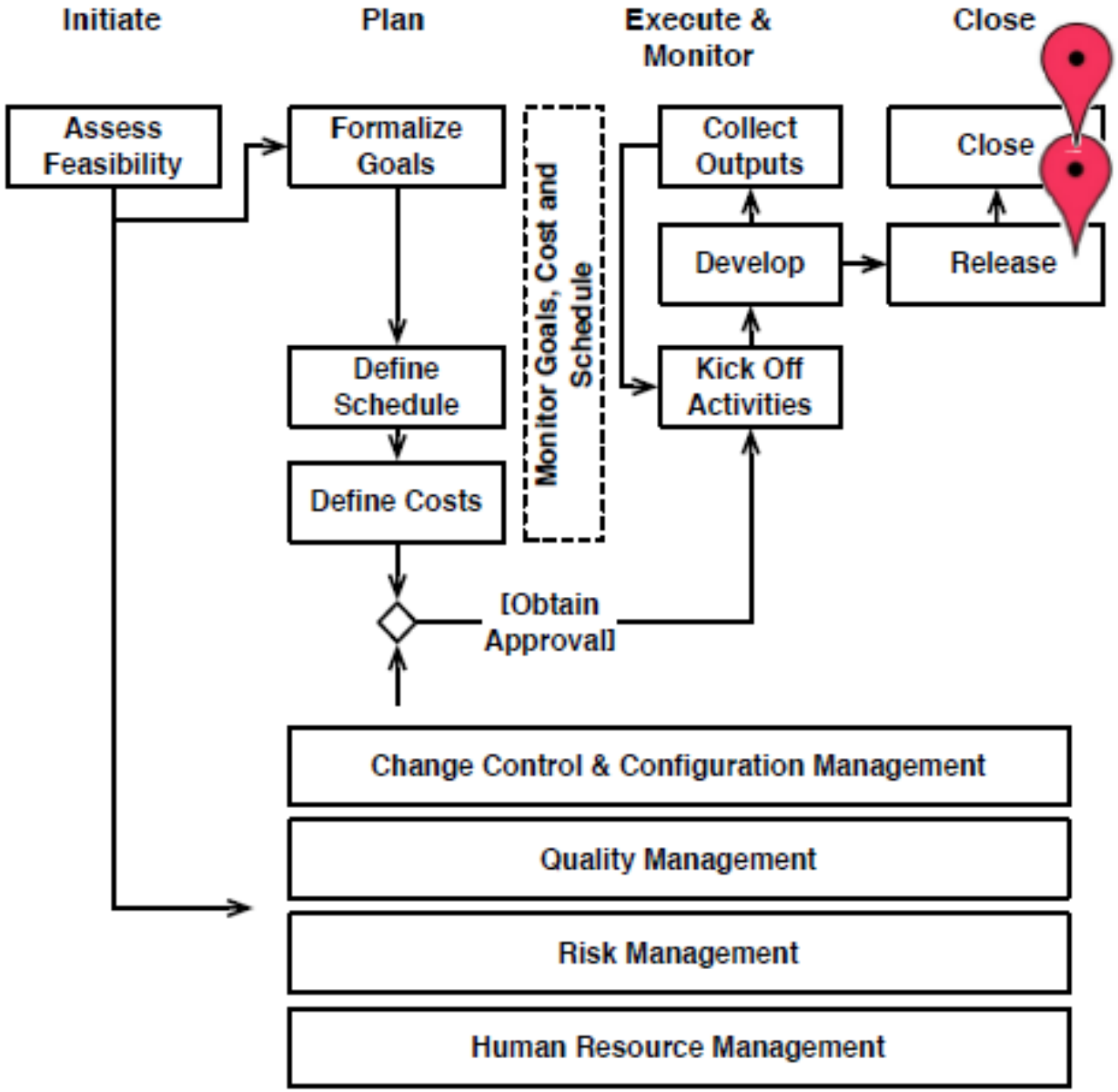
- Zgomot din cauza regulii 50%-50% (ex. săptămâna 1)
- Datele arată că noi suntem acum peste buget, dar devreme în planificare (ultima coloana)
- Totuși:
  - Eficiența reală a costurilor este, din cauza regulii 50%-50%, în activitatea 2 (am avut o creștere de 1600); datele vor fi mai exacte atunci când vom termina activitatea 2 (cheltuielile vor fi cel mai probabil 4000 Euro și BCWS 3200)

Finalizarea proiectului



Obiective

- Toate proiectele ajung la un sfârșit
- Multe dintre ele, totuși, trăiesc în uitare pentru o lungă perioadă de timp, nefiind active, dar nici închise (finalizate)
- Scopul unui bun manager de proiect este să se asigure că proiectele au fost finalizate cum trebuie
- Obiectivele acestui capitol:
  - Înțelegerea activităților ce trebuie efectuate pentru a închide un proiect
  - Înțelegerea motivelor pentru care proiectele nu sunt finalizate cum trebuie
  - Înțelegerea riscurilor apărute dacă un proiect nu este finalizat cum trebuie


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Tipuri de finalizari de proiect

- **Terminare prin intergrare și terminare prin adăugare**

Cazurile de succes: rezultatele proiectului integrate și/sau folosite ca intrări pentru alt proiect/producție

- **Terminare prin înfometare**

Proiectul se termină pentru ca s-au terminat resursele

- **Terminare prin extincție**

Proiectul terminat de către conducere, deoarece proiectul a eșuat (obiectivele neîndeplinite, înlocuit, neprofitabil)

Finalizarea proiectului

Finalizarea proiectului reprezintă ultima fază a unui proiect, în care ieșirile proiectului sunt trimise părților implicate, acordurile contractuale sunt gestionate cum se cuvine, iar registrele proiectului sunt stocate pentru referințe viitoare

- Scopuri principale:
  - Asigurarea posibilitatii utilizării ieșirilor proiectului
  - Asigurarea că nu există alte obligații viitoare / în așteptare
  - Învățare și creare de stocuri

De ce proiectele nu sunt finalizate cum trebuie

- Pentru proiectele care nu au succes:
  - Lipsă de interes din partea echipei
- Pentru toate proiectele:
  - Scăderea interesului din partea echipei
  - Costul realizării activităților necesare pentru finalizare
  - Activitățile de finalizare necesită puțină (sau chiar deloc) creativitate
  - Subestimarea cantității de cunoștințe implicite prezente
  - Subestimarea rapidității de pierdere a cunoștințelor
  - Repulsia eliberării resurselor din motive oportuniste
  - Factorul emoțional

Procesul de finalizare a proiectului

- Obținerea acceptului din partea clientului
- Instalarea livrabilelor proiectului
- Arhivarea livrabilelor vechi
- Documentarea proiectului
- Realizarea unei închideri definitive
- Realizarea auditului post-implementare
- Eliberarea staff-ului

Obținerea acceptului din partea clientului

- Acceptul ceremonial
  - Fără proceduri formale sau înregistrări formale pentru acceptarea livrabilelor proiectului
  - Scenarii: gentlemen agreement; atingerea deadline-urilor proiectului
- Acceptul formal
  - Procedură formală de acceptarea a livrabilelor proiectului
  - Testarea sistemului / aprobarea clientului

Audit post-implementare (post-mortem)

- Ne displace să repetăm aceleași greșeli
- Scopul acestui audit este de a analiza critic proiectul pentru a învăța și a îmbunătăți, evitând să facem aceleași greșeli
- Diferite formate și niveluri de formalitate sunt posibile
- Proiectele care nu au succes ne aduc o mulțime de informații
- Lecții utile și din proiectele de succes (ce a funcționat, ce am fi putut face mai bine)

Structura unui post-mortem

- **Conducerea de sondaje de proiect**

Evidențierea principalelor puncte tari și puncte slabe ale proiectului

- **Colectarea informației obiective**

Evidențierea măsurătorilor cantitative ale proiectului

- **Conducerea unei întâlniri pentru a extrage rezultatele**
- **Conducerea unei întâlniri de “istorie a proiectului”**

Găsirea cauzelor problemelor

- **Publicarea rezultatelor**

Asigurarea ca noi, organizația și echipa putem învăța din această experiență

Metrici post-mortem

- O evaluare cantitativă permite o interpretare mai precisă a proiectului
- Datele pot fi folosite si pentru estimări viitoare
- Metricile includ:

Metricile de Cost	Metricile de Planificare	Metricile de Calitate
Efortul Planificat și SLOC Estimat Efortul Real și SLOC Real Istoria schimbărilor cerințelor și a codului	Planificare Originală Planificare Finală Istoria evenimentelor ce au deraiat de la planificare	Erori în fiecare stadiu

Structura rezultatelor post-mortem

- Rezultatele unui audit post-mortem sunt publicate într-un document
- Acest document poate fi folosit pentru a disemina lecția învățată și poate fi folosit drept referință pentru viitoarele proiecte similare
- Un raport post-mortem poate fi organizat cum urmează:
  - Descrierea proiectului: informatii despre proiect, pentru a crea un context
  - Partea bună: ce a funcționat bine
  - Partea rea: cei mai rai 3 factori care au împiedicat echipele să atingă obiectivele
  - Partea urată: o recomandare pentru îmbunătățire

Eliberarea staff-ului

- Tranziția către noi activități poate dăuna echipei (să considerăm, de exemplu, că un proiect a lucrat ani de zile)
- Două aspecte importante:
  - Asigurarea recunoașterii corespunzătoare a experienței câștigate în timpul proiectului și a rezultatelor obținute
  - Asigurarea că sarcinile potrivite sunt atribuite membrilor echipei

Managementul schimbării

Obiective

- Importanța păstrării proiectului în același scop
- Cum cererile pot influența pozitiv sau negativ proiectul
- Tehnici de a gestiona schimbările

Cere schimbări și schimbările vor apărea în proiectul tău.

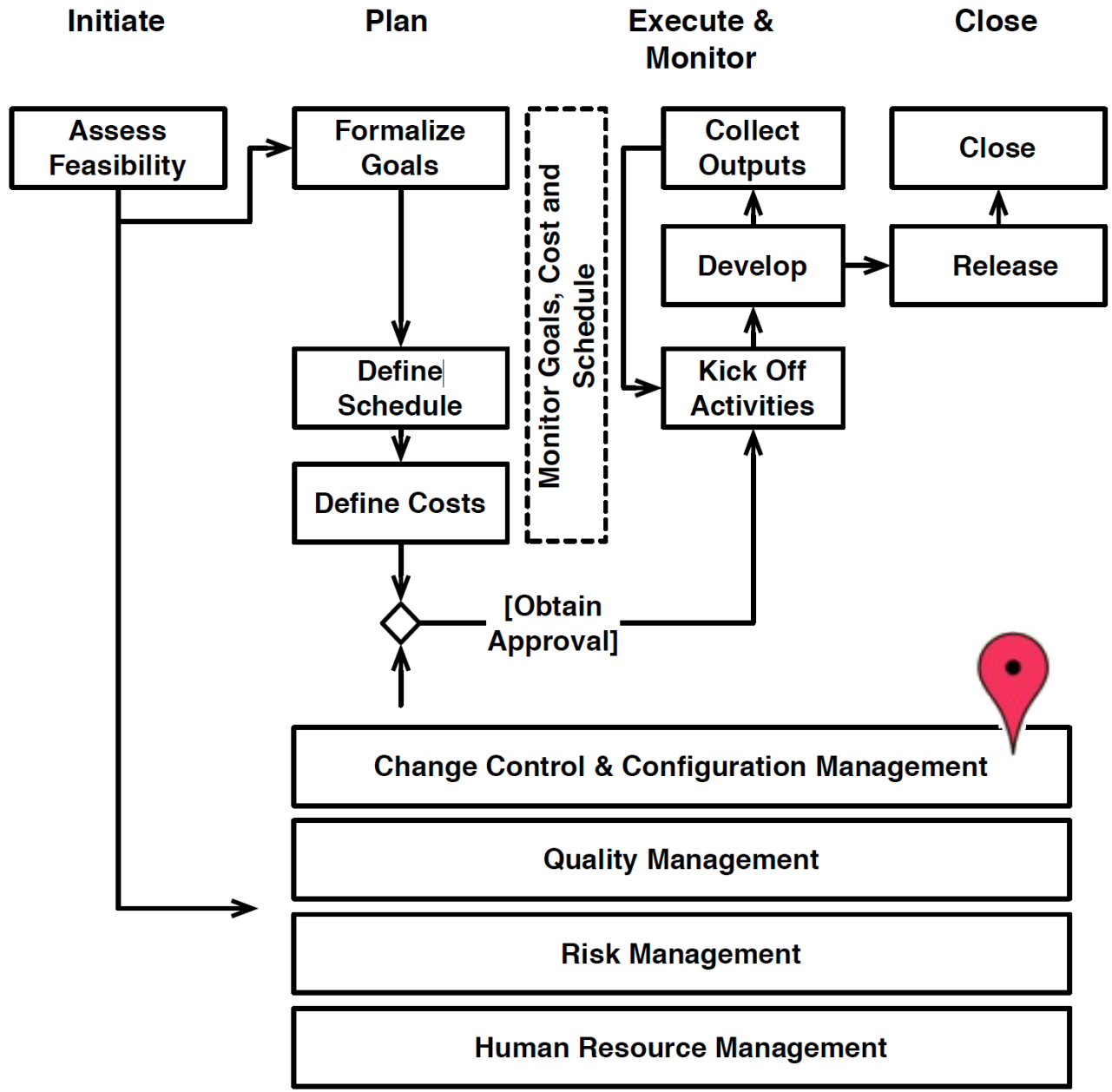
Framework

- Document formal ce cuprinde obiectivele proiectului
- Odată ce obiectivele sunt stabilite, atenția se va muta pe design-ul și implementarea pe faze ale proiectului, mentinând o continuă ajustare a acestuia
- Orice decizie ce alterează cursul acțiunii stabilite este o pertubație ce schimbă obiectivele, planurile, costurile, rezultatele, volumul de lucru etc.

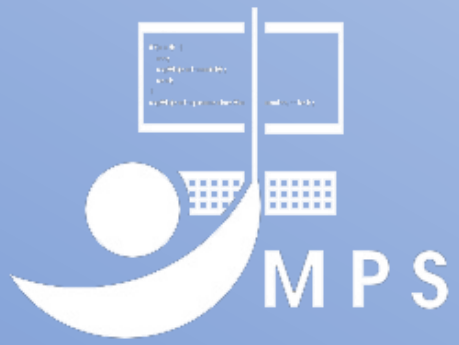
Concepte Fundamentale

- Change Control → un ansamblu de măsuri care să asigure că cererile de schimbare sunt tratate corespunzător

- Configuration Management → un ansamblu de măsuri care să asigure că obiectivul proiectului rămâne la fel de-a lungul timpului

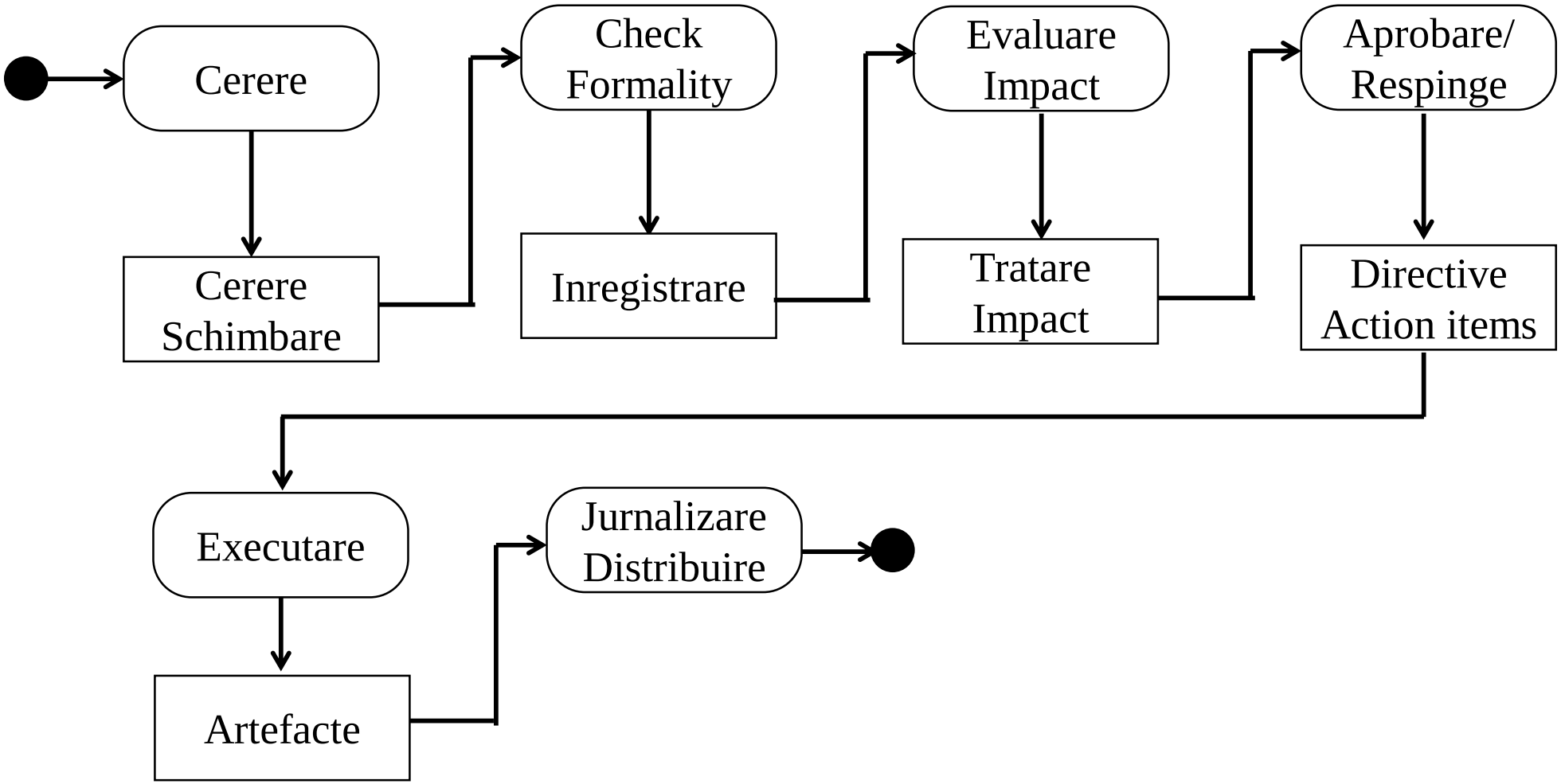

Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
[Obtain Approval]
Collect Outputs
Develop
Kick Off Activities
Close
Release
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Change Control



Cauze cerere schimbare

- Inconsistența a cerintelor de proiect
- O mai bună dezvoltare a sistemului utilizat
- O oportunitate tehnică
- O provocare tehnică
- O modificare a mediului exterior
- Nerespectarea livrării proiectului


Cerere
Cerere Schimbare
Check Formality
Inregistrare
Evaluare Impact
Tratare Impact
Aprobare/ Respinge
Directive Action items
Executare
Artefacte
Jurnalizare Distribuire

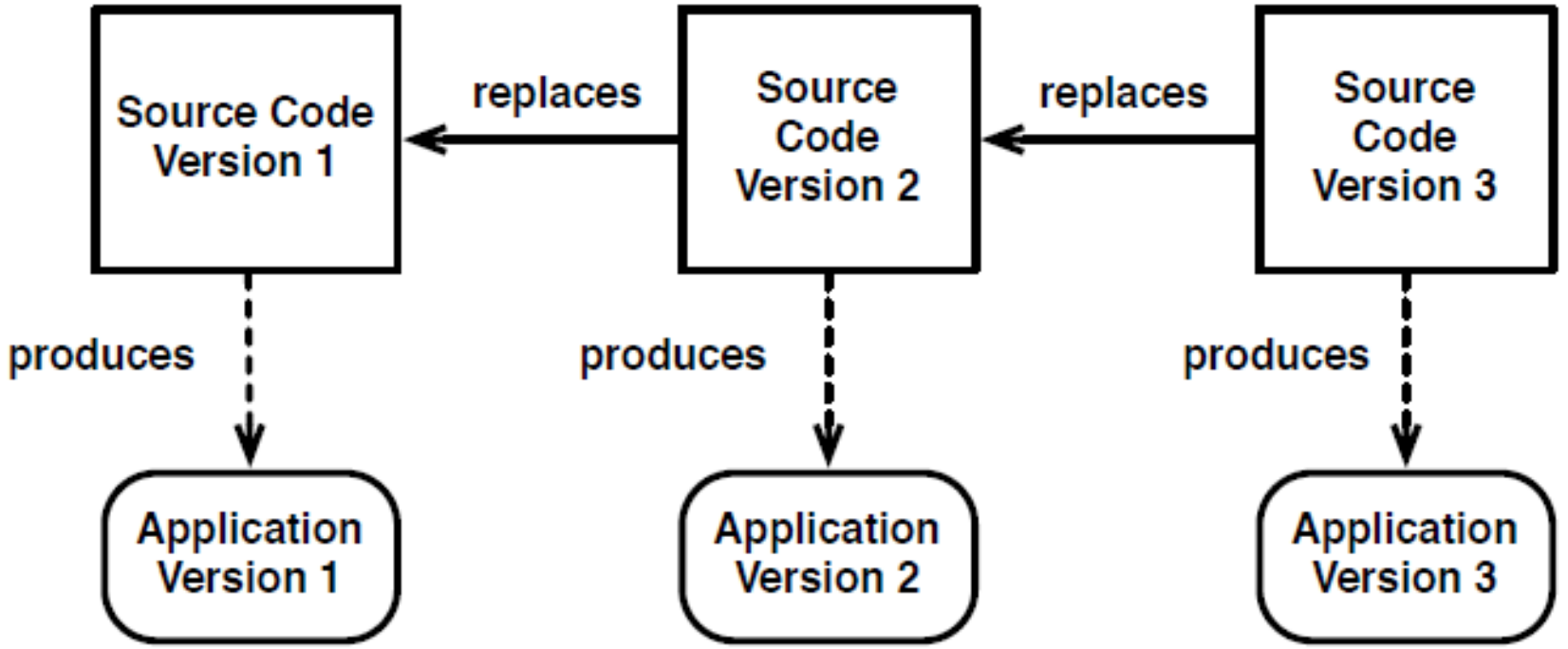
Comentarii

- Poate să existe un comitet executiv care să aprobe/respingă cereri
- Costurile și riscurile cresc odată cu înaintarea proiectului
- Procesul asigură existența unui jurnal și o procedură clară de pentru evaluarea impactului schimbării
- Schimbările și schimbările manageriale sunt controlate de tehnici agile (schimbările sunt considerate cerințe)

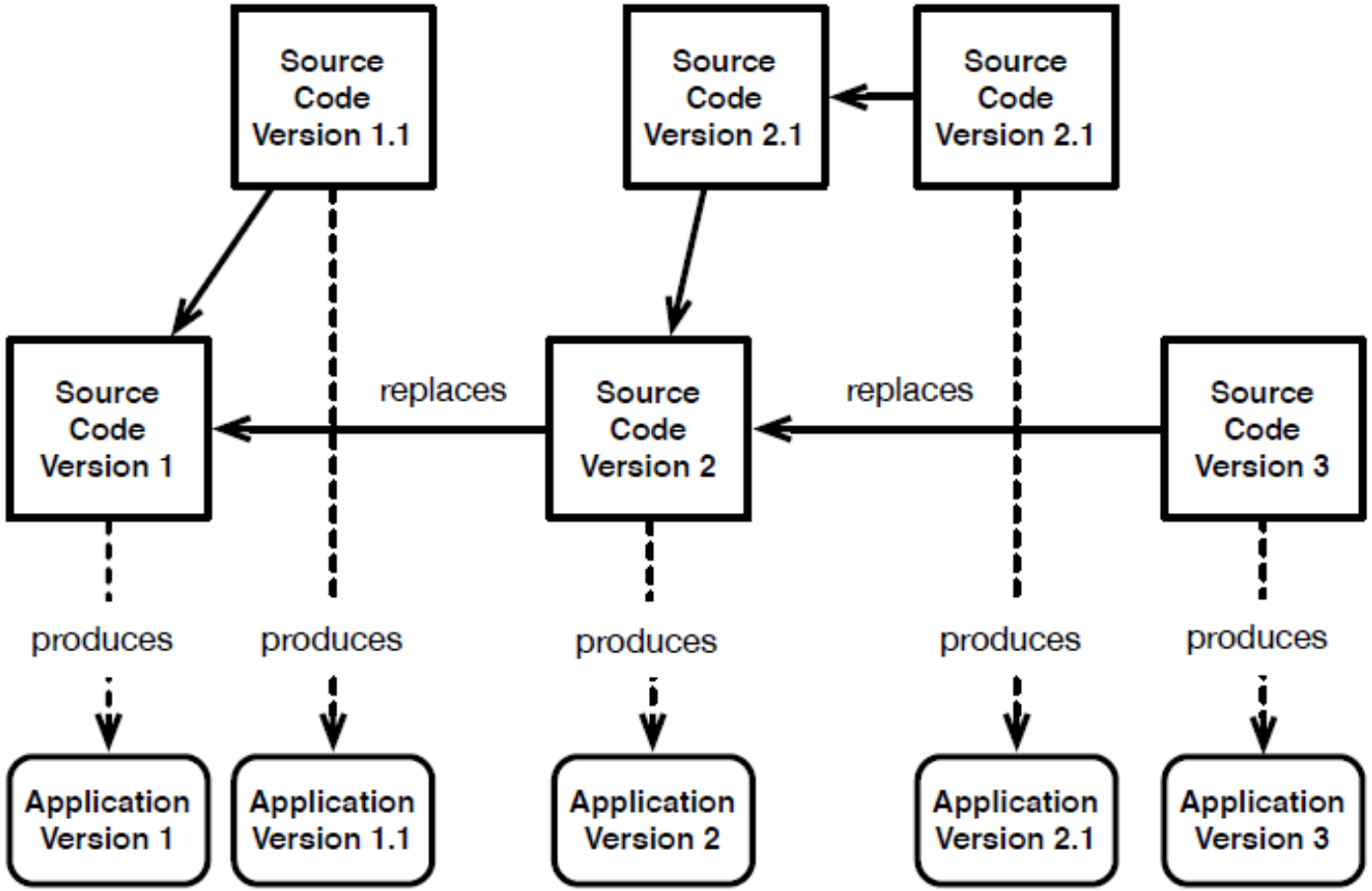
Evoluția modelelor software

- Sistemele software sunt alcătuite din mai multe artefacte:
  - Surse
  - Biblioteci
  - Biblioteci externe
  - Documentație
  - Scripturi
  - Baze de date etc.
- Sistemele software pot rula în mai multe configurații
- Două elemente cheie de complexitate trebuie adresate:
  - Identificarea artefactelor
  - Dezvoltare

Model de dezvoltare liniar



Branching Development Model



Comentarii

- Model de dezvoltare linear
  - Doar o singură versiune a aplicației poate rula la un anumit moment de tip

- Branching Development Model
  - Mai multe versiuni ale aplicației pot rula la un anumit moment de tip

Management de configurare

Management de configurare (CM)

Management de configurare – Configuration Management → Ansamblu de activități rulate în paralel pe procese de dezvoltare, al caror obiectiv este acela de a stabili și de a menține coerența sistemului de-a lungul timpului

Obiective principale - CM

- Construirea unui sistem dintr-un set consistent de componente
- Abilitatea de a retrage o anumită componentă software
- Abilitatea de a vizualiza istoricul modificărilor aduse sistemului
- Abilitatea de a face backup la o versiune anterioară

Remarcă: o legătură strânsă între procesul de managementul schimbării

Exemple

- Un bug este raportat de un utilizator pentru o aplicație care este pe piață de 10 ani
- Un client cere un upgrade a unui sistem inactiv ce a fost vândut în 2005
- E necesar să reproducem / înțelegem un comportament neobișnuit la un software de control din cadrul unei probe exploratorii din spațiu care orbitează în jurul planetei Jupiter

Pași și intrumente – stabilirea punctului de plecare

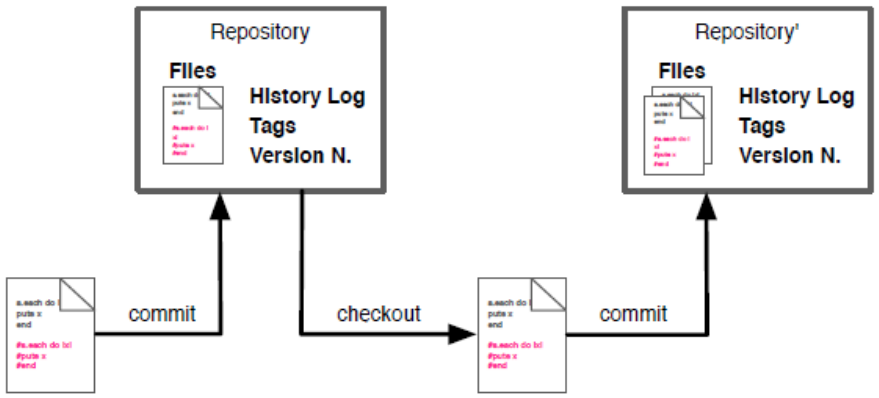
- Primul pas este acela de a stabili ce este un produs
- Un bun CM necesită:
  - Claritate în identificarea elementelor ce constituie produsul
  - Identificarea relațiilor dintre aceste elemente
  - Stabilește convenții de identificare și notare a versiunilor
  - Face snapshots → punctele de start/stop

Pași și intrumente – gestionarea schimbărilor

- Al doilea pas este păstrarea coerenței pe parcursul timpului
- Un bun CM necesită:
  - Definește punctul de start
  - Identifică și aprobă cererile de schimbare
  - Înregistrarea formală a modificărilor și menținerea unui istoric
  - Mentenanța vechilor versiuni

Version Control Systems

- Acesta implementează numeroase funcții
- Numai uneltele de proiectare nu sunt suficiente
- Concepte de baza
  - Working version → fișiere curente care sunt modificate
  - Repository → storage unde toate versiunile fișierelor sunt stocate împreună



Software Configuration Manager

Managementul riscului

Motivație

- În analiza unui proiect se ia în considerare partea financiara
- In documentul de management al domeniului am subliniat importanța realizarii obiectivelor noastre (A din SMART), dar este o mare diferență între realizabil și realizat.
- In etapa de planificare se ivesc diverse incertitudini (ex. estimare) care necesită o rezolvare generică (de ex. buffers)
- Se stabilește un plan, dar lumea este non-nominală, astfel apar modificări, atât negative, cât și pozitive

Managementul riscului

Managementul riscurilor colectează tehnici, metode cunoscute și procese care ajută la identificarea, evaluarea și monitorizarea riscurilor

- Obiectivele managementului riscurilor de proiect sunt:
  - Creșterea probabilității impactului evenimentelor positive
  - Scăderea probabilității și impactului evenimentelor nefavorabile proiectului.

Obiective

- Proiect suficient de bun pentru a fi abordat
- Rafinarea bugetului asignat
- Creșterea șanselor de a încheia proiectul cu succes:
  - în sfera de aplicare
  - de calitate
  - în bugetul alocat
  - la timp

Definiții

Traditional riscul este definit ca probabilitatea de a suferi pierderi

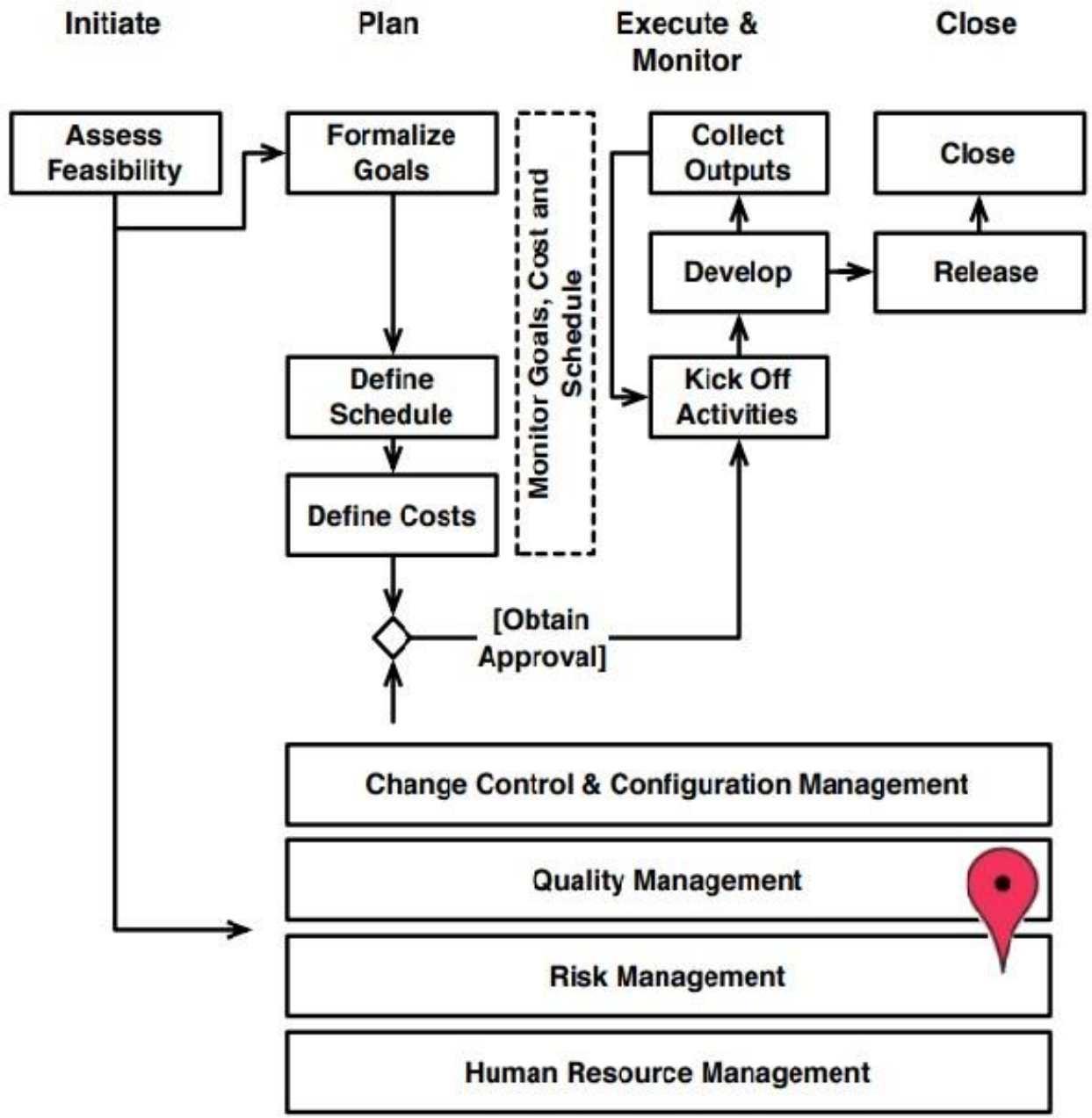
În managementul proiectelor riscul este un eveniment sau o condiție, care dacă se produce va avea un impact negativ sau pozitiv asupra proiectului

Managementul riscului

- Se folosește în mai multe domenii: finanțe, asigurări, inginerie (securitate)
- Riscul software este recunoscut de diferite standarde:
  - ISO / IEC 12207 (tehnologia informației - procesele ciclului de viață al software-ului)
  - UNI EN 29000-3 (ghid pentru aplicarea ISO 9001 la software dezvoltare și întreținere)
  - UNI ISO 10006 (Linii pentru gestionarea proiectelor)

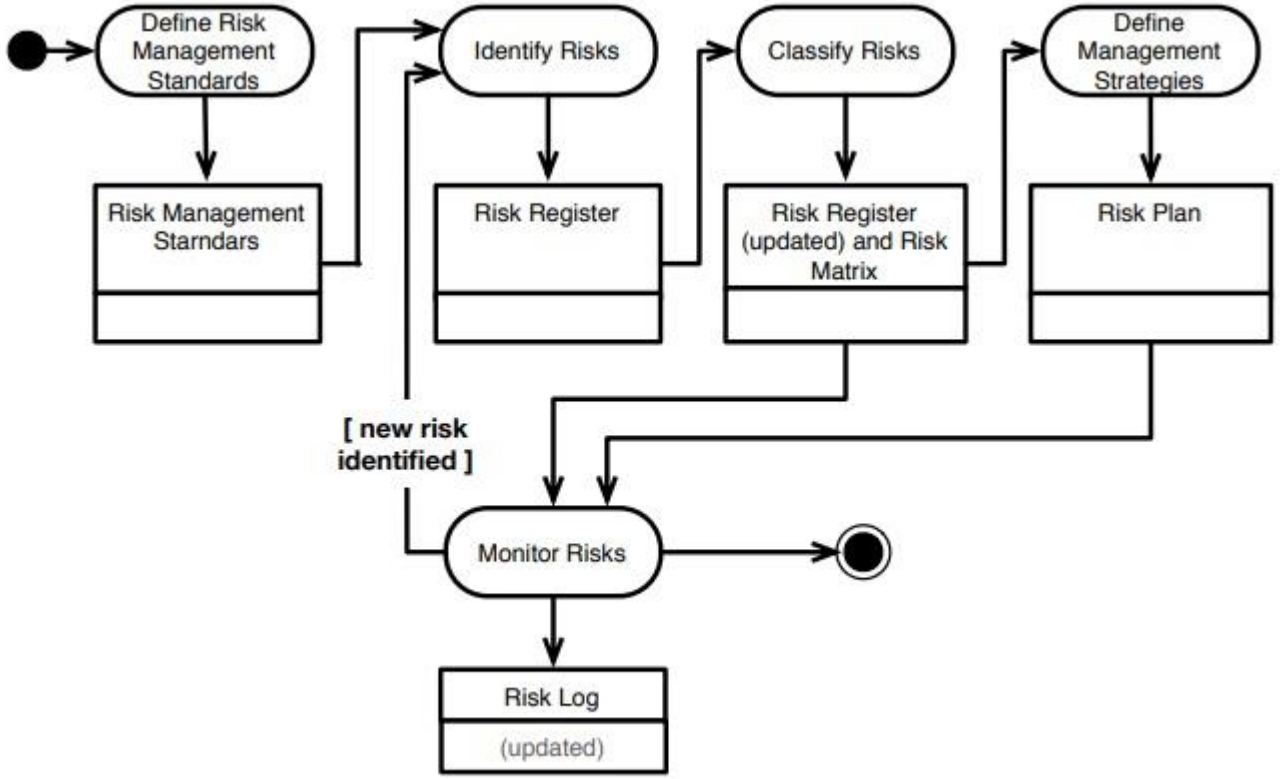
Obiectivele cursului

- Învățarea tehnicilor de identificare, evaluare, prioritizare, gestionare și controlarea riscurilor proiectului
- Învățarea celor mai frecvente riscuri în managementul de proiect
- Învățarea metodelor de protejare în condițiile apariției riscurilor


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
[Obtain Approval]
Collect Outputs
Develop
Kick Off Activities
Close
Release
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Managementul Riscurilor
Managementul Proiectelor

Procesul de gestiune al riscurilor



Standarde

Scop descrierea modului de structurare și aplicare a managementului riscurilor

- Rezultat: document (sau seturi de documente și template-uri
- Parte din planul de management
- Ajută la definirea standardelelor și bunelor practici

Definirea standardelor de MR

- Documentul include :
  - Procedurile de actualizare
  - Procedurile de aplicare a planurilor de urgență
  - Împărțirea pe task-uri și pe persoane responsabile de acestea

- La noi schimbări
  - definirea probabilităților de risc și a impactului
  - format de raport

- Planul de management trebuie standardizat și adoptat de toată organizația
- Proiecte diferite implică diferite metode de gestionare a riscurilor

Identificarea riscurilor

Scop Înțelegerea riscurilor ce pot influența proiectul și documentarea caracteristicilor acestora

- Identificare riscurilor este un proces iterativ, riscuri noi pot apărea pe parcursul proiectului
- Rezultat listarea riscurilor, analiză calitativă / cantitativă pe baza riscurilor

Identificare riscurilor și clasificare

- Proces iterativ
  - Colectare
    - Identificarea riscurilor specifice proiectului
    - Descrierea riscului
  - Analiză
    - Identificare cauzelor
    - Definirea categoriei și probabilității
- Rezultat registru de riscuri

Analiza riscurilor
Calitative vs Cantitative

Scorul riscului de 70%

Evaluare

Simplă

Riscuri cu

Evaluam riscurile folosind un set de standarde definite în planul de risc management

Riscuri ce necesită răspuns

Produsul dintre impact si probabilitate

Nu sunt suficiente informații despre probabilitățile de risc

Evaluare obiectivă

Sistematică, cere timp

Oferă cifre ale impactului economic, cuantificată în timp, bani, planificare

Evaluare numerică, Analiză matematică

Date anterioare calitative

Tool-uri: Monte Carlo Simulation, Decision Tree Analysis

Risc cu impact negativ de 1k$ asupra costurilor

Metode de identificare a riscurilor

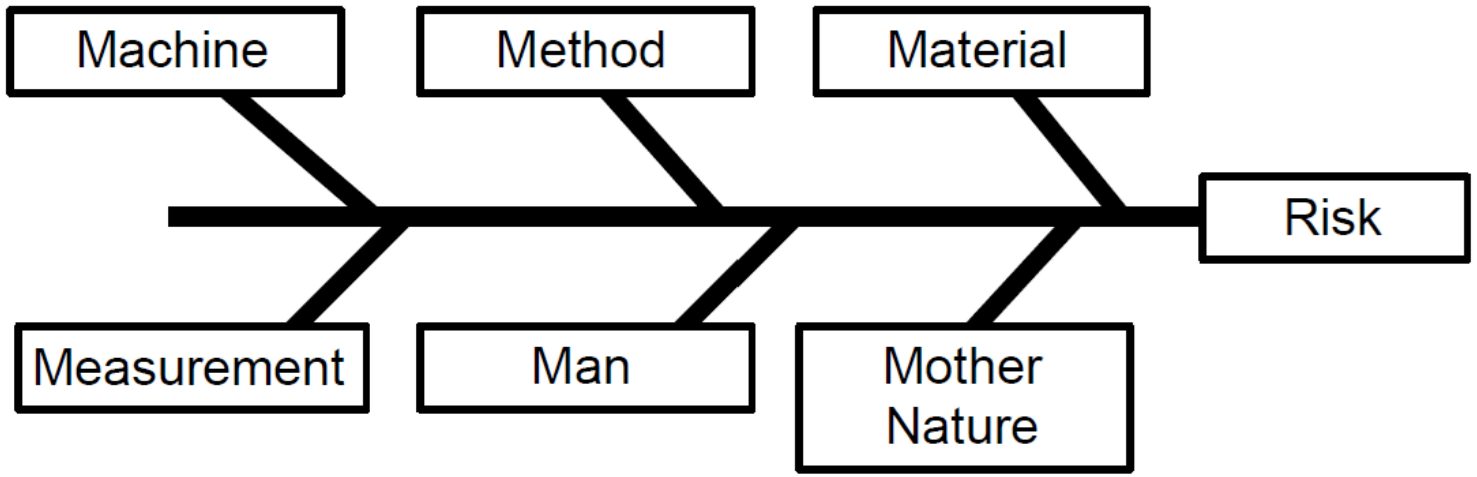
- Sedințe
- Analizarea documentelor
- Structuri de defalcare a riscurilor, liste de verificare, template-uri
- Analogii

Cele mai importante 10 cauze în eșecul unui proiect ale lui Boehm

- Scăderea personalului sau a subcontractanților
- Programul și bugetul nerealist
- Dezvoltarea unor funcții software / interfața cu utilizatorul greșite
- Gold Plating (greșirea priorităților)
- Controlul schimbării ineficient
- Riscuri tehnice

Tehnici de analiză a cauzelor de bază

- Diagrama cauza-efect (Ishikawa)
- Fault Trees / Failure Modes and Effect Analysis



Diagrame Fishbone - Puncte de pornire

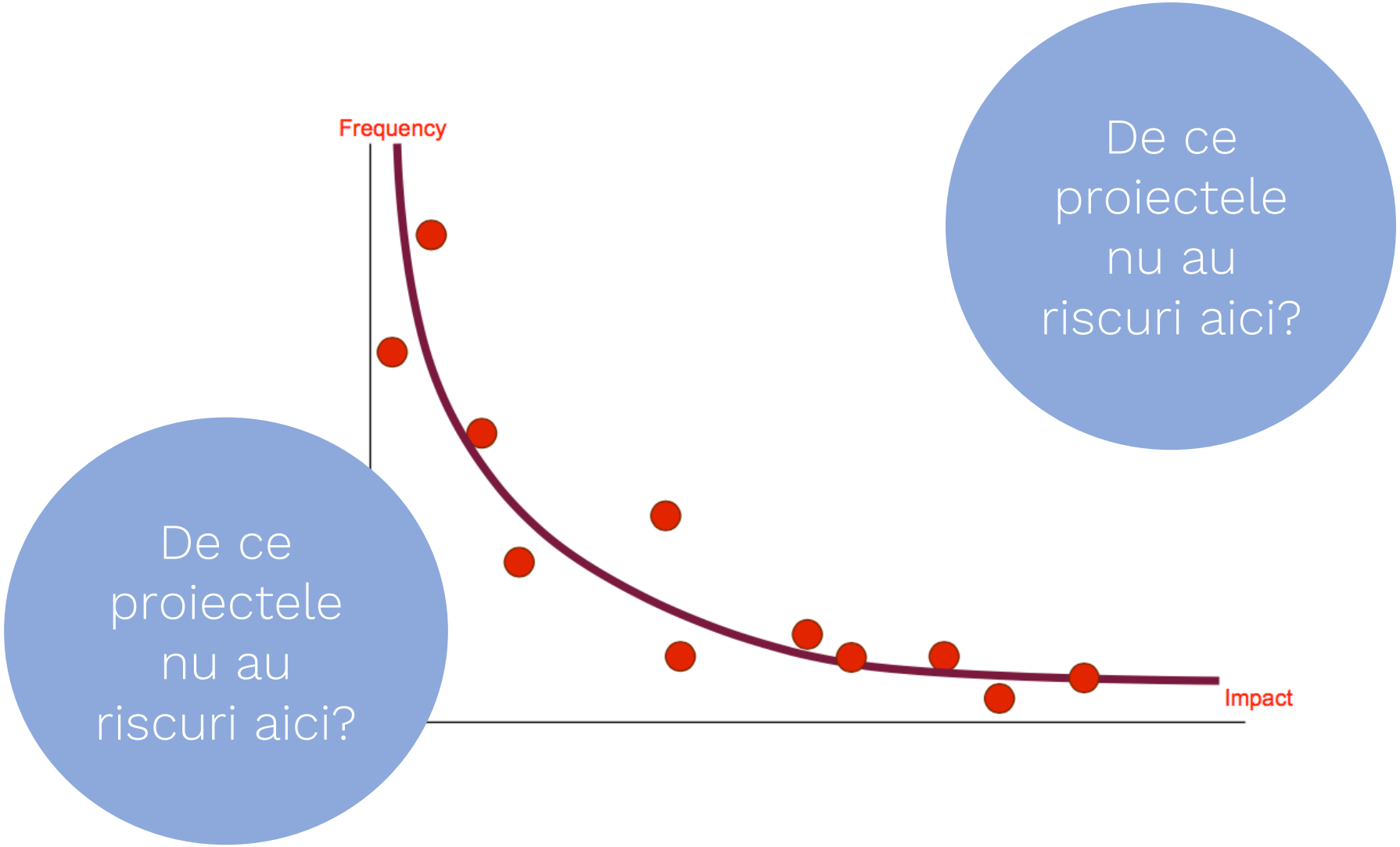
- Cei 6 M
  - Mașina, Metoda, Materialele, Măsurătorile, Man(Omul) și Mama Natura(Mediul de dezvoltare)
- Cei 8 P
  - Prețul, Promovarea, People(Oamenii), Procesele, Places(Locul), Politicile, Procedurile și Produsul
- Cei 4 S
  - Surroundings(Împrejurimi), Suppliers(Furnizori), Sisteme, Skills(Aptitudini)

Evaluarea riscurilor și strategiile de management al riscurilor

Evaluare a riscurilor

Scop prioritizați riscurile în funcție de impactul și asemănarea lor cu proiectul

Probabilitate / Impact



Tehnici

- **Analiza calitativă a riscului**
  - Mai simplu
  - Poate fi utilizată atunci când nu există informații exacte despre probabilitățile de risc
- **Analiza cantitativă a riscurilor**
  - Mai sistematică
  - Poate fi potrivită pentru analiza matematică
  - Furnizează cifre privind impactul economic al riscurilor

Analiza riscului calitativ

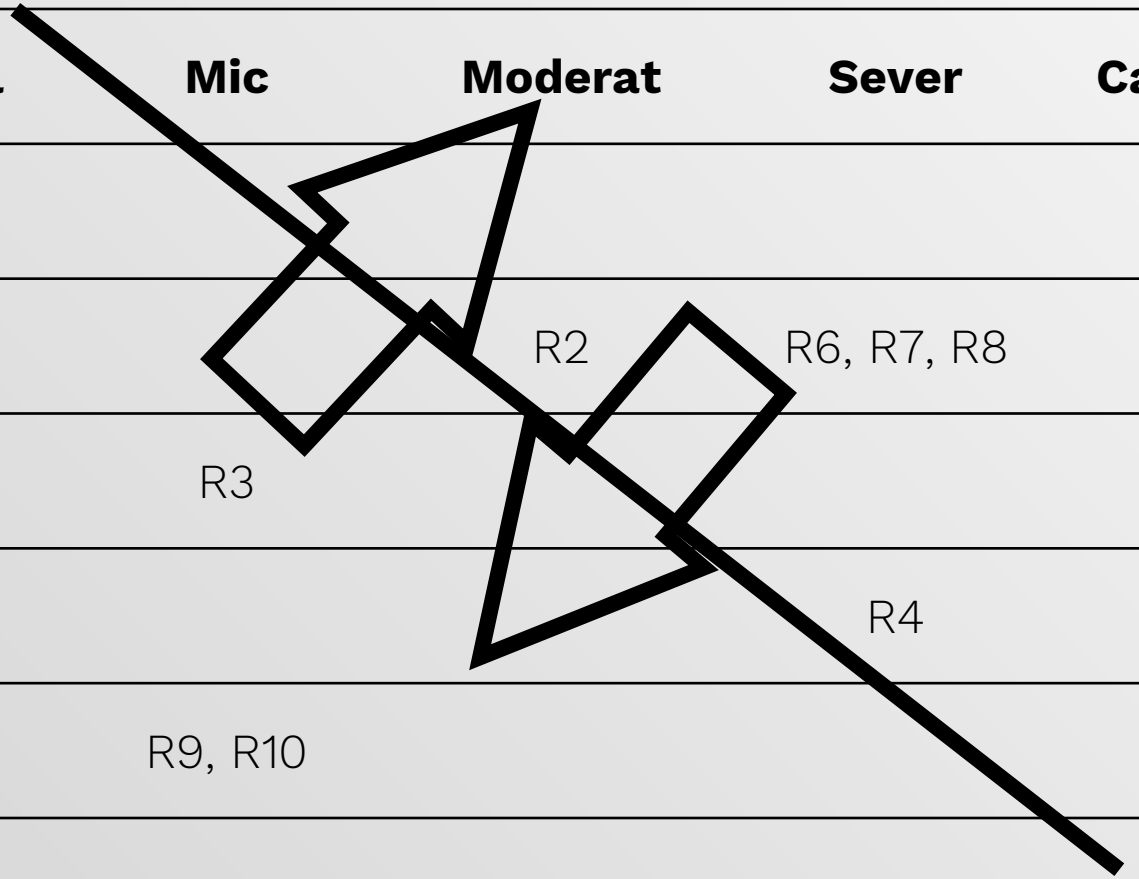
Matricea de risc

	Neglijabil	Mic	Moderat	Sever	Catastrofic
Foarte înalt	R1				R5
Înalt			R2	R6, R7, R8	
Moderat		R3			
Mic				R4	
Foarte mic		R9, R10			

Matricea de risc

Decidem unde vrem să tragem linia

	Neglijabil	Mic	Moderat	Sever	Catastrofic
Foarte înalt	R1				R5
Înalt			R2	R6, R7, R8	
Moderat		R3			
Mic				R4	
Foarte mic		R9, R10			



Matricea de risc

Roșu necesită tratament special (sau renunță la proiect)

Portocaliu Necesită monitorizare

Verde stardarul unui proiect

	Neglijabil	Mic	Moderat	Sever	Catastrofic
Foarte înalt	R1				R5
Înalt			R2	R6, R7, R8	
Moderat		R3			
Mic				R4	
Foarte mic		R9, R10			

Evaluarea riscului

- Defineste clasele de probabilități și clase de clase de impact calitative sau numerice
- Exemple
  - Probabilitate foarte scăzută, scăzută, moderată, înaltă, foarte ridicată
  - Impactul neglijabil, scăzut, moderat, sever, catastrofal
  - Scorul de risc scăzut, mediu, ridicat sau numeric: **SCORE = P x I**

Risc construit social

Două probleme cu risc calitativ:

- **Modelatori**: suntem “analfabeți în riscuri” (credem că anumite lucruri sunt mai riscante, deși statisticile spun altfel)

- **Modele**: cine spune ce sunt probabilitățile? Cum calculăm expunerile de risc în mod obiectiv?

Exemple de riscuri: Cauzele morții

- Boală de inimă: 597,689
- Cancer: 574,743
- Bolile cronice ale căilor respiratorii inferioare: 138,080
- Accident vascular cerebral (boli cerebrovasculare): 129,476
- Accidente (răni neintenționate): 120,859
- Boala Alzheimer: 83,494
- Diabet: 69,071
- Nefrită, sindrom nefrotic și nefroză: 50,476
- Gripă și pneumonie: 50,097
- Auto-vătămare intenționată (sinucidere): 38.364

Sursă: http://www.cdc.gov/nchs/fastats/lcod.htm

Strategiile managementului de risc

Strategia managementului de risc

Scop găsirea unei rezolvări pentru riscurile inacceptabile și deciderea asupra unei strategii pentru riscurile rămase, în cazul în care acestea apar pe durata proiectului

Scenariul

	Neglijabil	Mic	Moderat	Sever	Catastrofic
Foarte înalt	R1				R5
Înalt			R2	R6, R7, R8	
Moderat		R3			
Mic				R4	
Foarte mic		R9, R10			

Strategii – Amenințări

- Evitare schimbarea planului pentru a evita amenințările (mărește timpul pentru a realiza cerințele)

- Transfer deplasarea rezultatului către un terț; transferă responsabilitatea, dar nu elimină riscul (ex. asigurările)

- Diminuare reduce probabilitatea sau impactul (ex. prototipurile)

Strategii – Oportunități

- Exploatare eliminarea incertitudinii (atribuirea oamenilor talentați asigură o calitate mai bună)

- Distribuire atribuirea responsabilității de exploatare către un terț (parteneriate)

- Sporire modificarea mărimii unei oportunități prin creșterea probabilității și/sau un impact pozitiv

Strategii – Obișnuințe

- Acept
  - **Pasiv**: lăsarea echipei să se ocupe de riscuri
  - **Activ**: furnizarea unui buffer (timp, bani etc.)

  - De ce?
    - Probabilitate sau impact mic
    - Mai simplu să te ocupi de risc, în cazul in care el apare, poți planifica un răspuns în avans

Planificarea unui răspuns

- Plan:
  - Strategiile pentru tratarea riscurilor trebuie să fie concrete
  - Declanșatoare (elemente utilizate pentru a monitoriza dacă a apărut un risc)
  - Persoane responsabile de monitorizarea riscului
  - Persoanele responsabile de aplicarea planurilor pentru evenimente neprevăzute

Registrul riscurilor

- Fiecare risc este caracterizat de:
    - ID, Titlu, Descriere, Categorie
    - Probabilitate, Impact, Scor
    - Cauză de bază
    - Interval de timp
    - Modalități de monitorizare
    - Status (activ, inactiv)

Controlul si monitorizarea riscurilor

Controlul și monitorizarea riscurilor

- Input:
  - registrul de risc
- Proces:
  - analizarea abaterilor de la planul nominal
  - identificarea cazurilor
  - evaluarea corecțiilor
  - modificarea planului curent
- Gândire:
  - riscurile planificate trebuie tratate ca mai sus
  - riscurile neplanificate necesită întregul proces

Concluzii

Și principalul risc al ... managementului de risc

Erori comune

- **În timpul fazei de planificare:**
  - **Neidentificarea valorii maxime a riscului**
    * renunțarea la un proiect prea riscant
  - **Nescrierea unui plan balansat**
    * mărimea și complexitatea trebuie să fie la un nivel potrivit
  - **Interpretarea greșită a efectelor ca fiind cauze**
    * îngrijirea față de un eveniment greșit, netratarea actualei probleme

Erori comune

- **În timpul fazei de monitorizare:**

  - **Homeostazia riscului:** tendința de a ne asuma mai multe riscuri pe măsură ce proiectul devine mai sigur

  - **Ancore și cadre:** tendința de a rămâne la ancore și cadre cu privirea la oportunități sau la nevoia de a schimba cursul

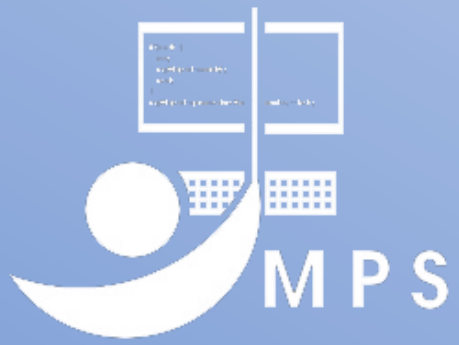
  - **Costuri de scufundare:** o ipoteza economică incorectă

  - **Disonanță cognitiva:** inconsecvențe neplăcute

Erori comune

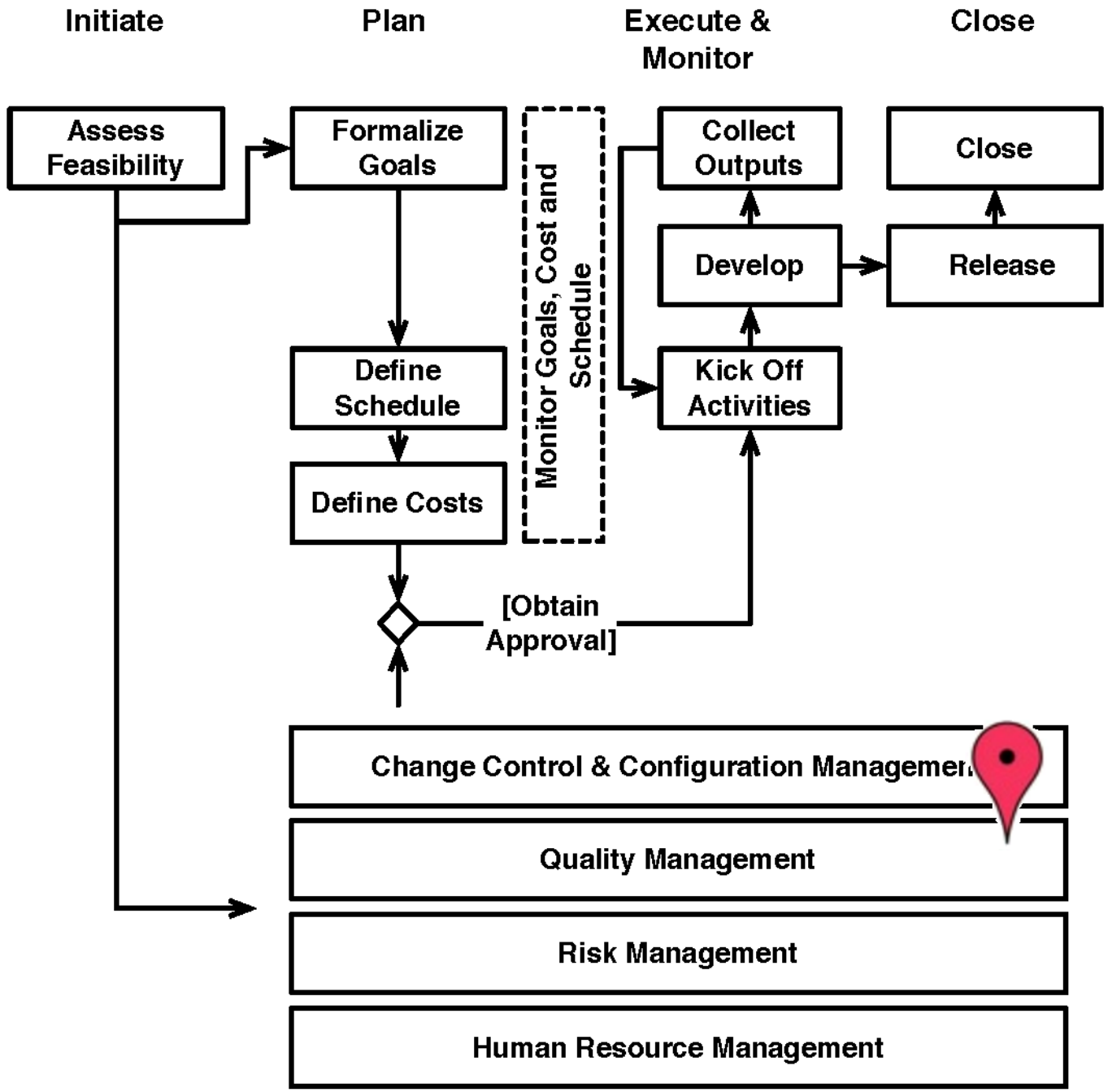
- **În timpul fazei de monitorizare:**
  - **Nu aplicați planurile de urgență** → utilizarea unor metode găsite în momentul apariției unui risc în loc de o strategie concepută în avans
  - **Nu implicați actori** → asigurați-vă că stakeholderii cunosc consecințele riscului, implicarea lor în rezolvarea lor
  - **Nu actualizați planul** → planul trebuie mereu adaptat la ultimele riscuri și condiții apărute

Managementul calității

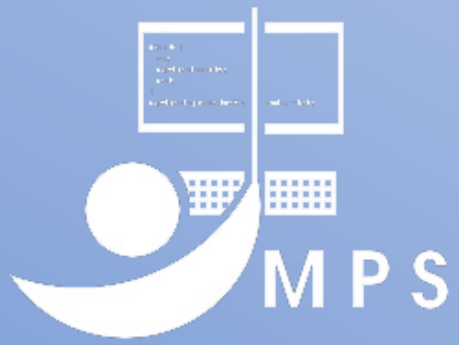


Obiective

- Înțelegerea imporatanței calității managementului în proiectele software
- Principalele tehnici pentru gestionarea calității în proiecte
- Pricipalele tehnici pentru gestionarea calității proiectelor livrabile
- Diferențele dintre testare software și managementul calității


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Asigurarea calității software



Asigurarea calității software

- Aplicarea planificată și sistematica a activităților pentru a asigura conformitatea ciclului de viata a proceselor și produselor la cerințele, standardele și procedurile cerute

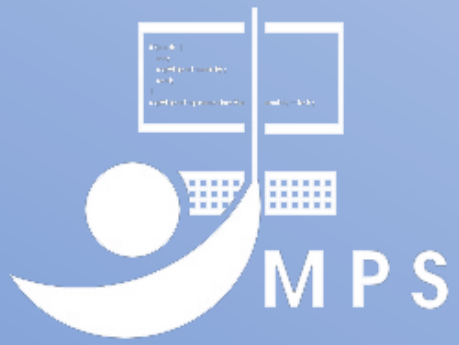
Observații

- Definiția se aplică atât proceselor cât și produselor
- Asigurarea calității este planificată și sistematică că multe alte activități
- Conformitatea este necesară, toate elemenetele caracterizând mediul software operațional

Procesul de asigurare a calității

- Planificarea calității – idenifica standardul și practicile relevante și modul în care le implementăm
- Asigurarea calității –pune accent pe asigurarea standardelor calității idenificate la pasul anterior
- Controlul calității –are rolul de a asigură că produsele respectă standardele calității identificate în etapă de planificare

Film



Planificare calitativă

- Scop: asigurarea îndeplinirii obiectivelor managementului calității într-un proiect
- Mijloace:

- Identificarea constrângerilor și obiectivelor de calitate în domeniul de aplicare
- Identificarea standardelor și procedurilor care trebuie aplicate - Identificarea tehnicilor care trebuie aplicate
- Alocarea resurselor (timp, oameni, buget) la activități de asigurare de calitate
- Roluri si responsabilități

- Output: document de planificare a asigurării calității

Observații

• Calitatea trebuie să fie echilibrată cu constrângerile celuilalt proiect (de exemplu, timp și costuri)

• Nu toate sistemele sunt la fel de critice: de exemplu, NASA distinge opt clase diferite de sisteme software

• Echipa de asigurare a calității trebuie să fie independentă de la echipa de dezvoltare

• Diferite "nivele" de independență:

- roluri diferite în proiect
- structuri diferite în cadrul organizației
- organizații independente

Asigurarea calității

- Obiectiv: asigurați-vă că proiectul aplică și urmează standarde de calitate
- Instrumentul principal: audituri de calitate
- Declanșator: timpul, etapele importante sau evenimentele critice din proiect (conform planului de calitate)
- Auditurile de calitate includ
  - Inspecții
  - Recenzii
  - Încercări
- Ieșire:
  - Principalele constatări și recomandări

Organizarea întâlnirilor de audit

• Au loc reuniuni de audit și revizuire pentru evaluarea statutului a unui produs sau a unui proiect

• Trei roluri "conflictuale" :

- Auditorii: analiza produselor și documentația proiectului
- Membrii proiectului, responsabili de furnizarea de clarificări și de explicarea alegerilor și stărilor proiectului
- Moderatorul, care asigură ordinea de zi și mediul de întâlnire rămâne productiv

Structura procesului de audit

- Definirea scopurilor și limitelor auditului
- Identificarea comitetului de audit (independență, competență, profesionalism)
- Distribuția întregului material relevant
- Pregătirea auditului de către auditori
- Întâlnire de audit
- Pregătirea raportului final

Semnele proiectelor problematice

- Potrivit datelor NASA, proiectele problematice includ:

- Schimbări frecvente ale etapelor importante

- Fluctuațiile inexplicabile ale personalului

- Întârzieri continue în livrarea de software

- Număr nerezonabil de rapoarte de neconformitate sau modificarea solicitărilor.

Controlul calității

- Scop: Asigurarea respectării standardelor de calitate propuse pentru produs în fază de planificare
- Metode folosite pentru asigurarea calității:
  - Inspecții
  - Analiză
  - Testare
- Factori declanșatori: milestone-uri sau evenimente critice (în funcție de planul de calitate)
- Rezultat: o listă cu rapoarte de neconformitate

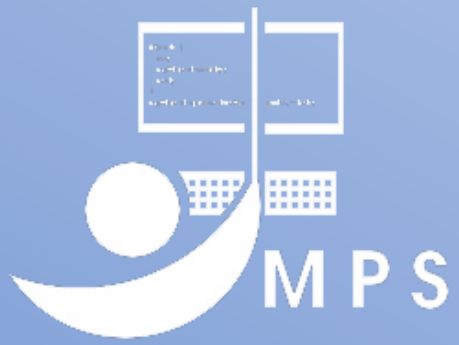
Controlul calității

- Controlul calității pentru sistemele software este greu deoarece:
  - Sistemele software se pot găsi în multe stări, deci testarea tuturor stărilor posibile este nepractică, sau chiar imposibilă
  - Mediul de operare este impredictibil
  - Discontinuitatea: orice mici schimbări în input-ul dat programului pot produce schimbări mari în output
  - Cerințele nefuncționale pot fi greu de stabilit
  - Automarea testelor poate fi foarte grea sau costisitoare (de exemplu testarea automată a unui GUI)
  - Sistemele din zilele noastre folosesc multe tehnologii diferite (HTML/CSS, Javascript, PHP, WebServer, OS)

Tehnici de control al calității pentru software

- Inspecții ale codului:
  - Sistemul este analizat de către o echipă independentă
- Analize:
  - Checkere statice care verifică folosirea corectă a diverselor construcții sintactice
  - Checkere dinamice care verifică diverse anomalii și situații suspecte executând cod instrumentat
  - Metrici ale codului sunt colectate pentru a măsură alte caracteristici de calitate (de exemplu procentul comentariilor în raport cu liniile de cod, acoperirea testelor unitare)
  - Verificarea formală (demonstrarea de teoreme, verificarea modelelor) arată proprietăți despre resprezentarea abstractă a sistemului
- Testarea:
  - Testele sunt derulate pe un sistem pentru a verifica comportamentul acestuia în diverse circumstanțe

Stabilirea unui program al metricilor



Stabilirea unui program de metrici

- Un **program de colectare a metricilor** apreciază cantitativ modul în care obiectivele proiectului sunt atinse
- Metricile procesului → măsoară diverse caracteristici ale proiectului
- Metricile produsului → măsoară diverse caracteristici ale unui produs al proiectului
- De obicei trendurile sunt mai importante decât numerele exacte
- Metricile sunt mai bune dacă sunt automatizate

Metricile unui produs - dimensiunea

- Metrici orientate pe dimensiune:
  - Numărul liniilor de cod (Source lines of code – SLOC)
  - Numărul claselor
- Metrici orientate pe funcționalitate:
  - Puncte funcționale (o “unitate de masură” care arată câtă funcționalitate oferă un sistem)
  - Puncte de obiecte (estimează efortul)
- Observații:
  - Metricile de dimensiune pot fi colectate automat
  - Numărătoarea făcută în cazul SLOC nu prea este consistentă
  - Metricile orientate pe funcționalitate necesită personal calificat care să le colecteze

Metricile unui produs - complexitatea

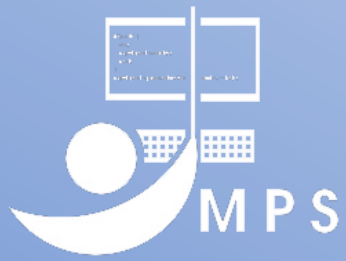
- Complexitatea ciclică
- Legăturile dintre obiecte
- Adâncimea în moștenire
- Fan-in, fan-out

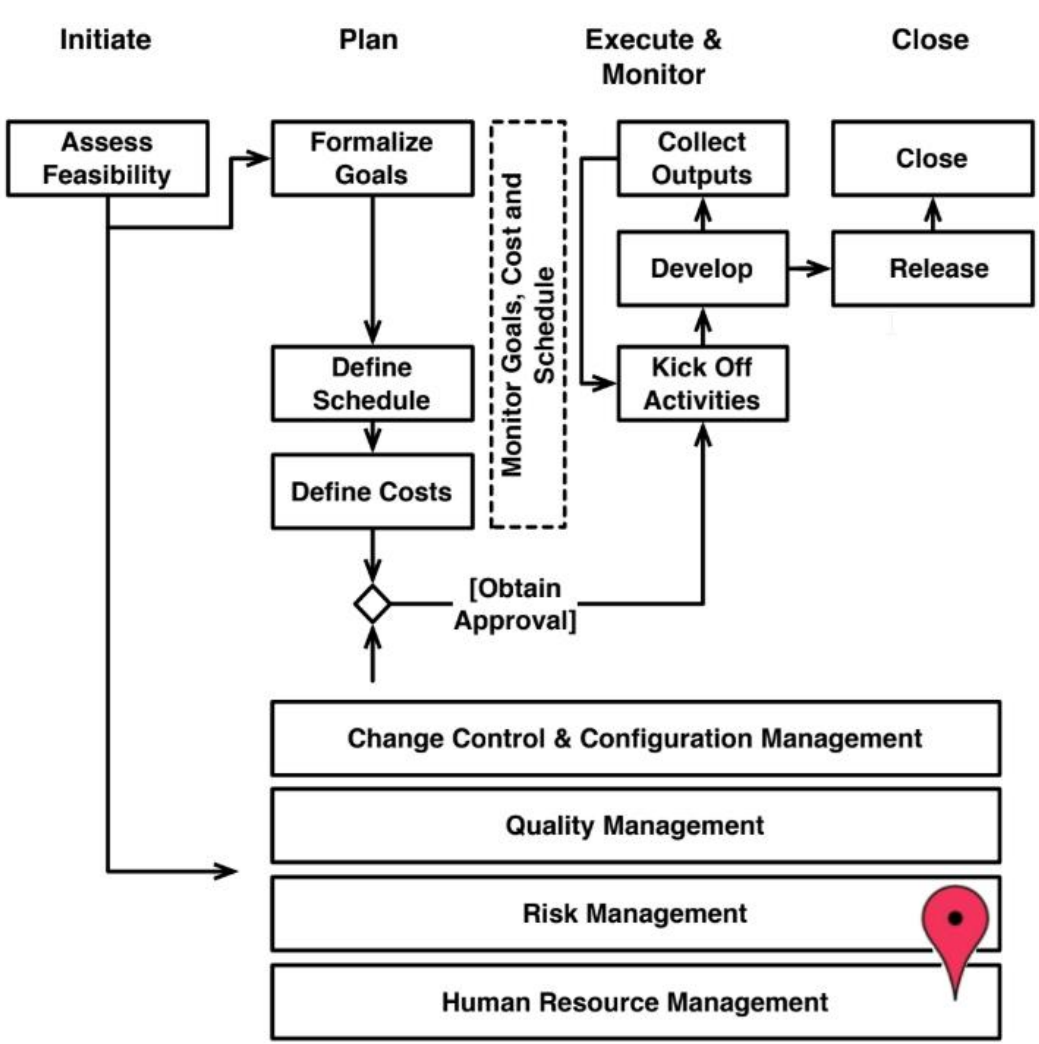
Metricile unui produs - metrici de calitate

- Raportul dintre numărul liniilor de comentarii și numărul liniilor de cod → indicativ al “mentenabilității” (maintainability) unui sistem
- Numărul de probleme care nu au fost încă rezolvate → arată dacă proiectul “converge”
- Densitatea erorilor: numărul de erori găsite în raport cu numărul de linii de cod -> ajută în a înțelege dacă în procesul de dezvoltare au apărut erori sistematice

Managementul oamenilor

Selectarea celor mai potriviți oameni și managementul acestora pentru lucrul individual și în echipă




Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Procesul de selectare a Echipei de Proiect

Pas 1 - Definirea personalului necesar

Pas 2 - Alegerea persoanelor potrivite

Pas 3 - Organizarea staff-ului

Pas 4 - Punerea pe liber a personalului

Pas 5 - Profit

Definirea personalului necesar

- Scopul → identificarea skillset-ului necesar pentru realizarea proiectului
- Acțiunea → Traducerea și împărțirea proiectului în activități și competențe
- Exemple de criterii
  - Hard/soft skills
  - Experiența
  - Implicarea
  - Recomandări de la alți angajați

Skill Matrix

Activitate	Deprindere
Design	Creativitate Experiență în Photoshop
Baza de date	Cunoștințe bune de SQL
Front end development	Experiență în HTML&CSS Experiență în JavaScript
Back end development	Cunoștințe de PHP

Alcătuirea Echipei de Proiect

- Două abordări:
  - Internă
  - Externă
- Factori care influențează alegerea:
  - Disponibilitate
  - Încărcare curentă
  - Prioritatea proiectului
  - Politicile companiei

Alcătuirea Echipei de Proiect

- Intern
  - Tehnici:
    - Preemtiv: Angajații știu dinainte de startul proiectului că vor fi asignați pentru acesta;
    - Negociere: Staff-ul este “negociat” între managerii de proiect din cadrul companiei.
- Extern
  - Procesul:
    - Definirea și scoaterea la concurs a posturilor vacante;
    - Analizarea CV-urilor + organizarea unui interviu.
  - În colaborare cu departamentul de HR

Managementul oamenilor

Premise

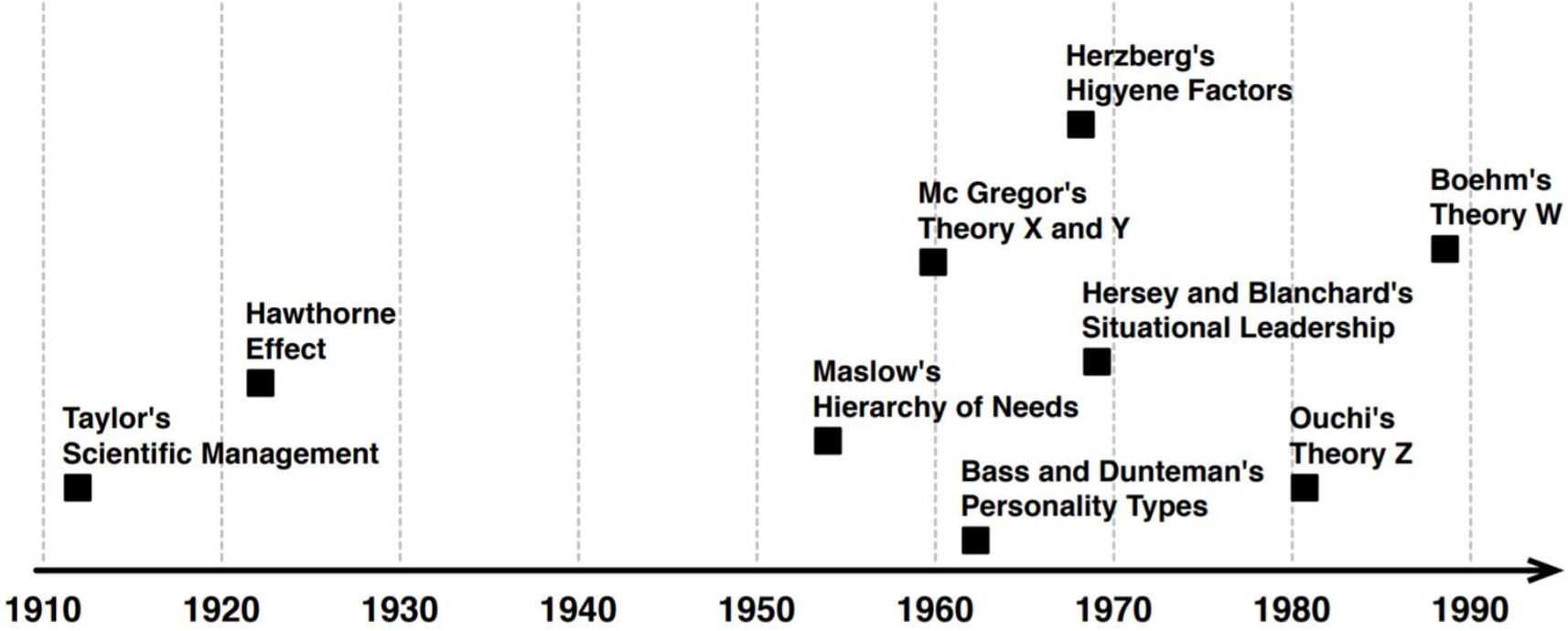
- Este dificil (dar important) de înțeles ce anume îi motivează pe oameni la locul de muncă
- Este foarte dificilă formularea unei teorii exacte, cu aplicabilitate generală

Studii principale în domeniu

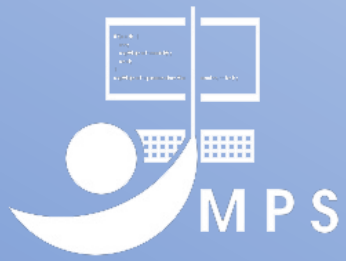
Categorii:

- Factori motivaționali
- Trăsături de personalitate și tipuri de interacțiune
- Strategii de management

Cronologie


Herzberg's
Higyene Factors
Mc Gregor's
Theory X and Y
Boehm's
Theory W
Hersey and Blanchard's
Situational Leadership
Hawthorne
Effect
Maslow's
Hierarchy of Needs
Taylor's
Scientific Management
Ouchi's
Theory Z
Bass and Dunteman's
Personality Types
1910
1920
1930
1940
1950
1960
1970
1980
1990

Factori motivaționali



Managementul științific (1911) - Taylor

- Obiective principale: îmbunătățirea **eficienței economice** și a **productivității muncii**
- Probleme expuse*:*
  - Muncitorii care trebuie să repete task-uri tind să lucreze în cel mai încet ritm care trece nepedepsit.
  - Nu te poți baza pe oameni pentru talent sau inteligență, toți muncitorii sunt motivați doar de bani.

Managementul științific (1911) - Frederick Taylor

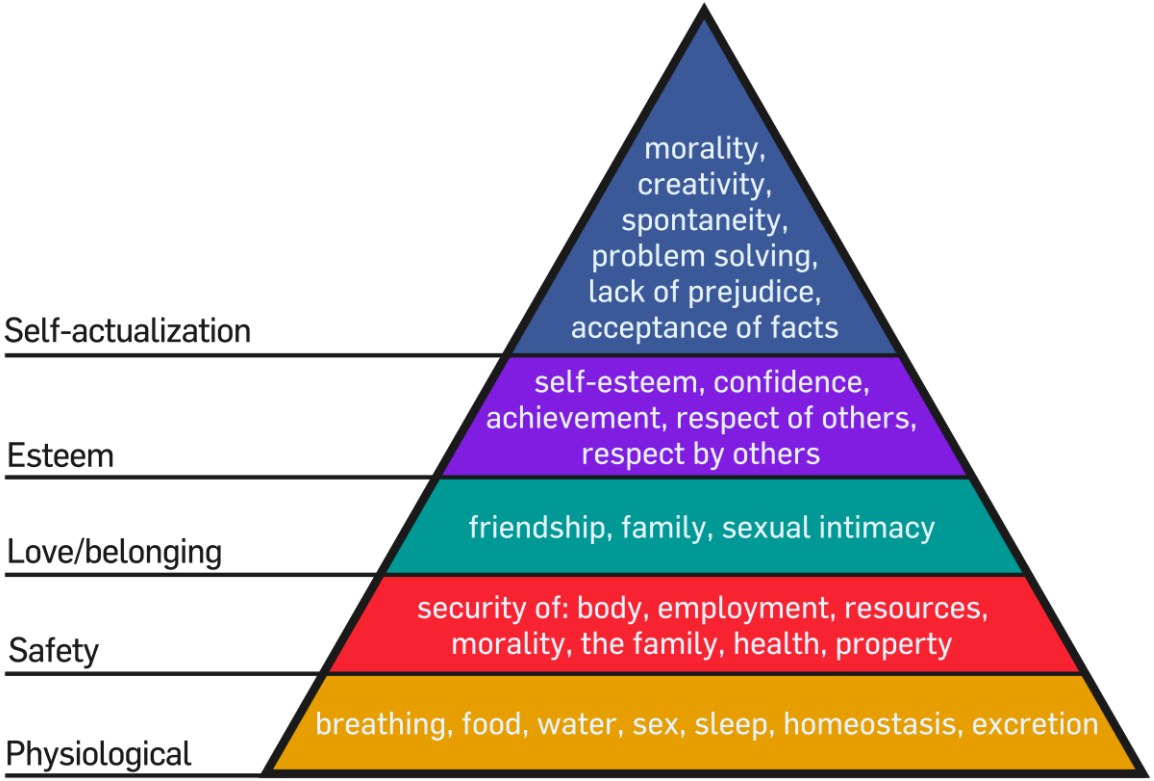
Soluții:

- Sistem de remunerare progresiv, în funcție de rezultate
- Selectarea angajaților în mod *științific* - instruire și perfecționare
- Pauzele = o necesitate pentru îmbunătățirea eficienței

Efectul Hawthorne (1924 - 1932) - Elton Mayo

- **Obiectiv inițial**: determinarea măsurii în care intensitatea luminii din fabrică are efect asupra productivității oamenilor
- **Rezultat**: Angajații și-au schimbat comportamentul (productivitatea a sporit) deoarece s-au simțit „sub lupă”, parte a unui experiment.
- **Concluzie**: Salariul și condițiile de lucru sunt factori mai puțin importanți decât recunoașterea importanței individului și apartenența, integrarea „în haită”

Piramida lui Maslow (1954)



Sursă foto: https://en.wikipedia.org/wiki/File:Maslow%27s_Hierarchy_of_Needs.svg

Piramida lui Maslow (1954)

- Ființa umană este motivată de o ierarhie de nevoi.
- Deficiency needs (D-needs) - urmare a lipsurilor:
  - Fiziologice
  - Siguranță
  - Dragoste / Sentimentul de apartenență
  - Stimă
- Growth needs (*being needs*: B-needs) - urmare a dorinței de creștere, împlinire
  - Dezvoltarea personală, autorealizarea
- Management bun → asigurarea ambelor tipuri de nevoi

Teoriile X și Y (1960) - McGregor

- **Teoria X** - Management *autocratic*, fondat pe idei precum:
  - Angajaților le displace munca, ar face orice să o evite
  - Angajații nu sunt ambițioși, nu doresc responsabilități
  - Ființele umane sunt lipsite de creativitate, au nevoie de instrucțiuni clare
  - Managerii trebuie să își controleze și să manipuleze angajații (principiul „biciul și zăhărelul”)
  - Satisfacerea angajatului → satisfacerea nevoilor fizice și de siguranță

Teoriile X și Y (1960) - McGregor

- **Teoria Y** - Management bazat pe *încredere*, fondat pe idei precum:
  - Munca este o activitate naturală, ca joaca și odihna.
  - Oamenii sunt în general foarte creativi.
  - Munca aduce împlinire omului.
  - Angajații devin responsabili atunci când se ține cont de părerea lor, sunt conștienți de sarcini.
  - Angajații sunt orientați către autorealizare și acceptă provocări.
  - Motivația vine din dorința de autorealizare și stimă de sine.

Teoriile X și Y (1960) - McGregor

Managerii aleg să folosească stilul de management corespunzător teoriei X sau teoriei Y în funcție de oamenii cu care au de a face.

„Cei mai mulți conducători acționează în concordanță cu teoria X, dar consideră că teoria Y este mai potrivită și reprezintă un ghid efectiv și eficient pentru acțiunea managerială.” - McGregor

Teoria Z - Ouchi (1981)

- Mix între stiluri de management americane și japoneze
- Principii de bază:
  - Pentru menținerea motivației, sunt necesare **obiective clare**.
  - **Motivația este esențială**.
  - **Chiar și oamenii motivați pot face greșeli**.
  - Task-urile trebuie să fie cât mai **standardizate** pentru o eficiență crescută.
  - **Obiectivele angajaților trebuie să se alinieze** cu nevoile companiei și cu condițiile de muncă.

Teoria W - Boehm (1988)

- Toate părțile interesate trebuie să aibă un câștig.
- Principii:
  - Înțelegerea a ce înseamnă conceptul de „câștig” pentru fiecare.
  - Crearea de așteptări realiste pentru fiecare proiect.
  - Identificarea clară a task-urilor bazată pe capacitățile fiecărui angajat.

Teoria factorilor duali a lui Herzberg (1968)

- KITA (Kick in the ass)
  - KITA fizic negativ
  - KITA psihologic negativ
  - KITA positiv

Teoria factorilor duali a lui Herzberg (1968)

- KITA (Kick in the ass)
  - KITA fizic negativ
  - KITA psihologic negativ
  - KITA positiv

NU FUNCȚIONEAZĂ!

Factorii duali

Factori MOTIVATORI (intrinseci, generatori de satisfacție)	Factori IGIENICI (extrinseci, generatori de insatisfacție)
Realizarea	Politicile companiei, administrarea
Recunoașterea	Supervizarea
Activitatea în sine	Relațiile interpersonale
Responsabilitatea	Condițiile de lucru & salariul
Avansarea	Statutul
Dezvoltarea profesională	Siguranța

!(SATISFACȚIE) ≠ INSATISFACȚIE

!(SATISFACȚIE) = LIPSA SATISFACȚIEI

!(INSATISFACȚIE) = LIPSA INSATISFACȚIEI

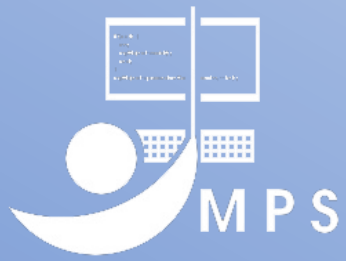
Teoria factorilor duali a lui Herzberg (1968)

Motivarea angajaților:

1. Eliminarea factorilor determinanți de insatisfacție (igienici)
2. Construirea factorilor determinanți de satisfacție (motivaționali)

Tipuri de interacțiune

Trăsături de personalitate



Trăsături de personalitate

- Oamenii sunt alcătuiți din trăsături de caractere diferite
- Echilibrul dintre aceste trăsături se modifică în funcție de circumstanțele personale și factorii externi
- Trăsăturile determină modul în care reacționăm la diverși stimuli
- Alegerea stimulului greșit poate da rezultate greșite

Bass and Dunteman (1963)

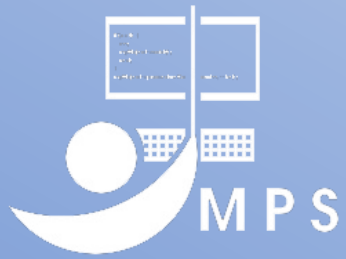
În mediul de lucru sunt definite trei tipuri de personalitate:

- **Task-oriented**
  - Plin de resurse, introvertit, competitiv, independent
  - Motivația de a lucra este lucrul însuși
- **Self-oriented**
  - Dezagreabil, agresiv, competitiv, introvertit, gelos, dogmatic
  - Câștig pe plan personal
- **Interaction-oriented**
  - Neagresiv, atent, fără nevoie de a se remarca sau de a câștiga
  - Motivația: acțiunile și rezultatele lucrului în echipă

Dr. David W Merrill and Roget Reid 1964 TRACOM™ Group 2004

- **Analitic**
  - Atent la detalii, înclină puțin spre risc, are nevoie de timp pentru a lua o decizie și vrea să aibă mereu dreptate
- **Amiabil**
  - Preferă lucrul în echipă și colaborările, nu ia decizii și nu îi place riscul, vrea să se simtă mereu în siguranță
- **Lider**
  - Intransigent, eficient, îi place riscul, vrea mereu rezultate vizibile
- **Creativ/expresiv**
  - Ambițios, entuziast, nu îi plac detaliile, îi place riscul

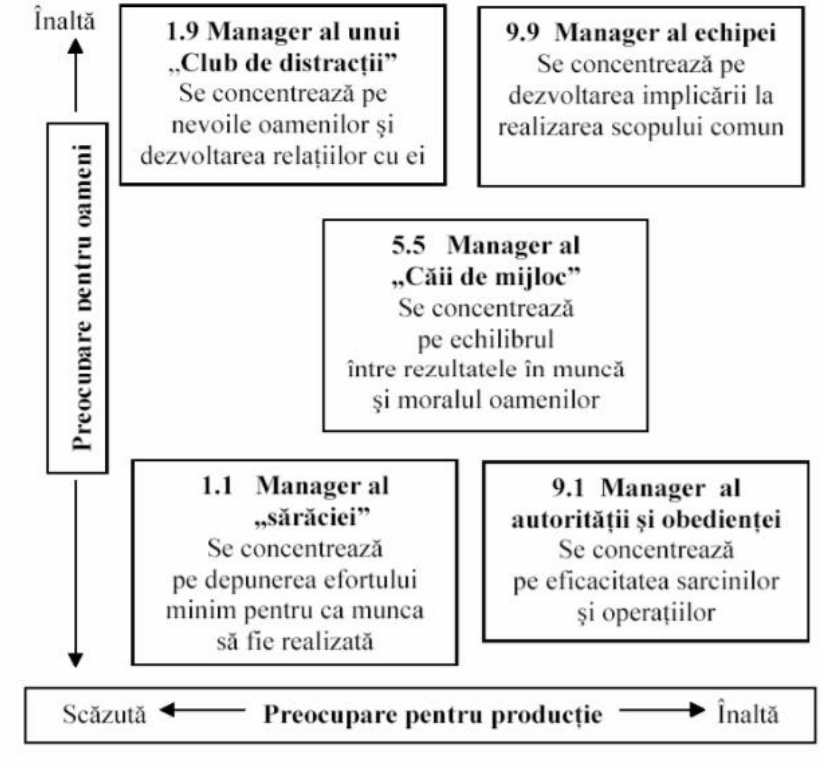
Strategii de management



Stiluri de management

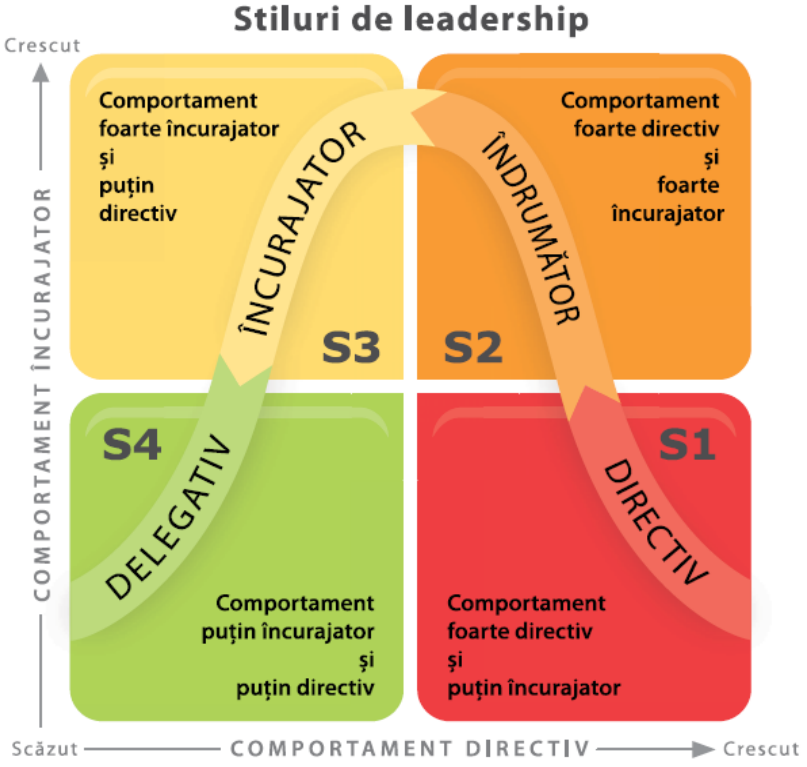
- **Autocratic**: managerul ia toate deciziile
- **Persuasiv**: managerul convinge subordonații să facă totul (tot autocratic)
- **Consultativ**: managerul implică echipa în procesul decizional, păstrând controlul asupra luării deciziilor
- **Democratic**: managerul permite echipei să ia parte în procesul decizional, alegând de comun acord
- **Laissez-faire**: managerul este mentor, acordând echipei libertate în luarea deciziilor

Modelul grilei manageriale
Robert Blake și Jane Mouton (1964)



Leadership situațional

Blanchard și Hersey (1969)



Structuri organizaționale

Ce influențează un proiect?

- Sistemul de management
  - Organizații bazate pe proiect
  - Organizații care nu sunt bazate pe proiect
- Cultura și stilul organizației
- Structura organizațională

Structura organizațională

- Necesară pentru a lucra eficient
- Nivelul de implicare al fiecărui participant depinde de modul de organizare
- Alegerea structurii adecvate este importantă.

Minim de informații

1.Cine este răspunzător de proiect?
2.Cum este împărțita informația intre participanții la proiect ?
3.Cum sunt remediate conflictele?

Tipuri de structuri organizatorice

- Ierarhică
- Matricială
- RACI
- Agile

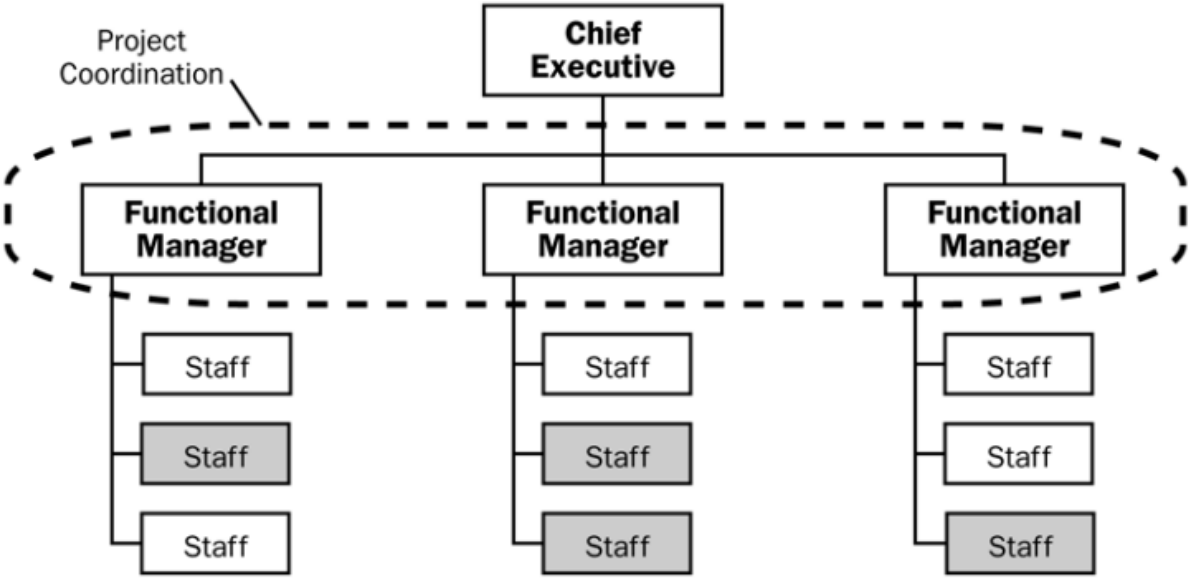
Exista 2 puncte de vedere:

- Structura organizației de care proiectul aparține
- Structura organizatorică aleasă pentru dezvoltarea proiectului

Structura ierarhică într-o organizație

- Fiecare participant are un superior clar stabilit
- Membrii personalului sunt împărțiți pe specializări
- Managerul de proiect are atât competente manageriale cât și tehnice pentru a administra proiectul
  - Variație: leadership-ul tehnic și administrativ sunt distincte.
- Deciziile sunt luate în vârful ierarhiei și propagate către celelalte departamente.

Structura ierarhică într-o organizație

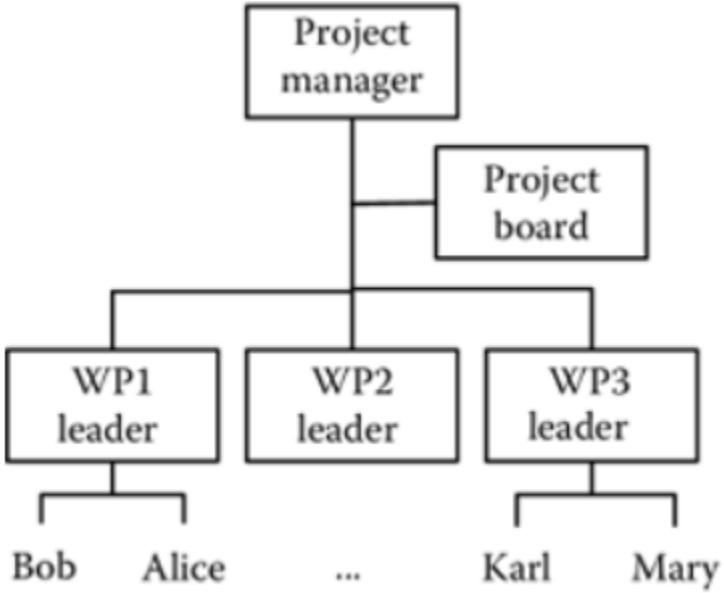


(Gray boxes represent staff engaged in project activities.)

Structura ierarhică într-un proiect

- PM are control asupra problemelor tehnice și manageriale
- Organizează munca în echipa de proiect.
- În proiectele mari se introduce un nivel intermediar: work-package leaders
- Comitet de management de proiect: compus din liderul fiecărui departament
  - Ajută la o mai bună comunicare
- Comitet de direcționare

Structura ierarhică într-un proiect



Structura divizională

- În cadrul organizațiilor mari
- Structura ierarhică nu mai face față
- Împărțire pe divizii
  - Concentrarea pe o activitate cu întreaga responsabilitate și autoritate
- Compartimentarea divizională se face pe baza produsului, ariei geografice, consumatorului sau, mai rar, procesului.

Structura divizională

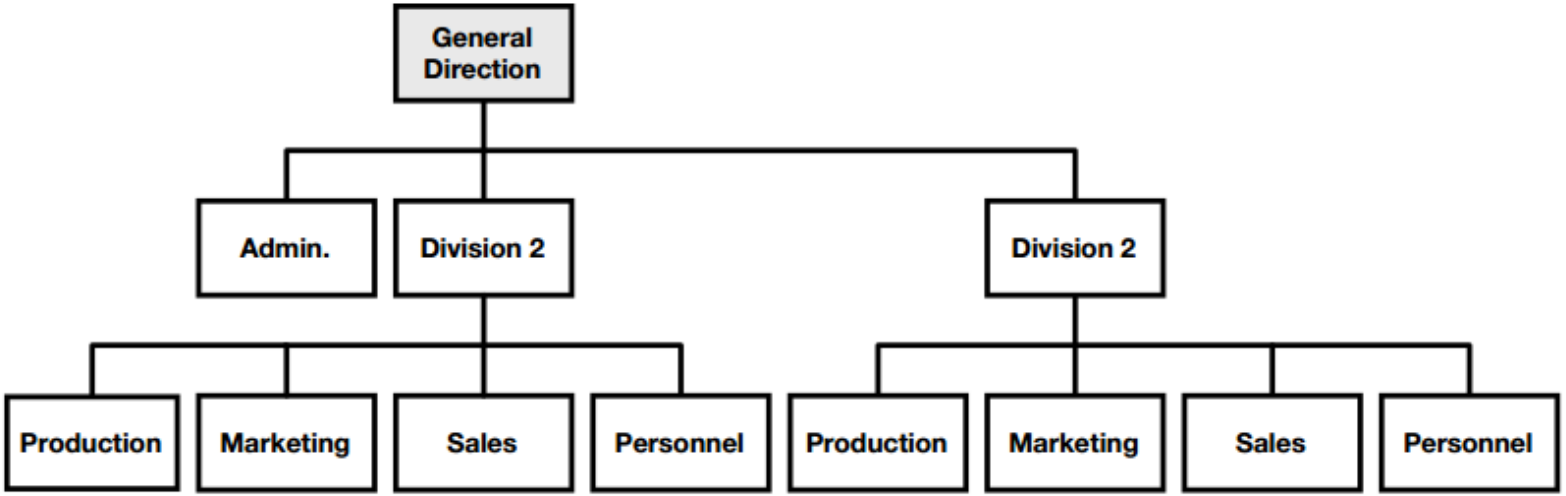
Avantaje

- concentrare asupra afacerii
- măsurare facilă a performanței
- focalizare asupra unei strategii a diviziei

Dezavantaje

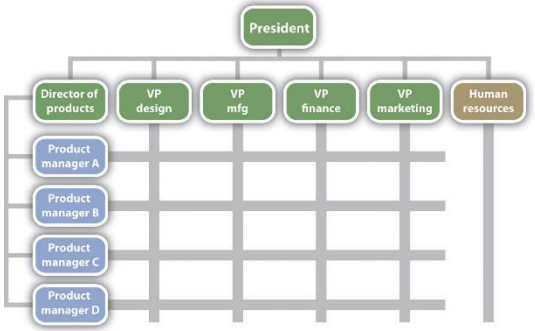
- conflicte între divizii
- costuri administrative ridicate
- complexitatea coordonării

Structura divizională



Structura matriceală într-o organizație

- Acest mod de organizare încearcă să găsească un echilibru între mai multe proiecte ce au nevoie de resurse funcționale comune.
- Funcționează cel mai bine în organizații cu multe proiecte mici ce se înscriu în tipare asemănătoare.
- În proiecte foarte mari în schimb, există posibilitatea de formare a unor grupuri, atunci când anumite necesități sunt acaparate de un anumit proiect, pierzându-se practice scopul acestei organizări.

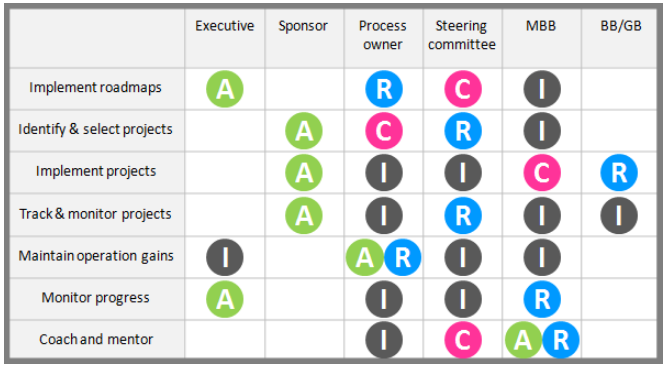


Structura matriceala într-un proiect

- Membrii sunt împărțiți în mai multe grupuri după funcția pe care o îndeplinesc
- Asemănător ca în cazul organizației, fiecare parte a proiectului va prelua după necesități membrii care să îndeplinească funcțiile necesare
- Este o soluție bună atunci când necesitățile bucăților din proiect sunt în continuă schimbare, drept pentru care resursele trebuiesc redistribuite
- Un dezavantaj provine din luarea de decizii și managementul indivizilor care vine din mai multe linii

Structura matrice RACI

- Această structură descrie responsabilitățile indivizilor sau rolurilor în cadrul unui proiect.
- Acestea sunt:
  - Responsabilitate
  - Răspundere
  - Consultare
  - Informare
- Specifică clar modul în care informațiile circulă, cât și modul în care deciziile sunt luate.



	Executive	Sponsor	Process owner	Steering committee	MBB	BB/GB
Implement roadmaps	A		R	C	I	
Identify & select projects		A	C	R	I	
Implement projects		A	I	I	C	R
Track & monitor projects		A	I	R	I	I
Maintain operation gains	I		A R	I	I	
Monitor progress	A		I	I	R	
Coach and mentor			I	C	A R	

Structura matrice RACI in proiect

- În cadrul unui proiect se face distincția între rol și individ:
  - Un individ poate avea mai multe roluri
  - Mai mulți indivizi pot avea același rol
- Fiecare task individual trebuie să aibă:
  - Cel puțin un rol R, dar cel mult un rol A
  - Este de preferat să nu existe sloturi goale într-o matrice RĂCI, dar în același timp să nu se exagereze rolurile (Consulted în loc de Informed de ex.)

Project Deliverable (or Activity)	Project Manager	Strategist	Designer	Front End Developer	Back End Developer
Design site map	C	R	A	I	I
Design wireframes	C	A	R	I	I
Create style guide	A	C	R	C	I
Code templates	A	I	C	R	C

Agile basics

- Ce este?
- Waterfall vs Agile
- Valorile și practicile Agile:
  - Indivizii și interacțiunile au întâietate asupra proceselor și instrumentelor
  - Software funcțional înaintea documentației complexe
  - Colaborarea cu clientul deasupra negocierilor contractuale
  - Răspunsul la schimbare înaintea urmăririi unui plan strict

© Scott Adams, Inc./Dist. by UFS, Inc.

Echipe Agile

- Echipe mici și dedicate
- Grup "cross-funcțional" de oameni
- Se preferă 5-9 oameni/echipă
- Rotație de roluri și funcții

Scrum

- Ce este?
- Caracteristici workflow Scrum:
  - Planificarea sprintului
  - Discuții interne zilnice
  - Demonstrație sprint
  - Retrospectivă sprint
- Roluri esențiale echipă Scrum:
  - ScrumMaster-ul
  - Clientul
  - Echipă

Pair-Programming

- Ce este?
- De ce este adoptată această tehnică?
- Beneficii:
  - Calitate
  - Integrare
  - Mentorat și învățare
  - Flexibilitate

Adantaje și dezavantaje echipe Agile

Avantaje

- Vizibilitate continuă
- Testare continuă
- Livrare rapidă
- Planificare efectivă pe termen scurt

Dezavantaje

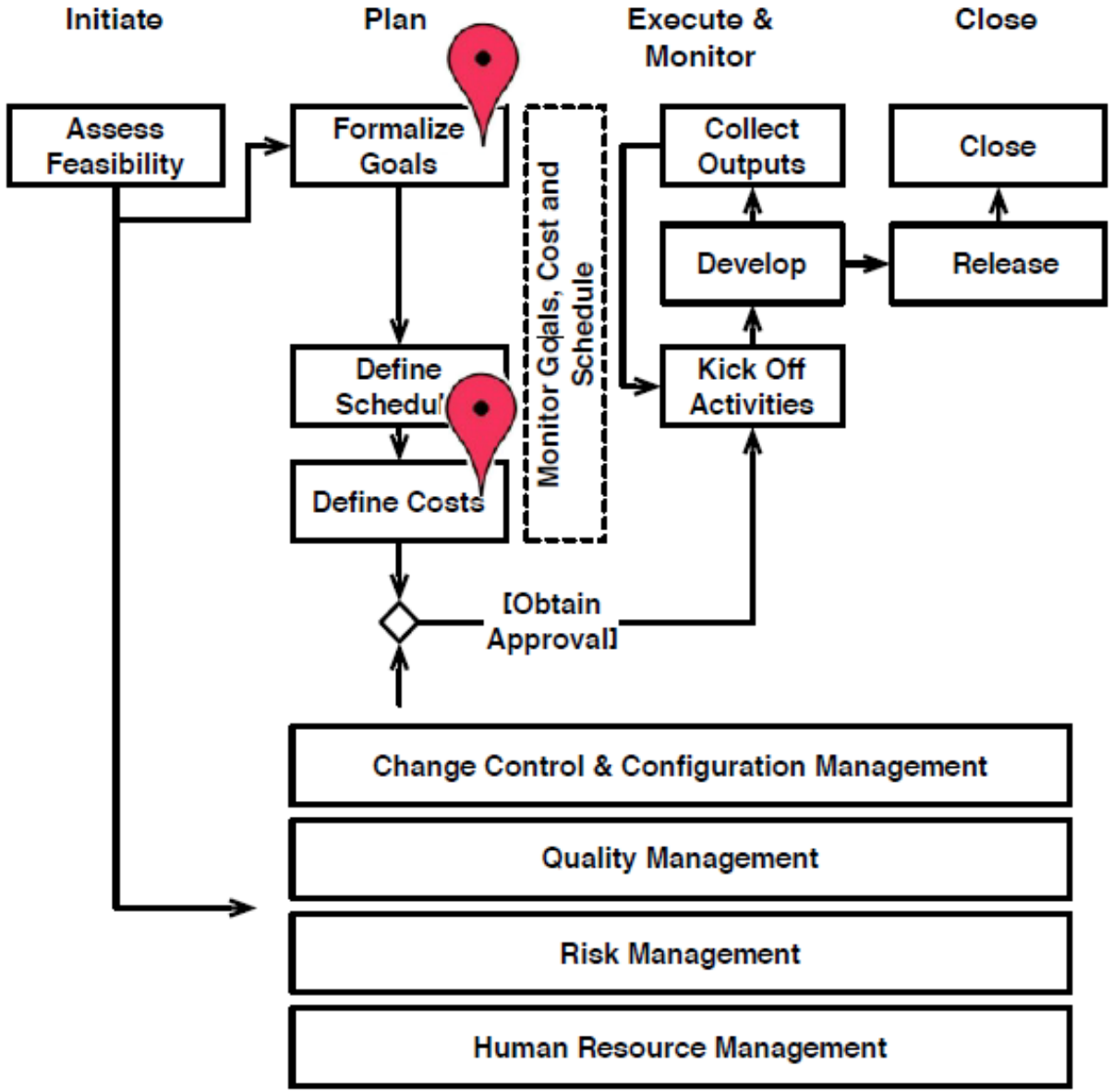
- Lipsă clarității poate duce la probleme
- Sunt necesare skilluri avansate
- Training insuficient
- Prea multă ambiție

Stabilirea prețurilor proiectului

Scop

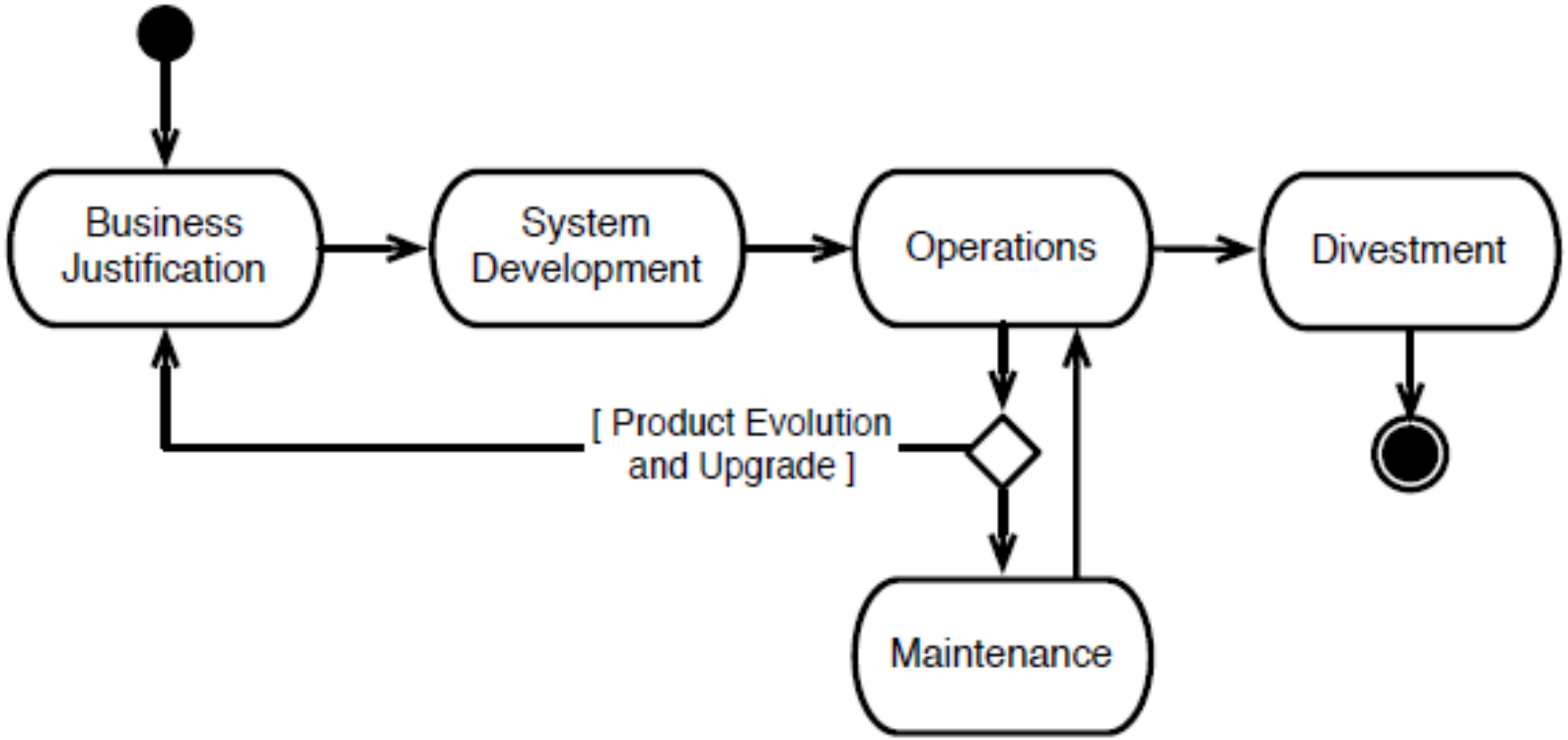
Cum determinăm costul rezultatelor proiectului.

Înțelegerea procesului de achiziție.


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedul
Define Costs
Monitor Goals, Cost and Schedule
Collect Outputs
Develop
Kick Off Activities
Close
Release
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

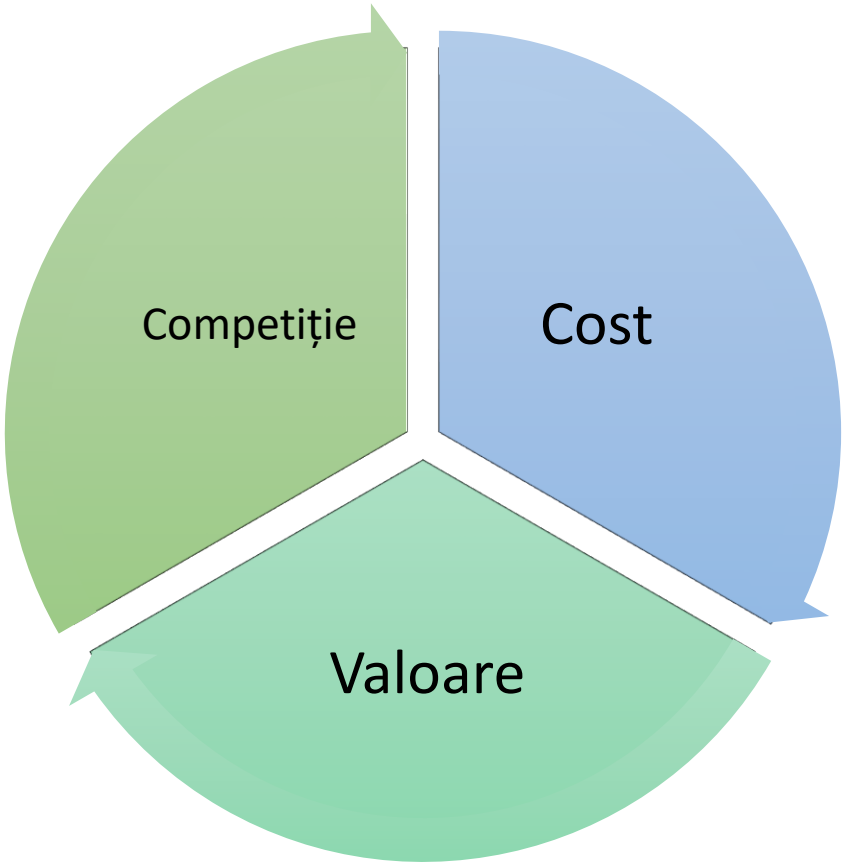
Durata de viață a unui proiect

Toate fazel



Modele de stabilire a prețului

Prețurile software-ului



Vânzare, licențiere, leasing

Deținerea codului sursă sau a executabilelor reprezintă un factor important în stabilirea prețului.

Software-ul este, de regulă, sub licență.

Leasing: folosirea temporară a produsului.

Software gratuit și Open Source

Caracteristici:

- Drepturi de folosire
- Datorii

Exploatare comercială:

- Instalare
- Pregătire
- Documentație
- Gadget-uri
- Donații

Sursă: https://tableplus.io/assets/images/open-source/open-source-community.png

Stabilirea prețului pentru proiect

Acorduri contractuale

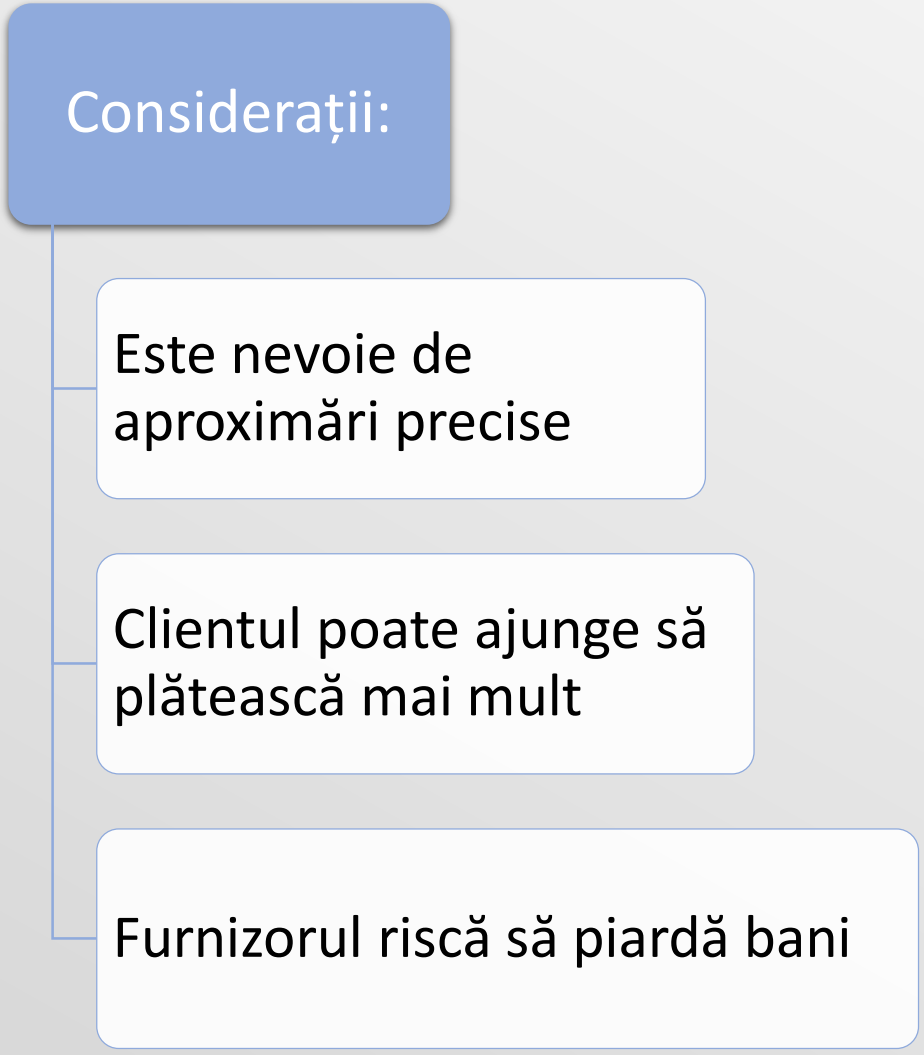
01

Clientul și contractorul au scopuri diferite pentru stabilirea prețului și planificarea plăților

02

Contractele reglementează aceste scopuri.

Contracte cu preț fix



Timp și materiale

- Furnizorul expune costurile către client și facturează conform costurilor efective.
- Considerații:
  - Se ține evidența timpului consumat
  - Risc de ineficiență

Reținere de fonduri

- Preț fix plătit către furnizor în schimbul unor servicii oferite mai târziu
- Considerații:
  - Potrivit dacă cererile nu sunt clare
  - Echivalent cu angajarea pe o perioadă determinată a forței de muncă
  - Plățile pot fi împărțite pe perioade fixe (lunar)

Cost Plus

Prețul de vânzare este format din prețul individual + o adiție specifică.

Tipuri:

- Contracte cu recompensă
- Contracte cu stimulent
- Contracte cu onorarii fixe

Considerații:

- Împarte riscul între client și furnizor
- Controlul costului suplimentar poate fi mai dificil

Acorduri contractuale și bugetul proiectului

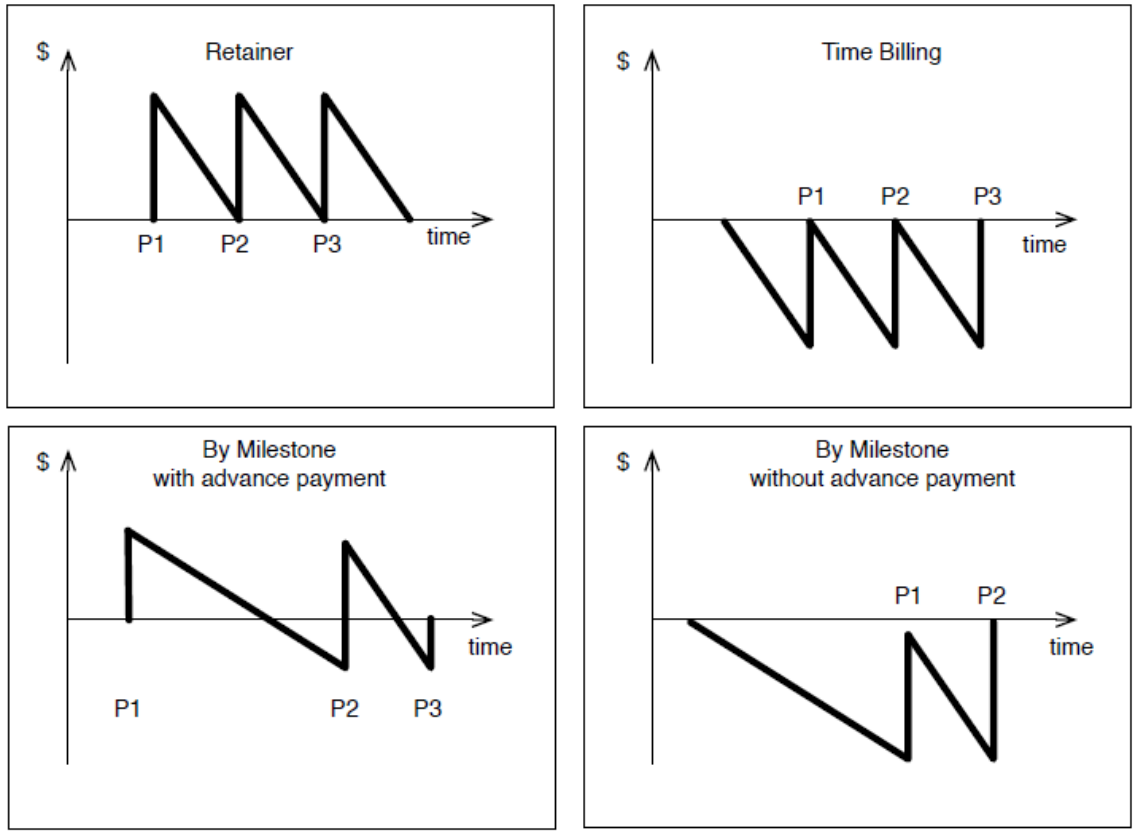
Intervalele de plată trebuie să fie reglementate pentru a echilibra riscul și implicarea

Din punct de vedere pur economic, costul de finalizare determină dacă are sens continuarea proiectului

Structuri de plată

Plățile și fluxul de bani

Nivelul de implicare al furnizorului depinde de structura plății



Achiziții și delegare

Procesul de achiziție

Multe proiecte au nevoie de servicii sau produse de la alți vendori

Trebuie alocat un timp suficient pentru achiziții și resurse suficiente pentru a monitoriza procesul.

Procesul de achiziție

Activități :

- Identificarea necesităților
- Identificarea și selectarea vendorilor (solicitare, selecție, recompensare)
- Administrarea îndeplinirii contractului
- Acceptarea produsului final

Invitație la licitare

Trebuie să cuprindă:

- Specificarea produselor și serviciilor oferite
- Constrângeri
- Modalități de propunere și criteriile de selectare ce vor fi folosite pentru îndeplinirea contractului

BOTSWANA
Qualifications Authority

INVITATION TO TENDER

Tender No BQA PR 48/18

Botswana Qualifications Authority (BQA) invites interested companies to tender for the **SUPPLY, DELIVERY AND INSTALLATION OF OFFICE FURNITURE.**

REQUIRED PPADB CODES:

- Code: 213 General Manufacturers/Producers
  - Sub Code: 06 Furniture

Sursă: https://www.bqa.org.bw/sites/default/files/news-image/screen_shot_2018-09-03_at_10.35.59_am.png

Solicitări

Ce poate fi făcut depinde de constrângerile legale și organizaționale.

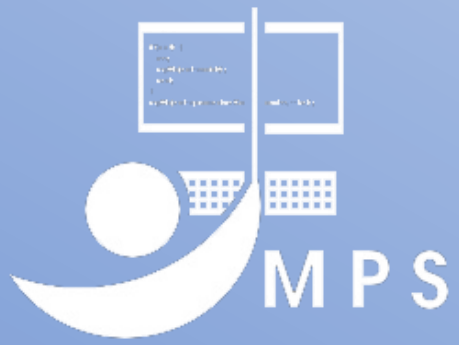
Invitația la licitație este făcută publică prin mai multe modalități

Motivație: Gestionarea compromisului dintre preț, calitate și timp

Considerente temporale ale licitațiilor deschise

- Trebuie să se țină cont de durata următoarelor activități:
  - Pregătirea invitației pentru licitație
  - Luarea la cunoștință a ofertei de către potențialii clienți
  - Evaluarea propunerilor
  - Acordarea contractului
  - Livrarea produselor sau serviciilor

Procese pentru Dezvoltarea Software



Procese pentru dezvoltarea software
Motivație

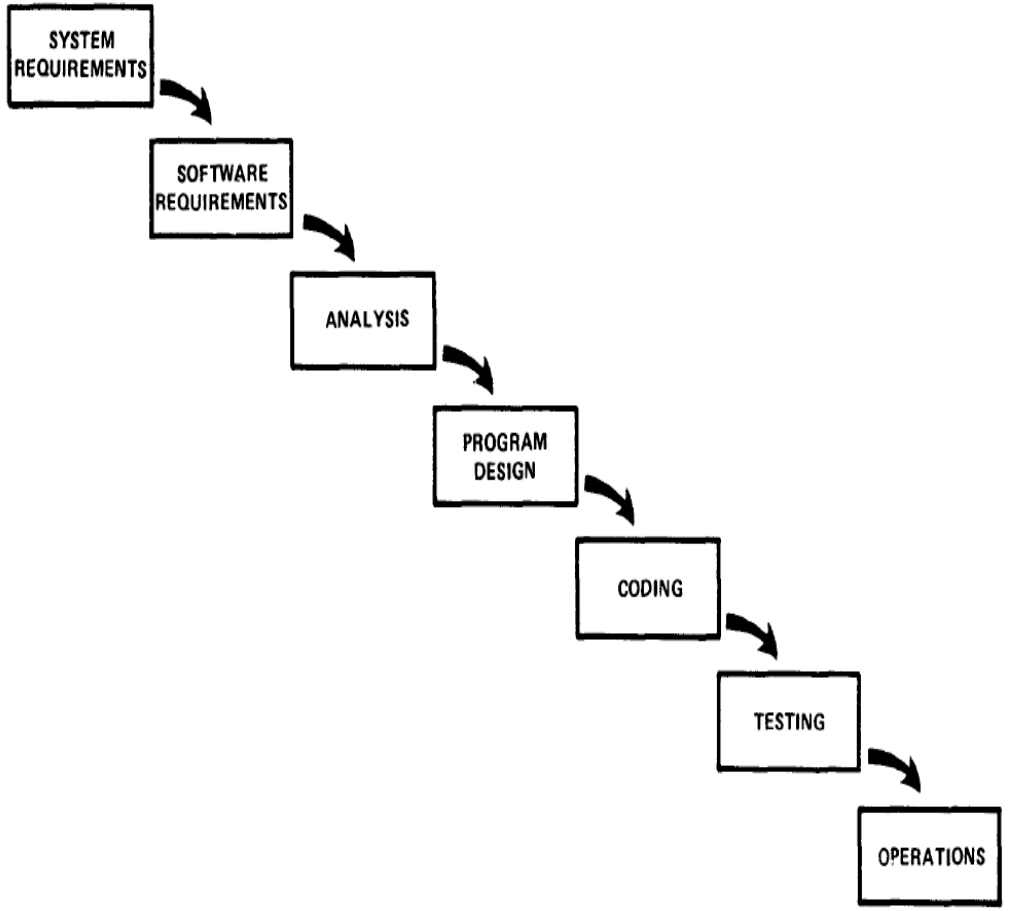
- Noi procese de dezvoltare software au fost introduse pentru a aduce o serie de îmbunătățiri la modelul clasic de dezvoltare
- **Procese de Dezvoltare Software** → model structurat și progresiv de la o idee la aplicația finală
- Îmbunătățiri:
  - Mai multă flexibilitate în procesul de dezvoltare
  - Satisfacția utilizatorului (aplicațiile sunt mai appropriate de cerințele clientului)
  - Eficientă
  - Timp și costuri
  - Calitate

Modele de dezvoltare

- Waterfall
- Prototype
- Incremental
- Spiral
- V-Model
- Cleanroom
- RUP
- XP
- SCRUM
- RAD
- ASAP
- MSF
- DFDM

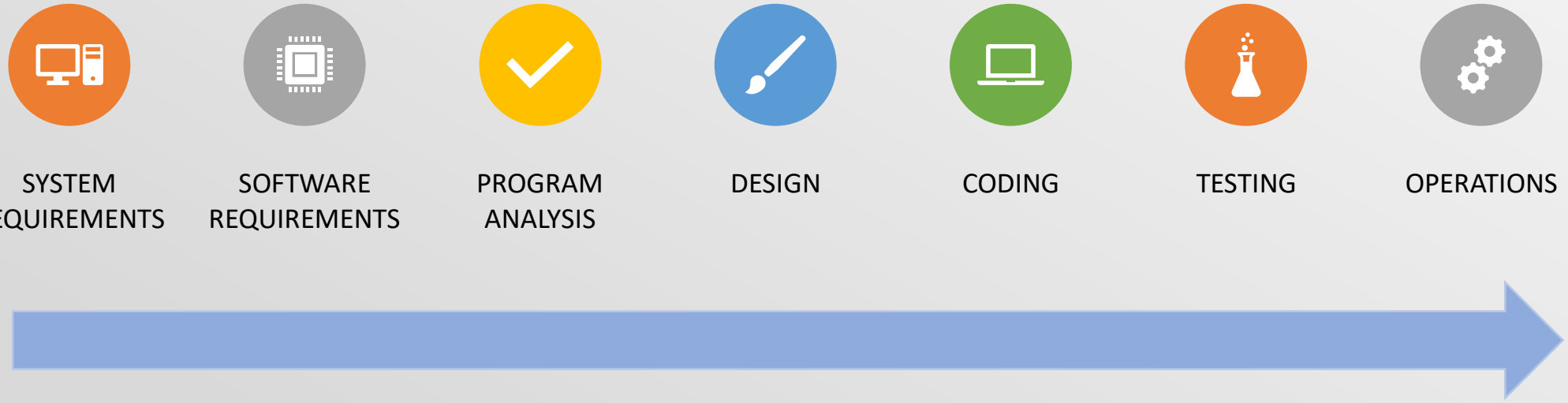
Modelul Waterfall

- Modelul Waterfall reprezintă o abordare relativ secvențială a procesului de dezvoltare software.
- Flow-ul modelului Waterfall decurge în principal într-o singură direcție, etapele unui proiect fiind abordate secvențial.



Etapele modelului Waterfall

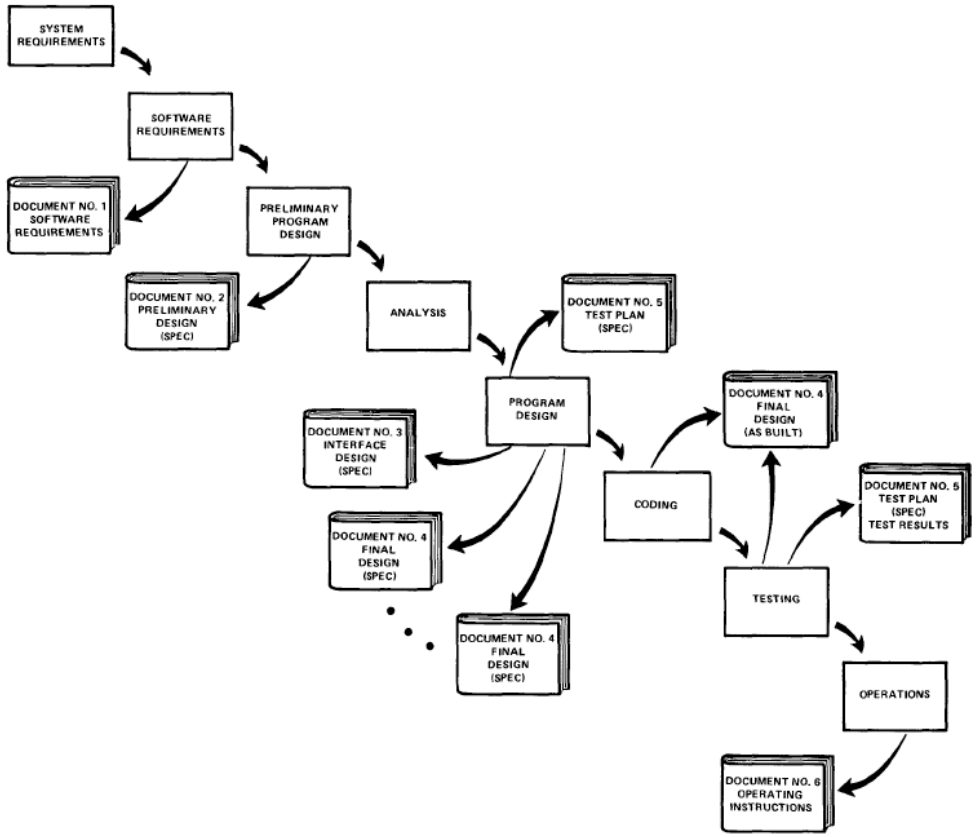
Etapele modelului Waterfall



Etapele Modelului Waterfall

- System and Software Requirements: sunt documentate toate specificațiile hardware și software ale aplicației → Product Requirements Document(PRD) / Software Requirements Document(SRD) (include toate cerințele aplicației)
- Analysis: din această etapă rezultă modele, scheme ale aplicației
- Program Design: arhitectură software-ului
- Coding:
  - Etapă de development propriu-zisă;
  - De asemenea, codul trece prin prima etapă de testare(unit-testing);
  - Software Integration: integrarea tuturor modulelor și serviciilor într-o singură aplicație funcțională.
- Testing: etapă de testare sistematică a aplicației
- Operations: Include instalarea, migrarea, suportul și mententanta aplicației.

Etapele Modelului Waterfall detaliat

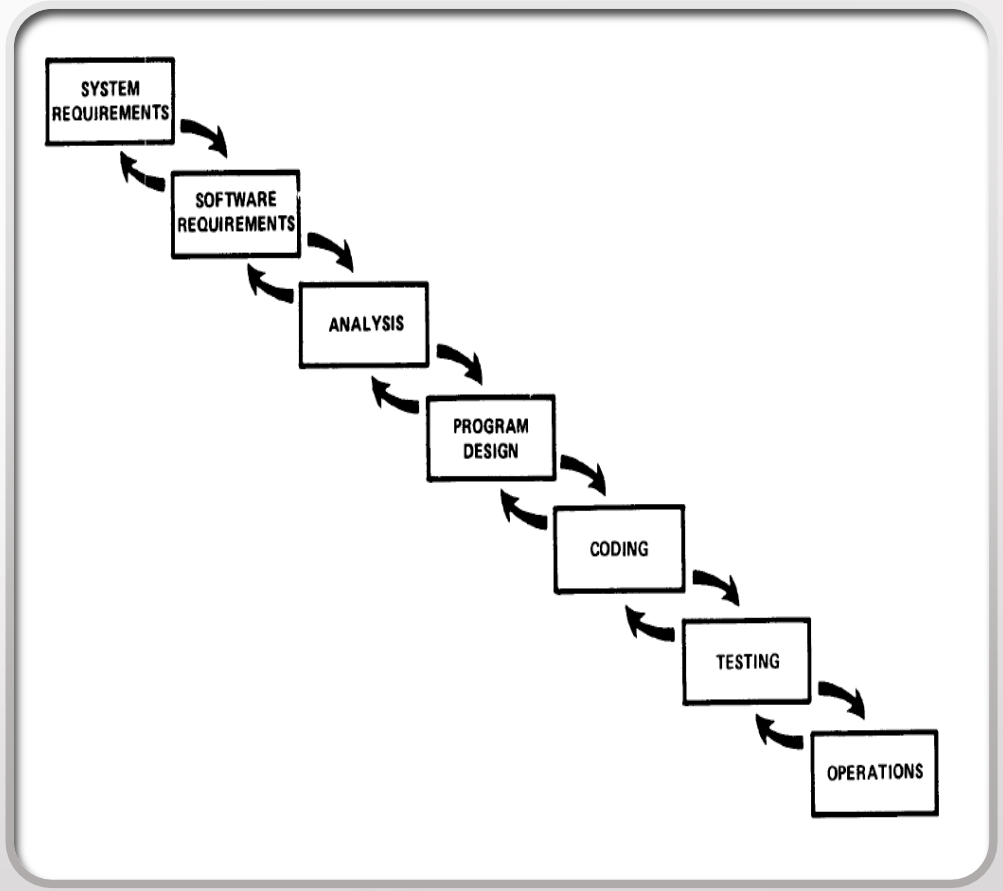


Dezavantaje metoda Waterfall

- Partiționare ineficiența a proiectului în stagii distincte
- Flow-ul sevential al modelului Waterfall face dificilă schimbarea pe parcurs a cerințelor utilizator → interacțiune cu clientul redusă
- Acest model este recomandat atunci când:
  - Cerințele sunt stabilite clar de la început și nu vor mai fi schimbate pe parcursul dezvoltării
  - “Freeze” the requirement document

Waterfall – Modelul final al lui Winston W. Royce

- S-a constatat faptul că este necesar și recomandat un feedback între etape, care să meargă în ambele sensuri
  - Feedback de la testing la design(în etapă de testare se pot identifică erori de design)
  - Feedback de la design la requirements(probelemele de design ar putea necesită modificarea cerințelor)
- De asemenea, se urmărește implicarea clientului cât mai mult în procesul de dezvoltare al proiectului.
- Se recomandă cât mai minuțioasă a documentației(“doing the job twice if possible”

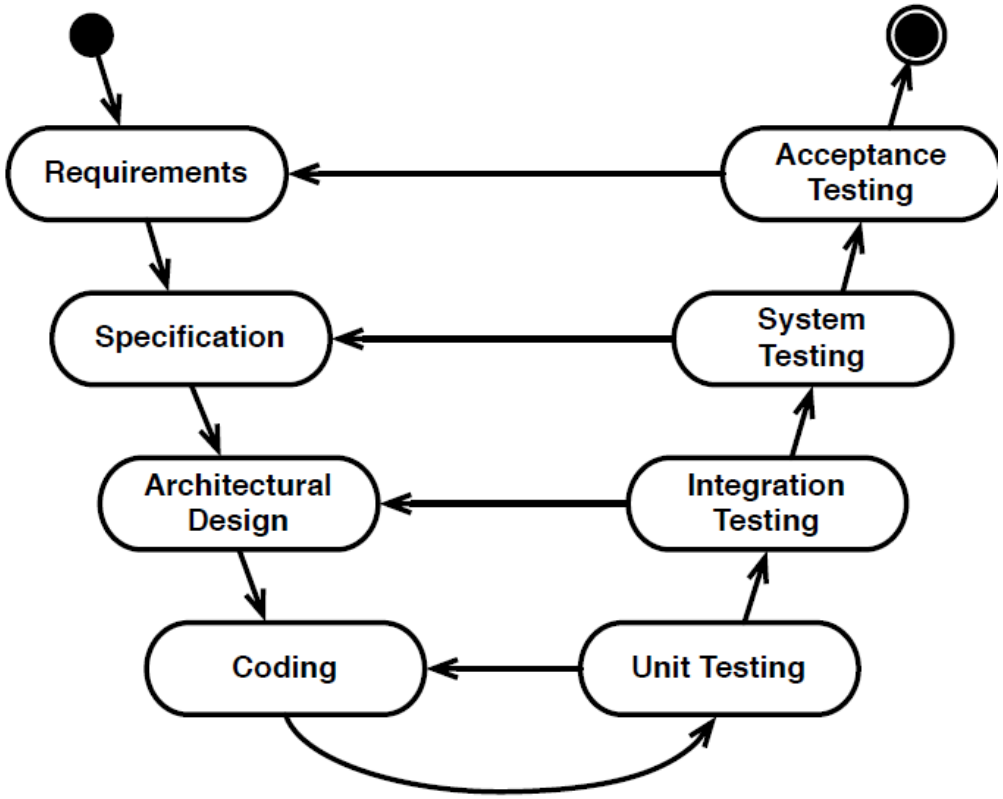


Variații ale modelului Waterfall

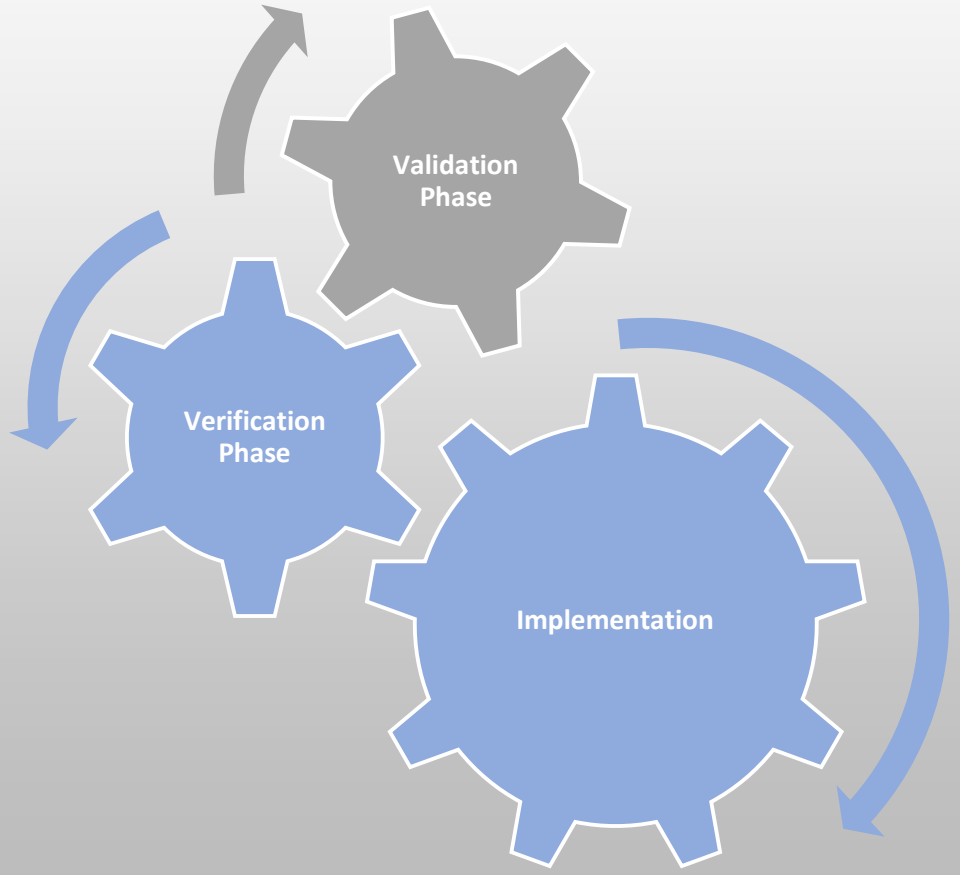
- Sashimi Waterfall: activitățile se pot suprapune.
- Waterfall cu sub-proiecte: implementarea în parallel a componentelor.
- Waterfall cu risk-reduction: se realizează o analiză inițială a riscurilor posibile, astfel încât să se diminueze impactul problemelor care pot apărea în fazele ulterioare ale procesului de dezvoltare.

V-Cycle

- Considerat o extensie a modelului Waterfall → accent pe testare
- În loc să se urmeze traseul clasic al Waterfall(etapele se succed în jos), în modelul V-Cycle etapele procesului de dezvoltare se succed în sus după etapă de coding, de aici rezultând formă în V a modelului.
- Modelul în V demonstrează relațiile dintre fiecare fază a ciclului de dezvoltare și fază asociată de testare.

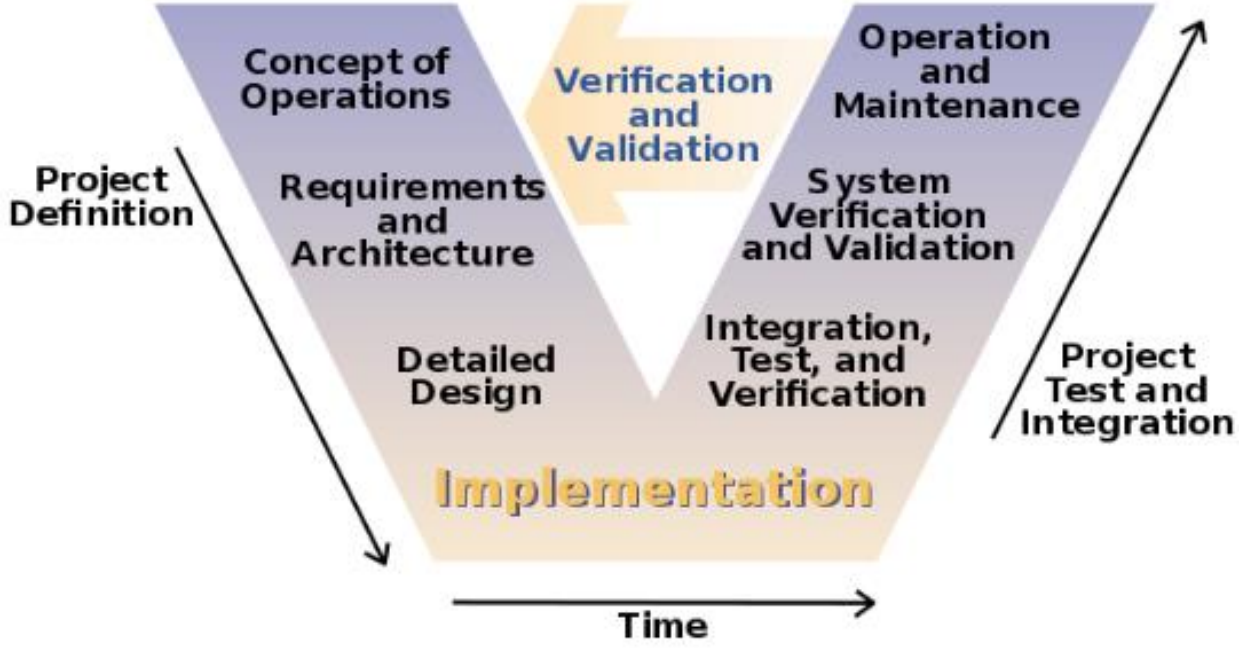


V-Cycle



V-Cycle

În acest model, fiecare faza a etapei de verificare are un corespondent în etapa de validare (testare).



V-Cycle – Verification Phases

Requirement Analysis: sunt colectate cerințele sistemului analizând cerințele utilizatorului → *User Requirements Document(URD)*

System Design: se analizează tehnicile prin care se pot implementa cerințele utilizatorului așa cum sunt descrise în URD → *Software Specification Document (*blueprint pentru etapa de dezvoltare)

Arhitecture Design*(high-level design):* design-ul arhitecturii hardware și software

Module Design*(low-level design):* se “sparge” aplicația în module mai mici, iar fiecare modul este explicat în detaliu. Se realizează o logică detaliată (în pseudocod) a functionalității modulului.

V-Cycle – Validation Phases

Unit Testing: sunt dezvoltate în etapa de *Module Design*.

Scop: eliminarea bug-urilor la nivel de modul.

Integration Testing: sunt dezvoltate în etapa de *Arhitectural Design*

Scop: verificarea comunicării dintre module după ce acestea au fost integrate.

System Testing: sunt dezvoltate în etapa de *System Design*, de către o echipă din partea clientului.

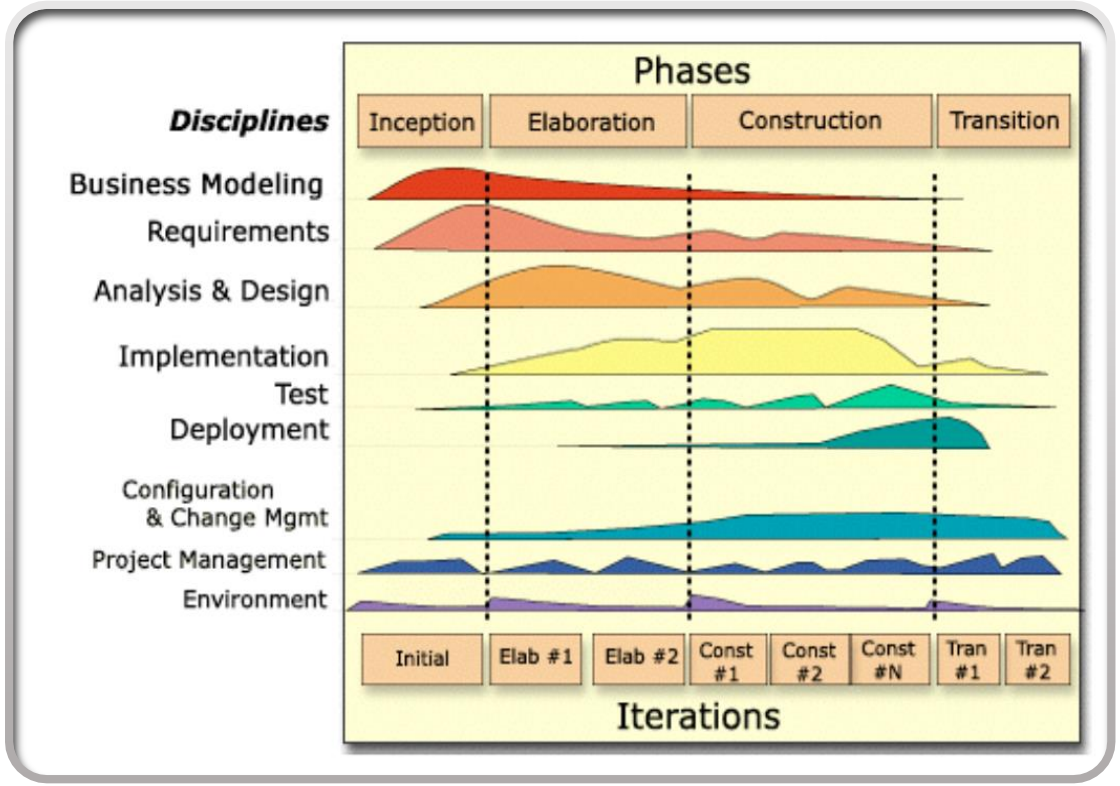
Scop: se verifică dacă au fost îndeplinite cerințele aplicatiei. Se testează întreaga aplicație.

User Acceptance Testing: dezvoltate în etapa de *Requirement Analysis,* de către utilizatorii bussiness.

Scop: se testează aplicația într-un mediu realistic, asemănător celui de producție; se verifică dacă aplicația îndeplineste cerințele user-ului și dacă este gata de dat în folosință în mediul de producție.

Rational Unified Process(RUP)

- Procesul de dezvoltare este împărțit în 2 dimensiuni:
  - **Fazele**, organizate în iterații, corespund unor niveluri diferite de maturitate
  - **Workflows**, concentrate pe un anumit aspect al dezvoltării
- Se suprapun și sunt caracterizate de niveluri diferite de intensitate



Rational Unified Process – Best Practices

- Există 6 practici principale ce defines principiile RUP:
  - Dezvoltare software în mod iterativ
  - Managementul cerințelor(inclusive evaluarea alternativelor)
  - Utilizarea de arhitecturi component-based(componente robuste)
  - Reprezentare simplă și neambiguă a modelului software
  - Verificarea calității software-ului
  - Schimbări de control ale software-ului (atât pentru calitate cât și pentru management)

Open Unified Process (OpenUP)

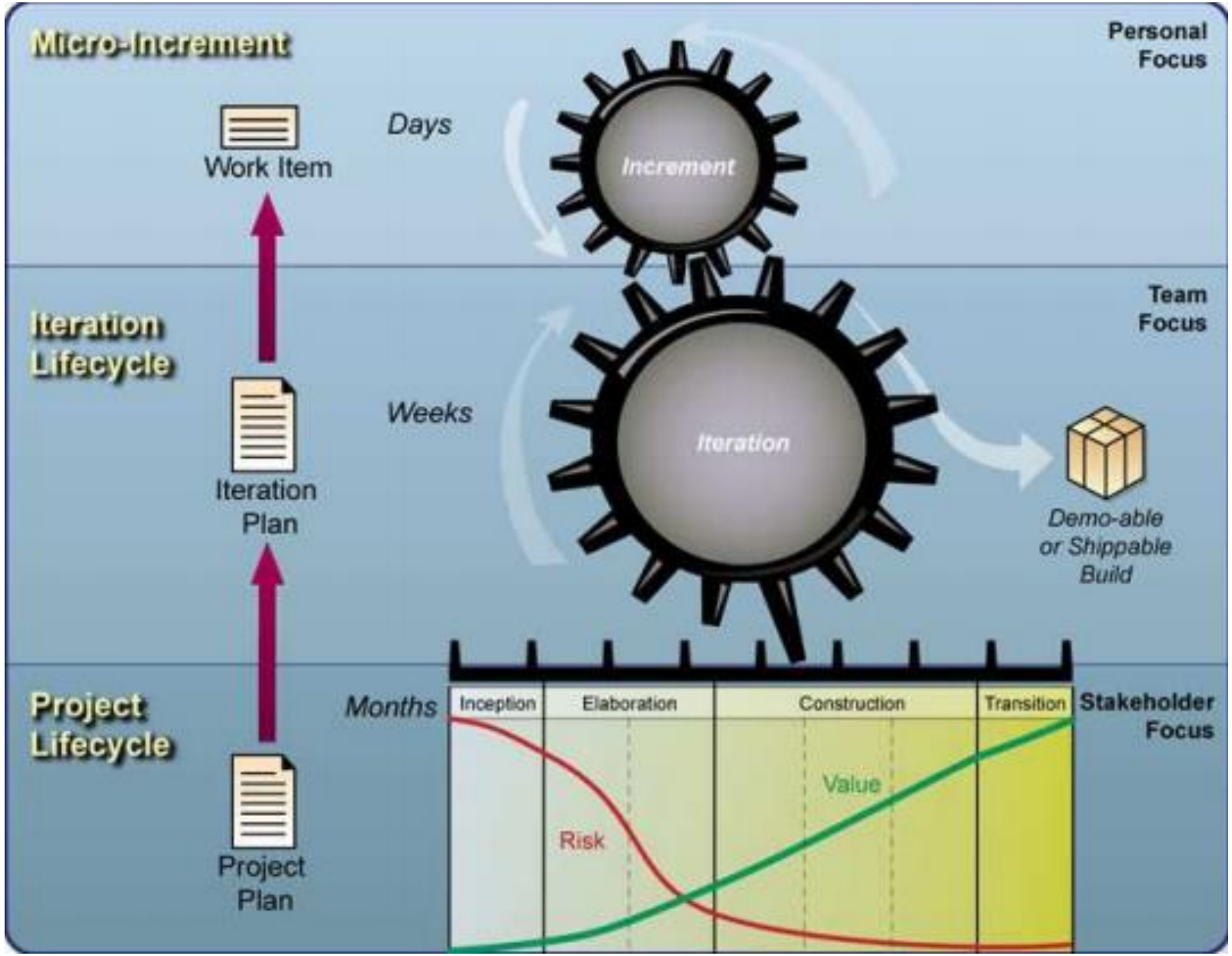
- Evoluție a Rational Unified Process (RUP)
- Păstrează caracteristicile esențiale ale RUP
  - Dezvoltare iterativă
  - Cazuri de utilizare
  - Scenarii
  - Evaluarea riscurilor
- Marea majoritate a părților opționale ale RUP au fost excluse

Open Unified Process (OpenUP)

- Conține componente RUP ce au fost unite
- Rezultă un proces mult mai simplu care respecta principiile RUP
- Dedicat echipelor mici (3-6 oameni) care doresc o abordare agile si iterativă => 3-6 luni de dezvoltare

OpenUp vs Agile

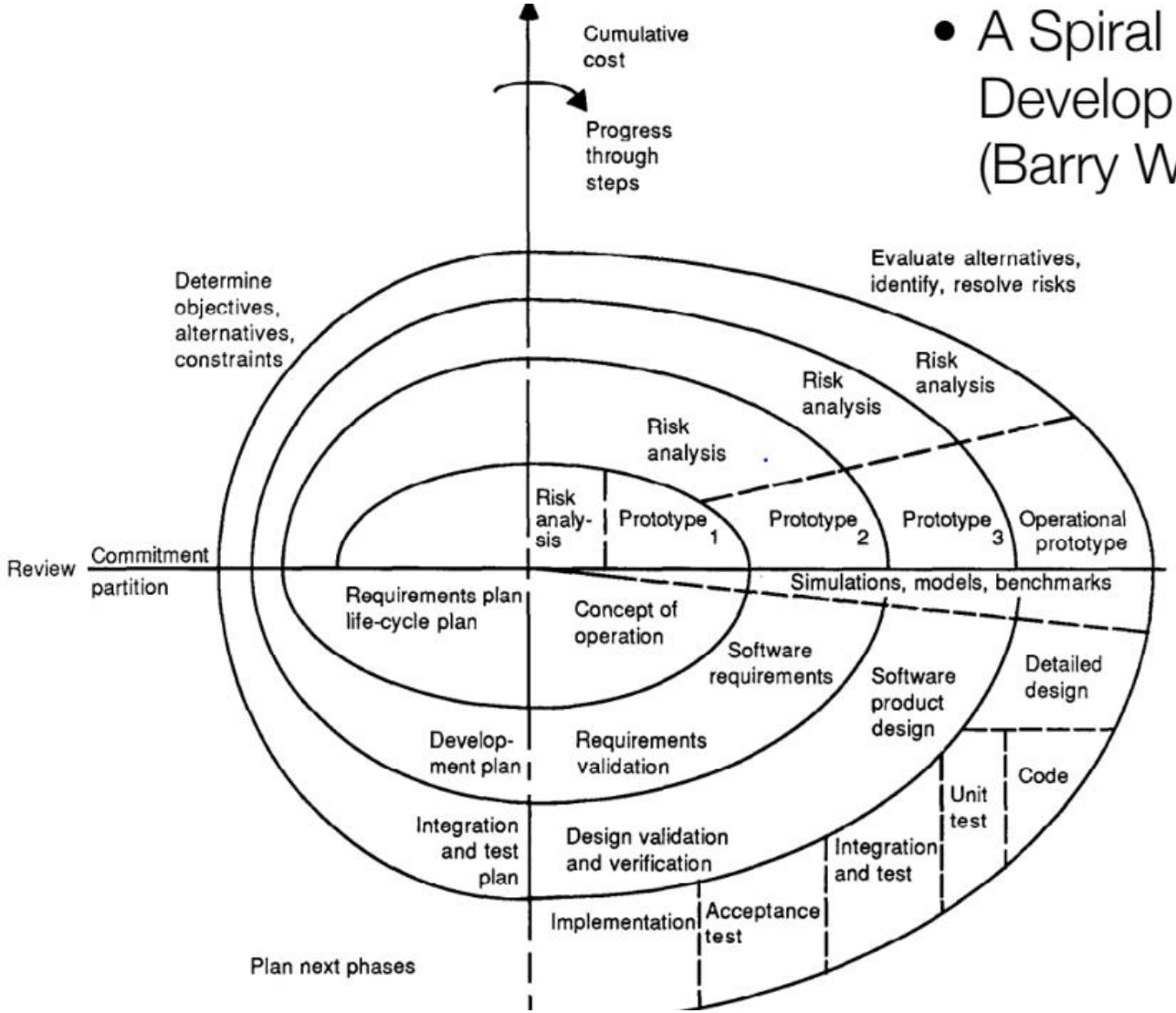
OpenUP	AGILE
Colaborare pentru stabilirea obiectivelor și pentru a înțelege demersul proiectului	Interacțiuni între membrii pentru a stabili modul de desfășurare si tehnologiile folosite
Cântărirea priorităților pentru a maximiza profitul părților implicate	Colaborare cu clienții/ negociere contract
Focusarea pe arhitectură in stagiile incipiente pentru a minimiza riscurile si a organiza procesul de dezvoltare	Dezvoltarea softului având o documentație completă
Se lucrează la produs pentru a obține feedback continuu și a îmbunătății	Aplicarea unor schimbări în timp ec se urmează un plan


Micro-Increment
Personal Focus
Work Item
Days
Increment
Iteration Lifecycle
Team Focus
Weeks
Iteration
Iteration Plan
Demo-able or Shippable Build
Project Lifecycle
Months
Inception
Elaboration
Construction
Transition
Stakeholder Focus
Value
Risk
Project Plan

Spiral

- Poate fi considerat o altă vedere a modelului în cascadă
- Softul este dezvoltat în cicluri → metodă iterativă
- Fiecare ciclu reprezintă o fază
- Fiecare ciclu este ales în funcție de ce e nevoie → nu există o ordine prestabilită
- Prima fază constituie o evaluare a riscurilor și alternativelor

- A Spiral Model of Software Development and Enhancement (Barry W. Boehm)



Spiral

Avantaje

- Bazată pe analiză de risc și alternative
- Preia diferite practici de dezvoltare software
- Evoluția softului (mentenanța reprezintă un ciclu din spirală)

Dezavantaje

- Respectarea cerințelor de soft din contract
- Dificultăți în a face estimări la început
- Flexibilitate scăzută

Activităţile unui ciclu de producţie

- Planificare
  - Stabilirea obiectivelor, alternativelor de rezolvare şi a restricţiilor pentru ciclul curent
- Analiza riscurilor
  - Analizează alternativele de rezolvare şi restricţiile din (1)
  - Identifică factorii de risc
  - Decizia GO/NO GO (continuă/renunţă)
- Inginerie - începutul unui ciclu nou
  - Dezvoltarea produsului pe următorul nivel de detaliere
  - Se pot folosi
    - Modelul clasic
    - Prototipizarea - pentru clarificarea unor cerinţe
- Evaluarea clientului

Prototiparea

- După ce sunt stabilite cerințele, un prototip al aplicației este dezvoltat (de exemplu GUI-ul) și este evaluat de client
- Se poate focusa pe aplicație sau pe o funcție specifică
- Poate fi de 3 tipuri
  - Throw away
  - Evoluționar
  - Incremental

Tipuri de prototipare

- **Throw away**
  - Presupune a fi folosit doar pentru a testa unele idei după care se renunță la el când se dorește dezvoltarea sistemului
- **Evoluționar**
  - Este dezvoltat și modificat până când devine sistem funcțional
- **Incremental**
  - Este dezvoltat si modificat in etape mici, dupa fiecare etapă fiind primit un feedback

Avantaje și dezavantaje – Model Spiral

Avantaje

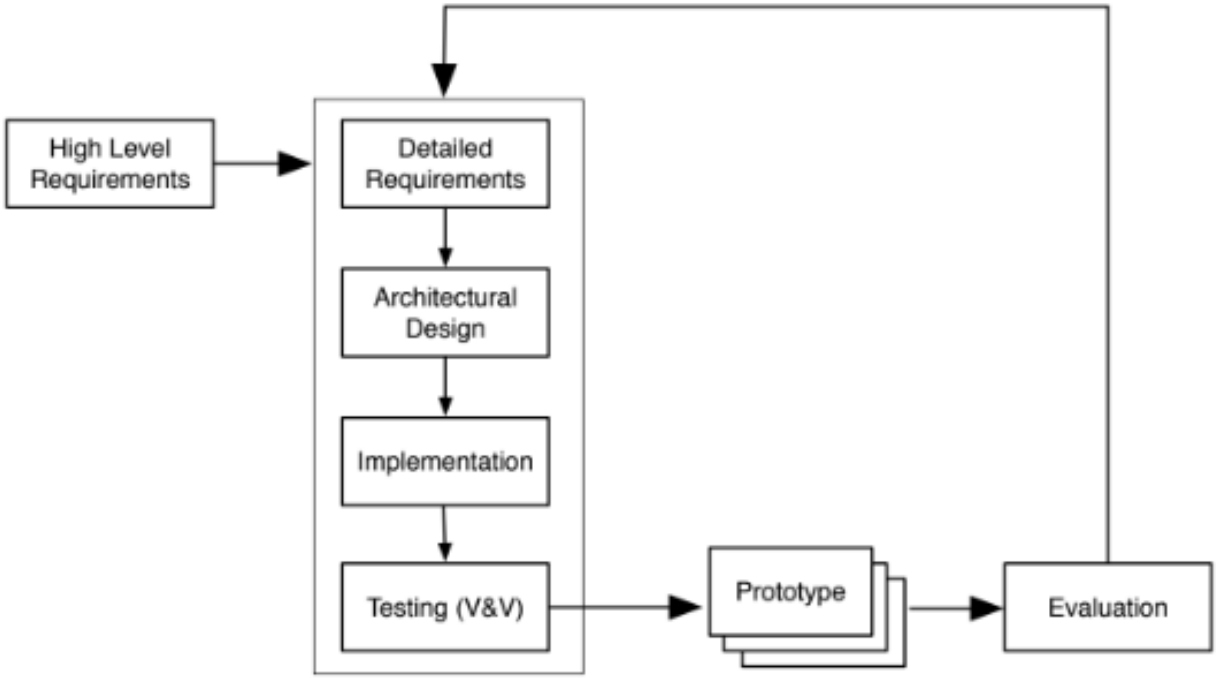
- Comunicare client – dezvoltator → clientul poate oferi feedback mai ușor
- Se pot testa diferite prototipuri si poate fi ales cel mai bun
- Documentație redusă datorita testării practice a prototipurilor
- Componentele mai greu de implementat pot fi dezvoltate la început, astfel testându-se dacă este o abordare potrivită

Dezavantaje

- Este nevoie ca unele componente să fie reimplementate
- Nu se cunosc foarte clar etapele procesului de implementare
- Abordarea evoluționară conduce la arhitecturi slabe (sunt construite cu informații incomplete)

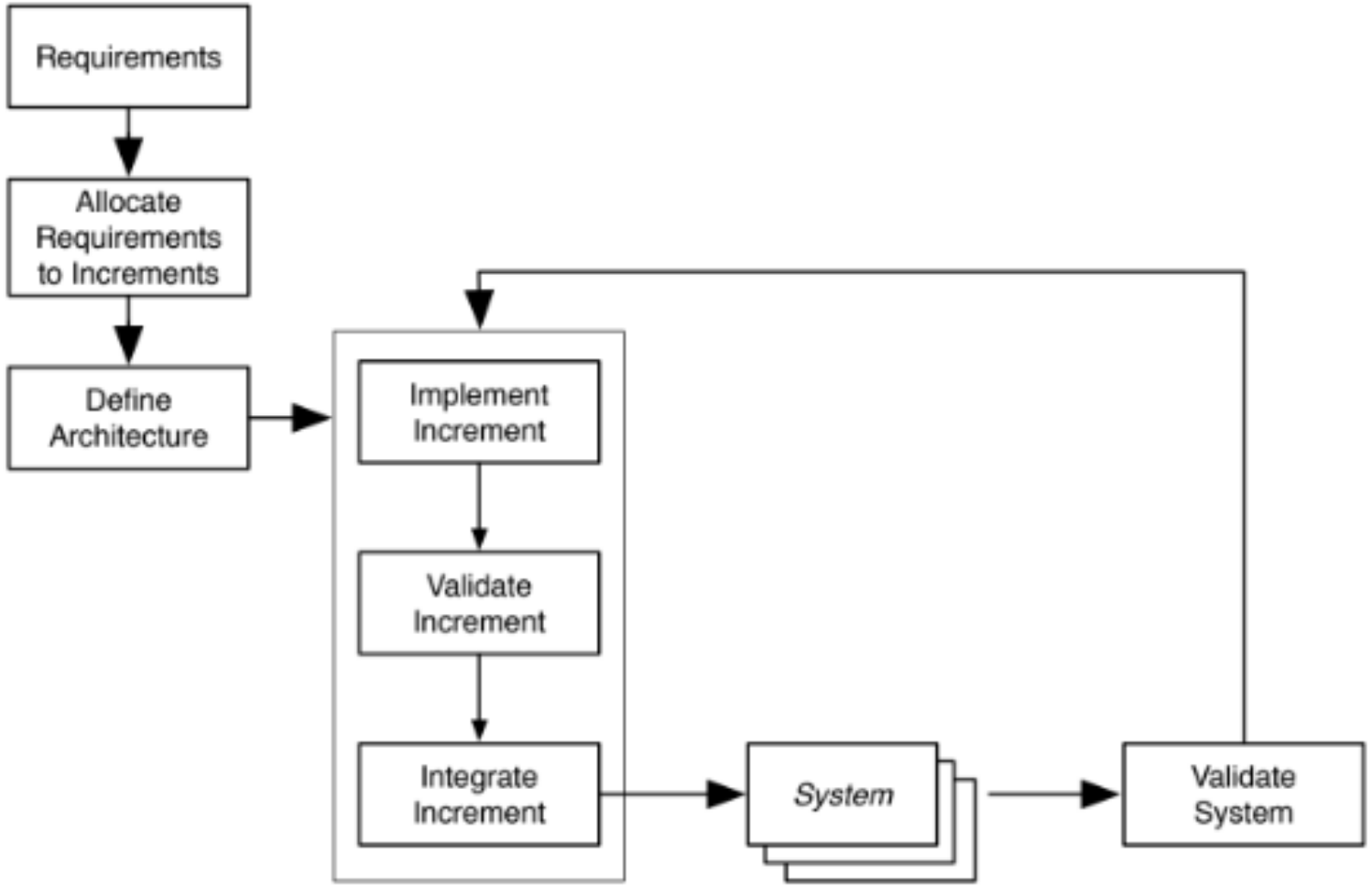
Prototipare

- Aplicabilitate
  - Sisteme de dimensiune mică sau medie
  - Părți ale unor sisteme mai mari
  - Sisteme care nu vor opera un timp indelungat



Modelul incremental

- Dezvoltarea și livrarea sunt împărțite în părți (incrementuri)
- Fiecare increment livrează o parte din funcționalități
- La început sunt livrate incrementuri cu cerințe mai importante
- Cerințele pentru incrementuri ulterioare pot evolua
- Cerințele nu pot fi modificate daca se lucrează la un increment


Requirements
Allocate Requirements to Increments
Define Architecture
Implement Increment
Validate Increment
Integrate Increment
System
Validate System

Avantaje și dezavantaje – Model incremental

Avantaje

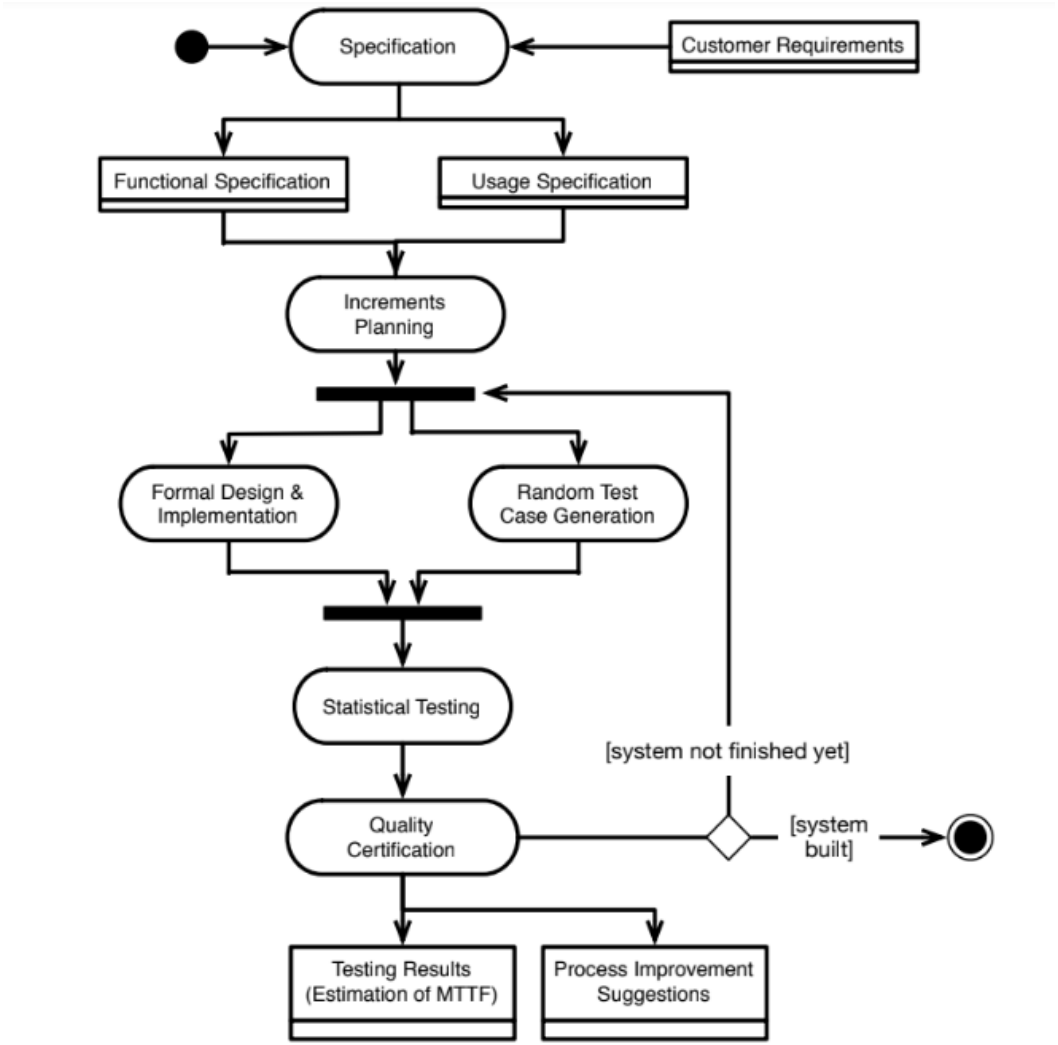
- Se obține un sistem funcțional mai rapid deoarece la icneput sunt livrate cerințele critice
- Incrementurile inițiale servesc drept prototip ajutând pentru elaborarea viitoarelor cerințe
- Risc scăzut de eșec al proiectului
- Cerințele cu prioritatea cea mai amre au parte de mai multă atenție (testare)

Dezavantaje

- Arhitecturile de sistem tind să se degradeze
- Unele incrementuri pot conduce la reimplementări semnificativ

Clean Room Software Engineering

- Proces pentru software fără defecte
- Bazat pe metode formale și teste statistice
- Rapid și corect
- Tip incremental
- Orientat pe prevenirea defectelor, nu pe rezolvarea lor


Specification
Customer Requirements
Functional Specification
Usage Specification
Increments Planning
Formal Design & Implementation
Random Test Case Generation
Statistical Testing
[system not finished yet]
Quality Certification
[system built]
Testing Results (Estimation of MTTF)
Process Improvement Suggestions

Open source development process

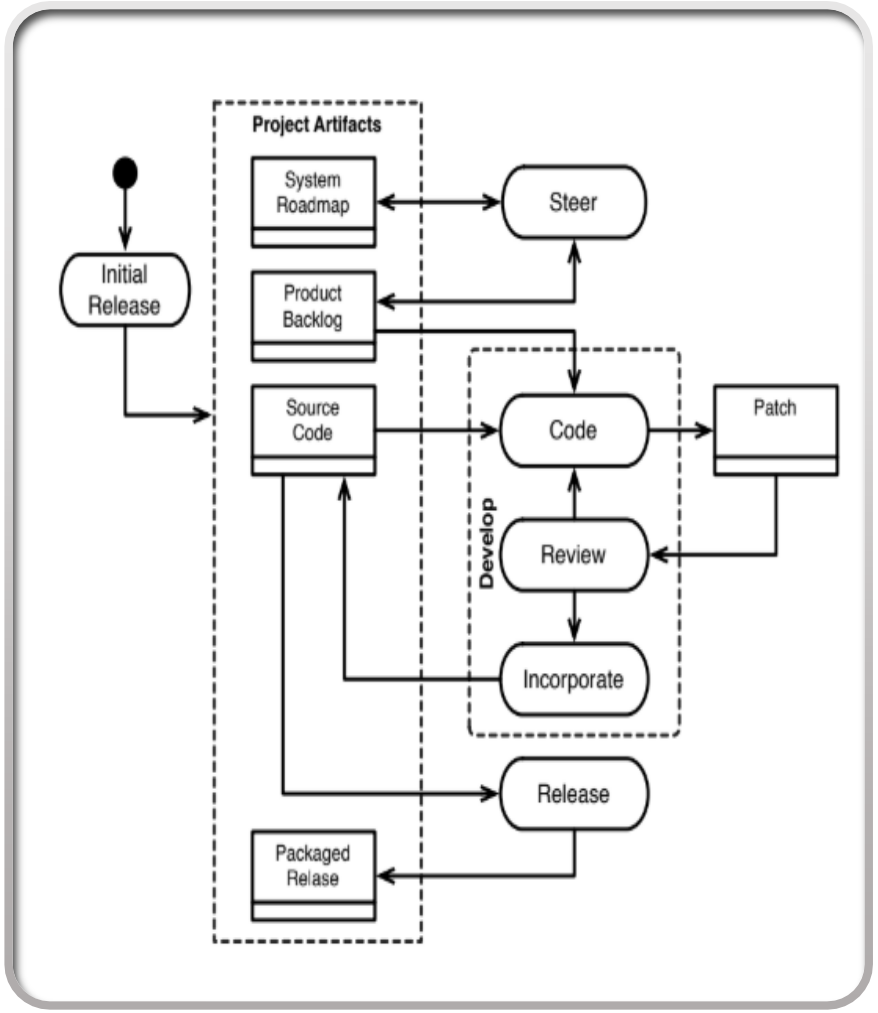
- Proiectul începe de la cod existent, uneori fiind prezente cantități considerabile de cod
- Comunitate de dezvoltatori din diferite zone cu diferite experiențe
- Provocări diverse
  - Implicare în diferite roluri
  - Control al calității
  - Lansare dicil de programat

Open source development process

- O hartă definește evoluția procesului de dezvoltare
- Dezvoltare pe cod neterminat/ cu erori, lansate ca patchuri
- Calitate asigurată de re-evaluări ale codului
- Există două modele
  - Funcționalități fixe (livrat cand e gata)
  - Livrare la date fixe (cu funcționalitățile implementate până la momentul respectiv)

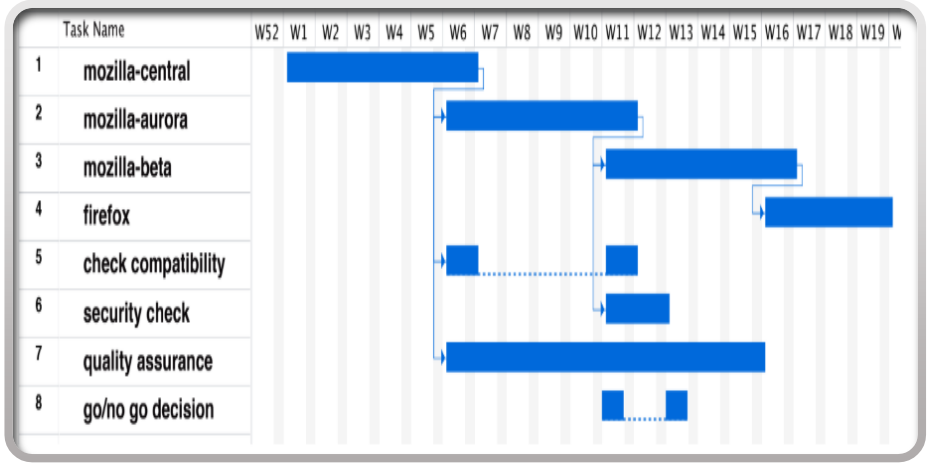
Open Source Development Process

- Primul pas în crearea și dezvoltarea open-source este alegerea unei platforme și definitivarea acestei alegeri.
- Ca pas următor , avem aducerea tool-urilor necesare pentru developer(precum IDEs , licențe și tot ce ține de un mediu de dezvoltare propice).
- Urmează pasul dezvoltării software propriu-zise , unde se face develop , se testează , se încorporează servicii și se integrează alte aplicații cu cea curentă. Există două modele:
  - funcționalități care sunt livrate când sunt terminate(dă o oarecare libertate)
  - cu deadline și livrare la o data fixă(se apropie de modelul AGILE)
- Se fac și reviews care conduc mai departe la patch-uri , o versiune oarecum mai bună decât cea curentă.
- Succesul este constituit de cât mai mulți contribuitori, care aduc funcționalități noi.



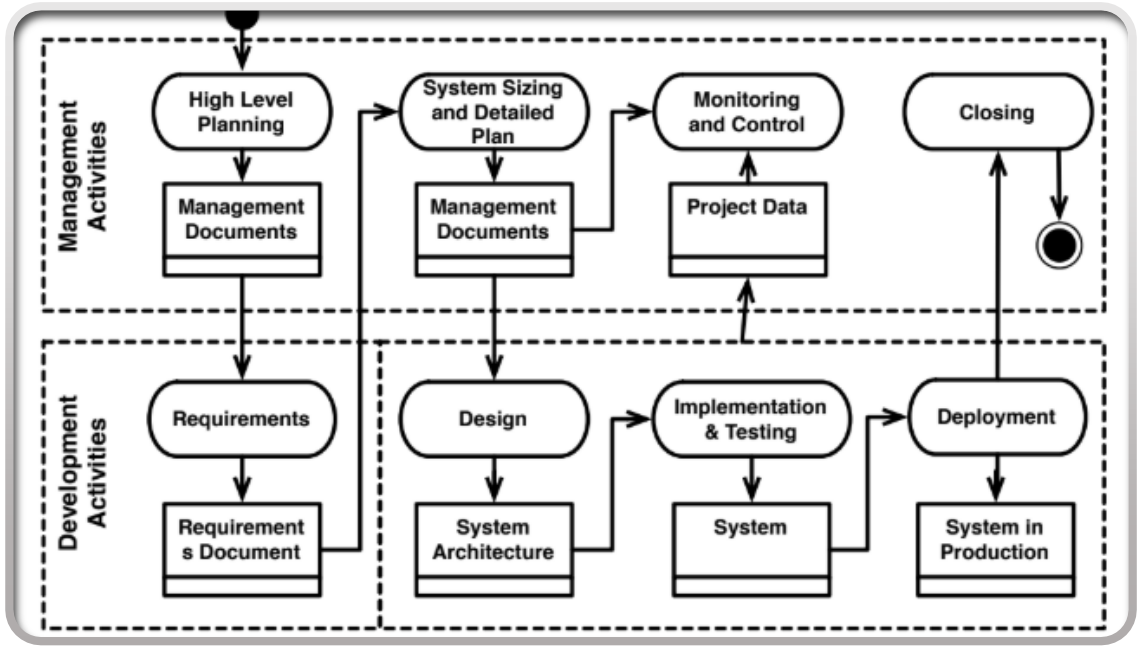
Firefox Release Schedule

- Putem observă în decurs de un an release-urile pentru firefox , dar și dependințele între acestea.
- Astfel firefox-central a fost lansat primul, după care se verifică funcționalitățile.
- Se verifică compatibilitatea cu alte aplicații , calitatea produsului , dar și versiunea livrată că urmare a acesteia, fiind vorba de aurora.
- În pasul următor se verifică din nou factorii de la pasul anterior pentru versiunea curentă . După cum putem observă, mozilla a intrat după mai multe release-uri în beta, fiind o versiune înaintea ultimelor release-uri
- Astfel, ultima versiune și cea stabile se poate consideră firefox, care derivă din versiunea beta.

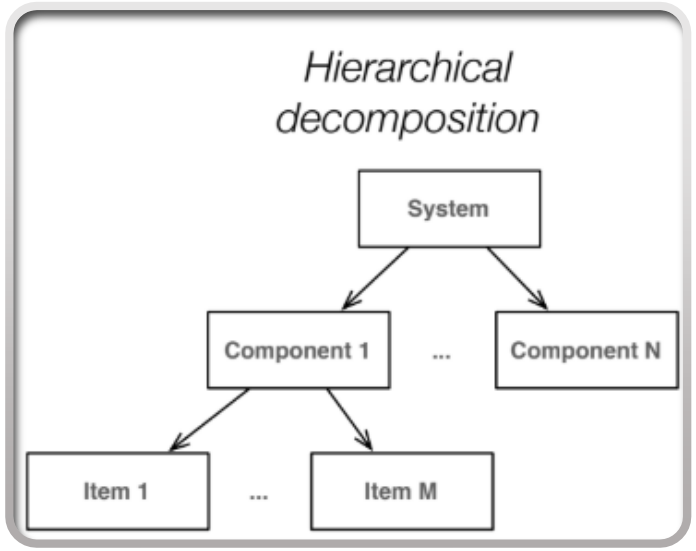


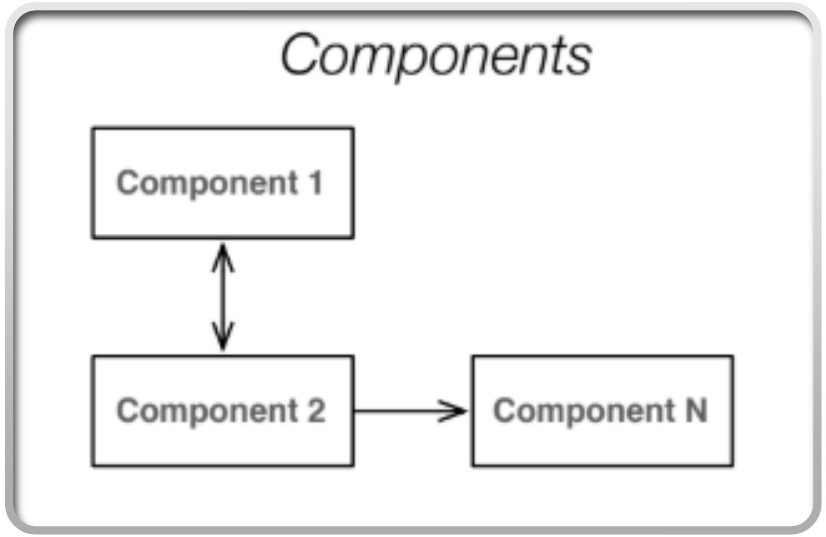
Additional Concerns

- Apar și anumite îngrijorări datorită modului mai greu de gestionare a aplicațiilor open-source între management și activitățile de dezvoltare în șine.
- Trebuie monitorizat controlul și flow-ul aplicației , design-ul trebuie să corespundă cu planul în detaliu generate de management.
- Arhitectura sistemului trebuie să fie în concordanță cu documentul generat prin activitățile de management.



Organizarea și descompunerea sistemului





- Sistemele complexe sunt structurate în mai multe componente elementare.
- Putem avea structură de arbore
- Structură ierarhică, în care avem dependințe între componente sub formă de fii și părinte (putem avea oricâte componente). Este un arbore cu o lista de fii.
- Putem avea sisteme cu dependințe liniare între componente

Strategii de organizare

Top-Down

- Se abordează aplicația în componente mari, din care se deduc componentele mai mici, adică fii unei componente ce acoperă o funcționalitate mai mare.
- Frunzele arborelui de componente și dependențe ar trebui să fie componente elementare, sau chiar funcții, fiind la cel mai jos nivel al arborelui.

Bottom-Up

- Abordarea bottom-up presupune dezvoltarea de componente mici, după care, mai departe acestea trebuie integrate între ele pentru a obține componentele mai mari de care avem nevoie, funcționalități integrate cu alte funcționalități, independente sau nu între ele.
- Se bazează pe obținerea rădăcinii din Frunze. Se pornește de jos în ierarhie și se ajunge la părinți.

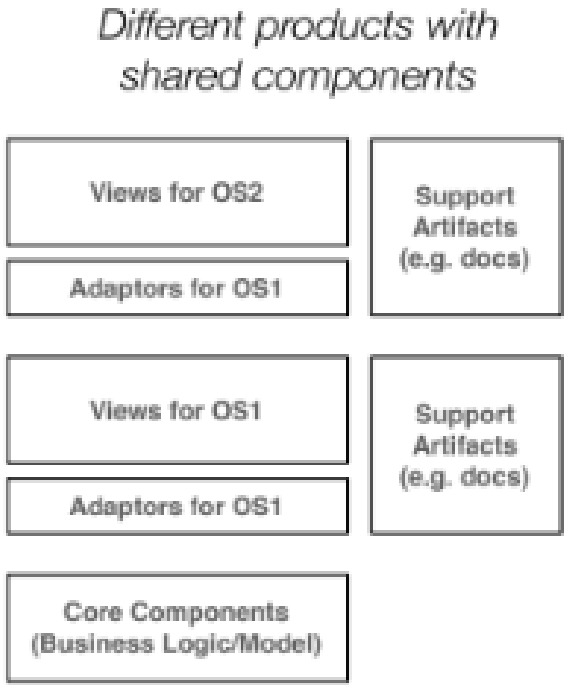
Familii de produse

Probleme

- Sisteme de operare
- Diferență dintre platforme
- Device-uri(probleme ce se leagă de inerfata cum ar fi setări de bază(capacitate , putere))
- Tehnologiile disponibile

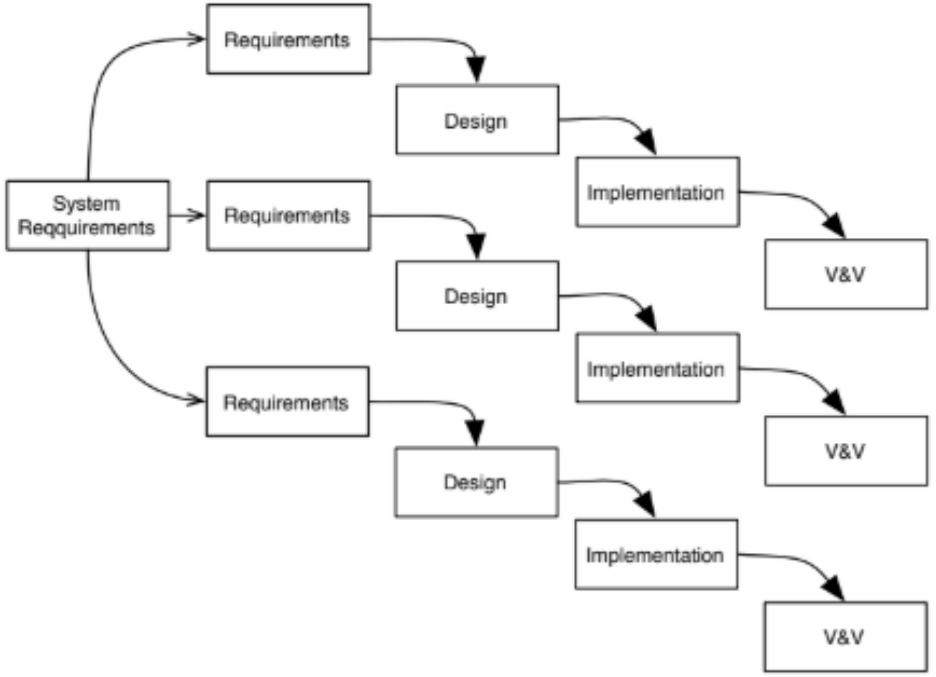
Abordări

- Folosirea unei aplicații cu o singură arhitectură
- Componente adaptive (design care să permită să rescrii componente ușor , integrarea să fie soft)
- Produse și aplicații diferite cu componente commune
- Dezvoltare independența cu interfețe comune sau API-uri.
- API-ul este separate de user interface și poate fi considerate o componența de server ce nu cauzează probleme dacă este izolată într-un mod rațional.



Consecinte

- Pentru sistemele complexe dezvoltarea de software trebuie făcută într-un mod rațional în așa fel încât:
  - Design-ul aplicației trebuie să fie făcut astfel încât componentele să se integreze ușor între ele(integrare soft), componentele să poată fi rescrise ușor.
  - Dezvoltarea software e compusă din procese care rulează în paralel
  - Dependențele între componente au de cele mai multe ori o arie ierarhică , ceea ce da senzația de recursive între activitățile dezvoltării software.
  - Aplicația trebuie încadrată în cererile de sistem



Exercițiu

Vrem să dezvoltăm o aplicație web. Această este definită că o bibliotecă online, cu toate funcționalitățile necesare acestui subiect.

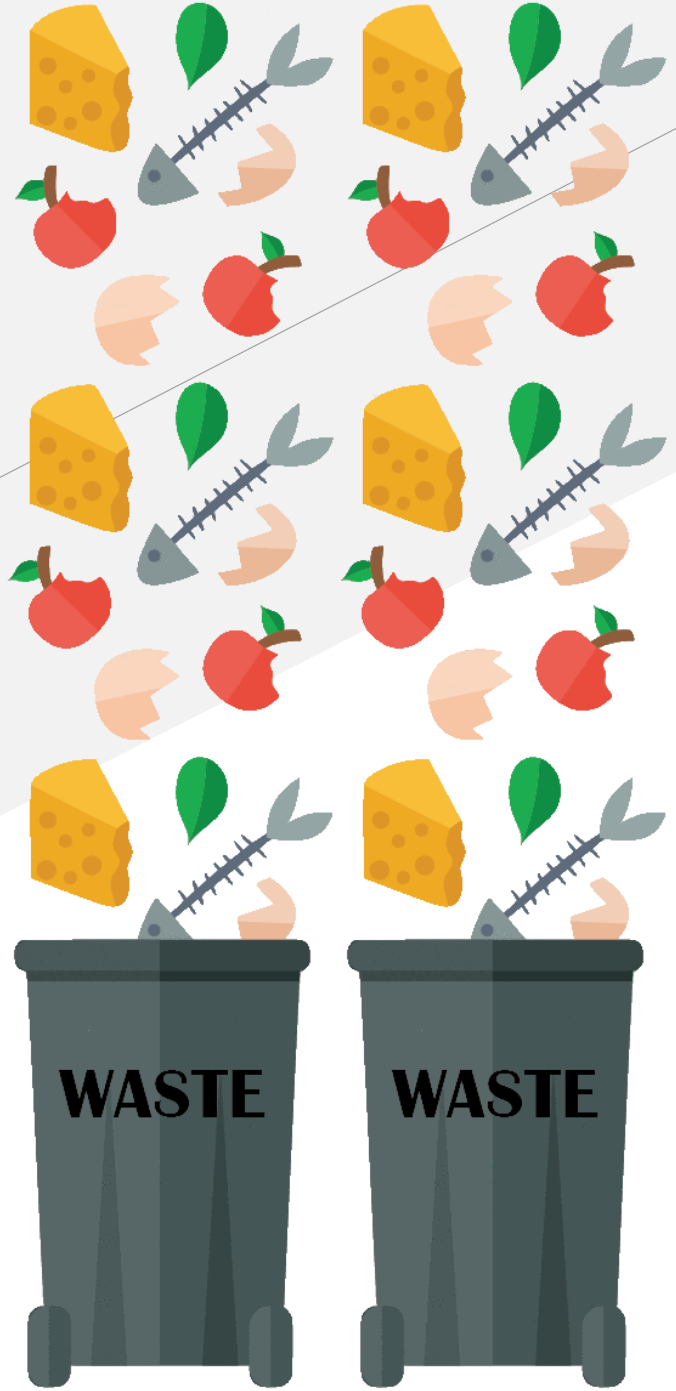
- Ce activități avem ?
- Ce procese avem ?
- Ce produse avem ?
- Cum folosim API-ul?
- Ce fel de componente avem?
- Ce dependențe există între acestea?
- Cum integrăm componentele?

Agile Software Development

Anghel Bogdan-Ștefan
Carpen Diana-Maria
Țuicu Daniel-George

Outline

- Despre risipă
- Introducere în Metodologia Agile
- Procese de dezvoltare Agile
  - DSGM
  - eXtreme Programming
  - Scrum



Scurtă discuție despre risipă

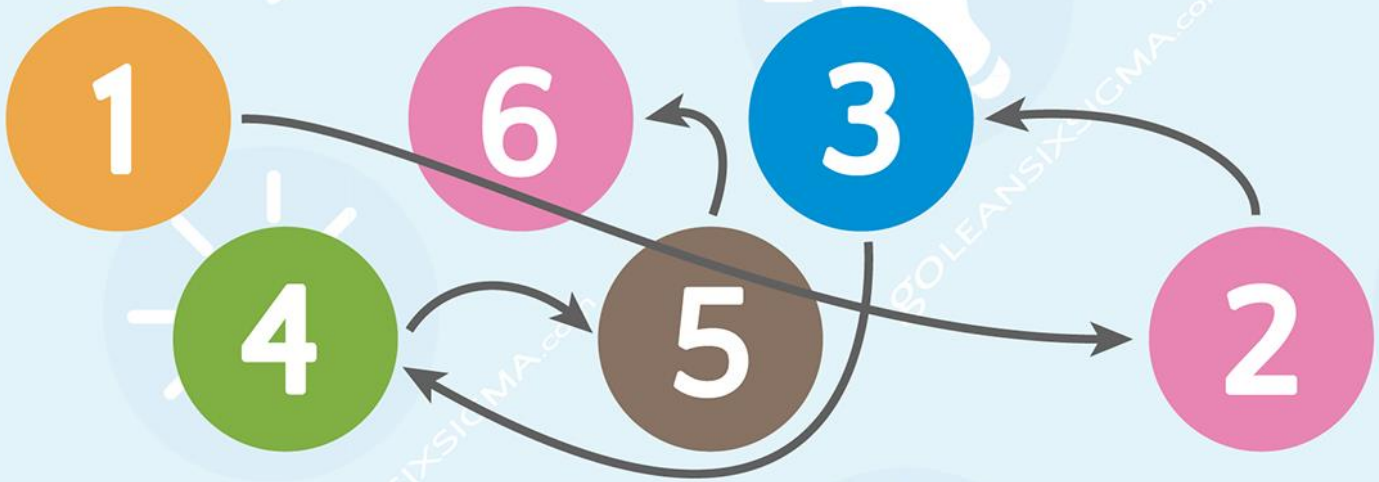
LEAN

Ce este LEAN?

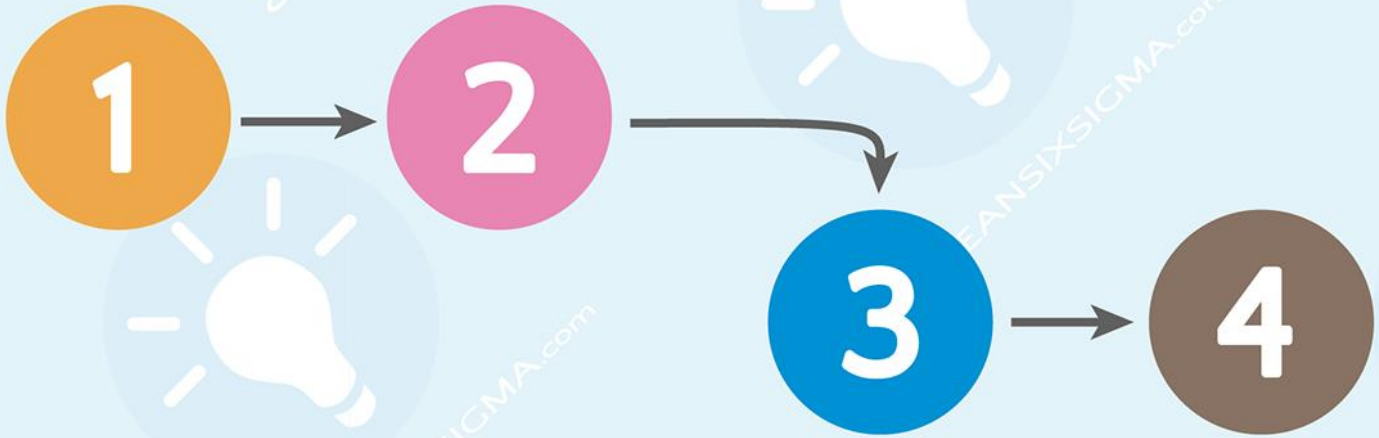
- Lean este o abordare sistematică prin care se identifică și se elimină risipa.
- Risipa reprezintă activitățile care nu produc valoare
- Valoare = ceea ce clientul este dispus să plătească

Lean: Process Before & After

PROCESS BEFORE LEAN:



PROCESS AFTER LEAN:



goLEANSIXSIGMA.com

© Copyright 2016 GoLeanSixSigma.com. All Rights Reserved.

FOURPRINCIPLES

THE LEAN MANAGEMENT EXPERTS

Get Lean in 90 Seconds

video by Quality Schnallity Inc.

LEAN

Tipuri de risipă în LEAN

- MURI - suprasolicitare

Muri

- MURA - dezechilibru, neregulat, inegalitate

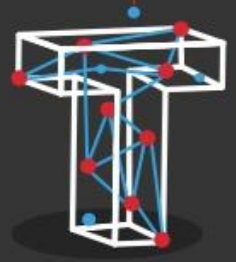
Mura

- MUDA - activități care nu aduc valoare

Muda

TRANSPORTATION

Refers to the relocation of finished or semi-finished products liable to deterioration, loss, longer lead times and additional workloads.



INVENTORIES

Refers to intermediate storage and to inventories made during the process. These do not contribute to the conversion of the product and generate additional costs that do not create added value.

MOTION

Refers to the pointless relocation of human resources and equipment during the manufacturing process that can have an impact in terms of pointless deterioration, wear and safety.

WAITING

Refers to the waiting time for an activity or task to be undertaken when the products or services are not in the process of transport, manufacture or conversion.

OVER-PROCESSING

Refers to pointless activities and tasks and/or those that do not create added value for the client, and to practical tools (software, procedures, work instructions) that are more specific, more complex or more costly than necessary.

OVER-PRODUCTION

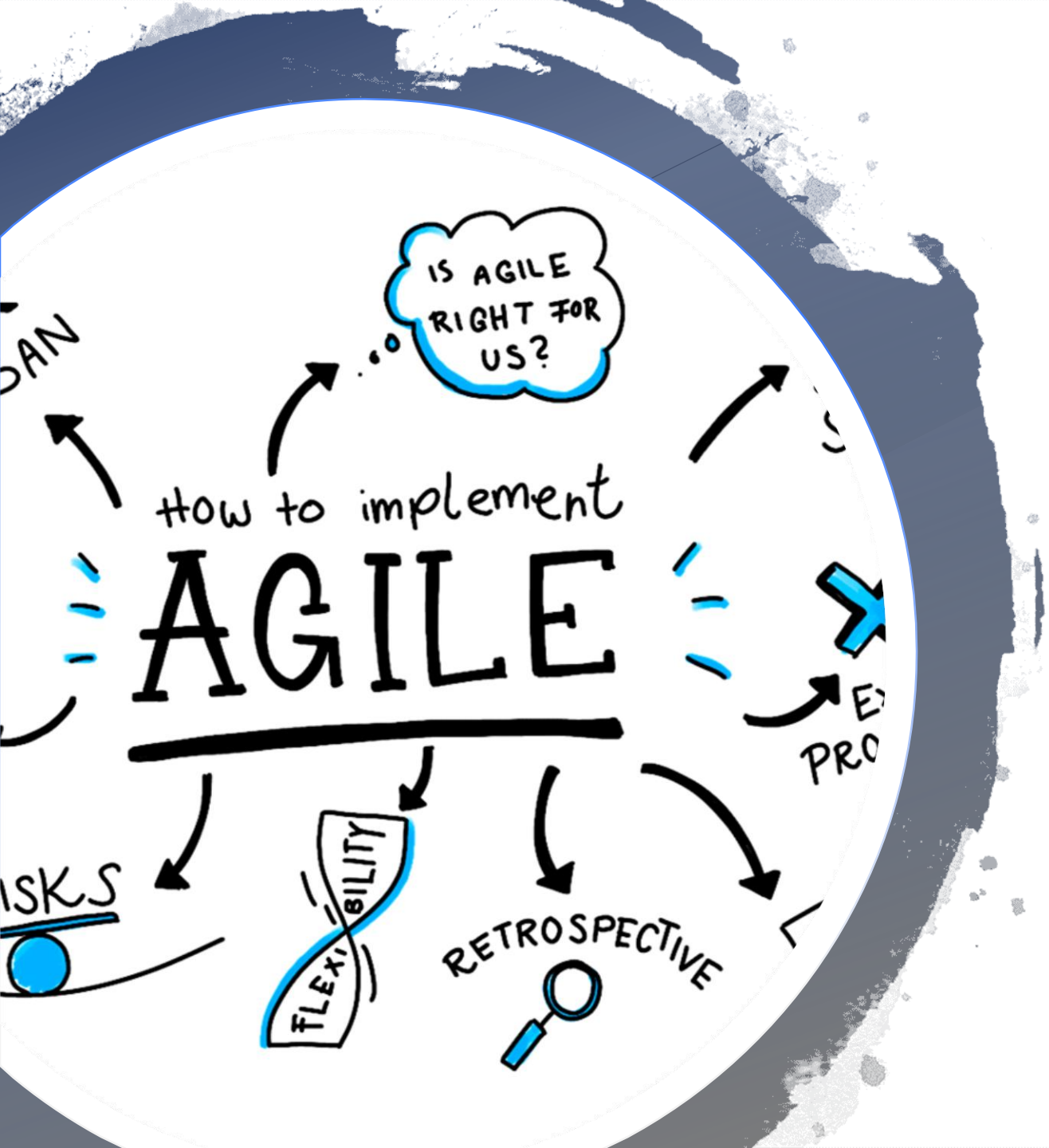
Refers to the overproduction of manufactured products in relation to client demand / orders. In the field of IT Operations, there is also the issue of underproduction.

DEFECTS

Every time defective and rejected finished and semi-finished products occur, this means additional costs to rework the products, destroy defective products, reprogram production etc.

Cauzele risipei în dezvoltarea software

- Software-ul nu poate fi măsurat (nu este material/palpabil)
- Flexibilitate în adăugarea de noi funcționalități
- Complexitate
- Tehnici de dezvoltare noi, care nu sunt perfecționate



INTRODUCERE ÎN

Metodologia Agile

De ce Agile?

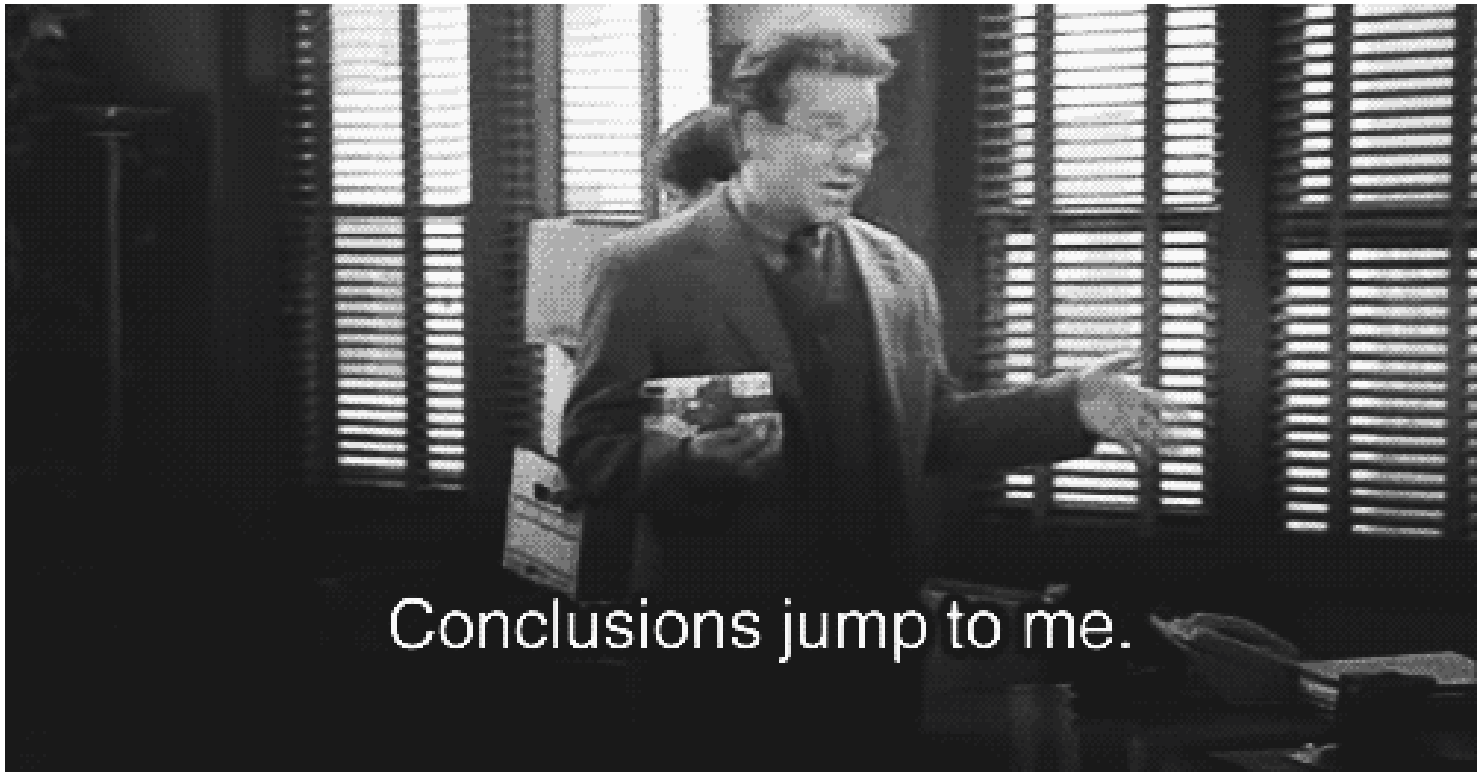
- **Agile Software Development:** un grup de metodologii de software development bazate pe următoarele principii:
  - Un proces de management care încurejează inspectarea și adaptarea frecventă
  - Lucru în echipă și self organization
  - Livrarea rapidă de software de calitate
  - O abordarea care unește cerințele clienților cu rezultatul software și cu obiectivele companiei

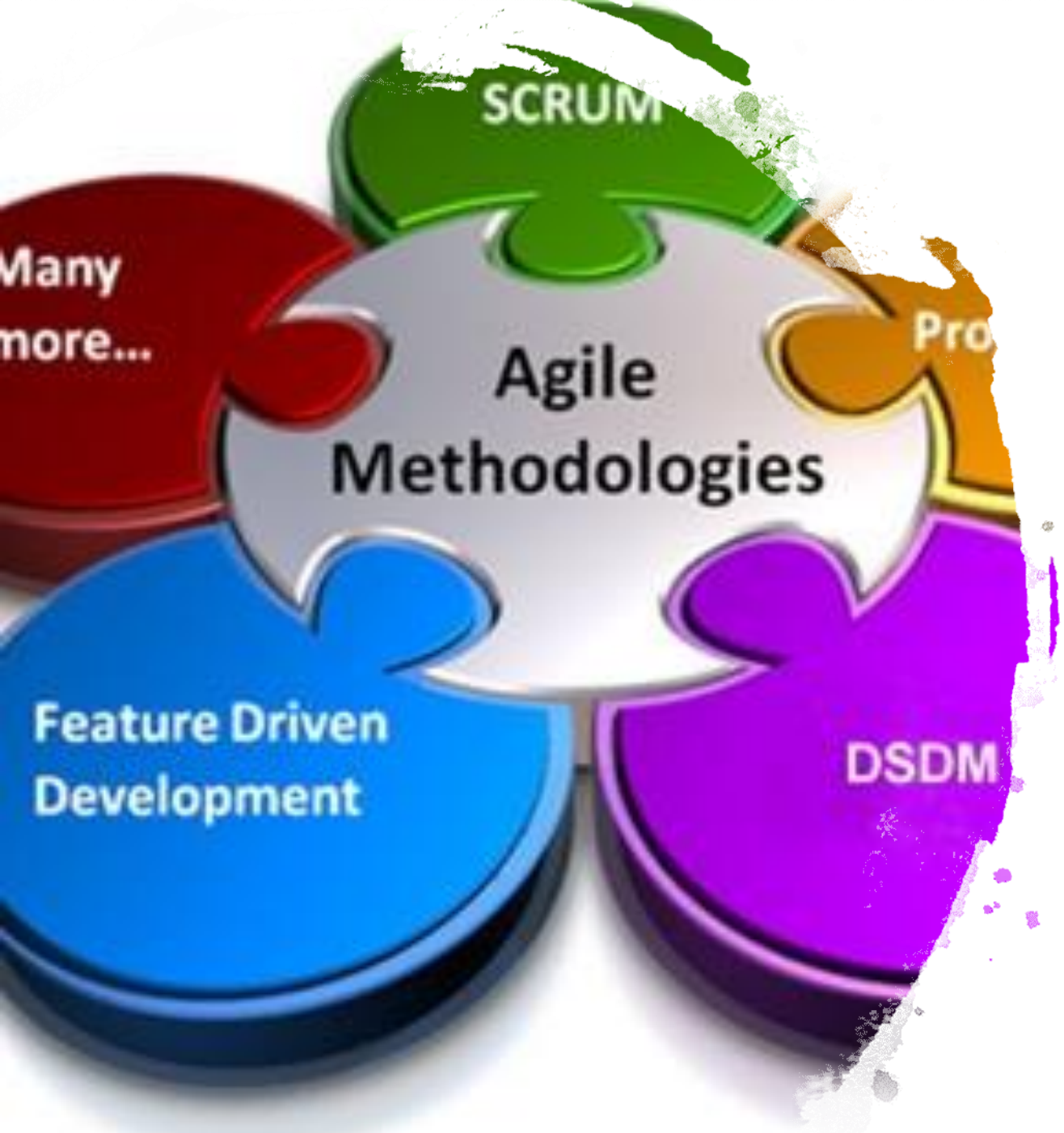
Ce consecințe sunt

- Clienți safisfăcuți
- Software funcțional livrat rapid
- Software-ul funcțional este modul de a măsura progresul
- Schimbări pe parcursul dezvoltării = no problem
- Simplitate

1 2 3 4 5 6 7 8 9 10
A
B
C
D
E
F
G
H
I
J
XTH 73
XAP 49
XJU 80
VCH

Concluzii Agile vs Waterfall





Procese de dezvoltare Agile

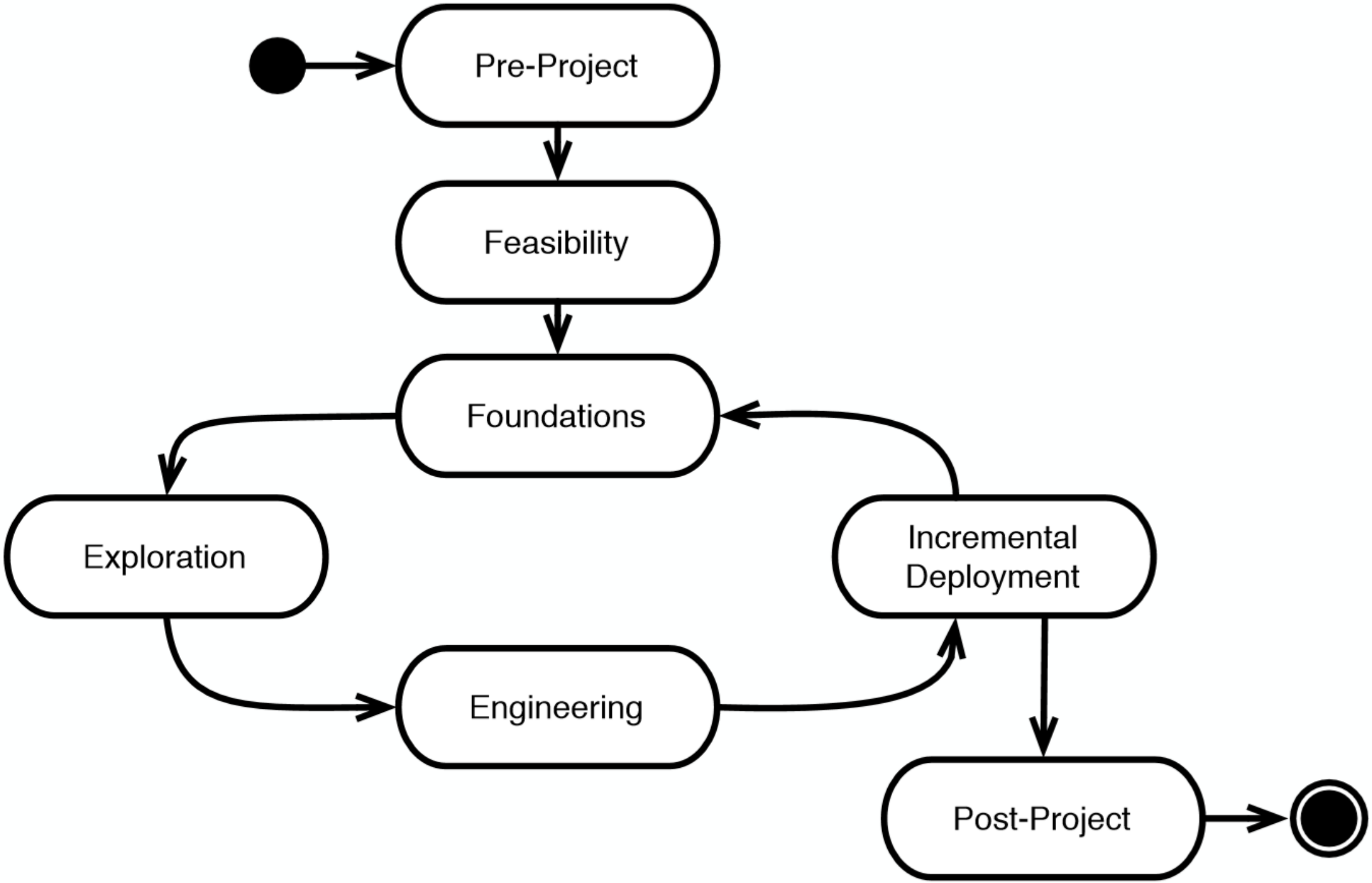
Dynamic System Development Method

- Metodologie Agile
- Se axează pe întregul ciclu de viață al proiectului
- Abordare iterativă, incrementală
- Bazată pe RAD (Rapid Application Development) 1950

Dynamic System Development Method

Practici utilizate

- Facilitated Workshops
- Modelling and Iterative Development
- MoSCoW Prioritisation
- Time boxing


Pre-Project
Feasibility
Foundations
Exploration
Incremental
Deployment
Engineering
Post-Project

Extreme Programming

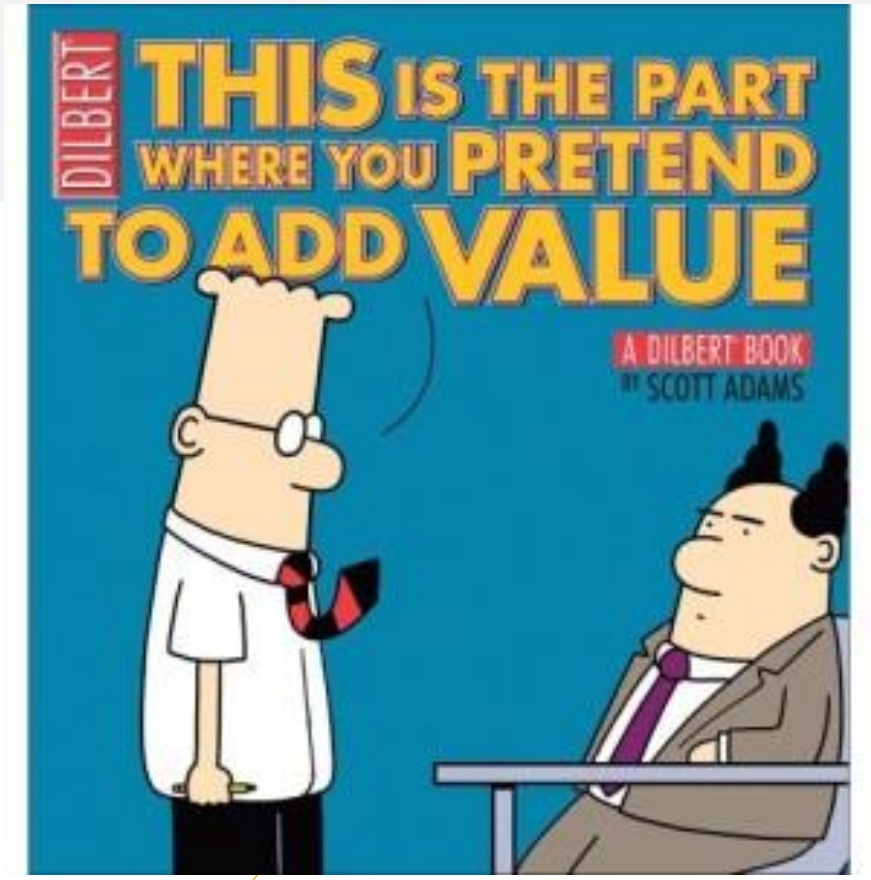
Extreme Programming

Definiție

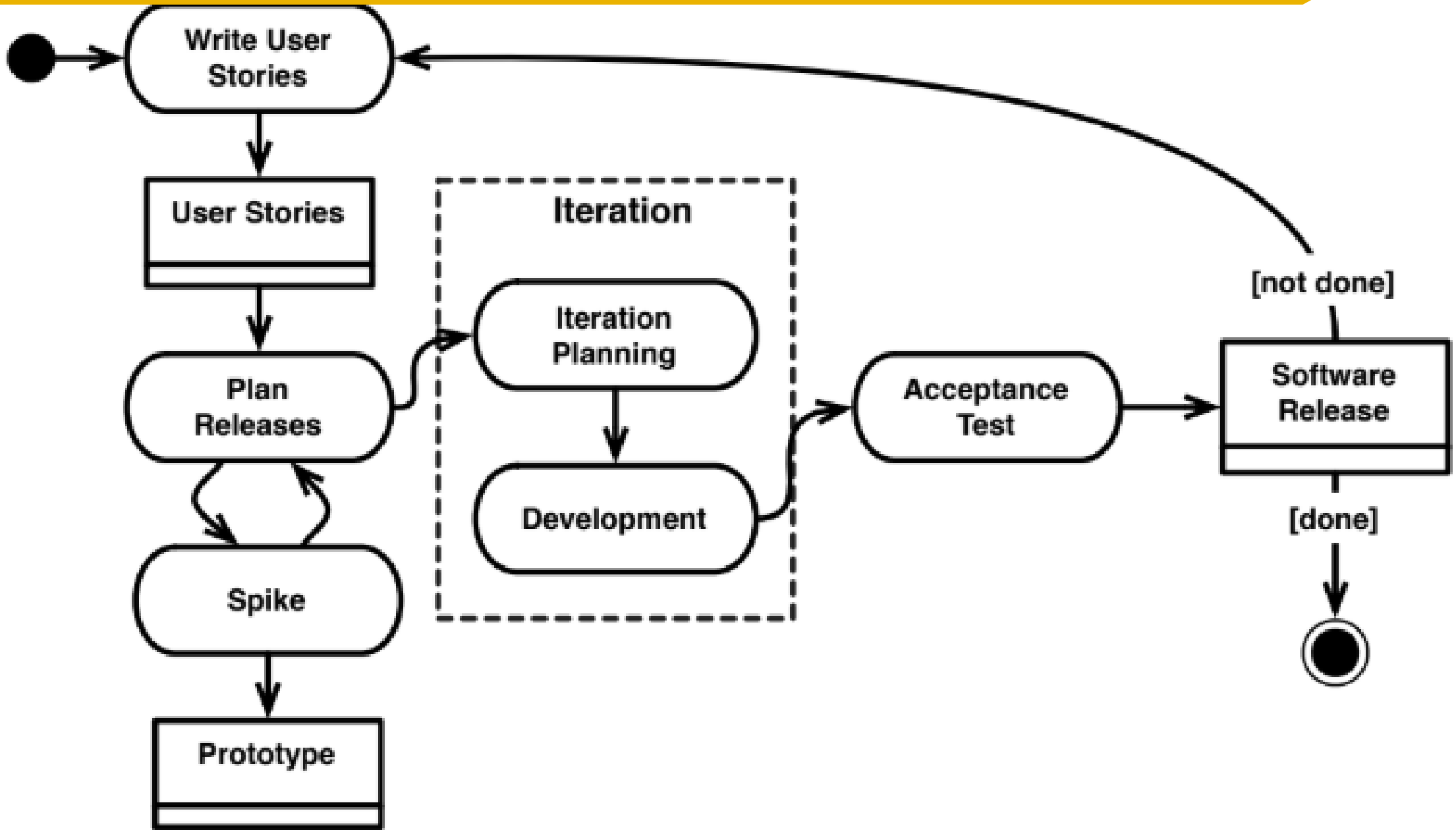
- XP este un Framework de dezvoltare software agile care are ca scop realizarea de software de calitate și îmbunătățirea mediului în care lucrează echipa de dezvoltare

Valori

- Comunicare
- Feedback
- Simplitate
- Curaj
- Respect



Procesul XP


Write User Stories
User Stories
Plan Releases
Spike
Prototype
Iteration
Iteration Planning
Development
Acceptance Test
[not done]
Software Release
[done]

Planificarea XP

- User stories
- Planificarea termenelor creează programul de lucru
- Modificări mici și frecvente
- Proiectul este împărțit în iterații
- Fiecare iterație începe cu planificarea iterației

XP Management

- Asigurarea unui mediu de lucru optim
- Ritmul potrivit de dezvoltare
- Discuții zilnice (daily standups)
- Măsurarea progresului
- Realocarea sarcinilor
- Change the rules

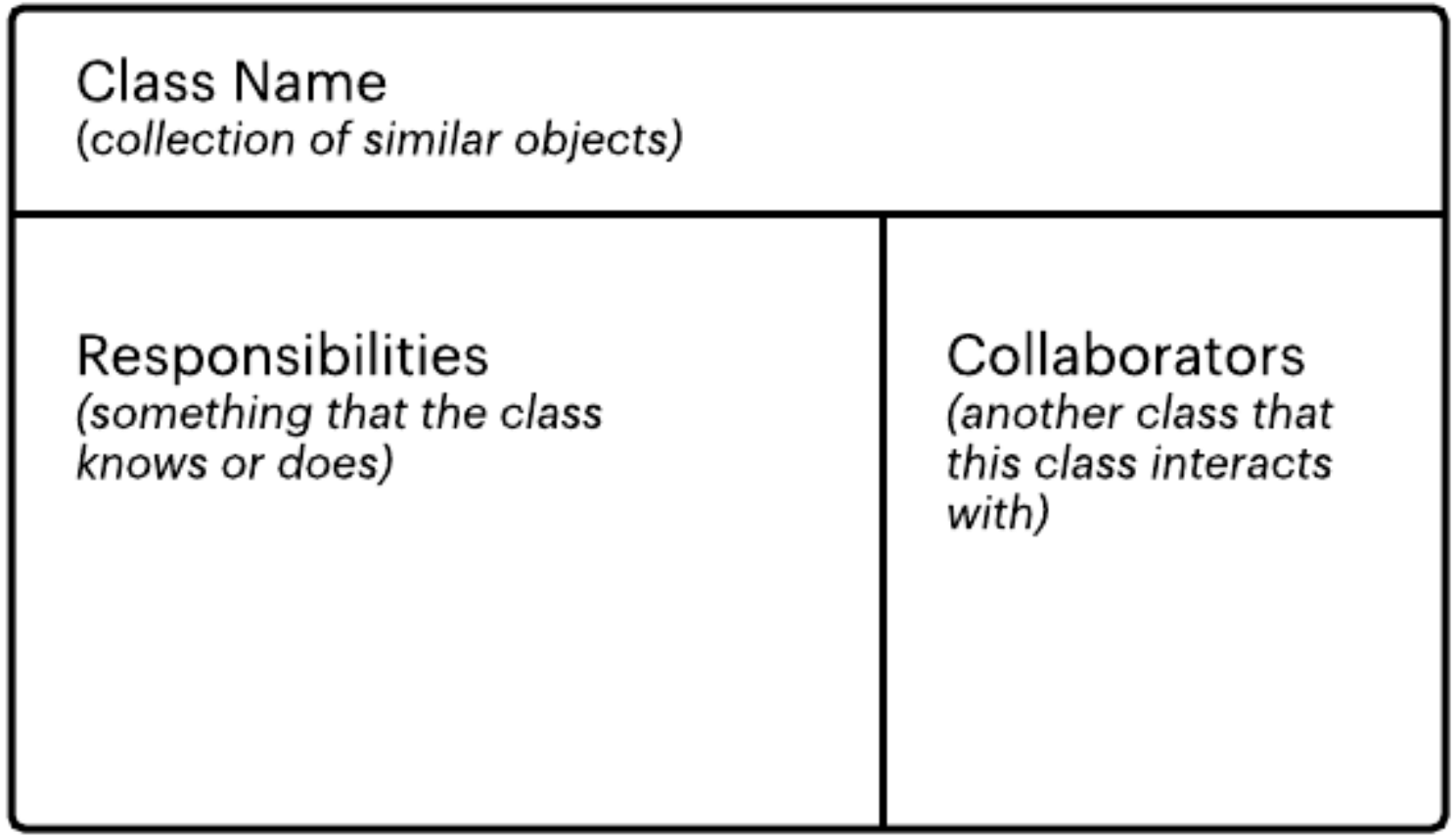
XP Design

- Simplitate
- Sistemul de metafore
- Soluții Spike
- Nicio funcționalitate prea devreme
- Refactor

XP Design

continuare

- Utilizare cartonașe CRC (class-responsibility-collaboration)



XP Coding

- Clientul este disponibil oricând
- Standarde de coding
- Unit Testing first
- Tot codul scris -» Pair programming
- Integrarea se face des și doar de o echipa la un moment dat
- Integrare pe un computer dedicat
- Collective ownership

XP Testing

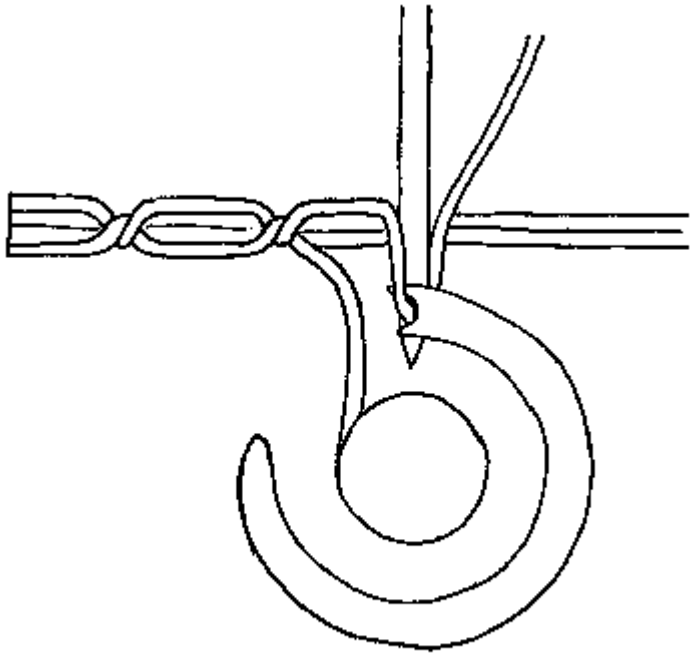
- Tot codul este testat
- Tot codul trebuie sa treacă toate testele
- Teste create la găsirea unui bug
- Teste de acceptare rulate des; scor publicat

Pair Programming

- Doi programatori lucrează la același calculator
- Câteva considerente:
  - Rezultă aproximativ 15% mai puțin cod decât 2 programatori solo
  - Code review continuu => design mai bun, mai puține erori
  - Fiecare învață de la celălalt (technical skills++)

Simple design that works

- Implementarea celor mai simple features care funcționează
- Trec toate testele
- Nu există cod duplicat
- Cât mai puține clase și metode



Refactorizarea

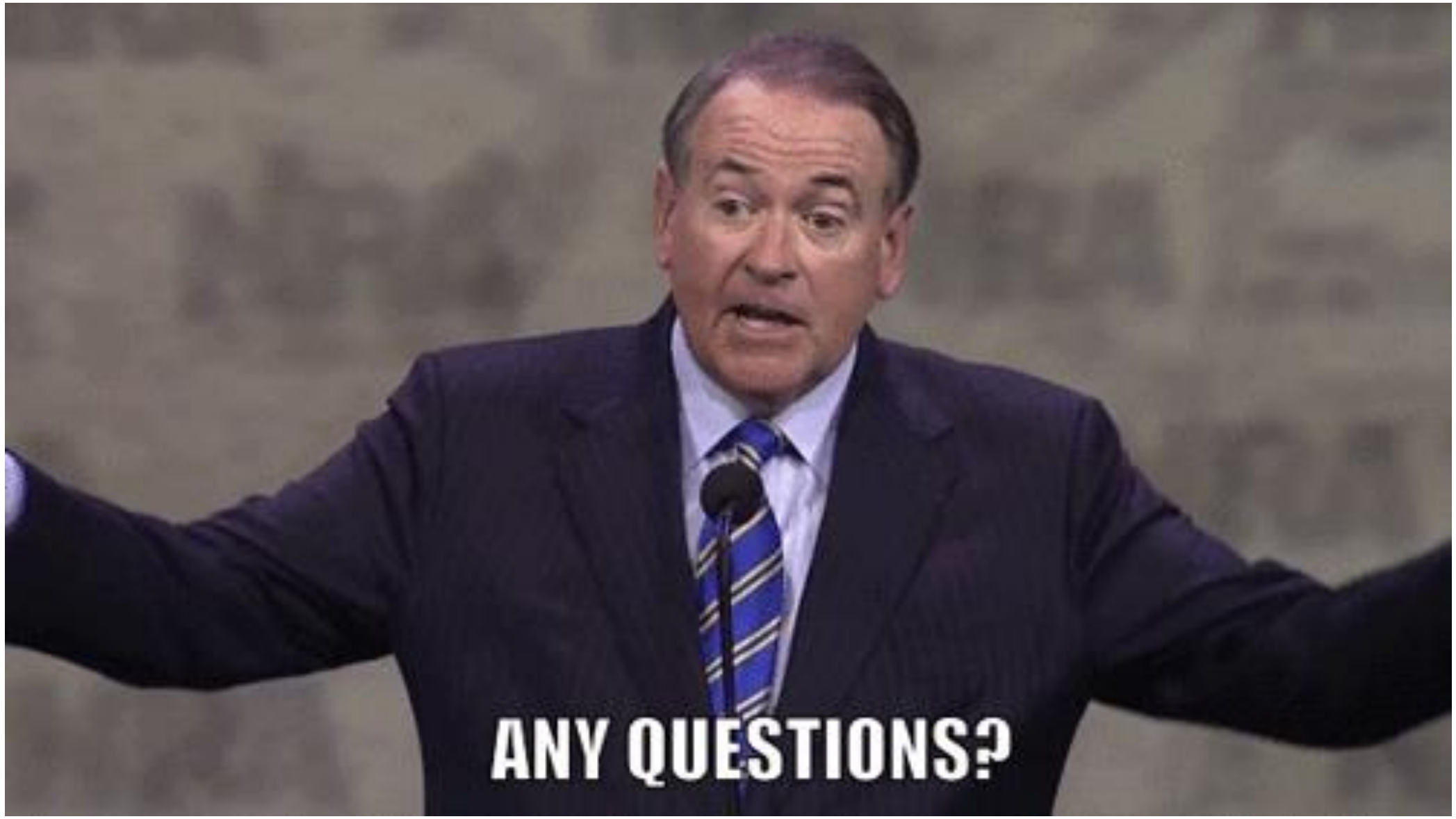
- Sofware design devine o activitate zilnică pentru toată lumea
- Îmbunătățire continuă a calității codului
- Consecințe:
  - Dezvoltare software rapidă
  - Codul poate fi schimbat cu ușurință

De ce XP?

- Light-weight: disciplină fără birocrație
- În condiții de stres, oamenii aleg de obicei calea mai ușoară
- XP are beneficii atât pe lungă durată cât și pe scurtă durată
- Codul servește drept documentație
- XP is fun

Critici XP

- Prea multe modificări = costuri ridicate
- Generează stres
- Cerințele se modifică foarte des => poate rezulta in muncă ineficientă (staring over)
- Poate fi foarte dificil de implementat din cauza lipsei de documentație si structură oferite de la început


ANY QUESTIONS?

THANK
YOU

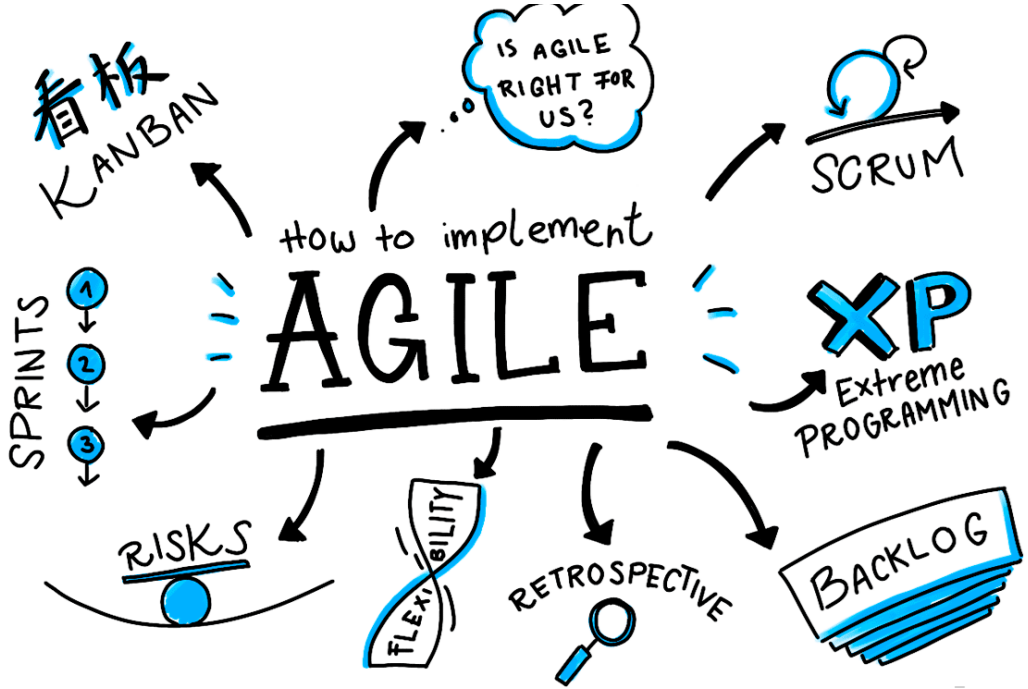
Cursul 13

Bouleanu Oana-Bianca
Preda Andreea-Cristina
Grupa 343C5

Cursul 13

Metodologia Scrum

Recap. de data trecuta..



Ce este Agile?

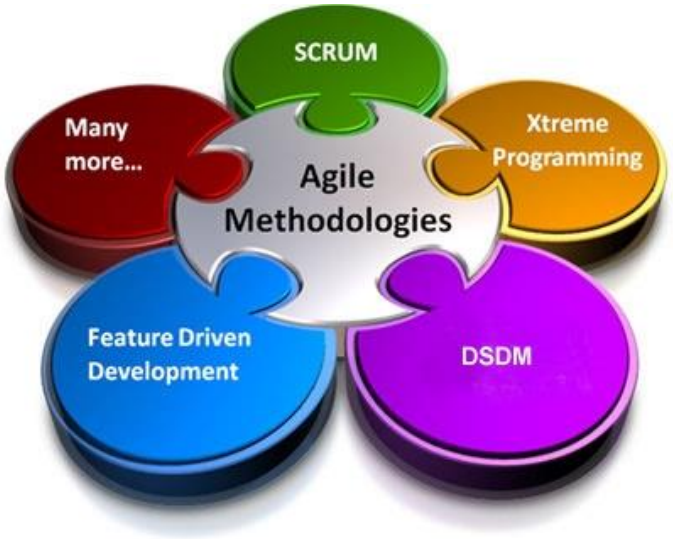
Metodologie de management a proiectelor ce incearca sa micsoreze riscurile de dezvoltare si timpul de executie prin implementarea proiectelor in forma foarte flexibila si interactiva.

Metodologii care deriveaza din Agile

AGILE exista in multe feluri:

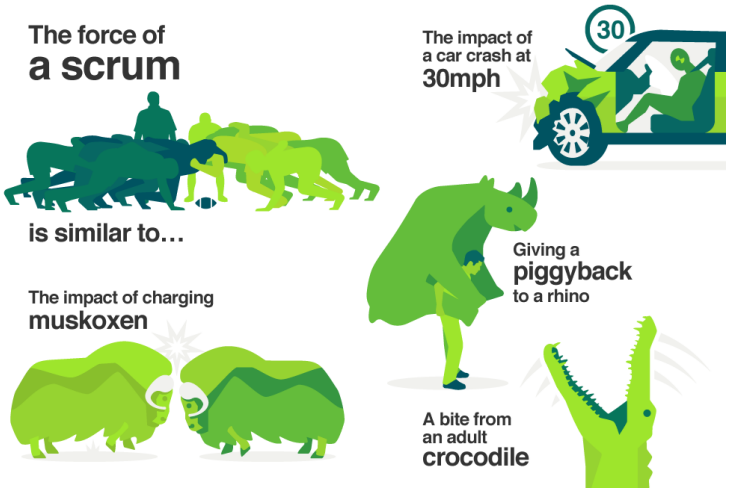
- XP
- SCRUM
- DSDM
- Crystal
- Feature Driven
- Etc.

Toate acestea folosesc principiile de baza ale filozofiei AGILE, dar o implementeaza in moduri diferile.



Abordarea scrum

O abordare de echipa in care toti membrii se indreapta, impreuna, catre acelasi scop, stabilind ritmul si adaptandu-se schimbarilor, similar a ceea ce se intampla la rugby.



Principii scrum

1. Instabilitate: scopuri extensive si directii strategice generale
2. Echipele de proiect se organizeaza singure
3. Faze de dezvoltare suprapuse
4. Multi-invatare astfel incat echipa poate invata atat din surse interne, cat si externe si sa se adapteze repede la conditii si medii schimbatoare
5. Control subtil: directionarea proiectului fara a interveni prea mult (ex: selectarea oamenilor potriviti pentru fiecare rol, crearea unui mediu deschis, tolerarea si anticiparea greselilor)
6. Transfer de invatare organizational prin asigurarea transferului si reutilizarii cunostintelor acumulate in urma unui proiect in cadrul altui proiect

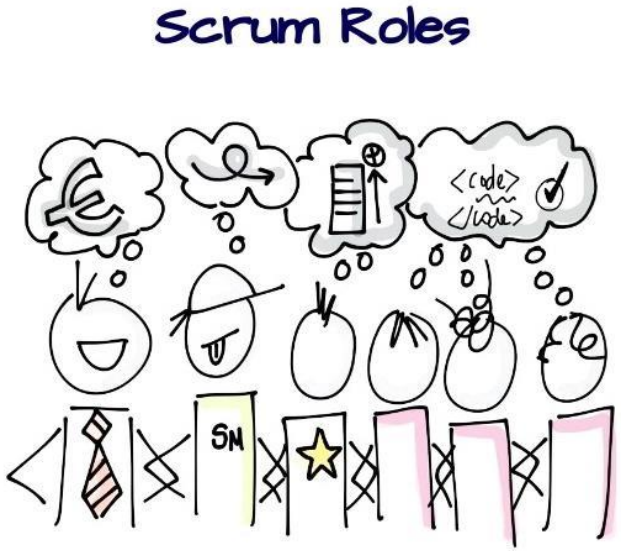
Scrum a fost folosit de catre:

- Microsoft
- Yahoo
- Google
- Electronic Arts
- IBM
- Lockheed Martin
- Philips
- Turner Broadcasting
- Oce
- Siemens
- Nokia
- Capital One
- Nielsen Media
- First American Real Estate
- BMC Software
- Ipswitch
- John Deere
- Lexis Nexis
- Intuit
- Sabre
- Salesforce.com
- Time Warner
- BBC

Scrum a fost folosit pentru:

- Software comercial
- Aplicatii in-house
- Aplicatii la comanda
- Proiecte cu pret fix
- Aplicatii financiare
- Aplicatii certificate ISO 9001
- Sisteme inglobate (embedded)
- Sisteme cu cerinte de disponibilitate 99.999%, 24x7
- Programul Joint Strike Fighter
- Dezvoltarea de jocuri video
- Sisteme aprobate de FDA, life-critical
- Software de control al satelitilor
- Site-uri web
- Software pentru dispozitive mobile
- Telefoane mobile
- Aplicatii de switching in retele
- Aplicatii ISV (vanzatori independenti de software)
- Unele din cele mai mari aplicatii utilizate

Rolurile SCRUM



Stakeholders, Scrum Master, Product Owner, Development Team

Rolurile scrum intr-un singur slide

- Scrum Master
- Facilitator, se asigura ca Scrum este aplicat si ca echipa poate opera si lucra
- Protejeaza echipa de interferente externe

- Detinatorul produsului
- Defineste caracteristicile produsului, decide prioritatile, accepta sau refuza rezultatele activitatii

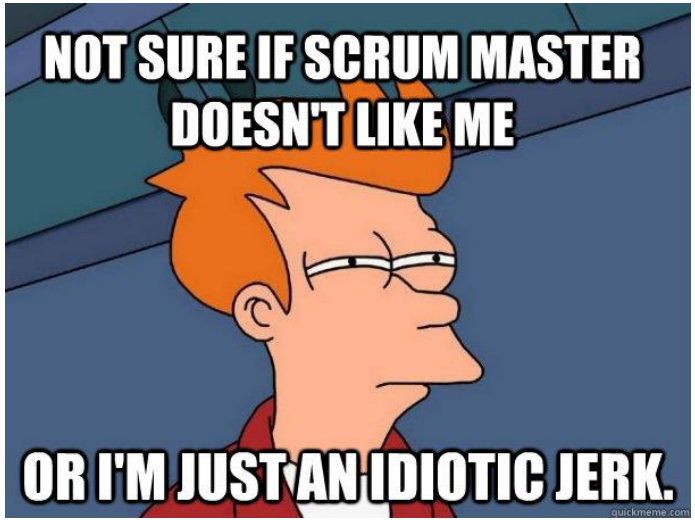
- Echipa
- Se organizeaza singura, ideal lucreaza full-time, restransa (5-9 membri)

Scrum Master

- Reprezinta managementul pentru proiect
- Responsabil de implementarea valorilor si practicilor Scrum-ului
- Elimina impedimentele
- Se asigura ca echipa este complet functionala si productiva
- Asigura cooperarea apropiata intre toate roulrile si functiile
- Protejeaza echipa de interferentele externe

Detinatorul proiectului

- Defineste proprietatile proiectului
- Decide data lansarii si continutul
- Responsabil de profitabilitatea produsului (ROI)
- Prioritizeaza caracteristici in functie de cerintele pietii
- Ajusteaza functiile si prioritatile la fiecare iteratie, in functie de necesitati
- Accepta sau refuza rezultatele muncii

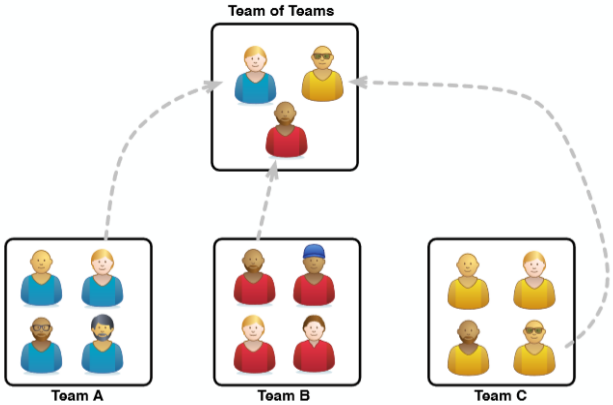


Echipa

- In mod normal, formata din 5-9 membri
- Inter-functionala
- Programatori, testeri, designeri de experienta de utilizator etc.
- Membri ar trebui sa lucreze full-time
- Pot exista exceptii (ex: administratorul de baza de date)
- Se autoorganizeaza
- Apartenenta la echipa se poate schimba numai intre cicluri de dezvoltare

Echipa de echipe

- Pentru proiecte mari se pot organiza echipe de echipe



Scrum in 100 de cuvinte

Scrum e un proces agil care ne permite sa ne concentram pe livrarea a ceea ce e mai valoros pentru o afacere in timpul cel mai scurt.

Ne permite ca in mod rapid si repetat sa evaluam software care intr-adevar functioneaza (odata la doua saptamanai sau lunar).

Clientul seteaza prioritatile. Echipele se organizeaza sigure pentru a determina cel mai bun mod de a livra functionalitatile cele mai prioritare.

Odata la doua saptamani sau cel mult lunar oricine poate vedea un software functional si sa decida sa il lanseze asa cum e sau sa continue sa il imbunatateasca in nou sprint.

Joke Time

Knock knock.
Who's there?
Done.
Done who?
Depends on who you ask.

"Knock knock."
Who's there?
"Carrie."
Carrie who?
"Carrie over to next sprint."

Knock knock.
Who's there?
Processes and tools.
Processes and tools who?
See, I told you he was ~~Agile!~~

A Pig and a Chicken are walking down the road.

The Chicken says: "Hey Pig, I was thinking we should open a restaurant!"

Pig replies: "Hm, maybe, what would we call it?"

The Chicken responds: "How about 'ham-n-eggs'?"

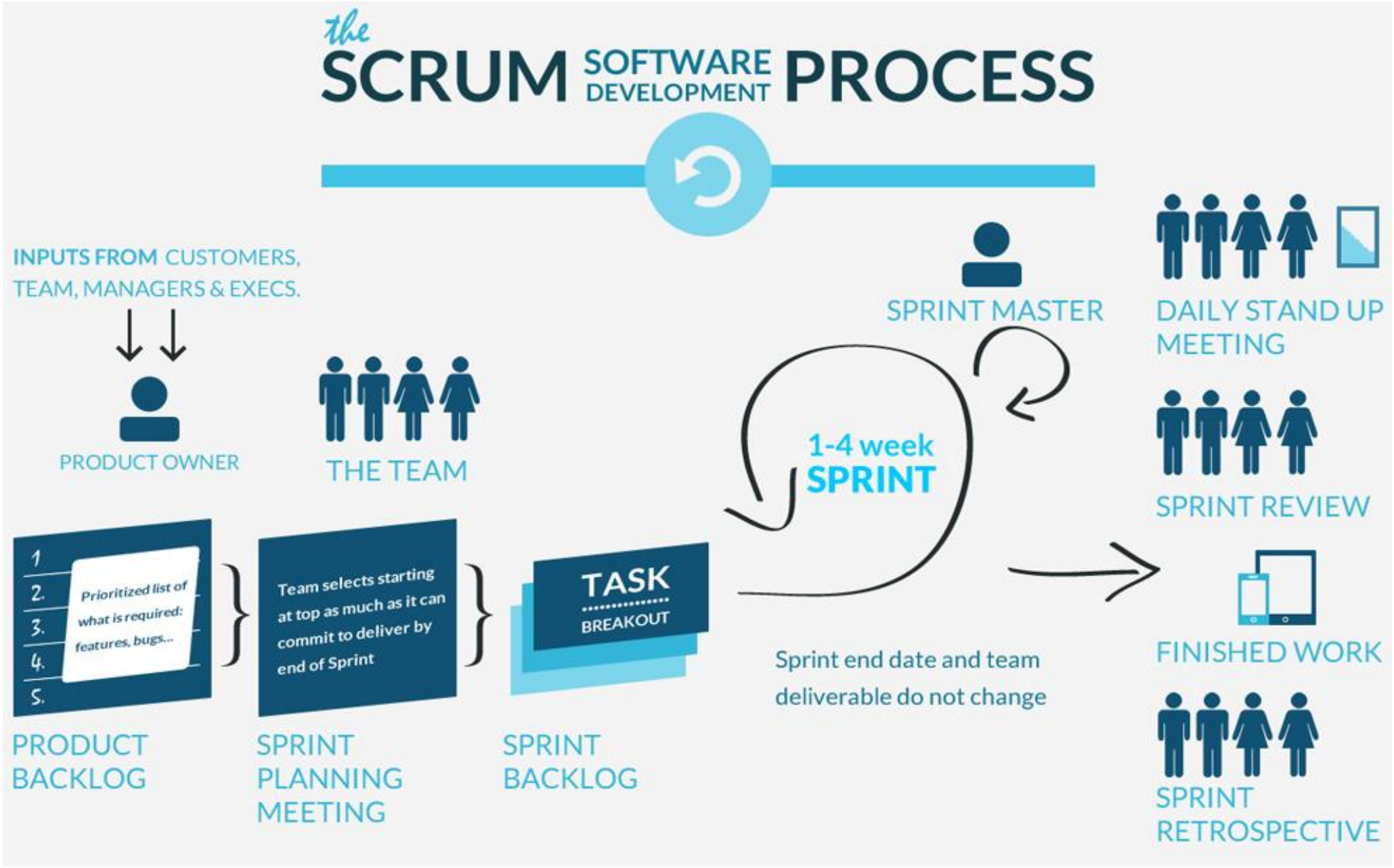
The Pig thinks for a moment and says: "No thanks. I'd be committed, but you'd only be involved."

Data viitoare..

- Procesul Scrum
- Produsul Backlog
- Planificarea iteratiilor
- Rularea si urmarirea iteratiilor
- Gestionarea proceselor Scrum

AGILE SOFTWARE DEVELOPMENT

Besea Georgiana
Draghici Ruxandra
342C5


the
SCRUM SOFTWARE DEVELOPMENT PROCESS
INPUTS FROM CUSTOMERS, TEAM, MANAGERS & EXECS.
PRODUCT OWNER
THE TEAM
SPRINT MASTER
DAILY STAND UP MEETING
1-4 week SPRINT
SPRINT REVIEW
1
2.
3.
4.
5.
Prioritized list of what is required: features, bugs...
Team selects starting at top as much as it can commit to deliver by end of Sprint
TASK
BREAKOUT
Sprint end date and team deliverable do not change
FINISHED WORK
PRODUCT BACKLOG
SPRINT PLANNING MEETING
SPRINT BACKLOG
SPRINT RETROSPECTIVE

Product backlog

- Lista ordonata ce contine tot ce este necesar produsului
- Compus din **User stories**
- Dinamic (produs adecvat, competitiv si util)
- Verificat de **Product Owner** (continut, disponibilitate, indeplinirea cererii)

User story

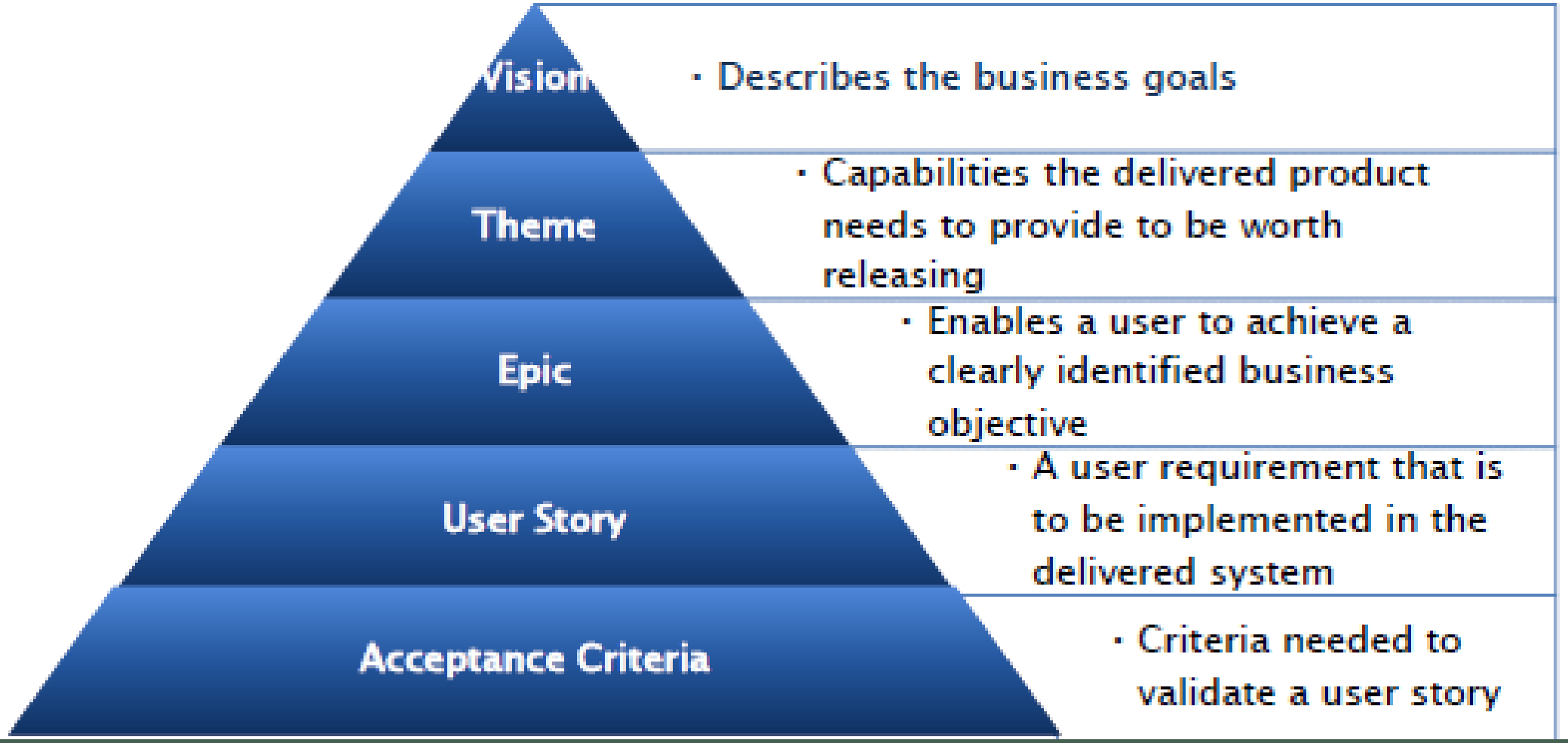
Format din:

- Rolul utilizatorului
- Descriere feature dorit
- Motiv

"As a **user,** I want **to upload photos** so that **I can share photos with others**."

Formula user story:

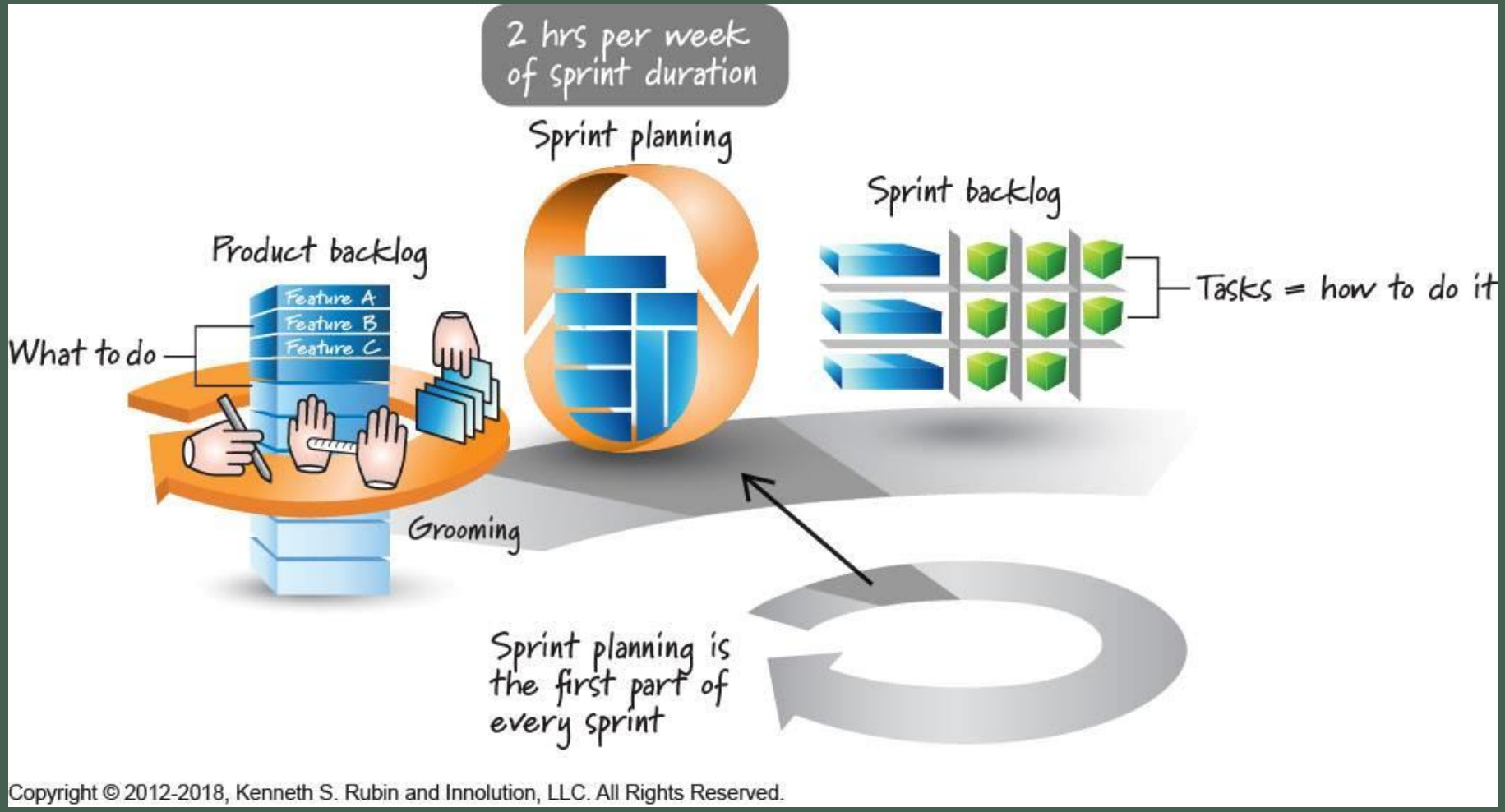
Card + Conversation + Confirmation (Creare User Story + Dezbatere pe tema user story-ului + Obiective atinse)


Vision
Describes the business goals
Theme
Capabilities the delivered product needs to provide to be worth releasing
Epic
Enables a user to achieve a clearly identified business objective
User Story
A user requirement that is to be implemented in the delivered system
Acceptance Criteria
Criteria needed to validate a user story

Pro user stories

- Prioritizare
- Colaborare
- Libertatea alegerii modului de dezvoltare
- Intelegerea scopului indiferent de pozitie

Sprint planning



Copyright © 2012-2018, Kenneth S. Rubin and Innolution, LLC. All Rights Reserved.

Participanti si roluri

Product Owner:

- scopul sprintului
- definirea user story-urilor in detaliu
- prioritizarea listei de USER
- descrierea rezultatului ce se doreste a fi obtinut pentru US si pentru sprint

Scrum Master:

- asigura desfasurarea in bune conditii a planningului
- verifica disponibilitatea si capacitatea de lucru a membrilor echipei

Dev Team:

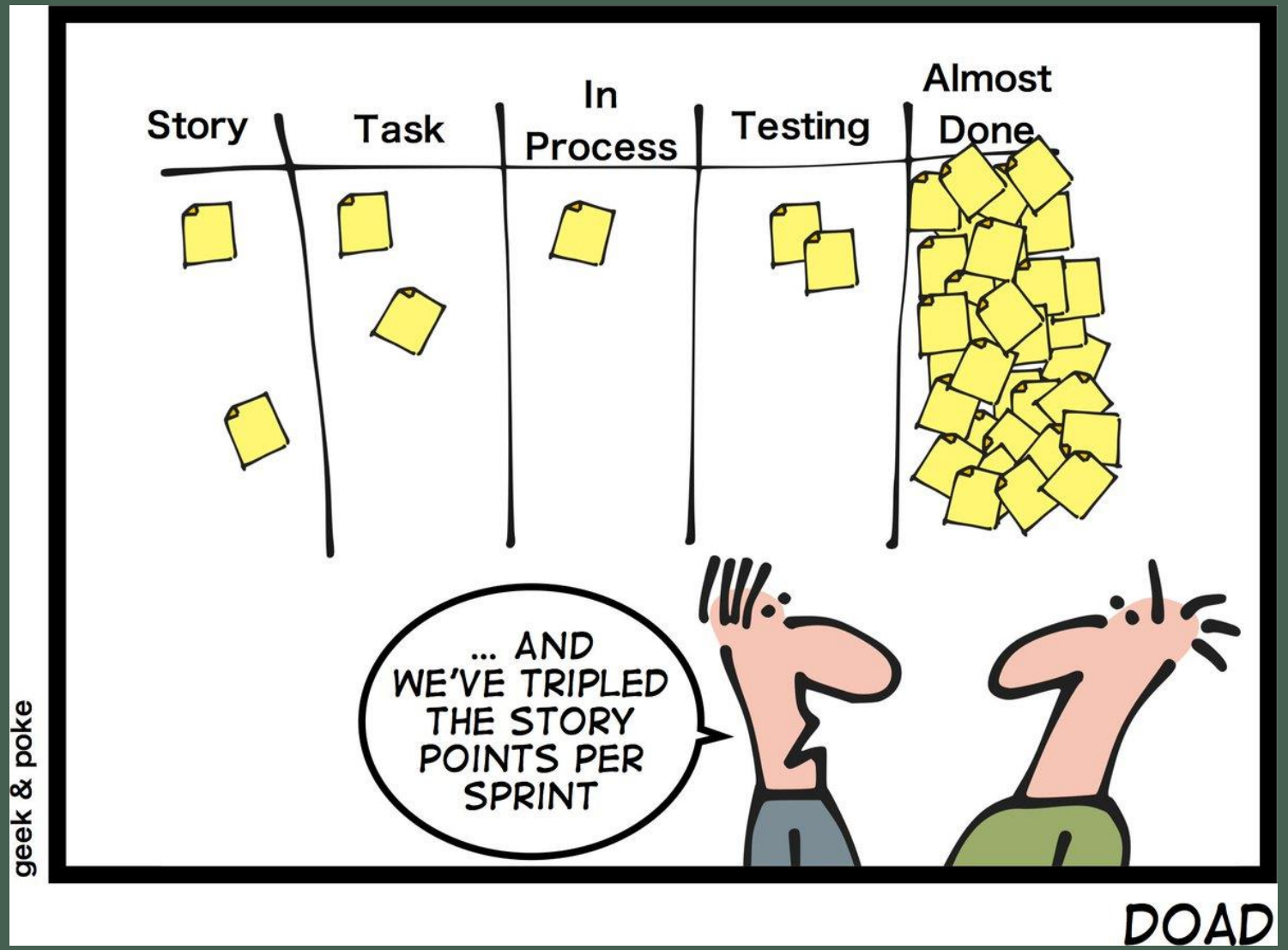
- adresare intrebari pentru clarificarea ambiguitatilor
- isi prezinta disponibilitatea in urmatorul sprint
- participa la estimarea fiecarui US

Estimarea

Metoda de masurare: Story points

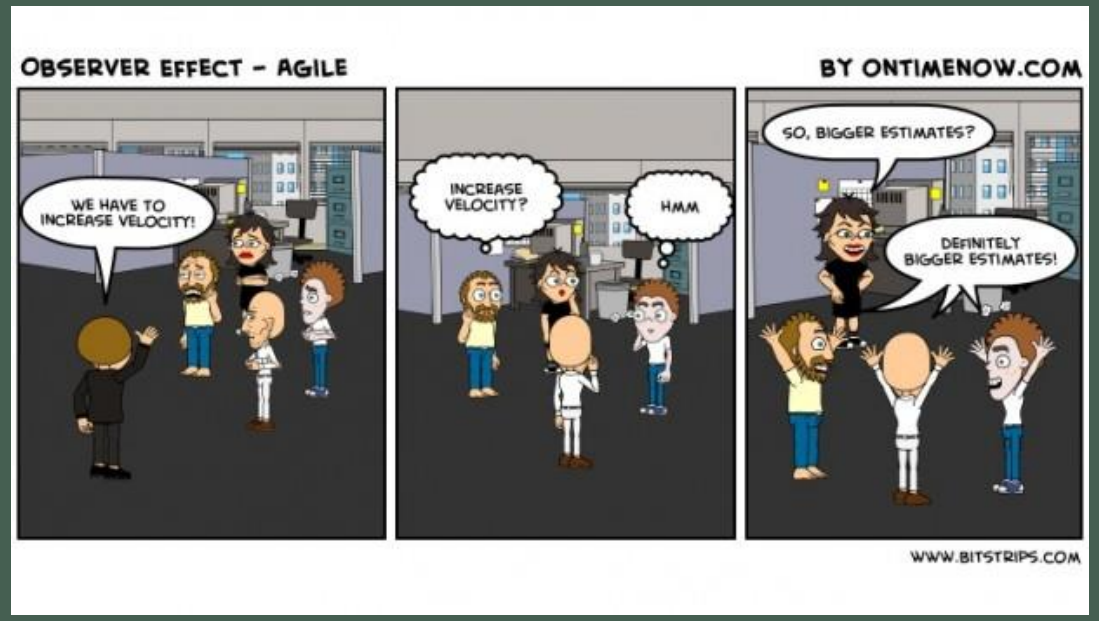
Metode de estimare a sprintului:

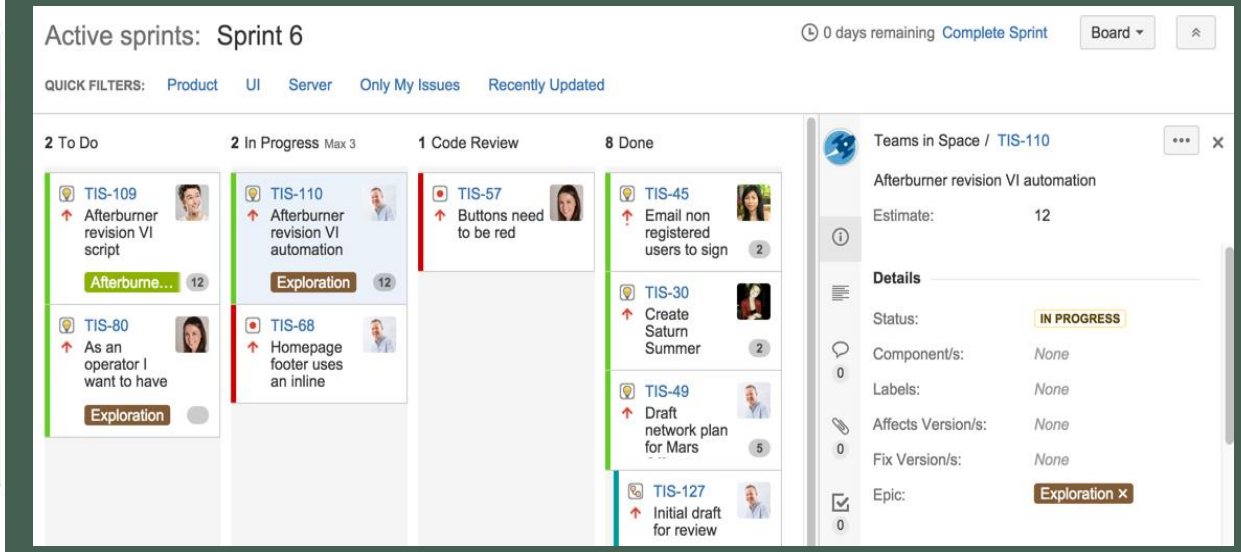
White Elephant Sizing Game https://www.youtube.com/watch?v=L94URgiPSGA



Scrum Tracking

- Masuratori: viteza, eficienta, efort.
- Metoda: Aplicatii de software development tracking – Jira, Trello.



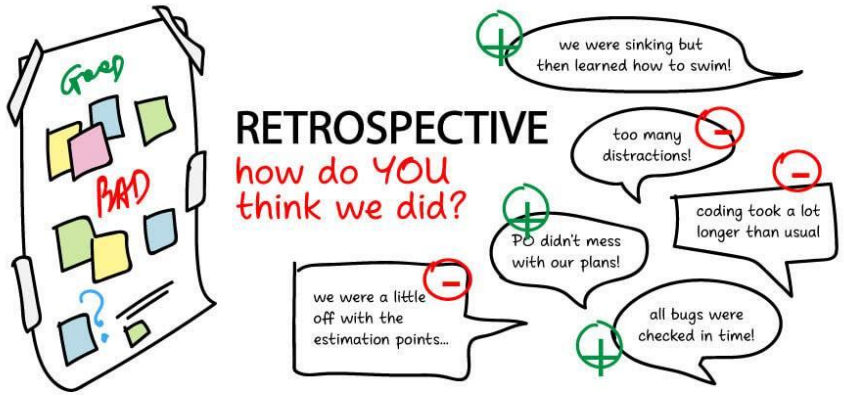


Daily Scrum

- Intalniri de 15 minute
- Fiecare participant raspunde la intrebarile:
  - Ce ai facut ieri?
  - Ce vei face azi?
  - Probleme intampinate – discutate in intalnirea de Problem Solving
- Abordare corecta: https://www.youtube.com/watch?v=H02BITXpcto

Sprint review

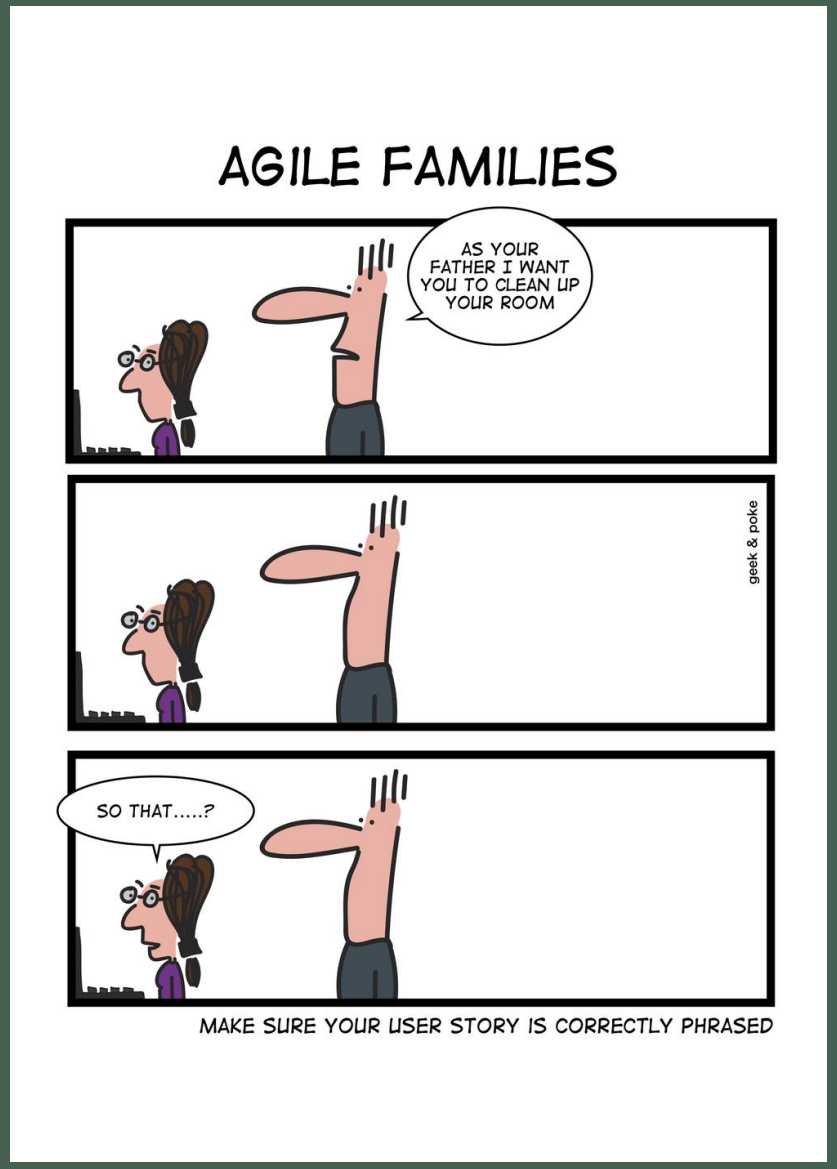
- Prezentarea produsului obtinut = **demo**
- **Cine** prezinta? – Scrum Master
- **Cui**? – Product owneri
- **Metode**:
  - inregistrari pentru fiecare feature/task
  - Lansarea ultimei versiuni a software-ului
  - Mock up uri



Sprint retrospective

- Ce a mers bine?
- Ce a mers prost?
- Ce putem schimba?
- Au avut effect schimbarile din ultimul sprint retrospective?

- Metode: Board stickers cu idei/ discutie libera/ jocuri



As a presenter of this topic I want to finish the presentation here so that all of us can have a break